

| 人工智能技术丛书 |

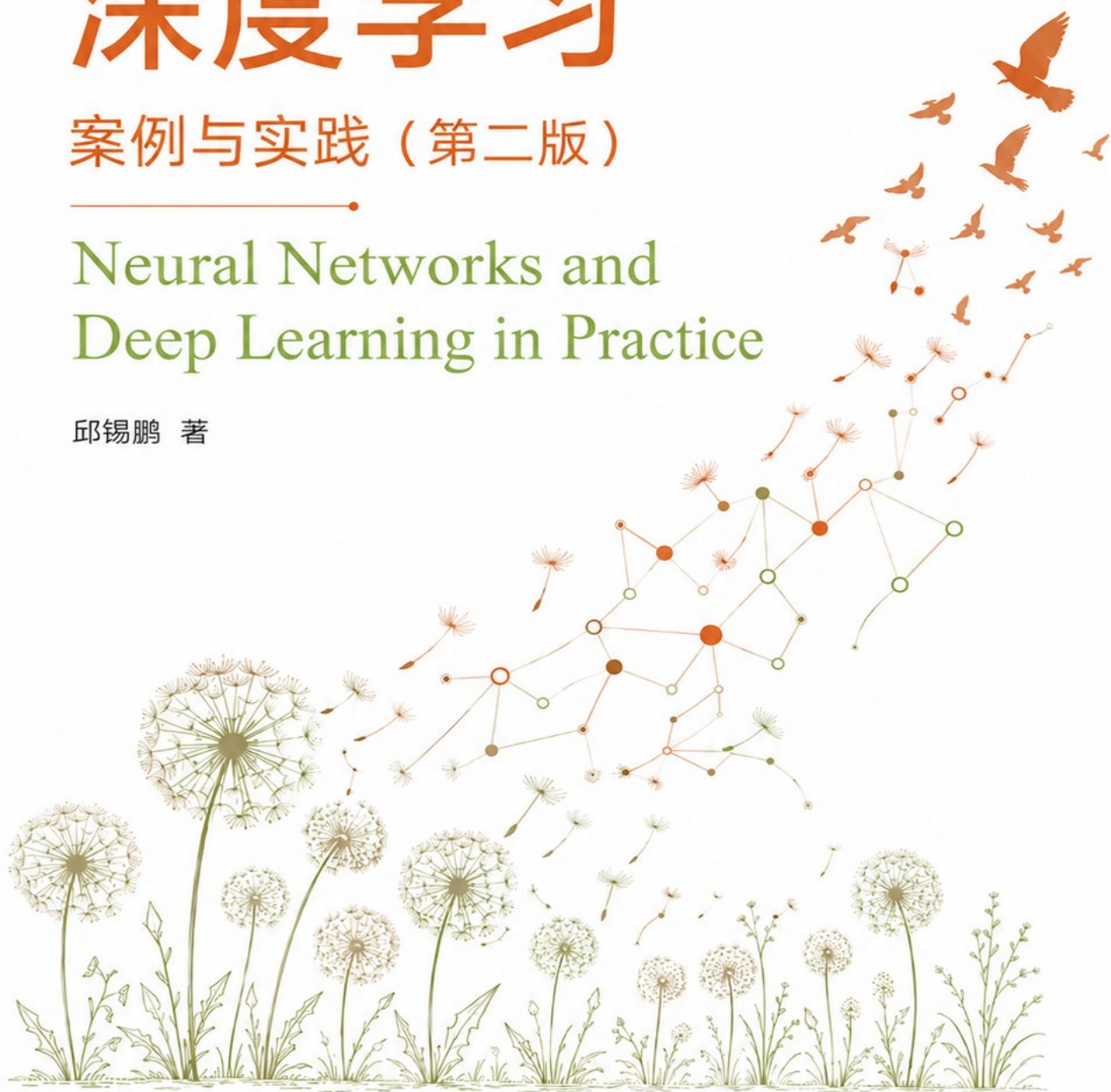
理论与实践相结合 · 案例与代码相驱动 · 学习与应用相促进

神经网络与 深度学习

案例与实践（第二版）

Neural Networks and
Deep Learning in Practice

邱锡鹏 著



神经网络与深度学习：案例与实践

(第二版)

Neural Networks and Deep Learning in Practice

邱锡鹏

复旦大学

2026 年 9 月 7 日

前言

反向传播公式能够逐项推导, 写出的网络却不一定学得会. 一个转置、一次重复累积的梯度, 或一段把测试数据用于调参的代码, 都可能让结果偏离预期. 实践的价值就在这些细节里: 把公式变成可运行的程序, 再用实验检查自己究竟理解了什么.

《神经网络与深度学习》电子版^①从2015年年底在网上开放共享以来, 读者的反馈一直帮助我们改进内容. 2022年, 本书第1版与百度飞桨团队合作完成, 使用飞桨(PaddlePaddle)框架编写实践内容, 并配合AI Studio在线平台运行, 为理论学习提供了配套的动手实践.

本书第2版采用PyTorch重写实践代码. 读者先用张量运算实现前向计算、梯度和参数更新, 再与框架提供的模块对照, 最后把模型放进包含数据处理、训练和评价的完整实验中. 第2版还补充了注意力机制、图神经网络、大语言模型与智能体等内容, 让前面建立的知识能够继续用于新的问题.

本书的定位是把神经网络模型、深度学习原理和工程实践结合起来, 让读者在动手实践中更深入地理解深度学习的模型和原理. 本书选取《神经网络与深度学习》中的主要模型和方法, 按实践所需的先后关系组织为10章. 各章结合模型解读与案例实践展开:

1) **模型解读**主要聚焦如何从零开始一步步实现机器学习模型和算法, 并结合简单的任务来加深读者对模型和算法的理解.

2) **案例实践**主要强调如何在实际应用的任务中使用PyTorch来更好地实现模型和算法, 会涉及一些在实际任务上碰到的数据预处理等问题.

贯穿前几章的两条线索是训练流程和模型构建.

1) 从**学习流程**出发, 逐步构建教学用训练器Runner. 模型、损失函数、优化器和评价指标各有职责, Runner将它们串联起来, 帮助读者看清一次训练迭代中发生了什么. 随着任务变复杂, 训练器再加入小批量训练、验证和检查点保存等功能.

2) 从**模型构建**出发, 用张量运算实现基本算子, 再将算子组合成模型, 形成配套的nndl代码库. 手写实现与PyTorch实现的对照, 可以把公式中的求和、广播和链式法则落实到张量形状与梯度上.

^① 本书出版后因封面上专门设计了蒲公英图案, 寓意帮助更多的读者进入深度学习以及人工智能领域, 为人工智能领域注入新的生机与活力, 而被广大读者昵称为蒲公英书. ——编辑注

Runner 和 nndl 服务于教学. 图上的消息传递、语言模型的自回归生成等任务有不同的计算过程, 后续章节会据此使用专门的训练或推断循环. 读者也可以借这些差异判断: 哪些步骤值得抽象成公共接口, 哪些细节需要在具体任务里保留.

阅读本书需要基本的 Python 编程、线性代数和微积分知识. 初学者可以先顺序完成前 4 章, 亲手检查一次前向计算和梯度, 再按兴趣进入图像、序列或图数据的案例. 读每个实验时, 不妨先写下自己的预期: 增加网络深度是否一定更好, 验证损失下降是否意味着测试结果更好, 模型能复述资料是否就能回答资料之外的问题. 运行结果与预期的分歧, 往往是最值得追查的地方.

为便于在普通计算机上复核, 部分案例采用明确标出的数据子集和较短训练预算; 较大配置作为进一步实验保留. 书中的实测数值与相应的数据划分、随机种子和训练条件配套, 不应直接当作完整数据集上的性能结论. 比较模型时应固定评价数据和选择规则, 用验证集选择模型, 最后再查看测试结果. 配套 Notebook 提供代码执行顺序和必要的运行说明.

此外, 本书还对《神经网络与深度学习》中的一些数学公式和术语翻译进行更新:

1) 在**数学公式**方面,《神经网络与深度学习》以矩阵表示一组样本时, 每一列为一个样本. 本书通常采用每一行为一个样本的约定, 与批量输入形状 `[batch, features]` 保持一致. 这是数据排列的选择, 张量本身并不要求按行存放样本. 相应公式中的矩阵乘法和转置也随之调整, 便于直接对照代码.

2) 在**术语翻译**方面, 机器学习领域的很多名词存在难翻译和乱翻译的现象. 2021 年, 在李航老师的建议下, 我有幸与周志华老师、李沐、阿斯顿·张一起讨论了机器学习相关术语的翻译问题, 对各自教材中不一致的译法进行统一, 因此本书中采用我们当时讨论确定的建议译法, 比如将 Dropout 翻译为“暂退法”, 将 Normalization 翻译为“规范化”等.

本书第 1 版的完成离不开飞桨教材编写组的大力支持, 他们为本书撰写了逻辑严谨的模型解读内容和简洁易用的实践代码. 第 2 版是在第 1 版基础上, 重新使用 PyTorch 进行编写和案例更新. 配套代码位于 `nndl-practice` 仓库的 `pytorch` 目录^②, 欢迎广大读者在仓库的 Issues 中提交问题和建议.

因为个人能力有限, 书中难免有不当和错误之处, 还望读者海涵和指正, 不胜感激.

邱锡鹏

于上海

2026 年 5 月

^② <https://github.com/nndl/nndl-practice/tree/main/pytorch>

目 录

前言	1
第 1 章 实践基础	1
1.1 如何运行本书的代码	1
1.1.1 环境准备	1
1.1.2 安装 PyTorch	2
1.1.3 代码下载与使用方法	3
1.1.4 依赖库与版本约定	3
1.2 张量	4
1.2.1 创建张量	4
1.2.2 张量的属性	7
1.2.3 张量与 Numpy 数组转换	10
1.2.4 张量的访问	11
1.2.5 张量的运算	13
1.3 算子	18
1.3.1 算子定义	19
1.3.2 自动微分机制	22
1.3.3 预定义的算子	25
1.3.4 本书中实现的算子	26
1.3.5 本书中实现的优化器	27
1.4 本书中使用的数据集和实现的 Dataset 类	28
1.4.1 数据集	28
1.4.2 Dataset 类	29
1.5 一个最小的训练循环	29
1.6 本书中实现的 Runner 类	30
1.7 模型的保存与加载	31
1.8 小结	33
第 2 章 机器学习概述	34
2.1 机器学习实践五要素	35
2.1.1 数据	36
2.1.2 模型	37

2.1.3	学习准则	37
2.1.4	优化算法	38
2.1.5	评价指标	38
2.2	实现一个简单的线性回归模型	38
2.2.1	数据集构建	38
2.2.2	模型构建	41
2.2.3	损失函数	43
2.2.4	优化器	44
2.2.5	模型训练	46
2.2.6	模型评价	46
2.3	多项式回归	47
2.3.1	数据集构建:ToySin25	47
2.3.2	模型构建	48
2.3.3	模型训练	49
2.3.4	模型评价	50
2.3.5	通过引入正则化项来缓解过拟合	52
2.4	构建 Runner 类	53
2.5	实践:基于线性回归的加州房价预测	55
2.5.1	数据处理	55
2.5.2	模型构建	58
2.5.3	完善 Runner 类:RunnerV1	58
2.5.4	模型训练	59
2.5.5	模型评价	60
2.5.6	模型预测	60
2.6	小结	61
第 3 章	线性模型	62
3.1	基于 Logistic 回归的二分类任务	63
3.1.1	数据集构建	63
3.1.2	模型构建	66
3.1.3	损失函数	67
3.1.4	模型优化	68
3.1.5	评价指标	71
3.1.6	完善 Runner 类:RunnerV2	72
3.1.7	模型训练	74
3.1.8	模型评价	75
3.2	基于 Softmax 回归的多分类任务	75
3.2.1	数据集构建	76
3.2.2	模型构建	78
3.2.3	损失函数	80
3.2.4	模型优化	82
3.2.5	模型训练	83

3.2.6	模型评价	84
3.3	实践:基于 Softmax 回归完成鸢尾花分类任务	84
3.3.1	数据处理	85
3.3.2	模型构建	86
3.3.3	模型训练	87
3.3.4	模型评价	87
3.3.5	模型预测	87
3.4	小结	88
第 4 章	前馈神经网络	89
4.1	神经元	89
4.1.1	激活函数	91
4.2	基于前馈神经网络的二分类任务	93
4.2.1	数据集构建	93
4.2.2	模型构建	93
4.2.3	损失函数	96
4.2.4	模型优化	97
4.2.5	完善 Runner 类:RunnerV2_1	103
4.2.6	模型训练	105
4.2.7	模型评价	106
4.3	自动梯度计算和预定义算子	106
4.3.1	利用预定义算子重新实现前馈神经网络	107
4.3.2	完善 Runner 类:RunnerV2_2	108
4.3.3	模型训练	110
4.3.4	模型评价	110
4.4	优化问题	111
4.4.1	参数初始化	111
4.4.2	梯度消失问题	112
4.4.3	死亡 ReLU 问题	115
4.5	实践:基于前馈神经网络完成鸢尾花分类任务	117
4.5.1	小批量梯度下降	118
4.5.2	数据处理	119
4.5.3	模型构建	121
4.5.4	完善 Runner 类:RunnerV3	121
4.5.5	模型训练	125
4.5.6	模型评价	126
4.5.7	模型预测	126
4.6	小结	127
第 5 章	卷积神经网络	128
5.1	卷积	129
5.1.1	二维卷积运算	129

5.1.2	二维卷积算子	130
5.1.3	卷积的变种	131
5.1.4	带步长和零填充的二维卷积算子	133
5.1.5	使用卷积运算完成图像边缘检测任务	134
5.2	卷积神经网络的基础算子	135
5.2.1	卷积层算子	136
5.2.2	汇聚层算子	139
5.3	基于 LeNet 实现手写体数字识别任务	141
5.3.1	数据集构建	141
5.3.2	模型构建	144
5.3.3	模型训练	147
5.3.4	模型评价	148
5.3.5	模型预测	148
5.4	基于残差网络的手写体数字识别	149
5.4.1	模型构建	150
5.4.2	没有残差连接的 ResNet18	155
5.4.3	带残差连接的 ResNet18	157
5.5	实践:基于 ResNet18 网络完成图像分类任务	158
5.5.1	数据处理	159
5.5.2	模型构建	162
5.5.3	模型训练	162
5.5.4	模型评价	164
5.5.5	模型预测	164
5.5.6	进一步实践:从零训练还是迁移学习	165
5.6	小结	166
第 6 章	循环神经网络	167
6.1	循环神经网络的记忆能力实验	168
6.1.1	数据集构建	169
6.1.2	模型构建	172
6.1.3	模型训练	177
6.1.4	模型评价	180
6.2	梯度爆炸实验	181
6.2.1	先观察时间传播本身	181
6.2.2	梯度范数记录函数	182
6.2.3	复现梯度爆炸问题	183
6.2.4	使用梯度截断解决梯度爆炸问题	185
6.3	LSTM 的记忆能力实验	187
6.3.1	模型构建	189
6.3.2	模型训练	190
6.3.3	模型评价	192
6.4	实践:基于双向 LSTM 模型完成文本分类任务	194

6.4.1	数据处理	194
6.4.2	模型构建	199
6.4.3	模型训练	202
6.4.4	模型评价	203
6.4.5	模型预测	203
6.5	小结	204
第7章	网络优化与正则化	205
7.1	小批量梯度下降法	206
7.2	批大小与学习率线性缩放实验	206
7.3	不同优化算法的比较分析	209
7.3.1	优化算法的实验设定	209
7.3.2	学习率调整	216
7.3.3	梯度估计修正	223
7.3.4	不同优化器的3D可视化对比	229
7.4	参数初始化	234
7.4.1	基于固定方差的参数初始化	234
7.4.2	基于方差缩放的参数初始化	235
7.5	逐层规范化	240
7.5.1	批量规范化	240
7.5.2	层规范化	247
7.6	网络正则化方法	249
7.6.1	数据集构建	249
7.6.2	模型构建	250
7.6.3	ℓ_1 和 ℓ_2 正则化	255
7.6.4	权重衰减	257
7.6.5	暂退法	259
7.6.6	从验证集选择,再评价测试集	263
7.6.7	进一步实践:训练预算与模型选择	263
7.7	小结	263
第8章	注意力机制	265
8.1	基于双向 LSTM 和注意力机制的文本分类	266
8.1.1	数据介绍	267
8.1.2	模型构建	268
8.1.3	使用加性注意力模型进行实验	274
8.1.4	使用点积注意力模型进行实验	275
8.2	基于双向 LSTM 和多头自注意力的文本分类实验	278
8.2.1	自注意力模型	278
8.2.2	基于 LSTM 和多头自注意力的文本分类的模型构建	288
8.2.3	模型训练	290
8.2.4	模型评价	291

8.2.5	模型预测	292
8.3	实践:基于自注意力模型的文本语义匹配	293
8.3.1	数据集构建	294
8.3.2	模型构建	298
8.3.3	模型训练	308
8.3.4	模型评价	309
8.3.5	模型预测	310
8.3.6	注意力可视化	310
8.4	现代Transformer的若干优化	312
8.4.1	Pre-LayerNorm vs Post-LayerNorm	312
8.4.2	旋转位置编码RoPE	312
8.4.3	三种架构范式	313
8.4.4	混合专家MoE	314
8.4.5	长上下文与FlashAttention	315
8.5	小结	316
第9章	图神经网络	318
9.1	图数据的基本表示	319
9.1.1	Zachary's Karate Club:一个“入门级”数据集	319
9.1.2	图数据可视化	320
9.1.3	节点特征的选择	321
9.2	消息传递机制	322
9.2.1	抽象基类:把消息传递独立出来	322
9.2.2	一个示例:用基类实现Mean Aggregator	324
9.3	GCN:图卷积网络	325
9.4	GraphSAGE:邻居采样+聚合	328
9.5	GAT:图注意力网络	329
9.6	GIN:图同构网络	330
9.7	四种GNN在Karate Club上的对比	331
9.8	过度平滑与深层训练	334
9.8.1	实验:把GCN堆到2层、4层、6层、8层	334
9.8.2	缓解过度平滑的三种思路	335
9.9	链接预测:让模型“补全”图	337
9.9.1	思路:编码器+解码器	337
9.9.2	负采样:从“缺边”里造训练样本	337
9.9.3	切分训练、验证与测试边	338
9.9.4	完整的GCN链接预测模型	338
9.9.5	评估:AUC与平均精确率	340
9.10	图级任务:随机图回归	341
9.11	PyTorch Geometric:从“学具”到“工程”	344
9.11.1	把前面的GCN换成PyG版	344
9.12	真实数据集实战:Cora论文引用网络	345

9.12.1	数据处理	345
9.12.2	模型构建	345
9.12.3	模型训练	346
9.12.4	模型评价	347
9.12.5	模型预测	349
9.13	图分类实战:蛋白质结构图分类	350
9.13.1	数据处理	350
9.13.2	模型构建	351
9.13.3	模型训练	351
9.13.4	模型评价	353
9.13.5	模型预测	354
9.14	小结	355
第 10 章	大语言模型与智能体	356
10.1	语言模型回顾	357
10.2	实践:从零搭建字符级 nanoGPT	357
10.2.1	数据处理	357
10.2.2	模型构建	362
10.2.3	模型训练	364
10.2.4	模型评价	368
10.2.5	模型预测	369
10.3	监督微调与 LoRA	373
10.3.1	LoRA 的结构与参数量	374
10.3.2	全量微调与 LoRA 的存储开销	374
10.3.3	把 LoRA 注入预训练 nanoGPT	375
10.3.4	SFT 数据格式与损失遮蔽	376
10.3.5	SFT 训练曲线	377
10.4	偏好对齐:DPO	379
10.5	推理与行动智能体	382
10.6	检索增强生成 (RAG)	384
10.7	小结	386
索引		388

第1章 实践基础

第一次写神经网络,模型常常只有几行,报错却可能出在另一处:两组数据的形状对不上,标签用了浮点数,或者模型已经移到GPU上,输入还留在CPU里。修好这些问题之后,训练终于开始,损失却纹丝不动。此时需要知道数据怎样进入模型、梯度怎样产生、参数又在何时更新。

深度学习框架把张量运算、自动微分和设备调度等工作组织成可复用的接口,让我们可以逐步检查这条计算过程。本书选用**PyTorch** (PyTorch)。我们会先用它做几个能手算核对的小实验,再让同一套工具处理图像、文本和图数据。

学习框架也有取舍。直接调用现成的层可以迅速搭起模型,手写一个小算子的前向和反向计算则能帮助我们理解它为什么这样工作。本书会交替使用这两种方式。

本章为后续章节铺垫两个最基础的概念:

- **张量 (Tensor)**: 多维数组形式的数据容器,是深度学习中存储和传递数据的统一形式。后续小节将熟悉它的形状、数据类型、设备位置等属性,以及PyTorch中的常用API。
- **算子 (Operator, Op)**: 完成一项计算的基本单元,例如加法、矩阵乘法和激活函数。对需要求导的可微运算,框架还提供相应的反向计算规则;比较、取整等操作则不能一概视为可微。

本章后半部分会把全书反复出现的资源整理成索引——包括我们自实现的算子、优化器、数据集类,以及轻量训练框架**Runner**,方便后续章节按图索骥。

1.1 如何运行本书的代码

1.1.1 环境准备

本书代码基于Python和PyTorch开发。建议为本书建立独立虚拟环境;以下是配套代码采用的环境约定,具体操作系统和硬件支持范围还需核对所安装版本的官方说明:

- 操作系统: Linux、macOS 或 Windows
- Python 版本: 优先使用 3.11 或 3.12 (64 位); 使用其他版本前, 检查依赖库是否提供对应安装包

- `pip`: 在虚拟环境中升级到与当前 Python 兼容的版本
- PyTorch 版本: 配套代码以 2.7 及后续 2.x 版本为基线
- torchvision 版本: 与 PyTorch 对应的版本
- GPU 加速 (可选): 使用 NVIDIA GPU 时, 选择硬件与驱动支持的 CUDA 安装包; 无需把 CUDA 版本统一限定为 12.6 及以上

全书代码示例与 API 说明均以 PyTorch 2.x 版本的常用 API 为基准, 具体依赖版本以配套代码仓库的环境配置为准. 如果读者参考的旧教程出现已被弃用的参数、类名或调用形式, 请优先使用配套代码中采用的 PyTorch 和 torchvision 写法.

使用如下命令确认 Python 版本:

```
1 python --version
```

使用如下命令确认 pip 版本:

```
1 python -m pip --version
```

1.1.2 安装 PyTorch

前几章的张量运算、线性模型和小型网络用 CPU 就能练习. 第 5 章的图像模型及后续较大的文本模型训练耗时更长, 使用 GPU 可以加快实验. 先运行小规模示例, 确认数据、损失和梯度都正常, 再增加样本数或训练轮数; 完整实验的耗时取决于模型、数据和硬件配置.

通过如下命令安装 CPU 版本的 PyTorch:

```
1 python -m pip install torch torchvision --index-url https://download.pytorch.org/whl/cpu
```

上述 CPU 安装命令适用于官网提供对应安装包的 Windows 或 Linux 环境. macOS 及 GPU 环境请在 PyTorch 官网^①选择操作系统和计算平台后复制安装命令. 预编译的 CUDA 安装包提供所需运行时依赖, 通常不要求单独安装同版本的 CUDA 开发工具包; 本机 NVIDIA 驱动仍须满足安装包的要求. 只有编译 PyTorch 或自定义 CUDA 扩展等情形才另需开发工具链.

动手练习 1.1

依次完成以下三步自检, 确认本机环境就绪:

- 1) 终端运行 `python` 进入解释器, 执行 `import torch`——无报错即说明 PyTorch 模块可以被导入.
- 2) 接着执行 `torch.__version__`, 确认实际安装的版本号与预期一致.
- 3) 最后执行 `torch.cuda.is_available()`, 返回 `True` 表示当前安装可使用 CUDA 设备; CPU 安装返回 `False` 也属正常. macOS 的 MPS 加速另有接口, 不能用这一项判断.



^① <https://pytorch.org/get-started/locally/>.

1.1.3 代码下载与使用方法

本书代码托管在 GitHub 项目^②, 第 2 版内容位于 `pytorch/` 目录. 下载仓库并安装 PyTorch 后, 在仓库根目录执行以下命令安装其余依赖、打开第一个 Notebook:

```
1 python -m pip install -r pytorch/requirements.txt
2 jupyter notebook "pytorch/chap1实践基础/实践基础.ipynb"
```

Notebook 会保留先前运行的变量. 修改前面的数据或模型定义后, 应重启内核并从头顺序执行, 确认结果不依赖旧变量. 提问时请附上章节、代码单元、Python 和 PyTorch 版本、完整报错及复现步骤.



笔记

本书包含大量代码示例, 建议在 Jupyter Notebook 中边读边运行, 配合源码观察输出, 以加深对模型构建与训练流程的理解.



1.1.4 依赖库与版本约定

除 PyTorch 和 torchvision 外, 本书代码还会用到以下常见的第三方库, 均列在配套仓库的 `pytorch/requirements.txt` 中:

- `numpy`, `matplotlib`: 贯穿全书, 分别用于数值计算与绘图.
- `scikit-learn`: 第 2 章、第 3 章加载加州房价、鸢尾花等数据集时使用.
- `networkx`, `torch_geometric`: 第 9 章图神经网络构建图数据与工程版对照实验时使用.

为减少版本差异带来的困扰, 本书统一遵循以下两条约定:

- **以 PyTorch 2.x 为基准**: 书中代码均采用 PyTorch 2.x 的官方推荐写法. 少数 API 对版本较敏感, 例如加载权重时显式指定 `torch.load(..., weights_only=True)`, 张量范数统一使用 `torch.linalg` 命名空间下的范数函数 (如 `norm`、`vector_norm`), 不再使用已弃用的 `torch.norm`. 若旧资料中出现已弃用的参数、类名或调用形式, 一律以配套代码仓库的写法为准.
- **记录随机性与实验条件**: 在数据生成和模型构建前设置 `torch.manual_seed`; 若还使用 NumPy、Python 的随机数或带 `random_state` 的工具, 也要分别设置它们的种子. 固定种子不保证跨版本、跨设备逐位一致. 书中的输出与曲线对应特定实验配置; 比较方法时, 应固定数据划分并重复多个随机种子, 同时记录样本量、超参数和软件版本.

^② <https://github.com/nndl/nndl-practice>.

1.2 张量

深度学习中的计算几乎都建立在向量、矩阵运算之上。例如 $w_1x_1 + w_2x_2 + \dots + w_Dx_D$ 可以改写成 $\mathbf{w}^T \mathbf{x}$ (其中 $\mathbf{w} = [w_1 w_2 \dots w_D]^T$, $\mathbf{x} = [x_1 x_2 \dots x_D]^T$), 这种紧凑形式既便于推导, 也方便借助 CPU/GPU 的并行能力实现高效计算。

深度学习框架沿用了这一思路, 把数据统一组织为张量 (Tensor)。张量可以看作矩阵向高阶的推广: 1 阶对应向量, 2 阶对应矩阵, 更高阶就是嵌套层数更多的多维数组。如果熟悉 Numpy, 那么张量与 Numpy 的多维数组 (ndarray) 非常类似, 二者都允许任意阶数。



笔记

这里的“维度”指张量的“阶”，与线性代数中向量的“维数”含义不同，注意区分。

张量的大小用形状 (Shape) 描述。例如一个三阶张量形状为 $[2, 2, 5]$, 每个数字表示对应轴 (Axis) 上的元素个数——第 0 轴 2 个, 第 1 轴 2 个, 第 2 轴 5 个。图 1.1 给出了不同阶张量的可视化对照。

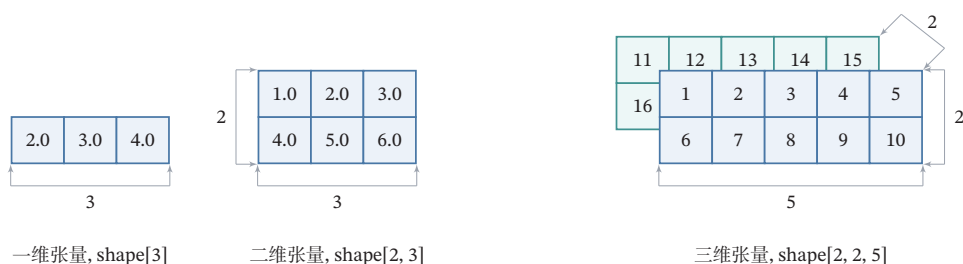


图 1.1 不同维度的张量可视化表示

同一张量里不允许混用元素类型——所有元素必须共享一个数据类型 (dtype) (可选: 布尔、若干位宽的整数、浮点数或复数)。这一约束让底层算子可以按固定步长寻址张量缓冲区, 是高性能批量计算的前提。

1.2.1 创建张量

PyTorch 提供了多种创建张量的方式, 常见的有: 直接由数据创建、按形状创建、按区间创建。

1.2.1.1 指定数据创建张量

把 Python 列表 (List) 作为输入, 即可创建任意阶数的张量。

1) 由列表 $[2.0, 3.0, 4.0]$ 创建一维张量:

```
1 # 导入PyTorch
2 import torch
3 # 创建一维张量
4 ndim_1_Tensor = torch.tensor([2.0, 3.0, 4.0])
5 print(ndim_1_Tensor)
```

输出结果为:

```
tensor([2., 3., 4.])
```

2) 使用嵌套列表创建对应矩阵的二维张量:

```
1 # 创建二维张量
2 ndim_2_Tensor = torch.tensor([[1.0, 2.0, 3.0],
3                               [4.0, 5.0, 6.0]])
4 print(ndim_2_Tensor)
```

输出结果为:

```
tensor([[1., 2., 3.],
        [4., 5., 6.]])
```

3) 同理可创建 3 阶、4 阶乃至 N 阶的多维张量:

```
1 # 创建多维张量
2 ndim_3_Tensor = torch.tensor([[[1, 2, 3, 4, 5],
3                               [6, 7, 8, 9, 10]],
4                               [[11, 12, 13, 14, 15],
5                               [16, 17, 18, 19, 20]]])
6 print(ndim_3_Tensor)
```

输出结果为:

```
tensor([[[ 1, 2, 3, 4, 5],
         [ 6, 7, 8, 9, 10]],
        [[11, 12, 13, 14, 15],
         [16, 17, 18, 19, 20]]])
```

需要注意: 张量同一维度上的元素数量必须相等, 否则会抛出形状不匹配的异常. 例如, 下面这段代码就会失败:

```
1 # 尝试定义在不同维度上元素数量不等的张量
2 ndim_2_Tensor = torch.tensor([[1.0, 2.0],
3                               [4.0, 5.0, 6.0]])
```

输出结果为:

```
ValueError: expected sequence of length 2 at dim 1 (got 3)
```

异常信息提示同一维度上的元素数量不一致,与上面的说明一致.

1.2.1.2 指定形状创建张量

如果只关心形状而希望元素全部相同,可使用`torch.zeros`、`torch.ones`、`torch.full`等 API:

```
1 m, n = 2, 3
2 # 使用 torch.zeros 创建数据全为 0, 形状为 [m, n] 的张量
3 zeros_Tensor = torch.zeros(m, n)
4 # 使用 torch.ones 创建数据全为 1, 形状为 [m, n] 的张量
5 ones_Tensor = torch.ones(m, n)
6 # 使用 torch.full 创建数据全为指定值, 形状为 [m, n] 的张量, 这里我们指定数据为 10
7 full_Tensor = torch.full((m, n), 10)
8
9 print('zeros Tensor: ', zeros_Tensor)
10 print('ones Tensor: ', ones_Tensor)
11 print('full Tensor: ', full_Tensor)
```

输出结果为:

```
zeros Tensor: tensor([[0., 0., 0.],
                      [0., 0., 0.]])
ones Tensor: tensor([[1., 1., 1.],
                     [1., 1., 1.]])
full Tensor: tensor([[10, 10, 10],
                     [10, 10, 10]])
```

1.2.1.3 指定区间创建张量

需要在某个数值区间内按规则取点时,可以使用`torch.arange`、`torch.linspace`等 API:

```
1 # 使用 torch.arange 创建以步长 step 均匀分隔数值区间 [start, end) 的一维张量
2 arange_Tensor = torch.arange(start=1, end=5, step=1)
3 # 使用 torch.linspace 创建以元素个数 steps 均匀分隔数值区间 [start, end] 的张量
4 linspace_Tensor = torch.linspace(start=1, end=5, steps=5)
5
6 print('arange Tensor: ', arange_Tensor)
7 print('linspace Tensor: ', linspace_Tensor)
```

输出结果为:

```
arange Tensor: tensor([1, 2, 3, 4])
linspace Tensor: tensor([1., 2., 3., 4., 5.])
```

1.2.2 张量的属性

1.2.2.1 张量的形状

PyTorch 通过几个属性暴露张量的形状信息, 调试或写算子时随时会用到:

- **Tensor.ndim**: 维度个数, 标量为 0、向量为 1、矩阵为 2, 依此类推.
- **Tensor.shape**: 各维度长度组成的元组 (即每一维上有多少个元素).
- **Tensor.shape[n]**: 单独取第 n 维的长度.
- **Tensor.numel()**: 张量总元素数, 等于 **shape** 内各项的乘积.

为了更好地理解 **ndim**、**shape**、**shape[n]**、**numel** 四种属性间的区别, 创建一个如图 1.2 所示的四维张量.

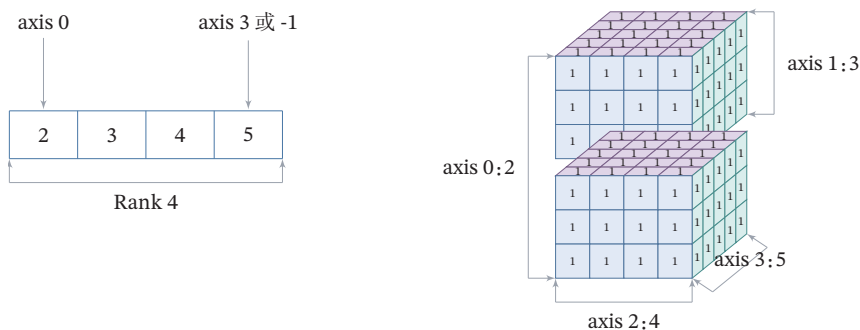


图 1.2 形状为 [2, 3, 4, 5] 的四维张量

创建一个四维张量, 并打印出 **shape**、**ndim**、**shape[n]**、**numel** 属性.

```
1 ndim_4_Tensor = torch.ones(2, 3, 4, 5)
2
3 print("Number of dimensions:", ndim_4_Tensor.ndim)
4 print("Shape of Tensor:", ndim_4_Tensor.shape)
5 print("Elements number along axis 0 of Tensor:", ndim_4_Tensor.shape[0])
6 print("Elements number along the last axis of Tensor:", ndim_4_Tensor.shape[-1])
7 print('Number of elements in Tensor: ', ndim_4_Tensor.numel())
```

输出结果为:

```
Number of dimensions: 4
Shape of Tensor: torch.Size([2, 3, 4, 5])
Elements number along axis 0 of Tensor: 2
Elements number along the last axis of Tensor: 5
Number of elements in Tensor: 120
```

1.2.2.2 形状的改变

读取形状之外,还经常需要主动修改形状,以便把数据塞进算子要求的接口里. 最常用的工具是`torch.reshape`,它在不动元素值的前提下重组维度.

```
1 # 定义一个形状为 [3,2,5] 的三维张量
2 ndim_3_Tensor = torch.tensor([[[1, 2, 3, 4, 5],
3                               [6, 7, 8, 9, 10]],
4                               [[11, 12, 13, 14, 15],
5                               [16, 17, 18, 19, 20]],
6                               [[21, 22, 23, 24, 25],
7                               [26, 27, 28, 29, 30]]])
8 print("the shape of ndim_3_Tensor:", ndim_3_Tensor.shape)
9 # torch.reshape可以在保持输入数据不变的情况下改变数据形状, 这里我们设置reshape为 [2,5,3]
10 reshape_Tensor = torch.reshape(ndim_3_Tensor, (2, 5, 3))
11 print("After reshape:", reshape_Tensor)
```

输出结果为:

```
the shape of ndim_3_Tensor: torch.Size([3, 2, 5])
After reshape: tensor([[[ 1, 2, 3],
                        [ 4, 5, 6],
                        [ 7, 8, 9],
                        [10, 11, 12],
                        [13, 14, 15]],
                      [[16, 17, 18],
                        [19, 20, 21],
                        [22, 23, 24],
                        [25, 26, 27],
                        [28, 29, 30]]])
```

对比输出可以看到,把`[3, 2, 5]`重塑为`[2, 5, 3]`保留了按原逻辑顺序读取的元素序列. 这个例子中的张量连续存储,重塑后可以共享存储;对于转置等操作得到的非连续张量,`reshape`也可能复制数据. 若要交换轴的含义,例如把图像从“高、宽、通道”变为“通道、高、宽”,应使用`permute`,不能只改形状.

笔记



调用`reshape`时有一个常用小技巧:

- 某一维写成`-1`表示让框架按“总元素数除以其余维度乘积”自动反推该维长度,因此一次`reshape`里最多只能出现一个`-1`.



分别对上文定义的`ndim_3_Tensor`进行`reshape`为`[-1]`和`reshape`为`[3, 5, 2]`两种操作,观察新张量的形状.

```

1 new_Tensor1 = ndim_3_Tensor.reshape(-1)
2 print('new Tensor 1 shape: ', new_Tensor1.shape)
3 new_Tensor2 = ndim_3_Tensor.reshape(3, 5, 2)
4 print('new Tensor 2 shape: ', new_Tensor2.shape)

```

输出结果为:

```

new Tensor 1 shape: torch.Size([30])
new Tensor 2 shape: torch.Size([3, 5, 2])

```

从结果看:第一次调用使用-1让框架推断出唯一一个维度,得到长度为30的一维向量;第二次调用显式给出三维形状[3, 5, 2],效果与传入元组等价。

除了整体重塑,还有一类高频需求是在指定位置“增加一个长度为1的轴”,常用于补齐广播维度。`torch.unsqueeze`专门做这件事,传入dim指定插入的位置。

```

1 ones_Tensor = torch.ones(5, 10)
2 new_Tensor1 = torch.unsqueeze(ones_Tensor, dim=0)
3 print('new Tensor 1 shape: ', new_Tensor1.shape)
4 # 可以多次调用unsqueeze在不同维度插入
5 new_Tensor2 = ones_Tensor.unsqueeze(1).unsqueeze(2)
6 print('new Tensor 2 shape: ', new_Tensor2.shape)

```

输出结果为:

```

new Tensor 1 shape: torch.Size([1, 5, 10])
new Tensor 2 shape: torch.Size([5, 1, 1, 10])

```

1.2.2.3 张量的数据类型

PyTorch 中可以通过`Tensor.dtype`来查看张量的数据类型,支持 bool、float16、float32、float64、uint8、int8、int16、int32、int64 和复数类型数据。

1) 从 Python 字面量创建张量时可通过`dtype`显式指定类型;若不指定,PyTorch 按字面量推断:

- Python 整数对应int64(有符号 64 位整数)。
- Python 浮点数采用当前默认浮点类型,未修改默认设置时为float32。

2) 由 Numpy 数组创建(如`torch.from_numpy()`)时,张量直接沿用原数组的`dtype`,不做隐式转换。

```

1 # 通过已知数据来创建张量并查看dtype
2 print("Tensor dtype from Python integers:", torch.tensor(1).dtype)
3 print("Tensor dtype from Python floating point:", torch.tensor(1.0).dtype)

```

输出结果为:

```
Tensor dtype from Python integers: torch.int64
Tensor dtype from Python floating point: torch.float32
```

如果想改变张量的数据类型,可以通过调用`Tensor.to()`方法来实现.

```
1 # 定义dtype为float32的张量
2 float32_Tensor = torch.tensor(1.0)
3 # Tensor.to可以将输入张量转换为指定的dtype
4 int64_Tensor = float32_Tensor.to(torch.int64)
5 print("Tensor after cast to int64:", int64_Tensor.dtype)
```

输出结果为:

```
Tensor after cast to int64: torch.int64
```

1.2.2.4 张量的设备位置

创建张量时可以传`device`参数显式选择存储位置. 本书涉及的取值只有两类: CPU与GPU (PyTorch里写作`cuda`).

未指定设备位置时,张量默认创建在CPU上. 要把张量移到GPU上,可以使用`tensor.to('cuda')`或`tensor.cuda()`. 设备位置不一致的张量之间不能直接做运算.

如下代码分别创建了CPU和GPU上的张量,并通过`Tensor.device`查看张量所在的设备位置.

```
1 # 创建CPU上的张量
2 cpu_Tensor = torch.tensor(1, device='cpu')
3 print('cpu Tensor: ', cpu_Tensor.device)
4 # 如果检测到GPU, 则创建GPU上的张量
5 if torch.cuda.is_available():
6     gpu_Tensor = torch.tensor(1, device='cuda')
7     print('gpu Tensor: ', gpu_Tensor.device)
```

在检测到GPU的机器上,输出结果为:

```
cpu Tensor: cpu
gpu Tensor: cuda:0
```

1.2.3 张量与Numpy数组转换

PyTorch张量和NumPy数组之间可以双向转换: 第1.2.2.3节已经出现过`torch.from_numpy()`把数组转为张量; 对于位于CPU、不需要梯度且数据类型与布局受NumPy支持的张量,可直接调用`Tensor.numpy()`. 处理模型输出时常用`tensor.detach().cpu().numpy()`, 先脱离求导过程,再移到CPU. 转换后的数组不再参与PyTorch的自动微分.

```

1 import numpy as np
2
3 ndim_1_Tensor = torch.tensor([1., 2.])
4 # 将当前张量转化为numpy.ndarray
5 print('Tensor to convert: ', ndim_1_Tensor.numpy())
6 # 将numpy.ndarray转化为张量
7 ndim_1_arr = np.array([3., 4.])
8 print('NumPy to Tensor: ', torch.from_numpy(ndim_1_arr))

```

输出结果为:

```

Tensor to convert: [1. 2.]
NumPy to Tensor: tensor([3., 4.], dtype=torch.float64)

```



笔记

注意: `torch.from_numpy()` 得到的张量与原 Numpy 数组共享底层内存, 对其中一个的修改会影响另一个. 如果想要一个独立的副本, 可以使用 `torch.tensor(arr)`.

1.2.4 张量的访问

1.2.4.1 索引和切片

访问和修改张量元素都依赖索引/切片语法, 规则与 Python 列表、Numpy 数组保持一致, 主要有两条:

1) 下标从 0 计起, 负值表示从尾端倒数.

2) 切片写作 `start:stop:step`, 正步长时在半开区间 $[start, stop)$ 内取元素; 三个参数省略时分别取 0、长度上限和 1. PyTorch 张量不支持 Python 列表的负步长切片, 反转顺序可用 `flip`.

1.2.4.2 访问张量

针对一维张量, 对单个轴进行索引和切片.

```

1 # 定义1个一维张量
2 ndim_1_Tensor = torch.tensor([0, 1, 2, 3, 4, 5, 6, 7, 8])
3
4 print("Origin Tensor:", ndim_1_Tensor)
5 print("First element:", ndim_1_Tensor[0])
6 print("Last element:", ndim_1_Tensor[-1])
7 print("All element:", ndim_1_Tensor[:])
8 print("Before 3:", ndim_1_Tensor[:3])
9 print("Interval of 3:", ndim_1_Tensor[::3])

```

```
10 # PyTorch暂不支持负步长[::-1], 反转用flip
11 print("Reverse:", ndim_1_Tensor.flip(0))
```

输出结果为:

```
Origin Tensor: tensor([0, 1, 2, 3, 4, 5, 6, 7, 8])
First element: tensor(0)
Last element: tensor(8)
All element: tensor([0, 1, 2, 3, 4, 5, 6, 7, 8])
Before 3: tensor([0, 1, 2])
Interval of 3: tensor([0, 3, 6])
Reverse: tensor([8, 7, 6, 5, 4, 3, 2, 1, 0])
```

维数高于一维时,索引/切片以逗号分隔,按位置对应第0维、第1维、第2维……;省略掉的尾部维度等价于写一个全选切片“:”。

```
1 # 定义1个二维张量
2 ndim_2_Tensor = torch.tensor([[0, 1, 2, 3],
3                               [4, 5, 6, 7],
4                               [8, 9, 10, 11]])
5 print("Origin Tensor:", ndim_2_Tensor)
6 print("First row:", ndim_2_Tensor[0])
7 print("First row:", ndim_2_Tensor[0, :])
8 print("First column:", ndim_2_Tensor[:, 0])
9 print("Last column:", ndim_2_Tensor[:, -1])
10 print("All element:", ndim_2_Tensor[:])
11 print("First row and second column:", ndim_2_Tensor[0, 1])
```

输出结果为:

```
Origin Tensor: tensor([[ 0, 1, 2, 3],
                        [ 4, 5, 6, 7],
                        [ 8, 9, 10, 11]])
First row: tensor([0, 1, 2, 3])
First row: tensor([0, 1, 2, 3])
First column: tensor([0, 4, 8])
Last column: tensor([ 3, 7, 11])
All element: tensor([[ 0, 1, 2, 3],
                      [ 4, 5, 6, 7],
                      [ 8, 9, 10, 11]])
First row and second column: tensor(1)
```

1.2.4.3 修改张量

索引/切片不止能取值,也能直接出现在赋值号左侧,按相同的位置规则覆盖元素。

**提醒**

这类赋值是原地写操作，旧值被直接覆盖且不会保留副本。如果反向传播需要被覆盖的旧值，PyTorch 通常会通过张量版本检查发现冲突并报错；对需要求梯度的叶子张量进行原地修改也受到限制。需要保留旧值时应先clone再修改。



```
1 # 定义1个二维张量
2 ndim_2_Tensor = torch.ones(2, 3, dtype=torch.float32)
3 print('Origin Tensor: ', ndim_2_Tensor)
4 # 修改首行为0
5 ndim_2_Tensor[0] = 0
6 print('change Tensor: ', ndim_2_Tensor)
7 # 用切片把首行改为2.1
8 ndim_2_Tensor[0:1] = 2.1
9 print('change Tensor: ', ndim_2_Tensor)
10 # 修改全部张量
11 ndim_2_Tensor[...] = 3
12 print('change Tensor: ', ndim_2_Tensor)
```

输出结果为：

```
Origin Tensor: tensor([[1., 1., 1.],
                       [1., 1., 1.]])
change Tensor: tensor([[0., 0., 0.],
                       [1., 1., 1.]])
change Tensor: tensor([[2.1000, 2.1000, 2.1000],
                       [1.0000, 1.0000, 1.0000]])
change Tensor: tensor([[3., 3., 3.],
                       [3., 3., 3.]])
```

1.2.5 张量的运算

张量类暴露了上百个算子，涵盖逐元素数学、逻辑判断、矩阵乘法等场景。以加法为例，调用方式可归为两类（共三种具体写法）：

- 1) 模块级函数`torch.add(x, y)`。
- 2) 张量方法`x.add(y)`，或更常用的运算符重载`x + y`。

代码实现如下：

```
1 # 定义两个张量
2 x = torch.tensor([[1.1, 2.2], [3.3, 4.4]], dtype=torch.float64)
3 y = torch.tensor([[5.5, 6.6], [7.7, 8.8]], dtype=torch.float64)
```

```

4 # 第一种调用方法: torch.add逐元素相加算子
5 print('Method 1: ', torch.add(x, y))
6 # 第二种调用方法
7 print('Method 2: ', x.add(y))
8 # 第三种调用方法 (最常用)
9 print('Method 3: ', x + y)

```

输出结果为:

```

Method 1: tensor([[ 6.6000,  8.8000],
                  [11.0000, 13.2000]], dtype=torch.float64)
Method 2: tensor([[ 6.6000,  8.8000],
                  [11.0000, 13.2000]], dtype=torch.float64)
Method 3: tensor([[ 6.6000,  8.8000],
                  [11.0000, 13.2000]], dtype=torch.float64)

```

三种写法得到的张量数值与形状完全一致。



笔记

从可读性看, 运算符与方法调用最贴近数学表达。本书后续除非特别需要模块级API的命名空间, 否则一律使用`x.add(y)`/`x + y`风格。



笔记

更多张量操作相关的API, 请参考PyTorch官方文档。



1.2.5.1 数学运算

张量类的基础数学函数如下:

```

1 x.abs()           # 逐元素取绝对值
2 x.ceil()          # 逐元素向上取整
3 x.floor()         # 逐元素向下取整
4 x.round()         # 取最近整数; 恰好居中时取偶数
5 x.exp()           # 逐元素计算自然常数为底的指数
6 x.log()           # 逐元素计算x的自然对数
7 x.reciprocal()    # 逐元素求倒数
8 x.square()        # 逐元素计算平方
9 x.sqrt()          # 逐元素计算平方根
10 x.sin()          # 逐元素计算正弦
11 x.cos()          # 逐元素计算余弦
12 x.add(y)         # 逐元素加
13 x.sub(y)         # 逐元素减

```

```

14 x.mul(y)                # 逐元素乘（积）
15 x.div(y)                # 逐元素除
16 x.fmod(y)               # 逐元素除并取余
17 x.pow(y)                # 逐元素幂
18 x.max()                 # 指定维度上元素最大值，默认为全部维度
19 x.min()                 # 指定维度上元素最小值，默认为全部维度
20 x.prod()                # 指定维度上元素累乘，默认为全部维度
21 x.sum()                 # 指定维度上元素的和，默认为全部维度

```

PyTorch 同时重写了 Python 的数学运算魔法方法，以下两种写法在效果上完全等价。

```

1 x + y -> x.add(y)        # 逐元素加
2 x - y -> x.sub(y)        # 逐元素减
3 x * y -> x.mul(y)        # 逐元素乘（积）
4 x / y -> x.div(y)        # 逐元素除
5 x % y -> x.remainder(y)  # 余数符号与除数一致
6 x ** y -> x.pow(y)       # 逐元素幂

```

1.2.5.2 逻辑运算

张量类的逻辑运算函数如下：

```

1 torch.isfinite(x)        # 判断张量中元素是否是有限的数字，即不包括 inf 与 nan
2 torch.equal(x, y)        # 判断两个张量的全部元素是否相等，返回 Python bool
3 x.eq(y)                  # 判断两个张量的每个元素是否相等，并返回形状相同的布尔型张量
4 x.ne(y)                  # 判断两个张量的每个元素是否不相等
5 x.lt(y)                  # 判断张量 x 的元素是否小于张量 y 的对应元素
6 x.le(y)                  # 判断张量 x 的元素是否小于或等于张量 y 的对应元素
7 x.gt(y)                  # 判断张量 x 的元素是否大于张量 y 的对应元素
8 x.ge(y)                  # 判断张量 x 的元素是否大于或等于张量 y 的对应元素
9 torch.allclose(x, y)     # 判断两个张量的全部元素是否接近，返回 Python bool

```

逻辑比较运算同样支持魔法方法（如 `==`、`<`），写法与数学运算类似，这里不再赘述。

1.2.5.3 矩阵运算

除了逐元素运算，张量类还提供常用的矩阵运算，如转置、范数和矩阵乘法等：

```

1 x.T                      # 矩阵转置（仅对二维张量；高阶用 transpose / permute）
2 x.transpose(0, 1)        # 交换第 0 维与第 1 维的顺序
3 torch.linalg.norm(x, ord='fro') # 矩阵的弗罗贝尼乌斯范数（Frobenius 范数）
4 torch.linalg.vector_norm(x - y, ord=2) # (x-y) 的 L2 范数
5 x.matmul(y)              # 矩阵乘积，等价于 x @ y

```

部分矩阵运算也能作用在高阶张量上。以 `matmul` 为例，它只对最后两个维度做矩阵乘法，其余维度被当作批维度（batch dim）。若 *x* 形状为 `[j, k, n, m]`，*y* 形状为 `[j, k, m, p]`，则 `x.matmul(y)` 的输出形状为 `[j, k, n, p]`。

1.2.5.4 广播机制

PyTorch 许多运算都支持**广播** (Broadcasting): 当参与运算的张量形状不同时, 框架按规则把它们视为同一形状, 再逐元素计算. 例如形状 $[3, 1]$ 与 $[1, 4]$ 相加时, 两者都要扩展, 结果为 $[3, 4]$. 广播不要求先把输入显式复制成完整数组.

广播机制的条件 PyTorch 沿用 Numpy 的广播规则, 可触发广播的两个张量需要满足:

- 1) 从最后一维起对齐, 缺少的前导维度视为长度 1; 零维标量也可以参与广播.
- 2) 对齐后的每一维长度要么相等, 要么其中一个为 1.

代码实现如下:

```
1 # 当两个张量的形状一致时, 可以广播
2 x = torch.ones(2, 3, 4)
3 y = torch.ones(2, 3, 4)
4 z = x + y
5 print('broadcasting with two same shape tensor: ', z.shape)
6
7 x = torch.ones(2, 3, 1, 5)
8 y = torch.ones(3, 4, 1)
9 # 从后往前依次比较:
10 # 第一次: y的维度大小是1
11 # 第二次: x的维度大小是1
12 # 第三次: x和y的维度大小相等, 都为3
13 # 第四次: y的维度不存在
14 # 所以x和y是可以广播的
15 z = x + y
16 print('broadcasting with two different shape tensor:', z.shape)
```

输出结果为:

```
broadcasting with two same shape tensor: torch.Size([2, 3, 4])
broadcasting with two different shape tensor: torch.Size([2, 3, 4, 5])
```

两组张量都满足广播规则, 所以相加成功. 下面再换一组形状为 $[2, 3, 4]$ 和 $[2, 3, 6]$ 的张量, 观察是否还能广播:

```
1 x = torch.ones(2, 3, 4)
2 y = torch.ones(2, 3, 6)
3 z = x + y
```

输出结果为:

```
RuntimeError: The size of tensor a (4) must match the size of tensor b (6) at non-
singleton dimension 2
```

最后一维上 4 与 6 既不相等也都不是 1, 违反广播规则, 因此运算失败.

广播机制的计算规则 确认两个张量可以广播后,广播结果的形状按如下两步推导:

- 1) 若两个张量的阶数不同,则在较短形状的前面补1,直到两者阶数相同;
- 2) 对齐后长度相等就保留该长度;若其中一个为1,则取另一个的长度.对本节长度均大于0的例子,这等价于逐维取较大值;长度0与1广播时结果长度为0.

举例: x 形状为 $[2, 3, 1, 5]$, y 形状为 $[3, 4, 1]$.先把 y 前补1,记为 $[1, 3, 4, 1]$;再按位取最大:第一维 $\max(2, 1) = 2$,第二维 $\max(3, 3) = 3$,第三维 $\max(1, 4) = 4$,第四维 $\max(5, 1) = 5$,因此结果形状为 $[2, 3, 4, 5]$.

矩阵乘法`torch.matmul`在深度学习中使用极频繁,它的广播规则需要单独记忆:

- 1) 两个张量都是一维,结果为标量(点积).
- 2) 两个张量都是二维,结果为矩阵乘积.
- 3) x 是一维、 y 是二维:把 x 临时视为 $[1, D]$ 做矩阵乘,再去掉前置维度.
- 4) x 是二维、 y 是一维:结果为矩阵与向量的乘积.
- 5) 至少一个张量高于二维时,最后两维做矩阵乘,其余维度按广播规则对齐;若另一输入是一维,先按前述规则补轴,再在结果中去掉该轴.例如 x 形状为 $[j, l, n, m]$ 、 y 形状为 $[k, m, p]$,则输出形状为 $[j, k, n, p]$.

代码示例如下:

```
1 x = torch.ones(10, 1, 5, 2)
2 y = torch.ones(3, 2, 5)
3 z = torch.matmul(x, y)
4 print('After matmul: ', z.shape)
```

输出结果为:

```
After matmul: torch.Size([10, 3, 5, 5])
```

从输出可见,高阶张量上的`matmul`也按上述规则广播了非矩阵维度.

动手练习 1.2

给定 x 形状为 $[4, 1, 6]$ 、 y 形状为 $[1, 5, 1]$,先在纸上按广播规则推断 $x + y$ 的结果形状,再用代码验证你的推断:

- 1) 不运行代码,先用本节“补1对齐、逐维取较大”的两步规则手算 $x + y$ 的形状;随后构造这两个张量、做相加并打印`z.shape`,核对手算结果是否一致.
- 2) 把 y 改成形状 $[1, 5, 2]$ 再相加,观察报错信息,结合广播条件说明此时为何无法广播.
- 3) 用`unsqueeze`或`reshape`改写其中一个张量,使这一对张量重新满足广播规则、运算得以成立;写出改写后的形状并打印相加结果,再说明你的改写是如何让冲突的那一维恢复可广播的.



笔记



PyTorch 的 API 分为 `inplace` (原位) 和非原位两类: 原位操作直接覆盖原张量, 非原位操作则返回新张量. 约定上, 函数名以下划线 “_” 结尾的版本即原位操作, 例如 `x.add(y)` 返回新张量, 而 `x.add_(y)` 会改写 `x` 本身. 原位操作虽节省内存, 但可能破坏计算图, 使用时需谨慎.

动手练习 1.3



动手把本节讲过的索引切片、数学运算、逻辑运算与矩阵运算各试几个, 熟悉常用张量操作及其返回形状.

1.3 算子

神经网络等复杂模型本质上是一个复合函数, 输入是特征, 输出是预测或损失. 把一个 L 层网络写成嵌套形式:

$$y = f_L(\cdots f_2(f_1(x))), \quad (1.1)$$

其中每个 $f_l(\cdot)$ 既可带参数也可不带参数, x 为输入特征, y 为损失. 从 x 到 y 的过程称为**前向计算**. 要训练网络, 则需要计算损失对所有参数的偏导数 (即梯度). 先用标量说明: 假设 $a_l = f_l(a_{l-1})$ 包含参数 θ_l , 由链式法则有

$$\frac{\partial y}{\partial \theta_l} = \frac{\partial a_l}{\partial \theta_l} \frac{\partial y}{\partial a_l} \quad (1.2)$$

$$= \frac{\partial f_l}{\partial \theta_l} \frac{\partial a_{l+1}}{\partial a_l} \cdots \frac{\partial y}{\partial a_{L-1}}, \quad (1.3)$$

其中 $y = a_L$ 是最末层 f_L 的输出.

如果令 $\delta_l \triangleq \frac{\partial y}{\partial a_l}$, 则相邻层之间的梯度满足递推关系

$$\delta_{l-1} = \frac{\partial a_l}{\partial a_{l-1}} \delta_l \quad (1.4)$$

$$\triangleq b_l(\delta_l). \quad (1.5)$$

沿前向方向计算 $a_l = f_l(a_{l-1})$ 的是**前向函数**, 把上游梯度传回输入的 $\delta_{l-1} = b_l(\delta_l)$ 是**反向函数**. 推广到向量和矩阵时, 反向函数要按张量的形状完成相应的雅可比乘积; 不能把标量乘法原封不动地用于所有张量.

只要为每个基础函数同时实现前向和反向两套计算, 复杂模型就能通过它们的组合得到, 梯度也能由链式法则自动反向传播. 深度学习框架把这种带前向、反向双计算的基础单元称为**算子**.

1.3.1 算子定义

算子是搭建模型的最小复用单元,每个算子绑定一对函数:前向用于计算结果,反向用于计算梯度.为了让后续示例统一接口,本书把算子抽象成基类Op:

```

1  class Op:
2      """算子基类: 子类按需实现 forward / backward.
3
4      - 实例可以像函数那样调用: model(...) 等价于 model.forward(...),
5        这一约定与 PyTorch 内置层 (如 nn.Linear) 保持一致.
6      - forward 的参数数量与含义由子类约定: 算子常用单输入,
7        损失常用 (logits, labels).
8      - backward 由用户手动推导: 通常接收上游对本算子输出的梯度,
9        返回对输入或参数的梯度; 本书的损失/模型算子有时直接接收 labels,
10       具体签名以各子类为准.
11     """
12
13     def __call__(self, *args, **kwargs):
14         return self.forward(*args, **kwargs)
15
16     def forward(self, *args, **kwargs):
17         raise NotImplementedError
18
19     def backward(self, *args, **kwargs):
20         raise NotImplementedError

```

基类把“怎么调用”和“必须实现什么”两件事钉死: `__call__`用`*args, **kwargs`透传到`forward`, 因此调用模型既可写`model.forward(...)`也可像函数一样`model(...)`; `forward`和`backward`都留作待重写的钩子,子类按各自语义补全签名与计算逻辑.

下面以 $g = \exp(a \times b + c \times d)$ 为例,先分别实现加法、乘法和指数三个算子,再把它们组合起来计算 g . 这里的输入都是标量;若扩展到带广播的张量运算,反向时还要把广播展开的维度求和,恢复各输入的形状.

1.3.1.1 加法算子

加法算子最为简单,图1.3给出其前向和反向流程.

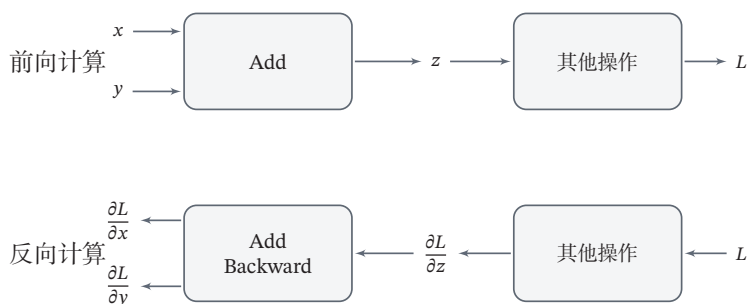


图 1.3 加法算子的前向和反向计算过程

前向计算 前向直接给出 $z = x + y$.

反向计算 设最终损失为 L , 记 $\delta_z = \frac{\partial L}{\partial z}$ 、 $\delta_x = \frac{\partial L}{\partial x}$ 、 $\delta_y = \frac{\partial L}{\partial y}$. 算子接收上游梯度 δ_z , 需要返回输入端的 δ_x 与 δ_y . 由链式法则可得 $\delta_x = \delta_y = \delta_z$, 即加法算子把梯度原样向两个输入分发.

代码实现如下:

```

1 class add(Op):
2     def forward(self, x, y):
3         self.x = x
4         self.y = y
5         outputs = x + y
6         return outputs
7
8     def backward(self, grads):
9         # 加法的偏导数对两个输入都是1, 因此上游梯度直接透传给两端
10        return grads, grads

```

取 $x = 1$ 、 $y = 4$, 并以上游梯度为 1 调用 `backward`, 可以验证 x 、 y 的梯度:

```

1 x = 1
2 y = 4
3 add_op = add()
4 z = add_op(x, y)
5 grads_x, grads_y = add_op.backward(grads=1)
6 print("x's grad is: ", grads_x)
7 print("y's grad is: ", grads_y)

```

输出结果为:

```

x's grad is: 1
y's grad is: 1

```

1.3.1.2 乘法算子

乘法 $z = x \times y$ 的前向同样平凡. 反向时由链式法则得到 $\delta_x = \delta_z \times y, \delta_y = \delta_z \times x$ ——即梯度需要与“另一侧”的输入相乘. 代码实现如下:

```

1 class multiply(Op):
2     def forward(self, x, y):
3         self.x = x
4         self.y = y
5         outputs = x * y
6         return outputs
7
8     def backward(self, grads):
9         grads_x = grads * self.y
10        grads_y = grads * self.x
11        return grads_x, grads_y

```

1.3.1.3 指数算子

指数运算 $z = \exp(x)$ 的反向更简单: 由 $dz/dx = \exp(x)$ 得 $\delta_x = \delta_z \times \exp(x)$. 代码实现如下:

```

1 import math
2
3 class exponential(Op):
4     def forward(self, x):
5         self.x = x
6         outputs = math.exp(x)
7         return outputs
8
9     def backward(self, grads):
10        grads = grads * math.exp(self.x)
11        return grads

```

给定 a, b, c, d 的具体取值, 依次实例化加法、乘法、指数算子, 按 $g = \exp(a \times b + c \times d)$ 的顺序组合调用即可:

```

1 a, b, c, d = 2, 3, 2, 2
2 # 实例化算子
3 multiply_ab = multiply()
4 multiply_cd = multiply()
5 add_op = add()
6 exp_op = exponential()
7 g = exp_op(add_op(multiply_ab(a, b), multiply_cd(c, d)))
8 print('g: ', g)

```

输出结果为:

g: 22026.465794806718

**动手练习 1.4**

从`exp_op.backward(1)`开始,依次调用加法和两个乘法算子的反向函数,验证四个输入的梯度分别为`bg`、`ag`、`dg`、`cg`。再试着复用同一个乘法实例计算两次,观察它缓存的`x`、`y`怎样被覆盖,说明为什么本例要创建两个乘法实例。

**1.3.2 自动微分机制**

主流深度学习框架几乎都内置了**自动微分** (Automatic Differentiation, AD), 开发者只需写前向计算, 框架便能依据它自动构造对应的反向计算。

**笔记**

自动微分把数值计算拆分为若干**原子操作** (Elementary Operation)。反向模式自动微分记录这些操作之间的依赖, 构造**计算图** (Computational Graph), 并保存求导需要的前向信息。沿图反向执行各操作的求导规则, 就能用链式法则得到梯度, 无须预先生成完整的符号表达式。



PyTorch 采用动态计算图: 在前向执行过程中边算边记录算子与张量, 并在每个有梯度需求的张量上挂接对应的反向操作, 调用`backward()`时再按记录回放。

**笔记**

查看叶子张量的梯度可访问其`.grad`属性, 例如`x.grad`。



下面以一个最小例子说明这一过程。定义张量`a`和`b`, 只让`a`需要梯度。虽然不求`b`的梯度, 但计算 $\partial(ab)/\partial a = b$ 仍然需要它的值。

```
1 # 定义张量a, requires_grad=True代表参与梯度计算
2 a = torch.tensor(2.0, requires_grad=True)
3 # 定义张量b, requires_grad=False代表不参与梯度计算
4 b = torch.tensor(5.0, requires_grad=False)
5 c = a * b
6 # 自动计算反向梯度
7 c.backward()
8 print(f"Tensor a's grad is: {a.grad}")
9 print(f"Tensor b's grad is: {b.grad}")
10 print(f"Tensor c's grad is: {c.grad}")
```

输出结果为：

```
Tensor a's grad is: 5.0
Tensor b's grad is: None
Tensor c's grad is: None
```

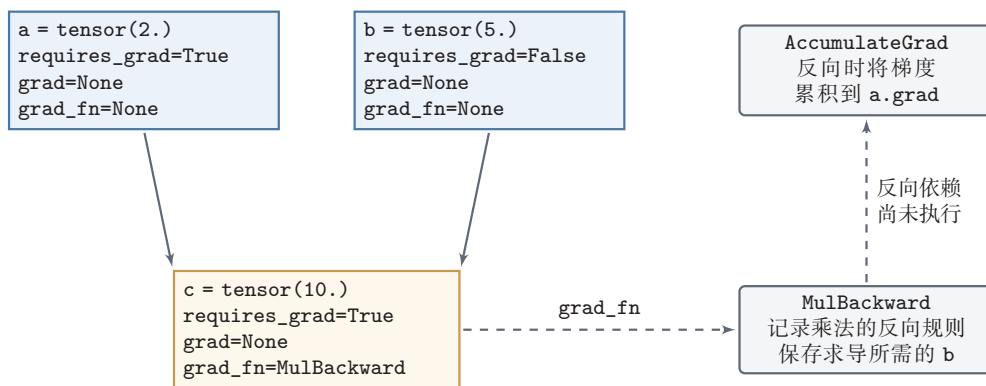
下面分阶段拆解上述代码的执行过程。

1.3.2.1 前向执行

运行到`c=a*b`时, 前向结果已经算出, 反向计算所需的依赖也已记录. 此时可以直接检查三个张量的属性:

- 1) `a` 是用户创建的叶子张量, `a.requires_grad=True`, 但 `a.grad` 仍为 `None`. 设置求导标记不会立即生成一个装着梯度的张量; 叶子张量的 `grad_fn` 也为 `None`.
- 2) `b.requires_grad=False`, 本次计算不为 `b` 累积梯度.
- 3) `c` 由可微运算产生且依赖 `a`, 因此 `c.requires_grad=True`. `c.grad_fn` 指向乘法的反向节点; 这个节点记录如何把上游梯度传向 `a` 的梯度累积节点.

图1.4区分了张量的前向数值、反向节点和 `.grad` 属性. `grad_fn` 描述结果如何产生, `.grad` 保存已经累积的梯度, 两者不能混为一谈.

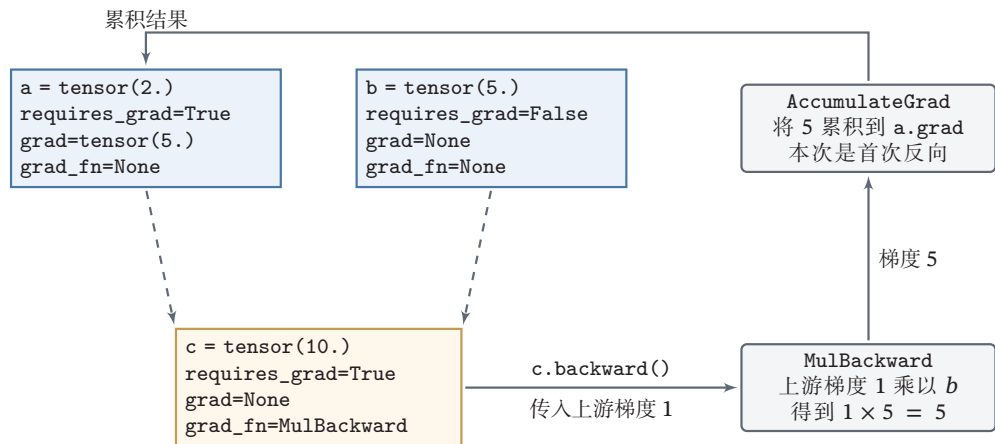


前向数值已经算出; 虚线表示已记录的反向依赖。
尚未调用 `backward()`, 三个张量的 `.grad` 均为 `None`。

图 1.4 调用 `backward()` 前的计算图状态

1.3.2.2 反向执行

调用 `c.backward()` 时, 标量输出的上游梯度默认为 1. 乘法的反向计算把 $1 \times b = 5$ 传给 `a`, 因此 `a.grad` 得到 `tensor(5.)`, 而 `b.grad` 仍为 `None`. 图1.5中的上游梯度 1 是计算过程中传递的值, 并不意味着 `c.grad` 被保存: `c` 是非叶子张量, 默认不保留它的梯度, 直接访问还会产生提示. 调试时可在反向之前调用 `c.retain_grad()`, 此后才会保存 `c.grad`.



实线表示本次反向计算,虚线保留前向依赖。
c 的梯度参与了计算,但默认不保存在 *c.grad* 中;*b* 不需要梯度。

图 1.5 调用backward()后的计算图状态

1.3.2.3 梯度累加与清零

PyTorch 每次backward()都会把新算出的梯度加到.grad上,而不是覆盖. 这种设计便于实现“梯度累积”一类技巧,但常规训练时必须先把上一步的梯度清零,否则会越来越大,导致参数更新方向出错。

```
1 x = torch.tensor([1.0, 2.0, 3.0], requires_grad=True)
2 # 第一次backward
3 y = (x ** 2).sum()
4 y.backward()
5 print('1st backward, dy/dx =', x.grad) # [2., 4., 6.]
6 # 不清零, 再做一次 —— 梯度会累加
7 y2 = (x ** 2).sum()
8 y2.backward()
9 print('2nd backward, dy/dx =', x.grad) # [4., 8., 12.] —— 累加了
10 # 训练循环里要先清零再backward
11 x.grad.zero_()
12 (x ** 2).sum().backward()
13 print('after zero_(), dy/dx =', x.grad) # 回到 [2., 4., 6.]
```

输出结果为:

```
1st backward, dy/dx = tensor([2., 4., 6.])
2nd backward, dy/dx = tensor([4., 8., 12.])
after zero_(), dy/dx = tensor([2., 4., 6.])
```

动手练习 1.5

用一个`requires_grad=True`的张量, 亲手对比“每步清零”与“故意不清零”两种训练步对梯度的影响:

1) 选定一个简单的标量损失(例如`(x ** 2).sum()`), 固定`x`不更新, 每次重新计算损失. 一种写法在反向前检查`x.grad is not None`再调用`x.grad.zero_()`, 另一种不清零. 初次反向前`x.grad`为`None`, 不能直接调用其方法.

2) 两种写法各连续求导若干次, 每次打印`x.grad`, 记录梯度的变化. 若进一步更新`x`, 应预期其梯度也随之改变.

3) 对比两条趋势: 清零的那条每步是否稳定, 不清零的那条为何会越加越大; 结合本节“`.grad`是累加而非覆盖”的说明, 解释为什么常规训练循环必须先清零再`backward()`, 并说说“忘记清零”会让参数更新方向出现什么偏差.

**1.3.2.4 关闭梯度**

推断阶段只做前向时, 用`torch.no_grad()`上下文管理器包裹相关代码, 可以省去记录反向图的开销. `tensor.detach()`返回脱离当前计算图的张量, 但仍与原张量共享存储; 需要独立副本时使用`tensor.detach().clone()`. `no_grad`不会自动切换模型的训练与推断行为, `model.eval()`也不会自动关闭求导, 这两个操作各有用途.

```
1 x = torch.tensor([1.0, 2.0, 3.0], requires_grad=True)
2 # 推断阶段关掉autograd, 节省内存
3 with torch.no_grad():
4     pred = x * 2
5     print('pred.requires_grad:', pred.requires_grad)
6 # detach: 切断计算图, 但仍共享存储
7 z = (x * 3).detach()
8 print('z.requires_grad:', z.requires_grad)
```

输出结果为:

```
pred.requires_grad: False
z.requires_grad : False
```

1.3.3 预定义的算子

从零造轮子虽然透明, 但代码会膨胀且容易出错. PyTorch 直接内置了两类常用算子可以复用: 一类是`+`, `-`, `×`, `/`, `exp()`等基础数学运算; 另一类是仿射变换、Logistic、卷积等中间层算子, 可拼装成完整模型.

这些层使用统一接口`torch.nn.Module`, 便于登记参数、组合子模块、切换训练模式以及保存状态. 自动微分则来自`forward`中实际执行的可微张量运算, 即使不继承`nn.Module`也能使用. 调用模块时通常写`model(x)`, 这样 PyTorch 还能执行注册的钩子等逻辑; 直接调用`model.forward(x)`会绕过这些处理.

1.3.4 本书中实现的算子

为了让读者深入理解模型与算法的内部细节,本书额外维护了一个手写算子库(nndl),并据此组合模型.书中自定义算子分两类:一类继承第1.3.1节定义的Op类,需要显式实现前向和反向,便于剖析梯度推导过程;另一类继承PyTorch的`torch.nn.Module`,方便与官方算子混搭,专注于复杂模型的搭建.

完整算子清单见表1.1,名称以Model_开头者为完整模型.

表 1.1 本书中使用的算子

算子名	章节	功能
Linear	2.2.2.1	线性算子,基于自定义Op类实现
Model_LR	3.1.2	Logistic 回归模型,是线性函数与 Logistic 函数的组合
Model_SR	3.2.2.2	Softmax 回归模型,是线性函数与 Softmax 函数的组合
Logistic	4.2.4.3	Logistic 激活函数算子,基于自定义Op类实现
Linear	4.2.4.4	线性层算子,基于自定义Op类实现
Model_MLP_L2	4.2.4.5	两层前馈神经网络,基于自定义Op类实现
Model_MLP_L2_V2	4.3.1	两层前馈神经网络,基于 PyTorch 算子实现,激活函数为 Sigmoid
IrisMLP	4.5.3	两层前馈神经网络,基于 PyTorch 算子实现,输出层不带激活函数
BinaryCrossEntropy	4.2.4.2	二分类交叉熵损失函数,基于自定义Op类实现
Model_MLP_L5	4.4.2.1	五层前馈神经网络
Conv2D	5.2.1	带步长和零填充的二维卷积层算子
Pool2D	5.2.2	汇聚层算子
Model_LeNet	5.3.2	LeNet-5 网络
ResBlock	5.4.1.1	浅层残差网络中的残差单元结构
Model_ResNet18	5.4.1.2	ResNet18 残差网络
SRN	6.1.2.2	SRN 层算子
Model_RNN4SeqClass	6.1.2.4	基于 RNN 的序列分类模型,使用最后时刻的状态进行分类
LSTM	6.3.1	LSTM 层算子
Model_BiLSTM_FC	6.4.2	基于双向 LSTM 的文本分类模型
MLP	7.4.2.1	自定义多层感知器,指定层数、初始化方法、激活函数等

表 1.1 本书中使用的算子（续）

算子名	章节	功能
BatchNorm	7.5.1.1	批量规范化算子
LayerNorm	7.5.2.1	层规范化算子
BinaryCrossEntropyWithLogits	7.6.2.3	以 logits 为输入的二分类交叉熵损失函数，基于自定义 <code>Op</code> 类实现
BinaryCrossEntropyWithLogits	7.6.3	以 logits 为输入的二分类交叉熵损失函数，支持带 ℓ_2 正则化项
AdditiveScore	8.1.2.1	加性注意力打分算子
DotProductScore	8.1.2.1	点积注意力打分算子
Model_LSTMAttention	8.1.2.5	双向 LSTM 和注意力机制网络
QKVAttention	8.2.1.2	QKV 注意力算子
MultiHeadSelfAttention	8.2.1.3	多头自注意力算子
Model_LSTMSelfAttention	8.2.2.1	双向 LSTM 和多头自注意力机制网络
TransformerEmbeddings	8.3.2.1	Transformer 嵌入层
TransformerBlock	8.3.2.2	Transformer 组块
Model_Transformer	8.3.2.3	Transformer 模型（编码器部分）

1.3.5 本书中实现的优化器

继承 `Op` 类的算子需要搭配手写优化器使用，本书提供的优化器列表见表 1.2。

表 1.2 本书中实现的优化器

优化器	章节	功能
Optimizer	3.1.4.2	优化器基类
SimpleBatchGD	3.1.4.3	梯度下降法优化器
BatchGD	4.2.4.6	梯度下降法优化器，支持更复杂的多层模型
AdaGrad	7.3.2.1	Adagrad 优化器
RMSprop	7.3.2.2	RMSprop 优化器
Momentum	7.3.3.1	Momentum 优化器
Adam	7.3.3.2	Adam 优化器

1.4 本书中使用的数据集和实现的 Dataset 类

“模型解读”章节多采用简化的玩具数据集，便于展示算法细节；“案例实践”章节则切换到公开的真实数据集，验证模型在复杂场景下的表现。本节先汇总这些数据集，再介绍配套的 Dataset 类。

1.4.1 数据集

本书使用的数据集如下：

1) **ToyLinear150**（一维线性回归）：第 2.2.1.2 节构造，规模 150 条，按 100/50 划分训练集与测试集，由第 2.2.1.1 节的 `create_toy_data` 加噪声合成。

2) **ToySin25**（多项式回归）：第 2.3.1 节构造，规模 25 条，划分为 15 条训练 + 10 条测试，同样调用 `create_toy_data` 得到。

3) **加州房价数据集**：含 20 640 条样本，特征是 8 项街区描述（中位收入、房龄、平均房间数等），目标是房价中位数；在第 2.5 节用作回归实战。

4) **Moon1000**（弯月二分类）：第 3.1 节构造，从两簇弯月形分布带噪采样得到，二维输入、二分类标签，按 640/160/200 划分训练/验证/测试。第 3.1 与第 4.2 节直接使用该集；其底层构造函数 `make_moons` 在第 7.4.2.3 与第 7.6 节复用。

5) **Multi1000**（三簇三分类）：第 3.2.1 节构造，三个簇各对应一个类别，规模划分同 Moon1000。

6) **鸢尾花数据集 Iris**：经典花卉分类基准，3 类共 150 条，每类 50 条；每条样本由花萼长宽、花瓣长宽 4 个属性描述；第 3.3 与第 4.5 节使用。

7) **MNIST**：手写数字识别入门基准，训练集 60 000 张、测试集 10 000 张，每张为 28×28 灰度图；第 5.3.1 与第 7.2 节使用。

8) **CIFAR-10**：物体识别基准，60 000 张 32×32 彩色图，覆盖 10 个类别（各类 6 000 张）；第 5.5 节用于实战。

9) **IMDB 电影评论**：英文情感二分类，训练/测试集各 25 000 条，按评分阈值划分极性（ ≥ 7 积极、 ≤ 4 消极）；第 6.4、8.1、8.2 节贯穿 RNN 与注意力实验。

10) **DigitSum**（数字求和）：第 6.1.1 节构造，输入是一串长度可控的数字序列，仅前两位从 0~9 随机采样、其余位主要为 0，目标是预测前两位之和——用于压力测试模型的长程记忆。

11) **LCQMC**（中文问题匹配）：百度知道开放领域的语义匹配语料，弥补中文相似度数据匮乏的现状，规模为训练 238 766 条 / 验证 8 802 条 / 测试 12 500 条；第 8.3 节使用。

1.4.2 Dataset类

PyTorch通过`torch.utils.data.Dataset`抽象基类与配套的`DataLoader`,把数据集封装成可按批次迭代的对象,正好契合随机梯度下降的训练范式.本书按需扩展该基类,得到一组具体的`Dataset`子类.

具体清单见表1.3,使用细节在第4.5.1.1节集中介绍.

表 1.3 本书中构建的Dataset类

Dataset类	章节	对应的数据集
MNISTDataset	5.3.1.2	手写体数字识别数据集
CIFAR10Dataset	5.5.1.3	CIFAR10 数据集
DigitSumDataset	6.1.1.3	数字求和数据集
IMDBDataset	6.4.1.2	IMDB 数据集
LCQMCdataset	8.3.1.3	LCQMC 数据集

1.5 一个最小的训练循环

在引入`Runner`之前,先把前面学到的算子、自动微分、`Dataset`串起来,搭一个最小可运行的训练流程,便于看清整体脉络.一个标准训练循环可以拆成4步:用`nn.Module`定义模型;准备好数据;指定损失函数与优化器;写循环——前向、算损失、反向、更新.

下面以拟合二维线性关系 $y = x_1 + x_2 + \epsilon$ 为例,先定义一个两层小型多层感知器TinyMLP:

```

1 import torch.nn as nn
2 import torch.optim as optim
3
4 class TinyMLP(nn.Module):
5     """一个两层的小型多层感知器, 用来拟合y=x1+x2+噪声. """
6
7     def __init__(self, in_dim=2, hid=16, out=1):
8         super().__init__()
9         self.net = nn.Sequential(
10             nn.Linear(in_dim, hid),
11             nn.ReLU(),
12             nn.Linear(hid, out),
13         )
14
15     def forward(self, x):
16         return self.net(x)

```

随后构造合成数据、实例化模型,按标准模板写训练循环:

```
1 torch.manual_seed(0)
2 # 合成回归数据  $y = \text{sum}(x) + \text{噪声}$ 
3 X = torch.randn(256, 2)
4 y = X.sum(dim=1, keepdim=True) + 0.1 * torch.randn(256, 1)
5
6 model = TinyMLP()
7 opt = optim.Adam(model.parameters(), lr=0.05)
8 loss_fn = nn.MSELoss()
9
10 for epoch in range(200):
11     pred = model(X)           # 前向
12     loss = loss_fn(pred, y)    # 求 loss
13     opt.zero_grad()           # 清零梯度
14     loss.backward()            # 反向
15     opt.step()                 # 更新参数
16
17     if epoch % 50 == 0:
18         print(f'epoch {epoch:3d} loss {loss.item():.4f}')
```

循环的核心是前向—损失—反向—更新这一固定节拍,几乎所有基于梯度的训练任务都共享这套骨架。



笔记

每次反向传播前都要调用`opt.zero_grad()`清零梯度。如果忘记清零,PyTorch会把上一步算出的梯度继续累加到`.grad`,参数更新方向会越来越偏。

这套四步骨架在切换任务时只需局部替换:模型部分换成CNN/RNN/Transformer、损失改成交叉熵、数据切到图像或文本,循环主体几乎原样保留。下一节即将登场的Runner就是把这套骨架连同数据加载、评估和模型保存等环节统一封装成一个类,后续章节就可以只聚焦模型设计、不必每次重抄训练循环。

1.6 本书中实现的 Runner 类

一次完整的实验流程通常贯穿数据准备、模型搭建、损失与优化器选择、评价指标定义、训练、评估、预测、模型保存与加载等环节。Runner的设计目标就是把这些环节封装为统一接口,把样板代码集中维护,章节正文则专注于模型变化的部分(详见第2.4节)。

为匹配各章不断升级的需求,本书在`pytorch/nndl/runner.py`里同时维护了三个版本,复杂度逐版递增,使用时按`from nndl.runner import ...`导入:

1) **RunnerV1** (第2.5.3节): 面向有解析解的简单模型, 直接调用最小二乘等求解器, 并负责参数的保存与加载;

2) **RunnerV2** (第3.1.6节): 引入**梯度下降法**, 在训练过程中记录训练集与验证集上的损失和指标, 并保留验证集最优模型;

3) **RunnerV3** (第4.5.4节): 基于**DataLoader**做小批量训练, 支持任意评价指标, 并通过**state_dict**保存和加载 PyTorch 模型参数.

RunnerV3已能覆盖书中绝大多数任务, 后续章节都以它为起点, 根据具体场景做微调.

1.7 模型的保存与加载

训练结束后, 关掉 Notebook 再打开, 内存中的参数便消失了. 要让第二天的预测使用同一个模型, 需要先保存它的状态. PyTorch 常用**state_dict**保存模型参数和持久缓冲区, 例如 Batch-Norm 的运行统计量; 加载时再按原配置建立模型.

state_dict是一个OrderedDict, 键为参数路径 (如net.0.weight), 值为对应张量. 本节涉及的主要 API 有:

- **torch.save(obj, path)**: 把任意可序列化对象 (张量、字典、模型**state_dict**等) 保存到文件.
- **torch.load(path, weights_only=True)**: 从文件读出对象. **weights_only=True**会限制反序列化可使用的对象类型, 适合加载本书保存的参数字典; 它能降低任意代码执行的风险, 但不应据此信任来源不明的文件.
- **Module.state_dict()**: 返回参数和持久缓冲区的状态字典; 其中张量仍引用模型当前状态, 若在内存中保留某一步的快照, 需要另行复制.
- **Module.load_state_dict(d)**: 用字典d里的参数覆盖当前模型的参数.

下面以上一节训练好的TinyMLP为例, 演示保存与加载的完整流程:

```

1 import tempfile
2 from pathlib import Path
3
4 with tempfile.TemporaryDirectory() as td:
5     save_path = Path(td) / 'tinymlp.pt'
6     # 保存: 把state_dict写到文件
7     torch.save(model.state_dict(), save_path)
8     # 加载: 先新建一个同结构的模型, 再load_state_dict
9     model2 = TinyMLP()
10    model2.load_state_dict(torch.load(save_path, weights_only=True))
11    model.eval()
```

```

12     model2.eval()          # 两份模型都切换到推断模式
13     # 验证：两份模型在同样输入下输出应当一致
14     with torch.no_grad():
15         a = model(X[:5])
16         b = model2(X[:5])
17     print('max abs diff:', (a - b).abs().max().item())

```

比较两份模型时,要让它们处于相同模式.推断时调用`eval()`,会使Dropout停止随机置零,并使采用默认设置的BatchNorm使用运行统计量.本例的TinyMLP只有线性层和ReLU,两种模式的计算相同;加入Dropout或BatchNorm后,这个区别才会影响输出.恢复训练则应调用`train()`.

笔记



保存`state_dict`还是保存整个模型? 官方推荐前者: 它只与参数名和形状有关,与类的具体定义路径解耦——即便后续改名、移动模型代码,只要结构不变,旧权重仍能加载.反之,保存整个模型对象写法虽短,但反序列化时`pickle`会去找原始类定义,一旦类被改名或移动便会失败.

如果还希望保存优化器状态和当前epoch,以便从已有训练进度继续,通常把它们打包成字典统一`torch.save`:

```

1 checkpoint = {
2     'epoch':    epoch,
3     'model':    model.state_dict(),
4     'optimizer': opt.state_dict(),
5     'loss':     loss.item(),
6 }
7 torch.save(checkpoint, 'ckpt.pt')

```

加载时按字典逐项还原: 先用`ckpt = torch.load('ckpt.pt', weights_only=True)`读入字典,再分别通过`model.load_state_dict(ckpt['model'])`、`opt.load_state_dict(ckpt['optimizer'])`写回各模块,并从下一epoch继续.若希望尽量重现中断前的训练轨迹,还需保存随机数状态、学习率调度器状态和数据迭代进度等信息.书中RunnerV3的保存接口主要保存模型状态,不能据此假定优化器和训练进度也已保存.

**动手练习 1.6**

在本节TinyMLP的保存/加载示例上故意“踩坑”，观察并记录两个易漏细节各自的后果：

1) 把加载后的`model2.eval()`这一行去掉，重跑“两份模型在相同输入下输出比较”的代码，观察`max abs diff`是否仍为0；再思考：如果模型里含有`Dropout`或`BatchNorm`，漏掉`eval()`会让推断输出出现什么问题，为什么。

2) 把`torch.load(..., weights_only=True)`改成`weights_only=False`，观察加载是否仍然成功、是否出现警告或报错；再结合正文说明`weights_only=True`为何是更安全的默认选项（加载来源不明的权重时它能挡住什么）。

3) 综合上述两次实验，用一两句话总结`eval()`与`weights_only`各自的作用，以及在部署或复现他人模型时为什么这两步都不能省。

1.8 小结

现在可以让一个小模型读入数据、计算损失、更新参数，再保存到文件并重新加载。遇到训练异常时，先检查张量的形状、类型和设备，再检查梯度是否产生、是否按预期清零，比盲目增加网络层数更容易找到原因。下一章从一条直线开始，把这些操作放进完整的学习实验：如何准备数据，又如何判断模型学到了规律。具体API可随用随查PyTorch官方文档^③。

^③ <https://pytorch.org/docs/stable/index.html>.

第2章 机器学习概述

给出一组房屋的特征和价格,画一条尽量贴近这些样本的直线并不难.但换一批房屋,这条直线还能预测得准吗?如果改用一条弯曲的曲线,训练样本拟合得更好了,预测也一定更好吗?本章从这两个问题出发,学习如何用数据建立模型,又如何检验模型学到的规律. **机器学习** (Machine Learning, ML) 正是研究这类从数据中学习、并将所学用于未见数据的方法.

本章内容与《神经网络与深度学习》第2章**机器学习概述**对应.在动手实践之前,建议先回顾图2.1列出的关键知识点,以便后续概念衔接顺畅.

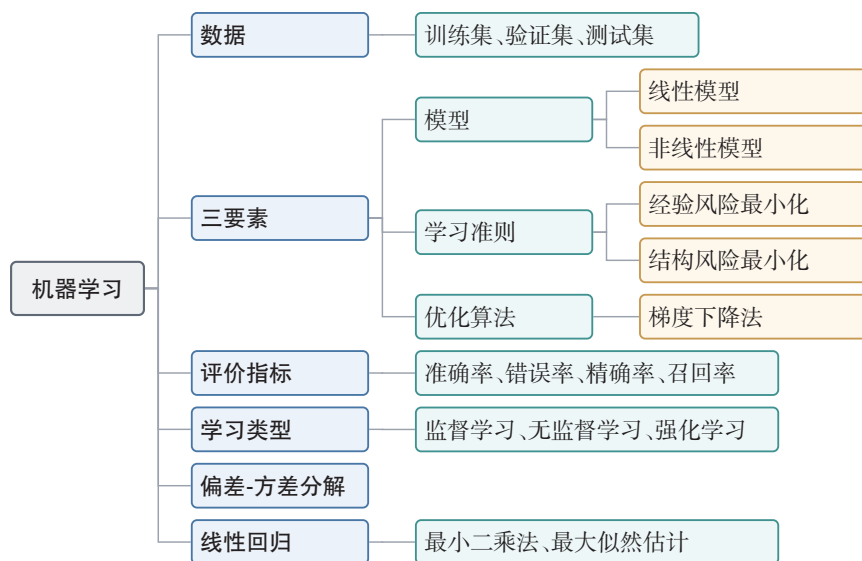


图 2.1 机器学习关键知识点回顾

本章分为两部分:

- **模型解读**: 拆解机器学习实践中的五个要素 (数据、模型、学习准则、优化算法、评价指标), 把原理讲解和代码实现交织在一起, 逐项说明它们在程序中的职责.
- **案例实践**: 以线性回归为载体, 用加州房价预测任务串起数据处理、模型构建、训练配置、**Runner**封装、训练与预测全流程.

2.1 机器学习实践五要素

本书把监督学习实践组织为下面 5 个要素：

- 1) **数据**：与任务匹配的数据集，通常进一步切分为训练集、验证集和测试集。
- 2) **模型**：从样本特征到标签的映射，通常写成带参数的函数；机器学习的目标是找到一组让预测尽量“准”的参数。
- 3) **学习准则**：定义“准”的标准，一般由训练集平均损失叠加正则化项构成，把建模问题转化为最优化问题。
- 4) **优化算法**：求解上述最优化问题的方法，常用的是梯度下降及其变种。
- 5) **评价指标**：衡量模型预测好坏的尺子，与任务目标对齐。

笔记



《神经网络与深度学习》第 2.2 节将机器学习抽象为“模型、学习准则、优化算法”三要素。本书面向实践，因此把“数据”和“评价指标”也纳入进来，形成五要素的框架。《通识版》也以这三要素解释训练，并分别介绍数据与评价；五要素是将这些实践条件一并列出。

按流程视角，机器学习实践可以划分为 4 个阶段：

- 1) **模型准备**：备齐五要素，并完成数据预处理，例如特征归一化、类别编号映射等。
- 2) **模型训练**：结合学习准则与优化算法，在训练集上更新参数、在验证集上挑选超参数，直到学习准则收敛或验证指标饱和。
- 3) **模型评价**（又称**模型测试**（Model Test））：在保留的测试集上按给定指标度量训练成果。
- 4) **模型预测**（又称**模型推断**（Model Inference））：用训练好的模型预测新样本的标签；预测前需对输入做与训练阶段一致的预处理，预测后再把模型输出转回业务可理解的形式。

笔记



工业场景中往往还有一个**模型部署**（Model Deployment）阶段：把训练好的模型挂到生产环境对外提供推断服务，并配套做推断加速、在线更新、监控与日志等工作。本书重点放在训练侧，部署细节不展开。

图 2.2 对照说明了五要素如何在训练与评价阶段协同工作：**训练阶段**消费训练集与验证集，结合模型、损失函数、优化算法产出学习好的模型；**评价阶段**则把测试集和评价指标接入到学习好的模型上，得到性能数值。

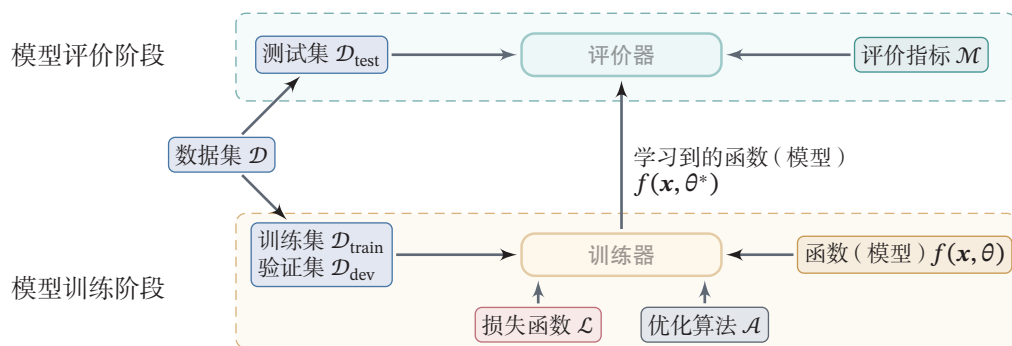


图 2.2 机器学习实践流程中五要素在模型训练和模型评价阶段的使用示例

下面依次拆解五个要素。

2.1.1 数据

如果训练数据没有覆盖将来要处理的情形，模型即使把现有样本拟合得很好，也可能在使用时失效。增加有代表性的样本、核查错误标签，往往比单纯增加样本数量更有效。数据预处理应避免信息泄漏：完成必要的的数据核验与去重后，先划分训练集、验证集和测试集，再只用训练集估计归一化等变换所需的统计量，并将同一变换应用于验证集和测试集。

- **训练集**：用来更新模型参数；在其上的平均损失称为**训练误差**，也称**经验误差**或**经验风险**。
- **验证集**：用于超参数选择。常见做法是用同一份训练集，在多组超参数设置下分别训练出多个模型，再在验证集上比较，挑选最好的一组用于最终评价。
- **测试集**：用于估计模型面对未见数据时的表现。这一估计的可信度取决于测试样本是否独立、是否代表实际使用场景，以及样本数量是否充足。

数据划分时存在一个权衡：训练数据较多通常有助于参数估计，测试数据较多则有助于降低评价的不确定性。独立同分布样本可按 7:3 或 8:2 先切出测试集，再从剩余数据中留出验证集；比例应随数据规模调整。来自同一用户、同一文档或相邻时间段的样本可能高度相关，此时应按组或按时间划分，不能只随机打乱行号。



提醒

测试集只能在最后一步使用。训练过程中绝不能让测试数据影响任何决策——包括调超参、改模型、做特征工程，否则评价结果会失真。



2.1.2 模型

有了数据,下一步就是从候选函数集合 $\mathcal{F} = \{f_1(\mathbf{x}), f_2(\mathbf{x}), \dots\}$ 中挑出最贴近样本 $(\mathbf{x}, y)$ 真实映射关系的 $f^*(\mathbf{x})$. $\mathcal{F}$ 被称为**假设空间**,在工程实现中往往写成参数化的形式:

$$\mathcal{F} = \{f(\mathbf{x}; \theta) \mid \theta \in \mathbb{R}^D\}, \quad (2.1)$$

其中 $f(\mathbf{x}; \theta)$ 记为模型, θ 是 D 个可学习参数.

按形式分类,假设空间可分为线性和非线性两类. 最简单的**线性模型**写作

$$f(\mathbf{x}; \theta) = \mathbf{w}^\top \mathbf{x} + b, \quad (2.2)$$

其参数 θ 由权重向量 $\mathbf{w}$ 和偏置 b 组成.

将输入 $\mathbf{x}$ 先经过**非线性基函数** $\phi(\mathbf{x})$, 再做线性组合, 就把线性模型扩展为**非线性模型**, 从而拟合更复杂的关系:

$$f(\mathbf{x}; \theta) = \mathbf{w}^\top \phi(\mathbf{x}) + b, \quad (2.3)$$

其中 $\phi(\mathbf{x}) = [\phi_1(\mathbf{x}), \phi_2(\mathbf{x}), \dots, \phi_K(\mathbf{x})]^\top$ 是 K 个基函数组成的特征向量, θ 同样由 $\mathbf{w}$ 和 b 构成. 这里的非线性是相对于原始输入而言; 当基函数固定时, 模型对可学习参数仍是线性的. 后面的多项式回归正属于这种情况.

2.1.3 学习准则

想说一个模型“好不好”, 得先给“好”一个量化标准. **损失函数** (Loss Function) $\mathcal{L}(y, f(\mathbf{x}; \theta))$ 承担的就是这件事: 它是一个非负实数函数, 量化预测值与真实标签的差距; 常见的有 0-1 损失、平方损失、交叉熵损失等.

理想情况下, 我们希望让损失在真实数据分布上的**期望**最小. 但实际工作中拿不到真实分布, 只能用手头训练集上的**平均损失**作为它的经验估计.

模型在训练集 $\mathcal{D} = \{(\mathbf{x}^{(n)}, y^{(n)})\}_{n=1}^N$ 上的平均损失称为**经验风险** (Empirical Risk):

$$\mathcal{R}_{\mathcal{D}}^{\text{emp}}(\theta) = \frac{1}{N} \sum_{n=1}^N \mathcal{L}(y^{(n)}, f(\mathbf{x}^{(n)}; \theta)). \quad (2.4)$$

通常, **经验风险最小化**就能得到可用的模型. 但当模型容量过大或训练数据量不足时, 光最小化经验风险, 可能在训练集上几乎零误差, 到测试集上却表现差——这就是**过拟合** (Overfitting). 为此, 经验风险后常追加正则化项以限制模型容量.

一种做法是惩罚参数的大小, 例如使用 ℓ_1 范数或 ℓ_2 范数的平方. 把惩罚函数 $\Omega(\theta)$ 加到经验风险上, 本书称之为**结构风险** (Structure Risk):

$$\mathcal{R}_{\mathcal{D}}^{\text{struct}}(\theta) = \mathcal{R}_{\mathcal{D}}^{\text{emp}}(\theta) + \lambda \Omega(\theta), \quad (2.5)$$

其中 $\lambda \geq 0$ 为正则化系数, 控制拟合数据与限制参数大小之间的权衡. 具体使用范数还是范数平方, 以及是否包含 $1/2$ 等系数, 需要与实现保持一致.

2.1.4 优化算法

学习准则一旦确定, 就得到一个具体的优化问题. 线性回归的平方损失具有特殊的二次结构, 可以通过求解线性方程得到最优参数. 但“目标函数是凸函数”并不意味着一定存在简单的解析解: 下一章的 Logistic 回归就是通常需要迭代求解的凸优化问题. 对可微凸函数, 梯度为零的点是全局最优; 非凸目标的驻点则未必是最小值.

深度学习里最常用的是**梯度下降法** (Gradient Descent Method). 它对学习率非常敏感——取得太大, 损失会振荡甚至发散; 取得太小, 收敛又极慢.

梯度下降通常还要搭配**提前停止** (Early Stopping) 这一类“轻量正则化”: 训练时同步观察验证集指标, 一旦指标长时间没有改善就提前终止迭代, 避免模型在训练集上越走越远而失去泛化能力.

2.1.5 评价指标

评价指标 (Evaluation Metric) 用于衡量模型的实际效果: 将预测与目标对照, 得到与任务相关的分数. 训练集和验证集上的指标用于诊断与模型选择, 测试集上的指标用于最终评价. 回归任务常用均方误差等连续值指标, 分类任务则常用准确率、召回率、F1 等.

笔记



评价指标和损失函数关系紧密但并不重合. 一般先依据业务需求确定评价指标, 再据此选择尽量贴近的损失函数; 但许多评价指标不可微 (如准确率), 无法直接用作训练目标, 此时只能选用一个数学性质良好的替代品.

2.2 实现一个简单的线性回归模型

回归 (Regression) 是监督学习里的一类核心任务, 目标是从样本输入预测一个连续标签 (房价、票房、温度等); 其中以线性函数描述输入-输出关系的**线性回归** (Linear Regression) 是最朴素的一种.

本节借线性回归这一最小完整例子走一遍机器学习全流程, 方法上选用闭式的最小二乘法求解, 便于在没有循环迭代的前提下先建立整体直觉.

2.2.1 数据集构建

在动手训练前, 先合成一个小规模的线性回归数据集.

2.2.1.1 数据集构建函数create_toy_data()

create_toy_data()函数用于按指定函数或分布生成样本。

它接收的参数包括目标函数func、自变量区间、样本数量,还可选地控制高斯噪声标准差、是否注入异常点以及异常点比例,便于模拟真实数据中的噪声和离群情况。

函数实现如下:

```
1 import torch
2
3 def create_toy_data(func, interval, sample_num, noise=0.0, add_outlier=False,
4                     outlier_ratio=0.001):
5     """根据给定的函数生成样本
6     输入:
7         - func: 函数
8         - interval: x 的取值范围
9         - sample_num: 样本数目
10        - noise: 噪声标准差
11        - add_outlier: 是否生成异常值
12        - outlier_ratio: 异常值占比
13    输出:
14        - X: 特征数据, shape=[n_samples, 1]
15        - y: 标签数据, shape=[n_samples, 1]
16    """
17    # 在 interval 区间均匀采样
18    X = torch.rand(sample_num, 1) * (interval[1] - interval[0]) + interval[0]
19    y = func(X)
20    # 生成高斯分布的标签噪声
21    epsilon = torch.normal(mean=0.0, std=noise, size=y.shape)
22    y = y + epsilon
23    if add_outlier:
24        # 生成额外的异常点
25        outlier_num = int(len(y) * outlier_ratio)
26        if outlier_num != 0:
27            outlier_idx = torch.randint(low=0, high=len(y), size=(outlier_num,))
28            y[outlier_idx] = y[outlier_idx] * 5
29    return X, y
```



动手练习 2.1

`create_toy_data()` 提供了 `add_outlier` 与 `outlier_ratio` 两个参数用于注入异常点,但本章正文从未启用它们. 请尝试:

1) 把生成 `ToyLinear150` 的调用改成 `add_outlier=True`, 并把 `outlier_ratio` 依次设为 0.05、0.2 (注意默认值 0.001 在样本量 100/50 下取整为 0, 不会产生任何异常点), 重新用最小二乘法拟合, 观察估计出的 w 、 b 以及训练误差、测试误差如何随异常点比例变化.

2) 结合平方损失对较大残差的放大作用, 解释为什么最小二乘法对离群点格外敏感.

2.2.1.2 构建线性回归数据集: `ToyLinear150`

设输入特征和输出标签均为标量 (一维), 待拟合的真实函数实现如下:

```
1 # 真实函数的参数缺省值为 w=1.2, b=0.5
2 def linear_func(x, w=1.2, b=0.5):
3     return w * x + b
```

借助 `torch.rand()` 在区间内均匀采样 x , 代入真实函数得到 y , 再叠加高斯噪声, 从而模拟真实场景. 我们用这套机制生成 150 个样本 (100 训 + 50 测), 并命名为 `ToyLinear150`:

```
1 torch.manual_seed(0)
2 # 生成训练数据 (100) 与测试数据 (50)
3 X_train, y_train = create_toy_data(func=linear_func, interval=(-10, 10), sample_num=100,
4                                   noise=2)
5 X_test, y_test = create_toy_data(func=linear_func, interval=(-10, 10), sample_num=50,
6                                  noise=2)
```

绘制散点图 (绘图代码见配套 `notebook`), 结果如图 2.3 所示: 散点紧贴在真实直线附近, 上下波动体现了噪声的存在.

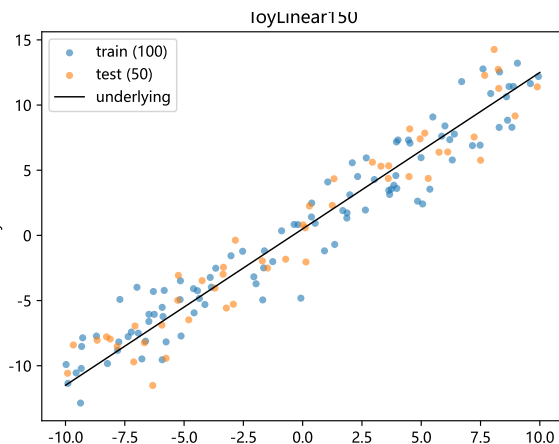


图 2.3 ToyLinear150 数据集的可视化



提醒

本书中给出的图例的颜色和实际代码的设置有一定差异。



2.2.2 模型构建

线性回归的输入是特征向量 $\mathbf{x} \in \mathbb{R}^D$, 输出是连续标签 $y \in \mathbb{R}$. 模型形式为

$$f(\mathbf{x}; \mathbf{w}, b) = \mathbf{w}^\top \mathbf{x} + b, \quad (2.6)$$

其中 $\mathbf{w} \in \mathbb{R}^D$ 与 $b \in \mathbb{R}$ 是可学习参数。



笔记

《神经网络与深度学习》主书使用增广权重向量把偏置吸收进 $\mathbf{w}$, 便于公式推导. 本书为了与代码一致, 仍保留显式的 b .



为充分利用 GPU 的并行能力, 工程中通常把 N 个样本组成一批一起计算. 批处理形式写作

$$\mathbf{y} = \mathbf{X}\mathbf{w} + b, \quad (2.7)$$

其中 $\mathbf{X} \in \mathbb{R}^{N \times D}$ 是按行堆叠的样本矩阵, $\mathbf{y} \in \mathbb{R}^N$ 为 N 个预测值。



提醒

本章采用“样本数 $\times$ 特征数”的张量布局, 每行代表一个样本 $\mathbf{x}$. 这与《神经网络与深度学习》主书中 $\mathbf{x}$ 为列向量的约定刚好转置, 对应推导时需留意维度匹配。



2.2.2.1 线性算子

公式(2.7)本身只是矩阵乘法,可以一行代码搞定:

```
1 # X: tensor, shape=[N, D]
2 # w: shape=[D, 1]
3 # b: shape=[1]
4 # y_pred: tensor, shape=[N, 1]
5 y_pred = X @ w + b
```

但为了与全书的算子化范式保持一致,下面把它封装成第1.3节定义的算子形式:

```
1 import torch
2 from nndl import Op
3
4 torch.manual_seed(0)          # 设置随机种子
5
6 class Linear(Op):
7     def __init__(self, input_size):
8         """
9         输入:
10             - input_size: 模型要处理的数据特征向量长度
11         """
12         self.input_size = input_size
13         # 模型参数
14         self.params = {
15             'w': torch.randn(input_size, 1),
16             'b': torch.zeros(1),
17         }
18         # 前向函数
19     def forward(self, X):
20         """
21         输入:
22             - X: tensor, shape=[N, D]
23             注意,这里的 X 矩阵是由 N 个 x 行向量拼接成的
24         输出:
25             - y_pred: tensor, shape=[N, 1]
26         """
27         N, D = X.shape
28         assert D == self.input_size          # 输入数据维度合法性验证
29         return X @ self.params['w'] + self.params['b']
30     # 注意,这里的 X 矩阵是由 N 个 x 行向量拼接成的
31     input_size = 3
32     N = 2
33     model = Linear(input_size)
34     X = torch.randn(N, input_size)          # 生成 2 个维度为 3 的数据
35     y_pred = model(X)
```

```
36 print("y_pred:", y_pred)                                # 输出结果个数也是 2 个
```

输出结果为:

```
y_pred: tensor([[4.2414],
               [1.9428]])
```

2.2.3 损失函数

回归任务关注连续值预测——给定特征, 输出一个实数. 衡量预测与真实之间的差距时, 最常用的学习准则是均方误差 (Mean Squared Error, MSE).

记 $\mathbf{y}, \hat{\mathbf{y}} \in \mathbb{R}^N$ 分别为真实标签和预测值. 通常的均方误差为 $\|\hat{\mathbf{y}} - \mathbf{y}\|^2/N$; 为使后面的导数更简洁, 本章采用带 1/2 系数的平方损失

$$\mathcal{L}(\mathbf{y}, \hat{\mathbf{y}}) = \frac{1}{2N} \|\hat{\mathbf{y}} - \mathbf{y}\|^2 = \frac{1}{2N} \|\mathbf{X}\mathbf{w} + \mathbf{b} - \mathbf{y}\|^2, \quad (2.8)$$

其中 $\mathbf{b}$ 为 N 维向量, 所有元素取值都为 b . 下面的 `mean_squared_error` 函数及损失曲线均沿用这一带 1/2 系数的口径; 其数值是 `nn.MSELoss` 默认结果的一半. 常数缩放不改变无正则化问题的最优参数, 但会改变梯度大小, 不能直接混用学习率和指标数值.

均方误差的代码实现如下:

```
1 import torch
2
3 def mean_squared_error(y_true, y_pred):
4     """
5     输入:
6         - y_true: tensor, 样本真实标签
7         - y_pred: tensor, 样本预测标签
8     输出:
9         - error: float, 误差值
10    """
11    assert y_true.shape == y_pred.shape
12    # 带 1/2 系数, 与损失定义  $L=1/(2N) \cdot || \cdot ||^2$  一致 (求导时约掉平方的 2)
13    return 0.5 * ((y_true - y_pred) ** 2).mean()
14    # 构造一个简单的样例进行测试: [N, 1], N=2
15    y_true = torch.tensor([[-0.2], [4.9]])
16    y_pred = torch.tensor([[1.3], [2.5]])
17
18    error = mean_squared_error(y_true=y_true, y_pred=y_pred).item()
19    print("error:", error)
```

输出结果为:

```
error: 2.002500057220459
```

2.2.4 优化器

在经验风险最小化框架下, 线性回归可以借助最小二乘法直接得到 $\mathbf{w}$ 和 b 的解析解. 先对公式(2.8)中的损失关于 b 求偏导, 得

$$\frac{\partial \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})}{\partial b} = \frac{1}{N} \mathbf{1}^\top (\mathbf{X}\mathbf{w} + \mathbf{b} - \mathbf{y}), \quad (2.9)$$

其中 $\mathbf{1}$ 为 N 维的全 1 向量.

令导数等于零时, 两边的共同系数 $1/N$ 可以约去.

令上式为零, 可解出

$$b^* = \bar{y} - \bar{\mathbf{x}}^\top \mathbf{w}, \quad (2.10)$$

其中 $\bar{y} = \frac{1}{N} \mathbf{1}^\top \mathbf{y}$ 为所有标签的平均值, $\bar{\mathbf{x}} = \frac{1}{N} (\mathbf{1}^\top \mathbf{X})^\top$ 为所有特征向量的平均值. 把 b^* 代回损失函数, 再对 $\mathbf{w}$ 求偏导, 可得

$$\frac{\partial \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})}{\partial \mathbf{w}} = \frac{1}{N} (\mathbf{X} - \bar{\mathbf{x}}^\top)^\top ((\mathbf{X} - \bar{\mathbf{x}}^\top) \mathbf{w} - (\mathbf{y} - \bar{y})), \quad (2.11)$$

其中 $(\mathbf{X} - \bar{\mathbf{x}}^\top)$ 借助广播机制, 对 $\mathbf{X}$ 的每一行都减去同一个均值向量 $\bar{\mathbf{x}}^\top$.

再次令上式为零. 若中心化后的特征矩阵列满秩, 相应矩阵可逆, 最优参数为

$$\mathbf{w}^* = \left((\mathbf{X} - \bar{\mathbf{x}}^\top)^\top (\mathbf{X} - \bar{\mathbf{x}}^\top) \right)^{-1} (\mathbf{X} - \bar{\mathbf{x}}^\top)^\top (\mathbf{y} - \bar{y}), \quad (2.12)$$

$$b^* = \bar{y} - \bar{\mathbf{x}}^\top \mathbf{w}^*. \quad (2.13)$$

若存在重复特征等导致的秩亏, 逆矩阵不存在, 最优权重也可能不唯一, 但最小二乘问题仍可求解. 实际代码用 `torch.linalg.lstsq` 直接处理设计矩阵, 避免显式构造逆矩阵.

若改用结构风险最小化, 在公式(2.8)上加 $\lambda \|\mathbf{w}\|^2 / (2N)$, 且不惩罚偏置, 则最优解变为

$$\mathbf{w}^* = \left((\mathbf{X} - \bar{\mathbf{x}}^\top)^\top (\mathbf{X} - \bar{\mathbf{x}}^\top) + \lambda \mathbf{I} \right)^{-1} (\mathbf{X} - \bar{\mathbf{x}}^\top)^\top (\mathbf{y} - \bar{y}), \quad (2.14)$$

其中 $\lambda > 0$ 为预先设置的正则化系数, $\mathbf{I} \in \mathbb{R}^{D \times D}$ 为单位矩阵. 若把惩罚项写成 $\lambda \|\mathbf{w}\|^2 / 2$, 括号内就应为 $N\lambda \mathbf{I}$; 比较不同实现的正则化强度时, 要先核对这一约定.

动手练习 2.2

验证公式(2.14).



最小二乘法 (Least Squares Method, LSM) 是以残差平方和最小为目标的拟合方法; 本节通过线性代数求解, 也可以用迭代算法求解同一个目标. 下面用 `optimizer_lsm` 函数实现该过程, 并把可选的 ℓ_2 正则化系数一并暴露出来:

```

1  def optimizer_lsm(model, X, y, reg_lambda=0.0):
2      """最小二乘法（可带 L2 正则化）求解 Linear 算子的参数.
3      输入:
4          - model: 模型
5          - X: tensor, 特征数据, shape=[N, D]
6          - y: tensor, 标签数据, shape=[N, 1]
7          - reg_lambda: float, 正则化系数, 默认为 0
8      输出:
9          - model: 优化好的模型
10     """
11     N, D = X.shape
12     # 对输入特征数据所有特征向量求平均值（行向量）
13     x_bar = X.mean(dim=0, keepdim=True)      # [1, D]
14     # 求标签的平均值, 标量
15     y_bar = y.mean()
16     # 通过广播实现矩阵减向量
17     x_sub = X - x_bar
18     if reg_lambda < 0:
19         raise ValueError("reg_lambda 必须大于或等于 0")
20
21     y_sub = y - y_bar
22     if reg_lambda == 0:
23         # 本章使用 CPU; SVD 求解可处理秩亏矩阵
24         w = torch.linalg.lstsq(x_sub, y_sub, driver="gelsd").solution
25     else:
26         #  $(X'X + \lambda I)^{-1} X' (y - y_{\text{bar}})$ 
27         A = x_sub.T @ x_sub + reg_lambda * torch.eye(
28             D, dtype=X.dtype, device=X.device)
29         rhs = x_sub.T @ y_sub
30         w = torch.linalg.solve(A, rhs)
31     b = y_bar - x_bar @ w                      # [1, 1]
32
33     model.params['w'] = w                      # [D, 1]
34     model.params['b'] = b.squeeze(0)          # [1]
35     return model

```

本章的小规模最小二乘实验在 CPU 上运行. 上面显式选择的 `gelsd` 是基于奇异值分解的求解方法; CUDA 版 `lstsq` 所支持的 `gels` 要求输入列满秩, 不能不加检查地用于后面的常数模型. 高阶多项式还会产生病态矩阵, 应结合数值精度和有效秩判断结果.

2.2.5 模型训练

数据、模型、损失函数和参数求解器都准备就绪后,就可以开始训练了. 本节用同一个带 1/2 系数的平方损失比较训练集和测试集.

下面调用上文的线性回归算子拟合训练数据,并打印训练集上的损失:

```
1 input_size = 1
2 model = Linear(input_size)
3
4 model = optimizer_lsm(model, X_train.reshape(-1, 1), y_train.reshape(-1, 1))
5 print("w_pred:", model.params['w'].item(), " b_pred:", model.params['b'].item())
6
7 y_train_pred = model(X_train.reshape(-1, 1))
8 train_error = mean_squared_error(y_true=y_train, y_pred=y_train_pred).item()
9 print("train error:", train_error)
```

输出结果为:

```
w_pred: 1.1314142942428589 b_pred: 0.47851675748825073
train error: 1.5823547840118408
```

从输出可见,估计出的参数与真实值 $w=1.2$ 、 $b=0.5$ 略有差距,差距主要来自训练数据中的噪声和样本数量限制.

2.2.6 模型评价

接着,用同一模型在测试集上预测并计算损失:

```
1 y_test_pred = model(X_test.reshape(-1, 1))
2 test_error = mean_squared_error(y_true=y_test, y_pred=y_test_pred).item()
3 print("test error:", test_error)
```

输出结果为:

```
test error: 1.7429876327514648
```

动手练习 2.3

为加深对机器学习流程的理解,请尝试:

- 1) 把训练样本量从 100 增加到 5000,观察训练误差与测试误差的变化趋势.
- 2) 调整正则化系数 λ (如 0、0.01、0.1、1),观察模型在训练/测试集上的表现差异.



动手练习 2.4

本节直接求解最小二乘问题,得到了线性回归的最优参数。请尝试改用梯度下降法(如`torch.optim.SGD`或自己手写参数更新)在同一份ToyLinear150上训练同一个线性模型,并比较:

- 1) 不同学习率下损失曲线的收敛与振荡情况;
- 2) 迭代足够多步后得到的 $\mathbf{w}$ 、 b 是否收敛到与最小二乘法相同的解。

2.3 多项式回归

多项式回归本质上仍然是一种回归任务,只是将假设空间扩展为 M 次多项式族:

$$f(\mathbf{x}; \mathbf{w}, b) = w_1x + w_2x^2 + \dots + w_Mx^M + b \quad (2.15)$$

$$= \mathbf{w}^T \phi(x) + b, \quad (2.16)$$

其中 $\mathbf{w} = [w_1, \dots, w_M]^T$ 为多项式系数, $\phi(x)$ 为多项式基函数(Polynomial Basis Function),把标量 x 映射为 M 维特征向量:

$$\phi(x) = [x, x^2, \dots, x^M]^T, \quad (2.17)$$

其中 M 是多项式阶数;当 $M = 0$ 时模型退化为常数 $f(\mathbf{x}; \mathbf{w}, b) = b$ 。

笔记

上式假定输入 $\mathbf{x}$ 只有一维。若 $\mathbf{x}$ 是多维特征,还需考虑特征之间的交互项。例如二维输入、二阶多项式时,模型扩展为

$$f(\mathbf{x}; \mathbf{w}, b) = w_1x_1 + w_2x_2 + w_3x_1^2 + w_4x_1x_2 + w_5x_2^2 + b.$$

2.3.1 数据集构建: ToySin25

为演示非线性拟合,准备一个目标函数是缩放 sin 曲线的小数据集:

```
1 import math
2 # sin 函数: sin(2 pi x)
3 def sin(x):
4     return torch.sin(2 * math.pi * x)
```

复用第2.2.1.1节的`create_toy_data()`:取自变量区间(0,1)、高斯噪声标准差0.1,生成15个训练样本和10个测试样本,记为ToySin25。为减少高次幂计算和求解过程中的舍入误差,将样本转成双精度:

```

1 interval = (0, 1)
2 torch.manual_seed(0)
3 # 生成 15 个训练样本、10 个测试样本，记为 ToySin25
4 X_train, y_train = create_toy_data(func=sin, interval=interval, sample_num=15, noise=0.1)
5 X_test, y_test = create_toy_data(func=sin, interval=interval, sample_num=10, noise=0.1)
6 X_train, y_train = X_train.double(), y_train.double()
7 X_test, y_test = X_test.double(), y_test.double()
8 # 真实曲线（无噪声），供后面拟合预测与对比绘图使用
9 X_underlying = torch.linspace(interval[0], interval[1], steps=200, dtype=torch.float64).
   unsqueeze(1)
10 y_underlying = sin(X_underlying)

```

绘制散点图（绘图代码见配套 notebook），结果见图 2.4：黑色曲线是周期为 1 的 sin 函数，红色圆圈是带噪声的训练样本。

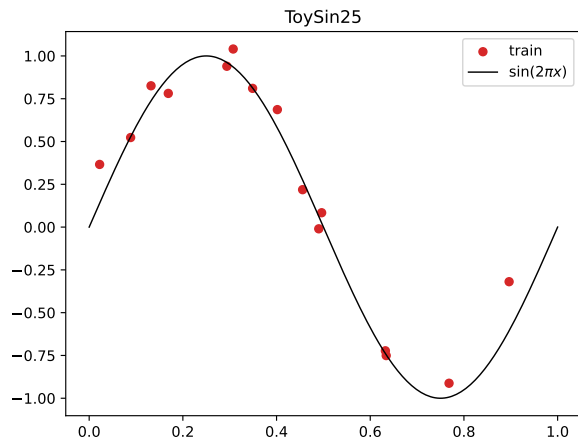


图 2.4 多项式回归的目标函数和采样数据可视化

2.3.2 模型构建

从公式(2.16)可以看出，多项式回归只是把原始特征 x 经过基函数 $\phi(x)$ 升维之后再做线性回归。只要 $\phi(x)$ 产生的特征矩阵就位，就能套用线性回归的解析解求解 w 和 b 。

笔记



当输入与输出之间存在非线性关系时，可以引入**非线性基函数（Non-linear Basis Function）**对原始特征做映射，让线性回归算法获得拟合非线性曲线的能力。

实现上分两步：先补充函数 `polynomial_basis_function` 负责特征变换，再直接复用

第2.2节的`Linear`算子和`optimizer_lsm()`:

```

1  # 多项式变换
2  def polynomial_basis_function(x, degree=2):
3      """
4      输入:
5          - x: tensor, 输入的数据, shape=[N, 1]
6          - degree: int, 多项式的阶数
7          example Input: [[2], [3], [4]], degree=2
8          example Output: [[2^1, 2^2], [3^1, 3^2], [4^1, 4^2]]
9          注意: degree 为 0 时生成形状与输入相同的全 1 张量
10     输出:
11         - x_result: tensor
12     """
13     # degree 为 0 时生成形状与输入相同的全 1 张量
14     if degree == 0:
15         return torch.ones_like(x)
16
17     return torch.cat([x ** k for k in range(1, degree + 1)], dim=-1)
18 # 简单测试
19 data = [[2.0], [3.0], [4.0]]
20 X = torch.tensor(data, dtype=torch.float32)
21 degree = 3
22 transformed_X = polynomial_basis_function(X, degree=degree)
23 print("变换前: ", X)
24 print("阶数为", degree, "变换后: ", transformed_X)

```

输出结果为:

```

变换前:  tensor([[2.],
                [3.],
                [4.]])
阶数为 3 变换后: tensor([[ 2.,  4.,  8.],
                        [ 3.,  9., 27.],
                        [ 4., 16., 64.]])

```

2.3.3 模型训练

多项式回归同样复用第2.2节的`Linear`算子、训练函数和`mean_squared_error`损失. 训练目标依旧是最小化损失,且仍可走最小二乘法的解析解路线.

下面分别取多项式阶数 $M = 0, 1, 3, 8$, 在 ToySin25 训练集上拟合, 并对比四条预测曲线与 `sin` 真值的差距:

```

1  # 取多项式阶数 M=0,1,3,8 分别拟合, 并在 X_underlying 上预测, 用于与 sin 真值对比

```

```

2 for degree in [0, 1, 3, 8]:
3     model = Linear(degree if degree > 0 else 1)
4     X_train_transformed = polynomial_basis_function(X_train.reshape(-1, 1), degree)
5     X_underlying_transformed = polynomial_basis_function(X_underlying.reshape(-1, 1),
6                                                         degree)
7     optimizer_lsm(model, X_train_transformed, y_train.reshape(-1, 1)) # 最小二乘拟合
8     y_underlying_pred = model(X_underlying_transformed).squeeze() # 预测曲线
9     # 四条拟合曲线的绘图代码见配套notebook

```

不同阶多项式拟合数据的结果如图2.5所示（绘图代码见配套 notebook）。

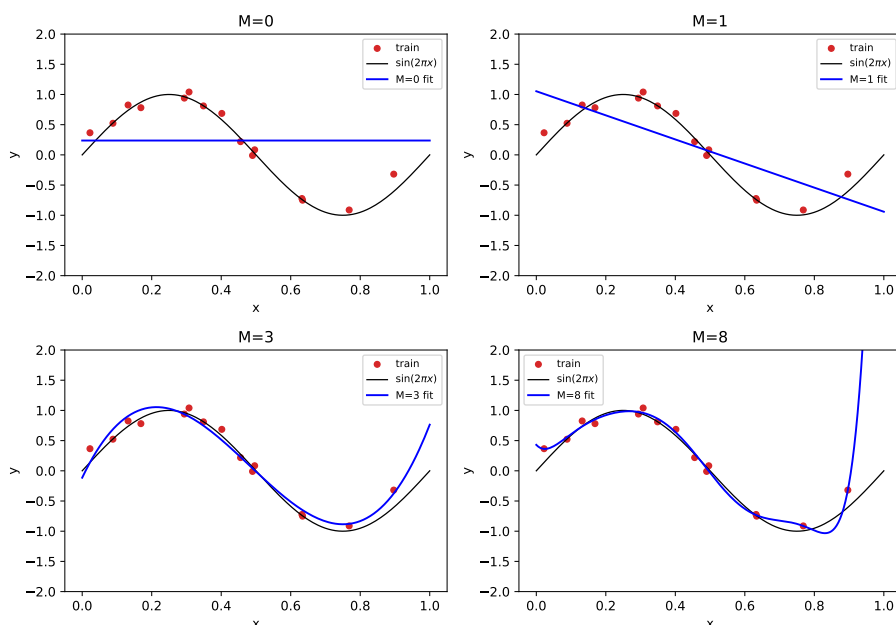


图 2.5 多项式回归训练结果

直线遇到了限制： $M = 0$ 、 $M = 1$ 时，模型无法跟随正弦曲线的弯曲，表现为欠拟合。增加到 $M = 3$ 后，模型已经能描出主要走势。 $M = 8$ 的曲线能更贴近训练点，却在样本稀疏处出现较大波动。高阶模型拥有更大的调整余地，但这份自由也可能用于追随噪声；是否值得增加阶数，还要看未参与拟合的数据。

2.3.4 模型评价

下面用带 $1/2$ 系数的平方损失量化两类指标：训练误差（train error）和测试误差（test error）。多项式阶数从 0 扫到 8，便于观察拟合能力随阶数的演变：

```

1 # 训练误差和测试误差

```

```

2 training_errors = []
3 test_errors = []
4 # 遍历多项式阶数
5 for i in range(9):
6     model = Linear(i if i > 0 else 1)
7     X_train_transformed = polynomial_basis_function(X_train.reshape(-1, 1), i)
8     X_test_transformed = polynomial_basis_function(X_test.reshape(-1, 1), i)
9     optimizer_lsm(model, X_train_transformed, y_train.reshape(-1, 1))
10    y_train_pred = model(X_train_transformed)
11    y_test_pred = model(X_test_transformed)
12    # 计算训练集和测试集上的均方误差
13    training_errors.append(mean_squared_error(y_true=y_train, y_pred=y_train_pred).item())
14    test_errors.append(mean_squared_error(y_true=y_test, y_pred=y_test_pred).item())
15
16 print("train errors:\n", training_errors)
17 print("test errors:\n", test_errors)
18 # 误差随阶数变化的绘图代码见配套notebook

```

结果如图2.6所示. 阶数增加后, 训练误差下降, 而测试误差并不随之单调下降. 这里模型族逐级包含前一个模型族, 因此在精确求解同一无正则化目标时, 最小训练误差不会因阶数增加而上升; 若实验明显违背这一点, 应先排查求解精度和有效秩, 再解释泛化现象.

这组测试曲线用于演示预先约定的对照实验, 不能同时作为挑选阶数和报告最终成绩的依据. 若要据此选出一个模型, 应另留验证集选阶数, 再用独立测试集评价; 10个测试样本也不足以说明某个阶数普遍更好.

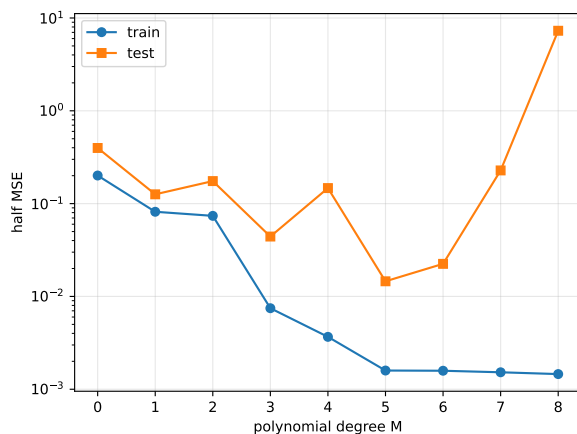


图 2.6 多项式阶数对模型性能的影响



思考

把高阶多项式实验改为单精度，并比较直接求解正规方程与`lstsq`的结果。用`torch.linalg.cond`查看设计矩阵的条件数，观察训练误差是否仍随阶数下降。数值不稳定与统计过拟合会怎样同时影响曲线？

2.3.5 通过引入正则化项来缓解过拟合

缓解过拟合的常用做法是给损失函数加上**惩罚项**，约束模型参数不要走得太极端。下面在损失中加入 ℓ_2 正则化，对比正则化前后的拟合曲线：

```
1 degree = 8                                # 多项式阶数
2 reg_lambda = 0.0001                        # 正则化系数
3 X_train_transformed = polynomial_basis_function(X_train.reshape(-1, 1), degree)
4 X_test_transformed = polynomial_basis_function(X_test.reshape(-1, 1), degree)
5 X_underlying_transformed = polynomial_basis_function(X_underlying.reshape(-1, 1), degree)
6 # 训练没有正则化项的模型
7 model = Linear(degree)
8 optimizer_lsm(model, X_train_transformed, y_train.reshape(-1, 1))
9 # 预测
10 y_test_pred = model(X_test_transformed)
11 y_underlying_pred = model(X_underlying_transformed).squeeze()
12 # 训练有正则化项的模型
13 model_reg = Linear(degree)
14 optimizer_lsm(model_reg, X_train_transformed, y_train.reshape(-1, 1), reg_lambda=
    reg_lambda)
15 # 预测
16 y_test_pred_reg = model_reg(X_test_transformed)
17 y_underlying_pred_reg = model_reg(X_underlying_transformed).squeeze()
18 # 计算测试集上的损失
19 mse = mean_squared_error(y_true=y_test, y_pred=y_test_pred).item()
20 print("mse:", mse)
21 mse_reg = mean_squared_error(y_true=y_test, y_pred=y_test_pred_reg).item()
22 print("mse_with_l2_reg:", mse_reg)
23 # 两条拟合曲线（正则 vs 不正则）的绘图代码见配套notebook
```

图2.7展示了本次采样下的结果：正则化约束了高阶系数，减少了曲线在样本稀疏处的摆动。但正则化系数过大也会丢失真实的弯曲，带来欠拟合；实际任务应在验证集上选择系数。真正正弦曲线只在这个合成实验中已知，现实任务通常无法直接与它比较。

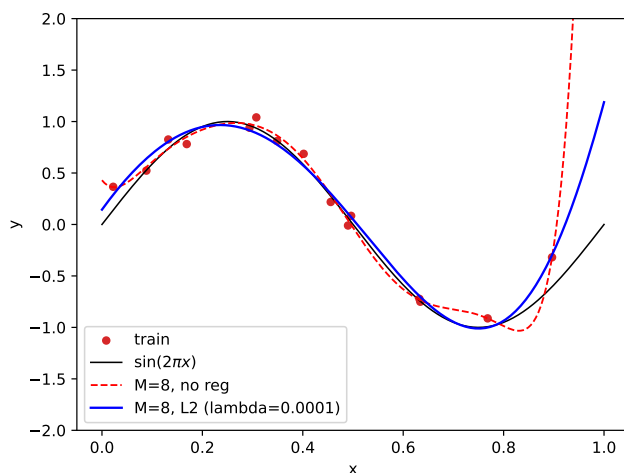


图 2.7 多项式回归的正则化效果示例

动手练习 2.5

不断改变训练样本数量，观察在不同模型复杂度下抑制过拟合所需的样本规模，验证“样本量与模型容量需要匹配”这一直觉。

思考

面对实际任务时有两种思路：(1) 使用复杂模型并配大正则化系数；(2) 直接使用简单模型。请结合数据规模、解释性和工程成本，讨论二者各自的适用场景。

2.4 构建 Runner 类

回看前几节的实验，机器学习从头到尾涉及一长串环节：数据集构建、模型构建、损失函数与优化器、评价指标、训练、评价、预测、模型保存/加载。其中若干步骤会在后续章节重复出现，适合提取成可复用代码；序列生成、图任务和智能体的流程仍需按任务调整。

为避免每章重写一遍这套样板代码，本书把它们打包进 `Runner` 类。`Runner` 对外的主要成员函数如下：

- `__init__`: 构造时接收模型、损失函数、优化器、评价指标等组件。
- `train`: 执行训练流程，接受训练集与可选的验证集。
- `evaluate`: 在验证集或测试集上计算评价指标。

- `predict`: 对单条样本（或一批样本）做前向预测。
- `save_model/load_model`: 负责模型参数的持久化与回读。

Runner类的骨架定义如下：

```
1 class Runner:
2     def __init__(self, model, optimizer, loss_fn, metric):
3         self.model = model          # 模型
4         self.optimizer = optimizer  # 优化器
5         self.loss_fn = loss_fn      # 损失函数
6         self.metric = metric        # 评价指标
7     # 模型训练
8     def train(self, train_dataset, dev_dataset=None, **kwargs):
9         pass
10    # 模型评价
11    def evaluate(self, data_set, **kwargs):
12        pass
13    # 模型预测
14    def predict(self, x, **kwargs):
15        pass
16    # 模型保存
17    def save_model(self, save_path):
18        pass
19    # 模型加载
20    def load_model(self, model_path):
21        pass
```

笔记



上面给出的是**Runner**在概念层的职责划分，具体实现会按任务做精简或调整：本章稍后实现的**RunnerV1**走最小二乘法解析解，没有训练循环，接口收敛为`fit/save/load`；线性模型章的**RunnerV2**加入小批量训练，依旧保留`train/save_model/load_model`，凸显训练循环与最优模型保存；前馈神经网络章的**RunnerV3**回到`fit/save/load`，与工程仓库接口完全对齐。

Runner类的使用流程见图2.8，可分为4个阶段：

- 1) **模型准备**: 初始化阶段，备齐模型、损失函数、优化器和评价指标，并准备好数据集传入**Runner**。
- 2) **模型训练**: 调用`train()`训练模型，用`evaluate()`跟踪验证集指标，并用`save_model()`保存最优模型。
- 3) **模型评价**: 调用`evaluate()`在测试集上得到最终性能。

4) **模型预测**:调用`predict()`对新样本预测. 除前向计算外,往往还要做两件事:一是把原始特征转换为模型所需的输入张量,二是把模型输出(如类别概率)还原为业务可读的标签.

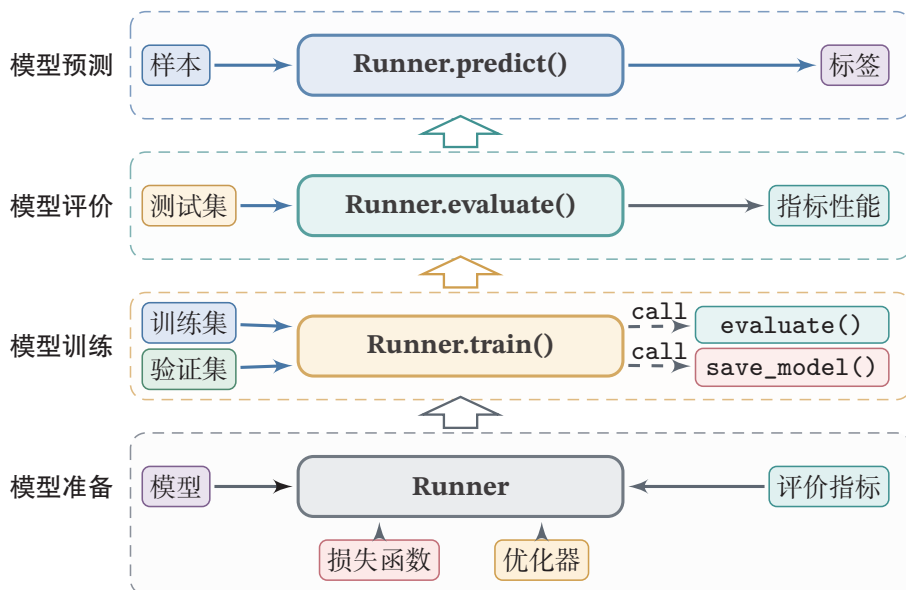


图 2.8 Runner类的使用流程

2.5 实践:基于线性回归的加州房价预测

合成数据里,真实规律由我们预先指定;现在换成一份不知道生成规律的房价数据. 模型能利用哪些特征?误差又该怎样解释?下面用加州房价预测检验前面的流程,依次完成:

- 1) **数据处理**:加载数据、划分训练/测试集并对特征做标准化,确保数值范围合理.
- 2) **模型构建**:实例化线性回归模型.
- 3) **完善Runner类**:实现RunnerV1以统一管理训练、评价和预测.
- 4) **模型训练**:通过Runner完成训练.
- 5) **模型评价**:通过Runner在测试集上汇报指标.
- 6) **模型预测**:通过Runner对新样本做预测.

2.5.1 数据处理

2.5.1.1 数据集介绍

加州房价预测数据集 (California Housing Prices) 共有 20 640 条记录,数据来自 1990 年美国人口普查,每条记录给出普查街区组层面的 8 个特征(如中位收入、房龄、平均房间数等)以及该街区房价中位数作为标签. 各字段含义见表 2.1.

表 2.1 加州房价字段含义

字段名	类型	含义
MedInc	float	街区中位家庭收入(单位 \$10 000)
HouseAge	float	街区房屋的中位年龄
AveRooms	float	每户的平均房间数
AveBedrms	float	每户的平均卧室数
Population	float	街区总人口
AveOccup	float	户均人口数
Latitude	float	街区纬度
Longitude	float	街区经度
MedHouseVal	float	该街区房价中位数(单位 \$100 000)

笔记



本案例早期使用的波士顿房价数据集因涉及种族特征等伦理问题,已于 scikit-learn 1.2 版本中移除。本版改用加州房价数据集演示回归流程,首次加载时需要下载并缓存数据。它反映历史街区统计,目标也有截顶处理,适合作为教学基准;这里的模型不能直接用于今天的单套住宅估价。



通过 scikit-learn 加载数据集并预览前 5 条样本:

```

1 from sklearn.datasets import fetch_california_housing
2 import pandas as pd                                # 开源数据分析工具
3
4 ds = fetch_california_housing(as_frame=True)
5 data = ds.frame                                    # 8 特征 + 1 目标列
6 # 预览前 5 行数据
7 data.head()
```

输出结果为:

	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	Latitude	Longitude	MedHouseVal
0	8.3252	41.0	6.984127	1.023810	322.0	2.555556	37.88	-122.23	4.526
1	8.3014	21.0	6.238137	0.971880	2401.0	2.109842	37.86	-122.22	3.585
2	7.2574	52.0	8.288136	1.073446	496.0	2.802260	37.85	-122.24	3.521
3	5.6431	52.0	5.817352	1.073059	558.0	2.547945	37.85	-122.25	3.413
4	3.8462	52.0	6.281853	1.081081	565.0	2.181467	37.85	-122.25	3.422

该数据集无缺失值 (可用 `data.isna().sum()` 验证), 可直接进入下一步。

2.5.1.2 数据集划分与特征标准化

本例预先固定为无正则化的线性回归, 不进行超参数选择, 因此按 8:2 划分训练集和测试集。若后续要选择特征、模型或正则化系数, 须再划出验证集。原始特征的量纲差异很大, 收入、人口和经纬度的数值范围不同, 先做**特征缩放** (Feature Scaling) 有助于改善数值条件和比较系数。对于带偏置、无正则化且精确求解的线性回归, 可逆缩放不改变能表示的函数集合, 因此不会凭空提高模型的拟合能力。

笔记



常见的特征缩放方式有两种: **最小—最大归一化** (Min-Max Normalization) 把训练特征的取值范围映射到 $[0, 1]$; **标准化** (Standardization) 让每维特征均值为 0、标准差为 1。本案例采用标准化。

提醒



均值和标准差只能在训练集上估计, 并用同一组统计量变换测试集, 否则测试数据会参与模型准备, 评价不再独立。实际部署的新样本也必须使用这组训练集统计量, 不能重新按每批输入计算。

划分与标准化的代码如下:

```
1 import torch
2
3 torch.manual_seed(0)
4
5 X = torch.tensor(data.drop(columns=['MedHouseVal']).values, dtype=torch.float32)
6 y = torch.tensor(data['MedHouseVal'].values, dtype=torch.float32).unsqueeze(1)
7 # 8:2 划分训练 / 测试集
8 N = len(X)
9 perm = torch.randperm(N)
10 split = int(N * 0.8)
11 tr, te = perm[:split], perm[split:]
12 X_train, y_train = X[tr], y[tr]
13 X_test, y_test = X[te], y[te]
14 # mean/std 只在训练集上拟合
15 mean, std = X_train.mean(dim=0), X_train.std(dim=0)
16 std = torch.where(std == 0, torch.ones_like(std), std) # 常量特征避免除零
17 X_train = (X_train - mean) / std
18 X_test = (X_test - mean) / std
19
20 train_dataset = (X_train, y_train)
21 test_dataset = (X_test, y_test)
```

2.5.2 模型构建

直接复用第2.2.2.1节的`Linear`算子,输入维度设置为8,与数据集的特征数一致:

```
1 # 模型实例化
2 input_size = 8
3 model = Linear(input_size)
```

2.5.3 完善 Runner 类:RunnerV1

数据和模型就位后,还需要把损失函数、优化器、评价指标,以及模型存储路径等信息组装进`Runner`,并确认每个组件在本任务中是否确实需要。

第一个版本`RunnerV1`只承担求解器调用、评估、预测和参数读写——由于损失函数已隐含在最小二乘法的解析解中,无需在`RunnerV1`里另外维护。实现如下:

```
1 from pathlib import Path
2
3 class RunnerV1:
4     def __init__(self, model, optimizer):
5         self.model = model
6         self.optimizer = optimizer
7
8     def fit(self, X, y, **kwargs):
9         self.optimizer(self.model, X, y, **kwargs)
10
11     def predict(self, X):
12         return self.model(X)
13
14     def evaluate(self, X, y, metric_fn):
15         return metric_fn(y, self.model(X)).item()
16
17     def save(self, path):
18         Path(path).parent.mkdir(parents=True, exist_ok=True)
19         torch.save(self.model.params, path)
20
21     def load(self, path):
22         self.model.params = torch.load(path, weights_only=True)
```

`Runner`组装好后,便可以按训练—评价—预测的顺序逐步推进。

2.5.4 模型训练

实例化RunnerV1, 通过fit调用第2.2.4节实现的最小二乘求解器optimizer_lsm(). 这里打印带1/2系数的平方损失, 与前面的函数保持一致:

```
1 def mse(y_true, y_pred):
2     return 0.5 * ((y_true - y_pred) ** 2).mean() # 与 mean_squared_error 同口径: 1/(2N)
3
4 runner = RunnerV1(model, optimizer_lsm)
5 # 模型训练
6 runner.fit(X_train, y_train, reg_lambda=0)
7
8 print(f'train half MSE: {runner.evaluate(X_train, y_train, mse):.4f}')
9 print(f'test half MSE: {runner.evaluate(X_test, y_test, mse):.4f}')
```

输出结果为:

```
train half MSE: 0.2631
test half MSE: 0.2584
```

打印训练得到的权重, 并按数值从小到大排序, 便于解读每个特征的影响方向:

```
1 columns_list = data.drop(columns=['MedHouseVal']).columns.to_list()
2 weights = runner.model.params['w'].squeeze().tolist()
3 b = runner.model.params['b'].item()
4
5 for name, weight in sorted(zip(columns_list, weights), key=lambda kv: kv[1]):
6     print(f'{name:12s} weight: {weight:+.4f}')
7
8 print(f'bias      {b:+.4f}')
```

输出结果为:

```
Latitude  weight: -0.8974
Longitude weight: -0.8688
AveRooms  weight: -0.2692
AveOccup  weight: -0.0406
Population weight: -0.0050
HouseAge  weight: +0.1162
AveBedrms weight: +0.3079
MedInc    weight: +0.8284
bias      +2.0765
```

在其他输入特征保持不变时, MedInc增加一个训练集标准差, 模型预测值约增加0.83个房价单位. 纬度和经度的负系数也只表示模型中的条件关联; 它们与沿海位置、收入等因素相关, 不能

据此概括某个方向上的真实房价走势,更不能把系数解释为因果作用. 相关特征一起进入模型时,系数的正负也可能与单独观察该特征时不同.

2.5.5 模型评价

用`RunnerV1.save`把训练好的参数保存到磁盘,再新建一个同结构模型并通过`load`加载,对比测试集均方误差,验证保存/加载链路是否完整:

```
1 save_path = './checkpoint/california_linear.pt'
2 runner.save(save_path)
3
4 model2 = Linear(input_size=X_train.shape[1])
5 runner2 = RunnerV1(model2, optimizer_lsm)
6 runner2.load(save_path)
7 print(f'reloaded test half MSE: {runner2.evaluate(X_test, y_test, mse):.4f}')
```

输出结果为:

```
reloaded test half MSE: 0.2584
```

2.5.6 模型预测

模型参数加载完成后,调用`predict`即可对单条样本做预测:

```
1 pred = runner2.predict(X_test[:1])
2 print(f'真实房价: {y_test[:1].item():.2f}')
3 print(f'预测房价: {pred.item():.2f}')
```

输出结果为:

```
真实房价: 1.92
```

```
预测房价: 1.95
```

这条样本的预测值与真实标签较为接近,但一个样本不能代表整体效果. 测试集上带 1/2 系数的损失约为 0.258,换算为通常的 MSE 约为 0.517, RMSE 约为 0.719 个房价单位,即约 7.19 万美元. 这是历史街区房价中位数预测的误差量级. 随机划分衡量的是同一数据来源中的预测表现;若要考察对新地区的迁移,还应按地理区域划分.

上面的参数文件只保存了 $\mathbf{w}$ 和 $\mathbf{b}$. 在新程序中处理原始输入时,还需要保存特征名称及顺序、训练集的 `mean/std` 和目标单位,否则即使成功加载权重,也可能得到错误预测. 可以把这些信息与参数一起保存:

```
1 torch.save({
2     "params": runner.model.params,
```



```
3     "feature_names": columns_list,  
4     "mean": mean, "std": std, "target_unit_usd": 100000,  
5 }, "./checkpoint/california_bundle.pt")
```



动手练习 2.6

若跳过第2.5.1.2节中的特征标准化,会对线性回归模型的学习过程和最终精度产生哪些影响?



思考

当训练数据中混入异常样本时,最小二乘法得到的模型会被怎样影响?常见的应对手段(如健壮回归、异常点过滤)各有什么权衡?

2.6 小结

从一条带噪声的直线,到可以任意弯曲的多项式,再到真实的房价数据,本章反复遇到同一个问题:把训练样本拟合好之后,还剩下什么需要检验?独立评价、数值稳定性、指标口径和预处理的一致性,都会改变我们对实验的判断。

RunnerV1把最小二乘求解、评价和参数读写封装起来。下一章转向分类任务,参数通常不能再靠求解一组线性方程得到;我们将实现梯度下降,并观察参数如何在一次次更新中改变分类边界。

第3章 线性模型

上一章预测房价时,输出是一个连续数值.如果现在要判断一朵鸢尾花属于哪一类,输出就变成了离散的**类别**.这类任务称为**分类**.本章先尝试用一条直线分开两群弯月形样本,观察参数更新怎样移动分界线;再让多个类别同时竞争,完成鸢尾花分类.

线性分类利用一个或多个线性函数配合决策规则把样本分到不同类别.这里的“线性”并不是说输入特征到预测标签的整体映射是线性的,而是指不同类别之间的分界面是样本空间中的超平面.常见线性分类模型包括感知器、线性支持向量机、Logistic 回归和 Softmax 回归,它们之间的差异主要体现在模型形式、损失函数和优化算法上.其中 Logistic 回归是处理二分类问题的常用线性模型,Softmax 回归是它在多分类上的自然推广.

本章对应《神经网络与深度学习》第3章**线性模型**.动手前可先扫一眼图3.1的关键知识点,便于在实践代码与原书概念之间建立映射.

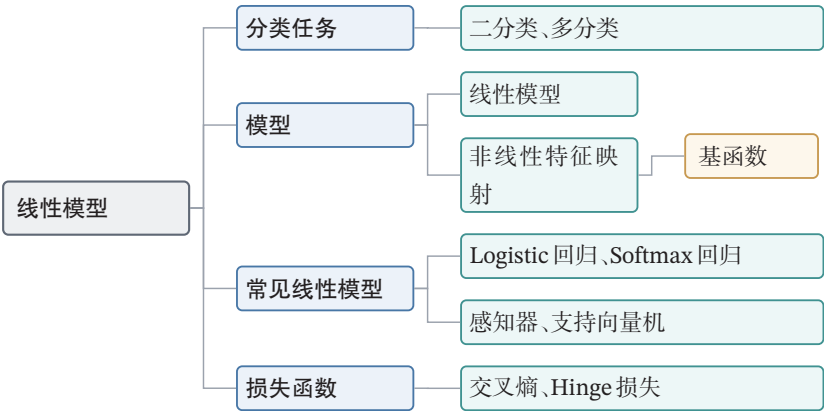


图 3.1 线性模型关键知识点回顾

本章分为两部分:

- **模型解读**: 聚焦 Logistic 回归和 Softmax 回归两种常用的线性分类模型. 先构造简单的二分类与三分类数据集,再用梯度下降法逐步学习参数,把公式推导与代码实现相互印证.

- **案例实践**: 以 Softmax 回归完成鸢尾花分类, 在小样本条件下检查模型选择、数据预处理和评价的可靠性。

3.1 基于 Logistic 回归的二分类任务

Logistic 回归是一种常用的二分类线性模型。它和线性回归一样, 把特征经权重线性叠加; 不同之处在于额外套了一个非线性的“挤压”函数 $\sigma: \mathbb{R} \rightarrow (0, 1)$, 把线性输出转换为后验概率 $p(y = 1|\mathbf{x})$, 从而把连续的线性值映射到合理的概率区间。

$$p(y = 1|\mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x} + b), \quad (3.1)$$

这里的 $\sigma(\cdot)$ 即 Logistic 函数, 亦称激活函数, 负责把线性结果 $f(\mathbf{x}; \mathbf{w}, b)$ 从实数轴“挤”到 $(0, 1)$ 区间, 正好对应概率取值。其表达式为

$$\sigma(x) = \frac{1}{1 + \exp(-x)}. \quad (3.2)$$

Logistic 函数 Logistic 函数在 0 附近近似线性, 两端则快速饱和——输入趋向负无穷时输出趋近 0, 趋向正无穷时输出趋近 1。数学上其输出落在 $(0, 1)$ 区间, 模型将它解释为类别 1 的条件概率估计; 落在概率区间并不自动保证估计准确。浮点计算中, 大幅度输入还可能使结果舍入为 0 或 1。

笔记



直接套用公式实现 Logistic 函数时, 当 x 取很大的负数, `torch.exp(-x)` 会数值溢出。实际代码可调用 `torch.sigmoid(x)`, 避免自行处理指数计算。但得到有限概率并不意味着后续取对数也安全: 饱和后的 0 或 1 仍会使 $\log(p)$ 或 $\log(1-p)$ 出现问题。训练时把线性分数直接交给融合损失函数更可靠, 后文会给出对照。

动手练习 3.1



在配套 notebook 中采样并用 Matplotlib 画出 Logistic 函数的曲线, 直观感受它“两端饱和、中间近似线性”的形态。

下面动手实现一个 Logistic 回归模型, 并在小型二分类数据集上完成训练与评估。

3.1.1 数据集构建

首先构造一个二分类数据集——**Moon1000**。数据由两条带噪声的弯月形函数生成, 每条弯月对应一类, 每条样本含 2 个特征, 共 1 000 条。

构造函数 `make_moons` 的实现如下:

```

1 import math
2 import torch
3
4 def make_moons(n_samples=1000, shuffle=True, noise=None, seed=None):
5     """
6     生成带噪声的弯月形状数据
7     输入:
8         - n_samples: 样本数量, 数据类型为 int
9         - shuffle: 是否打乱数据, 数据类型为 bool
10        - noise: 以多大的程度增加噪声, noise 为 None 时表示不增加噪声
11        - seed: 随机种子, 非 None 时使打乱与噪声可复现
12    输出:
13        - X: 特征数据, shape=[n_samples, 2]
14        - y: 标签数据, shape=[n_samples]
15    """
16    g = torch.Generator().manual_seed(seed) if seed is not None else None
17    n_out = n_samples // 2
18    n_in = n_samples - n_out
19    # 采集第 0 类数据 (外圈), 特征 (x, y)
20    outer_x = torch.cos(torch.linspace(0, math.pi, n_out))
21    outer_y = torch.sin(torch.linspace(0, math.pi, n_out))
22    # 采集第 1 类数据 (内圈)
23    inner_x = 1 - torch.cos(torch.linspace(0, math.pi, n_in))
24    inner_y = 0.5 - torch.sin(torch.linspace(0, math.pi, n_in))
25
26    X = torch.stack([
27        torch.cat([outer_x, inner_x]),
28        torch.cat([outer_y, inner_y]),
29    ], dim=1)
30    y = torch.cat([torch.zeros(n_out), torch.ones(n_in)])
31
32    if shuffle:
33        idx = torch.randperm(X.shape[0], generator=g)
34        X = X[idx]
35        y = y[idx]
36    if noise is not None:
37        X = X + torch.normal(mean=0.0, std=noise, size=X.shape, generator=g)
38    return X, y

```

采样 1000 条数据 (噪声 noise=0.2):

```

1 torch.manual_seed(0)
2 n_samples = 1000
3 X, y = make_moons(n_samples=n_samples, shuffle=True, noise=0.2)

```

绘制散点图（绘图代码见配套 notebook），结果见图3.2，两类样本呈对扣的弯月形状。

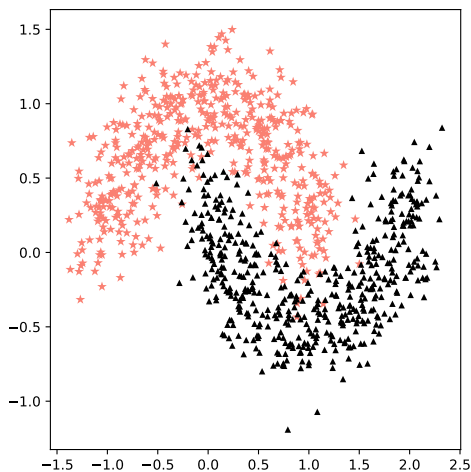


图 3.2 Moon1000 数据集的数据分布

按 640、160、200 条样本切分训练集、验证集和测试集：

```
1 num_train = 640
2 num_dev = 160
3 num_test = 200
4
5 X_train, y_train = X[:num_train], y[:num_train]
6 X_dev, y_dev = X[num_train:num_train + num_dev], y[num_train:num_train + num_dev]
7 X_test, y_test = X[num_train + num_dev:], y[num_train + num_dev:]
8
9 y_train = y_train.reshape(-1, 1)
10 y_dev = y_dev.reshape(-1, 1)
11 y_test = y_test.reshape(-1, 1)
```

至此Moon1000数据集构建完成. 打印一下张量形状方便核对：

```
1 print("X_train shape:", X_train.shape, "y_train shape:", y_train.shape)
```

输出结果为：

```
X_train shape: torch.Size([640, 2]) y_train shape: torch.Size([640, 1])
```



动手练习 3.2

打印前 5 个数据的标签, 检查数据划分后的格式是否正确。



3.1.2 模型构建

Logistic 回归模型 Logistic 回归本质上就是**线性层 + Logistic 函数**的串联. 这里把权重和偏置简单地初始化为 0 (这里只有一个输出单元, 无须打破隐藏单元之间的对称性); 为充分利用矩阵运算, 预测时一次性处理 N 个样本:

$$\hat{\mathbf{y}} = \sigma(\mathbf{X}\mathbf{w} + \mathbf{b}), \quad (3.3)$$

其中 $\mathbf{X} \in \mathbb{R}^{N \times D}$ 是按行堆叠的样本矩阵, $\hat{\mathbf{y}}$ 是 N 个预测概率组成的向量. 线性分数 $z = \mathbf{w}^T \mathbf{x} + b$ 称为**对数几率** (Logit), 满足 $z = \log(\hat{y}/(1 - \hat{y}))$; 它和经 Logistic 函数得到的概率是两种不同的量.

实现代码如下:

```

1  from nndl import op
2
3  class Model_LR(op.Op):
4      def __init__(self, input_size):
5          super().__init__()
6          self.params = {
7              # 将线性层的权重参数全部初始化为 0
7              'w': torch.zeros(input_size, 1),
9              # 将线性层的偏置参数初始化为 0
9              'b': torch.zeros(1),
11         }
12
13     def forward(self, inputs):
14         """
15         输入:
16             - inputs: shape=[N, D], N 是样本数量, D 是特征维度
17         输出:
18             - outputs: 预测标签为 1 的概率, shape=[N, 1]
19         """
20         # 线性计算
21         score = inputs @ self.params['w'] + self.params['b']
22         # Logistic 函数
23         return torch.sigmoid(score)

```

思考



在最终决策阶段, 其实只需判断线性输出 score 是否大于等于 0 即可决定类别 (概率阈值取 0.5 时), 不必显式调用 Logistic 函数. 这一观察对训练阶段是否也成立? 是否存在更优的工程改进?



测试一下 随机生成 3 条长度为 4 的特征向量喂入模型, 查看输出:

<https://nndl.ai/>

```

1 # 固定随机种子，保持每次运行结果一致
2 torch.manual_seed(0)
3 # 随机生成 3 条长度为 4 的数据
4 inputs = torch.randn(3, 4)
5 print('Input is:', inputs)
6 # 实例化模型
7 model = Model_LR(4)
8 outputs = model(inputs)
9 print('Output is:', outputs)

```

输出结果为：

```

Input is: tensor([[ 1.5410, -0.2934, -2.1788, 0.5684],
                  [-1.0845, -1.3986, 0.4033, 0.8380],
                  [-0.7193, -0.4033, -0.5966, 0.1820]])
Output is: tensor([[0.5000],
                  [0.5000],
                  [0.5000]])

```

所有样本的预测概率都是 0.5。原因很简单：权重和偏置全部为 0，线性输出恒为 0，再经 Logistic 得到 $\sigma(0) = 0.5$ 。

3.1.3 损失函数

训练阶段需要一把“尺子”衡量预测值与真实值之间的差距。对于确定类别的监督样本，把真实类别写成概率为 1、其余类别概率为 0 的分布。交叉熵损失函数惩罚模型给真实类别分配过小的概率：真实类别的预测概率越小，负对数损失越大。

二分类场景里只需估计 $\hat{y} = p(y = 1|\mathbf{x})$, $p(y = 0|\mathbf{x})$ 即 $1 - \hat{y}$ 。给定训练集 $\{(\mathbf{x}^{(n)}, y^{(n)})\}_{n=1}^N$ ，Logistic 回归的经验风险写作

$$\mathcal{R}(\mathbf{w}, b) = -\frac{1}{N} \sum_{n=1}^N \left(y^{(n)} \log \hat{y}^{(n)} + (1 - y^{(n)}) \log(1 - \hat{y}^{(n)}) \right). \quad (3.4)$$

写成向量形式为

$$\mathcal{R}(\mathbf{w}, b) = -\frac{1}{N} \left(\mathbf{y}^\top \log \hat{\mathbf{y}} + (1 - \mathbf{y})^\top \log(1 - \hat{\mathbf{y}}) \right), \quad (3.5)$$

其中 $\hat{\mathbf{y}} \in [0, 1]^N$ 为预测概率向量。

二分类交叉熵损失的代码实现如下：

```

1 class BinaryCrossEntropyLoss(op.Op):
2     def __init__(self):

```

```

3         # predicts / labels 由 forward 写入, 供调用方读取使用
4         self.predicts = None
5         self.labels = None
6
7     def forward(self, predicts, labels):
8         """
9         输入:
10             - predicts: 预测值, shape=[N, 1]
11             - labels: 真实标签, shape=[N, 1]
12         输出:
13             - 损失值: 零维标量张量
14         """
15         self.predicts = predicts
16         self.labels = labels
17         N = predicts.shape[0]
18         # float32 下 1-1e-9 会舍入为 1.0、clamp 上界失效; eps 取 1e-7 才能保证 1-eps < 1
19         eps = max(1e-7, torch.finfo(predicts.dtype).eps)
20         p = predicts.clamp(eps, 1 - eps)      # 数值稳定
21         loss = -1 / N * (labels.t() @ torch.log(p) + (1 - labels).t() @ torch.log(1 - p))
22         return loss.squeeze()
23
24 # 测试一下
25 labels = torch.ones(3, 1)
26 bce_loss = BinaryCrossEntropyLoss()
27 print(bce_loss(outputs, labels))

```

输出结果为:

```
tensor(0.6931)
```

3.1.4 模型优化

Logistic 回归的交叉熵目标对参数是凸的, 但一般没有像线性最小二乘那样的闭式解, 通常采用迭代优化. 下面使用**梯度下降法**. 还需留意, 完全可分的数据在不加正则化时, 可能通过不断增加权重降低损失而没有有限的最优权重; 损失变小并不总意味着参数会收敛到一个有限值.

梯度下降的基本步骤是: 先初始化参数 $\mathbf{w}$ 、 b (本节取零), 再反复计算损失关于参数的梯度, 沿梯度反方向小步更新, 直到损失收敛.

3.1.4.1 梯度计算

对 Logistic 回归的经验风险 $\mathcal{R}(\mathbf{w}, b)$ 分别求 $\mathbf{w}$ 和 b 的偏导, 可得

$$\frac{\partial \mathcal{R}(\mathbf{w}, b)}{\partial \mathbf{w}} = -\frac{1}{N} \sum_{n=1}^N \mathbf{x}^{(n)} (y^{(n)} - \hat{y}^{(n)}) = -\frac{1}{N} \mathbf{X}^T (\mathbf{y} - \hat{\mathbf{y}}), \quad (3.6)$$

$$\frac{\partial \mathcal{R}(\mathbf{w}, b)}{\partial b} = -\frac{1}{N} \sum_{n=1}^N (y^{(n)} - \hat{y}^{(n)}) = -\frac{1}{N} \mathbf{1}^\top (\mathbf{y} - \hat{\mathbf{y}}), \quad (3.7)$$

其中 $\mathbf{X}$ 是按行堆叠的样本矩阵, $\mathbf{y}$ 是真实标签向量, $\hat{\mathbf{y}}$ 是模型预测的“标签为 1”概率向量, $\mathbf{1}$ 表示 N 维全 1 向量。

下面给 Logistic 回归算子加上 `backward` 函数: 它按上述公式计算梯度, 并写入算子的 `grads` 属性, 供优化器读取使用。

```

1  class Model_LR(op.Op):
2      def __init__(self, input_size):
3          super().__init__()
4          # 存放线性层参数
5          self.params = {
6              # 将线性层的权重参数全部初始化为 0
7              'w': torch.zeros(input_size, 1),
8              # 将线性层的偏置参数初始化为 0
9              'b': torch.zeros(1),
10         }
11         # 存放参数的梯度
12         self.grads = {}
13         self.X = None
14         self.outputs = None
15
16     def forward(self, inputs):
17         self.X = inputs
18         # 线性计算
19         score = inputs @ self.params['w'] + self.params['b']
20         # Logistic 函数
21         self.outputs = torch.sigmoid(score)
22         return self.outputs
23
24     def backward(self, labels):
25         """
26         输入:
27             - labels: 真实标签, shape=[N, 1]
28         """
29         N = labels.shape[0]
30         # 计算偏导数
31         self.grads['w'] = -1 / N * (self.X.t() @ (labels - self.outputs))
32         self.grads['b'] = -1 / N * (labels - self.outputs).sum()

```

提醒



这里`backward`并不返回`forward`本身的导数,而是合并了交叉熵损失之后的总梯度.这一公式对应未截断的理论交叉熵.上面的概率截断用于避免演示程序出现`log(0)`;一旦触发截断,其数值损失与这里的解析梯度便不再严格配对.正式训练应使用接收 logits 的融合损失,避免在极端预测时掩盖大损失.

动手练习 3.3



本节的`backward`把“交叉熵损失 + Logistic 函数”的复合梯度一次性手工推导出来,绕过了自动微分.请验证这个手写公式是否正确:取一小批样本,先调用`backward`得到`grads['w']`、`grads['b']`;再用有限差分(对某个参数分量加减一个很小的 ϵ ,用损失的差商近似偏导)或`torch.autograd`对同一批数据另算一遍梯度,与`backward`写入的结果逐元素比较,观察两者的差异量级,据此判断手写梯度公式是否成立.

与 PyTorch 融合损失对照 二分类也可以直接使用融合损失函数:输入线性分数,由函数内部完成稳定计算.下面调用`binary_cross_entropy_with_logits`,故意给出“非常自信却预测错误”的分数:

```
1 import torch.nn.functional as F
2 binary_logits = torch.tensor([[ -1000.], [1000.]], requires_grad=True)
3 binary_targets = torch.tensor([[1.], [0.]])
4 stable_loss = F.binary_cross_entropy_with_logits(binary_logits, binary_targets)
5 stable_loss.backward()
6 print(stable_loss.item(), binary_logits.grad.flatten())
```

损失为 1000, 梯度为 $[-0.5, 0.5]$, 均保持有限. 它通过稳定的对数计算保留了错误预测应有的惩罚; 先算饱和概率再截断则会低估这一损失.

3.1.4.2 优化器

得到梯度后,按梯度下降的更新公式更新参数:

$$\mathbf{w} \leftarrow \mathbf{w} - \alpha \frac{\partial \mathcal{R}(\mathbf{w}, b)}{\partial \mathbf{w}}, \quad (3.8)$$

$$b \leftarrow b - \alpha \frac{\partial \mathcal{R}(\mathbf{w}, b)}{\partial b}, \quad (3.9)$$

其中 α 为学习率.

把参数更新过程封装成优化器,方便不同算法共享接口.先定义一个基类`Optimizer`:初始化时传入学习率和待优化模型,`step`方法留给子类实现具体的更新逻辑.代码如下:

```
1 from abc import ABC, abstractmethod
2 # 优化器基类
```

```

3 class Optimizer(ABC):
4     def __init__(self, init_lr, model):
5         """
6         优化器类初始化
7         """
8         # 初始化学习率, 用于参数更新的计算
9         self.init_lr = init_lr
10        # 指定优化器需要优化的模型
11        self.model = model
12
13    @abstractmethod
14    def step(self):
15        """
16        定义每次迭代如何更新参数
17        """
18        pass

```

3.1.4.3 梯度下降优化器

继承`Optimizer`得到具体的梯度下降优化器`SimpleBatchGD`: 每次调用`step`时, 从模型的`grads`里取出梯度, 按上式更新对应参数. 实现如下:

```

1 class SimpleBatchGD(Optimizer):
2     def step(self):
3         # 参数更新
4         # 遍历所有参数, 按照梯度下降公式更新
5         if isinstance(self.model.params, dict):
6             for key in self.model.params.keys():
7                 self.model.params[key] = self.model.params[key] - self.init_lr * self.model.grads[key]

```

3.1.5 评价指标

分类任务最常用的评价指标是**准确率** (Accuracy) —— 预测正确的样本占总样本的比例. 设真实标签向量 $\mathbf{y} \in \{0, 1\}^N$ 、预测类别向量 $\hat{\mathbf{y}} \in \{0, 1\}^N$ (由概率阈值化或取最大值得到), 准确率的定义为

$$\mathcal{A} = \frac{1}{N} \mathbf{1}^\top I(\mathbf{y} = \hat{\mathbf{y}}), \quad (3.10)$$

其中 $I(\cdot)$ 是指示函数. 本章类别数量较均衡, 用准确率便于观察; 若 99% 的样本属于同一类, 永远预测这一类也能达到 99% 的准确率, 此时还需要查看各类召回率、精确率或混淆矩阵. 代码实现如下:

```

1 def accuracy(preds, labels):
2     """

```

```

3  输入:
4      - preds: 预测值, shape=[N, 1] 是二分类; shape=[N, C] 是多分类, C 为类别数
5      - labels: 真实标签, shape=[N, 1]
6  输出:
7      - 准确率: Python 浮点数
8  """
9  # 判断是二分类还是多分类. preds.shape[1]==1 时为二分类
10 if preds.dim() > 1 and preds.shape[1] == 1:
11     preds = (preds >= 0.5).float()
12 elif preds.dim() > 1:
13     preds = preds.argmax(dim=1)
14     labels = labels.reshape(preds.shape)
15     return (preds == labels).float().mean().item()
16 # 假设模型预测值为 [[0.], [1.], [1.], [0.]], 真实类别为 [[1.], [1.], [0.], [0.]]
17 preds = torch.tensor([[0.], [1.], [1.], [0.]])
18 labels = torch.tensor([[1.], [1.], [0.], [0.]])
19 print("accuracy is:", accuracy(preds, labels))

```

输出结果为:

```
accuracy is: 0.5
```

3.1.6 完善 Runner 类: RunnerV2

第2章的RunnerV1面向有解析解的情形,没有显式训练循环.本章改用梯度下降迭代优化,必须把训练循环加进Runner:每轮迭代算损失、反向求梯度、调用优化器更新参数;同时引入提前停止(Early Stopping)思想,在验证集上跟踪指标并保留最优模型。



笔记

提前停止是梯度下降训练中常见的正则化技巧;当验证集指标连续若干轮没有改善时终止训练.下面的RunnerV2实现了其中的模型选择环节,即保留验证集上指标最优的一份模型快照;它仍会完成指定的训练轮次,尚未实现提前终止。

RunnerV2的核心是下面这个保存最优模型快照的训练循环,并给出评估、预测和参数读写方法,便于连续运行;配套工具包位于nndl/runner.py:

```

1  import os
2
3  class RunnerV2:
4      def __init__(self, model, optimizer, metric, loss_fn):
5          self.model = model
6          self.optimizer = optimizer

```

```

7         self.loss_fn = loss_fn
8         self.metric = metric
9         # 训练过程指标统一收纳到 history 字典中, 便于扩展和绘图
10        self.history = {
11            'train_loss': [], 'dev_loss': [],
12            'train_score': [], 'dev_score': [],
13        }
14
15    def train(self, train_set, dev_set, num_epochs=100, log_epochs=100,
16              save_path='model_best.pt'):
17        # 用 -inf 初始化保证首轮一定触发对比
18        best_score = -float('inf')
19        X, y = train_set
20        for epoch in range(num_epochs):
21            # 训练一步
22            probabilities = self.model(X)
23            trn_loss = self.loss_fn(probabilities, y).item()
24            trn_score = self.metric(probabilities, y)
25            self.model.backward(y)
26            self.optimizer.step()
27            self.history['train_loss'].append(trn_loss)
28            self.history['train_score'].append(trn_score)
29            # 验证集评估并记录历史
30            dev_score, dev_loss = self.evaluate(dev_set)
31            self.history['dev_loss'].append(dev_loss)
32            self.history['dev_score'].append(dev_score)
33            # 保存最优模型
34            if dev_score > best_score:
35                self.save_model(save_path)
36                print(f'best score updated: {best_score:.5f} -> {dev_score:.5f}')
37                best_score = dev_score
38            if (epoch + 1) % log_epochs == 0:
39                print(f'[Train] epoch {epoch+1} loss {trn_loss:.4f} score {trn_score:.4f}')
40                print(f'[Dev] epoch {epoch+1} loss {dev_loss:.4f} score {dev_score:.4f}')
41    def evaluate(self, data_set):
42        X, y = data_set
43        probabilities = self.model(X)
44        return self.metric(probabilities, y), self.loss_fn(probabilities, y).item()
45
46    def predict(self, X):
47        return self.model(X)
48
49    def save_model(self, path):
50        os.makedirs(os.path.dirname(path) or ".", exist_ok=True)
51        torch.save(self.model.params, path)

```

```
52
53     def load_model(self, path):
54         self.model.params = torch.load(path, weights_only=True)
```



思考

RunnerV2 虽已支持梯度下降和最优模型保存，但仍可在多个方面继续完善（例如批量化、学习率调度、训练日志的结构化等），请尝试列出改进点并动手实现一个版本。



3.1.7 模型训练

下面用交叉熵损失和梯度下降训练 Logistic 回归模型：训练 500 轮，每 50 轮打印一次训练指标。代码如下：

```
1  input_size = 2
2  model = Model_LR(input_size)
3  optimizer = SimpleBatchGD(init_lr=0.1, model=model)
4  loss_fn = BinaryCrossEntropyLoss()
5  metric = accuracy
6
7  runner = RunnerV2(model, optimizer, metric, loss_fn)
8  runner.train([X_train, y_train], [X_dev, y_dev],
9              num_epochs=500, log_epochs=50,
10             save_path='./checkpoint/model_lr_best.pt')
```



提醒

梯度下降式训练的日志输出格式高度相似，为节省篇幅，后续章节不再重复展示运行日志。



训练过程可视化 把训练集与验证集上的准确率、损失曲线画在同一张图上，便于观察收敛趋势（绘图代码见配套 notebook）。结果见图 3.3：训练集与验证集的损失均稳定下降，准确率收敛在一个较高水平，说明训练已较充分，过拟合迹象尚不明显。

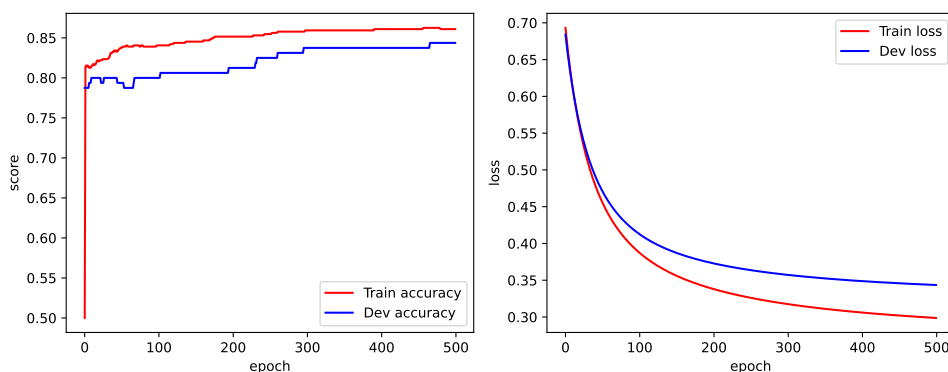


图 3.3 损失和准确率变化趋势

3.1.8 模型评价

训练结束后,先加载按验证准确率保存的最佳快照,再在测试集上评价.这里准确率相同时保留较早的模型;如果要改用损失打破平局,也应在查看测试集之前约定:

```
1 runner.load_model('./checkpoint/model_lr_best.pt')
2 score, loss = runner.evaluate([X_test, y_test])
3 print(f"[Test] score/loss: {score:.4f}/{loss:.4f}")
```

输出结果为:

```
[Test] score/loss: 0.8400/0.3262
```



动手练习 3.4

在验证集上比较学习率与训练轮数,记录收敛速度.再画出阈值0.5对应的决策边界:它始终是一条直线.对于弯月数据中直线无法分开的部分,仅延长训练会有帮助吗?



3.2 基于 Softmax 回归的多分类任务

Logistic 回归只覆盖了类别数为 2 的特殊场景;面对类别数 $C > 2$ 的**多分类**(Multi-class Classification)问题,需要把它推广到 C 条线性判别同时存在的形式——这就是 Softmax 回归.

记类别标签 $y \in \{1, 2, \dots, C\}$. 给定样本 $\mathbf{x}$, Softmax 回归把它属于类别 c 的条件概率写作

$$p(y = c | \mathbf{x}) = \frac{\exp(\mathbf{w}_c^\top \mathbf{x} + b_c)}{\sum_{j=1}^C \exp(\mathbf{w}_j^\top \mathbf{x} + b_j)}, \quad (3.11)$$

其中 $\mathbf{w}_c$ 、 b_c 分别是类别 c 对应的权重向量和偏置。需要说明的是, 这里数学上的类别索引从 1 开始, 而代码实现中类别索引从 0 开始。

结构上, Softmax 回归 = 线性变换 + Softmax 函数 的组合。下面按与二分类相同的步骤, 先合成一份多分类数据, 再实现模型并训练。

3.2.1 数据集构建

先构造一个 3 类合成数据集——Multi1000: 1000 条样本、每条 2 个特征, 标签来自 3 个空间分布不同的簇。为了既能看出 3 个类别的大致区域, 又保留一定的交叠区, 下面把 3 个簇中心放在等边三角形的三个顶点上, 再用类内扰动和特征噪声控制任务难度。

构造函数 `make_multiclass_classification` 的实现如下:

```

1  import math
2
3  def make_multiclass_classification(n_samples=100, n_features=2, n_classes=3,
4                                     shuffle=True, noise=0.1, seed=None,
5                                     class_sep=1.9, cluster_std=1.45,
6                                     label_noise=0.0):
7      """
8      生成类中心可控的多类别簇数据
9      """
10     g = torch.Generator().manual_seed(seed) if seed is not None else None
11     # 计算每个类别的样本数量
12     n_per_class = [n_samples // n_classes] * n_classes
13     for i in range(n_samples - sum(n_per_class)):
14         n_per_class[i % n_classes] += 1
15
16     X = torch.zeros(n_samples, n_features)
17     y = torch.zeros(n_samples, dtype=torch.int64)
18
19     # 把类别中心放在规则多边形上; 3 类 2 维时就是等边三角形
20     if n_features == 1:
21         centroids = torch.linspace(-class_sep, class_sep, n_classes).unsqueeze(1)
22     else:
23         angles = torch.linspace(0, 2 * math.pi, n_classes + 1)[-1:] + math.pi / 2
24         centroids = torch.zeros(n_classes, n_features)
25         centroids[:, 0] = class_sep * torch.cos(angles)
26         centroids[:, 1] = class_sep * torch.sin(angles)
27
28     stop = 0
29     for k, centroid in enumerate(centroids):
30         start, stop = stop, stop + n_per_class[k]
31         y[start:stop] = k

```

```

32         X[start:stop] = centroid + cluster_std * torch.randn(
33             n_per_class[k], n_features, generator=g)
34
35     # noise 控制特征扰动; label_noise 可选控制误标比例
36     if noise > 0:
37         X = X + noise * torch.randn(n_samples, n_features, generator=g)
38     if label_noise > 0:
39         noise_mask = torch.rand(n_samples, generator=g) < label_noise
40         n_noisy = int(noise_mask.sum())
41         if n_noisy > 0:
42             y[noise_mask] = (
43                 y[noise_mask] + torch.randint(1, n_classes, (n_noisy,), generator=g)
44                 ) % n_classes
45     if shuffle:
46         idx = torch.randperm(n_samples, generator=g)
47         X = X[idx]
48         y = y[idx]
49     return X, y

```

采样 1000 条样本:

```

1  # 采样 1000 个样本, 并用 seed 固定随机性
2  n_samples = 1000
3  X, y = make_multiclass_classification(
4      n_samples=n_samples, n_features=2, n_classes=3,
5      noise=0.15, label_noise=0.0, seed=287)

```

绘制散点图 (绘图代码见配套 notebook), 结果见图 3.4, 三类样本分别聚集在三个不同区域.

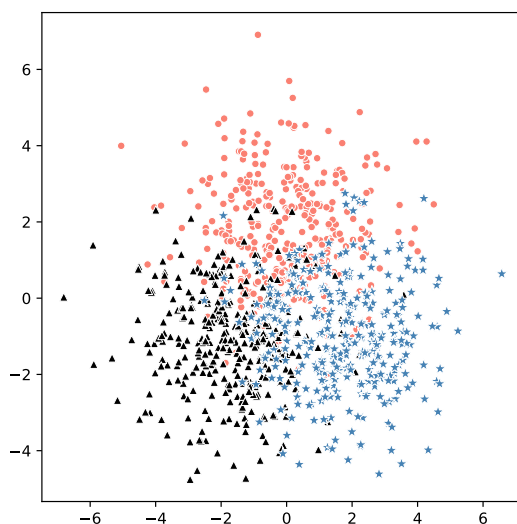


图 3.4 Multi1000 数据集的数据分布

按 640、160、200 条样本切分训练集、验证集和测试集：

```

1 num_train, num_dev, num_test = 640, 160, 200
2 X_train, y_train = X[:num_train], y[:num_train]
3 X_dev, y_dev = X[num_train:num_train + num_dev], y[num_train:num_train + num_dev]
4 X_test, y_test = X[num_train + num_dev:], y[num_train + num_dev:]
5
6 print("X_train shape:", X_train.shape, "y_train shape:", y_train.shape)

```

输出结果为：

```
X_train shape: torch.Size([640, 2]) y_train shape: torch.Size([640])
```

至此Multi1000数据集就绪。

3.2.2 模型构建

Softmax 回归对每个样本同时输出 C 个类别的条件概率。与 Logistic 回归只输出一个数不同，它需要 C 个线性分支，再通过 Softmax 把这 C 个分数归一化为概率分布。

3.2.2.1 Softmax 函数

Softmax 函数把一个 K 维向量 $\mathbf{x} = [x_1, \dots, x_K]$ 映射为一个总和为 1 的概率分布：

$$[\text{softmax}(\mathbf{x})]_k = \frac{\exp(x_k)}{\sum_{i=1}^K \exp(x_i)}. \quad (3.12)$$

直接按上式实现会面临两类数值问题：

- **上溢出**: 当 $\mathbf{x}$ 中存在很大的正数时, $\exp(\cdot)$ 结果可能超过浮点数表示范围.
- **下溢出**: 若 $\mathbf{x}$ 元素普遍为很大的负数, $\exp(\cdot)$ 结果会被舍入为 0, 分母变成 0.

常见的稳态化技巧是先减去最大值: 用 $x_k - \max(\mathbf{x})$ 替换 x_k . 这样指数中的最大值降为 0, 避免上溢; 对于有限输入, 分母至少包含 $\exp(0) = 1$, 避免分母下溢为零. 不过较小的分子仍可能下溢为 0, 因此后续计算对数概率时应使用 `log_softmax`, 而不是先 Softmax 再取对数.

Softmax 函数的实现如下, 沿每行的类别维归一化:

```

1  # X 为张量
2  def softmax(X):
3      """
4      输入:
5          - X: shape=[N, D]
6      """
7      x_max = X.max(dim=1, keepdim=True).values
8      x_exp = torch.exp(X - x_max)
9      partition = x_exp.sum(dim=1, keepdim=True)
10     return x_exp / partition
11 # 观察 softmax 的计算方式
12 X = torch.tensor([[0.1, 0.2, 0.3, 0.4]])
13 print(softmax(X))

```

输出结果为:

```
tensor([[0.2138, 0.2363, 0.2612, 0.2887]])
```

3.2.2.2 Softmax 回归模型

为了和代码实现一致, 模型以批处理形式给出: $\mathbf{X} \in \mathbb{R}^{N \times D}$ 是按行堆叠的样本矩阵, $\hat{\mathbf{Y}} \in \mathbb{R}^{N \times C}$ 是预测概率矩阵. 模型公式为

$$\hat{\mathbf{Y}} = \text{softmax}(\mathbf{X}\mathbf{W} + \mathbf{b}), \quad (3.13)$$

其中 $\mathbf{W} \in \mathbb{R}^{D \times C}$ 的每一列对应一个类别的权重, $\mathbf{b} \in \mathbb{R}^C$ 是偏置向量. $\mathbf{X}\mathbf{W}$ 与 $\mathbf{b}$ 相加时利用广播机制, $\mathbf{X}\mathbf{W}$ 的第 c 列统一加上 b_c .

按公式(3.13)实现 Softmax 回归算子:

```

1  class Model_SR(op.Op):
2      def __init__(self, input_size, output_size):
3          super().__init__()
4          self.params = {
5              # 将线性层的权重参数全部初始化为 0
6              'W': torch.zeros(input_size, output_size),
7              # 将线性层的偏置参数初始化为 0

```

```

8         'b': torch.zeros(output_size),
9     }
10    self.outputs = None
11
12    def forward(self, inputs):
13        """
14        输入:
15        - inputs: shape=[N, D], N 是样本数量, D 是特征维度
16        输出:
17        - outputs: 预测值, shape=[N, C], C 是类别数
18        """
19        # 线性计算
20        score = inputs @ self.params['W'] + self.params['b']
21        # Softmax 函数
22        self.outputs = softmax(score)
23        return self.outputs
24    # 随机生成 1 条长度为 4 的数据
25    torch.manual_seed(0)
26    inputs = torch.randn(1, 4)
27    print('Input is:', inputs)
28    # 实例化模型, 输入长度 4, 输出类别数 3
29    model = Model_SR(input_size=4, output_size=3)
30    outputs = model(inputs)
31    print('Output is:', outputs)

```

输出结果为:

```

Input is: tensor([[ 1.5410, -0.2934, -2.1788,  0.5684]])
Output is: tensor([[0.3333, 0.3333, 0.3333]])

```

输出每个类别的概率都是 $\frac{1}{C}$. 原因仍然是参数全零: 线性输出恒为 0, Softmax 把全零向量映射成均匀分布.

3.2.3 损失函数

Softmax 回归同样使用交叉熵损失. 把多分类标签写成 C 维 one-hot 向量 $\mathbf{y} \in \{0, 1\}^C$: 当真实类别为 c 时

$$\mathbf{y} = [I(1 = c), I(2 = c), \dots, I(C = c)]^T, \quad (3.14)$$

其中 $I(\cdot)$ 是指示函数: 括号内表达式成立时取 1, 否则取 0.

给定训练集 $\{(\mathbf{x}^{(n)}, \mathbf{y}^{(n)})\}_{n=1}^N$, 记 $\hat{\mathbf{y}}^{(n)} = \text{softmax}(\mathbf{W}^T \mathbf{x}^{(n)} + \mathbf{b})$ 为样本 n 的类别概率分布. 多分

类交叉熵损失定义为

$$\mathcal{R}(\mathbf{W}, \mathbf{b}) = -\frac{1}{N} \sum_{n=1}^N (\mathbf{y}^{(n)})^\top \log \hat{\mathbf{y}}^{(n)} = -\frac{1}{N} \sum_{n=1}^N \sum_{c=1}^C \mathbf{y}_c^{(n)} \log \hat{\mathbf{y}}_c^{(n)}. \quad (3.15)$$

由于 $\mathbf{y}_c^{(n)}$ 只在真实类别上为 1, 其余位置都是 0, 交叉熵其实只关心模型对真实类别赋的那一个概率. 借助这一点, 损失可以化简为

$$\mathcal{R}(\mathbf{W}, \mathbf{b}) = -\frac{1}{N} \sum_{n=1}^N \log[\hat{\mathbf{y}}^{(n)}]_{\mathbf{y}^{(n)}}, \quad (3.16)$$

其中 $\mathbf{y}^{(n)}$ 表示第 n 个样本的真实类别索引.

多类交叉熵损失的实现如下, 其中专门按索引取出真实类别上的概率, 避免显式构造 one-hot:

```

1  class MultiCrossEntropyLoss(op.Op):
2      def __init__(self):
3          self.predicts = None
4          self.labels = None
5
6      def forward(self, predicts, labels):
7          """
8          输入:
9              - predicts: 预测值, shape=[N, C]
10             - labels: 真实标签, shape=[N] (long)
11          输出:
12              - 损失值: 标量张量
13          """
14          self.predicts = predicts
15          self.labels = labels
16          N = predicts.shape[0]
17          eps = torch.finfo(predicts.dtype).tiny
18          # 取每个样本正确类别对应的预测概率
19          p_correct = predicts[torch.arange(N, device=predicts.device), labels].clamp(eps,
20                                     1.0)
21          return -torch.log(p_correct).mean()
22
23  # 测试一下
24  labels = torch.tensor([0])
25  mce_loss = MultiCrossEntropyLoss()
26  print(mce_loss(outputs, labels))

```

输出结果为:

```
tensor(1.0986)
```



笔记

PyTorch 的 `CrossEntropyLoss` 接收未经 Softmax 的类别分数 (logits), 目标可以是本章使用的长整数类别索引. 它内部结合了 `LogSoftmax` 与负对数似然; 不要先对输入做 Softmax. 若目标是每个样本可同时拥有多个标签, 则应对各标签分别做二分类, 不能直接套用这里互斥类别的 Softmax 模型.

3.2.4 模型优化

仍采用梯度下降法迭代求解.

3.2.4.1 梯度计算

对 Softmax 回归的经验风险 $\mathcal{R}(\mathbf{W}, \mathbf{b})$ 分别求 $\mathbf{W}$ 和 $\mathbf{b}$ 的偏导, 可得

$$\frac{\partial \mathcal{R}(\mathbf{W}, \mathbf{b})}{\partial \mathbf{W}} = -\frac{1}{N} \sum_{n=1}^N \mathbf{x}^{(n)} (\mathbf{y}^{(n)} - \hat{\mathbf{y}}^{(n)})^{\top} = -\frac{1}{N} \mathbf{X}^{\top} (\mathbf{y} - \hat{\mathbf{y}}), \quad (3.17)$$

$$\frac{\partial \mathcal{R}(\mathbf{W}, \mathbf{b})}{\partial \mathbf{b}} = -\frac{1}{N} \sum_{n=1}^N (\mathbf{y}^{(n)} - \hat{\mathbf{y}}^{(n)})^{\top} = -\frac{1}{N} \mathbf{1}^{\top} (\mathbf{y} - \hat{\mathbf{y}}), \quad (3.18)$$

其中 $\mathbf{X} \in \mathbb{R}^{N \times D}$ 是样本矩阵, $\mathbf{y} \in \mathbb{R}^{N \times C}$ 是真实标签的 one-hot 矩阵, $\hat{\mathbf{y}} \in \mathbb{R}^{N \times C}$ 是预测概率矩阵, $\mathbf{1}$ 为 N 维全 1 向量.

把上式落到 `backward` 函数中:

```
1 import torch.nn.functional as F
2
3 class Model_SR(op.Op):
4     def __init__(self, input_size, output_size):
5         super().__init__()
6         self.params = {
7             'W': torch.zeros(input_size, output_size),
8             'b': torch.zeros(output_size),
9         }
10        self.grads = {}
11        self.X = None
12        self.outputs = None
13        self.output_size = output_size
14
15    def forward(self, inputs):
16        self.X = inputs
17        score = inputs @ self.params['W'] + self.params['b']
18        self.outputs = softmax(score)
19        return self.outputs
20
```

```

21     def backward(self, labels):
22         """
23         输入:
24             - labels: 真实标签, shape=[N], long 类型
25         """
26         N = labels.shape[0]
27         y_onehot = F.one_hot(labels, num_classes=self.output_size).float()
28         self.grads['W'] = -1 / N * (self.X.t() @ (y_onehot - self.outputs))
29         self.grads['b'] = -1 / N * (y_onehot - self.outputs).sum(dim=0)

```



提醒

`backward`里的梯度已经把交叉熵损失一并合并进来,对应的是最终损失关于参数的偏导,而不是`forward`本身的导数。这里同样使用未截断交叉熵的解析梯度;概率发生下溢、触发截断时,应改用融合损失实现。



3.2.4.2 参数更新

得到梯度后,复用第3.1.4.3节的SimpleBatchGD优化器更新参数即可。

3.2.5 模型训练

实例化RunnerV2并传入相应组件,训练500轮(Epoch),每50轮打印一次训练指标:

```

1 input_size = 2
2 output_size = 3
3 model = Model_SR(input_size, output_size)
4 optimizer = SimpleBatchGD(init_lr=0.1, model=model)
5 loss_fn = MultiCrossEntropyLoss()
6 metric = accuracy
7
8 runner = RunnerV2(model, optimizer, metric, loss_fn)
9 runner.train([X_train, y_train], [X_dev, y_dev],
10             num_epochs=500, log_epochs=50,
11             save_path='./checkpoint/model_sr_best.pt')

```

训练曲线见图3.5: 训练损失和验证损失都持续下降并逐渐趋于平稳,训练准确率与验证准确率也稳定在较高水平。由于数据集故意保留了一部分交叠区域和特征噪声,模型不会把所有样本都完美分开,这比完全分离的玩具数据更接近真实分类任务。

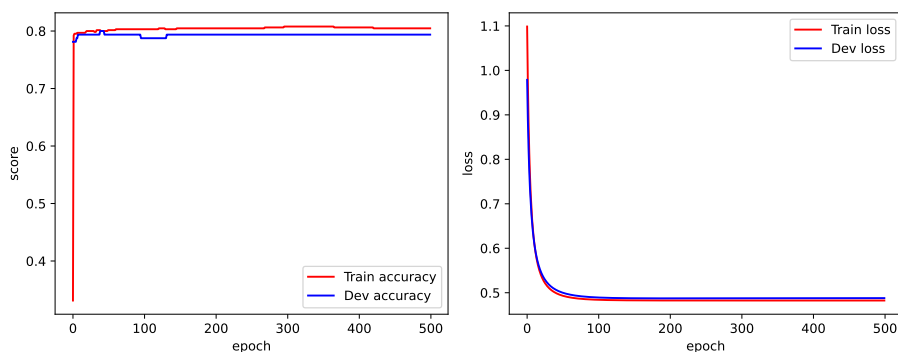


图 3.5 损失和准确率变化趋势



动手练习 3.5

多分类还有另一种思路:把它拆分成 C 个二分类子问题,每次区分一个类别与其余类别 (one-vs-rest),最后聚合输出.请分别用 Softmax 回归和 one-vs-rest 两种方案完成本数据集,比较准确率与训练成本。

3.2.6 模型评价

在测试集上做一次最终评估,量化泛化表现:

```
1 runner.load_model('./checkpoint/model_sr_best.pt')
2 score, loss = runner.evaluate([X_test, y_test])
3 print(f"[Test] score/loss: {score:.4f}/{loss:.4f}")
```

输出结果为:

```
[Test] score/loss: 0.7900/0.5291
```

3.3 实践:基于 Softmax 回归完成鸢尾花分类任务

一朵花的四个长度测量值,能否支持可靠的类别判断?本节用 150 条鸢尾花样本检验 Softmax 回归.样本少,训练很快;但测试集错一朵花,就会使准确率产生明显变化,这也让模型评价成为本例的一部分.

一次机器学习实践通常包含 8 个步骤:数据处理、模型构建、损失函数、优化器、评价指标、训练、评价、预测——前 5 步合称“模型准备”,正是五要素就位的过程.围绕 **Runner**,这条流水线落到代码上就是:

1) 数据处理:把原始数据加工成模型可读取的张量格式.

- 2) 模型构建:本实验使用 Softmax 回归模型.
- 3) 训练配置:交叉熵作损失、梯度下降作优化器、准确率作评价指标.
- 4) 组装Runner:把上述组件传入并实例化.
- 5) 训练—评价—预测:交给Runner串起整条链路.



笔记

后续章节会反复沿用这一模板,不再每次重复说明.



3.3.1 数据处理

3.3.1.1 数据集介绍

Iris 数据集 (俗称鸢尾花数据集) 共有 150 条样本,覆盖 3 种鸢尾花类别 (Setosa、Versicolour、Virginica), 每类 50 条. 每条样本由花萼长度、花萼宽度、花瓣长度、花瓣宽度 4 个特征组成,单位均为厘米. 任务目标是基于这 4 个特征推断样本所属类别.

特征字段见表3.1,类别标签见表3.2.

表 3.1 鸢尾花属性

属性 1	属性 2	属性 3	属性 4
sepal_length	sepal_width	petal_length	petal_width
花萼长度	花萼宽度	花瓣长度	花瓣宽度

表 3.2 鸢尾花类别

英文名	中文名	标签
Setosa Iris	山鸢尾	0
Versicolour Iris	杂色鸢尾	1
Virginica Iris	弗吉尼亚鸢尾	2

3.3.1.2 数据读取

把 150 条样本按 80%、10%、10% 切分成训练、验证和测试三部分;然后仅用训练集估计最小值与最大值,对三部分数据应用相同的最小—最大归一化. 训练特征映射到 $[0, 1]$, 验证和测试特征超出这个范围是允许的,不应重新用各自的极值缩放:

```

1  from sklearn.datasets import load_iris
2  # 加载数据集
3  def load_data(shuffle=True):
4      """
5      加载鸢尾花数据
6      输出:
7          - X: 特征数据, shape=[150, 4]
8          - y: 标签数据, shape=[150]
9      """
10     iris = load_iris()
11     X = torch.tensor(iris.data, dtype=torch.float32)
12     y = torch.tensor(iris.target, dtype=torch.int64)
13
14     if shuffle:
15         idx = torch.randperm(X.shape[0])
16         X = X[idx]
17         y = y[idx]
18     return X, y
19 # 固定随机种子
20 torch.manual_seed(102)
21
22 num_train, num_dev, num_test = 120, 15, 15
23
24 X, y = load_data(shuffle=True)
25 X_train, y_train = X[:num_train], y[:num_train]
26 X_dev, y_dev = X[num_train:num_train + num_dev], y[num_train:num_train + num_dev]
27 X_test, y_test = X[num_train + num_dev:], y[num_train + num_dev:]
28 # 只用训练集估计归一化参数, 避免验证/测试信息泄漏
29 X_min = X_train.min(dim=0).values
30 X_range = (X_train.max(dim=0).values - X_min).clamp_min(1e-12)
31 X_train = (X_train - X_min) / X_range
32 X_dev = (X_dev - X_min) / X_range
33 X_test = (X_test - X_min) / X_range

```

3.3.2 模型构建

复用第3.2.2.2节实现的Model_SR, 输入维度设为4(与特征数一致), 输出维度设为3(与类别数一致):

```

1  input_size = 4
2  output_size = 3
3  model = Model_SR(input_size, output_size)

```

3.3.3 模型训练

实例化RunnerV2,用训练集更新参数、用验证集选择快照,共训练 300 轮,每 50 轮打印一次训练指标,并自动保留验证集上表现最好的模型作为最优快照:

```
1 optimizer = SimpleBatchGD(init_lr=1.0, model=model)
2 loss_fn = MultiCrossEntropyLoss()
3 metric = accuracy
4 runner = RunnerV2(model, optimizer, metric, loss_fn)
5 runner.train([X_train, y_train], [X_dev, y_dev],
6             num_epochs=300, log_epochs=50,
7             save_path='./checkpoint/model_iris_best.pt')
```

训练曲线见图3.6:损失收敛、准确率稳步上升,整体训练过程平稳.

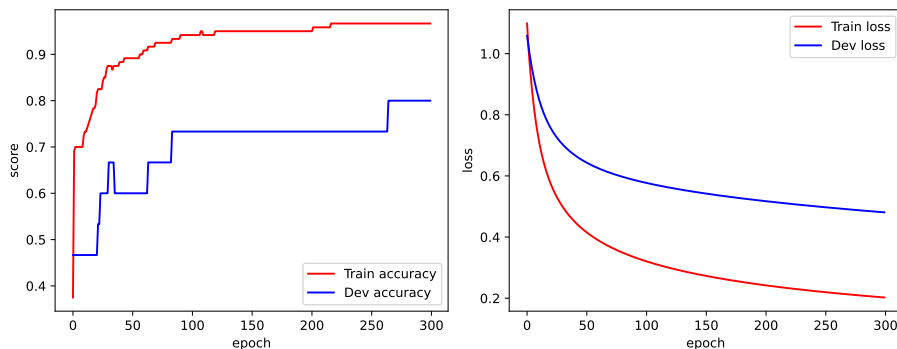


图 3.6 损失和准确率变化趋势

3.3.4 模型评价

载入训练阶段保存的最优模型,并在测试集上评估其准确率:

```
1 runner.load_model('./checkpoint/model_iris_best.pt')
2 score, loss = runner.evaluate([X_test, y_test])
3 print(f"[Test] score/loss: {score:.4f}/{loss:.4f}")
```

输出结果为:

```
[Test] score/loss: 0.9333/0.3149
```

3.3.5 模型预测

用保存好的模型对单条测试样本做预测,演示单点推断.

由于`Model_SR`默认接收形状为 $N \times D$ 的批量矩阵，而单条样本只有 D 维，调用前需先用`unsqueeze`把它升为 $1 \times D$ ：

```
1 # 获取测试集中第一条数据
2 X_one = X_test[0].unsqueeze(0)
3 label = y_test[0].item()
4 # 进行模型推断
5 predicts = runner.predict(X_one)
6 # 获取概率最大的类别
7 pred = predicts.argmax(dim=1).item()
8 print(f"The true category is {label} and the predicted category is {pred}")
```

输出结果为：

```
The true category is 2 and the predicted category is 2
```



动手练习 3.6

尝试实现《神经网络与深度学习》第3章介绍的其他线性分类模型（如感知器、支持向量机）应用于Iris数据集，比较它们与Softmax回归在准确率、训练成本上的差异。



动手练习 3.7

Iris测试集只有15条样本，单次随机划分得到的准确率方差很大。请把数据读取处的`torch.manual_seed`换成多个不同的取值，多次重新划分并训练，统计测试准确率的均值与波动范围；据此说明为什么这种小数据集上的单点准确率（如本节报告的0.9333）不能全信，并尝试用交叉验证来缓解这种不确定性。

3.4 小结

Logistic回归和Softmax回归把线性分数转成类别概率，交叉熵则把这些概率与标签联系起来。手写梯度让我们看到参数怎样移动分类边界；融合损失让这套计算在极端分数下仍能可靠运行。RunnerV2加入训练循环和验证集快照选择后，还需显式加载所选快照，才能评价它的测试表现。

弯月数据也暴露了线性边界的限制。下一章加入隐藏层，观察网络能否学到弯曲的边界，以及更多参数会给训练和泛化带来什么变化。

第4章 前馈神经网络

上一章中, 无论怎样调整 Logistic 回归的参数, 弯月数据的分类边界始终是一条直线. 要让边界弯起来, 可以先变换特征, 再做分类. 本章进一步让这些特征变换也从数据中学习: 把若干线性变换与非线性激活连接起来, 就得到神经网络 (Neural Network, NN).

我们从前馈神经网络 (Feedforward Neural Network, FNN) 开始. 它的前向计算没有反馈回路, 对应一张有向无环图. 除了搭建模型, 还要逐层检查梯度怎样传回来; 更多层带来更多表达能力, 也可能使参数更难训练.

本章对应《神经网络与深度学习》第4章前馈神经网络. 阅读前可先扫一眼图4.1的关键知识点, 便于把概念与后续代码逐一对应.

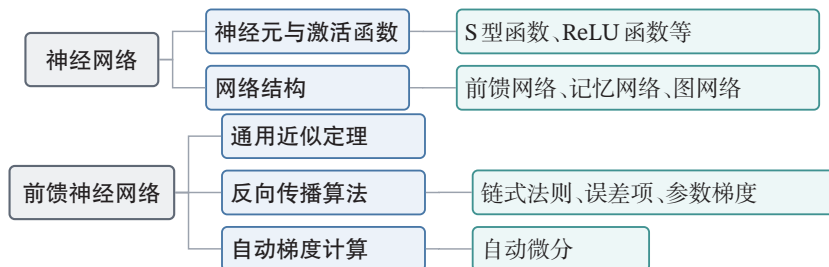


图 4.1 前馈神经网络关键知识点回顾

本章分为两部分:

- **模型解读**: 从神经元到多层前馈网络, 按层次拆解概念与代码实现; 以一个分类任务串起反向传播流程, 并通过两组小实验直观展示**梯度消失**和**死亡 ReLU**两类典型问题及缓解思路.
- **案例实践**: 基于前馈神经网络完成鸢尾花分类任务, 再次走通完整训练-评价-预测流程.

4.1 神经元

神经网络的最小构件是带非线性激活函数的**神经元**, 这里采用一种受生物神经元启发的简化数学模型: 接收一组输入信号, 按权重线性叠加, 再经非线性激活后输出. 记神经元的输入向量为

$\mathbf{x} \in \mathbb{R}^D$, 权重向量为 $\mathbf{w} \in \mathbb{R}^D$, 偏置为 b . 神经元先把输入做加权求和, 得到净活性值 z :

$$z = \mathbf{w}^T \mathbf{x} + b. \quad (4.1)$$

实际工程中通常一次处理 N 个样本以提高吞吐:

$$\mathbf{z} = \mathbf{X}\mathbf{w} + b, \quad (4.2)$$

其中 $\mathbf{X} \in \mathbb{R}^{N \times D}$ 是按行堆叠的样本矩阵, $\mathbf{z} \in \mathbb{R}^N$ 是对应的净活性值向量.

下面用 PyTorch 对一组样本计算净活性值:

```
1 import torch
2 torch.manual_seed(0)
3 # 两个特征数为5的样本
4 X = torch.rand(size=[2, 5])
5 # 含有5个参数的权重向量
6 w = torch.rand(size=[5, 1])
7 # 偏置项
8 b = torch.rand(size=[1, 1])
9 # 使用'torch.matmul'实现矩阵相乘
10 z = torch.matmul(X, w) + b
11 print("input X:", X)
12 print("weight w:", w, "\nbias b:", b)
13 print("output z:", z)
```

输出结果为:

```
input X: tensor([[0.4963, 0.7682, 0.0885, 0.1320, 0.3074],
                 [0.6341, 0.4901, 0.8964, 0.4556, 0.6323]])
weight w: tensor([[0.3489],
                  [0.4017],
                  [0.0223],
                  [0.1689],
                  [0.2939]])
bias b: tensor([[0.5185]])
output z: tensor([[1.1149],
                  [1.2194]])
```



笔记

工程实现中可以直接调用 `nn.Linear` 完成上述线性变换, 无需手动维护权重和偏置。



4.1.1 激活函数

净活性值 z 经过**激活函数** $f(\cdot)$ 得到神经元的活性值 $a = f(z)$. 隐藏层之间需要引入非线性; 若只有线性变换和偏置, 多层复合仍可合并为一次仿射变换. 输出层是否使用激活函数则取决于任务和损失接口. 常用的两类激活函数是**Sigmoid 型函数**和**ReLU 函数**.

4.1.1.1 Sigmoid 型函数

Sigmoid 型函数^①泛指曲线形如“S”、两端饱和的一类函数; 最常用的两个代表是**Logistic 函数**和**Tanh 函数**:

$$\text{Logistic 函数:} \quad \sigma(z) = \frac{1}{1 + \exp(-z)}, \quad (4.3)$$

$$\text{Tanh 函数:} \quad \tanh(z) = \frac{\exp(z) - \exp(-z)}{\exp(z) + \exp(-z)}. \quad (4.4)$$

两个函数的实现如下 (绘图代码见配套 notebook):

```
1 # Logistic函数
2 def logistic(z):
3     return 1.0 / (1.0 + torch.exp(-z))
4 # Tanh函数
5 def tanh(z):
6     return (torch.exp(z) - torch.exp(-z)) / (torch.exp(z) + torch.exp(-z))
```

笔记



工程上可直接使用 `torch.sigmoid` 和 `torch.tanh`; 由框架提供实现, 避免手动展开指数造成的数值问题. 浮点输出仍可能饱和到边界值; 后续取对数时还要使用相应的稳定损失. 这里的手写版本只用于有限区间内的公式演示.

提醒



在 PyTorch 及多数论文里, “Sigmoid” 默认特指 Logistic 函数. 但严格地说, Sigmoid 型是一类 S 形函数的统称, 还包括 Tanh、ArcTan、平滑阶跃函数等.

4.1.1.2 ReLU 函数

ReLU (Rectified Linear Unit, 修正线性单元) 及其变体中, 常见的是 ReLU 本身与**带泄露的 ReLU** (Leaky ReLU), 定义为

$$\text{ReLU}(z) = \max(0, z), \quad (4.5)$$

^① 参考: https://en.wikipedia.org/wiki/Sigmoid_function.

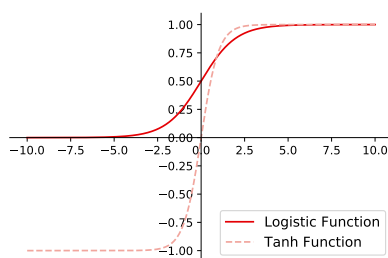
$$\text{LeakyReLU}(z) = \max(0, z) + \lambda \min(0, z), \quad (4.6)$$

其中 λ 是带泄露 ReLU 的超参数 (常取 0.01 或 0.1), 用于在负半轴保留非零斜率, 使梯度不因负输入而直接截断。

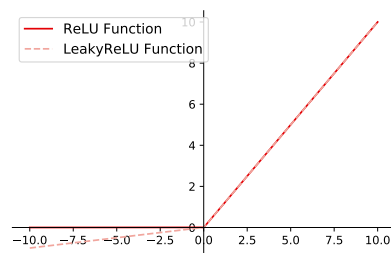
下面给出 ReLU 与 Leaky ReLU 的实现:

```
1 # relu
2 def relu(z):
3     return z.clamp_min(0)
4 # 带泄露的relu
5 def leaky_relu(z, negative_slope=0.1):
6     return torch.where(z > 0, z, negative_slope * z)
```

在 $[-10, 10]$ 区间内取一系列输入值代入上面几个函数, 绘制出的曲线见图 4.2 (绘图代码见配套 notebook)。Logistic 和 Tanh 两条 S 形函数在 0 附近近似线性, 两端迅速饱和, 输出区间分别为 $(0, 1)$ 和 $(-1, 1)$; ReLU 在负半轴恒为 0, Leaky ReLU 仍保留一段小斜率, 避免梯度完全消失。



(a) Logistic 和 Tanh 函数



(b) ReLU 和带泄露的 ReLU 函数

图 4.2 常用激活函数曲线

笔记

PyTorch 里的对应实现为 `torch.nn.functional.relu` 和 `torch.nn.functional.leaky_relu`。

动手练习 4.1

本节只展开了 Sigmoid 与 ReLU 两族最常用的激活函数。请尝试实现《神经网络与深度学习》第 4.1 节中介绍的其他激活函数, 例如 Hard-Logistic、Hard-Tanh、ELU、Softplus、Swish 等, 比较它们在小型分类任务上的差异。

4.2 基于前馈神经网络的二分类任务

前馈网络的连接方式见图4.3：信号从输入层一层层向输出层推进，每一层把上一层的活性值乘权重、加偏置、再过激活函数。整条链路是单向的、无反馈回路，最终在输出层得到 $\mathbf{a}^{(L)}$ 。

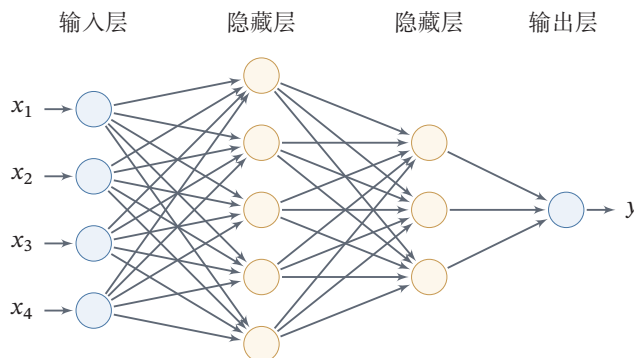


图 4.3 前馈神经网络结构

下面动手实现一个简单的前馈神经网络，并在一个二分类数据集上跑通训练。

4.2.1 数据集构建

继续使用第3.1.1节构造的Moon1000数据集（640、160、200 条样本切分），数据由两条带噪声的弯月形分布生成，每条样本含 2 个特征。下面从配套工具包加载同一生成函数，固定数据随机性：

```
1 from nndl.data import make_moons
2 X_moon, y_moon = make_moons(n_samples=1000, noise=0.2, seed=0)
3 y_moon = y_moon.unsqueeze(1)
4 X_train, y_train = X_moon[:640], y_moon[:640]
5 X_dev, y_dev = X_moon[640:800], y_moon[640:800]
6 X_test, y_test = X_moon[800:], y_moon[800:]
```

4.2.2 模型构建

把网络拆分为可复用的层算子，再按需要的层数堆叠，是搭建前馈网络的常见做法。

设第 l 层的输入是第 $l-1$ 层的活性值 $\mathbf{a}^{(l-1)}$ 。仿射变换得到净活性值 $\mathbf{z}$ ，再经激活函数变成成本层活性值 $\mathbf{a}$ 。

为了充分利用矩阵运算，工程中一次性处理 N 个样本。记第 l 层的输入为 $\mathbf{A}^{(l-1)} \in \mathbb{R}^{N \times M_{l-1}}$ （每行一个样本），第 l 层的计算公式为

$$\mathbf{Z}^{(l)} = \mathbf{A}^{(l-1)} \mathbf{W}^{(l)} + \mathbf{b}^{(l)} \in \mathbb{R}^{N \times M_l}, \quad (4.7)$$

$$\mathbf{A}^{(l)} = f_l(\mathbf{Z}^{(l)}) \in \mathbb{R}^{N \times M_l}, \quad (4.8)$$

其中 $\mathbf{Z}^{(l)}$ 是该层 N 个样本的净活性值, $\mathbf{A}^{(l)}$ 是激活后的活性值; $\mathbf{W}^{(l)} \in \mathbb{R}^{M_{l-1} \times M_l}$ 与 $\mathbf{b}^{(l)} \in \mathbb{R}^{1 \times M_l}$ 是层参数; $f_l(\cdot)$ 是该层激活函数。



提醒

本章的全连接网络使用“样本数 $\times$ 特征数”的张量布局, $\mathbf{X}$ 由 N 个 $\mathbf{x}$ 的行向量堆叠而成;这与《神经网络与深度学习》主书中 $\mathbf{x}$ 为列向量的写法刚好转置,因此 $\mathbf{W}$ 和 $\mathbf{b}$ 的形状也对应转置。

为了让模型搭建更模块化,把公式(4.7)与公式(4.8)分别封装为算子,全部继承自Op基类。

4.2.2.1 线性层算子

公式(4.7)对应一个线性层算子,权重默认随机初始化、偏置默认置零:

```
1 from nndl.op import Op
2 # 实现线性层算子
3 class Linear(Op):
4     def __init__(self, input_size, output_size, name, weight_init=torch.randn, bias_init=
        torch.zeros):
5         """
6         输入:
7             - input_size: 输入层神经元数量
8             - output_size: 输出层神经元数量
9             - name: 算子名称
10            - weight_init: 权重初始化方式, 默认用'torch.randn'进行标准正态分布初始化
11            - bias_init: 偏置初始化方式, 默认使用全0初始化
12        """
13        self.params = {}
14        # 初始化权重
15        self.params['W'] = weight_init(size=[input_size, output_size])
16        # 初始化偏置
17        self.params['b'] = bias_init(size=[1, output_size])
18        self.inputs = None
19        self.name = name
20
21    def forward(self, inputs):
22        """
23        输入:
24            - inputs: size=[N, input_size], N是样本数量
25        输出:
26            - outputs: 预测值, size=[N, output_size]
27        """
28        self.inputs = inputs
```

```

29         outputs = torch.matmul(self.inputs, self.params['W']) + self.params['b']
30         return outputs

```

4.2.2.2 Logistic 算子

把公式(4.8)中的 Logistic 激活也封装为算子,便于与线性层组合:

```

1 class Logistic(Op):
2     def __init__(self):
3         self.inputs = None
4         self.outputs = None
5
6     def forward(self, inputs):
7         """
8         输入:
9             - inputs: size=[N,D]
10        输出:
11            - outputs: size=[N,D]
12        """
13        outputs = torch.sigmoid(inputs)
14        self.outputs = outputs
15        return outputs

```

4.2.2.3 层的串行组合

定义好线性层和激活算子之后,按“线性→激活”的节奏反复堆叠就能得到任意层数的前馈网络.下面以二分类任务为例,搭一个两层、激活函数为 Logistic 的小网络:

```

1 # 实现一个两层前馈神经网络
2 class Model_MLP_L2(Op):
3     def __init__(self, input_size, hidden_size, output_size):
4         """
5         输入:
6             - input_size: 输入层神经元数量
7             - hidden_size: 隐藏层神经元数量
8             - output_size: 输出层神经元数量
9         """
10        self.fc1 = Linear(input_size, hidden_size, name="fc1")
11        self.act_fn1 = Logistic()
12        self.fc2 = Linear(hidden_size, output_size, name="fc2")
13        self.act_fn2 = Logistic()
14
15    def forward(self, X):
16        """
17        输入:
18            - X: size=[N,input_size], N是样本数量

```

```

19         输出:
20         - a2: 预测值, size=[N,output_size]
21         """
22         z1 = self.fc1(X)
23         a1 = self.act_fn1(z1)
24         z2 = self.fc2(a1)
25         a2 = self.act_fn2(z2)
26         return a2

```

测试一下 实例化一个输入维度 5、隐藏层 10、输出维度 1 的两层前馈网络,并喂一条长度为 5 的随机样本,观察输出形状是否符合预期:

```

1  torch.manual_seed(0)
2  # 实例化模型
3  model = Model_MLP_L2(input_size=5, hidden_size=10, output_size=1)
4  # 随机生成一条长度为5的数据
5  X = torch.rand(size=[1, 5])
6  result = model(X)
7  print ("result: ", result)

```

输出结果为:

```
result: tensor([[0.6587]])
```

4.2.3 损失函数

二分类任务沿用**二分类交叉熵**损失,公式定义参见第3.1.3节。

这里把它也封装为算子,便于和后续模型组合使用:

```

1  # 实现交叉熵损失函数
2  class BinaryCrossEntropyLoss(Op):
3      def __init__(self, model):
4          self.predicts = None
5          self.labels = None
6          self.num = None
7
8          self.model = model
9
10     def forward(self, predicts, labels):
11         """
12         输入:
13         - predicts: 预测值, size=[N, 1], N为样本数量
14         - labels: 真实标签, size=[N, 1]
15         输出:

```

```

16         - 损失值：零维标量张量
17         """
18         # 避免饱和概率导致  $\log(0)$ ;  $\text{float32}$  下  $1e-7$  可保持上界严格小于 1
19         eps = max(1e-7, torch.finfo(predicts.dtype).eps)
20         self.clip_mask = (predicts >= eps) & (predicts <= 1.0 - eps)
21         self.predicts = predicts.clamp(eps, 1.0 - eps)
22         self.labels = labels
23         self.num = self.predicts.shape[0]
24         loss = -1. / self.num * (torch.matmul(self.labels.t(), torch.log(self.predicts))
25                                 + torch.matmul((1-self.labels.t()), torch.log(1-self.predicts)))
26
27         loss = loss.squeeze()
28         return loss
29
30     def backward(self):
31         # 计算损失函数对输入的导数
32         inputs_grads = -1.0 * (self.labels / self.predicts -
33                                (1 - self.labels) / (1 - self.predicts)) / self.num
34         self.model.backward(inputs_grads * self.clip_mask)

```

4.2.4 模型优化

神经网络仍以**梯度下降法**为基本优化手段, 区别在于: 线性模型可以直接写出梯度, 而神经网络是一个层层嵌套的复合函数, 必须借助链式法则做反向传播。

4.2.4.1 反向传播算法

前馈神经网络的梯度计算遵循**误差反向传播**算法, 整个训练过程可拆为三步:

- 1) 前向计算每层的净活性值 $\mathbf{Z}^{(l)}$ 和活性值 $\mathbf{A}^{(l)}$, 直到输出层. 由每个算子的 `forward()` 实现.
- 2) 反向计算每层的误差项 $\delta^{(l)} = \frac{\partial \mathcal{L}}{\partial \mathbf{Z}^{(l)}}$, 由 `backward()` 实现.
- 3) 在 `backward()` 里顺带把每层参数的梯度也算好, 再交给**优化器**统一更新参数.

训练循环内: 先 `forward()`, 再 `backward()` 得到所有梯度, 最后由优化器迭代参数.

以本节的两层网络 `Model_MLP_L2` 为例, 图 4.4 描绘了完整的前向与反向流程.

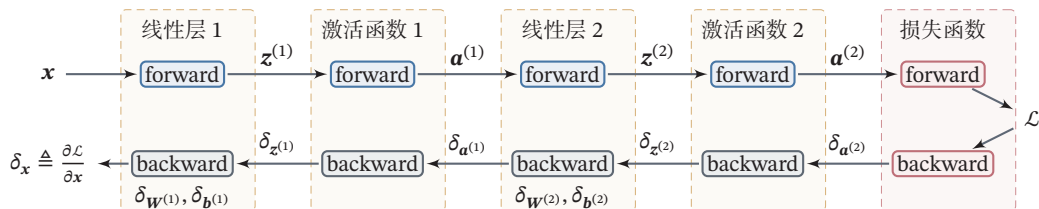


图 4.4 两层全连接前馈神经网络的前向和反向计算过程示例

接下来按反向传播的顺序, 从损失函数开始, 逐个算子补上`backward()`方法, 并在线性层中顺便算好参数的梯度。

4.2.4.2 损失函数

公式(3.5)定义的二分类交叉熵, 其输入是网络输出 $\hat{\mathbf{y}}$. 损失 $\mathcal{R}$ 对 $\hat{\mathbf{y}}$ 的偏导数为:

$$\frac{\partial \mathcal{R}}{\partial \hat{\mathbf{y}}} = -\frac{1}{N} \left(\text{diag} \left(\frac{1}{\hat{\mathbf{y}}} \right) \mathbf{y} - \text{diag} \left(\frac{1}{1 - \hat{\mathbf{y}}} \right) (1 - \mathbf{y}) \right) \quad (4.9)$$

$$= -\frac{1}{N} \left(\frac{1}{\hat{\mathbf{y}}} \odot \mathbf{y} - \frac{1}{1 - \hat{\mathbf{y}}} \odot (1 - \mathbf{y}) \right), \quad (4.10)$$

其中 $\text{diag}(\mathbf{x})$ 把向量 $\mathbf{x}$ 放到对角线得到对角阵, $\frac{1}{x}$ 表示逐元素求倒数, $\odot$ 表示逐元素乘积。

为该损失函数补上`backward()`方法. 为防止演示代码取到 $\log(0)$, 前向会截断极端概率; 反向也须乘上截断操作的导数掩码. 这种处理与第3.1.3节讨论的一样, 不适合代替基于logits的稳定损失; 本节用它展示逐算子的链式法则。

```

1  # 实现交叉熵损失函数
2  class BinaryCrossEntropyLoss(Op):
3      def __init__(self, model):
4          self.predicts = None
5          self.labels = None
6          self.num = None
7
8          self.model = model
9
10     def forward(self, predicts, labels):
11         """
12         输入:
13             - predicts: 预测值, size=[N, 1], N为样本数量
14             - labels: 真实标签, size=[N, 1]
15         输出:
16             - 损失值: 零维标量张量
17         """
18         # 避免饱和概率导致 log(0); float32 下 1e-7 可保持上界严格小于 1
19         eps = max(1e-7, torch.finfo(predicts.dtype).eps)
20         self.clip_mask = (predicts >= eps) & (predicts <= 1.0 - eps)
21         self.predicts = predicts.clamp(eps, 1.0 - eps)
22         self.labels = labels
23         self.num = self.predicts.shape[0]
24         loss = -1. / self.num * (torch.matmul(self.labels.t(), torch.log(self.predicts))
25                                 + torch.matmul((1-self.labels.t()), torch.log(1-self.predicts)))
26         loss = loss.squeeze()
27         return loss
28
29     def backward(self):

```

```

30     # 计算损失函数对输入的导数
31     inputs_grads = -1.0 * (self.labels / self.predicts -
32                           (1 - self.labels) / (1 - self.predicts)) / self.num
33     self.model.backward(inputs_grads * self.clip_mask)

```

4.2.4.3 Logistic 算子



提醒

为了使算子与具体层数解耦,下面的推导不再带层标 l ,与《神经网络与深度学习》主书中按层标的写法略有不同。

接着给 Logistic 激活算子加上反向函数。

Logistic 的前向过程为 $\mathbf{A} = \sigma(\mathbf{Z})$, 其中 $\mathbf{Z}, \mathbf{A} \in \mathbb{R}^{N \times D}$ 的每一行对应一个样本。

为简洁起见,用单样本向量 $\mathbf{a}, \mathbf{z} \in \mathbb{R}^D$ 代替矩阵,二者关系为 $\mathbf{a} = \sigma(\mathbf{z})$, 则 $\mathbf{a}$ 对 $\mathbf{z}$ 的 Jacobian 为

$$\frac{\partial \mathbf{a}}{\partial \mathbf{z}} = \text{diag}(\mathbf{a} \odot (1 - \mathbf{a})) \in \mathbb{R}^{D \times D}, \quad (4.11)$$

其中 $\text{diag}(\mathbf{x})$ 把向量放到对角线, $\odot$ 是逐元素积。

按反向传播规则,记 $\delta_{\mathbf{a}} = \frac{\partial \mathcal{R}}{\partial \mathbf{a}} \in \mathbb{R}^D$ 为上游传来的梯度,则有

$$\delta_{\mathbf{z}} \triangleq \frac{\partial \mathcal{R}}{\partial \mathbf{z}} = \frac{\partial \mathbf{a}}{\partial \mathbf{z}} \delta_{\mathbf{a}} \quad (4.12)$$

$$= \text{diag}(\mathbf{a} \odot (1 - \mathbf{a})) \delta_{\mathbf{a}} \quad (4.13)$$

$$= \mathbf{a} \odot (1 - \mathbf{a}) \odot \delta_{\mathbf{a}}. \quad (4.14)$$

把上式推广到批量数据:记 $\delta_{\mathbf{A}} = \frac{\partial \mathcal{R}}{\partial \mathbf{A}} \in \mathbb{R}^{N \times D}$ 为输出 $\mathbf{A}$ 对应的梯度,公式(4.14)化为

$$\delta_{\mathbf{Z}} = \mathbf{A} \odot (1 - \mathbf{A}) \odot \delta_{\mathbf{A}} \in \mathbb{R}^{N \times D}, \quad (4.15)$$

$\delta_{\mathbf{Z}}$ 就是反向算子要返回的梯度。

Logistic 函数本身没有可学习参数,所以 `backward()` 只需把梯度反传出去,无须维护参数梯度. 实现如下:

```

1 class Logistic(Op):
2     def __init__(self):
3         self.inputs = None
4         self.outputs = None
5         self.params = None
6
7     def forward(self, inputs):

```

```

8         outputs = torch.sigmoid(inputs)
9         self.outputs = outputs
10        return outputs
11
12    def backward(self, outputs_grads):
13        # 计算Logistic激活函数对输入的导数
14        # torch.mul是逐元素相乘算子
15        outputs_grad_inputs = torch.mul(self.outputs, (1.0 - self.outputs))
16        return torch.mul(outputs_grads, outputs_grad_inputs)

```



动手练习 4.2

仿照 Logistic 算子的写法，自行实现一个 ReLU 算子并补上反向计算。本书在第 7.6.2.1 节中也给出了一份参考实现。

4.2.4.4 线性层算子

线性层算子 **Linear** 前向计算为 $\mathbf{Y} = \mathbf{X}\mathbf{W} + \mathbf{b}$ ，其中输入 $\mathbf{X} \in \mathbb{R}^{N \times M}$ ，输出 $\mathbf{Y} \in \mathbb{R}^{N \times D}$ ；参数包括权重 $\mathbf{W} \in \mathbb{R}^{M \times D}$ 和偏置 $\mathbf{b} \in \mathbb{R}^{1 \times D}$ 。 $\mathbf{X}$ 和 $\mathbf{Y}$ 的每一行都是一个样本。

用单样本向量 $\mathbf{x} \in \mathbb{R}^M$ 、 $\mathbf{y} \in \mathbb{R}^D$ 来推导更简洁： $\mathbf{y} = \mathbf{W}^\top \mathbf{x} + \mathbf{b}^\top$ ， $\mathbf{y}$ 对 $\mathbf{x}$ 的偏导为

$$\frac{\partial \mathbf{y}}{\partial \mathbf{x}} = \mathbf{W}^\top \in \mathbb{R}^{D \times M}. \quad (4.16)$$

计算线性层输入 $\mathbf{X}$ 的梯度 记 $\delta_{\mathbf{y}} = \frac{\partial \mathcal{R}}{\partial \mathbf{y}} \in \mathbb{R}^D$ 为上游传入的梯度，则

$$\delta_{\mathbf{x}} \triangleq \frac{\partial \mathcal{R}}{\partial \mathbf{x}} = \mathbf{W} \delta_{\mathbf{y}}. \quad (4.17)$$

推广到批量形式，记 $\delta_{\mathbf{Y}} = \frac{\partial \mathcal{R}}{\partial \mathbf{Y}} \in \mathbb{R}^{N \times D}$ ，公式(4.17)变为

$$\delta_{\mathbf{X}} = \delta_{\mathbf{Y}} \mathbf{W}^\top, \quad (4.18)$$

$\delta_{\mathbf{X}}$ 即反向算子向上游返回的梯度。

计算线性层参数 $\mathbf{W}$ 和 $\mathbf{b}$ 的梯度 线性层带有可学习参数，因此 **backward()** 除了传梯度外，还要算参数的梯度。借助《神经网络与深度学习》主书公式(4.68)、(4.69)的结论，在单样本下有

$$\delta_{\mathbf{W}} \triangleq \frac{\partial \mathcal{R}}{\partial \mathbf{W}} = \mathbf{x} \delta_{\mathbf{y}}^\top, \quad (4.19)$$

$$\delta_{\mathbf{b}} \triangleq \frac{\partial \mathcal{R}}{\partial \mathbf{b}} = \delta_{\mathbf{y}}^\top. \quad (4.20)$$

推广到批量形式，公式(4.19)和公式(4.20)化为

$$\delta_{\mathbf{W}} = \mathbf{X}^\top \delta_{\mathbf{Y}}, \quad (4.21)$$

$$\delta_b = 1^T \delta_Y. \quad (4.22)$$

把上述公式落到`Linear`的`backward()`方法里,参数梯度写入`grads`属性,便于优化器读取:

```

1 class Linear(Op):
2     def __init__(self, input_size, output_size, name, weight_init=torch.randn, bias_init=
      torch.zeros):
3         self.params = {}
4         self.params['W'] = weight_init(size=[input_size, output_size])
5         self.params['b'] = bias_init(size=[1, output_size])
6         self.inputs = None
7         self.grads = {}
8         self.name = name
9
10    def forward(self, inputs):
11        self.inputs = inputs
12        outputs = torch.matmul(self.inputs, self.params['W']) + self.params['b']
13        return outputs
14
15    def backward(self, grads):
16        """
17        输入:
18            - grads: 损失函数对当前层输出的导数
19        输出:
20            - 损失函数对当前层输入的导数
21        """
22        self.grads['W'] = torch.matmul(self.inputs.T, grads)
23        self.grads['b'] = torch.sum(grads, dim=0, keepdim=True)
24        return torch.matmul(grads, self.params['W'].T)

```

动手练习 4.3

把本节手写的`Linear`与`Logistic`算子组合成的两层前馈网络,对同一条输入和标签验证反向传播是否正确:

1) 用手写算子完成一次`forward`与`backward`, 记录各线性层参数的梯度`grads['W']`、`grads['b']`.

2) 用`torch.nn.Linear`和`torch.sigmoid`搭一个结构相同、权重与偏置逐一对齐的网络,借助 PyTorch 的自动梯度计算对同一批数据再算一遍各层参数梯度。

3) 逐元素比较两套梯度,统计每个参数张量上的最大绝对差,确认手写反向公式与自动微分的结果是否一致;若出现明显偏差,回到误差项 δ 的推导逐层定位问题所在。

4.2.4.5 整个网络

把上述各层串起来,得到完整的两层网络,前向和反向都按序调用各算子:

```

1 class Model_MLP_L2(Op):
2     def __init__(self, input_size, hidden_size, output_size):
3         #线性层
4         self.fc1 = Linear(input_size, hidden_size, name="fc1")
5         #Logistic激活函数层
6         self.act_fn1 = Logistic()
7         self.fc2 = Linear(hidden_size, output_size, name="fc2")
8         self.act_fn2 = Logistic()
9         self.layers = [self.fc1, self.act_fn1, self.fc2, self.act_fn2]
10        #前向计算
11        def forward(self, X):
12            z1 = self.fc1(X)
13            a1 = self.act_fn1(z1)
14            z2 = self.fc2(a1)
15            a2 = self.act_fn2(z2)
16            return a2
17        #反向计算
18        def backward(self, loss_grad_a2):
19            loss_grad_z2 = self.act_fn2.backward(loss_grad_a2)
20            loss_grad_a1 = self.fc2.backward(loss_grad_z2)
21            loss_grad_z1 = self.act_fn1.backward(loss_grad_a1)
22            loss_grad_inputs = self.fc1.backward(loss_grad_z1)

```



提醒

我们手写的算子目前还不会自动构建计算图,所以模型的backward()里要手动按顺序调用每一层的backward();同样地,下面的优化器也需要显式遍历各层参数。

4.2.4.6 优化器

参数梯度算好后,更新逻辑放到优化器里实现。

与第3.1.4.3节里的SimpleBatchGD相比,这里需要遍历模型的每一层,针对各自的参数字典分别做更新:

```

1 from nndl.optim import Optimizer
2
3 class BatchGD(Optimizer):
4     def __init__(self, init_lr, model):
5         super().__init__(init_lr=init_lr, model=model)
6

```

```

7     def step(self):
8         # 参数更新
9         for layer in self.model.layers: # 遍历所有层
10             if isinstance(layer.params, dict):
11                 for key in layer.params.keys():
12                     layer.params[key] = layer.params[key] - self.init_lr * layer.grads[key]

```

4.2.5 完善 Runner 类: RunnerV2_1

第3.1.6节的RunnerV2面向较简单的单层模型, 对本章的多算子组合模型并不够用. 这里再做一次增量升级RunnerV2_1, 主要新增两点能力:

- 1) 支持自定义算子的反向计算: 训练时通过`self.loss_fn.backward()`从损失函数开始反向传梯度;
- 2) 支持把各层参数按层名打包到同一个文件, 并按层名加载; 没有参数的激活层不参与存取.

代码实现如下:

```

1  import os
2  from pathlib import Path
3
4  class RunnerV2_1:
5      """支持多算子模型反向传播的训练框架（手写算子时代过渡版）. """
6
7      def __init__(self, model, optimizer, metric, loss_fn):
8          self.model = model
9          self.optimizer = optimizer
10         self.loss_fn = loss_fn
11         self.metric = metric
12         # 训练过程指标统一收纳到 history 字典中
13         self.history = {
14             'train_loss': [], 'dev_loss': [],
15             'train_score': [], 'dev_score': [],
16         }
17
18     def train(self, train_set, dev_set, num_epochs=100, log_epochs=100,
19              save_path=None):
20         # 用 -inf 初始化保证首轮一定触发对比
21         best_score = -float('inf')
22         for epoch in range(num_epochs):
23             X, y = train_set
24             # 训练一步
25             probabilities = self.model(X)

```

```

26         trn_loss = self.loss_fn(probabilities, y)
27         trn_score = self.metric(probabilities, y)
28         self.loss_fn.backward()
29         self.optimizer.step()
30         self.history['train_loss'].append(trn_loss.item())
31         self.history['train_score'].append(trn_score)
32         # 验证集评估
33         dev_score, dev_loss = self.evaluate(dev_set)
34         self.history['dev_loss'].append(dev_loss)
35         self.history['dev_score'].append(dev_score)
36         # 保存最优
37         if dev_score > best_score:
38             print(f'[Evaluate] best score updated: {best_score:.5f} -> {dev_score:.5f}')
39             best_score = dev_score
40             if save_path:
41                 self.save_model(save_path)
42         if log_epochs and (epoch + 1) % log_epochs == 0:
43             print(f'[Train] epoch {epoch+1}/{num_epochs} loss {trn_loss.item():.4f}')
44
45     def evaluate(self, data_set):
46         """纯查询: 不写入历史. """
47         X, y = data_set
48         probabilities = self.model(X)
49         loss = self.loss_fn(probabilities, y).item()
50         score = self.metric(probabilities, y)
51         return score, loss
52
53     def predict(self, X):
54         return self.model(X)
55
56     def save_model(self, save_path):
57         # 把所有层的 params 字典统一打包成 {layer_name: params} 一次性写盘
58         state = {layer.name: layer.params for layer in self.model.layers
59                  if isinstance(getattr(layer, 'params', None), dict)}
60         Path(save_path).parent.mkdir(parents=True, exist_ok=True)
61         torch.save(state, save_path)
62
63     def load_model(self, save_path):
64         state = torch.load(save_path, weights_only=True)
65         for layer in self.model.layers:
66             if getattr(layer, 'name', None) in state:
67                 layer.params = state[layer.name]

```

4.2.6 模型训练

评价指标沿用第3.1.5节定义的准确率。

下面基于RunnerV2_1训练前馈神经网络，用训练集更新参数、用验证集选择快照，共训练1000轮：

```
1 from nndl import accuracy
2 # 输入层神经元数量为2
3 input_size = 2
4 # 隐藏层神经元数量为5
5 hidden_size = 5
6 # 输出层神经元数量为1
7 output_size = 1
8 # 固定初始化，便于与配套 Notebook 对照
9 torch.manual_seed(0)
10 model = Model_MLP_L2(input_size, hidden_size, output_size)
11 # 损失函数
12 loss_fn = BinaryCrossEntropyLoss(model)
13 # 优化器
14 lr = 0.2
15 optimizer = BatchGD(lr, model)
16 # 评价指标
17 metric = accuracy
18 # 实例化RunnerV2_1类，并传入训练配置
19 runner = RunnerV2_1(model, optimizer, metric, loss_fn)
20 runner.train([X_train, y_train], [X_dev, y_dev], num_epochs=1000, log_epochs=50, save_path
              = "./checkpoint/model_best.pt")
```

把训练集与验证集的损失同时画出来（绘图代码见配套 notebook），便于观察收敛情况。

结果见图4.5：两条损失曲线共同下降，没有明显的过拟合迹象。

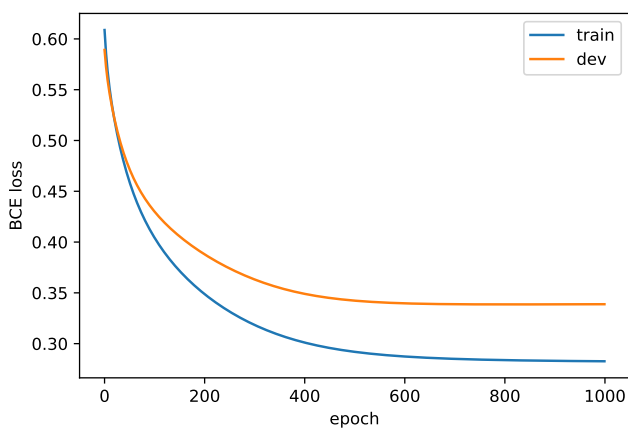


图 4.5 损失函数变化趋势

4.2.7 模型评价

载入最优模型,在测试集上做最终评价:

```
1 # 加载训练好的模型
2 runner.load_model("./checkpoint/model_best.pt")
3 # 在测试集上对模型进行评价
4 score, loss = runner.evaluate([X_test, y_test])
5 print(f"[Test] score/loss: {score:.4f}/{loss:.4f}")
```

输出结果为:

```
[Test] score/loss: 0.8500/0.3244
```

模型能否利用隐藏层学到弯曲的边界,还需结合决策图和验证误差判断. 增加隐藏层并不保证在当前初始化、宽度和训练轮数下超过上一章的线性模型;参数更多只是提供了可能性.

4.3 自动梯度计算和预定义算子

手动按层写反向虽然概念清晰,但实战中极易在细节上出错;现代框架早已把这一步交给自动梯度计算 (autograd) ——开发者只描述前向,框架顺着计算图自动推导反向.

`torch.nn.Module`负责组织参数、子模块和状态,autograd 则记录实际执行的可微张量运算. 只要相关参数需要梯度,且计算没有被`detach`或`no_grad`切断,反向传播就能沿图计算梯度;优化器再更新交给它的参数. 自定义层也可以使用普通张量函数,并非每一步都必须封装为 `Module`.

4.3.1 利用预定义算子重新实现前馈神经网络

下面用 PyTorch 的预定义算子重新实现第 4.2.4.5 节的两层前馈网络, 并复用第 4.2 节的二分类任务对比效果. 核心组件是 `torch.nn.Linear`:

```
1 layer = torch.nn.Linear(in_features=5, out_features=3, bias=True)
```

`nn.Linear` 的输入张量形状为 `[*, in_features]` (`*` 表示任意数量的前导维度, 也允许只有一个特征向量), 内部保存的权重形状为 `[out_features, in_features]`, 前向计算相当于输入乘权重的转置再加偏置, 产出形状为 `[*, out_features]` 的输出. 默认带偏置, 可通过 `bias=False` 关掉.

基于预定义算子的两层网络实现如下:

```
1 import torch.nn as nn
2 import torch.nn.functional as F
3 from functools import partial
4 # 用 functools.partial 固定初始化参数, 得到接收一个张量的初始化函数.
5 def Normal(mean=0.0, std=1.0):
6     return partial(nn.init.normal_, mean=mean, std=std)
7 def Constant(value=0.0):
8     return partial(nn.init.constant_, val=value)
9 def Uniform(low=-1.0, high=1.0):
10    return partial(nn.init.uniform_, a=low, b=high)
11 def Kaiming():
12     # He (Kaiming) 初始化: std=sqrt(2/fan_in), 适配ReLU; 在相应统计假设下控制信号尺度
13     return partial(nn.init.kaiming_normal_, nonlinearity='relu')
14
15 class Model_MLP_L2_V2(nn.Module):
16     def __init__(self, input_size, hidden_size, output_size):
17         super().__init__()
18         # 使用 'nn.Linear' 定义线性层.
19         # 其中 in_features 为线性层输入维度, out_features 为线性层输出维度.
20         self.fc1 = nn.Linear(input_size, hidden_size)
21         self.fc2 = nn.Linear(hidden_size, output_size)
22         # 用 'torch.nn.init.normal_' 对权重做随机高斯分布初始化,
23         # 用 'torch.nn.init.constant_' 把偏置初始化为常数.
24         nn.init.normal_(self.fc1.weight)
25         nn.init.constant_(self.fc1.bias, 0.0)
26         nn.init.normal_(self.fc2.weight)
27         nn.init.constant_(self.fc2.bias, 0.0)
28         # 使用 'torch.sigmoid' 定义 Logistic 激活函数
29         self.act_fn = torch.sigmoid
30         # 前向计算
31         def forward(self, inputs):
```

```

32         z1 = self.fc1(inputs)
33         a1 = self.act_fn(z1)
34         z2 = self.fc2(a1)
35         a2 = self.act_fn(z2)
36         return a2

```

4.3.2 完善 Runner 类: RunnerV2_2

第4.2.5节的RunnerV2_1从自定义损失算子的`backward`启动反向传播,并按手写层的参数字典保存模型.切换到PyTorch模型后,这两处接口都要改:调用损失张量的`backward`,并用`state_dict`存取状态.下面实现RunnerV2_2.

新增的四点能力可归纳如下:

- 1) **自定义日志**: 训练循环对外暴露`custom_print_log()`回调,可以打印每层梯度、监控指标等任意自定义信息.
 - 2) **训练/评价切换**: 评价和预测路径加`@torch.no_grad()`避免无谓的梯度计算与显存占用;同时配合`model.train()` / `model.eval()`切换 Dropout、BatchNorm 等算子的两种行为模式.
 - 3) **模型保存**: 通过`state_dict()`一次性收集模块及其子模块的所有参数和持久化缓冲区,统一落盘.
 - 4) **模型加载**: 通过`load_state_dict()`把这些张量回填到目标模型中.
-

```

1 class RunnerV2_2:
2     """PyTorch autograd 版: state_dict 保存、@torch.no_grad 评估. """
3
4     def __init__(self, model, optimizer, metric, loss_fn):
5         self.model = model
6         self.optimizer = optimizer
7         self.loss_fn = loss_fn
8         self.metric = metric
9         self.history = {
10             'train_loss': [], 'dev_loss': [],
11             'train_score': [], 'dev_score': [],
12         }
13
14     def train(self, train_set, dev_set, num_epochs=100, log_epochs=100,
15              save_path='model_best.pt', custom_print_log=None):
16         best_score = -float('inf')
17         X, y = train_set
18         for epoch in range(num_epochs):
19             self.model.train()
20             # 训练一步
21             self.optimizer.zero_grad()

```

```

22         probabilities = self.model(X)
23         trn_loss = self.loss_fn(probabilities, y)
24         trn_score = self.metric(probabilities, y)
25         trn_loss.backward()
26         if custom_print_log is not None:
27             custom_print_log(self.model)
28         self.optimizer.step()
29         self.history['train_loss'].append(trn_loss.item())
30         self.history['train_score'].append(trn_score)
31         # 验证集评估
32         dev_score, dev_loss = self.evaluate(dev_set)
33         self.history['dev_loss'].append(dev_loss)
34         self.history['dev_score'].append(dev_score)
35         # 保存最优
36         if dev_score > best_score:
37             self.save_model(save_path)
38             print(f'[Evaluate] best score updated: {best_score:.5f} -> {dev_score:.5f}')
39             best_score = dev_score
40         if log_epochs and (epoch + 1) % log_epochs == 0:
41             print(f'[Train] epoch {epoch+1}/{num_epochs} loss {trn_loss.item():.4f}')
42
43     @torch.no_grad()
44     def evaluate(self, data_set):
45         """纯查询：切换 eval 模式但不写入历史。"""
46         self.model.eval()
47         X, y = data_set
48         probabilities = self.model(X)
49         loss = self.loss_fn(probabilities, y).item()
50         score = self.metric(probabilities, y)
51         return score, loss
52
53     @torch.no_grad()
54     def predict(self, X):
55         self.model.eval()
56         return self.model(X)
57
58     def save_model(self, save_path):
59         Path(save_path).parent.mkdir(parents=True, exist_ok=True)
60         torch.save(self.model.state_dict(), save_path)
61
62     def load_model(self, save_path):
63         self.model.load_state_dict(torch.load(save_path, weights_only=True))

```

4.3.3 模型训练

实例化RunnerV2_2,采用SGD 优化器(学习率0.2):

```
1 # 设置模型
2 input_size = 2
3 hidden_size = 5
4 output_size = 1
5 torch.manual_seed(0)
6 model = Model_MLP_L2_V2(input_size, hidden_size, output_size)
7 # 设置损失函数
8 loss_fn = F.binary_cross_entropy
9 # model.parameters() 返回参数迭代器, 供优化器管理
10 optimizer = torch.optim.SGD(model.parameters(), lr=0.2)
11 # 设置评价指标
12 metric = accuracy
13 saved_path = 'best_model.pt'
14 # 实例化RunnerV2类, 并传入训练配置
15 runner = RunnerV2_2(model, optimizer, metric, loss_fn)
16 runner.train([X_train, y_train], [X_dev, y_dev], num_epochs=1000, log_epochs=50, save_path
              = "./checkpoint/model_best.pt")
```

把训练过程中损失和准确率的变化分别绘制出来(绘图代码见配套 notebook), 方便整体观察.

训练曲线见图4.6:损失下降平稳,准确率随之逐步收敛.

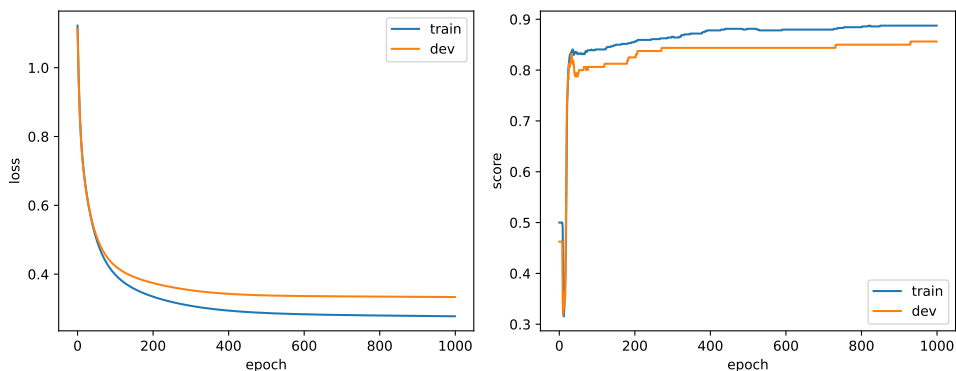


图 4.6 损失和准确率变化趋势

4.3.4 模型评价

载入最优模型,在测试集上做一次最终评价:

```
1 # 模型评价
2 runner.load_model("./checkpoint/model_best.pt")
3 score, loss = runner.evaluate([X_test, y_test])
4 print(f"[Test] score/loss: {score:.4f}/{loss:.4f}")
```

输出结果为:

```
[Test] score/loss: 0.8600/0.3182
```

两种实现描述相同的网络结构,但随机初始化的采样顺序不同,不能仅凭最终准确率判断反向传播是否一致。更直接的核验是把权重转置并逐一复制,使用同一批输入比较输出和参数梯度;本章前面的练习给出了这一办法。

4.4 优化问题

网络一旦深起来,训练就不再是“加几层就好”那么简单。本节安排两组小实验,分别复现**梯度消失**与**死亡 ReLU**这两类深层网络中最常见的失败模式,并讨论实践中的缓解办法。

4.4.1 参数初始化

训练神经网络之前必须先初始化参数。如果把所有权重和偏置都置为 0,前向传播时同一层所有神经元会输出相同的激活值,反向传播时它们也会获得相同的梯度,更新后仍然一致——多个神经元退化成同一个,这就是**对称权重现象**。



动手练习 4.4

将神经网络的权重全部初始化为 0 并训练,观察损失和最终性能与正常初始化的差异。

破解对称权重最直接的办法就是随机初始化,最常用的两种分布是**高斯分布**和**均匀分布**。从这两种分布中采样并绘制直方图,结果如图 4.7 所示。

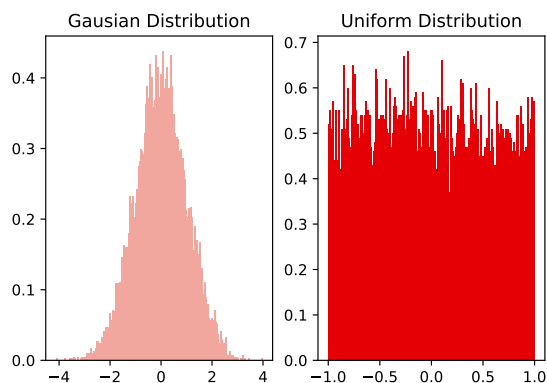


图 4.7 高斯分布和均匀分布



动手练习 4.5

在配套 notebook 中用 `torch.normal` 和 `Tensor.uniform_` 分别从高斯分布和均匀分布中采样, 绘制直方图对比两种分布的形态。

4.4.2 梯度消失问题

增加深度可以更高效地表达某些复合函数, 但这不保证给定网络更容易优化。反向传播需要连续乘上各层的 Jacobian, 激活函数的导数与权重矩阵共同影响梯度尺度。

Sigmoid 的导数至多为 0.25, 进入饱和区后更小。当各层 Jacobian 的连乘不断收缩时, 靠近输入端的梯度可能很小, 参数更新缓慢, 这就是**梯度消失问题**。权重尺度也会放大或缩小这一连乘, 因此不能只根据激活函数名称判断梯度一定如何变化。

一种常用对策是采用 ReLU 等正半轴导数不衰减的激活, 并配合合适的初始化。下面就构造一个对照实验, 先复现 Sigmoid 带来的梯度消失, 再切换到 ReLU 观察改善效果。

4.4.2.1 模型构建

构造一个 5 层前馈网络 (4 个隐藏层 + 1 个输出层), 通过 `act` 参数在 Sigmoid、ReLU、Leaky ReLU 三种激活之间切换:

```
1 # 定义多层前馈神经网络
2 class Model_MLP_L5(nn.Module):
3     def __init__(self, input_size, output_size, act='sigmoid', w_init=Kaiming(), b_init=
4         Constant(value=1.0)):
5         super().__init__()
6         self.fc1 = nn.Linear(input_size, 3)
7         self.fc2 = nn.Linear(3, 3)
8         self.fc3 = nn.Linear(3, 3)
```

```

8         self.fc4 = nn.Linear(3, 3)
9         self.fc5 = nn.Linear(3, output_size)
10        # 定义网络使用的激活函数
11        if act == 'sigmoid':
12            self.act = torch.sigmoid
13        elif act == 'relu':
14            self.act = F.relu
15        elif act == 'lrelu':
16            self.act = F.leaky_relu
17        else:
18            raise ValueError("Please enter sigmoid relu or lrelu!")
19        # 初始化线性层权重和偏置参数
20        self.init_weights(w_init, b_init)
21        # 初始化线性层权重和偏置参数
22        def init_weights(self, w_init, b_init):
23            # 使用 'named_modules' 遍历所有网络层
24            for n, m in self.named_modules():
25                # 如果是线性层, 则使用指定方式进行参数初始化
26                if isinstance(m, nn.Linear):
27                    w_init(m.weight)
28                    b_init(m.bias)
29
30        def forward(self, inputs):
31            outputs = self.fc1(inputs)
32            outputs = self.act(outputs)
33            outputs = self.fc2(outputs)
34            outputs = self.act(outputs)
35            outputs = self.fc3(outputs)
36            outputs = self.act(outputs)
37            outputs = self.fc4(outputs)
38            outputs = self.act(outputs)
39            outputs = self.fc5(outputs)
40            outputs = torch.sigmoid(outputs)
41            return outputs

```

4.4.2.2 使用 Sigmoid 型函数进行训练

实现一个梯度打印工具, 在每次反向传播后输出每层权重梯度的 ℓ_2 范数, 便于跨层对比:

```

1 def print_grads(model):
2     # 打印每一层权重的 L2 范数. 用 isinstance(nn.Linear) 与初始化遍历的判断保持一致,
3     # 避免基于“参数数恰好为 2”这种脆弱条件
4     print('The gradient of the Layers: ')
5     for name, item in model.named_modules():
6         if isinstance(item, nn.Linear) and item.weight.grad is not None:
7             print(name, item.weight.grad.norm(p=2).item())

```

激活函数指定为 Sigmoid, 仅训练 1 个回合——这样不必跑完整训练就能从单步梯度看到衰减规律:

```
1 torch.manual_seed(102)
2 # 定义网络
3 model = Model_MLP_L5(input_size=2, output_size=1, act='sigmoid')
4 # 定义优化器
5 optimizer = torch.optim.SGD(model.parameters(), lr=0.01)
6 # 定义损失函数, 使用交叉熵损失函数
7 loss_fn = F.binary_cross_entropy
8 # 定义评价指标
9 metric = accuracy
10 # 指定梯度打印函数
11 custom_print_log=print_grads
```

把模型、优化器、损失、指标交给 RunnerV2_2:

```
1 # 实例化 RunnerV2_2 类
2 runner = RunnerV2_2(model, optimizer, metric, loss_fn)
```

模型训练, 打印网络每层梯度值的 ℓ_2 范数. 代码实现如下:

```
1 # 模型训练
2 runner.train([X_train, y_train], [X_dev, y_dev],
3             num_epochs=1, log_epochs=None,
4             save_path="./checkpoint/model_best.pt",
5             custom_print_log=custom_print_log)
```

输出结果为:

```
The gradient of the Layers:
fc1 0.001497069257311523
fc2 0.010207749903202057
fc3 0.03565644472837448
fc4 0.22919254004955292
fc5 0.46500515937805176
```

可以看到: 这次使用 He 初始化的 Sigmoid 网络, 靠近输入的权重梯度明显更小——从输出层的约 0.47 降到第一层的约 1.5×10^{-3} , 跨度约 2.5 个数量级. Sigmoid 的饱和导数参与了这一衰减. He 初始化主要针对 ReLU, 并不能排除初始化对 Sigmoid 网络的影响; 这组结果只描述当前数据与参数设置.

4.4.2.3 使用 ReLU 函数进行模型训练

重新把随机种子设为 102，以相同初始权重和数据只更换隐藏层激活函数；输出层仍使用 Sigmoid。配套 Notebook 把两组权重梯度绘在图 4.8 中。当前实验中 ReLU 各层梯度的差异较小，但“正区导数为 1”不能保证整个网络的梯度不消失：权重矩阵、负区门控、层宽和输出层都会参与反向计算。还可进一步比较 Sigmoid 配 Xavier 初始化、ReLU 配 He 初始化的结果。

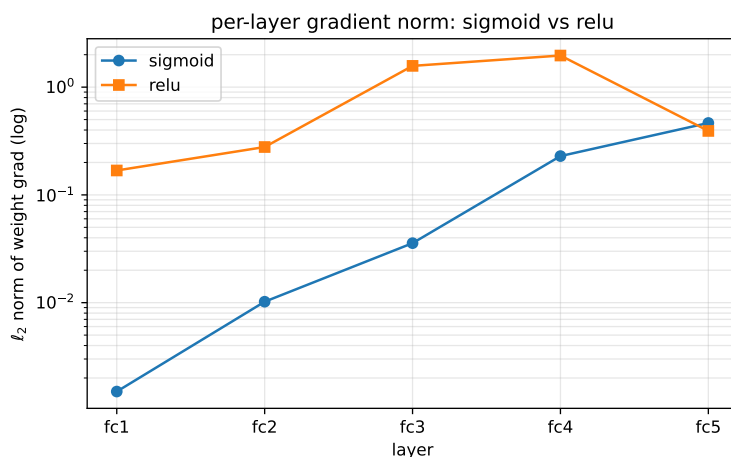


图 4.8 网络每层梯度值的 l_2 范数变化趋势

动手练习 4.6

在本节 5 层网络的梯度消失实验基础上，改动以下变量并各跑一遍单步反向传播，记录并对比每层权重梯度的 l_2 范数：

- 1) 把权重初始化的标准差从 0.01 依次调到 0.1、0.5，观察各层梯度范数随尺度怎样变化。
- 2) 把网络层数从 5 层加深到 8 层、10 层，看梯度衰减是否随深度加剧。
- 3) 把激活函数从 Sigmoid 换成 Tanh，比较两者在相同深度下的梯度衰减程度。

综合三组实验，分析初始化尺度、网络深度、激活函数中哪个因素对梯度消失影响最大，并说说为什么深而窄的网络更容易出现梯度消失。

4.4.3 死亡 ReLU 问题

ReLU 缓解了梯度消失，但它自己也带来一个新风险——**死亡 ReLU 问题**。原因很简单： $x < 0$ 时 ReLU 输出恒为 0、导数也恒为 0。如果某个神经元在**所有训练样本**上的输入都处于负区，经过它的梯度就为 0；在输入特征不变、仅用普通梯度更新入边参数时，它可能持续不被激活。上游特征变化、动量或正则化等因素可能改变这种状态，因此不能仅凭一个批次的零输出判定神经元永久死亡。

工程上的常见对策是把激活函数换成带泄露的 ReLU (Leaky ReLU) 或 ELU 等保留负半轴小斜率的变种,使神经元在负半轴仍有非零梯度,从而能继续更新。

4.4.3.1 使用 ReLU 进行模型训练

复用第4.4.2节的 5 层网络,把激活换成 ReLU,并把偏置故意初始化为一个较大的负数——这样输入经线性变换后大概率落入负半轴,恰好触发死亡 ReLU:

```
1 # 定义网络, 并使用较大的负值来初始化偏置
2 torch.manual_seed(102)
3 model = Model_MLP_L5(input_size=2, output_size=1, act='relu', b_init=Constant(value=-8.0))
4 # 新模型要配一个绑定到它自己参数上的优化器
5 optimizer = torch.optim.SGD(model.parameters(), lr=0.01)
```

把模型交给RunnerV2_2训练一回合,仍打印各层梯度范数:

```
1 # 实例化Runner类
2 runner = RunnerV2_2(model, optimizer, metric, loss_fn)
3 # 模型训练
4 runner.train([X_train, y_train], [X_dev, y_dev],
5             num_epochs=1, log_epochs=None,
6             save_path="./checkpoint/model_best.pt",
7             custom_print_log=custom_print_log)
```

输出结果为:

```
The gradient of the Layers:
fc1 0.0
fc2 0.0
fc3 0.0
fc4 0.0
fc5 0.0
```

这里打印的各层权重梯度都为 0. 隐藏激活被负偏置压成零, 输出层权重也收不到输入信号; 但输出层偏置仍可能有非零梯度. 它可以改变统一的预测概率, 却不能恢复已截断的隐藏特征通路. 可打印model.fc5.bias.grad验证这一点.

下面把激活换成 Leaky ReLU, 看是否能让这些神经元恢复更新.

4.4.3.2 使用带泄露的 ReLU 进行模型训练

仅换激活函数,其他配置与上面完全相同:

```
1 # 重新定义网络, 使用带泄露的ReLU激活函数
2 torch.manual_seed(102)
3 model = Model_MLP_L5(input_size=2, output_size=1, act='lrelu', b_init=Constant(value=-8.0)
4 )
5 # 新模型要配一个绑定到它自己参数上的优化器
```

```

5 optimizer = torch.optim.SGD(model.parameters(), lr=0.01)
6 # 实例化RunnerV2_2类
7 runner = RunnerV2_2(model, optimizer, metric, loss_fn)
8 # 模型训练
9 runner.train([X_train, y_train], [X_dev, y_dev],
10             num_epochs=1, log_epochs=None,
11             save_path="./checkpoint/model_best.pt",
12             custom_print_log=custom_print_log)

```

输出结果为:

```

The gradient of the Layers:
fc1 1.3553411726263676e-08
fc2 1.37136311195718e-07
fc3 1.3334247341845185e-05
fc4 0.0013541706139221787
fc5 0.0685686320066452

```

这组实验中各层权重梯度恢复为非零值,参数重新获得更新信号;这尚不等于网络已经学好。但梯度仍随层数沿反向衰减,这是因为 Leaky ReLU 负半轴斜率默认只有 0.01——在这个负区实验中避免了完全截断,但仍可能很小。可以调整负半轴斜率,并结合初始化重新实验;换成 ELU、GELU 或 Swish 也不保证负区梯度足够大。

动手练习 4.7

沿用本节触发死亡 ReLU 的 5 层网络 (`b_init=Constant(value=-8.0)`、ReLU 激活、单步反向),逐一尝试以下几种把神经元“救活”的手段,每次只改一处,重新跑一遍并记录各层权重梯度的 ℓ_2 范数:

1) 把偏置初始化从 -8 改回 0 或某个正值,观察输入是否重新落回 ReLU 的正半轴。

2) 保持 -8 不变,把学习率 `lr` 调小,检验它是否能改变第一次更新前的零梯度。再从正常初始化开始,用不同学习率训练多步,区分“预防一次大更新造成失活”和“恢复初始化时已为零的梯度”。

3) 换用带泄露的 ReLU (`act='lrelu'`), 把 `F.leaky_relu` 的 `negative_slope` 从默认的 0.01 逐步调大。

4) 给 `Model_MLP_L5` 增设一个 `act='elu'` 分支调用 `F.elu`,再跑一遍。

综合各组结果,总结哪些手段能让原本全零的梯度恢复成非零值、各自付出的代价是什么。



4.5 实践:基于前馈神经网络完成鸢尾花分类任务

本节把第 3.3 节的 Softmax 分类器替换为本章实现的前馈神经网络,再在 Iris 数据集上完成小批量训练。损失取交叉熵,优化器使用自适应梯度方法 Adam,评价指标取准确率。本节的数据划

分、预处理和优化器与上一章不同,不能直接用两个测试分数判断模型孰优;严格比较时应统一这些条件。

4.5.1 小批量梯度下降

经典梯度下降一次性用整个训练集 N 条样本算梯度,称为**批量梯度下降法**(Batch Gradient Descent, BGD)。它的“代价”很直接:每步都要遍历整张训练集,当 N 或参数量很大时既费内存又费时间。另一种极端是每次只用一条样本——**随机梯度下降法**(Stochastic Gradient Descent, SGD),每步处理的样本少,但硬件利用率可能较低。梯度噪声会影响优化轨迹,不能保证避开所有不良驻点。

在两者之间取折衷的,就是**小批量梯度下降**(Mini-Batch Gradient Descent):每步从训练集随机抽取 K 条样本(即一个 mini-batch)算梯度,既能利用 GPU 并行加速、又能保持一定噪声。

第 t 步从训练集中随机抽出 K 条样本 S_t ,用它们的平均梯度更新参数:

$$\theta_{t+1} \leftarrow \theta_t - \alpha \frac{1}{K} \sum_{(\mathbf{x}, y) \in S_t} \frac{\partial \mathcal{L}(y, f(\mathbf{x}; \theta))}{\partial \theta}, \quad (4.23)$$

其中 K 为**批大小**(Batch Size)。 K 的常用范围大致是几十到几百,常从 16、32、64、128 等数值试起,再依据显存、吞吐和验证表现选择。批大小不必是 2 的幂,实际效率还取决于模型维度、精度和硬件。

小批量梯度下降兼顾了计算效率与梯度噪声,是大规模机器学习中最常用的训练方式。完整的训练流程见算法 4.1。

算法 4.1 小批量随机梯度下降法

输入: 训练集 $\mathcal{D} = \{(\mathbf{x}^{(n)}, y^{(n)})\}_{n=1}^N$, 验证集 $\mathcal{V}$, 学习率 α

```

1 随机初始化  $\theta$ ;
2 for  $epoch = 1 \cdots E$  do
3   将训练集  $\mathcal{D}$  打乱后切成大小为  $K$  的 mini-batch 序列  $\mathcal{B}_1, \mathcal{B}_2, \dots, \mathcal{B}_T$ ;
4   for  $iter = 1 \cdots T$  do
5     计算  $\mathcal{B}_{iter}$  上的梯度,并更新参数;
6   end
7 end
输出:  $\theta$ 
```

思考

当样本总数不能被批大小 K 整除时,最后一个小批量将不足 K 个样本,应该如何处理?(可了解 `DataLoader` 的 `drop_last` 参数。)

4.5.1.1 数据分组

小批量梯度下降在每一步都需要拿到“一小撮”样本。手动管理索引既繁琐又容易出错,工程上的标准做法是把这件事交给**数据迭代器**:训练前注册整张数据集,训练时每次迭代自动按指定批大小返回一批数据。数据迭代器的工作流程如图4.9所示。

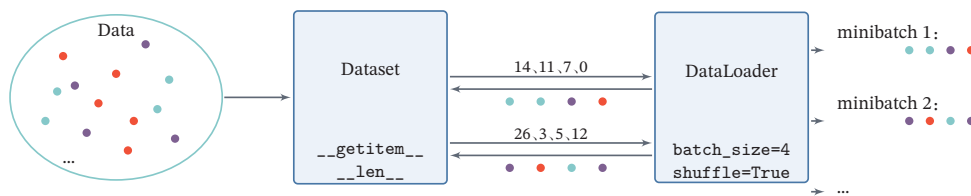


图 4.9 数据迭代器的实现原理

实现上由两类对象协作完成:

1) **Dataset**:负责描述“一条样本怎么取”——实现`__getitem__(idx)`给定索引返回对应样本,`__len__`返回总样本数。

2) **DataLoader**:负责按批迭代——传入**Dataset**和`batch_size`后,自行生成索引列表(`shuffle=True`时还会随机打乱),按`batch`切片返回数据。

PyTorch 提供了数据集基类`torch.utils.data.Dataset`和加载器`torch.utils.data.DataLoader`。常见用法分两档:特征和标签都已经是张量的小数据集,直接套现成的**TensorDataset**即可;样本需要在线读取或额外预处理时,则继承**Dataset**写一个自定义子类。**DataLoader**负责按设定的批大小(可选`shuffle=True`打乱)从**Dataset**中取批。

4.5.2 数据处理

4.5.2.1 加载与划分数据

通过 `scikit-learn` 提供的接口加载鸢尾花数据集,并把特征和标签转换为张量。代码实现如下:

```
1 import torch
2 from sklearn.datasets import load_iris
3
4 X, y = load_iris(return_X_y=True)
5 X = torch.tensor(X, dtype=torch.float32)
6 y = torch.tensor(y, dtype=torch.long)
7 print('X:', X.shape, 'y:', y.shape, 'classes:', y.unique().tolist())
```

输出结果为:

```
X: torch.Size([150, 4]) y: torch.Size([150]) classes: [0, 1, 2]
```

鸢尾花数据集中的样本是按类别顺序排列的,需要先洗牌再按比例切分. 这里按 6 : 2 : 2 的比例划分为训练集、验证集和测试集. 代码实现如下:

```

1 torch.manual_seed(0)
2 perm = torch.randperm(len(X))
3 n_train = int(0.6 * len(X))
4 n_dev = int(0.2 * len(X))
5 tr, dv, te = perm[:n_train], perm[n_train:n_train + n_dev], perm[n_train + n_dev:]
6 X_train, y_train = X[tr], y[tr]
7 X_dev, y_dev = X[dv], y[dv]
8 X_test, y_test = X[te], y[te]
```

这四个特征的单位都是厘米,但数值范围和方差不同. 训练前对各维做标准化,可以改善梯度优化的尺度条件.



提醒

均值和标准差只能在训练集上估计,再用同一组统计量去变换验证集和测试集——否则会造成测试集信息泄露。



```

1 mean, std = X_train.mean(dim=0), X_train.std(dim=0)
2 std = torch.where(std == 0, torch.ones_like(std), std)
3 X_train = (X_train - mean) / std
4 X_dev = (X_dev - mean) / std
5 X_test = (X_test - mean) / std
```

4.5.2.2 用 TensorDataset 和 DataLoader 封装

鸢尾花的特征和标签都是张量, 不需要额外的样本级处理, 可以直接用 PyTorch 内置的 **TensorDataset** 把它们配对成 **Dataset**, 再用 **Dataloader** 做小批量遍历. 训练集需要在每个回合重新打乱顺序, 验证集和测试集则不需要. 代码实现如下:

```

1 from torch.utils.data import TensorDataset, DataLoader
2
3 batch_size = 16
4 train_loader = DataLoader(TensorDataset(X_train, y_train), batch_size=batch_size, shuffle=True)
5 dev_loader = DataLoader(TensorDataset(X_dev, y_dev), batch_size=batch_size)
6 test_loader = DataLoader(TensorDataset(X_test, y_test), batch_size=batch_size)
7 print(f'train {len(X_train)} dev {len(X_dev)} test {len(X_test)}')
```

输出结果为:

```
train 90 dev 30 test 30
```

4.5.3 模型构建

沿用PyTorch自带算子搭一个两层全连接网络做鸢尾花分类:输入4维(对应4个特征)、隐藏层16个神经元、输出层3维(对应3个类别)。



提醒

与第4.3.1节的两层版本相比,这里输出层不再接激活函数,而是直接输出未归一化分数(Logits)。分类决策只需要比较类别得分的相对大小(或后续argmax),并不需要显式概率;同时`nn.CrossEntropyLoss`内部已经做了`log_softmax`,模型再叠一层Softmax会使损失函数把概率误当作Logits,再次进行归一化,从而改变损失及其梯度。

代码实现如下:

```
1 import torch.nn as nn
2 # 定义前馈神经网络
3 class IrisMLP(nn.Module):
4     def __init__(self, in_dim=4, hidden=16, n_class=3):
5         super().__init__()
6         self.net = nn.Sequential(
7             nn.Linear(in_dim, hidden),
8             nn.ReLU(),
9             nn.Linear(hidden, n_class),
10        )
11
12    def forward(self, inputs):
13        return self.net(inputs)
14
15 fnn_model = IrisMLP(in_dim=4, hidden=16, n_class=3)
```

4.5.4 完善 Runner 类:RunnerV3

前面几版Runner假设训练集可以一次全量加载,与真实工程脱节。本节继续升级到RunnerV3,把训练流程改为“小批量+DataLoader”的组织方式。相比RunnerV2,主要新增三点能力:

- 1) **数据分批加载**:使用数据迭代器DataLoader管理数据,每个回合内部按小批量遍历。
- 2) **评价指标解耦**:用`metric_fn(logits, y)`传入准确率等可按样本累积的评价指标,并用`higher_is_better`说明指标越大越好还是越小越好。
- 3) **最优模型保存**: 训练过程中实时比较验证集指标, 更优时把模型`state_dict`写到`best_path`。



笔记

这里展示的是RunnerV3的核心教学版。配套代码仓库nndl.runner中提供了功能更完整的工程化版本，与教学版的接口和训练主线一致，额外支持指定device、可选dev_loader、梯度截断grad_clip_norm、多输入batch等。后续章节可以直接from nndl.runner import RunnerV3复用工程化版本，不必重复粘贴这一段代码。

4.5.4.1 RunnerV3类

RunnerV3类的代码实现如下：

```

1  from pathlib import Path
2  import torch
3
4  class RunnerV3:
5      """小批量训练框架: DataLoader + state_dict 保存最优模型.
6
7      - model: nn.Module 子类
8      - optimizer: torch.optim.Optimizer
9      - loss_fn: 例如 nn.CrossEntropyLoss()
10     - metric_fn(out, y) -> float: 标量评价指标, 缺省时以 dev_loss 作 metric
11     - higher_is_better: metric 越大越好 (如准确率) 还是越小越好 (如 MAE)
12     - device: 若给定, 构造时把 model 搬到该 device, fit/_eval/predict 自动把
13       batch 张量也搬过去; 默认 None 表示不接管 device
14     """
15
16     def __init__(self, model, optimizer, loss_fn, metric_fn=None,
17                  higher_is_better=True, device=None):
18         self.device = torch.device(device) if device is not None else None
19         if self.device is not None:
20             model = model.to(self.device)
21         self.model = model
22         self.optimizer = optimizer
23         self.loss_fn = loss_fn
24         self.metric_fn = metric_fn
25         self.higher_is_better = higher_is_better if metric_fn is not None else False
26         self.history = {'train_loss': [], 'dev_loss': [], 'dev_metric': [], 'lr': []}
27
28     def _to_device(self, batch):
29         if self.device is None:
30             return batch
31         return tuple(t.to(self.device) if torch.is_tensor(t) else t for t in batch)
32
33     def fit(self, train_loader, dev_loader=None, num_epochs=100, log_every=10,
34            best_path=None, grad_clip_norm=None, lr_scheduler=None,

```

```

35         patience=None, seed=None):
36     """训练入口.
37
38     - dev_loader=None: 仅跟踪 train_loss, 跳过 dev 评估与 best-checkpoint
39     - log_every=None: 完全不打印 epoch 日志
40     - grad_clip_norm: 若非 None, 在 backward 与 step 之间按模截断梯度
41     - lr_scheduler: 每个 epoch 末调一次 .step()
42     - patience: 连续 N 轮无改善则提前停止 (仅当 dev_loader 提供时生效)
43     - seed: 固定 torch 随机数种子, 便于复现
44
45     train_loader 的每个 batch 解包为 *inputs, y, 模型按 model(*inputs) 调用——
46     既支持普通 (x, y), 也支持变长序列的 (padded, lengths, y) 等多输入约定.
47     """
48     if seed is not None:
49         torch.manual_seed(seed)
50     best = -float('inf') if self.higher_is_better else float('inf')
51     no_improve = 0
52     for epoch in range(num_epochs):
53         self.model.train()
54         running, n = 0.0, 0
55         for batch in train_loader:
56             *inputs, y = self._to_device(batch)
57             self.optimizer.zero_grad()
58             loss = self.loss_fn(self.model(*inputs), y)
59             loss.backward()
60             if grad_clip_norm is not None:
61                 torch.nn.utils.clip_grad_norm_(self.model.parameters(), grad_clip_norm)
62             self.optimizer.step()
63             bs = inputs[0].size(0)
64             running += loss.item() * bs
65             n += bs
66         train_loss = running / n
67         self.history['train_loss'].append(train_loss)
68         self.history['lr'].append(self.optimizer.param_groups[0]['lr'])
69         if lr_scheduler is not None:
70             lr_scheduler.step()
71
72         should_log = log_every is not None and (epoch + 1) % log_every == 0
73         if dev_loader is not None:
74             dev_loss, dev_metric = self._eval(dev_loader)
75             self.history['dev_loss'].append(dev_loss)
76             self.history['dev_metric'].append(dev_metric)
77             improved = (dev_metric > best) if self.higher_is_better else (dev_metric <
78                 best)
79             if improved:

```

```

79         best = dev_metric
80         no_improve = 0
81         if best_path is not None:
82             Path(best_path).parent.mkdir(parents=True, exist_ok=True)
83             torch.save(self.model.state_dict(), best_path)
84         else:
85             no_improve += 1
86         if should_log:
87             tag = ' *' if improved else ''
88             print(f'epoch {epoch+1:4d} train_loss={train_loss:.4f} '
89                   f'dev_loss={dev_loss:.4f} dev_metric={dev_metric:.4f}{tag}')
90         if patience is not None and no_improve >= patience:
91             if should_log or log_every is not None:
92                 print(f'early stop at epoch {epoch+1} (no improvement for {patience}
93                       epochs)')
94             break
95         elif should_log:
96             print(f'epoch {epoch+1:4d} train_loss={train_loss:.4f}')
97     @torch.no_grad()
98     def _eval(self, loader):
99         self.model.eval()
100         total_loss, total_m, n = 0.0, 0.0, 0
101         for batch in loader:
102             *inputs, y = self._to_device(batch)
103             out = self.model(*inputs)
104             bs = inputs[0].size(0)
105             total_loss += self.loss_fn(out, y).item() * bs
106             if self.metric_fn is not None:
107                 total_m += self.metric_fn(out, y) * bs
108             n += bs
109         dev_loss = total_loss / n
110         dev_metric = (total_m / n) if self.metric_fn is not None else dev_loss
111         return dev_loss, dev_metric
112
113     def evaluate(self, loader):
114         return self._eval(loader)
115
116     @torch.no_grad()
117     def predict(self, *inputs):
118         """与 fit 多输入约定对齐; 普通用法 runner.predict(x) 仍然成立. """
119         self.model.eval()
120         if self.device is not None:
121             inputs = tuple(t.to(self.device) if torch.is_tensor(t) else t for t in inputs)
122         return self.model(*inputs)

```



```

123
124     def save(self, path):
125         Path(path).parent.mkdir(parents=True, exist_ok=True)
126         torch.save(self.model.state_dict(), path)
127
128     def load(self, path):
129         state = torch.load(path, weights_only=True, map_location=self.device)
130         self.model.load_state_dict(state)

```

笔记



这个 Runner 假定损失返回批内样本平均值,评价指标也能按样本数加权汇总。准确率和普通平均损失符合这一条件; F1、AUC 等指标不能简单平均各批结果,应先在完整数据集上汇集预测或累计所需统计量。若未传入 `metric_fn`, 则按验证损失越小越好选择快照。训练损失还是各次参数更新时的批损失平均,验证损失则使用轮末的同一模型,二者的计算时点不同。



4.5.5 模型训练

实例化 `RunnerV3`, 优化器选 Adam (学习率 0.01), 用训练集更新参数、用验证集选择快照, 共训练 150 轮, 自动保留验证集准确率最高的模型快照。

提醒



损失函数取 `nn.CrossEntropyLoss`, 它能直接接收未归一化分数 (Logits) 作为输入: 内部先 `log_softmax` 再算负对数似然, 等价于 “LogSoftmax + NLL-Loss”, 并避免先计算 Softmax 再取对数带来的数值问题。因此模型最后一层不必再叠 Softmax。



代码如下:

```

1  import torch.optim as optim
2  from nndl import accuracy
3
4  torch.manual_seed(0)
5  model = IrisMLP(in_dim=4, hidden=16, n_class=3)
6  runner = RunnerV3(
7      model,
8      optim.Adam(model.parameters(), lr=0.01),
9      nn.CrossEntropyLoss(),
10     metric_fn=accuracy,
11     higher_is_better=True,
12 )
13 runner.fit(train_loader, dev_loader,

```

```
14         num_epochs=150, log_every=20,  
15         best_path='./models/iris_best.pt')
```

把训练损失、验证损失和验证准确率分别绘出来(绘图代码见配套 notebook),整体观察收敛过程。



提醒

这类“画损失+准确率”的样板代码在每个实验中都大同小异,后续章节默认不再重复,只保留与具体模型相关的关键曲线。

可视化结果见图4.10:两条损失曲线一同下降,验证集准确率随之稳步上升,后期趋于平稳。

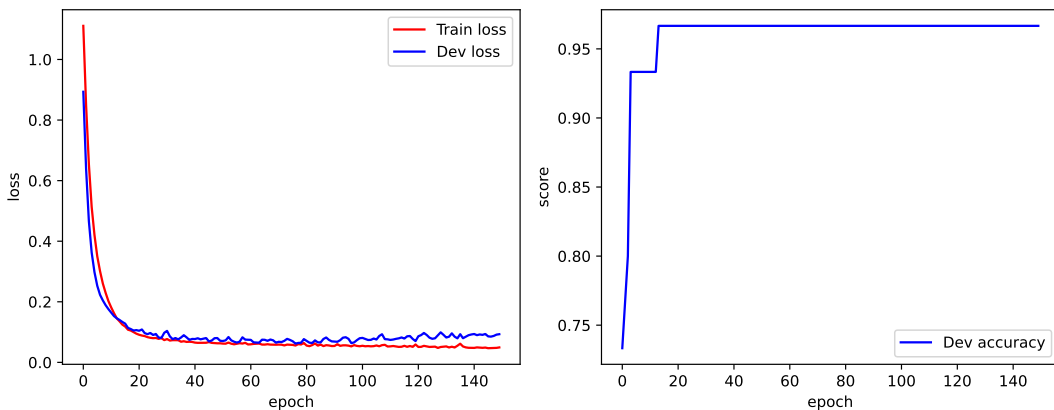


图 4.10 损失和准确率变化趋势

4.5.6 模型评价

载入训练阶段保存的最优快照,在测试集上做最终评价:

```
1 # 加载最优模型  
2 runner.load('./models/iris_best.pt')  
3 # 模型评价  
4 test_loss, test_acc = runner.evaluate(test_loader)  
5 print(f"[test] accuracy/loss: {test_acc:.4f}/{test_loss:.4f}")
```

4.5.7 模型预测

最后用同一份模型对测试集的单条样本做预测,验证端到端推断是否走得通:

```
1 # 获取测试集中第一条数据
```

```
2 X, label = next(iter(test_loader))
3 logits = runner.predict(X)
4
5 pred_class = torch.argmax(logits[0]).item()
6 label = label[0].item()
7 # 输出真实类别与预测类别
8 print(f"The true category is {label} and the predicted category is {pred_class}")
```



动手练习 4.8

迁移本节的训练管道到 MNIST 手写体识别: 设计一个合适层数和宽度的前馈神经网络, 在验证集上确定结构和超参数, 再报告独立测试集的准确率, 并分析主要错误类别。



4.6 小结

本章顺着“神经元—层—网络—训练”的路径, 把前馈神经网络从最小单元一路搭到完整训练流程. 先在Op框架下手写线性层、Logistic 算子与交叉熵损失, 组装出最简的两层网络; 再按误差反向传播逐算子补上 `backward` 方法, 把梯度回传链路打通; 与之配套, `Runner` 也升级到 `RunnerV2_1` 和 `RunnerV2_2`, 分别承载多算子模型的反向计算与 PyTorch 原生的存取流程.

随后我们用两组对照实验让常见的训练陷阱“看得见”: 在 5 层 Sigmoid 网络上复现梯度消失, 在偏置初始化偏负的 ReLU 网络上复现死亡 ReLU, 并通过更换激活函数演示对应的缓解思路.

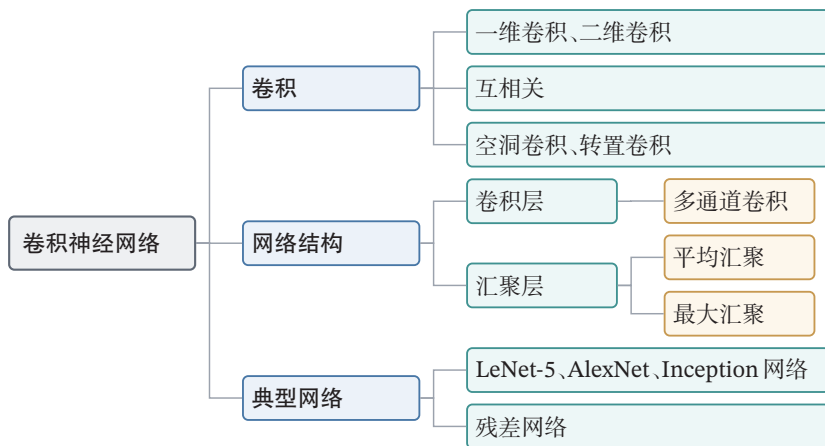
案例实践部分把训练框架推到 `RunnerV3`, 对齐工程惯例的“小批量 + `DataLoader`”写法, 并以鸢尾花分类完成一次端到端的训练—评价—预测闭环.

第5章 卷积神经网络

把手写数字“3”向右挪几个像素,它仍然是“3”.但若把整幅图展平成向量,位移就会改变许多输入坐标,前馈网络还得重新学习这些位置之间的关系. **卷积神经网络** (Convolutional Neural Network, CNN) 采用另一种组织方式:用同一组小窗口在不同位置寻找局部模式,再逐层组合. 这对应卷积层的两个特点:**局部连接**和**权重共享**.

这种结构利用了图像的空间关系,也减少了参数量.忽略边界效应时,步长为1的卷积具有平移等变性:输入平移,输出特征也相应平移.但等变不等于分类结果不变,缩放、旋转和下采样还会带来新的问题.本章先观察一个固定卷积核能检测什么,再让网络从手写数字和自然图像中学习卷积核.

本章对应《神经网络与深度学习》第5章**卷积神经网络**.建议先扫一眼图5.1中的关键知识点,便于后续概念与代码逐一对应.



本章分为两部分:

- **模型解读**:依次介绍卷积运算、卷积神经网络的基本结构、残差连接与残差网络,并用一个简单 CNN 与一个浅层残差网络完成手写体数字识别.

- **案例实践**: 基于 ResNet18 完成 CIFAR-10 图像分类, 体验残差网络在更复杂数据集上的表现。

5.1 卷积

卷积是分析数学中常见的一种运算, 长期服务于信号处理与图像处理。本节以二维卷积为切入点讲解 CNN 中的卷积操作。

5.1.1 二维卷积运算

在 CNN 中, 把卷积核沿图像滑动, 并在每个位置对重叠元素相乘求和, 得到一张新的特征图。这个运算严格来说是**互相关** (Cross-Correlation); 数学卷积还需要先翻转卷积核。对于不受额外约束、可自由学习的卷积核, 翻转只是参数的重新排列, 不改变模型可以表示的函数集合。因此, 深度学习框架通常直接实现互相关。若使用预先给定的滤波器, 两种运算的输出可能不同, 不能随意省略翻转。



提醒

除非特别强调, 本书后文中的“卷积”都默认指互相关。



对于一个输入矩阵 $\mathbf{X} \in \mathbb{R}^{M \times N}$ 和一个滤波器 $\mathbf{W} \in \mathbb{R}^{U \times V}$, 它们的卷积为

$$y_{ij} = \sum_{u=0}^{U-1} \sum_{v=0}^{V-1} w_{uv} x_{i+u, j+v}. \quad (5.1)$$

图 5.2 给出了卷积计算的示例。

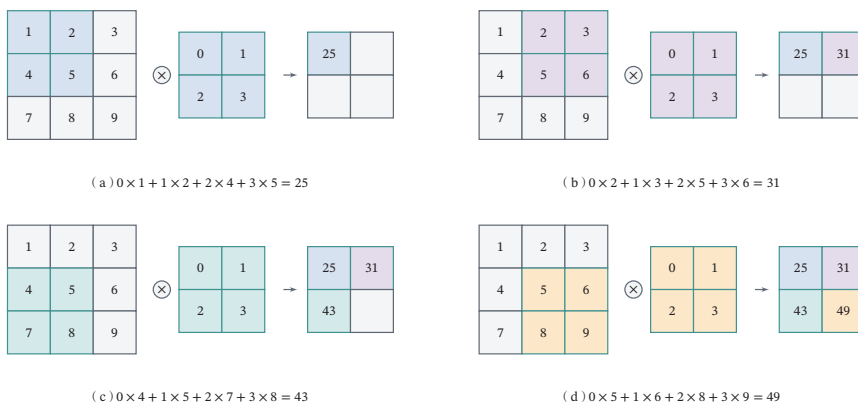


图 5.2 卷积操作的计算过程



提醒

本节二维卷积公式的空间下标从 0 开始, 与 NumPy 和 PyTorch 的索引对应; 后文按通道求和的数学公式仍可从 1 开始计数。对照代码时, 应先确认下标所指的轴及起点。

5.1.2 二维卷积算子

按公式(5.1)写一个最朴素的二维卷积算子:



笔记

本章的算子继承 `torch.nn.Module`, 内部使用支持自动微分的 PyTorch 运算, 因此可以直接由自动微分计算梯度, 不必为每个算子编写 `backward()`。

```

1  import torch
2  import torch.nn as nn
3
4  class Conv2d(nn.Module):
5      def __init__(self, kernel_size):
6          super().__init__()
7          self.weight = nn.Parameter(torch.randn(kernel_size, kernel_size, dtype=torch.
              float32))
8
9      def forward(self, X):
10         """
11         输入:
12             - X: 输入矩阵, size=[B, M, N], B是样本数量
13         输出:
14             - output: 输出矩阵
15         """
16         u, v = self.weight.shape
17         output = X.new_zeros([X.shape[0], X.shape[1] - u + 1, X.shape[2] - v + 1])
18         for i in range(output.shape[1]):
19             for j in range(output.shape[2]):
20                 output[:, i, j] = torch.sum(X[:, i:i+u, j:j+v]*self.weight, dim=(1, 2))
21         return output
22
23 #随机构造一个二维输入矩阵
24 torch.manual_seed(100)
25 inputs = torch.randn(size=[2, 3, 3])
26 conv2d = Conv2d(kernel_size=2)
27 outputs = conv2d(inputs)
28 print(f"input: {inputs}, \noutput: {outputs}")

```

输出结果为:

```
input: tensor([[[[ 0.1268, 1.3564, -0.0247],
                  [-0.8466, 0.0293, -0.5721],
                  [-1.2546, 0.0486, 0.2753]],

                [[-2.1550, -0.7116, 0.0575],
                  [ 0.6263, -1.7736, -0.2205],
                  [ 2.7467, -1.0480, 1.1239]]]],

output: tensor([[[[ 1.7117, 0.9436],
                  [-0.5147, -0.4481]],

                [[-3.7472, -0.4041],
                  [-2.2892, -1.1388]]]], grad_fn=<CopySlices>)
```

5.1.3 卷积的变种

在标准卷积之外,还可以借助**步长**和**零填充**两类设置来调整输出尺寸,使特征提取更灵活.

5.1.3.1 步长

特征图越大,后续计算通常越多.可以在两个空间方向上每隔 S 个元素计算一次卷积,这相当于对步长为 1 的输出做**下采样**(Downsampling). S 称为**步长**(Stride). 尺寸缩小会丢失部分空间信息,也可能使模型对微小位移更敏感;步长应根据需要保留的细节来选择.

此时,对于一个输入矩阵 $\mathbf{X} \in \mathbb{R}^{M \times N}$ 和一个滤波器 $\mathbf{W} \in \mathbb{R}^{U \times V}$,它们的卷积为

$$y_{ij} = \sum_{u=0}^{U-1} \sum_{v=0}^{V-1} w_{uv} x_{i \times S + u, j \times S + v}. \quad (5.2)$$

思考



给定输入 $\mathbf{X} \in \mathbb{R}^{M \times N}$ 和卷积核 $\mathbf{W} \in \mathbb{R}^{U \times V}$,按步长 S 做二维卷积时,输出特征图的高、宽分别是多少?请推导通项公式,并在 $M = N = 7$ 、 $U = V = 3$ 、 $S = 2$ 时验证。

$S = 2$ 时的卷积过程如图 5.3 所示.

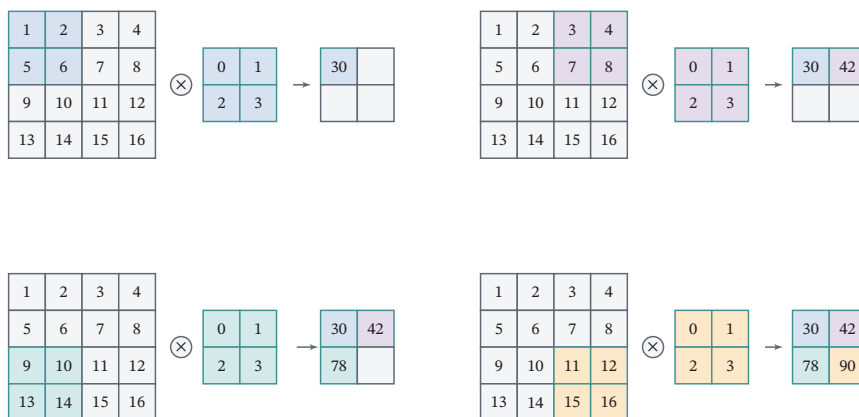


图 5.3 步长为 2 的二维卷积计算过程

5.1.3.2 零填充

当不填充、步长为 1 时, 大于 1×1 的卷积核会使相应方向的输出变小. 连续堆叠若干层后, 边缘位置参与计算的次数也少于中心位置. **零填充**可以缓解尺寸收缩, 但补入的 0 并非真实观测, 仍会影响边界附近的输出.

零填充 (Zero Padding) 就是在输入矩阵周围补若干圈值为 0 的元素.

图 5.4 给出 $P = 1$ 的示例.

0	0	0	0	0	0
0	1	2	3	4	0
0	5	6	7	8	0
0	9	10	11	12	0
0	13	14	15	16	0
0	0	0	0	0	0

图 5.4 $P = 1$ 的零填充

对输入 $\mathbf{X} \in \mathbb{R}^{M \times N}$ 、卷积核 $\mathbf{W} \in \mathbb{R}^{U \times V}$, 步长为 S 、零填充宽度为 P 时, 输出特征图的尺寸由下式给出:

$$M' = \left\lfloor \frac{M + 2P - U}{S} \right\rfloor + 1, \quad (5.3)$$

$$N' = \left\lfloor \frac{N + 2P - V}{S} \right\rfloor + 1. \quad (5.4)$$



动手练习 5.1

查阅资料给出感受野 (Receptive Field) 的定义, 并结合本节的步长、零填充与输出尺寸公式, 完成下面几问:

- 1) 推导一条规则: 当若干卷积层 (步长均为 1) 依次堆叠时, 深层单个神经元在原始输入上的感受野如何随卷积核大小逐层累积。
- 2) 计算本章 Model_LeNet 中 C5 层单个神经元在原始 32×32 输入上的感受野大小, 并说明它与 C5 输出特征图恰好为 1×1 之间的联系。
- 3) 尝试用两层 3×3 卷积替换一层 5×5 卷积, 比较二者覆盖的感受野是否相同; 再在相同输入、输出通道数下估算两种方案的卷积核参数量, 分析“小核堆叠”相对“大核单层”在感受野与参数量上的取舍。

5.1.4 带步长和零填充的二维卷积算子

把步长和零填充统一加进来, 得到的卷积算子如下:

```
1 class Conv2d(nn.Module):
2     def __init__(self, kernel_size, stride=1, padding=0):
3         super().__init__()
4         self.weight = nn.Parameter(torch.randn(kernel_size, kernel_size, dtype=torch.
5             float32))
6         # 步长
7         self.stride = stride
8         # 零填充
9         self.padding = padding
10
11     def forward(self, X):
12         # 零填充
13         new_X = X.new_zeros([X.shape[0], X.shape[1]+2*self.padding, X.shape[2]+2*self.
14             padding])
15         new_X[:, self.padding:X.shape[1]+self.padding, self.padding:X.shape[2]+self.padding
16             ] = X
17         u, v = self.weight.shape
18         output_w = (new_X.shape[1] - u) // self.stride + 1
19         output_h = (new_X.shape[2] - v) // self.stride + 1
20         output = X.new_zeros([X.shape[0], output_w, output_h])
21         for i in range(0, output.shape[1]):
22             for j in range(0, output.shape[2]):
23                 output[:, i, j] = torch.sum(new_X[:, self.stride*i:self.stride*i+u, self.
24                     stride*j:self.stride*j+v]*self.weight, dim=(1, 2))
25         return output
26
27 inputs = torch.randn(size=[2, 8, 8])
28 conv2d_padding = Conv2d(kernel_size=3, padding=1)
```

```

25 outputs = conv2d_padding(inputs)
26 print(f"When kernel_size=3, padding=1 stride=1, input's shape: {inputs.shape}, output's
    shape: {outputs.shape}")
27 conv2d_stride = Conv2d(kernel_size=3, stride=2, padding=1)
28 outputs = conv2d_stride(inputs)
29 print(f"When kernel_size=3, padding=1 stride=2, input's shape: {inputs.shape}, output's
    shape: {outputs.shape}")

```

输出结果为:

```

When kernel_size=3, padding=1 stride=1, input's shape: torch.Size([2, 8, 8]), output's
shape: torch.Size([2, 8, 8])
When kernel_size=3, padding=1 stride=2, input's shape: torch.Size([2, 8, 8]), output's
shape: torch.Size([2, 4, 4])

```

输出尺寸符合预期: 3×3 卷积、padding=1 时, stride=1 保持输入尺寸不变; stride=2 在本例的偶数尺寸输入上, 将宽高各减半; 奇数尺寸应按带向下取整的公式计算。

5.1.5 使用卷积运算完成图像边缘检测任务

在图像处理中, 可以用离散拉普拉斯滤波器突出局部灰度变化。本例采用一种 3×3 近似: 中心为 8, 其余位置为 -1。均匀区域的响应为 0, 轮廓附近常有较大响应; 它也会放大噪声, 因此边缘响应并不等同于物体边界。

下面把上一节的 `Conv2d` 权重设为拉普拉斯算子, 对一张灰度图做边缘检测:

```

1 from PIL import Image
2 import numpy as np
3 from pathlib import Path
4 from torchvision import datasets
5 # 使用公开数据集中的第一张灰度图, 避免依赖未提供的 number.jpg
6 CACHE = str(Path.home() / '.cache' / 'torch_data')
7 mnist_image = datasets.MNIST(CACHE, train=True, download=True)
8 img, image_label = mnist_image[0]
9 # 设置卷积核参数 (拉普拉斯算子)
10 w = torch.tensor([[[-1, -1, -1], [-1, 8, -1], [-1, -1, -1]], dtype=torch.float32)
11 # 创建卷积算子, 卷积核大小为  $3 \times 3$ , 并把上面的拉普拉斯核写入权重
12 conv = Conv2d(kernel_size=3, stride=1, padding=0)
13 with torch.no_grad():
14     conv.weight.copy_(w)
15 # 将读入的图像转化为 float32 类型的 numpy.ndarray
16 inputs = np.array(img.convert('L'), dtype=np.float32) / 255.0
17 # 将图像转为张量
18 inputs = torch.tensor(inputs)
19 inputs = torch.unsqueeze(inputs, dim=0)

```

```
20 outputs = conv(inputs)
```

把原图与输出特征图并排画出（绘图代码见配套 notebook），得到图 5.5。数字笔画内部比较平坦，笔画两侧的反应更明显。输出有正有负，绘图时使用以 0 为中心的色标，才能区分两侧的变化方向。

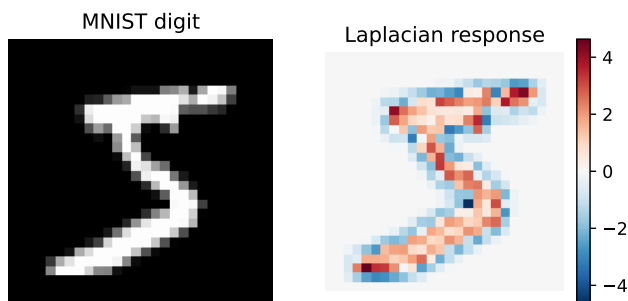


图 5.5 利用拉普拉斯算子来提取图像特征

动手练习 5.2

参考《神经网络与深度学习》图 5.3，尝试不同卷积核（如 Sobel、高斯模糊、锐化等）作用于同一张图像，比较它们在边缘、平滑度、纹理上提取出的特征差异。

5.2 卷积神经网络的基础算子

一种经典的 CNN 结构见图 5.6：先由若干卷积层与汇聚层交替堆叠负责特征抽取，再接上若干全连接层完成分类或回归。

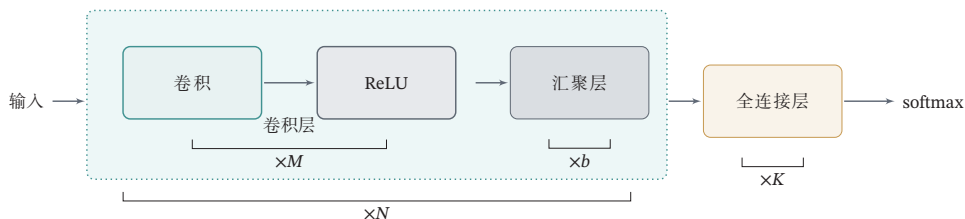


图 5.6 卷积神经网络的典型结构

搭建图中的分类网络，需要卷积、汇聚、线性变换和非线性激活。本节实现两个新组件：**卷积层算子**与**汇聚层算子**。后面还会看到，空间下采样也可以由带步长的卷积完成，分类前则可以用全局平均汇聚减少参数；CNN 并不只有一种固定的排列。

5.2.1 卷积层算子

卷积层就是用卷积操作实现的神经网络层。为了同时提取多种特征，一层中通常会并行使用多个不同的卷积核。

5.2.1.1 多通道卷积

前面定义的二维卷积默认输入是单张矩阵，可现实里像素并不是单一标量： $M \times N$ 彩色图像的每个像素是 R、G、B 三个分量，更一般情形下还可能是 D 维向量。这就把输入提升为三维张量：空间维度 $M \times N$ 之外加一个深度 D ，深度也称**输入通道数**——灰度图为 1、彩色图为 3、上一层输出特征图层数越多则越大。

输出端同样不止一张图。单个卷积核只能产出一张 $M' \times N'$ 的**特征图** (Feature Map)，对应一种特征模式；要在一层内并行提取多种模式，就需要安排 P 组卷积核，最终堆出 P 张特征图，这里的 P 称为**输出通道数**。



提醒

《神经网络与深度学习》将 Feature Map 译为“特征映射”，本书统一使用更口语化的“特征图”。



设输入特征图 $\mathbf{X} \in \mathbb{R}^{D \times M \times N}$ ，其中 (M, N) 是空间尺寸、 D 是通道数；卷积核 $\mathbf{W} \in \mathbb{R}^{P \times D \times U \times V}$ ，其中 (U, V) 是卷积核空间尺寸、 D 是输入通道数、 P 是输出通道数。



提醒

为了对齐主流深度学习框架的张量布局，本书把特征图写成 $D \times M \times N$ ，与《神经网络与深度学习》主书使用的 $M \times N \times D$ 顺序不同；卷积核也相应地写成 $P \times D \times U \times V$ 。



多通道卷积的实现可以拆成下面四步：

(1) **单通道特征图的卷积**。对二维输入 $\mathbf{X} \in \mathbb{R}^{M \times N}$ 与卷积核 $\mathbf{W} \in \mathbb{R}^{U \times V}$ 做基础卷积，输出仍是二维特征图 $\mathbf{Z} = \mathbf{W} \otimes \mathbf{X} \in \mathbb{R}^{M' \times N'}$ ，并支持步长与零填充。这一步对应上一节实现的 Conv2d。

(2) **多输入通道到单输出通道的卷积**。给 D 通道输入 $\mathbf{X} \in \mathbb{R}^{D \times M \times N}$ 配一个三维卷积核 $\mathbf{W} \in \mathbb{R}^{D \times U \times V}$ ，对各通道 $\mathbf{X}^{(d)}$ 与对应 $\mathbf{W}^{(d)}$ 分别做二维卷积，把 D 个结果按位相加得到一张单通道输出 $\mathbf{Z}$ ：

$$\mathbf{Z} = \sum_{d=1}^D \mathbf{W}^{(d)} \otimes \mathbf{X}^{(d)} + b \in \mathbb{R}^{M' \times N'}, \quad (5.5)$$

其中 b 是单一可学习偏置。该过程的示意图见图 5.7。

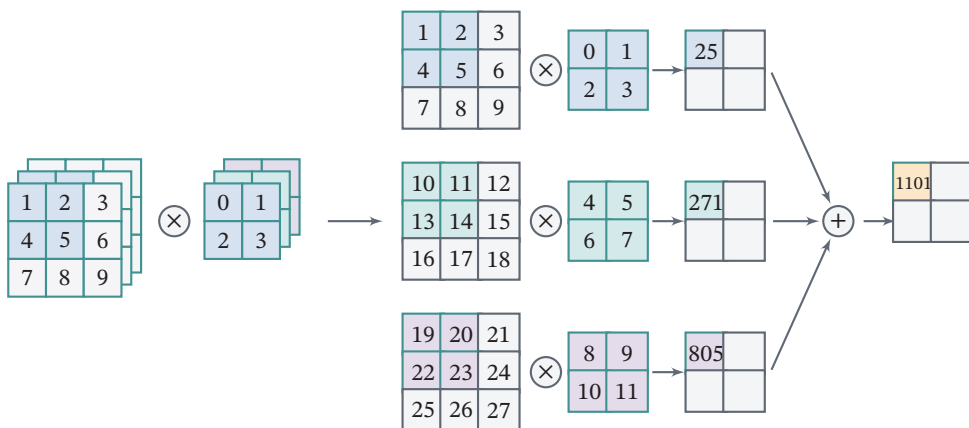


图 5.7 多输入通道到单输出通道的卷积运算

(3) 多输入通道到多输出通道的卷积. 把第(2)步重复 P 次: 用 P 组三维卷积核

$$\mathbf{W}^{(1)}, \dots, \mathbf{W}^{(P)} \in \mathbb{R}^{D \times U \times V}$$

分别对 $\mathbf{X}$ 做“多输入通道到单输出通道”的卷积, 得到 P 张单通道输出, 再按通道堆叠成 $\mathbf{Z} \in \mathbb{R}^{P \times M' \times N'}$. 整套流程见图 5.8.

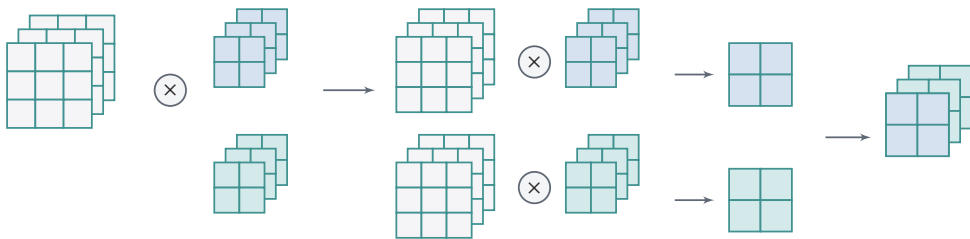


图 5.8 多输入通道到多输出通道的卷积运算

(4) 非线性激活函数. 对多通道输出 $\mathbf{Z}$ 逐元素施加非线性激活 $f(\cdot)$:

$$\mathbf{Y} = f(\mathbf{Z}) \in \mathbb{R}^{P \times M' \times N'}, \quad (5.6)$$

其中 $f(\cdot)$ 是一个逐元素的非线性函数.



笔记

工程实现中, 激活函数通常单独作为一个算子, 不放在卷积层内部, 便于卷积层与激活函数自由组合.

多通道卷积层的实现 把上述步骤合并到一个算子里. **注意:** 下面的实现采用 Python 嵌套循环逐位置滑窗, 目的是让读者看清四步分解的对应关系, 大量细粒度运算会使它远慢于框架实

现, 具体差距取决于输入形状和设备. 实际工程中应直接调用`torch.nn.Conv2d`或`torch.nn.functional.conv2d`——它们会调用针对设备和输入形状优化的底层实现, 可一次性处理整批多通道输入; `im2col + GEMM` 是常见实现思路之一.

```

1  class Conv2d(nn.Module):
2      def __init__(self, in_channels, out_channels, kernel_size, stride=1, padding=0):
3          super().__init__()
4          # 创建卷积核
5          self.weight = nn.Parameter(torch.randn(out_channels, in_channels, kernel_size,
6                                                    kernel_size, dtype=torch.float32))
7          # 创建偏置
8          self.bias = nn.Parameter(torch.zeros(out_channels, 1, dtype=torch.float32))
9          self.stride = stride
10         self.padding = padding
11         # 输入通道数
12         self.in_channels = in_channels
13         # 输出通道数
14         self.out_channels = out_channels
15         # 单通道特征图的卷积
16         def single_forward(self, X, weight):
17             # 零填充: 用 X.new_zeros 让新张量自动继承 X 的设备/精度, 避免显式串 device/dtype
18             new_X = X.new_zeros(X.shape[0], X.shape[1] + 2 * self.padding, X.shape[2] + 2 *
19                                   self.padding)
20             new_X[:, self.padding:X.shape[1]+self.padding, self.padding:X.shape[2]+self.padding
21                     ] = X
22             u, v = weight.shape
23             output_w = (new_X.shape[1] - u) // self.stride + 1
24             output_h = (new_X.shape[2] - v) // self.stride + 1
25             output = X.new_zeros(X.shape[0], output_w, output_h)
26             for i in range(0, output.shape[1]):
27                 for j in range(0, output.shape[2]):
28                     output[:, i, j] = torch.sum(new_X[:, self.stride*i:self.stride*i+u, self.
29                                                       stride*j:self.stride*j+v]*weight, dim=(1, 2))
30             return output
31         # 多输入通道到单输出通道的卷积运算
32         def multi2single_forward(self, inputs, weight, b):
33             """
34             输入:
35             - inputs: 输入矩阵, size=[B, D, M, N]
36             - weight: 一组二维卷积核, size=[D, U, V]
37             - b: 偏置, size=[1]
38             """
39             return torch.sum(torch.stack([self.single_forward(inputs[:, i, :, :], weight[i]) for i
40                                     in range(self.in_channels)], dim=1), dim=1) + b
41         # 多输入通道到多输出通道的卷积运算

```

```

37     def multi2multi_forward(self, inputs, weights, bias):
38         """
39         输入:
40         - inputs: 输入矩阵, size=[B, D, M, N]
41         - weights: P组二维卷积核, size=[P, D, U, V]
42         - bias: P个偏置, size=[P, 1]
43         """
44         # 进行多次多输入通道卷积运算, 并将结果进行堆叠
45         return torch.stack([self.multi2single_forward(inputs, w, b) for w, b in zip(weights
46                                     , bias)], dim=1)
47     # 完整运算
48     def forward(self, inputs):
49         return self.multi2multi_forward(inputs, self.weight, self.bias)

```

动手练习 5.3



把本节从零实现的多通道`Conv2d`与`torch.nn.Conv2d`在同一组随机权重、同一份输入上对比: 先把两者的卷积核权重和偏置设成完全相同(注意权重布局为 $P \times D \times U \times V$, 以及互相关与数学卷积是否翻转核的区别), 验证两者前向输出以及同一标量损失对输入、权重和偏置的梯度在数值上一致; 再分别对一批多通道输入计时, 感受纯Python嵌套循环与框架优化实现的运行时间差异。结合正文“实际工程中应直接调用`torch.nn.Conv2d`”的提示, 谈谈手写算子的教学价值与生产实现的取舍。

5.2.2 汇聚层算子

汇聚层 (Pooling) 的核心作用是对特征图做下采样: 把局部区域的多个响应压成一个, 从而减少特征数量、降低对窗口内部部分位置变化的敏感性, 但不保证严格的平移不变性。空间尺寸缩小后, 若紧跟全连接层, 还能直接降低后续层的参数量、节省显存。

常见的汇聚算子有两种:

1) **平均汇聚**: 在每个汇聚窗口(如 2×2 区域)内取像素均值作为该区域的代表值, 平滑性更好。

2) **最大汇聚**: 在每个汇聚窗口内取最强响应作为代表值, 保留显著特征更激进。

为了真正看清窗口如何滑动、两种汇聚如何得到不同输出, 可以把这个示例改成一次动手绘图练习。



动手练习 5.4

在配套 notebook 的“汇聚层算子（平均/最大）”小节中，构造一个 4×4 特征图，分别画出 2×2 、步长为 2 的平均汇聚和最大汇聚过程；在原特征图上标出 4 个不重叠窗口，并在旁边画出两种汇聚方式得到的输出矩阵，比较两者在哪些位置给出不同结果。

窗口越大，空间细节损失通常越多。 2×2 、步长为 2 的最大汇聚是常见选择，但不是必需组件。后面的 ResNet 也会用带步长的卷积下采样，用全局平均汇聚把每个通道压缩成一个数。

下面用 PyTorch 实现一个支持平均汇聚和最大汇聚两种模式的简易算子：

```
1 class Pool2D(nn.Module):
2     def __init__(self, size=(2,2), mode='max', stride=1):
3         super().__init__()
4         # 汇聚方式
5         self.mode = mode
6         self.h, self.w = size
7         if mode not in ("max", "avg"):
8             raise ValueError("mode must be max or avg")
9         self.stride = stride
10
11     def forward(self, x):
12         output_h = (x.shape[2] - self.h) // self.stride + 1
13         output_w = (x.shape[3] - self.w) // self.stride + 1
14         # 用 x.new_zeros 自动继承 x 的设备/精度
15         output = x.new_zeros(x.shape[0], x.shape[1], output_h, output_w)
16         # 汇聚
17         for i in range(output.shape[2]):
18             for j in range(output.shape[3]):
19                 # 最大汇聚
20                 if self.mode == 'max':
21                     output[:, :, i, j] = x[:, :, self.stride*i:self.stride*i+self.h, self.
22                                     stride*j:self.stride*j+self.w].flatten(2).max(dim=-1).values
23                 # 平均汇聚
24                 elif self.mode == 'avg':
25                     output[:, :, i, j] = torch.mean(x[:, :, self.stride*i:self.stride*i+self.
26                                     h, self.stride*j:self.stride*j+self.w], dim=(2, 3))
27         return output
```

最大汇聚遇到并列最大值时，前向结果仍然确定，反向传播则需要约定次梯度。本例使用 `max(dim=-1)`，把梯度传给窗口展平后的第一个最大值；`amax` 会在并列最大值之间均分梯度。矩形窗口和并列值的对照代码见配套 notebook。



动手练习 5.5

后面要搭建的 LeNet-5 里, S2 用的是最大汇聚、S4 用的是平均汇聚。读完本节的 Pool2D 算子后, 尝试把 S2、S4 两个汇聚层分别全部改成最大汇聚、再全部改成平均汇聚(其余超参与训练设置保持不变), 在 MNIST 子集上分别重训, 比较两种配置下验证集准确率与收敛曲线, 分析最大汇聚与平均汇聚在这个任务上的差异是否显著, 并结合两种汇聚“保留显著响应”与“平滑均值”的特点解释你观察到的现象。

5.3 基于 LeNet 实现手写体数字识别任务

本节用经典的 LeNet-5 完成手写体数字识别, 把前面实现的卷积层、汇聚层与全连接层在一个完整模型里串起来。



笔记

模型名称中的数字要结合论文约定理解。例如 ResNet18 的 18 通常统计主路径上的 17 个卷积层和 1 个全连接层, 不把投影直连边另外计入。LeNet-5 的名称也不能直接作为“所有可训练层数”的通用计数规则。

5.3.1 数据集构建

手写体数字识别属于入门级视觉任务: 把 0~9 这 10 种数字的手写图像各自归类。但“入门级”并不意味着简单——不同人的笔迹差异显著, 模型必须学会对这些扰动具备一定鲁棒性。

本实验使用经典的 MNIST 数据集。MNIST 提供 60 000 张训练样本和 10 000 张测试样本, 全部图像已统一做过尺寸归一化与居中对齐, 分辨率固定为 28×28 。

为了缩短实验运行时间, 本章只取 MNIST 的一个子集: 训练集 1 000 条、验证集 200 条、测试集 200 条。每个数据集都用两份列表分别存储图像和标签, 以训练集为例:

- 图像数据: 形状为 $[1000, 784]$, 1 000 幅图像, 每幅用长度为 784 的向量记录 28×28 灰度值。
- 标签数据: 形状为 $[1000]$, 对应每幅图像的真实数字 (0~9)。

下面读取并查看子集的样本数:

```
1 import numpy as np
2 from torchvision import datasets
3 # 使用与边缘检测示例相同的缓存目录
4 from pathlib import Path
5 CACHE = str(Path.home() / ".cache" / "torch_data")
6 mnist_train = datasets.MNIST(CACHE, train=True, download=True)
```

```

7 mnist_test = datasets.MNIST(CACHE, train=False, download=True)
8 # 取子集, 并把每幅图转成长度 784 的灰度向量, 存成 [图像列表, 标签列表]:
9 # 训练取前 1000、验证取训练集第 1000~1200、测试取前 200
10 def take(ds, idxs):
11     images = [np.array(ds[i][0]).reshape(-1).tolist() for i in idxs]
12     labels = [int(ds[i][1]) for i in idxs]
13     return [images, labels]
14 train_set = take(mnist_train, range(1000))
15 dev_set = take(mnist_train, range(1000, 1200))
16 test_set = take(mnist_test, range(200))
17 print(f'Length of train/dev/test set:{len(train_set[0])}/{len(dev_set[0])}/{len(test_set[0])}')

```

输出结果为:

```
Length of train/dev/test set:1000/200/200
```

把训练集第 0 条样本还原成 28×28 图像并打印对应标签 (绘图代码见配套 notebook), 样本图像见图 5.9:

```

1 image, label = train_set[0][0], train_set[1][0]
2 image, label = np.array(image).astype('float32'), int(label)
3 # 原始图像数据为长度 784 的行向量, 需要调整为 [28, 28] 大小的图像
4 image = np.reshape(image, [28, 28])
5 print(f"The number in the picture is {label}")

```

输出结果为:

```
The number in the picture is 5
```

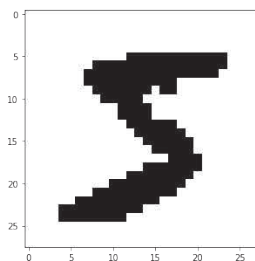


图 5.9 样本图像

5.3.1.1 数据预处理

图像在送入网络之前需要满足模型的尺寸、范围等约定. 本实验主要做两件事:

- 调整图像大小: LeNet 期望 32×32 输入, 而 MNIST 原始尺寸为 28×28 , 本例先缩放到这个尺寸; 四周补边也是一种方案, 但会得到不同的输入.

- 规范化: 原始灰度值位于 $[0, 255]$. `ToTensor` 把本例的 8 位 PIL 图像转换为 $[0, 1]$ 张量, 再用 `Normalize` 映射到 $[-1, 1]$. 这控制了输入量级, 并不意味着数据均值恰好为 0, 也不能单独保证训练稳定.



笔记

`torchvision.transforms` 里已经提供了常用的视觉预处理算子, 可以直接调用: `Resize` 缩放、`Normalize` 标准化、`Compose` 负责串联多个步骤.

实现如下:

```
1 from torchvision.transforms import Compose, Resize, ToTensor, Normalize
2 # 数据预处理: ToTensor 把 PIL 图转成 [0, 1] 张量, 再标准化到 [-1, 1]
3 transforms = Compose([Resize(32), ToTensor(),
4                        Normalize(mean=[0.5], std=[0.5]) ] )
```

5.3.1.2 构建 Dataset 类

把原始列表封装成 PyTorch 约定的 `Dataset` 类, 以便后续用 `DataLoader` 统一加载、洗牌、做小批量:

```
1 import random
2 from torch.utils.data import DataLoader, Dataset
3
4 class MNISTDataset(Dataset):
5     def __init__(self, dataset, transforms, mode='train'):
6         self.mode = mode
7         self.transforms = transforms
8         self.dataset = dataset
9         self.index_list = list(range(len(self.dataset[0])))
10
11     def __getitem__(self, idx):
12         idx = self.index_list[idx]
13         # 获取图像和标签
14         image, label = self.dataset[0][idx], self.dataset[1][idx]
15         image, label = np.array(image).astype('float32'), int(label)
16         image = np.reshape(image, [28, 28])
17         image = Image.fromarray(image.astype('uint8'))
18         image = self.transforms(image)
19
20         return image, label
21
22     def __len__(self):
23         return len(self.dataset[0])
```

利用上面的 `Dataset` 子类加载训练/验证/测试三套数据:

```
1 # 固定随机种子
2 random.seed(0)
3 # 加载mnist数据集
4 train_dataset = MNISTDataset(dataset=train_set, transforms=transforms, mode='train')
5 test_dataset = MNISTDataset(dataset=test_set, transforms=transforms, mode='test')
6 dev_dataset = MNISTDataset(dataset=dev_set, transforms=transforms, mode='dev')
```

5.3.2 模型构建

LeNet-5 把笔画附近的局部响应逐层组合为数字类别。早期的文档识别研究已经用这类卷积网络处理手写字符;这里关注的不是复现当年的识别系统,而是把每一层的输入、输出和参数对上。网络结构见图 5.10。

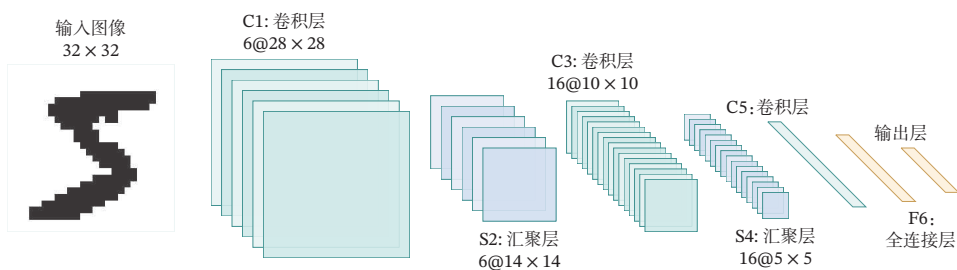


图 5.10 LeNet-5 网络结构

上一节的循环实现用于检查计算过程。训练时使用框架的卷积、最大汇聚与平均汇聚算子,分别为 `nn.Conv2d`、`nn.MaxPool2d` 和 `nn.AvgPool2d`。先对齐权重验证手写算子,再使用框架算子训练,便能把“公式是否正确”和“网络能否学会任务”分别检查清楚。

提醒



本节实现的 LeNet-5 与原始论文有 4 处差异:(1)C3 层不再使用连接表来削减卷积通道数;(2)汇聚层退化为无参数、无激活的最简形式;(3)卷积层激活函数统一改为 ReLU;(4)输出层直接使用全连接线性层。

网络共 7 层:3 个卷积层、2 个汇聚层、2 个全连接层(最后一层为分类输出)。输入是 32×32 的灰度图,输出对应 10 类得分:

```
1 import torch.nn.functional as F
2
3 class Model_LeNet(torch.nn.Module):
4     def __init__(self, in_channels, num_classes=10):
```

```

5         super().__init__()
6         # 卷积层: 输出通道数为6, 卷积核大小为5×5
7         self.conv1 = nn.Conv2d(in_channels=in_channels, out_channels=6, kernel_size=5)
8         # 汇聚层: 汇聚窗口为2×2, 步长为2
9         self.pool2 = nn.MaxPool2d(kernel_size=2, stride=2)
10        # 卷积层: 输入通道数为6, 输出通道数为16, 卷积核大小为5×5, 步长为1
11        self.conv3 = nn.Conv2d(in_channels=6, out_channels=16, kernel_size=5, stride=1)
12        # 汇聚层: 汇聚窗口为2×2, 步长为2
13        self.pool4 = nn.AvgPool2d(kernel_size=2, stride=2)
14        # 卷积层: 输入通道数为16, 输出通道数为120, 卷积核大小为5×5
15        self.conv5 = nn.Conv2d(in_channels=16, out_channels=120, kernel_size=5, stride=1)
16        # 全连接层: 输入神经元为120, 输出神经元为84
17        self.linear6 = nn.Linear(120, 84)
18        # 全连接层: 输入神经元为84, 输出神经元为类别数
19        self.linear7 = nn.Linear(84, num_classes)
20
21    def forward(self, x):
22        # C1: 卷积层+激活函数
23        output = F.relu(self.conv1(x))
24        # S2: 汇聚层
25        output = self.pool2(output)
26        # C3: 卷积层+激活函数
27        output = F.relu(self.conv3(output))
28        # S4: 汇聚层
29        output = self.pool4(output)
30        # C5: 卷积层+激活函数
31        output = F.relu(self.conv5(output))
32        # C5 输出空间尺寸已是 1×1, 用 squeeze 去掉两个空间维 [B,C,1,1] -> [B,C]
33        output = torch.squeeze(output, dim=(2, 3))
34        # F6: 全连接层
35        output = F.relu(self.linear6(output))
36        # F7: 全连接层
37        output = self.linear7(output)
38        return output

```

下面用一个 [1, 1, 32, 32] 的随机张量做一次前向推断, 逐层打印特征图形状, 便于直观验证网络结构是否符合预期:

```

1  # 用 np.random 创建一个随机数组作为输入数据
2  inputs = np.random.randn(1, 1, 32, 32).astype('float32')
3  # 创建 Model_LeNet 实例, 指定输入通道数和分类的类别数目
4  model = Model_LeNet(in_channels=1, num_classes=10)
5  # 打印模型结构, 查看 LeNet 中所包含的子层
6  print(model)
7

```

```

8 x = torch.tensor(inputs)
9 # 逐层向前, 打印每一层之后的特征图形状, 以及该层权重与偏置的形状
10 for name, layer in model.named_children():
11     if name == 'linear6':
12         # 进入全连接层前先把空间维度展平
13         x = torch.flatten(x, start_dim=1)
14     x = layer(x)
15     if getattr(layer, 'weight', None) is not None:
16         w_shape = tuple(layer.weight.shape)
17         b_shape = tuple(layer.bias.shape) if getattr(layer, 'bias', None) is not None else
            None
18         print(name, tuple(x.shape), w_shape, b_shape)
19     else:
20         # 汇聚层没有参数
21         print(name, tuple(x.shape))

```

输出结果为:

```

Model_LeNet(
  (conv1): Conv2d(1, 6, kernel_size=(5, 5), stride=(1, 1))
  (pool1): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  (conv2): Conv2d(6, 16, kernel_size=(5, 5), stride=(1, 1))
  (pool2): AvgPool2d(kernel_size=2, stride=2, padding=0)
  (conv3): Conv2d(16, 120, kernel_size=(5, 5), stride=(1, 1))
  (linear4): Linear(in_features=120, out_features=84, bias=True)
  (linear5): Linear(in_features=84, out_features=10, bias=True)
)
conv1 (1, 6, 28, 28) (6, 1, 5, 5) (6,)
pool1 (1, 6, 14, 14)
conv2 (1, 16, 10, 10) (16, 6, 5, 5) (16,)
pool2 (1, 16, 5, 5)
conv3 (1, 120, 1, 1) (120, 16, 5, 5) (120,)
linear4 (1, 84) (84, 120) (84,)
linear5 (1, 10) (10, 84) (10,)

```

逐层张量形状的演化清晰可见:

- 输入是 $1 \times 32 \times 32$ 的灰度图, 先经 C1 (6 个 5×5 卷积核) 变成 $6 \times 28 \times 28$.
- 经 S2 (2×2 、步长 2 的最大汇聚) 下采样为 $6 \times 14 \times 14$.
- 经 C3 (16 个 5×5 卷积核) 变为 $16 \times 10 \times 10$.
- 经 S4 (2×2 、步长 2 的平均汇聚) 下采样为 $16 \times 5 \times 5$.
- 经 C5 (120 个 5×5 卷积核) 变为 $120 \times 1 \times 1$.
- 把空间维展平后得到长度 120 的向量, 经 F6 ($120 \rightarrow 84$) 输出长度 84.
- 最后通过 F7 ($84 \rightarrow$ 类别数) 得到分类得分.

5.3.3 模型训练

训练时采用交叉熵损失, 优化器选随机梯度下降 (学习率 0.1); 通过 RunnerV3 在训练集上跑 20 个回合, 并保存验证集上准确率最高的模型:

```
1 import torch.optim as optim
2 from nndl import RunnerV3, accuracy
3
4 torch.manual_seed(100)
5 # 批大小
6 batch_size = 64
7 # 加载数据
8 train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True,
9                             generator=torch.Generator().manual_seed(100))
10 dev_loader = DataLoader(dev_dataset, batch_size=batch_size)
11 test_loader = DataLoader(test_dataset, batch_size=batch_size)
12 # 定义 LeNet 网络
13 model = Model_LeNet(in_channels=1, num_classes=10)
14 # 定义优化器
15 optimizer = optim.SGD(model.parameters(), lr=0.1)
16 # 定义损失函数
17 loss_fn = F.cross_entropy
18 # 定义评价指标
19 metric = accuracy
20 # 实例化 RunnerV3 类, 并传入训练配置
21 runner = RunnerV3(model, optimizer, loss_fn, metric_fn=metric)
22 # 模型训练
23 runner.fit(train_loader, dev_loader, num_epochs=20, log_every=1, best_path="./checkpoint/
    lenet_mnist_best.pt")
```

在本次运行中, 前 5 个回合的训练损失仍接近 2.3, 此时就停止训练会低估模型在这份数据上的学习能力. 延长到 20 个回合后, 训练损失明显下降. 训练集与验证集上的损失曲线和准确率变化如图 5.11 所示.

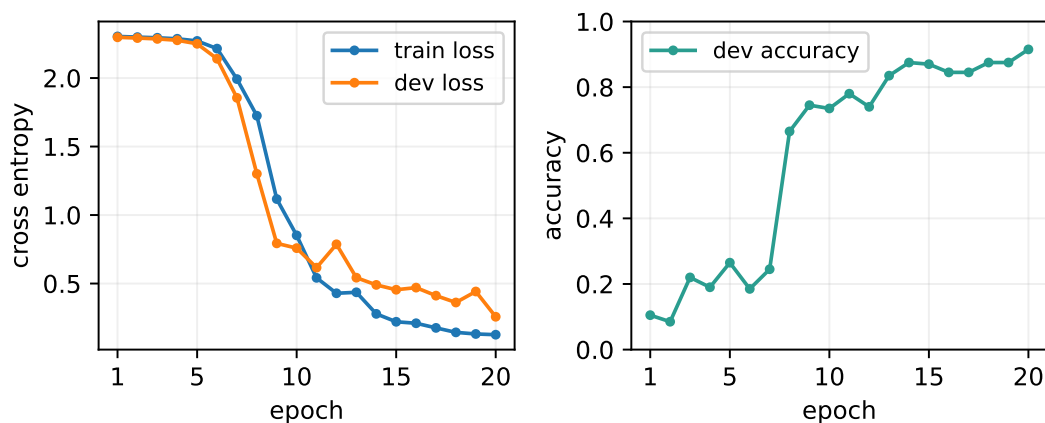


图 5.11 损失和准确率变化趋势

5.3.4 模型评价

载入训练阶段保存的最优模型,在测试集上做一次最终评估:

```
1 # 加载最优模型
2 runner.load("./checkpoint/lenet_mnist_best.pt")
3 # 模型评价
4 loss, score = runner.evaluate(test_loader)
5 print(f"[Test] accuracy/loss: {score:.4f}/{loss:.4f}")
```

输出结果为:

```
[Test] accuracy/loss: 0.9550/0.1642
```

5.3.5 模型预测

用同样的模型对测试集中的一条样本做预测,并对比真实标签:

```
1 # 获取测试集中的一个批次
2 X, label = next(iter(test_loader))
3 logits = runner.predict(X)
4 # 多分类, 沿类别维度做 softmax 得到预测概率
5 pred = F.softmax(logits, dim=1)
6 # 取出第 2 个样本的预测和真实类别
7 sample_id = 1
8 pred_class = pred[sample_id].argmax().item()
9 true_class = label[sample_id].item()
10 # 输出真实类别与预测类别
```



```
11 print(f"The true category is {true_class} and the predicted category is {pred_class}")
```

输出结果为:

```
The true category is 2 and the predicted category is 2
```

把该样本图像画出(绘图代码见配套 notebook), 预测结果见图 5.12.

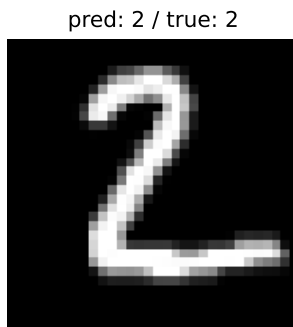


图 5.12 预测图像

5.4 基于残差网络的手写体数字识别

如果一个浅层网络已经能识别数字, 再加几层是否一定更好? 新增层在理论上可以学习恒等映射, 但优化器未必容易找到它. **残差网络** (Residual Network, ResNet) 把这条路径直接写进结构: 主干学习对输入的修正, **直连边** (Skip Connection) 把输入送到后面相加. 这种以相加方式合并跨层信息的连接称为**残差连接** (Residual Connection); 直连边也可以采用其他合并方式, 两者并非在所有结构中同义. 这样的残差单元改变了函数的参数化方式, 也为梯度提供了加性的传播路径, 能够改善深层网络的优化, 但不保证任意深度都能稳定训练.

设 $f(\mathbf{X}; \theta)$ 表示一段普通的非线性子网络, 残差单元在它的输入和输出之间加一条直连边:

$$\text{ResBlock}_f(\mathbf{X}) = f(\mathbf{X}; \theta) + \mathbf{X}, \quad (5.7)$$

其中 θ 为子网络的可学习参数. 式中省略了相加后的激活函数; 本节代码采用相加后再做 ReLU 的形式. 典型的残差单元结构见图 5.13: 由若干通道一致的卷积层串成主干, 再叠加一条跨越主干的直连边.

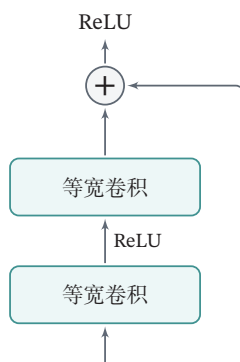


图 5.13 残差单元结构

下面搭一个经典的 ResNet18, 并再次完成手写体数字识别, 对照看深度网络相对于 LeNet 的差异.

5.4.1 模型构建

ResNet 家族在不同深度下使用了两种残差单元: 浅层版本 (ResNet18、ResNet34) 采用两层 3×3 等宽卷积加直连边; 深层版本 (ResNet50/101/152) 则采用“中间细、两头粗”的**瓶颈结构**——先用 1×1 卷积压通道, 再做 3×3 卷积, 最后用 1×1 卷积恢复通道, 从而控制参数量在合理范围. 两种残差单元的对比见图 5.14.

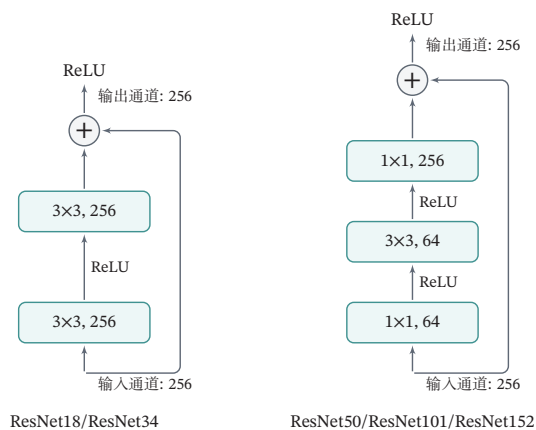


图 5.14 不同深度残差网络的残差单元结构示意图

5.4.1.1 残差单元

下面实现浅层残差单元 (图 5.14 左侧版本). 为了简洁, 主干卷积直接复用 PyTorch 的 `nn.Conv2d`:

```

1 conv = nn.Conv2d(in_channels=3, out_channels=16,
2                 kernel_size=3, stride=1, padding=1)

```

`Conv2d`根据`in_channels`、`out_channels`自动决定卷积核数量。卷积核张量布局是MCHW（M输出通道、C输入通道、H高、W宽），输入/输出默认NCHW（批量、通道、高、宽）。前向时根据`kernel_size`、`stride`、`padding`、`dilation`与输入尺寸计算输出尺寸。

公式(5.7)成立的前提是 $f(\mathbf{X}; \theta)$ 输入输出形状一致——可以直接相加。但残差单元在工程中经常承担“改通道、下采样”的任务，主干侧已经把形状变了，直连边也必须做对应变形，否则无法对齐。

补救方法很统一：在直连边上插一个 1×1 卷积 $\mathbf{X} \rightarrow \mathbf{X}'$ ，让它跟着主干一起变形——通道数不一致时，让 1×1 卷积把 D 通道映射到 P 通道；空间尺寸不一致时，让这个 1×1 卷积带上恰当的步长完成同步下采样。两种情形下直连边的具体形态对比见图5.15。

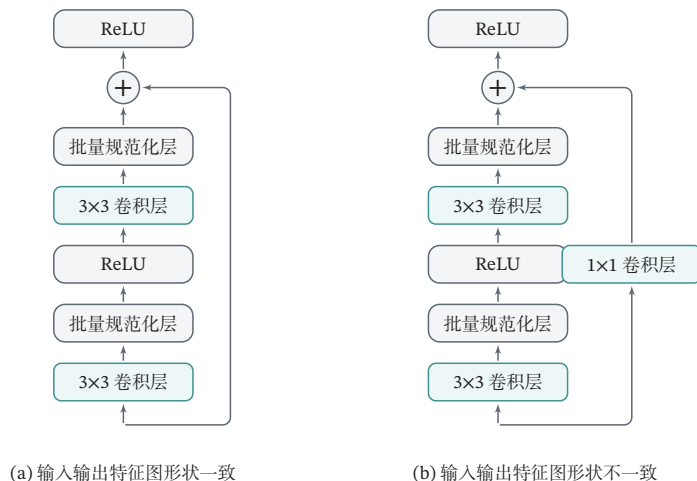


图 5.15 输入特征图与输出特征图形状一致和不一致时直连边的不同设置

笔记

残差网络往往很深，直接优化容易出现梯度问题，因此残差单元中通常额外加入**批量规范化层**（BatchNorm）来稳定训练。批量规范化的细节参见第7.5.1节。

下面按照图5.15的结构逐步实现残差单元。接下来的两段代码展示构造函数内部的逻辑；实际运行时使用后面的完整`ResBlock`类。

（1）先实现主干上的两个卷积层。第一个卷积层可以改变输出通道数与步长。当输入、输出通道数不同，或步长不为1时，输入输出形状不再相同，就需要在直连边上使用 1×1 卷积。

```

1 # 第一个卷积层, 卷积核大小为3×3, 可以设置不同输出通道数以及步长
2 conv1 = nn.Conv2d(in_channels, out_channels, 3, padding=1, stride=stride, bias=False)
3 # 第二个卷积层, 卷积核大小为3×3, 不改变输入特征图的形状, 步长为1
4 conv2 = nn.Conv2d(out_channels, out_channels, 3, padding=1, bias=False)
5 # 如果残差单元的输入和输出形状不一致, 就需要使用带1×1卷积的直连边
6 if in_channels != out_channels or stride != 1:
7     use_1x1conv = True
8 else:
9     use_1x1conv = False

```

布尔变量`use_1x1conv`用来标记是否需要带 1×1 卷积的直连边。

(2) 接着按需构造直连边: 若`use_1x1conv`为真, 就插入一个 1×1 卷积, 把输入特征图变换为与主干输出一致的形状。

```

1 if self.use_1x1conv:
2     shortcut = nn.Conv2d(in_channels, out_channels, 1, stride=stride, bias=False)

```



提醒

这里卷积层的`bias`都设为`False`。训练时, 紧随其后的 `BatchNorm` 会减去通道均值, 卷积偏置随之抵消; `BatchNorm` 自身的可学习平移参数仍可调整输出。若改为固定规范化统计量等其他训练方式, 需要重新检查这一条件。



完整的残差单元`ResBlock`如下: 通过`use_residual`参数控制是否启用直连边, 便于后续做对照实验。

```

1 class ResBlock(nn.Module):
2     def __init__(self, in_channels, out_channels, stride=1, use_residual=True):
3         """
4         残差单元
5         输入:
6             - in_channels: 输入通道数
7             - out_channels: 输出通道数
8             - stride: 残差单元的步长, 通过调整残差单元中第一个卷积层的步长来控制
9             - use_residual: 用于控制是否使用残差连接
10        """
11        super().__init__()
12        self.stride = stride
13        self.use_residual = use_residual
14        # 第一个卷积层, 卷积核大小为3×3, 可以设置不同输出通道数以及步长
15        self.conv1 = nn.Conv2d(in_channels, out_channels, 3, padding=1, stride=self.stride,
16                               bias=False)
17        # 第二个卷积层, 卷积核大小为3×3, 不改变输入特征图的形状, 步长为1

```

```

17     self.conv2 = nn.Conv2d(out_channels, out_channels, 3, padding=1, bias=False)
18     # 如果conv2的输出和此残差块的输入数据形状不一致, 则use_1x1conv = True
19     # 当use_1x1conv = True, 添加1个1x1的卷积作用在输入数据上, 使其形状变成和conv2一致
20     if in_channels != out_channels or stride != 1:
21         self.use_1x1conv = True
22     else:
23         self.use_1x1conv = False
24     # 当残差单元包裹的非线性层输入和输出通道数不一致时, 需要用1x1卷积调整通道数后再进行相加运算
25     if self.use_1x1conv:
26         self.shortcut = nn.Conv2d(in_channels, out_channels, 1, stride=self.stride, bias=False)
27     # 每个卷积层后会接一个批量规范化层, 批量规范化的内容在第7.5.1节中进行详细介绍
28     self.bn1 = nn.BatchNorm2d(out_channels)
29     self.bn2 = nn.BatchNorm2d(out_channels)
30     if self.use_1x1conv:
31         self.bn3 = nn.BatchNorm2d(out_channels)
32
33     def forward(self, inputs):
34         y = F.relu(self.bn1(self.conv1(inputs)))
35         y = self.bn2(self.conv2(y))
36         if self.use_residual:
37             if self.use_1x1conv: # 如果为真, 对inputs进行1x1卷积, 将形状调整成和conv2的输出y一致
38                 shortcut = self.bn3(self.shortcut(inputs))
39             else: # 否则直接将inputs和conv2的输出y相加
40                 shortcut = inputs
41             y = torch.add(shortcut, y)
42         out = F.relu(y)
43         return out

```



动手练习 5.6

仿照上面的写法实现瓶颈型残差单元（图5.14右侧版本），并对比在相同输入/输出通道下, 瓶颈结构与基础残差单元的参数量差异。

5.4.1.2 残差网络的整体结构

ResNet把若干残差单元串联起来形成深层网络. ResNet18的整体结构见图5.16.

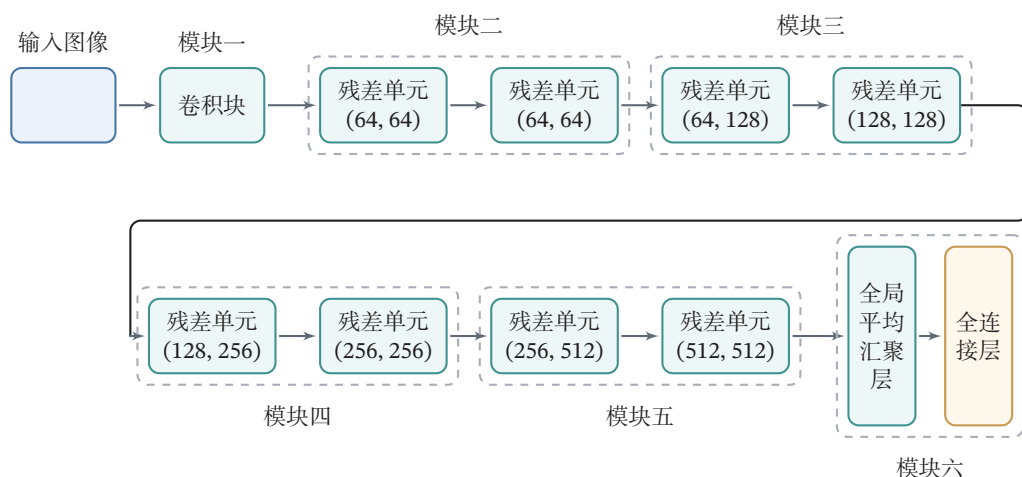


图 5.16 残差网络 ResNet18 的网络结构

为了便于理解,可以把 ResNet18 拆成 6 个模块:

- 1) 模块一:步长 2 的 7×7 卷积(输出 64 通道)、BN、ReLU 和步长 2 的 3×3 最大汇聚,整体把输入图像快速下采样。
- 2) 模块二:2 个残差单元,输入输出均 64 通道,空间尺寸不变。
- 3) 模块三:2 个残差单元,通道数从 64 升到 128,特征图尺寸减半。
- 4) 模块四:2 个残差单元,通道数从 128 升到 256,特征图尺寸再减半。
- 5) 模块五:2 个残差单元,通道数从 256 升到 512,特征图尺寸继续减半。
- 6) 模块六:全局平均汇聚把特征图压成 1×1 ,再接一个全连接层输出类别得分。

(1) 模块一的代码实现如下:

```

1 def make_first_module(in_channels):
2     # 模块一: 7*7卷积、批量规范化、汇聚
3     m1 = nn.Sequential(nn.Conv2d(in_channels, 64, 7, stride=2, padding=3),
4                         nn.BatchNorm2d(64), nn.ReLU(),
5                         nn.MaxPool2d(kernel_size=3, stride=2, padding=1))
6     return m1

```

(2) 模块二到模块五结构相似: 每个模块由 num_res_blocks 个残差单元串接而成。其中模块三到模块五的模块内第一个残差单元负责把通道数提升、空间尺寸减半,完成“维度过渡”;其余残差单元保持输入输出形状一致,专注于在新维度上做特征精修。模块二是个例外: 它的两个残差单元始终保持 64 通道、空间尺寸不变。

```

1 def resnet_module(input_channels, out_channels, num_res_blocks, stride=1, use_residual=
    True):
2     blk = []

```

```

3     # 根据num_res_blocks, 循环生成残差单元
4     for i in range(num_res_blocks):
5         if i == 0: # 创建模块中的第一个残差单元
6             blk.append(ResBlock(input_channels, out_channels,
7                                 stride=stride, use_residual=use_residual))
8         else:      # 创建模块中的其他残差单元
9             blk.append(ResBlock(out_channels, out_channels, use_residual=use_residual))
10    return blk

```

下面按上述约定具体定义模块二到模块五:

```

1 def make_modules(use_residual):
2     # 模块二: 包含两个残差单元, 输入通道数为64, 输出通道数为64, 步长为1, 特征图大小保持不变
3     m2 = nn.Sequential(*resnet_module(64, 64, 2, stride=1, use_residual=use_residual))
4     # 模块三: 包含两个残差单元, 输入通道数为64, 输出通道数为128, 步长为2, 特征图大小缩小一半.
5     m3 = nn.Sequential(*resnet_module(64, 128, 2, stride=2, use_residual=use_residual))
6     # 模块四: 包含两个残差单元, 输入通道数为128, 输出通道数为256, 步长为2, 特征图大小缩小一半.
7     m4 = nn.Sequential(*resnet_module(128, 256, 2, stride=2, use_residual=use_residual))
8     # 模块五: 包含两个残差单元, 输入通道数为256, 输出通道数为512, 步长为2, 特征图大小缩小一半.
9     m5 = nn.Sequential(*resnet_module(256, 512, 2, stride=2, use_residual=use_residual))
10    return m2, m3, m4, m5

```

完整的 ResNet18 模型代码如下:

```

1 class Model_ResNet18(nn.Module):
2     def __init__(self, in_channels=3, num_classes=10, use_residual=True):
3         super().__init__()
4         m1 = make_first_module(in_channels)
5         m2, m3, m4, m5 = make_modules(use_residual)
6         # 封装模块一到模块六
7         self.net = nn.Sequential(m1, m2, m3, m4, m5,
8                                   # 模块六: 全局平均汇聚层、全连接层
8                                   nn.AdaptiveAvgPool2d(1), nn.Flatten(), nn.Linear(512, num_classes) )
9
10
11    def forward(self, x):
12        return self.net(x)

```

下面用同一份 MNIST 划分做对照, 只切换 `use_residual`. 两组实验使用相同的初始参数、学习率 0.01、批次顺序和 5 个训练回合. 模型始终构造投影层, 关闭直连边时这些层不参与前向计算, 因此两种配置的参数名称和初值能逐项对齐. 一次小样本实验只能说明这一组条件下的现象, 不能代替多随机种子的比较.

5.4.2 没有残差连接的 ResNet18

先看缺少残差连接时的表现.

5.4.2.1 模型训练

在训练集、验证集上跑 5 个回合,并保留验证集准确率最高的模型作为最优模型:

```
1 torch.manual_seed(100)
2 # 批大小
3 batch_size = 64
4 # 加载数据
5 train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True,
6                             generator=torch.Generator().manual_seed(100))
7 dev_loader = DataLoader(dev_dataset, batch_size=batch_size)
8 test_loader = DataLoader(test_dataset, batch_size=batch_size)
9 # 定义网络, 没有使用残差结构的深层网络
10 model = Model_ResNet18(in_channels=1, num_classes=10, use_residual=False)
11 # 定义优化器
12 optimizer = optim.SGD(model.parameters(), lr=0.01)
13 # 实例化RunnerV3
14 runner = RunnerV3(model, optimizer, loss_fn, metric_fn=metric)
15 # 模型训练
16 runner.fit(train_loader, dev_loader, num_epochs=5, log_every=1, best_path="./checkpoint/
    plain_mnist_best.pt")
```

图5.17记录训练损失、验证损失和验证准确率. 先看训练损失是否下降, 再看训练与验证的差距; 验证曲线的波动还可能来自样本数量较少和批量规范化统计量的变化.

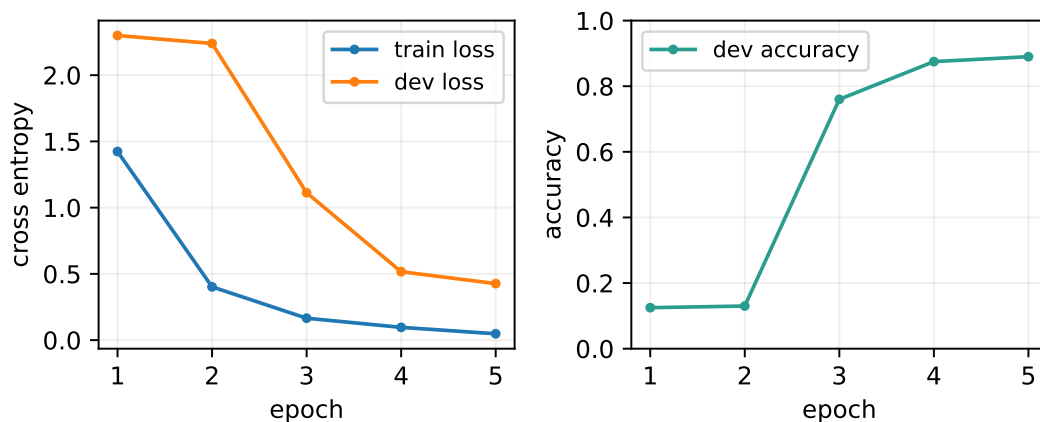


图 5.17 没有使用残差连接的 ResNet 的损失和准确率变化趋势

5.4.2.2 模型评价

在测试集上评估最优模型, 得到泛化性能:

```
1 # 加载最优模型
2 runner.load("./checkpoint/plain_mnist_best.pt")
```

```

3 # 模型评价
4 loss, score = runner.evaluate(test_loader)
5 print(f"[Test] accuracy/loss: {score:.4f}/{loss:.4f}")

```

输出结果为:

```
[Test] accuracy/loss: 0.9150/0.2673
```

这个网络与 LeNet 的层数、通道数、下采样方式和学习率都不同, 不能把两者的准确率差异全部归因于深度. 要分析直连边的作用, 应与下一组结构相同、只启用直连边的实验比较.

5.4.3 带残差连接的 ResNet18

5.4.3.1 模型训练

现在打开残差连接, 其他超参数保持一致, 重新训练:

```

1 # 批大小
2 batch_size = 64
3 # 加载数据
4 train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True,
5                             generator=torch.Generator().manual_seed(100))
6 dev_loader = DataLoader(dev_dataset, batch_size=batch_size)
7 test_loader = DataLoader(test_dataset, batch_size=batch_size)
8 # 在构造模型前重置种子, 使两组实验的初始参数一致
9 torch.manual_seed(100)
10 # 打开残差连接
11 model = Model_ResNet18(in_channels=1, num_classes=10, use_residual=True)
12 # 定义优化器
13 optimizer = optim.SGD(model.parameters(), lr=0.01)
14 # 实例化 RunnerV3
15 runner = RunnerV3(model, optimizer, loss_fn, metric_fn=metric)
16 # 模型训练
17 runner.fit(train_loader, dev_loader, num_epochs=5, log_every=1, best_path="./checkpoint/
    resnet_mnist_best.pt")

```

图 5.18 给出启用直连边后的训练曲线. 将它与上一图对照, 分别观察训练损失下降速度与验证集表现; 优化得更快和泛化得更好是两个需要区分的问题.

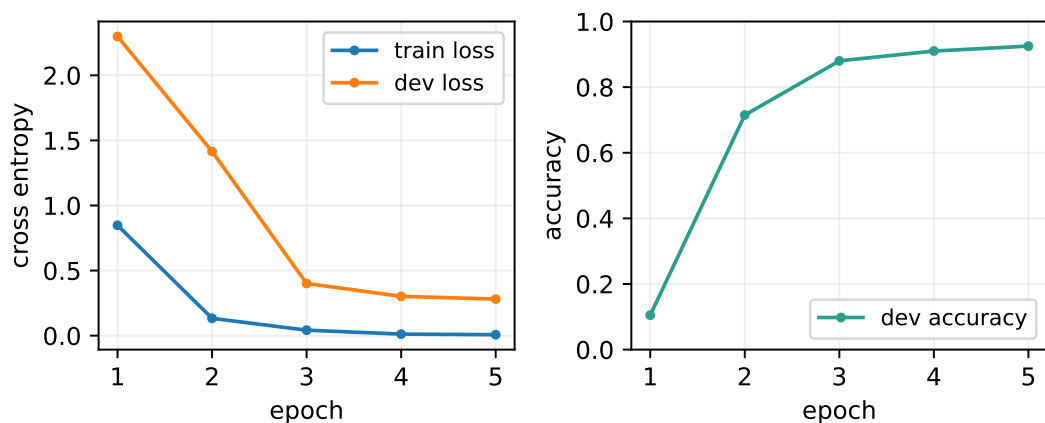


图 5.18 使用残差连接的 ResNet 的损失和准确率变化趋势

5.4.3.2 模型评价

载入训练阶段保存的最优模型快照,在测试集上做最终评估:

```
1 # 加载最优模型
2 runner.load("./checkpoint/resnet_mnist_best.pt")
3 # 模型评价
4 loss, score = runner.evaluate(test_loader)
5 print(f"[Test] accuracy/loss: {score:.4f}/{loss:.4f}")
```

输出结果为:

```
[Test] accuracy/loss: 0.9700/0.1270
```

本次运行中,关闭和开启直连边时的测试准确率分别为 91.5% 和 97.0%;残差组的训练损失下降也更快.比较这两次输出时,应同时查看曲线和验证集选择的回合.即使某一组测试准确率更高,也只能视为本次固定划分下的观察.更可靠的结论需要事先确定比较指标,重复多个随机种子,并报告均值和波动范围.



动手练习 5.7

尝试加深残差网络的层数,观察是否能够得到更高的精度。



5.5 实践:基于 ResNet18 网络完成图像分类任务

手写数字大多居中、背景简单.换成自然图像后,同一类物体可能出现在不同背景、角度和尺度下,问题更难了.本节保留训练框架,把输入换为 CIFAR-10 的彩色图像,检查数据预处理和网络

入口需要怎样调整。图像分类 (Image Classification) 给整幅图像赋予类别。例如,判断一张图是否包含人脸可以表述为二分类;若还要标出人脸的位置,就属于目标检测,需要额外的定位输出。这里每张图只有一个类别标签。

本实验配置如下:数据集采用 CIFAR-10、模型沿用 ResNet18、损失函数取交叉熵、优化器选 Adam、评价指标取准确率。



笔记

Adam 优化器的细节参见《神经网络与深度学习》第 7.2.4.3 节。



5.5.1 数据处理

5.5.1.1 数据集介绍

CIFAR-10 共有 60 000 张彩色图像,分为 10 个类别,每类 6 000 张,分辨率统一为 32×32 。示例见图 5.19。

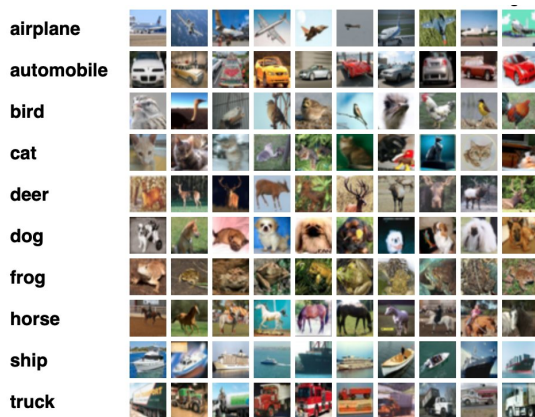


图 5.19 CIFAR-10 数据集示例

首次运行用 torchvision 下载并校验官方数据;已有缓存时会直接复用。后续仍手写批文件读取,以观察磁盘布局与张量布局的对应关系。

```
1 from pathlib import Path
2 from torchvision import datasets
3 CACHE = str(Path.home() / '.cache' / 'torch_data')
4 datasets.CIFAR10(CACHE, train=True, download=True)
5 datasets.CIFAR10(CACHE, train=False, download=True)
6 cifar_path = str(Path(CACHE) / 'cifar-10-batches-py')
```

原始数据按批存储:训练集 50 000 条切成 5 个批文件,测试集 10 000 条单独成批。

5.5.1.2 数据读取

为方便做模型选择,把原始训练集再拆分成训练集和验证集:前 4 个批文件合并为训练集(40 000 条),第 5 个批文件作为验证集(10 000 条),原始test_batch保持为测试集(10 000 条)。

- 训练集:40 000 条.
- 验证集:10 000 条.
- 测试集:10 000 条.

读取单个批数据的实现如下:

```

1 import os
2 import pickle
3
4 def load_cifar10_batch(folder_path, batch_id=1, mode='train'):
5     if mode == 'test':
6         file_path = os.path.join(folder_path, 'test_batch')
7     else:
8         file_path = os.path.join(folder_path, 'data_batch_'+str(batch_id))
9     # 加载数据集文件
10    with open(file_path, 'rb') as batch_file:
11        batch = pickle.load(batch_file, encoding = 'latin1')
12
13    imgs = batch['data'].reshape((len(batch['data']),3,32,32)) / 255.
14    labels = batch['labels']
15
16    return np.array(imgs, dtype=np.float32), np.array(labels)
17
18 imgs_batch, labels_batch = load_cifar10_batch(folder_path=cifar_path, batch_id=1, mode='
    train')

```

查看数据的维度:

```

1 # 打印每个批中X和y的维度
2 print ("batch of imgs shape: ",imgs_batch.shape, "batch of labels shape: ", labels_batch.
    shape)

```

输出结果为:

```
batch of imgs shape: (10000, 3, 32, 32) batch of labels shape: (10000,)
```

把第二条样本对照标签打印,确认数据已正确读入;再将其画出(绘图代码见配套 notebook),结果见图5.20:

```

1 image, label = imgs_batch[1], labels_batch[1]
2 print(f"The label in the picture is {label}")

```

输出结果为:

The label in the picture is 9

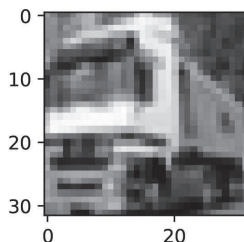


图 5.20 数据集样本图像

5.5.1.3 构造 Dataset 类

把上面的批读取逻辑封装成 CIFAR10Dataset 子类, 继承自 `torch.utils.data.Dataset`, 便于和 `DataLoader` 对接:

```

1 class CIFAR10Dataset(torch.utils.data.Dataset):
2     def __init__(self, folder_path, mode='train'):
3         if mode == 'train':
4             # 加载batch1~batch4作为训练集
5             self.imgs, self.labels = load_cifar10_batch(folder_path=folder_path, batch_id=1,
6                                                         mode='train')
7             for i in range(2, 5):
8                 imgs_batch, labels_batch = load_cifar10_batch(folder_path=folder_path,
9                                                                 batch_id=i, mode='train')
10                self.imgs, self.labels = np.concatenate([self.imgs, imgs_batch]), np.
11                    concatenate([self.labels, labels_batch])
12
13        elif mode == 'dev':
14            # 加载batch5作为验证集
15            self.imgs, self.labels = load_cifar10_batch(folder_path=folder_path, batch_id=5,
16                                                         mode='dev')
17
18        elif mode == 'test':
19            # 加载test作为测试集
20            self.imgs, self.labels = load_cifar10_batch(folder_path=folder_path, mode='test')
21
22        else:
23            raise ValueError("mode must be train, dev or test")
24
25        # 固定映射到 [-1, 1], 不使用验证集或测试集估计统计量
26        self.transform = Normalize(mean=[0.5]*3, std=[0.5]*3)
27
28    def __getitem__(self, idx):
29        # 数据加载阶段已经把图像 reshape 为 (3, 32, 32) 并归一化到 [0, 1], 这里转成张量后再做通道
30            # 级标准化

```

```

22         img = torch.from_numpy(self.imgs[idx])
23         label = int(self.labels[idx])
24         img = self.transform(img)
25         return img, label
26
27     def __len__(self):
28         return len(self.imgs)
29
30     torch.manual_seed(100)
31     # cifar_path 已由下载步骤设置
32     train_dataset = CIFAR10Dataset(cifar_path, mode='train')
33     dev_dataset = CIFAR10Dataset(cifar_path, mode='dev')
34     test_dataset = CIFAR10Dataset(cifar_path, mode='test')

```

5.5.2 模型构建

使用torchvision提供的 ResNet18, 从随机初始化开始训练 (weights=None), 输出 10 类。标准入口的 7×7 、步长 2 卷积和最大汇聚会过早压缩 32×32 图像, 因此本例把入口改成 3×3 、步长 1 卷积并移除首个最大汇聚。后面的 4 组残差模块和全局平均汇聚保持不变。

```

1  from torchvision.models import resnet18
2  torch.manual_seed(100)
3  resnet18_model = resnet18(weights=None, num_classes=10)
4  resnet18_model.conv1 = nn.Conv2d(3, 64, 3, stride=1, padding=1, bias=False)
5  resnet18_model.maxpool = nn.Identity()

```

笔记



PyTorch 的 torchvision 子库提供了常用视觉数据集、数据增强方法, 以及 MobileNet、ResNet 等模型和预训练权重。直接调用这些组件既能减少重复实现, 也方便在它们之上做二次开发。

5.5.3 模型训练

先用短程实验检查流程: 从上述三个互不重叠的数据划分中, 固定抽取 5 000 条训练样本、1 000 条验证样本和 1 000 条测试样本, 训练 5 个回合。将 full_run 设为 True 可切换到完整划分: 训练样本 40 000 条, 验证与测试样本各 10 000 条, 训练 30 个回合。两种规模的结果必须分开记录。

本例使用 Adam, 并通过 weight_decay 加入 ℓ_2 惩罚; 它与 AdamW 的解耦权重衰减不同, 详见第 7.6.4 节。

```

1  from torch.utils.data import Subset

```

```

2 full_run = False
3 num_epochs = 30 if full_run else 5
4 if not full_run:
5     g = torch.Generator().manual_seed(100)
6     train_dataset = Subset(train_dataset, torch.randperm(len(train_dataset), generator=g)
7                             [:5000].tolist())
8     dev_dataset = Subset(dev_dataset, torch.randperm(len(dev_dataset), generator=g)[:1000].
9                             tolist())
10    test_dataset = Subset(test_dataset, torch.randperm(len(test_dataset), generator=g)
11                             [:1000].tolist())
12    # 指定运行设备
13    device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
14    # 批大小
15    batch_size = 64
16    # 加载数据
17    train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True,
18                              generator=torch.Generator().manual_seed(100))
19    dev_loader = DataLoader(dev_dataset, batch_size=batch_size)
20    test_loader = DataLoader(test_dataset, batch_size=batch_size)
21    # 定义网络
22    model = resnet18_model
23    # 定义优化器
24    optimizer = optim.Adam(model.parameters(), lr=0.001, weight_decay=0.005)
25    # 定义损失函数
26    loss_fn = F.cross_entropy
27    # 定义评价指标
28    metric = accuracy
29    # 实例化RunnerV3, 把设备交给 Runner 一并管理模型与批数据的搬迁
30    runner = RunnerV3(model, optimizer, loss_fn, metric_fn=metric, device=device)
31    # 模型训练
32    runner.fit(train_loader, dev_loader, num_epochs=num_epochs, log_every=1, best_path="./
33               checkpoint/resnet_cifar10_best.pt")

```

图5.21对应上述默认短程配置,横轴按回合计数.训练集和验证集的差距可以提示过拟合,但5个回合不足以判断这一结构在完整数据上的性能上限.扩大训练规模时,应重新生成曲线并记录数据量、随机种子、设备、模型入口和超参数,不能沿用短程结果.

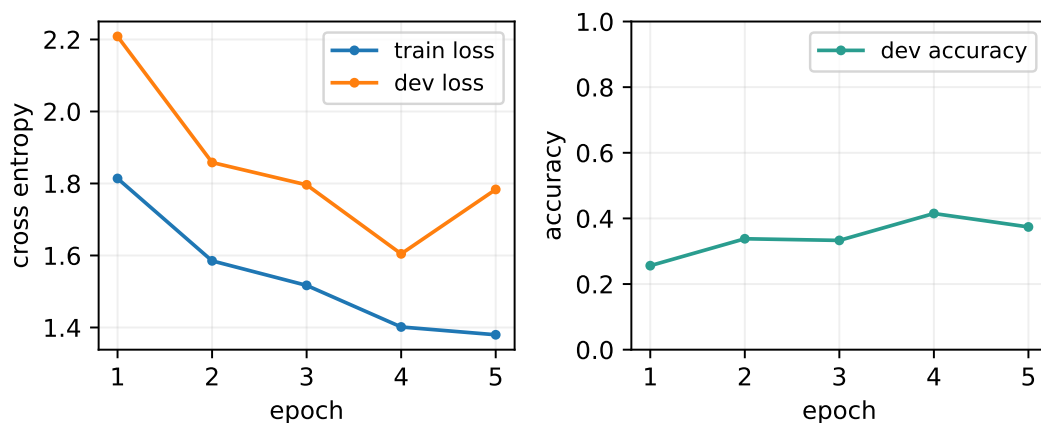


图 5.21 ResNet18 在 CIFAR-10 上的短程训练预览

动手练习 5.8

本实验默认采用 Adam。请在相同数据划分下尝试 SGD+Momentum 或 SGD+Momentum+Nesterov, 分别调节学习率与权重衰减, 并组合第 7 章介绍的学习率调度方法, 比较验证集精度。

**5.5.4 模型评价**

载入最优模型快照, 在测试集上做最终评价:

```
1 # 加载最优模型
2 runner.load("./checkpoint/resnet_cifar10_best.pt")
3 # 模型评价
4 loss, score = runner.evaluate(test_loader)
5 print(f"[Test] accuracy/loss: {score:.4f}/{loss:.4f}")
```

输出结果为:

```
[Test] accuracy/loss: 0.4370/1.5525
```

5.5.5 模型预测

最后用同一份模型对测试集的单条样本做预测, 对比真实类别与预测类别:

```
1 id2label = {0: 'airplane', 1: 'automobile', 2: 'bird', 3: 'cat', 4: 'deer',
2             5: 'dog', 6: 'frog', 7: 'horse', 8: 'ship', 9: 'truck'}
3 # 获取测试集中的一个批的数据
```



```
4 X, label_ids = next(iter(test_loader))
5 logits = runner.predict(X)
6 # 取出第3个样本的预测和真实类别
7 sample_id = 2
8 pred_class_id = logits[sample_id].argmax().item()
9 label_id = label_ids[sample_id].item()
10 pred_class = id2label[pred_class_id]
11 label = id2label[label_id]
12 # 输出真实类别与预测类别
13 print(f"The true category is {label} and the predicted category is {pred_class}")
```

输出结果为:

The true category is horse and the predicted category is horse

把该样本图像画出(绘图代码见配套 notebook),可视化预测图像如图5.22所示.



图 5.22 预测图像

动手练习 5.9

尝试使用 CIFAR-100 进行实验,观察不同模型在不同数据集上的学习效果.

5.5.6 进一步实践:从零训练还是迁移学习

当标注样本不多时,还可以把预训练模型作为另一个实验起点. torchvision 把权重与预处理约定关联起来,例如先取得 ResNet18_Weights.DEFAULT,再用该权重对象的 transforms() 处理图像. 加载预训练权重时,应先构建与权重相配的原始网络,再替换分类层;不能同时传入与预训练权重类别数不符的 num_classes=10,也不能把本例改过的入口直接当作原预训练网络.

迁移学习可以先冻结特征提取部分、只训练新分类层,也可以用较小学习率微调整个网络. 这里还要区分参数与状态:把参数设为 requires_grad=False 并不会停止 BatchNorm 更新运行统

计量. 比较两种方案时, 应明确模型模式、预处理、训练参数范围和验证集选择规则. 预训练模型可能提供有用特征, 收益仍需在当前数据上验证.

5.6 小结

本章从一个固定的边缘滤波器出发, 逐步实现多通道卷积和汇聚, 再用 LeNet 学习数字分类. 手写算子与框架算子的输出、梯度对照, 帮助我们确认计算是否正确; 训练与验证曲线则回答模型是否学到了有用的模式. 两类检查缺一不可.

残差网络实验进一步说明, 比较结构时必须控制学习率、初始化和数据顺序. CIFAR-10 案例则把问题推进了一步: 输入从灰度数字变成自然彩色图像后, 预处理和下采样策略都需要重新检查. 下一章转向序列数据, 输入之间的关系将由空间邻近变成时间先后.

第6章 循环神经网络

读到“这部电影的开头很平淡，但结尾让我改了主意”时，句子的后半段会改变我们对前半段的理解。处理序列时，模型也需要把先前的信息带到后面。循环神经网络（Recurrent Neural Network, RNN）用一个随输入更新的状态（State）完成这件事：每读入一个元素，就把它与上一时刻的状态结合。这个固定维度的状态能保留多少信息，又会在什么时候丢失信息，是本章要通过实验回答的问题。

本章对应《神经网络与深度学习》第6章循环神经网络。阅读前可先扫一眼图6.1中的关键知识点，方便概念与代码相互对照。

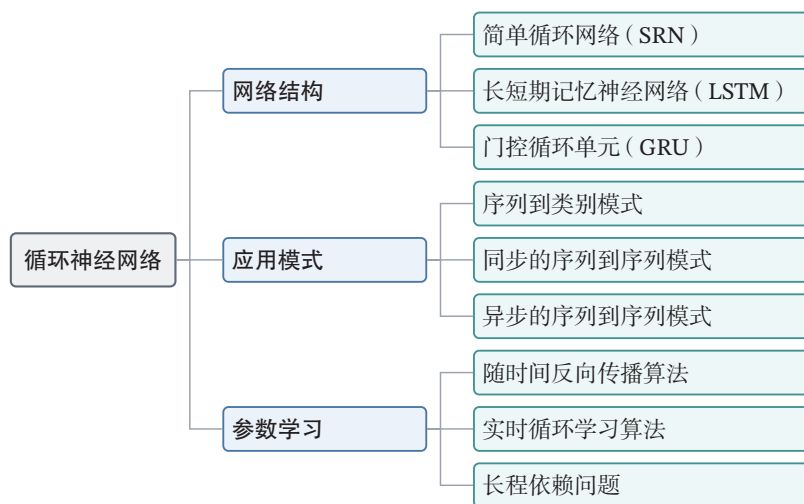


图 6.1 循环神经网络关键知识点回顾

本章分为两部分：

- **模型解读**：介绍经典 RNN 原理，并用一个简单的**数字求和任务**测试简单循环网络的记忆能力。围绕长程依赖问题，分别复现**梯度爆炸**并尝试缓解，再动手实现**长短期记忆网络（LSTM）**，观察其对**梯度消失**的改善效果。

- **案例实践**: 基于双向 LSTM 完成文本分类任务, 同时理解序列补齐、词向量化、补齐位置掩蔽等工程细节.

循环神经网络擅长处理长度可变的序列: 网络中带有自反馈的神经元, 使其能够维护一个跨时刻的状态. 给定序列 $[x_1, x_2, x_3, \dots]$, RNN 沿时间从左到右扫描, 每一步都调用同一个**组合函数** $f(\cdot)$, 这一函数被称为**循环神经网络单元** (RNN Cell). 时刻 t 时, 单元同时接收当前输入 $x_t \in \mathbb{R}^M$ 和前一时刻的隐状态 $h_{t-1} \in \mathbb{R}^D$, 输出新的隐状态 h_t :

$$h_t = f(h_{t-1}, x_t), \quad (6.1)$$

其中 $h_0 = 0$, $f(\cdot)$ 是非线性函数.

RNN 的参数在时间步之间共享. 将递推计算沿时间展开, 就得到一张可用反向传播求梯度的计算图, 这称为**随时间反向传播** (Backpropagation Through Time, BPTT). 自动微分框架替我们执行这套链式求导, 并没有绕开时间上的依赖关系.

长序列的困难在于: 末端损失对早期状态的影响, 要沿多个时间步的雅可比矩阵连乘传播. 乘积的某些方向可能不断放大, 也可能迅速缩小, 产生**梯度爆炸**或**梯度消失**. 早期信息是否能用于预测, 还取决于状态容量、输入干扰、初始化和训练方式, 不能仅由网络结构给出一个固定的“记忆长度”. 这些困难统称为**长程依赖问题** (Long-Term Dependencies Problem).

6.1 循环神经网络的记忆能力实验

RNN 最朴素的实现叫**简单循环网络** (Simple Recurrent Network, SRN).

记 $x_t \in \mathbb{R}^M$ 为时刻 t 的输入, $h_t \in \mathbb{R}^D$ 为同一时刻的隐藏层状态. h_t 同时取决于当前输入 x_t 和上一时刻的状态 h_{t-1} . SRN 的更新规则为

$$h_t = \tanh(Wx_t + Uh_{t-1} + b), \quad (6.2)$$

其中 $U \in \mathbb{R}^{D \times D}$ 是**状态-状态**权重矩阵, $W \in \mathbb{R}^{D \times M}$ 是**状态-输入**权重矩阵, $b \in \mathbb{R}^D$ 是偏置.

SRN 存在长程依赖问题, 对**长时间间隔** (Long Range) 的依赖建模乏力. 本节通过一个**数字求和**任务直观度量它的记忆长度.

任务规则: 输入是一串数字, 前两个数字为 0~9, 其余数字以 0 为主、零星填充随机数字; 模型需要预测前两个数字的和. 图 6.2 给出长度为 10 的样本示例.

数字序列										标签
1	2	0	0	0	0	0	0	0	0	3
2	4	0	0	3	0	0	0	0	0	6
3	2	0	0	0	0	1	0	0	0	5
4	1	0	0	0	0	0	0	4	0	5

图 6.2 数字求和任务示例

在相同数据生成规则和训练预算下, 较长序列上的表现可以提供信息保留能力的线索; 这不是与训练条件无关的容量测量. 下面通过让 SRN 在多种长度的数据集上分别训练、评估, 量化它的记忆距离.

这里还需要两个参照. 始终预测 9 的分类器准确率为 10%, 因为 100 种等可能的加数组合中有 10 种的和为 9; 只有均匀随机猜 19 类时, 期望准确率才是 $1/19$. 直接读取前两位并相加则能得到正确答案. 本任务有意让网络把信号带到末端, 测试的是这条传递过程, 并不检验对未见过的加数组合的算术泛化.

6.1.1 数据集构建

先构造不同序列长度的 DigitSum 数据集.

6.1.1.1 数据集的构建函数

前两位数字一共有 $10 \times 10 = 100$ 种组合, 可以一次性穷举; 剩下位置默认用 0 填充. 为了增加样本多样性, 我们再随机挑一个 0 填充位置, 把它替换为 $0 \sim 9$ 的随机数字.

通过参数 k 控制每个基础组合生成的样本数: $k = 3$ 用于训练集, $k = 5$ 用于验证集和测试集 (验证/测试集取较多样本, 可降低准确率估计的方差、让不同长度间的对比更稳定). 对每个长度都生成一套训练/验证/测试三个文件:

```

1 import random
2 import numpy as np
3 # 固定随机种子
4 random.seed(0)
5 np.random.seed(0)
6
7 def generate_data(length, k, save_path, seed=0):
8     rng = np.random.RandomState(seed)
9     if length < 3:
10         raise ValueError("The length of data should be greater than 2.")
11     if k <= 0:
```

```

12         raise ValueError("k should be greater than 0.")
13     # 生成长度为length的数字序列，除前两个字符外，序列其余数字暂用0填充
14     base_examples = []
15     for n1 in range(0, 10):
16         for n2 in range(0, 10):
17             seq = [n1, n2] + [0] * (length - 2)
18             label = n1 + n2
19             base_examples.append((seq, label))
20
21     # 用于记录数据增强后的数字序列
22     examples = []
23     for base_example in base_examples:
24         for _ in range(k):
25             # 随机生成替换的元素位置和元素
26             idx = rng.randint(2, length)
27             val = rng.randint(0, 10)
28             # 对序列中的对应0元素进行替换
29             seq = base_example[0].copy()
30             label = base_example[1]
31             seq[idx] = val
32             examples.append((seq, label))
33     # 保存增强后的数据；首次运行时创建父目录
34     from pathlib import Path
35     Path(save_path).parent.mkdir(parents=True, exist_ok=True)
36     with open(save_path, "w", encoding="utf-8") as f:
37         for example in examples:
38             # 将数据转为字符串类型，方便保存
39             seq = [str(e) for e in example[0]]
40             label = str(example[1])
41             line = " ".join(seq) + "\t" + label + "\n"
42             f.write(line)
43
44     print(f"generate data to: {save_path}.")
45     # 定义生成的数字序列长度
46     lengths = [5, 10, 15, 20, 25, 30, 35]
47     for length in lengths:
48         # 生成长度为length的训练数据
49         save_path = f"./datasets/{length}/train.txt"
50         k = 3
51         generate_data(length, k, save_path, seed=100 + length)
52         # 生成长度为length的验证数据
53         save_path = f"./datasets/{length}/dev.txt"
54         k = 5
55         generate_data(length, k, save_path, seed=200 + length)
56         # 生成长度为length的测试数据

```

```
57     save_path = f"./datasets/{length}/test.txt"
58     k = 5
59     generate_data(length, k, save_path, seed=300 + length)
```

6.1.1.2 加载指定长度数据集

先运行上一段生成代码,再按长度读取已保存的数据.同一次对照中的模型应复用这份数据;需要重新采样时,应同时记录生成种子和全部划分,避免只更新其中一个文件.

```
1  import os
2  # 加载数据
3  def load_data(data_path):
4      # 加载训练集
5      train_examples = []
6      train_path = os.path.join(data_path, "train.txt")
7      with open(train_path, "r", encoding="utf-8") as f:
8          for line in f.readlines():
9              # 解析一行数据,将其处理为数字序列seq和标签label
10             items = line.strip().split("\t")
11             seq = [int(i) for i in items[0].split(" ")]
12             label = int(items[1])
13             train_examples.append((seq, label))
14      # 加载验证集
15      dev_examples = []
16      dev_path = os.path.join(data_path, "dev.txt")
17      with open(dev_path, "r", encoding="utf-8") as f:
18          for line in f.readlines():
19              # 解析一行数据,将其处理为数字序列seq和标签label
20             items = line.strip().split("\t")
21             seq = [int(i) for i in items[0].split(" ")]
22             label = int(items[1])
23             dev_examples.append((seq, label))
24      # 加载测试集
25      test_examples = []
26      test_path = os.path.join(data_path, "test.txt")
27      with open(test_path, "r", encoding="utf-8") as f:
28          for line in f.readlines():
29              # 解析一行数据,将其处理为数字序列seq和标签label
30             items = line.strip().split("\t")
31             seq = [int(i) for i in items[0].split(" ")]
32             label = int(items[1])
33             test_examples.append((seq, label))
34
35      return train_examples, dev_examples, test_examples
36  # 设定加载的数据集的长度
```

```

37 length = 5
38 # 该长度的数据集的存放目录
39 data_path = f"./datasets/{length}"
40 # 加载该数据集
41 train_examples, dev_examples, test_examples = load_data(data_path)
42 print("dev example:", dev_examples[:2])
43 print("训练集数量: ", len(train_examples))
44 print("验证集数量: ", len(dev_examples))
45 print("测试集数量: ", len(test_examples))

```

输出结果为:

```

dev example: [[([0, 0, 0, 0, 4], 0), ([0, 0, 0, 0, 8], 0)]
训练集数量: 300
验证集数量: 500
测试集数量: 500

```

6.1.1.3 构造Dataset类

为配合`DataLoader`做小批量采样, 我们将 `DigitSum` 数据集封装为 PyTorch 的 `Dataset` 子类. 其中 `__getitem__` 按索引取出一条样本, 把数字序列与标签分别转为整型张量; `__len__` 返回总样本数. 实现如下:

```

1 from torch.utils.data import DataLoader, Dataset
2
3 class DigitSumDataset(Dataset):
4     def __init__(self, data):
5         self.data = data
6
7     def __getitem__(self, idx):
8         example = self.data[idx]
9         seq = torch.tensor(example[0], dtype=torch.int64)
10        label = torch.tensor(example[1], dtype=torch.int64)
11        return seq, label
12
13    def __len__(self):
14        return len(self.data)

```

6.1.2 模型构建

使用 SRN 进行数字求和任务的模型结构为如图 6.3 所示.

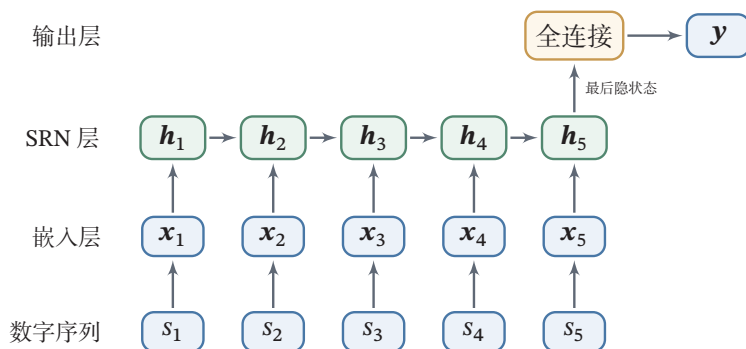


图 6.3 基于 SRN 模型的数字求和任务

整体由三段串联：

- 1) 嵌入层: 把每个数字查表为一个向量, 得到向量序列。
- 2) SRN 层: 沿时间步逐步刷新隐状态, 取末时刻输出作为整条序列的表示。
- 3) 输出层: 把序列表示映射到 19 个类别的未归一化分数。

6.1.2.1 嵌入层

本任务把数字当作离散符号, 为每个符号学习一个**嵌入 (Embedding)** 向量。向量不会增加原始输入的信息, 却提供了可学习的表示方式。直接输入数值也可以建模, 但会显式引入数值大小的关系, 两种表示对优化和泛化的影响可以通过实验比较。具体做法是维护一张**嵌入矩阵 (Embedding Matrix)** $E \in \mathbb{R}^{10 \times M}$, 第 i 行就是数字 i 的 M 维嵌入, 如图 6.4 所示。对于一批数字序列 $S \in \{0, \dots, 9\}^{B \times L}$ (B 为批大小, L 为序列长度), 按下标查表即可得到嵌入张量 $X \in \mathbb{R}^{B \times L \times M}$ 。

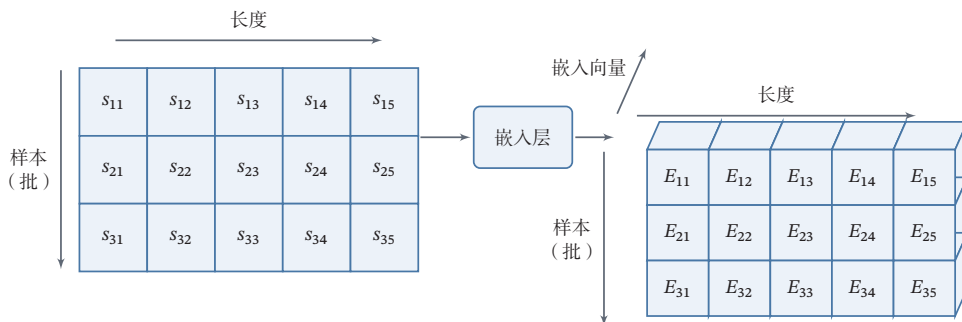


图 6.4 嵌入矩阵

提醒



本章张量约定与代码保持一致, 序列特征张量按“样本数 $\times$ 序列长度 $\times$ 特征维度”排列。单个时刻的状态没有序列轴; 框架返回的末状态还可能带有层数和方向轴, 需要分别核对。

也可以将每个数字表示为 10 维的 one-hot 向量,使用矩阵运算得到嵌入表示为

$$\mathbf{X} = \mathbf{S}'\mathbf{E}, \quad (6.3)$$

其中 $\mathbf{S}' \in \mathbb{R}^{B \times L \times 10}$ 是序列 $\mathbf{S}$ 对应的 one-hot 表示。



思考

如果不使用嵌入层,直接将数字作为 SRN 层的输入会有什么问题?



基于索引方式的嵌入层的代码实现如下:

```
1 import torch
2 import torch.nn as nn
3
4 class Embedding(nn.Module):
5     def __init__(self, num_embeddings, embedding_dim):
6         super().__init__()
7         # 定义嵌入矩阵
8         self.W = nn.Parameter(torch.randn(num_embeddings, embedding_dim, dtype=torch.
9                                         float32))
9
10    def forward(self, inputs):
11        # 根据索引获取对应嵌入向量
12        embs = self.W[inputs]
13        return embs
14
15 torch.manual_seed(0)
16 emb_layer = Embedding(10, 5)
17 inputs = torch.tensor([0,1,2,3])
18 emb_layer(inputs)
```

输出结果为:

```
tensor([[ -1.1258, -1.1524, -0.2506, -0.4339,  0.8487],
        [  0.6920, -0.3160, -2.1152,  0.3223, -1.2633],
        [  0.3500,  0.3081,  0.1198,  1.2377,  1.1168],
        [-0.2473, -1.3527, -1.6959,  0.5667,  0.7935]],
       grad_fn=<IndexBackward0>)
```



思考

请思考应该如何实现基于 one-hot 编码的嵌入层。



6.1.2.2 SRN 层

数字序列 $\mathbf{S} \in \{0, \dots, 9\}^{B \times L}$ 经嵌入层查表后变为 $\mathbf{X} \in \mathbb{R}^{B \times L \times M}$ (B 为批大小、 L 为序列长度、 M 为嵌入维度)。

在每个时间步 t , SRN 把当前输入 $\mathbf{X}_t \in \mathbb{R}^{B \times M}$ 与上一时刻隐状态 $\mathbf{H}_{t-1} \in \mathbb{R}^{B \times D}$ 分别做线性映射后相加, 再过一层非线性激活 $f(\cdot)$, 得到当前隐状态:

$$\mathbf{H}_t = \tanh(\mathbf{X}_t \mathbf{W} + \mathbf{H}_{t-1} \mathbf{U} + \mathbf{b}), \quad (6.4)$$

式中 $\mathbf{W} \in \mathbb{R}^{M \times D}$ 、 $\mathbf{U} \in \mathbb{R}^{D \times D}$ 、 $\mathbf{b} \in \mathbb{R}^{1 \times D}$ 均为可学习参数, D 为隐状态维度。

SRN 的代码实现如下:

```

1 import torch
2 import torch.nn as nn
3 import torch.nn.functional as F
4 # SRN模型
5 class SRN(nn.Module):
6     def __init__(self, input_size, hidden_size):
7         super().__init__()
8         # 嵌入向量的维度
9         self.input_size = input_size
10        # 隐状态的维度
11        self.hidden_size = hidden_size
12        # 定义模型参数; 权重用小尺度初始化 ( $\times 0.1$ ),
13        # 否则标准正态初始化会让  $\tanh$  一开始就饱和、SRN 难以训练
14        self.W = nn.Parameter(0.1 * torch.randn(input_size, hidden_size, dtype=torch.float32))
15        self.U = nn.Parameter(0.1 * torch.randn(hidden_size, hidden_size, dtype=torch.float32))
16        self.b = nn.Parameter(torch.zeros(1, hidden_size, dtype=torch.float32))
17        # 初始化向量
18        def init_state(self, batch_size):
19            # 用模型参数所在的device保证在GPU训练时隐状态也在GPU上;
20            # next(self.parameters())是PyTorch惯用法, 与子类参数命名解耦
21            ref = next(self.parameters())
22            return ref.new_zeros(batch_size, self.hidden_size)
23        # 定义前向计算
24        def forward(self, inputs, hidden_state=None):
25            batch_size, seq_len, input_size = inputs.shape
26            # 若未传入hidden_state, 则调用init_state进行初始化
27            if hidden_state is None:
28                hidden_state = self.init_state(batch_size)
29            # 循环执行RNN计算
30            for step in range(seq_len):
31                # 获取第step步的输入数据

```

```

32         step_input = inputs[:, step, :]
33         # 进行单元内的隐状态计算
34         hidden_state = torch.tanh(torch.matmul(step_input, self.W) + torch.matmul(
35             hidden_state, self.U) + self.b)
36     return hidden_state

```



提醒

为简化任务,这里只取最后一个时刻的隐状态作为输出,相当于把整段序列“摘要”成一个固定长度的向量。



动手练习 6.1

用`torch.nn.RNN`重写本节这个从零实现的SRN层,并验证两者在相同输入下输出完全一致。

1) 构造一组随机权重,分别赋给你手写的SRN (W 、 U 、 b) 和一个`nn.RNN`实例;注意`nn.RNN`的`weight_ih_l0`、`weight_hh_l0`与手写的 W 、 U 互为转置,且`nn.RNN`带`bias_ih_l0`与`bias_hh_l0`两份偏置,其有效偏置为二者之和。

2) 喂入同一批序列,取末时刻隐状态,比较两者输出的最大逐元素绝对差,确认手写实现与内置算子数值等价。

3) 思考: `nn.RNN`为何要拆成`bias_ih`与`bias_hh`两份偏置? 把它换进`Model_RNN4SeqClass`重跑数字求和任务,对比手写SRN与`nn.RNN`在单回合训练耗时上的差异。

6.1.2.3 线性层

把最后一个时刻的隐状态 $H_L \in \mathbb{R}^{B \times D}$ 送入一个线性层,得到各类别的未归一化分数(Logits):

$$Y = H_L W_o + b_o, \quad (6.5)$$

其中 $W_o \in \mathbb{R}^{D \times 19}$, $b_o \in \mathbb{R}^{19}$ 为可学习的权重矩阵和偏置。



提醒

工程上分类网络只输出未归一化分数(Logits)即可,不必显式过Softmax;PyTorch的`CrossEntropyLoss`内部已包含log-Softmax,直接接收未归一化分数更省也更稳。

线性层直接复用`torch.nn.Linear`。

6.1.2.4 模型汇总

把嵌入层、SRN层和线性层串起来,得到完整的数字求和模型`Model_RNN4SeqClass`。

构造时由外部传入一个具体的 RNN 算子（这里是 SRN），内部再分别构造嵌入层和线性层；forward 负责把数字序列依次过嵌入、RNN、线性输出。



提醒

为了便于做对比实验（例如把 SRN 换成 LSTM），我们把 RNN 算子的实例化放在 Model_RNN4SeqClass 外部，依赖注入的方式更灵活。常规写法中通常会把它放在模型内部。

完整实现如下：

```
1 # 基于RNN实现数字求和的模型
2 class Model_RNN4SeqClass(nn.Module):
3     def __init__(self, model, num_digits, input_size, hidden_size, num_classes):
4         super().__init__()
5         # 传入实例化的RNN层，例如SRN
6         self.rnn_model = model
7         # 词表大小，这里为10，即不同数字的数量
8         self.num_digits = num_digits
9         # 嵌入向量的维度
10        self.input_size = input_size
11        # 定义嵌入层
12        self.embedding = Embedding(num_digits, input_size)
13        # 定义线性层
14        self.linear = nn.Linear(hidden_size, num_classes)
15
16    def forward(self, inputs):
17        # 将数字序列映射为相应向量
18        inputs_emb = self.embedding(inputs)
19        # 调用RNN模型
20        hidden_state = self.rnn_model(inputs_emb)
21        # 使用最后一个时刻的状态进行数字求和
22        logits = self.linear(hidden_state)
23        return logits
```

6.1.3 模型训练

先把手写层的参数复制到框架优化算子，再进行重复训练。配套的NativeRecurrent适配器对 SRN 转置 W 、 U ；对 LSTM 按输入门、遗忘门、候选状态、输出门的顺序拼接参数。框架算子有两份偏置，适配器把第二份固定为 0，使有效偏置和可训练参数数量与手写层一致。前向与梯度对照代码见配套 notebook。

借助 RunnerV3 进行训练：通过 length 参数选择目标长度的数据集；优化器使用 Adam（学习率 0.001）；评价指标取 accuracy；训练 200 个回合：

```
1 import os
2 import random
3 import torch
4 import numpy as np
5 from nndl import RunnerV3, accuracy
6 from nndl.rnn import NativeRecurrent
7 # 为了实验的可重复性, 这里将所有的随机种子固定
8 np.random.seed(0)
9 random.seed(0)
10 seed = 0
11 torch.manual_seed(seed)
12 # 输入数字的类别数
13 num_digits = 10
14 # 将数字映射为向量的维度
15 input_size = 32
16 # 隐状态向量的维度
17 hidden_size = 32
18 # 数字求和的类别数
19 num_classes = 19
20 # 批大小
21 batch_size = 32
22 # 可以设置不同的length进行不同长度数据的求和实验
23 length = 10
24 print(f"\n====> Training SRN with data of length {length}.")
25 # 加载长度为length的数据
26 data_path = f"./datasets/{length}"
27 train_examples, dev_examples, test_examples = load_data(data_path)
28 train_set, dev_set, test_set = DigitSumDataset(train_examples), DigitSumDataset(
    dev_examples), DigitSumDataset(test_examples)
29 train_loader = DataLoader(train_set, batch_size=batch_size, shuffle=True,
30                             generator=torch.Generator().manual_seed(seed))
31 dev_loader = DataLoader(dev_set, batch_size=128)
32 test_loader = DataLoader(test_set, batch_size=128)
33 # 实例化模型
34 base_model = SRN(input_size, hidden_size)
35 torch.manual_seed(1000 + seed) # 两类模型共享嵌入层与输出层的初始参数
36 model = Model_RNN4SeqClass(base_model, num_digits, input_size, hidden_size, num_classes)
37 model.rnn_model = NativeRecurrent(base_model, kind="rnn")
38 # 指定优化器
39 optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
40 # 定义评价指标
41 metric = accuracy
42 # 定义损失函数
43 loss_fn = nn.CrossEntropyLoss()
```

```
44 # 基于以上组件，实例化Runner
45 runner = RunnerV3(model, optimizer, loss_fn, metric_fn=metric)
46 # 模型保存目录
47 save_dir = "./checkpoints"
48 model_save_path = os.path.join(save_dir, f"best_srn_model_{length}_seed{seed}.pt")
49 # 模型训练
50 runner.fit(train_loader, dev_loader, num_epochs=200, log_every=50, best_path=
    model_save_path)
```



动手练习 6.2

依次把length改为15、20、25、30、35,重复上面的训练流程,并保存各自的实验结果,便于下面统一观察长度对模型表现的影响。



图6.5汇总6种长度、3个训练种子(0、1、2)的实验。数据生成种子保持固定,每次训练200个回合,按各次验证准确率保存最佳快照。曲线为同一回合损失的均值,阴影为上下一个样本标准差,反映初始化和洗牌造成的波动。较长序列上即使训练损失下降,验证损失仍可能上升;这说明当前训练条件下的泛化困难,不能仅凭曲线确定模型遗忘了哪个位置。

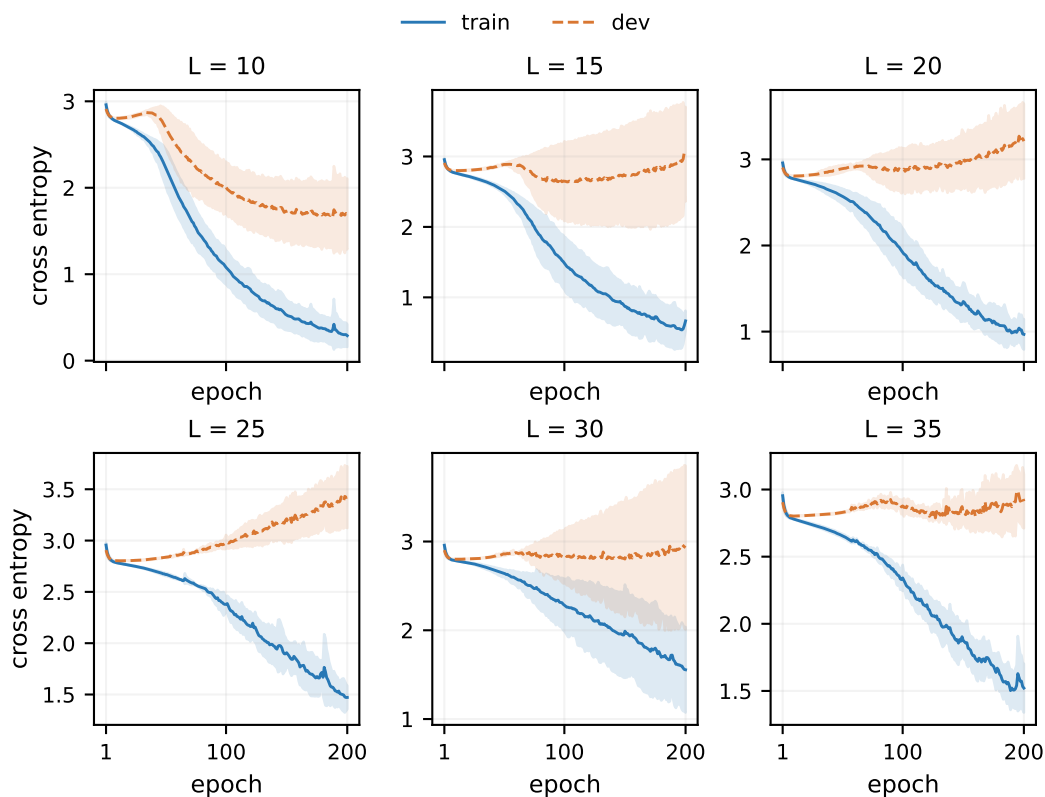


图 6.5 SRN 在不同长度下的损失:实线为训练、虚线为验证;阴影为三个随机种子的均值上下一个标准差

6.1.4 模型评价

载入训练阶段保留的最优模型,在测试集上评估准确率:

```
1 print(f"Evaluate SRN with data length {length}.")
2 # 加载训练过程中效果最好的模型
3 model_path = os.path.join(save_dir, f"best_srn_model_{length}_seed{seed}.pt")
4 runner.load(model_path)
5 # 使用测试集评价模型
6 _, score = runner.evaluate(test_loader)
7 print(f"[SRN] length:{length}, Score: {score: .5f}")
```

图6.6使用同一批实验中的最佳快照,分别评价验证集和测试集,并画出三个种子的均值及标准差。总体上,SRN 在较长序列上的准确率下降,但个别长度和种子会出现起伏。虚线给出始终预测 9 的 10% 基线;判断网络是否学到有效信息,应先与这条参照比较。

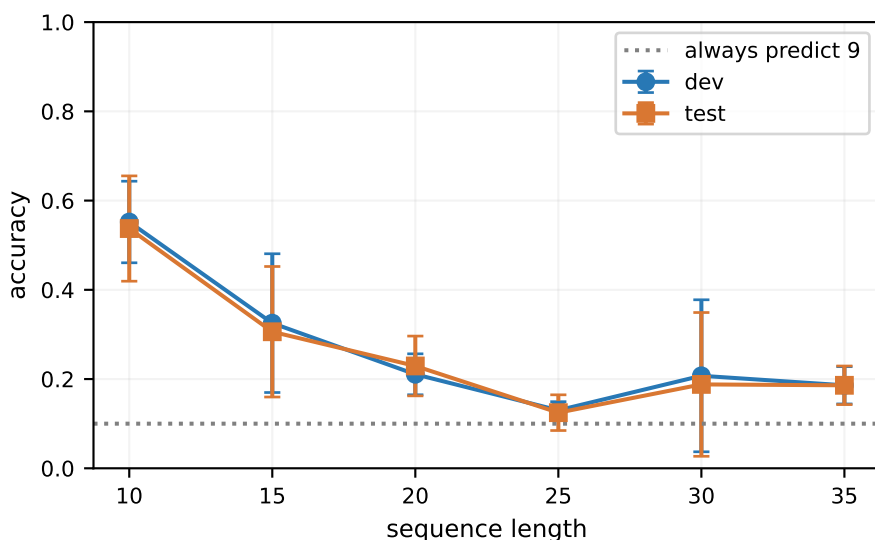


图 6.6 SRN 在不同长度的验证集和测试集的准确率变化

动手练习 6.3

参考《神经网络与深度学习》主书公式 (6.50), 给 SRN 的循环单元加上隐状态之间的残差连接 (相当于一种迷你“跳连”), 重新跑数字求和实验, 看看长程依赖是否能得到缓解。

6.2 梯度爆炸实验

SRN 处理长程依赖时遇到的两大障碍是**梯度爆炸**和**梯度消失**。梯度截断可以限制过大的梯度, 减少不稳定更新; 梯度消失则通常需要从网络结构和初始化等方面缓解。LSTM 的门控与加性状态更新为长期信息和梯度提供了更通畅的路径, 但并不能保证在任意长序列上都消除梯度消失。

先把时间传播与优化器更新分开观察, 再对完整 SRN 做训练压力测试。前者检查链式求导如何随序列长度变化, 后者检查大步更新会怎样影响训练, 以及梯度截断能限制什么。

6.2.1 先观察时间传播本身

考虑标量递推 $h_t = \tanh(a h_{t-1})$, 并令 $h_0 = 0$ 。此时每一步的状态都为 0, $\tanh$ 在 0 处的导数为 1, 因此 $\partial h_T / \partial h_0 = a^T$ 。下面用自动微分验证:

```
1 lengths_for_grad = list(range(1, 61))
2 temporal_grads = {}
```

```

3 for gain in (0.5, 1.1):
4     values = []
5     for steps in lengths_for_grad:
6         h0 = torch.zeros((), dtype=torch.float64, requires_grad=True)
7         h = h0
8         for _ in range(steps):
9             h = torch.tanh(gain * h)
10            grad, = torch.autograd.grad(h, h0)
11            values.append(abs(grad.item()))
12    temporal_grads[gain] = values

```

图6.7中,一个梯度随长度指数衰减,另一个指数增长. 这个例子特意把状态固定在0,便于单独观察乘积效应;一般情况下,激活函数的导数也参与连乘,不能仅凭循环权重的大小判断结果.

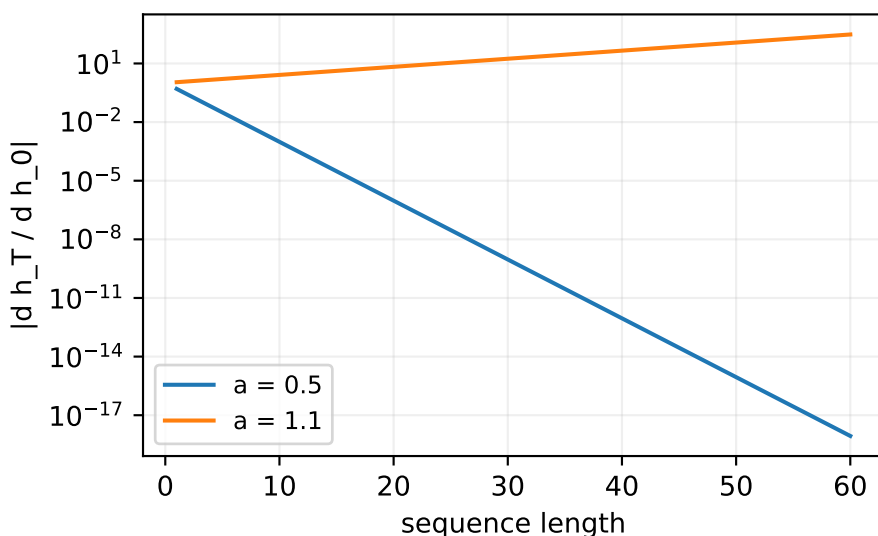


图 6.7 标量递推中,末状态对初状态的梯度随长度变化

6.2.2 梯度范数记录函数

训练实验还需要在反向传播后、参数更新前记录梯度. 下面分别保存全部参数梯度拼接后的整体范数,以及SRN中 $\mathbf{W}$ 、 $\mathbf{U}$ 、 $\mathbf{b}$ 的范数. 启用截断时,同时记录截断前后的整体范数,避免把“截断前仍然很大”误认为截断没有生效.

```

1 # 记录SRN循环单元中关键参数的梯度范数
2 def record_grad_norms(model, traces):
3     name_map = {
4         "rnn_model.W": "W_grad_l2",
5         "rnn_model.U": "U_grad_l2",

```

```

6         "rnn_model.b": "b_grad_l2",
7     }
8     for name, param in model.named_parameters():
9         if name in name_map and param.grad is not None:
10             traces[name_map[name]].append(param.grad.detach().norm().item())
11
12 def train_with_grad_trace(model, train_loader, optimizer, loss_fn,
13                           num_epochs=20, clip_norm=None):
14     traces = {"W_grad_l2": [], "U_grad_l2": [], "b_grad_l2": [],
15             "total_before": [], "total_after": [], "loss": []}
16     model.train()
17     for epoch in range(num_epochs):
18         for x, y in train_loader:
19             optimizer.zero_grad()
20             loss = loss_fn(model(x), y)
21             loss.backward()
22             grads = [p.grad for p in model.parameters() if p.grad is not None]
23             total = torch.stack([g.norm() for g in grads]).norm().item()
24             traces["total_before"].append(total)
25             traces["loss"].append(loss.item())
26             if clip_norm is not None:
27                 nn.utils.clip_grad_norm_(model.parameters(), clip_norm, error_if_nonfinite=
28                                         True)
29             traces["total_after"].append(torch.stack([g.norm() for g in grads]).norm().item
30                                           ())
31             record_grad_norms(model, traces)
32             optimizer.step()
33     return traces

```

得到traces后，把三组梯度范数按训练步画成对数坐标曲线即可（绘图代码见配套 notebook）。

6.2.3 复现梯度爆炸问题

压力测试采用长度 20 的序列，把批大小设为 64、SGD 学习率设为 0.2，并将交叉熵的reduction设为sum。求和损失会直接放大相对于平均损失的梯度，较大的学习率则放大参数更新；这两种人为放大不能单独作为时间传播发生梯度爆炸的证据。我们用它检查不稳定更新及其后果。

```

1 import os
2 import random
3 import torch
4 import torch.nn as nn
5 import numpy as np

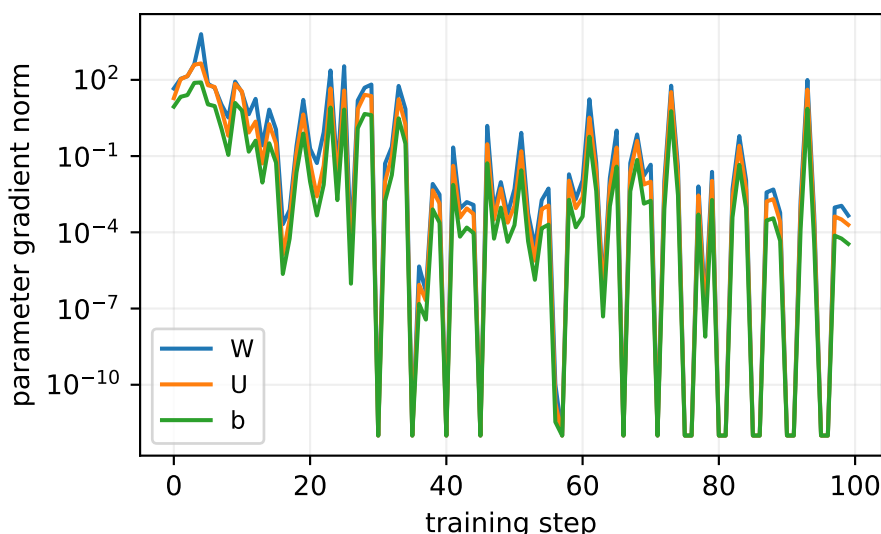
```

```

6  # 为了实验的可重复性, 这里将所有的随机种子固定
7  np.random.seed(0)
8  random.seed(0)
9  torch.manual_seed(0)
10 # 输入数字的类别数
11 num_digits = 10
12 # 将数字映射为向量的维度
13 input_size = 32
14 # 隐状态向量的维度
15 hidden_size = 32
16 # 数字求和的类别数
17 num_classes = 19
18 # 批大小
19 batch_size = 64
20 # 可以设置不同的length进行不同长度数据的求和实验
21 length = 20
22 print(f"\n====> Training SRN with data of length {length}.")
23 # 加载长度为length的数据
24 data_path = f"./datasets/{length}"
25 train_examples, dev_examples, test_examples = load_data(data_path)
26 train_set, dev_set, test_set = DigitSumDataset(train_examples), DigitSumDataset(
    dev_examples), DigitSumDataset(test_examples)
27 train_loader = DataLoader(train_set, batch_size=batch_size)
28 dev_loader = DataLoader(dev_set, batch_size=batch_size)
29 test_loader = DataLoader(test_set, batch_size=batch_size)
30 # 实例化模型
31 base_model = SRN(input_size,hidden_size)
32 model = Model_RNN4SeqClass(base_model, num_digits, input_size,hidden_size, num_classes)
33 # 指定优化器
34 optimizer = torch.optim.SGD(model.parameters(), lr=0.2)
35 # 定义损失函数
36 loss_fn = nn.CrossEntropyLoss(reduction="sum")
37 # 模型训练并记录梯度
38 grad_traces = train_with_grad_trace(model, train_loader, optimizer, loss_fn,
39                                     num_epochs=20)

```

图6.8记录这次压力测试中循环层参数的梯度。绘图采用对数坐标, 低于 10^{-12} 的范数显示在该下限处; 原始记录仍保留实际值。尖峰、衰减或接近0都需要结合状态和损失来解释: 例如Tanh进入饱和和区后, 导数会很小。只记录循环层还不够, 即使它们的梯度接近0, 输出层的梯度也可能仍很大, 因此下一图同时给出整体范数。

图 6.8 SRN 在训练过程中参数梯度的 ℓ_2 范数变化

6.2.4 使用梯度截断解决梯度爆炸问题

梯度截断 (Gradient Clipping, 也称梯度裁剪) 是限制过大梯度的常用手段: 一旦梯度的模超过预设阈值, 就把它压回预设范围. 常见做法有两种——按值截断 (对每个分量做硬性裁剪) 和按模截断 (对整体范数做缩放), 本实验采用后者.

按模截断针对的是梯度向量 $\mathbf{g}$ 的整体范数, 要求其模不超过阈值 b , 裁剪规则为

$$\mathbf{g} = \begin{cases} \mathbf{g} & \|\mathbf{g}\| \leq b \\ \frac{b}{\|\mathbf{g}\|} \mathbf{g} & \|\mathbf{g}\| > b \end{cases} \quad (6.6)$$

即模长不超过 b 时原样保留, 超过则等比例缩小到 b , 方向不变、长度受限.

笔记



在 PyTorch 中, 可以在反向传播之后、优化器更新之前调用 `torch.nn.utils.clip_grad_norm` 进行按模截断. 配套的 RunnerV3 提供了 `grad_clip_norm` 参数, 会在 `loss.backward()` 之后、`optimizer.step()` 之前执行这一操作.

下面把按模截断接入训练循环, 再跑一遍同样的实验来对照效果. 重新实例化模型与优化器, 把 `clip_norm` 传入 `train_with_grad_trace`——这里记录到的梯度范数已经是截断之后的版本, 也就是真正会影响参数更新的那一份. 代码如下:

```

1  # 与未截断实验使用相同的初始参数
2  torch.manual_seed(0)
3  # 实例化模型
4  base_model = SRN(input_size,hidden_size)
5  model = Model_RNN4SeqClass(base_model, num_digits, input_size,hidden_size, num_classes)
6  # 实例化优化器
7  optimizer = torch.optim.SGD(model.parameters(), lr=0.2)
8  # 定义损失函数
9  loss_fn = nn.CrossEntropyLoss(reduction="sum")
10 # 模型训练并记录截断后的梯度
11 clip_grad_traces = train_with_grad_trace(model, train_loader, optimizer, loss_fn,
12                                         num_epochs=20, clip_norm=5.0)

```

图6.9对照截断前后的整体范数。有限梯度超过5时会被按比例缩小；未达到阈值时保持原样。这限制的是本次交给优化器的梯度，并不保证损失下降。对带动量或自适应缩放的优化器，参数更新还取决于优化器状态，也不能把梯度阈值直接当作参数步长上限。遇到NaN或无穷大时，应先定位数值问题，截断不能把它们恢复成有效梯度。

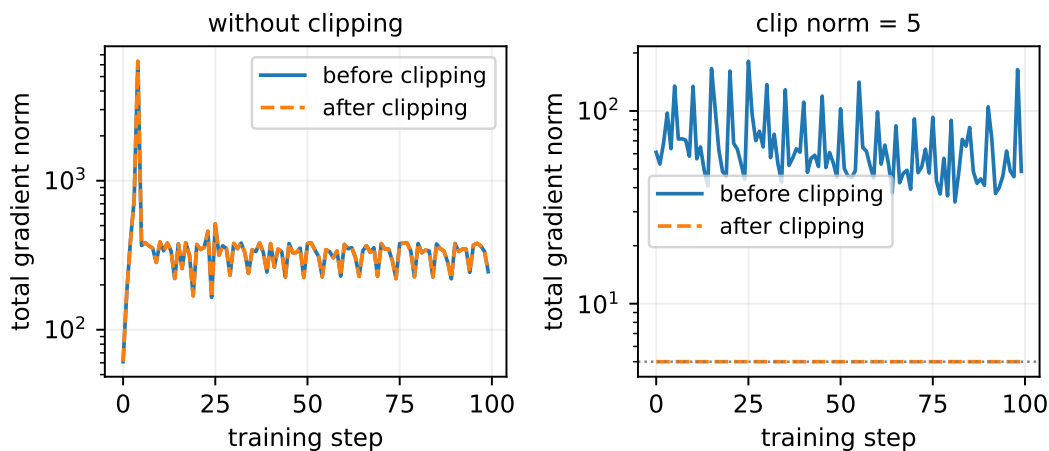


图 6.9 压力测试中,截断前后的整体梯度范数

动手练习 6.4

复用 `train_with_grad_trace`, 进一步探究梯度截断阈值与截断方式对训练稳定性的影响。

1) 把按模截断的阈值 `clip_norm` 依次设为 0.5、1.0、5.0、20.0, 分别重跑梯度记录实验, 比较各设置下 $\mathbf{W}$ 、 $\mathbf{U}$ 、 $\mathbf{b}$ 梯度 ℓ_2 范数的轨迹, 分析阈值过小或过大各自带来的现象。

2) 把每个阈值下训练得到的模型在验证集上评估准确率, 观察“截得太狠”是否反而拖慢收敛、“几乎不截”是否又压不住爆炸, 据此谈谈如何为这个任务挑一个合适的阈值。

3) 把“按模截断”换成“按值截断”(`torch.nn.utils.clip_grad_value_`), 在相同配置下重跑, 对比两种策略在梯度范数轨迹与训练稳定性上的差异。



6.3 LSTM 的记忆能力实验

长短期记忆网络 (Long Short-Term Memory, LSTM) 专门用于缓解 SRN 的长程依赖问题, 核心做了两点改造: 第一, 引入额外的**细胞状态** (Cell State, 也称内部状态) $\mathbf{c} \in \mathbb{R}^D$, 让旧状态经过遗忘门缩放后与新信息相加; 第二, 加入一组**门控机制** (Gating Mechanism) ——**输入门**过滤当前输入、**遗忘门**选择性丢弃历史、**输出门**决定向下游暴露多少, 让网络可以自己学会“记什么、忘什么、说什么”。

回到数字求和任务: 有效信号出现在开头, 后续输入大多无关。门控是否学会保留前两个数字的信息, 需要看模型的实际预测与对照实验, 而不能只凭结构推断。

LSTM 循环单元在第 t 步的内部结构见图 6.10。

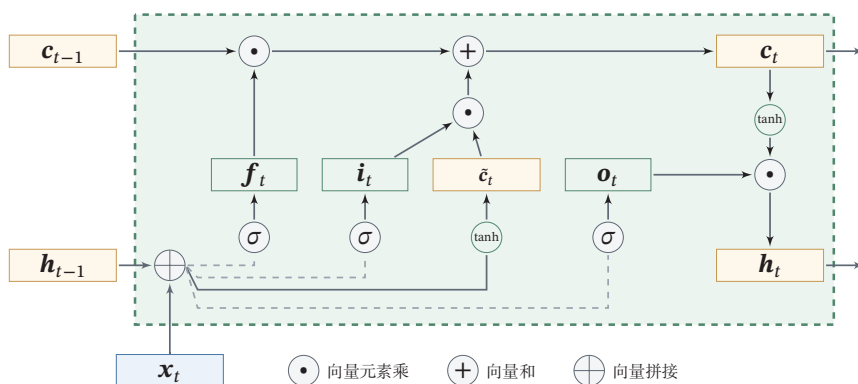


图 6.10 LSTM 网络的循环单元结构, 虚线框内为一个循环单元



提醒

本章张量约定与代码保持一致, 序列特征张量按“样本数 $\times$ 序列长度 $\times$ 特征维度”排列. 单个时刻的状态没有序列轴; 框架返回的末状态还可能带有层数和方向轴, 需要分别核对.

设输入序列张量为 $\mathbf{X} \in \mathbb{R}^{B \times L \times M}$ (B 批量、 L 序列长度、 M 输入维度), LSTM 沿时间扫描, 依次更新内部状态 $\mathbf{C}_t \in \mathbb{R}^{B \times D}$ 与隐状态 $\mathbf{H}_t \in \mathbb{R}^{B \times D}$.

整个过程可以拆为三步:

(1) 计算三个门. 时刻 t 用当前输入 $\mathbf{X}_t$ 与上一时刻隐状态 $\mathbf{H}_{t-1}$ 同时算出输入门 $\mathbf{I}_t$ 、遗忘门 $\mathbf{F}_t$ 、输出门 $\mathbf{O}_t$:

$$\mathbf{I}_t = \sigma(\mathbf{X}_t \mathbf{W}_i + \mathbf{H}_{t-1} \mathbf{U}_i + \mathbf{b}_i) \in \mathbb{R}^{B \times D}, \quad (6.7)$$

$$\mathbf{F}_t = \sigma(\mathbf{X}_t \mathbf{W}_f + \mathbf{H}_{t-1} \mathbf{U}_f + \mathbf{b}_f) \in \mathbb{R}^{B \times D}, \quad (6.8)$$

$$\mathbf{O}_t = \sigma(\mathbf{X}_t \mathbf{W}_o + \mathbf{H}_{t-1} \mathbf{U}_o + \mathbf{b}_o) \in \mathbb{R}^{B \times D}, \quad (6.9)$$

其中 $\mathbf{W}_*$ 、 $\mathbf{U}_*$ 、 $\mathbf{b}_*$ 均为可学习参数; σ 为 Logistic 函数, 把每个门的取值压到 $(0, 1)$, 相当于一个软性的“开关”.

(2) 计算内部状态. 先算出候选内部状态:

$$\tilde{\mathbf{C}}_t = \tanh(\mathbf{X}_t \mathbf{W}_c + \mathbf{H}_{t-1} \mathbf{U}_c + \mathbf{b}_c) \in \mathbb{R}^{B \times D}, \quad (6.10)$$

其中 $\mathbf{W}_c$ 、 $\mathbf{U}_c$ 、 $\mathbf{b}_c$ 是候选内部状态特有的可学习参数 (与三个门相区别, 同样在各时间步间共享).

接着用遗忘门控制上一时刻状态 $\mathbf{C}_{t-1}$ 的保留比例, 用输入门控制候选状态 $\tilde{\mathbf{C}}_t$ 的写入比例, 得到当前内部状态:

$$\mathbf{C}_t = \mathbf{F}_t \odot \mathbf{C}_{t-1} + \mathbf{I}_t \odot \tilde{\mathbf{C}}_t, \quad (6.11)$$

其中 $\odot$ 表示逐元素积 (Element-wise Product).

(3) 计算输出状态. 内部状态 $\mathbf{C}_t$ 再经 $\tanh$ 压缩并由输出门调节, 得到当前时刻可用于预测并传给下一时刻的隐状态 $\mathbf{H}_t$:

$$\mathbf{H}_t = \mathbf{O}_t \odot \tanh(\mathbf{C}_t). \quad (6.12)$$

汇总起来, LSTM 每一步以 $(\mathbf{C}_{t-1}, \mathbf{H}_{t-1})$ 为输入, 更新为 $(\mathbf{C}_t, \mathbf{H}_t)$. 沿内部状态的直接路径求导时, 每一步保留因子为遗忘门; 若忽略经过门和隐状态的其他路径, 跨多步的因子就是遗忘门的逐元素乘积. 门值接近 1 时有利于保留信息, 持续小于 1 时仍会衰减. 完整梯度还包括其他路径, 因此加性更新缓解了问题, 却不能保证梯度永不消失.

门控的开合程度由数据驱动学出来, 网络据此在不同上下文中决定保留什么、丢弃什么.

下面用 LSTM 重做一次数字求和实验, 直观对比它与 SRN 在长程依赖上的差距.

6.3.1 模型构建

复用第6.1.2.4节实现的Model_RNN4SeqClass: 只需新增一个LSTM算子, 再把它实例传入即可完成模型组装。

6.3.1.1 LSTM层

LSTM算子整体结构与SRN类似, 区别是为每一种门额外维护一组权重, 并在每一步内额外更新内部状态. 和SRN一样, LSTM算子只返回序列最后一时刻的隐状态:

```

1  import torch.nn.functional as F
2  # 声明LSTM和相关参数
3  class LSTM(nn.Module):
4      def __init__(self, input_size, hidden_size):
5          super().__init__()
6          self.input_size = input_size
7          self.hidden_size = hidden_size
8          # 初始化模型参数
9          self.W_i = nn.Parameter(torch.empty(input_size, hidden_size, dtype=torch.float32))
10         self.W_f = nn.Parameter(torch.empty(input_size, hidden_size, dtype=torch.float32))
11         self.W_o = nn.Parameter(torch.empty(input_size, hidden_size, dtype=torch.float32))
12         self.W_c = nn.Parameter(torch.empty(input_size, hidden_size, dtype=torch.float32))
13         self.U_i = nn.Parameter(torch.empty(hidden_size, hidden_size, dtype=torch.float32))
14         self.U_f = nn.Parameter(torch.empty(hidden_size, hidden_size, dtype=torch.float32))
15         self.U_o = nn.Parameter(torch.empty(hidden_size, hidden_size, dtype=torch.float32))
16         self.U_c = nn.Parameter(torch.empty(hidden_size, hidden_size, dtype=torch.float32))
17         self.b_i = nn.Parameter(torch.zeros(1, hidden_size, dtype=torch.float32))
18         self.b_f = nn.Parameter(torch.zeros(1, hidden_size, dtype=torch.float32))
19         self.b_o = nn.Parameter(torch.zeros(1, hidden_size, dtype=torch.float32))
20         self.b_c = nn.Parameter(torch.zeros(1, hidden_size, dtype=torch.float32))
21         # 控制初始权重尺度; 遗忘门偏置设为1是一项实验选择
22         for name, param in self.named_parameters():
23             if not name.startswith("b_"):
24                 nn.init.uniform_(param, -hidden_size**-0.5, hidden_size**-0.5)
25                 nn.init.ones_(self.b_f)
26         # 初始化状态向量和隐状态向量
27         def init_state(self, batch_size):
28             # 用模型参数所在的device保证在GPU训练时(h, c)也在GPU上
29             ref = next(self.parameters())
30             hidden_state = ref.new_zeros(batch_size, self.hidden_size)
31             cell_state = ref.new_zeros(batch_size, self.hidden_size)
32             return hidden_state, cell_state
33         # 定义前向计算
34         def forward(self, inputs, states=None):
35             batch_size, seq_len, input_size = inputs.shape
36             # 若未传入states, 则调用init_state进行初始化

```

```

37         if states is None:
38             states = self.init_state(batch_size)
39             hidden_state, cell_state = states
40             # 执行LSTM计算, 包括: 输入门、遗忘门、输出门、候选内部状态、内部状态和隐状态
41             for step in range(seq_len):
42                 input_step = inputs[:, step, :]
43                 I_gate = torch.sigmoid(torch.matmul(input_step, self.W_i) + torch.matmul(
44                     hidden_state, self.U_i) + self.b_i)
45                 F_gate = torch.sigmoid(torch.matmul(input_step, self.W_f) + torch.matmul(
46                     hidden_state, self.U_f) + self.b_f)
47                 O_gate = torch.sigmoid(torch.matmul(input_step, self.W_o) + torch.matmul(
48                     hidden_state, self.U_o) + self.b_o)
49                 C_tilde = torch.tanh(torch.matmul(input_step, self.W_c) + torch.matmul(
50                     hidden_state, self.U_c) + self.b_c)
51                 cell_state = F_gate * cell_state + I_gate * C_tilde
52                 hidden_state = O_gate * torch.tanh(cell_state)
53             return hidden_state

```

6.3.1.2 模型汇总

包装在外的Model_RNN4SeqClass与SRN实验完全一致, 唯一区别是构造时把RNN参数换成LSTM实例。



动手练习 6.5

上面的 LSTM 算子是逐时刻循环计算的, 运行效率不高. 请思考如何写一个更高效的 LSTM 实现.



6.3.2 模型训练

对照实验使用相同的数据、嵌入与隐状态维度、批大小 32、Adam 学习率 0.001 和 200 个训练回合, 并使嵌入层及输出层的初值一致. LSTM 有更多可训练参数, 循环结构与初始化规则也不同; 这里比较的是明确配置下的两类模型, 不能把全部差异仅归因于某一个门。

```

1 import os
2 import random
3 import torch
4 import numpy as np
5 from nndl import RunnerV3, accuracy
6 from nndl.rnn import NativeRecurrent
7
8 np.random.seed(0)
9 random.seed(0)

```

```

10 seed = 0
11 torch.manual_seed(seed)
12 # 输入数字的类别数
13 num_digits = 10
14 # 将数字映射为向量的维度
15 input_size = 32
16 # 隐状态向量的维度
17 hidden_size = 32
18 # 数字求和的类别数
19 num_classes = 19
20 # 批大小
21 batch_size = 32
22 # 模型保存目录
23 save_dir = "./checkpoints"
24 # 可以设置不同的length进行不同长度数据的求和实验
25 length = 10
26 print(f"\n====> Training LSTM with data of length {length}.")
27 # 加载长度为length的数据
28 data_path = f"./datasets/{length}"
29 train_examples, dev_examples, test_examples = load_data(data_path)
30 train_set, dev_set, test_set = DigitSumDataset(train_examples), DigitSumDataset(
    dev_examples), DigitSumDataset(test_examples)
31 train_loader = DataLoader(train_set, batch_size=batch_size, shuffle=True,
32                             generator=torch.Generator().manual_seed(seed))
33 dev_loader = DataLoader(dev_set, batch_size=128)
34 test_loader = DataLoader(test_set, batch_size=128)
35 # 实例化模型
36 base_model = LSTM(input_size,hidden_size)
37 torch.manual_seed(1000 + seed) # 两类模型共享嵌入层与输出层的初始参数
38 model = Model_RNN4SeqClass(base_model, num_digits, input_size,hidden_size, num_classes)
39 model.rnn_model = NativeRecurrent(base_model, kind="lstm")
40 # 指定优化器
41 optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
42 # 定义评价指标
43 metric = accuracy
44 # 定义损失函数
45 loss_fn = nn.CrossEntropyLoss()
46 # 实例化Runner
47 runner = RunnerV3(model, optimizer, loss_fn, metric_fn=metric)
48 # 模型训练
49 model_save_path = os.path.join(save_dir, f"best_lstm_model_{length}_seed{seed}.pt")
50 runner.fit(train_loader, dev_loader, num_epochs=200, log_every=50, best_path=
    model_save_path)

```

图6.11与SRN图使用相同的长度、数据和训练种子. 比较时既看均值, 也看阴影范围. LSTM在

这些配置下通常能得到更低的验证损失, 但训练损失与验证损失仍有差距, 门控并没有消除小样本下的过拟合。

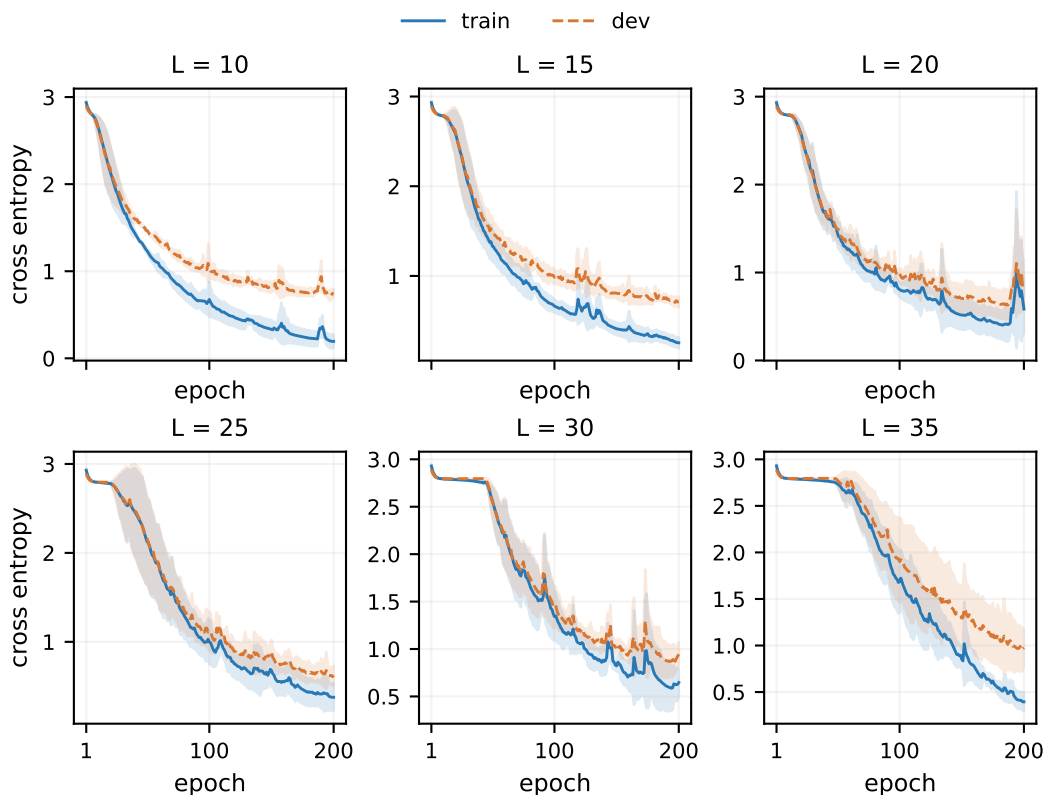


图 6.11 LSTM 在不同长度下的损失; 曲线与阴影采用与 SRN 相同的统计口径



动手练习 6.6

请扩展第 6.3.1.1 节实现的 LSTM 算子, 使其支持双向 (bidirectional) 模式, 把正、反两个方向的隐状态拼接作为最终输出。

6.3.3 模型评价

载入训练阶段保存的最优模型, 在测试集上评估准确率:

```
1 print(f"Evaluate LSTM with data length {length}.")
2 # 加载训练过程中效果最好的模型
3 model_path = os.path.join(save_dir, f"best_lstm_model_{length}_seed{seed}.pt")
4 runner.load(model_path)
5 # 使用测试集评价模型
```

```

6 _, score = runner.evaluate(test_loader)
7 print(f"[LSTM] length:{length}, Score: {score: .5f}")

```

图6.12给出同一批实验的测试结果。长度10时，SRN与LSTM的平均准确率分别约为53.7%和83.3%；长度35时分别约为18.6%和77.3%。本次实验中LSTM在长序列上保留了更多用于预测的信息，仍不能据此保证任意长度或其他数据分布上的表现。

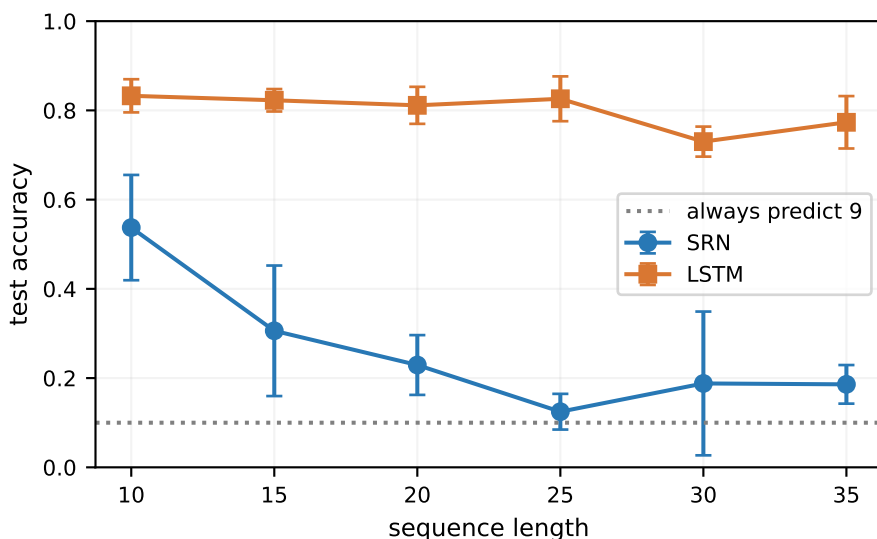


图 6.12 LSTM 与 SRN 模型在不同长度数据集上的准确率对比

动手练习 6.7

仿照 LSTM 算子的实现写一个 GRU 算子，再用同样的实验流程对比 GRU 与 LSTM 在长程依赖上的表现差异。

图6.13使用长度10、种子0的LSTM最佳快照，对输入[6, 7, 0, 0, 1, 0, 0, 0, 0, 0]逐步复算门值与内部状态。复算得到的末隐状态已与框架输出核对。横轴为输入数字，纵轴为32个状态维度；三个门的色标均为0到1，内部状态采用以0为中心的发散色标。这条序列的正确标签是13，当前模型预测为8。热图显示的是这一次计算中的数值轨迹，单凭颜色不能认定某个维度专门存储某个数字。还可以改变干扰数字的位置，或对选定门值做干预，观察预测如何变化。

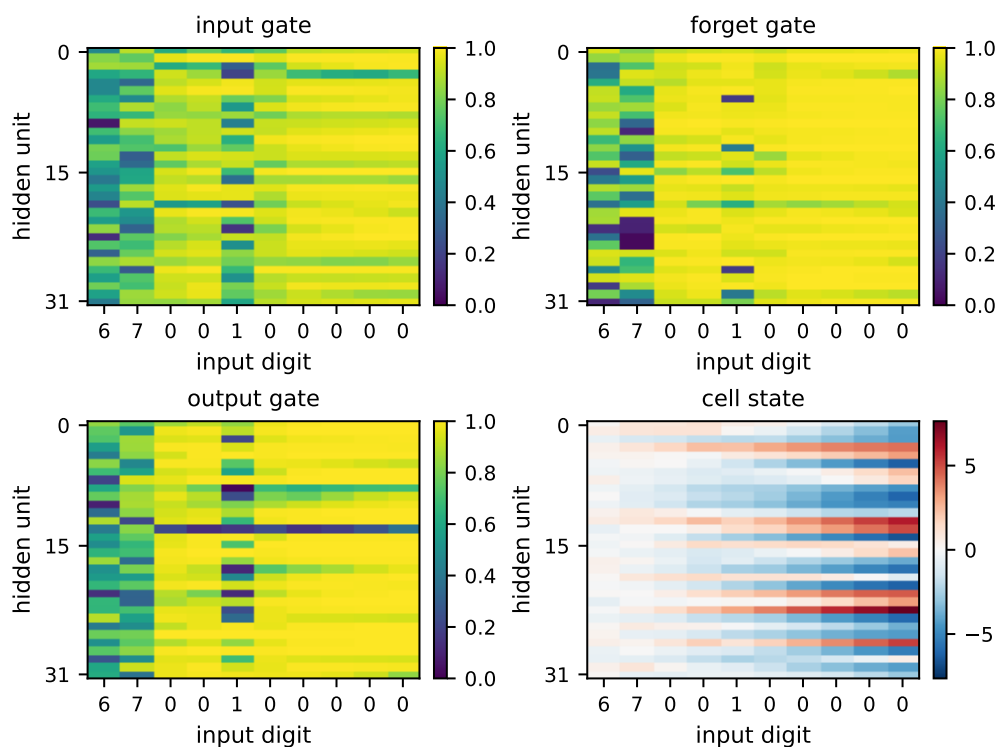


图 6.13 LSTM 在处理序列过程中三个门和内部单元状态的可视化

6.4 实践:基于双向 LSTM 模型完成文本分类任务

一句影评可能先批评情节,再肯定结尾;只看到一个褒义词还不足以判断整段倾向。情感分析 (Sentiment Analysis) 可以涉及极性、强度或评价对象等不同任务,本节将其限定为整段影评的积极 / 消极二分类。

本节以经典的 IMDB 影评数据集为例,搭建一个双向 LSTM 网络完成端到端的情感分类。

6.4.1 数据处理

IMDB 是常见的二分类影评数据集:评分 ≥ 7 被标记为积极, ≤ 4 被标记为消极。训练集和测试集各 25 000 条,每条样本是一段用户原始评价和对应的情感标签。文件目录结构如下:

```
1 |--train/
2   |--neg # 消极数据
3   |--pos # 积极数据
```

```

4     |--unsup # 无标签数据
5 |--test/
6     |--neg # 消极数据
7     |--pos # 积极数据

```

在 test/neg 目录中任选一条电影评论数据, 内容如下:

“Cover Girl” is a lacklustre WWII musical with absolutely nothing memorable about it, save for its signature song, “Long Ago and Far Away”.

这段文本需要先转成数值张量. 本例把词映射为可学习的向量, 这种映射称为**词嵌入** (Word Embedding), 所得向量俗称词向量. 为了让转换流程更高效, 常见的做法是先准备一张**词表** (Vocabulary) 把每个词映射成一个整数 ID, 再走第 6.1.2.1 节描述的查表路径换成向量. 词表里还需要留一个特殊符号 [UNK] 作为兜底, 遇到表外词一律用它代替.

6.4.1.1 数据加载

配套的 datasets/download.py 只从官方源获取数据, 并用固定种子 42 把原训练集划为 20 000 条训练样本和 5 000 条验证样本, 官方 25 000 条测试样本保持独立. 词表只由这 20 000 条训练样本构建, 保留最常见的 50 000 个词, 另加 [PAD] 和 [UNK]. 处理结果以 gzip 文件保存在配套代码的 pytorch/datasets/imdb/ 目录. 以下代码从章节 notebook 所在目录运行.

```

1 from pathlib import Path
2 from nnlib.text import load_imdb_data, load_vocab, tokenize, balanced_subset, build_vocab
3 DATASET_DIR = Path("../datasets/imdb")
4 # 如未准备数据, 从 pytorch 目录运行 python datasets/download.py --only=imdb
5 train_data, dev_data, test_data = load_imdb_data(DATASET_DIR)
6 full_run = False
7 if not full_run:
8     train_data = balanced_subset(train_data, 5000, seed=100)
9     dev_data = balanced_subset(dev_data, 1000, seed=101)
10    test_data = balanced_subset(test_data, 1000, seed=102)
11 print("train/dev/test:", len(train_data), len(dev_data), len(test_data))
12 # 打印加载后的数据样式
13 print("文本开头: ", train_data[1][0][:80], "标签: ", train_data[1][1])

```

输出结果为:

```

train/dev/test: 5000 1000 1000
文本开头:  artistically shot, actors portray exactly their role. you get a real feeling watch
          标签:  1

```

从输出结果看, 加载后的每条样本包含两部分内容: 文本串和标签.

默认抽取子集时, 不能直接取文件开头的若干条: 原始目录或处理文件可能按标签排列. 这里使用固定种子按标签抽取, 保证正负各半. tokenize 在训练和预测时都执行相同的小写、HTML 换

行清理和空白切分,并保留附着在词上的标点;这是简单的教学分词规则,不等同于后面章节的子词词元化。

6.4.1.2 构造Dataset类

首先,我们构造IMDBDataset类用于数据管理,它继承自`torch.utils.data.Dataset`类。

输入是文本序列,需要先把每个词换成词表中的ID,再交给嵌入层去查词向量;这套流程与第6.1.1节中处理数字时的流程一致。IMDBDataset中的`words_to_id`方法负责这一步:依据词表`word2id_dict`逐词查表,命中即取对应ID,未命中的词统一用[UNK]的ID代替。`word2id_dict`本身就是一张如表6.1所示的哈希表,键为词、值为ID。

表 6.1 `words_to_id`方法中使用的`word2id_dict`词表示例

词	ID
[PAD]	0
[UNK]	1
the	2
a	3
and	4
...	...

代码实现如下:

```
1 import torch
2 import torch.nn as nn
3 from torch.utils.data import DataLoader, Dataset
4 from nndl.text import load_vocab, tokenize
5
6 class IMDBDataset(Dataset):
7     def __init__(self, examples, word2id_dict):
8         super().__init__()
9         # 词表, 用于将词转为词表索引的数字
10        self.word2id_dict = word2id_dict
11        # 加载后的数据集
12        self.examples = self.words_to_id(examples)
13
14    def words_to_id(self, examples):
15        tmp_examples = []
16        for idx, example in enumerate(examples):
17            seq, label = example
```

```

18         # 将词映射为词表索引的ID, 对于词表中没有的词用 [UNK]对应的ID进行替代
19         seq = [self.word2id_dict.get(word, self.word2id_dict['[UNK]']) for word in
                tokenize(seq)]
20         if not seq:
21             seq = [self.word2id_dict["[UNK]"]]
22         label = int(label)
23         tmp_examples.append([seq, label])
24     return tmp_examples
25
26     def __getitem__(self, idx):
27         seq, label = self.examples[idx]
28         return seq, label
29
30     def __len__(self):
31         return len(self.examples)
32
33     # 加载词表
34     word2id_dict = (load_vocab(DATASET_DIR / "vocab.txt.gz") if full_run
35                     else build_vocab(train_data, max_words=20000))
36
37     # 实例化Dataset
38     train_set = IMDBDataset(train_data, word2id_dict)
39     dev_set = IMDBDataset(dev_data, word2id_dict)
40     test_set = IMDBDataset(test_data, word2id_dict)
41
42     print('训练集样本数: ', len(train_set))
43     print('样本前20个ID: ', train_set[0][0][:20], '标签: ', train_set[0][1])

```

输出结果为:

```

训练集样本数:  5000
样本前20个ID:  [12, 2199, 1299, 9865, 17, 42, 1314, 8, 350, 1, 3, 42, 5158, 8, 2, 14858, 5,
                18093, 64, 8948] 标签:  1

```

6.4.1.3 封装 DataLoader

有了Dataset还需要一个DataLoader来做批量采样. 文本任务比前几章多两步预处理:

1) **长度限制**: 评论长短不一, 过长样本增加计算与存储开销, 本例将其截到上限以内; 截断也可能删掉决定情感的内容.

2) **长度补齐**: 同一批的序列必须等长才能拼成张量, 因此短序列需要填充至本批最长长度.

具体做法是: 截断由max_seq_len控制, 把超长样本砍到该上限; 补齐则先扫描本批最长长度, 再用无语义占位符 [PAD] 补到等长. 例如下面两句:

```

句子1: This movie was craptacular.
句子2: I got stuck in traffic on the way to the theater

```

将上面的两个句子补齐,变为:

句子1: This movie was craptacular [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD]

句子2: I got stuck in traffic on the way to the theater

截断和填充由自定义的`collate_fn`承担,把它作为回调传入`DataLoader`后,每次取批前都会先经过它做整形,输出处理好的序列张量与标签。

填充会引入一批无意义的 [PAD] 位置,文本分类阶段需要绕开它们,所以`collate_fn`同时返回一个`seq_lens`向量,记录每条序列 [PAD] 之前的真实长度。配合`RunnerV3`的解包惯例`*inputs`, `y = batch` (除最后一项外的元素逐个传给模型), `collate_fn`直接以“序列张量、长度张量、标签张量”三元组返回,前两项会自动送进`forward`。

代码实现如下:

```
1 from functools import partial
2
3 def collate_fn(batch_data, pad_val=0, max_seq_len=256):
4     if max_seq_len < 1:
5         raise ValueError("max_seq_len must be positive")
6     seqs, seq_lens, labels = [], [], []
7     max_len = 0
8     for example in batch_data:
9         seq, label = example
10        # 对数据序列进行截断
11        seq = seq[:max_seq_len]
12        # 对数据截断并保存于seqs中
13        seqs.append(seq)
14        seq_lens.append(len(seq))
15        labels.append(label)
16        # 保存序列最大长度
17        max_len = max(max_len, len(seq))
18    # 对数据序列进行填充至最大长度
19    for i in range(len(seqs)):
20        seqs[i] = seqs[i] + [pad_val] * (max_len - len(seqs[i]))
21
22    return torch.tensor(seqs), torch.tensor(seq_lens), torch.tensor(labels)
23 # 最大序列长度
24 max_seq_len = 256 if full_run else 128
25 batch_size = 128 if full_run else 64
26 imdb_collate = partial(collate_fn, pad_val=word2id_dict["[PAD]"], max_seq_len=max_seq_len)
27 train_loader = DataLoader(train_set, batch_size=batch_size, shuffle=True, drop_last=False,
28                            collate_fn=imdb_collate,
29                            generator=torch.Generator().manual_seed(100))
28
29 dev_loader = DataLoader(dev_set, batch_size=batch_size, shuffle=False, drop_last=False,
30                          collate_fn=imdb_collate)
```

```
30 test_loader = DataLoader(test_set, batch_size=batch_size, shuffle=False, drop_last=False,
    collate_fn=imdb_collate)
```

6.4.2 模型构建

本实践的模型结构如图6.14所示.

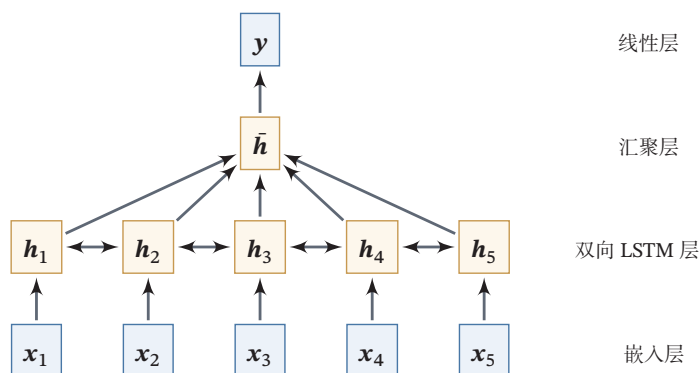


图 6.14 基于双向 LSTM 的文本分类模型结构

模型分为四段：

1) **嵌入层**: 把整数 ID 序列查表得到对应的词向量序列. 这里直接调用 PyTorch 的 `torch.nn.Embedding`:

```
1 embedding = nn.Embedding(num_embeddings=1000, embedding_dim=32, padding_idx=0)
```

最关键的两个参数是 `num_embeddings` (词表大小) 和 `embedding_dim` (向量维度), 二者共同决定了内部嵌入矩阵的形状. `padding_idx` 指向占位符 [PAD] 的 ID, 该行默认初始化为零, 且不会通过嵌入层的反向传播接收梯度; 若手动赋值或加载了非零权重, 输出会使用该行实际保存的向量. 约定俗成的做法是把 [PAD] 放在词表第 0 位, 本节也沿用这一惯例.

2) **双向 LSTM 层**: 在词向量序列上同时跑一遍正向和一遍反向 LSTM, 同一位置的正向与反向隐状态拼接得到双向表示. 直接使用 `torch.nn.LSTM`, 只需在构造时把 `bidirectional` 设为 `True`.

思考

思考在实现双向 LSTM 时, 因为需要进行序列补齐, 在计算反向 LSTM 时, 占位符 [PAD] 是否会对 LSTM 参数梯度的更新有影响。如果有的话, 如何消除影响? 注: `torch.nn.LSTM` 的 `forward` 本身没有 “按真实长度跳过 [PAD] 位置” 的参数; 在 PyTorch 里处理变长序列的标准做法是先用 `torch.nn.utils.rnn.pack_padded_sequence` 把填充后的序列连同各样本的真实长度打包成 `PackedSequence`, LSTM 内部会跳过填充位置; 再用 `pad_packed_sequence` 解包成 `[batch, length, hidden]` 的张量供下游使用。注意传给 `pack_padded_sequence` 的 `lengths` 需要在 CPU 上。

3) **汇聚层**: 对各时刻隐状态做平均, 把整条序列压缩成一个句向量。

4) **线性层**: 把句向量映射到各类别的 logits, 这里直接用 `torch.nn.Linear`。

动手练习 6.8

扩展第 6.3.1.1 节的 LSTM 算子, 使其能处理同一批次中不等长的序列, 并兼容双向运行模式。

嵌入层、双向 LSTM、线性层都可以直接借助 PyTorch, 因此我们只需要单独实现汇聚层算子。值得提醒的是: `pack_padded_sequence` 打包后的序列在 LSTM 内部确实会跳过 [PAD] 位置, 但 `pad_packed_sequence` 还原回 `[batch, length, hidden]` 时会在 [PAD] 位置写入零向量。为了让汇聚层与序列模型保持独立可替换, 本节仍按真实长度做一次显式掩码而不是依赖打包语义。

还要区分两种屏蔽位置。打包序列使 LSTM 不再用补齐位置更新状态; 输出端的掩码汇聚则避免补齐位置进入均值, 并按真实长度归一化。仅把补齐向量设为 0、或只在输出端乘掩码, 都不能替代前一项, 因为循环状态和偏置仍可能使 “读入 0” 改变状态。双向 LSTM 尤其需要注意反向分支先读到尾部补齐的问题。

双向计算使用了完整序列的两个方向, 适合本例的整段评论分类; 若预测时未来输入尚不可见, 就不能直接使用同样的双向结构。

汇聚层算子 汇聚层负责对双向 LSTM 输出的逐时刻隐状态做平均, 得到整条序列的表示。本节给出的 `AveragePooling` 先根据 `seq_lens` 生成掩码 (Mask) 矩阵屏蔽 [PAD] 位置, 再对剩余位置求和、按真实长度归一化。代码如下:

```
1 class AveragePooling(nn.Module):
2     def __init__(self):
3         super().__init__()
4
5     def forward(self, sequence_output, sequence_length):
6         sequence_length = sequence_length.to(device=sequence_output.device).unsqueeze(-1)
7         # 根据 sequence_length 生成掩码矩阵, 用于对 Padding 位置的信息进行掩蔽
8         max_len = sequence_output.shape[1]
```

```

9         mask = torch.arange(max_len, device=sequence_output.device) < sequence_length
10        mask = mask.to(sequence_output.dtype).unsqueeze(-1)
11        # 对序列中Padding部分进行掩蔽
12        sequence_output = torch.mul(sequence_output, mask)
13        # 对序列中的向量取均值
14        batch_mean_hidden = torch.div(torch.sum(sequence_output, dim=1), sequence_length.to(
            (sequence_output.dtype)))
15        return batch_mean_hidden

```

模型汇总 把嵌入层、双向 LSTM、汇聚层和线性输出层拼接为完整的情感分类网络,实现如下:

```

1  from torch.nn.utils.rnn import pack_padded_sequence, pad_packed_sequence
2
3  class Model_BiLSTM_FC(nn.Module):
4      def __init__(self, num_embeddings, input_size, hidden_size, num_classes=2):
5          super().__init__()
6          # 词表大小
7          self.num_embeddings = num_embeddings
8          # 词向量的维度
9          self.input_size = input_size
10         # LSTM隐藏单元数量
11         self.hidden_size = hidden_size
12         # 情感分类类别数量
13         self.num_classes = num_classes
14         # 实例化嵌入层
15         self.embedding_layer = nn.Embedding(num_embeddings, input_size, padding_idx=0)
16         # 实例化LSTM层, batch_first=True 表示输入形状是[batch, length, embedding]
17         self.lstm_layer = nn.LSTM(input_size, hidden_size,
18                                   batch_first=True, bidirectional=True)
19         # 实例化汇聚层
20         self.average_layer = AveragePooling()
21         # 实例化线性层
22         self.output_layer = nn.Linear(hidden_size * 2, num_classes)
23
24     def forward(self, input_ids, sequence_length):
25         # 获取词向量
26         inputs_emb = self.embedding_layer(input_ids)
27         # 打包变长序列, LSTM 内部会跳过 [PAD] 位置
28         packed = pack_padded_sequence(inputs_emb, sequence_length.cpu(),
29                                       batch_first=True, enforce_sorted=False)
30         packed_output, _ = self.lstm_layer(packed)
31         # 解包回 [batch, length, 2*hidden], [PAD] 处仍是 0
32         sequence_output, _ = pad_packed_sequence(packed_output, batch_first=True)
33         # 使用汇聚层汇聚sequence_output
34         batch_mean_hidden = self.average_layer(sequence_output, sequence_length)

```

```

35         # 输出文本分类 logits
36         logits = self.output_layer(batch_mean_hidden)
37         return logits

```

6.4.3 模型训练

默认先运行短程配置:5 000 条训练样本、1 000 条验证样本、1 000 条测试样本,正负各半;词表仅由这 5 000 条训练文本构建,最多保留 20 000 个普通词。词向量与每个方向的隐状态均为 64 维,序列最多 128 个词元,训练 5 个回合。full_run=True 则使用完整划分、256 维表示、最多 256 个词元和 3 个训练回合。两种配置的结果应分开记录;本章曲线与输出对应默认短程配置。

```

1  from nndl import RunnerV3, accuracy
2  # 指定嵌入向量的数量为词表大小
3  num_embeddings = len(word2id_dict)
4  # 嵌入向量的维度
5  input_size = 256 if full_run else 64
6  # LSTM 网络隐状态向量的维度
7  hidden_size = 256 if full_run else 64
8  # 实例化模型
9  torch.manual_seed(100)
10 model = Model_BiLSTM_FC(num_embeddings, input_size, hidden_size)
11 # 指定优化器
12 optimizer = torch.optim.Adam(model.parameters(), lr=0.001, betas=(0.9, 0.999))
13 # 指定损失函数
14 loss_fn = nn.CrossEntropyLoss()
15 # 指定评价指标
16 metric = accuracy
17 # 实例化 Runner
18 runner = RunnerV3(model, optimizer, loss_fn, metric_fn=metric)
19 # 模型训练
20 num_epochs = 3 if full_run else 5
21 runner.fit(train_loader, dev_loader, num_epochs=num_epochs, log_every=1, best_path="./
    checkpoints/best_imdb_bilstm.pt",
22           grad_clip_norm=1.0)

```

本次短程实验中,验证损失在第 3 轮达到最低值,之后回升;第 5 轮验证准确率为 73.7%,仅比第 3 轮高 0.1 个百分点。准确率只看类别是否正确,交叉熵还会惩罚高置信度的错误预测,因此两项指标可能给出不同的模型选择。本例按预先约定的验证准确率选择第 5 轮快照,测试准确率为 72.8%。

图 6.15 同时给出逐回合的训练损失、验证损失与验证准确率。若训练损失继续下降而验证表现不再改善,应先检查过拟合与截断造成的信息损失。Runner 按验证准确率保存快照,最后才在测试集评价;不能凭训练损失下降就认定模型泛化得更好。

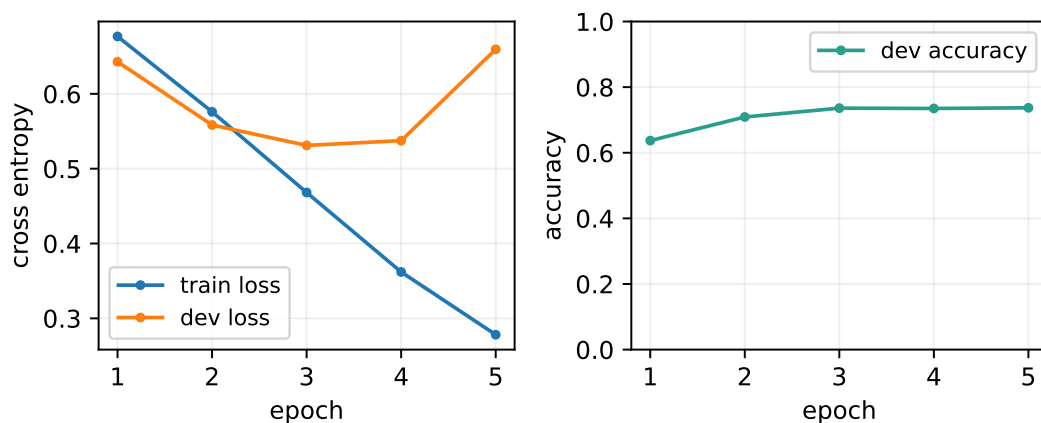


图 6.15 双向 LSTM 在 IMDB 短程配置下的训练与验证曲线

6.4.4 模型评价

加载训练过程中效果最好的模型,然后使用测试集进行测试。

```
1 runner.load("./checkpoints/best_imdb_bilstm.pt")
2 _, test_accuracy = runner.evaluate(test_loader)
3 print(f"Evaluate on test set, Accuracy: {test_accuracy:.2f}")
```

本次短程运行的输出为:

```
Evaluate on test set, Accuracy: 0.73
```

6.4.5 模型预测

随便给定一句话,让训练好的模型判定其情感极性。预测流程需要复用训练时的分词与截断规则,并把单句包装成形状为 $[1, L]$ 的批张量。代码如下:

```
1 id2label={0:"消极情绪", 1:"积极情绪"}
2 text = "this movie is so great. I watched it three times already"
3 # 处理单条文本
4 sentence = tokenize(text)
5 tokens = [word2id_dict[word] if word in word2id_dict else word2id_dict['[UNK]'] for word
             in sentence]
6 tokens = tokens[:max_seq_len] or [word2id_dict["[UNK]"]]
7 sequence_length = torch.tensor([len(tokens)], dtype=torch.int64)
8 tokens = torch.tensor(tokens, dtype=torch.int64).unsqueeze(0)
9 # 使用模型进行预测
10 logits = runner.predict(tokens, sequence_length)
```

```
11 max_label_id = torch.argmax(logits, dim=-1).item()
12 pred_label = id2label[max_label_id]
13 print("Label: ", pred_label)
```

输出结果为：

Label: 积极情绪



动手练习 6.9

LSTM 在实际工程中常以多层堆叠的形式使用，请把上面的模型替换成多层 LSTM，观察验证集准确率随层数如何变化，并尝试给出收益递减乃至下降时的可能改进方向。



动手练习 6.10

在同一份数据划分上增加一个词袋基线：只用训练文本拟合 TF-IDF 特征，再训练第 3 章的 Logistic 回归。为保持比较口径一致，两种模型都只读取相同长度的文本前缀，并用验证集选择超参数。比较它们的准确率、训练时间以及在否定句、转折句上的错误；不要预设循环网络一定优于线性分类器。

6.5 小结

数字求和实验把问题集中在一件事上：读过干扰数字后，末隐状态还能保留多少开头的信息？手写 SRN 与 LSTM 展示了状态更新的区别，多种长度和随机种子的实验则说明，门控带来的收益与训练条件有关。时间梯度的简单递推揭示了连乘的影响；全局梯度截断可以限制一次反向传播得到的梯度范数，却不能恢复已经消失的信号。

IMDB 实验进一步引入真实文本的长度差异和未知词。只从训练文本构建词表、跳过补齐位置、按真实长度汇聚，这些处理决定了模型实际读到了什么。训练损失下降而验证损失上升的曲线，也为下一章的正则化实验留下了一个具体问题：怎样改善泛化，而不只是继续拟合训练样本？

第7章 网络优化与正则化

上一章的影评分类器留下了两条方向不同的曲线：训练损失继续下降，验证损失却开始上升。如果此时只是延长训练，未必能改善新评论上的预测。另一些模型甚至还没有走到这一步，训练损失就迟迟降不下来。两种现象需要不同的处理。

本章先检查模型能否在给定预算内学好训练样本，再讨论怎样改善未见样本上的表现。优化器、初始化和规范化影响参数学习的过程；正则化则通过约束、扰动或提前停止影响最终选择的模型。它们的作用有交叉，但不能仅凭一条训练曲线判断泛化是否改善。

本章对应《神经网络与深度学习》第7章**网络优化与正则化**。阅读前可以扫一眼图7.1里的关键知识点，方便概念与代码彼此对照。

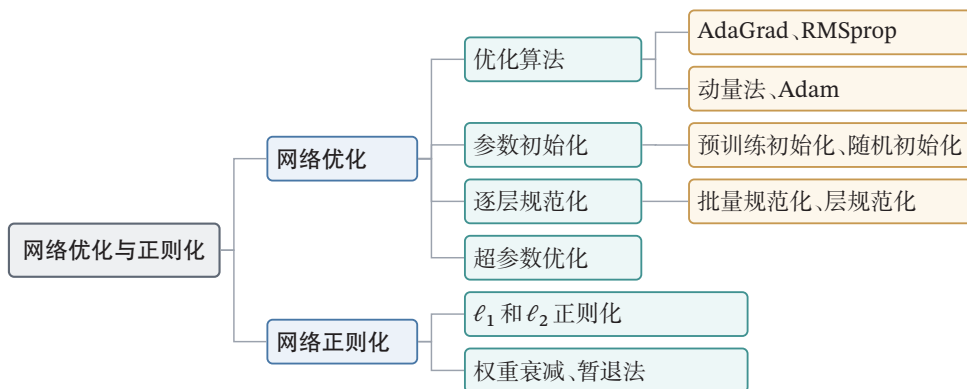


图 7.1 网络优化和正则化关键知识点回顾

本章分为两部分：

- **网络优化**：围绕优化算法、参数初始化、逐层规范化等技术，用案例与可视化进行对比实验，并落到代码层面剖析其实现。
- **网络正则化**：覆盖 ℓ_1/ℓ_2 正则化、权重衰减、暂退法等典型方法，同样通过实验直观展示它们对泛化能力的影响。



提醒

本书对部分主书术语做了调整: **Normalization** 译作规范化, **Dropout** 译作暂退法。



7.1 小批量梯度下降法

梯度下降根据当前训练目标更新神经网络参数; 该目标可以是经验风险, 也可以包含正则项。按每一步计算梯度时使用的样本数, 可分为三种形式: 批量梯度下降用全部样本、随机梯度下降只用单条样本、**小批量梯度下降** (Mini-Batch Gradient Descent) 居中——每步抽出一小批。三者的关键差异就在这一个超参**批大小** (Batch Size) 上, 需要结合数据规模与算力做权衡。本章余下内容以小批量梯度下降为主线展开。

记 $f(\mathbf{x}; \theta)$ 为神经网络模型、 θ 为参数、 $\mathcal{L}(\cdot)$ 为可微损失函数, 则 $\nabla_{\theta} \mathcal{L}(\mathbf{y}, f(\mathbf{x}; \theta))$ 表示损失对参数的梯度。小批量梯度下降每次取出 K 条样本 $\mathcal{S}_t = \{(\mathbf{x}^{(k)}, \mathbf{y}^{(k)})\}_{k=1}^K$, 第 t 步的梯度为

$$\mathbf{g}_t = \frac{1}{K} \sum_{(\mathbf{x}, \mathbf{y}) \in \mathcal{S}_t} \nabla_{\theta} \mathcal{L}(\mathbf{y}, f(\mathbf{x}; \theta_{t-1})), \quad (7.1)$$

其中 K 为批大小。

随后按梯度下降的更新规则刷新参数:

$$\theta_t \leftarrow \theta_{t-1} - \alpha \mathbf{g}_t, \quad (7.2)$$

其中 $\alpha > 0$ 为学习率。

可以看到, 影响优化效果的三个方面是:

- 1) 批大小 K ;
- 2) 学习率 α ;
- 3) 梯度估计 $\mathbf{g}_t$ (如何从 $\mathcal{S}_t$ 中得到“更好”的梯度)。

后续的各种优化算法基本都是在这三处做文章。下面通过几组对照实验, 通过数值与曲线认识它们的影响。

7.2 批大小与学习率线性缩放实验

在大数据集上, 每更新一次参数都遍历全部样本会带来较高延迟。小批量训练允许模型更早更新, 并利用矩阵运算提高吞吐量; 代价是每一步梯度含有抽样噪声。

“线性缩放规则” (linear scaling rule) 建议在一定范围内随批大小按比例调整学习率。它近似比较多次小批更新与一次大批更新的累计作用, 前提是这些步之间的梯度变化不大; 训练初期或

批量过大时可能失效,常需配合学习率预热.下面的实验同时改变批大小和学习率,观察这条规则下损失曲线的整体走向;这并不是“只改批大小”的单变量对照——如要严格度量批大小本身对收敛的影响,应固定学习率.实验仍以 LeNet 在 MNIST 上做图像分类,先用 `torchvision.datasets.MNIST` 加载数据并做规范化预处理:

```

1 import torch
2 import numpy as np
3 import torch.nn as nn
4 from torchvision import transforms
5 # 将图像转为张量,并按MNIST常用统计量规范化
6 transform = transforms.Compose([
7     transforms.ToTensor(),
8     transforms.Normalize((0.1307,), (0.3081,))
9 ])

```

为简化代码,沿用第4.5.4节的RunnerV3和配套LeNet5:用`DataLoader`按批大小切分数据,损失函数取交叉熵,优化器选`torch.optim.SGD`. RunnerV3会记录每轮的训练损失,便于稍后可视化比较.

实验组合为 $(K, \alpha) \in \{(16, 0.01), (32, 0.02), (64, 0.04), (128, 0.08), (256, 0.16)\}$, 即批大小与学习率始终保持线性同步放缩:

```

1 import torch.optim as optimizer
2 import torch.nn.functional as F
3 from torch.utils.data import DataLoader, Dataset, Subset
4 from pathlib import Path
5 from nndl import RunnerV3, LeNet5, accuracy
6 from torchvision.datasets import MNIST
7 CACHE = str(Path.home() / '.cache/torch_data')
8 mnist_train = MNIST(CACHE, train=True, download=True, transform=transform)
9 # 固定教学子集,先隔离验证样本
10 indices = torch.randperm(len(mnist_train), generator=torch.Generator().manual_seed(0))
11 train_dataset = Subset(mnist_train, indices[:5000].tolist())
12 mnist_dev = Subset(mnist_train, indices[5000:6000].tolist())
13 batch_runners = {}
14 for batch_size in [16, 32, 64, 128, 256]:
15     torch.manual_seed(0) # 各组从相同权重开始
16     model = LeNet5()
17     loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True,
18                         generator=torch.Generator().manual_seed(0))
19     opt = optimizer.SGD(model.parameters(), lr=0.01 * batch_size / 16)
20     runner = RunnerV3(model, opt, F.cross_entropy, metric_fn=accuracy)
21     runner.fit(loader, dev_loader=None, num_epochs=5, log_every=None)
22     batch_runners[batch_size] = runner

```

取出`batch_runners`中五组`history['train_loss']`逐条画在同一张图上即可比较（绘图代码见配套 notebook）。输出结果如图 7.2 所示。

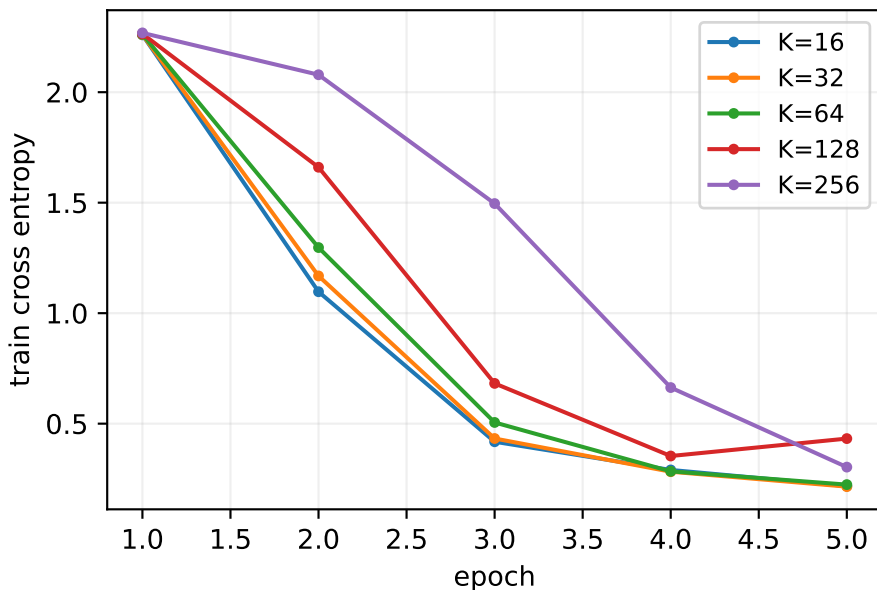


图 7.2 在 MNIST 数据集上批大小与学习率线性缩放时的训练损失曲线

本实验固定 5000 条训练样本、5 个回合，各组的初始化与每轮样本排列相同。批次分组改变了每次求梯度的位置，线性缩放后也不要求曲线重合。回合数相同比较的是相近的数据遍历量；更新步数和实际耗时仍不同。语言模型还常按序列数或词元数计量，并区分单设备批量与含梯度累积的有效批量。选择批大小时需记录显存、吞吐量和验证表现，不能从训练损失独自推断泛化优劣。

动手练习 7.1

改为按**迭代步**（而非回合）绘制损失曲线，对比不同批大小下损失下降速度的差异；思考“按回合”和“按迭代”两个视角下结论是否一致。

动手练习 7.2

对比下面两种实验设置，分析和比较它们的区别。

①批大小增大一倍，学习率不变。②批大小不变，学习率减小一半。



动手练习 7.3

本节的实验同时缩放批大小与学习率。现在把批大小固定为 64、其余设置保持不变，只把学习率依次设为 0.01、0.04、0.16、0.64，重新绘制训练损失曲线。

1) 观察学习率过小与过大时损失曲线分别呈现什么形态（下降缓慢、平稳收敛、振荡、还是发散）。

2) 据此说明你会如何快速圈定一个合理的学习率范围。

7.3 不同优化算法的比较分析

除了批大小之外，学习率与梯度估计是优化算法的另外两个因素。主流算法对它们的改良大致归为两类：

1) **学习率调整**：依据梯度历史给每个参数自适应缩放学习率，让优化更稳定——代表方法包括 AdaGrad、RMSprop、AdaDelta 等。

2) **梯度估计修正**：对每步实际使用的“方向”做修正以加速收敛——代表方法包括动量法、Nesterov 加速梯度等。

把两类思路同时纳入的代表是 Adam，本节后续也会专门介绍。

7.3.1 优化算法的实验设定

为统一对比口径，下面设计两个实验：**2D 可视化实验**和**简单拟合实验**。

先固定数据、函数和记录方式，再更换优化器。

7.3.1.1 2D 可视化实验

选一个二维凸函数，让不同优化算法去找它的最优解，并可视化参数更新轨迹，直观对比各自的收敛风格。

被优化函数 采用 Sphere 函数，其定义为

$$\text{Sphere}(\mathbf{x}) = \sum_{d=1}^D x_d^2 = \|\mathbf{x}\|_2^2, \quad (7.3)$$

其中 $\mathbf{x} \in \mathbb{R}^D$ ，后文记 $\mathbf{x}^2$ 为逐元素平方得到的向量。它显然在 $\mathbf{x}^* = 0$ 处取得唯一全局最优。

为方便可视化，下面把输入限定为二维，并把函数改造成带权重的 Sphere： $\text{Sphere}(\mathbf{x}) = \mathbf{w}^\top \mathbf{x}^2$ ，其中各权重为正，使不同维度上的曲率不同。对 $\mathbf{x}$ 的偏导为

$$\frac{\partial \text{Sphere}(\mathbf{x})}{\partial \mathbf{x}} = 2\mathbf{w} \odot \mathbf{x}, \quad (7.4)$$

其中 $\odot$ 表示逐元素积。

把它实现为 OptimizedFunction 算子：forward 计算函数值，backward 给出对 $\mathbf{x}$ 的偏导：

```

1  from nndl.op import Op
2
3  class OptimizedFunction(Op):
4      def __init__(self, w):
5          super().__init__()
6          self.w = w
7          self.params = {'x': 0}
8          self.grads = {'x': 0}
9
10     def forward(self, x):
11         self.params['x'] = x
12         return torch.matmul(self.w, torch.square(self.params['x']))
13
14     def backward(self):
15         self.grads['x'] = 2 * torch.mul(self.w, self.params['x'])

```

小批量梯度下降优化器 复用第3.1.4.3节实现的SimpleBatchGD:按 $\theta_t \leftarrow \theta_{t-1} - \alpha \mathbf{g}_t$ 更新参数.

训练函数 写一个最简训练循环,把每步的 $\mathbf{x}$ 和损失记录下来:

```

1  def train_f(model, optimizer, x_init, epoch):
2      """
3      训练函数
4      输入:
5          - model: 被优化函数
6          - optimizer: 优化器
7          - x_init: x初始值
8          - epoch: 训练回合数
9      """
10     x = x_init.clone() # 不改写调用者的共享起点
11     all_x = []
12     losses = []
13     for i in range(epoch):
14         all_x.append(x.clone())
15         loss = model(x)
16         losses.append(loss.item())
17         model.backward()
18         optimizer.step()
19         x = model.params['x']
20     all_x.append(x.clone())
21     losses.append(model(x).item())
22     return torch.stack(all_x), losses

```

可视化函数 再封装一个Visualization类,把函数在 $[-5, 5]^2$ 网格上的等值线和参数更新轨迹画在同一张图上 (contourf/contour画等值线、橙色折线画轨迹、五角星标最优); 上层

用`train_and_plot_f`把`train_f`与`Visualization`串起来: 设定初始值 $\mathbf{x} = (3, 4)$, 训练后调用`plot_2d`. 两者均为纯绘图样板, 完整实现见配套 notebook.

下面给出可直接运行的绘图入口. 轨迹包含更新前的起点及最后一次更新后的参数.

```

1 import matplotlib.pyplot as plt
2 def train_and_plot_f(model, optimizer, epoch):
3     x, losses = train_f(model, optimizer, torch.tensor([3., 4.]), epoch)
4     grid = torch.linspace(-5, 5, 150)
5     x1, x2 = torch.meshgrid(grid, grid, indexing='xy')
6     z = model.w[0] * x1.square() + model.w[1] * x2.square()
7     fig, ax = plt.subplots(figsize=(4.8, 3.5))
8     contour = ax.contourf(x1, x2, z, levels=12, cmap='viridis')
9     ax.contour(x1, x2, z, levels=12, colors='gray', linewidths=0.4)
10    ax.plot(x[:, 0], x[:, 1], '-o', color='#E76F51', markersize=3)
11    ax.plot(0, 0, '*', color='white', markeredgecolor='black', markersize=12)
12    ax.set(xlabel='x1', ylabel='x2', xlim=(-2, 5), ylim=(-2, 5))
13    fig.colorbar(contour, ax=ax, label='objective')
14    fig.tight_layout()
15    return x, losses

```

模型训练与可视化 先指定 Sphere 函数的权重 $\mathbf{w}$, 构造待优化函数. 下面用`SimpleBatchGD`做 20 步梯度下降, 把每步的 $\mathbf{x}$ 位置画到等值线图上:

```

1 from nndl import SimpleBatchGD
2 # 固定随机种子
3 torch.manual_seed(0)
4 w = torch.tensor([0.2, 2])
5 model = OptimizedFunction(w)
6 opt = SimpleBatchGD(init_lr=0.2, model=model)
7 train_and_plot_f(model, opt, epoch=20)

```

图7.3的色标给出函数值, 细线连接函数值相同的位置. 橙色折线记录实际更新路径; 两个方向曲率不同, 因此相同学习率下的收敛速度也不同.

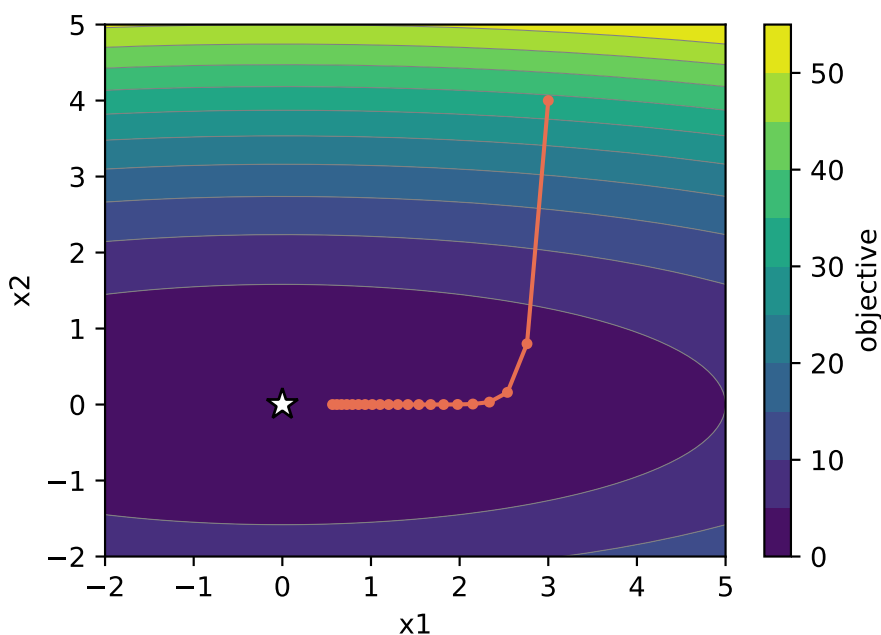


图 7.3 梯度下降法的参数更新轨迹

7.3.1.2 简单拟合实验

除了 2D 可视化, 下面再设计一个轻量级拟合任务, 用单层前馈网络对比各种优化器的收敛速度。

数据集构建 用`torch.randn`生成 1000 条二维输入 $\mathbf{X}$, 按真实关系 $y = 0.5x_1 + 0.8x_2 + 0.01\epsilon$ 生成 $\mathbf{y}$ (其中 ϵ 为噪声), 再将 $\mathbf{X}$ 和 $\mathbf{y}$ 拼成训练样本:

```

1  # 固定随机种子
2  torch.manual_seed(0)
3  # 随机生成shape为 (1000, 2) 的训练数据
4  X = torch.randn([1000, 2])
5  w = torch.tensor([0.5, 0.8])
6  w = torch.unsqueeze(w, dim=1)
7  noise = 0.01 * torch.rand([1000])
8  noise = torch.unsqueeze(noise, dim=1)
9  # 计算y
10 y = torch.matmul(X, w) + noise
11 # 打印X, y样本
12 print('X: ', X[0].numpy())
13 print('y: ', y[0].numpy())
14 # X, y组成训练样本数据
15 data = torch.cat((X, y), dim=1)

```



```

16 print('input data shape: ', data.shape)
17 print('data: ', data[0].numpy())

```

输出结果为:

```

X: [-1.1258398 -1.1523602]
y: [-1.4770346]
input data shape: torch.Size([1000, 3])
data: [-1.1258398 -1.1523602 -1.4770346]

```

模型构建 搭一个最简单的单层前馈网络: $\mathbf{X} \in \mathbb{R}^{K \times D}$ 为输入、 $\mathbf{w} \in \mathbb{R}^{D \times 1}$ 为权重、 $\mathbf{b} \in \mathbb{R}$ 为偏置:

$$\mathbf{z} = \mathbf{X}\mathbf{w} + \mathbf{b} \in \mathbb{R}^{K \times 1}, \quad (7.5)$$

其中 K 代表一个批次中的样本数量, D 为单层网络的输入特征维度.

损失函数 训练损失取通常均方误差的一半, 并按链式法则给出对参数 $\mathbf{w}$ 、 $\mathbf{b}$ 的偏导. 损失定义为

$$\mathcal{L} = \frac{1}{2K} \sum_{k=1}^K (\mathbf{y}^{(k)} - \mathbf{z}^{(k)})^2, \quad (7.6)$$

其中 $\mathbf{z}^{(k)}$ 是网络对第 k 个样本的预测值. 损失对参数的偏导为

$$\frac{\partial \mathcal{L}}{\partial \mathbf{w}} = \frac{1}{K} \sum_{k=1}^K \mathbf{x}^{(k)} (\mathbf{z}^{(k)} - \mathbf{y}^{(k)}) = \frac{1}{K} \mathbf{X}^\top (\mathbf{z} - \mathbf{y}), \quad (7.7)$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{b}} = \frac{1}{K} \sum_{k=1}^K (\mathbf{z}^{(k)} - \mathbf{y}^{(k)}) = \frac{1}{K} \mathbf{1}^\top (\mathbf{z} - \mathbf{y}). \quad (7.8)$$

据此实现 **Linear** 算子, 同时维护前向和反向过程:

```

1 class Linear(Op):
2     def __init__(self, input_size, weight_init=torch.randn, bias_init=torch.zeros):
3         super().__init__()
4         self.params = {}
5         self.params['W'] = weight_init(size=[input_size, 1])
6         self.params['b'] = bias_init(size=[1])
7         self.inputs = None
8         self.grads = {}
9
10    def forward(self, inputs):
11        self.inputs = inputs
12        self.outputs = torch.matmul(self.inputs, self.params['W']) + self.params['b']
13        return self.outputs
14
15    def backward(self, labels):

```

```

16         K = self.inputs.shape[0]
17         self.grads['w'] = 1. /K * torch.matmul(self.inputs.T, (self.outputs - labels))
18         self.grads['b'] = 1. /K * torch.sum(self.outputs - labels, dim=0)

```



笔记

backward里写的并不是**forward**本身的导数，而是把均方差损失整体合并后对参数的梯度。这种“合并写法”绕过了链式法则的中间项，是手算梯度时常见的简化策略。

训练函数 下面写一个简易训练循环，复用SimpleBatchGD进行小批量梯度下降。为了能横向对比不同优化器，这里做了两点权衡：

1) 固定训练数据和批次顺序，让不同优化器读取相同的样本；相同随机种子不保证跨设备与版本的逐位一致；

2) 与RunnerV2相比改为小批量梯度下降；与RunnerV3相比保留显式的**Optimizer**基类，便于切换不同优化器。

实现如下：

```

1  def train(data, num_epochs, batch_size, model, calculate_loss, optimizer, verbose=False):
2      """
3      训练神经网络
4      输入：
5          - data: 训练样本
6          - num_epochs: 训练回合数
7          - batch_size: 批大小
8          - model: 实例化的模型
9          - calculate_loss: 损失函数
10         - optimizer: 优化器
11         - verbose: 日志显示，默认为False
12     输出：
13         - iter_loss: 每次迭代的损失值
14         - epoch_loss: 每个回合的平均损失值
15     """
16     # 记录每个回合损失的变化
17     epoch_loss = []
18     # 记录每次迭代损失的变化
19     iter_loss = []
20     N = len(data)
21     for epoch_id in range(num_epochs):
22         epoch_total = 0.0
23         # np.random.shuffle(data) #不再随机打乱数据
24         # 将训练数据进行拆分，每个mini_batch包含batch_size条数据

```

```

25     mini_batches = [data[i:i+batch_size] for i in range(0, N, batch_size)]
26     for iter_id, mini_batch in enumerate(mini_batches):
27         # data中前两个分量为X
28         inputs = mini_batch[:, :-1]
29         # data中最后一个分量为y
30         labels = mini_batch[:, -1:]
31         # 前向计算
32         outputs = model(inputs)
33         # 计算损失
34         loss = calculate_loss(outputs, labels).item()
35         # 计算梯度
36         model.backward(labels)
37         # 梯度更新
38         optimizer.step()
39         iter_loss.append(loss)
40         epoch_total += loss * len(mini_batch)
41         # verbose = True 则打印当前回合的损失
42         if verbose:
43             print(f'Epoch {epoch_id:3d}, loss = {epoch_total / N:.4f}')
44         epoch_loss.append(epoch_total / N)
45     return iter_loss, epoch_loss

```

优化过程可视化 为了便于复用,把训练+可视化封装成`train_and_plot`:内部先调用`train`得到按迭代和按回合的两条损失序列,再分别画成左右两张子图(绘图代码见配套 notebook)。每次切换优化器只需传不同的`optimizer`实例,损失统一使用`0.5 * F.mse_loss`,与手写梯度的尺度一致:

```

1  import torch.nn as nn
2  def train_and_plot(optimizer):
3      """
4      训练网络并画出损失函数的变化趋势
5      输入:
6          - optimizer: 优化器
7      """
8      # 定义均方误差损失
9      mse = lambda output, target: 0.5 * F.mse_loss(output, target)
10     iter_loss, epoch_loss = train(data, num_epochs=30, batch_size=64, model=model,
11                                   calculate_loss=mse, optimizer=optimizer)
12     fig, axes = plt.subplots(1, 2, figsize=(6.2, 2.7))
13     axes[0].plot(iter_loss); axes[0].set_xlabel("update step")
14     axes[1].plot(range(1, len(epoch_loss) + 1), epoch_loss)
15     axes[1].set_xlabel("epoch")
16     for ax in axes:
17         ax.set_ylabel("half MSE"); ax.grid(alpha=0.2)
18     fig.tight_layout()

```

```
18     return iter_loss, epoch_loss
```

训练并查看损失曲线：

```
1 # 固定随机种子
2 torch.manual_seed(0)
3 # 定义网络结构
4 model = Linear(2)
5 # 定义优化器
6 opt = SimpleBatchGD(init_lr=0.01, model=model)
7 train_and_plot(opt)
```

输出结果如图7.4所示.

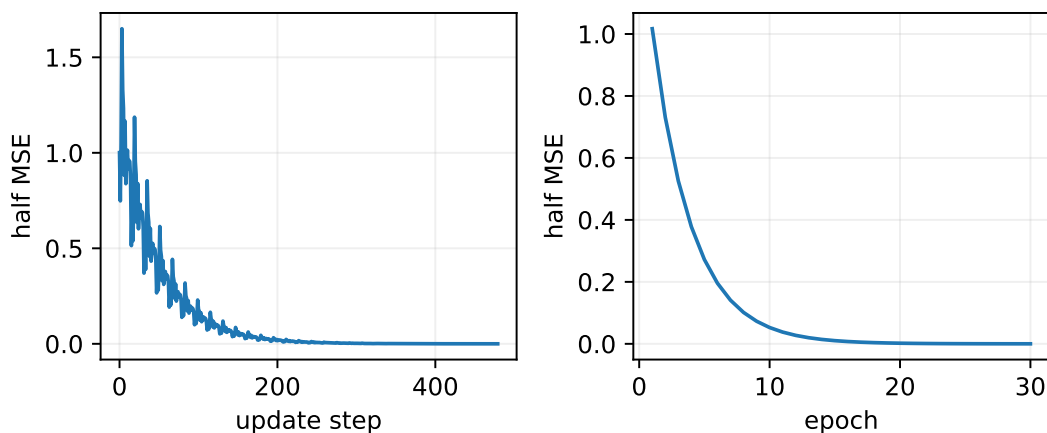


图 7.4 小批量梯度下降法的损失函数变化趋势

损失稳步下降,模型按预期完成收敛.



提醒

本节后面介绍的每一种优化算法都会同时跑这两组实验（2D 轨迹 + 拟合损失），便于横向比较。



7.3.2 学习率调整

学习率 α 是梯度下降最重要的超参数：太大会让损失发散，太小则训练效率低下。



动手练习 7.4

查阅《神经网络与深度学习》第 7.2.3.1 节列出的多种学习率衰减策略（分段、指数、余弦等），分别接入到本节的优化实验中，比较各自的最终损失与收敛速度。

常用的学习率调整手段大致可以分成两类：一类是**按规则调整**，包括学习率衰减、预热、周期性循环等；另一类是**自适应调整**，根据梯度历史动态调整每个参数的学习率，代表方法有 AdaGrad、RMSprop、AdaDelta 等。

下面重点讲 AdaGrad 与 RMSprop。

7.3.2.1 AdaGrad 算法

AdaGrad (Adaptive Gradient Algorithm) 根据各坐标的历史梯度调整更新尺度：先累积该参数历史梯度的平方

$$G_t = \sum_{\tau=1}^t \mathbf{g}_{\tau} \odot \mathbf{g}_{\tau}, \quad (7.9)$$

其中 $\odot$ 为逐元素积， $\mathbf{g}_{\tau}$ 是第 τ 次迭代时的梯度。再用累积量缩放当前梯度得到更新量：

$$\Delta \theta_t = -\frac{\alpha}{\sqrt{G_t} + \epsilon} \odot \mathbf{g}_t, \quad (7.10)$$

其中 α 是初始学习率； ϵ 为数值稳定项，常取 $10^{-7} \sim 10^{-10}$ 。开方、除法、加法等都按元素进行。直觉上：梯度大的方向被分母约束、更新步长变小，小梯度的方向则相对放大，自动取得不同参数的平衡。

构建优化器 继承 `Optimizer` 得到 `Adagrad`：step 方法按上述公式逐参数更新：

```
1 from nndl import Optimizer
2
3 class Adagrad(Optimizer):
4     def __init__(self, init_lr, model, epsilon):
5         """
6         Adagrad 优化器初始化
7         输入：
8             - init_lr: 初始学习率
9             - model: 模型, model.params 存储模型参数值
10            - epsilon: 保持数值稳定性而设置的非常小的常数
11         """
12        super().__init__(init_lr=init_lr, model=model)
13        self.G = {}
14        for key in self.model.params.keys():
15            self.G[key] = 0
```

```
16         self.epsilon = epsilon
17
18     def adagrad(self, x, gradient_x, G, init_lr):
19         """
20         adagrad算法更新参数, G为参数梯度平方的累积值
21         """
22         G += gradient_x ** 2
23         x -= init_lr / (torch.sqrt(G) + self.epsilon) * gradient_x
24         return x, G
25
26     def step(self):
27         """
28         参数更新
29         """
30         for key in self.model.params.keys():
31             self.model.params[key], self.G[key] = self.adagrad(self.model.params[key], self.
                model.grads[key], self.G[key], self.init_lr)
```

2D 可视化实验 用Sphere函数观察 AdaGrad 的更新轨迹:

```
1  # 固定随机种子
2  torch.manual_seed(0)
3  w = torch.tensor([0.2, 2])
4  model = OptimizedFunction(w)
5  opt = Adagrad(init_lr=0.5, model=model, epsilon=1e-7)
6  train_and_plot_f(model, opt, epoch=50)
```

输出结果如图7.5所示.

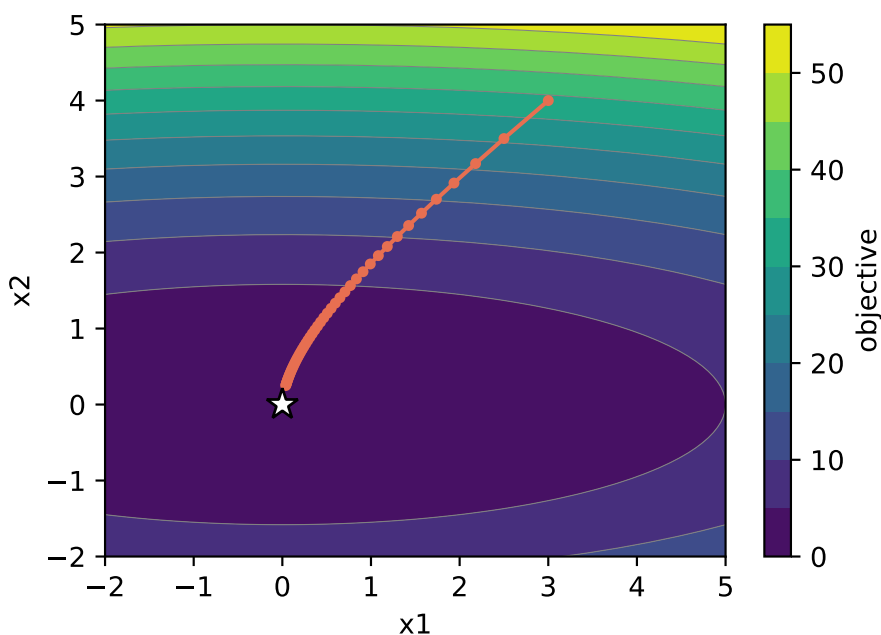


图 7.5 AdaGrad 算法的参数更新轨迹

轨迹显示, AdaGrad 在前几步更新幅度较大, 随着累积量增加, 分母变大, 步长被迅速压缩. 直觉上, 频繁出现大梯度的参数学习率会自动收紧, 而梯度较小的参数则被相对放大. 然而 G_t 只增不减, 当迭代时间一长、还没找到最优点时, 学习率会被压到极小, 模型就难再继续逼近最优.

简单拟合实验 再用单层线性网络验证 AdaGrad 在更接近真实场景的拟合任务上的收敛行为:

```
1 # 固定随机种子
2 torch.manual_seed(0)
3 # 定义网络结构
4 model = Linear(2)
5 # 定义优化器
6 opt = Adagrad(init_lr=0.1, model=model, epsilon=1e-7)
7 train_and_plot(opt)
```

输出结果如图 7.6 所示.

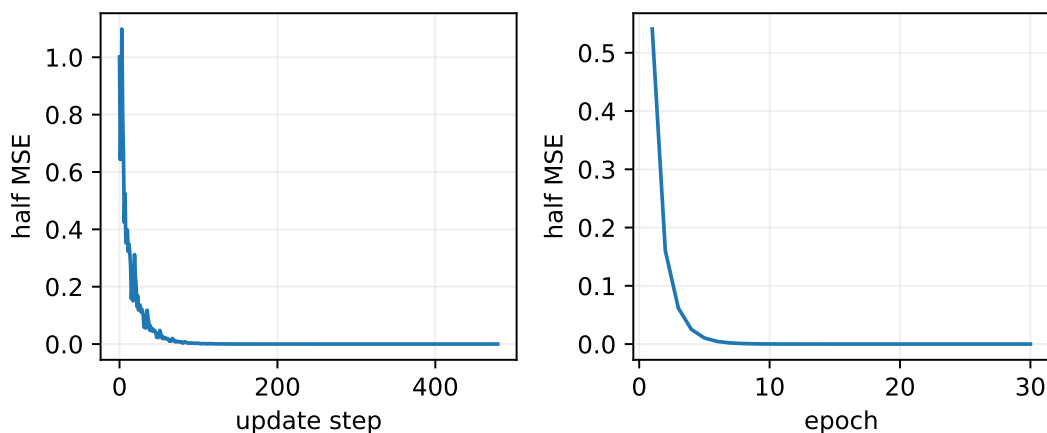


图 7.6 AdaGrad 算法的损失函数变化趋势

7.3.2.2 RMSprop 算法

RMSprop 针对 AdaGrad 有效步长可能过早变小的问题做了改进：用指数移动平均替代累计求和，让 G_t 既能升也能降。

具体地，每步计算梯度平方的加权移动平均：

$$G_t = \beta G_{t-1} + (1 - \beta) \mathbf{g}_t \odot \mathbf{g}_t, \quad (7.11)$$

其中 β 是衰减率，常取 0.9，相当于关注最近若干步的梯度统计。参数更新差值仍然类似 AdaGrad：

$$\Delta \theta_t = -\frac{\alpha}{\sqrt{G_t} + \epsilon} \odot \mathbf{g}_t, \quad (7.12)$$

其中 α 是初始学习率（如 0.001）。RMSprop 与 AdaGrad 的唯一区别就在于 G_t 用指数加权平均替换累积和，因此学习率可升可降，不再因历史累积量只增不减而必然持续缩小。

构建优化器 继承 `Optimizer` 得到 `RMSprop`：

```
1 class RMSprop(Optimizer):
2     def __init__(self, init_lr, model, beta, epsilon):
3         """
4         RMSprop 优化器初始化
5         输入：
6             - init_lr: 初始学习率
7             - model: 模型，model.params 存储模型参数值
8             - beta: 衰减率
9             - epsilon: 为保持数值稳定性而设置的常数
10        """
11        super().__init__(init_lr=init_lr, model=model)
12        self.G = {}
```



```
13         for key in self.model.params.keys():
14             self.G[key] = 0
15         self.beta = beta
16         self.epsilon = epsilon
17
18     def rmsprop(self, x, gradient_x, G, init_lr):
19         """
20         RMSprop算法更新参数, G为迭代梯度平方的加权移动平均
21         """
22         G = self.beta * G + (1 - self.beta) * gradient_x ** 2
23         x -= init_lr / (torch.sqrt(G) + self.epsilon) * gradient_x
24         return x, G
25
26     def step(self):
27         """参数更新"""
28         for key in self.model.params.keys():
29             self.model.params[key], self.G[key] = self.rmsprop(self.model.params[key],
30                 self.model.grads[key], self.G[key], self.init_lr)
```

2D 可视化实验 使用被优化函数展示 RMSprop 算法的参数更新轨迹. 代码实现如下:

```
1 # 固定随机种子
2 torch.manual_seed(0)
3 w = torch.tensor([0.2, 2])
4 model = OptimizedFunction(w)
5 opt = RMSprop(init_lr=0.1, model=model, beta=0.9, epsilon=1e-7)
6 train_and_plot_f(model, opt, epoch=50)
```

输出结果如图7.7所示.

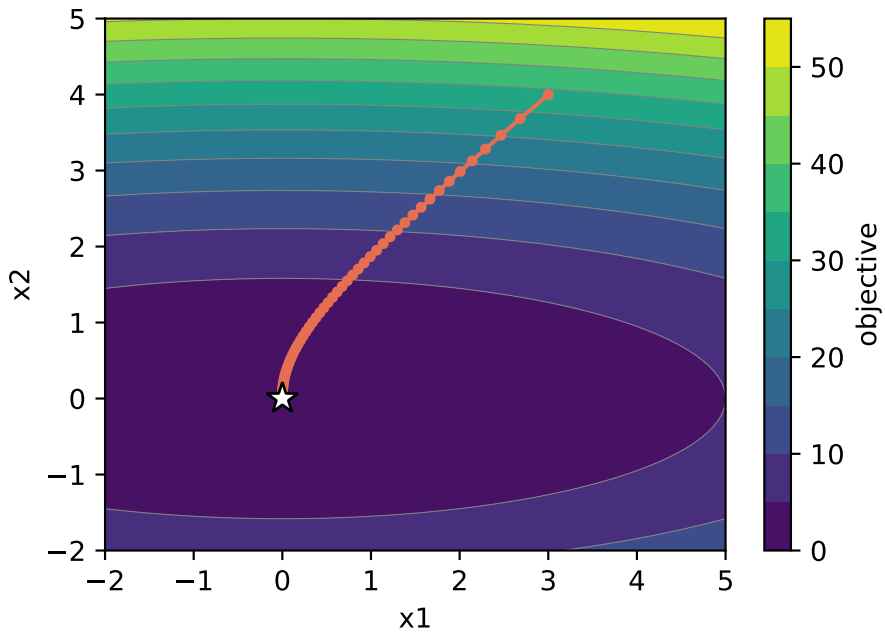


图 7.7 RMSprop 算法的参数更新轨迹

简单拟合实验 训练单层线性网络,进行简单的拟合实验. 代码实现如下:

```
1 # 固定随机种子
2 torch.manual_seed(0)
3 # 定义网络结构
4 model = Linear(2)
5 # 定义优化器
6 opt = RMSprop(init_lr=0.1, model=model, beta=0.9, epsilon=1e-7)
7 train_and_plot(opt)
```

输出结果如图7.8所示.

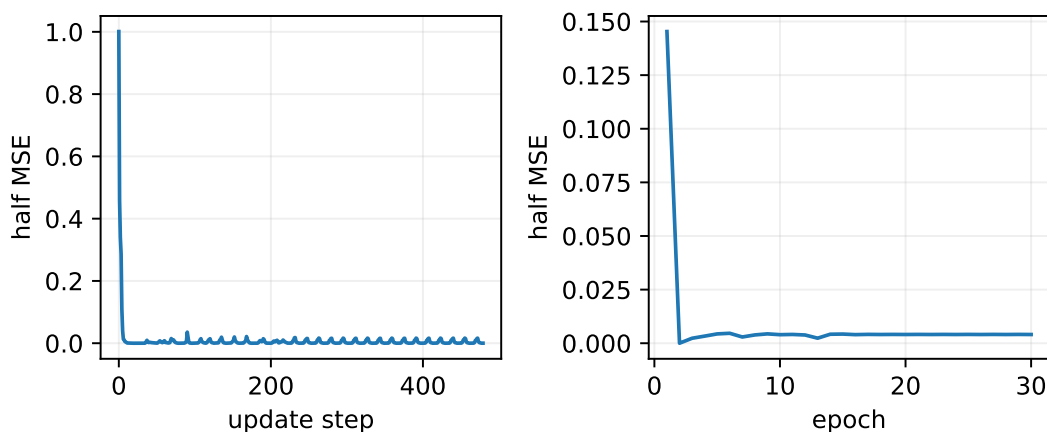


图 7.8 RMSprop 算法的损失函数变化趋势



动手练习 7.5

动手实现 AdaDelta 算法。



7.3.3 梯度估计修正

学习率调整解决了“每步走多远”的问题,剩下的就是“走的方向”——梯度估计本身。在固定参数处均匀抽取样本时,小批量平均梯度可以是全量梯度的无偏估计;单次估计仍有随机误差。批大小、样本相关性与采样方式共同影响噪声。

针对这一问题,一类直接有效的做法是把最近若干步梯度的“平均”作为更新方向,平滑掉单步噪声,从而走得更稳、更快。下面介绍这类思想下最具代表性的两种算法。

7.3.3.1 动量法

动量法 (Momentum Method) 用历史梯度的累积“动量”代替单次梯度,相当于把每一步梯度当作**加速度**而非位移。

具体地,第 t 步以负梯度的指数加权和平移作为更新方向:

$$\Delta\theta_t = \rho\Delta\theta_{t-1} - \alpha\mathbf{g}_t = -\alpha \sum_{\tau=1}^t \rho^{t-\tau} \mathbf{g}_\tau, \quad (7.13)$$

其中 ρ 是动量因子 (常用 0.9), α 是学习率。

直觉上:参数实际走的方向是“近期梯度的加权和”。如果最近若干步梯度方向不一致 (互相抵消),更新幅度会缩小;如果方向一致,则不断累加产生加速效应。持续一致的梯度可以积累速度,来回变化的分量可能抵消;但已有动量也会造成越过极小点和振荡,效果取决于学习率与动量系

数。换一个视角看,把当前梯度与历史梯度叠加,在效果上类似于利用了曲面的走向信息,但它本质上仍是一阶梯度的加权平均,并未真正利用曲率(Hessian)信息。

构建优化器 继承`Optimizer`基类得到`Momentum`:`step`方法逐参数调用`momentum`函数完成更新:

```

1  class Momentum(Optimizer):
2      def __init__(self, init_lr, model, rho):
3          """
4          Momentum优化器初始化
5          输入:
6              - init_lr: 初始学习率
7              - model: 模型, model.params存储模型参数值
8              - rho: 动量因子
9          """
10         super().__init__(init_lr=init_lr, model=model)
11         self.delta_x = {}
12         for key in self.model.params.keys():
13             self.delta_x[key] = 0
14         self.rho = rho
15
16     def momentum(self, x, gradient_x, delta_x, init_lr):
17         """
18         momentum算法更新参数, delta_x为梯度的加权移动平均
19         """
20         delta_x = self.rho * delta_x - init_lr * gradient_x
21         x += delta_x
22         return x, delta_x
23
24     def step(self):
25         """参数更新"""
26         for key in self.model.params.keys():
27             self.model.params[key], self.delta_x[key] = self.momentum(self.model.params[key],
28                                                                           self.model.grads[key], self.delta_x[key], self.init_lr)

```

2D 可视化实验 用`Sphere`函数观察动量法的更新轨迹:

```

1  # 固定随机种子
2  torch.manual_seed(0)
3  w = torch.tensor([0.2, 2])
4  model = OptimizedFunction(w)
5  opt = Momentum(init_lr=0.01, model=model, rho=0.9)
6  train_and_plot_f(model, opt, epoch=50)

```

输出结果如图7.9所示。

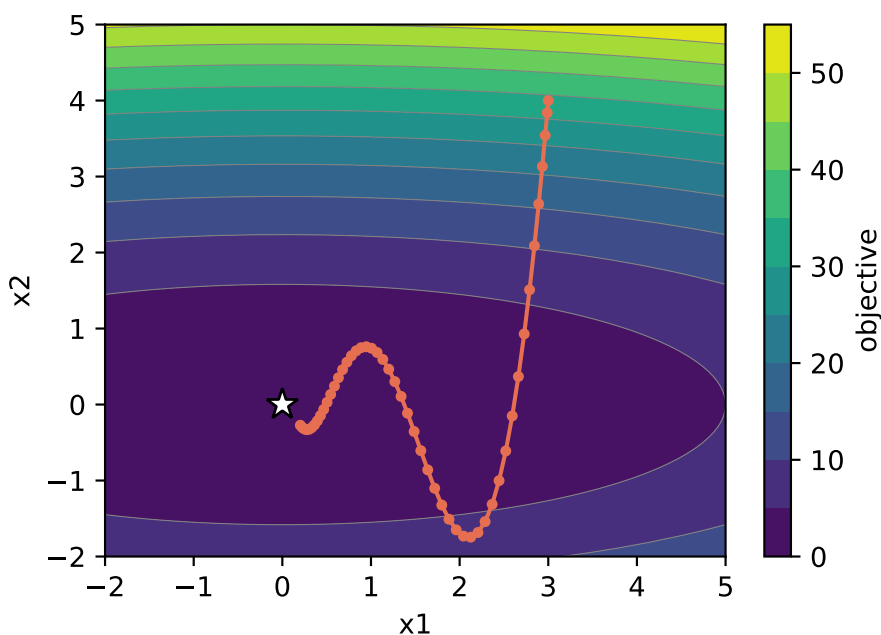


图 7.9 动量法的参数更新轨迹

轨迹符合直觉：训练初期梯度方向一致，步长被逐步放大，加速向最优点逼近；接近最优时方向频繁切换，动量自然衰减，参数在最优点附近做小幅度震荡。

简单拟合实验 再在单层线性网络上验证动量法的收敛行为：

```
1 # 固定随机种子
2 torch.manual_seed(0)
3 # 定义网络结构
4 model = Linear(2)
5 # 定义优化器
6 opt = Momentum(init_lr=0.01, model=model, rho=0.9)
7 train_and_plot(opt)
```

损失曲线见图7.10：相比纯SGD，动量法在前期更快接近极小值，且后期没有明显的发散迹象。

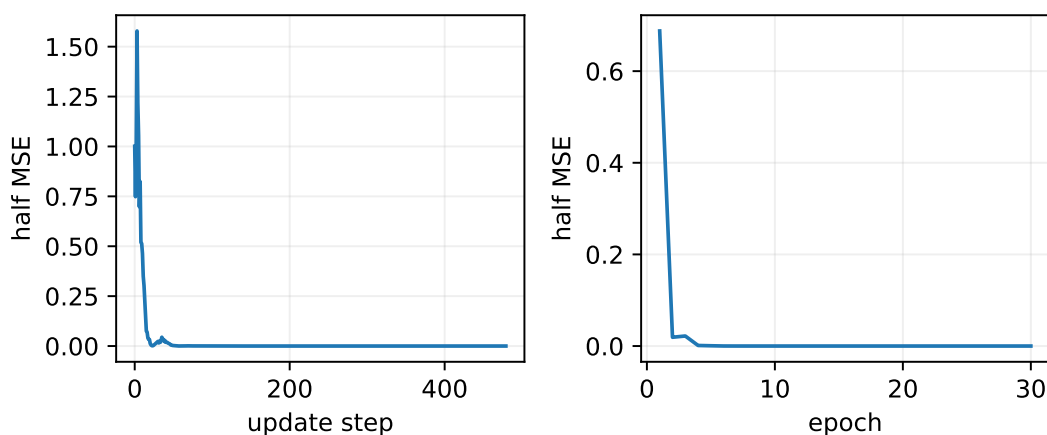


图 7.10 Momentum 算法的损失函数变化趋势

7.3.3.2 Adam 算法

Adam (Adaptive Moment Estimation) 把 RMSprop 的自适应学习率和动量法的方向修正合二为一: 既维护梯度的指数加权平均 (提供动量), 又维护梯度平方的指数加权平均 (提供自适应学习率). 两套统计量的更新规则分别为:

$$M_t = \beta_1 M_{t-1} + (1 - \beta_1) \mathbf{g}_t, \quad (7.14)$$

$$G_t = \beta_2 G_{t-1} + (1 - \beta_2) \mathbf{g}_t \odot \mathbf{g}_t, \quad (7.15)$$

其中 β_1 、 β_2 分别为两个移动平均的衰减率. Adam 原论文及 PyTorch 的常见默认值为 (0.9, 0.999), 本节手写实验取 (0.9, 0.99), 属于不同超参数设置. 从统计意义看, M_t 估计梯度的一阶矩, G_t 估计未中心化的二阶矩, 即梯度平方的均值.

初始值 $M_0 = G_0 = 0$ 使迭代初期的两项移动平均偏向零, 尤其在 β_1 、 β_2 接近 1 时更明显. G_t 对应二阶矩, 不是扣除均值平方后的方差. Adam 用下面的偏差修正补偿零初始化的影响:

$$\hat{M}_t = \frac{M_t}{1 - \beta_1^t}, \quad (7.16)$$

$$\hat{G}_t = \frac{G_t}{1 - \beta_2^t}. \quad (7.17)$$

Adam 算法的参数更新差值为

$$\Delta \theta_t = -\frac{\alpha}{\sqrt{\hat{G}_t} + \epsilon} \hat{M}_t, \quad (7.18)$$

其中学习率 α 通常取 0.001, 并可根据需要叠加衰减策略, 例如 $\alpha_t = \alpha_0 / \sqrt{t}$.

构建优化器 继承`Optimizer`基类得到`Adam:step`方法逐参数调用`adam`函数完成上述四步更新：

```

1  class Adam(Optimizer):
2      def __init__(self, init_lr, model, beta1, beta2, epsilon):
3          """
4              Adam优化器初始化
5              输入：
6                  - init_lr: 初始学习率
7                  - model: 模型, model.params存储模型参数值
8                  - beta1, beta2: 移动平均的衰减率
9                  - epsilon: 为保持数值稳定性而设置的常数
10             """
11             super().__init__(init_lr=init_lr, model=model)
12             self.beta1 = beta1
13             self.beta2 = beta2
14             self.epsilon = epsilon
15             self.M, self.G = {}, {}
16             for key in self.model.params.keys():
17                 self.M[key] = 0
18                 self.G[key] = 0
19             self.t = 1
20
21     def adam(self, x, gradient_x, G, M, t, init_lr):
22         """
23             adam算法更新参数
24             输入：
25                 - x: 参数
26                 - G: 梯度平方的加权移动平均
27                 - M: 梯度的加权移动平均
28                 - t: 迭代次数
29                 - init_lr: 初始学习率
30         """
31         M = self.beta1 * M + (1 - self.beta1) * gradient_x
32         G = self.beta2 * G + (1 - self.beta2) * gradient_x ** 2
33         M_hat = M / (1 - self.beta1 ** t)
34         G_hat = G / (1 - self.beta2 ** t)
35         x -= init_lr / (torch.sqrt(G_hat) + self.epsilon) * M_hat
36         return x, G, M
37
38     def step(self):
39         """参数更新"""
40         for key in self.model.params.keys():
41             self.model.params[key], self.G[key], self.M[key] = self.adam(
42                 self.model.params[key], self.model.grads[key], self.G[key], self.M[key],
43                 self.t, self.init_lr)

```

```
44     # 一个 step 内所有参数用同一个 t 做偏差修正, 更新完再自增
45     self.t += 1
```

2D 可视化实验 用Sphere函数观察 Adam 的更新轨迹:

```
1  # 固定随机种子
2  torch.manual_seed(0)
3  w = torch.tensor([0.2, 2])
4  model = OptimizedFunction(w)
5  opt = Adam(init_lr=0.2, model=model, beta1=0.9, beta2=0.99, epsilon=1e-7)
6  train_and_plot_f(model, opt, epoch=20)
```

输出结果如图7.11所示.

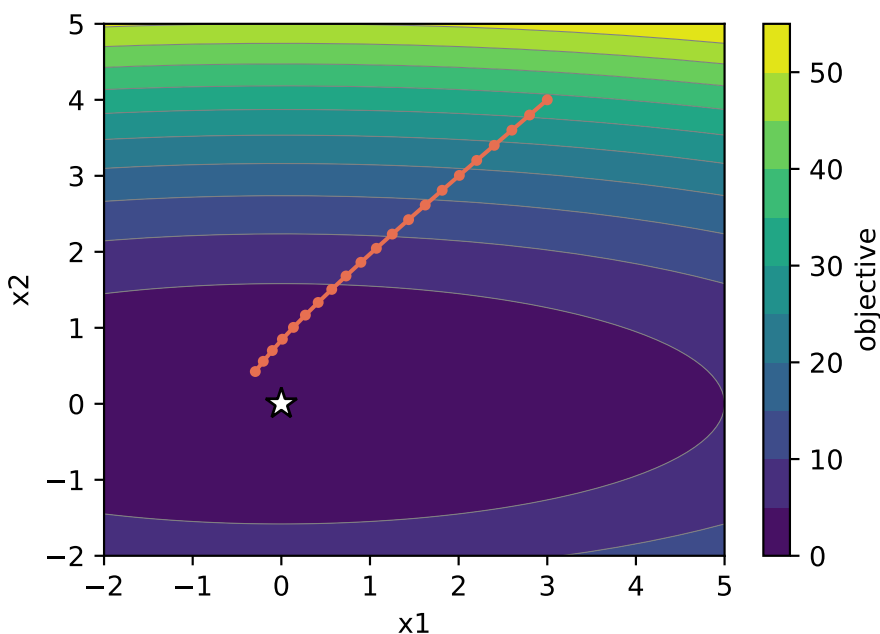


图 7.11 Adam 算法的参数更新轨迹

Adam 同时使用梯度均值和平方均值, 轨迹会随两项统计量变化. 本图采用的学习率与部分前例不同, 因此只展示该配置下的更新行为, 不能据此排出优化器的普遍优劣.

简单拟合实验 再用单层线性网络验证 Adam 在拟合任务上的收敛行为:

```
1  # 固定随机种子
2  torch.manual_seed(0)
3  # 定义网络结构
4  model = Linear(2)
```



```

5 # 定义优化器
6 opt = Adam(init_lr=0.1, model=model, beta1=0.9, beta2=0.99, epsilon=1e-7)
7 train_and_plot(opt)

```

输出结果如图7.12所示.

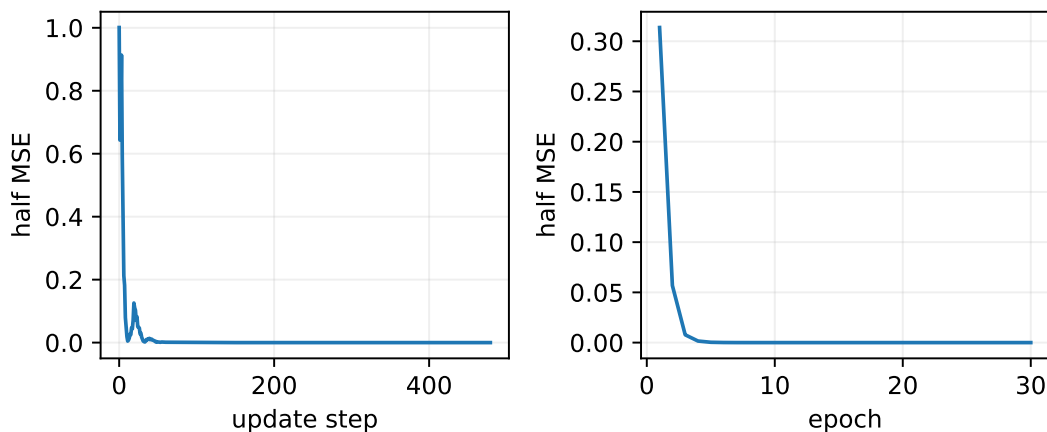


图 7.12 Adam 算法的损失函数变化趋势



动手练习 7.6

学习 AdamW 算法，通过 PyTorchAPI 调用 LeNet 网络和 MNIST 数据集，使用 `torch.optim.AdamW` 作为优化器训练网络，进行简单的拟合实验。

7.3.4 不同优化器的 3D 可视化对比

7.3.4.1 构建一个三维空间中的被优化函数

仍然优化两个参数，这次把函数值作为第三根坐标轴绘制曲面。OptimizedFunction3D 算子表示函数 $f(\mathbf{x}) = \mathbf{x}[0]^4 - 2\mathbf{x}[0]^2 + \mathbf{x}[1]^2$ ，其中 $\mathbf{x}[0]$ 、 $\mathbf{x}[1]$ 是两个参数。该函数在原点 (0, 0) 处有一个鞍点：沿 $\mathbf{x}[0]$ 方向它是一道向两侧下降的“脊”（局部极大），沿 $\mathbf{x}[1]$ 方向却是一条“谷”（局部极小），因此既非极大值也非极小值；脊的两侧各有一个真正的最小值 ($\pm 1, 0$)。我们的关注点是：不同优化器能否绕开鞍点、继续滚向某个最小值。算子实现如下：

```

1 class OptimizedFunction3D(Op):
2     def __init__(self):
3         super().__init__()
4         self.params = {'x': 0}
5         self.grads = {'x': 0}
6

```

```

7     def forward(self, x):
8         self.params['x'] = x
9         return x[0] ** 4 - 2 * x[0] ** 2 + x[1] ** 2
10
11    def backward(self):
12        x = self.params['x']
13        gradient1 = 4 * x[0] ** 3 - 4 * x[0]
14        gradient2 = 2 * x[1]
15        self.grads['x'] = torch.stack([gradient1, gradient2])

```

在同一个被优化函数上,分别用五种优化器各跑一轮训练,并记录其参数轨迹,便于后续放到同一张 3D 图上对比:

```

1  #构建5个模型, 分别配备不同的优化器
2  model1 = OptimizedFunction3D()
3  opt_gd = SimpleBatchGD(init_lr=0.01, model=model1)
4  model2 = OptimizedFunction3D()
5  opt_adagrad = Adagrad(init_lr=0.5, model=model2, epsilon=1e-7)
6  model3 = OptimizedFunction3D()
7  opt_rmsprop = RMSprop(init_lr=0.1, model=model3, beta=0.9, epsilon=1e-7)
8  model4 = OptimizedFunction3D()
9  opt_momentum = Momentum(init_lr=0.01, model=model4, rho=0.9)
10 model5 = OptimizedFunction3D()
11 opt_adam = Adam(init_lr=0.1, model=model5, beta1=0.9, beta2=0.99, epsilon=1e-7)
12
13 models = [model1, model2, model3, model4, model5]
14 opts = [opt_gd, opt_adagrad, opt_rmsprop, opt_momentum, opt_adam]
15
16 x_all_opts = []
17 z_all_opts = []
18 x_init = torch.tensor([0.2, 1.5], dtype=torch.float32) # x0=0.2 略偏向右侧盆地、x1=1.5 远离
    脊线
19 # 使用不同优化器训练
20 for model, opt in zip(models, opts):
21     x_one_opt, z_one_opt = train_f(model, opt, x_init, 150)
22     # 保存参数值
23     x_all_opts.append(x_one_opt.numpy())
24     z_all_opts.append(np.squeeze(z_one_opt))

```

封装 Visualization3D 类,将各优化器的参数轨迹叠加为一段 3D 动画:

```

1  from matplotlib import animation
2  from itertools import zip_longest
3
4  class Visualization3D(animation.FuncAnimation):
5      """

```

```

6     绘制动态图像，可视化参数更新轨迹
7     """
8     def __init__(self, *xy_values, z_values, labels=[], colors=[], fig, ax, interval=60,
9         blit=True, **kwargs):
10         """
11         初始化3d可视化类
12         输入：
13             xy_values: 三维中x、y维度的值
14             z_values: 三维中z维度的值
15             labels: 每个参数更新轨迹的标签
16             colors: 每个轨迹的颜色
17             interval: 帧之间的延迟（以毫秒为单位）
18             blit: 是否优化绘图
19         """
20         self.fig = fig
21         self.ax = ax
22         self.xy_values = xy_values
23         self.z_values = z_values
24         frames = max(xy_value.shape[0] for xy_value in xy_values)
25         self.lines = [ax.plot([], [], [], label=label, color=color, lw=2)[0]
26             for _, label, color in zip_longest(xy_values, labels, colors)]
27         super().__init__(fig, self.animate, init_func=self.init_animation, frames=frames,
28             interval=interval, blit=blit, **kwargs)
29
30     def init_animation(self):
31         # 数值初始化
32         for line in self.lines:
33             line.set_data([], [])
34             line.set_3d_properties([])
35         return self.lines
36
37     def animate(self, i):
38         # 将x、y、z三个数据传入，绘制三维图像
39         for line, xy_value, z_value in zip(self.lines, self.xy_values, self.z_values):
40             line.set_data(xy_value[:i+1, 0], xy_value[:i+1, 1])
41             line.set_3d_properties(z_value[:i+1])
42         return self.lines

```

先把被优化函数本身的3D曲面绘出,作为动画的背景:

```

1 from mpl_toolkits.mplot3d import Axes3D
2 import matplotlib.pyplot as plt
3 # 使用numpy.meshgrid生成x1,x2矩阵，矩阵的每一行为[-2, 2]，以0.1为间隔的数值
4 x1 = np.arange(-2, 2, 0.1)
5 x2 = np.arange(-2, 2, 0.1)

```

```
6 x1, x2 = np.meshgrid(x1, x2)
7 init_x = torch.tensor(np.stack([x1, x2]))
8 model = OptimizedFunction3D()
9 # 绘制 $f_{3d}$ 函数的三维图像
10 fig = plt.figure()
11 ax = plt.axes(projection='3d')
12 ax.plot_surface(init_x[0], init_x[1], model(init_x), cmap='viridis')
13 ax.set_xlabel('x1')
14 ax.set_ylabel('x2')
15 ax.set_zlabel('f(x1,x2)')
```

再将五条轨迹叠到这张 3D 图上:

```
1 from IPython.display import HTML
2 labels = ['SGD', 'AdaGrad', 'RMSprop', 'Momentum', 'Adam']
3 colors = ['r', 'b', 'y', 'g', 'c']
4 anim = Visualization3D(*x_all_opts, z_values=z_all_opts, labels=labels, colors=colors, fig
    =fig, ax=ax)
5 ax.legend(loc='upper left')
6 HTML(anim.to_jshtml())
```

输出结果如图7.13所示.

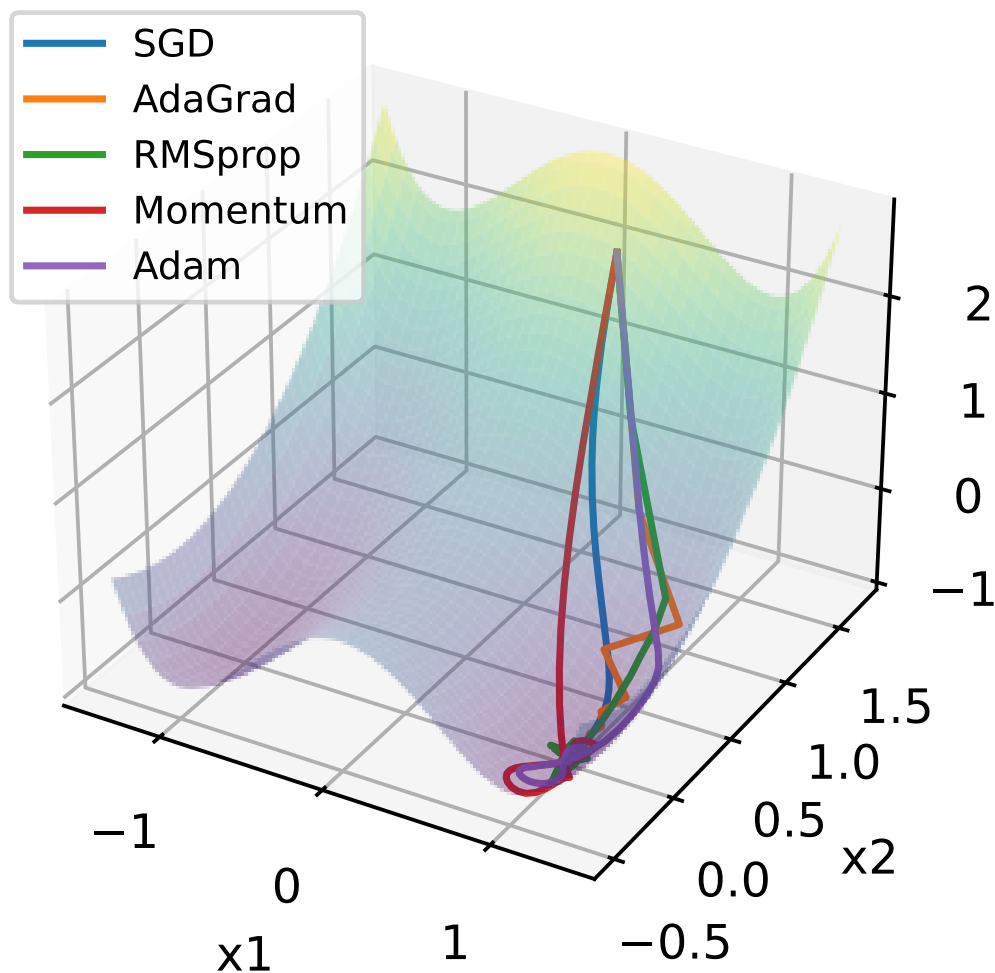


图 7.13 不同优化器参数变化轨迹

这些轨迹从(0.2, 1.5)出发, 已经偏向右侧盆地; 它们展示的是接近极小点时的路径与速度, 而非从鞍点脱困的证明. 若恰从(0, 0)出发且所有动量状态为0, 梯度就是0, 这些确定性更新都会停在那里. 可以在起点加入不同大小的扰动, 观察其影响. 比较图中速度时, 还应注意各算法的学习率并不相同.



动手练习 7.7

调用`torch.optim`对比 SGD、Adam、AdamW、RMSprop 等优化器在 MNIST 上的收敛速度和最终精度。



**动手练习 7.8**

本章从零实现了SimpleBatchGD、Adagrad、RMSprop、Momentum、Adam五个优化器。请把它们与torch.optim中的同名优化器做一次对账。

1) 在相同的初始点、相同的学习率与步数下,对同一个被优化函数分别用自实现版本和torch.optim版本各跑一遍,逐一对比两者的参数更新轨迹。

2) 找出数值差异最大的那一个优化器,结合它的实现细节(如Adam所有参数共用的迭代步 t 、分母中 ϵ 位于平方根之外、各超参的默认取值)分析差异的来源。

7.4 参数初始化

神经网络的训练本质上是在一个非凸损失面上做优化,起点选在哪里直接影响优化的难度与最终模型的泛化能力。第4.4.1节已经看到极端情形——同一层的隐藏神经元采用相同入边、偏置及对称出边时会出现**对称权重现象**:前向激活值一致、反向梯度也一致,多个神经元退化为同一个。因此初始化的第一原则就是给参数引入足够的差异。

主流初始化策略可归为三类:

- **随机初始化**:用一个随机分布逐参数采样。这是最常用的默认选项。
- **预训练初始化**:当目标任务数据稀少时,把一个已经在大规模数据上训练好的模型作为起点,再针对目标任务**微调**(Fine-Tuning)。
- **固定值初始化**:对带先验知识的关键参数显式赋值。例如使用ReLU时常把偏置设置为一个微小正数(如0.01),可能改变早期激活比例,但不能保证神经元不进入零梯度区域。

预训练权重能提供已有任务中学到的表示,是否适用取决于输入、任务与结构;未匹配的层仍可重新初始化。本节聚焦它的两类典型版本:固定方差随机初始化与方差缩放随机初始化。

7.4.1 基于固定方差的参数初始化

最简单的随机初始化是从均值为0、方差为 σ^2 的固定分布中采样。常用形式有两种:

- **高斯分布初始化**:使用一个高斯分布 $\mathcal{N}(0, \sigma^2)$ 对每个参数进行随机初始化。
- **均匀分布初始化**:在一个给定的区间 $[-r, r]$ 内使用均匀分布来初始化。

这两种初始化的代码实现可直接复用第4.4.1节给出的版本。

7.4.2 基于方差缩放的参数初始化

在深网络中,固定方差初始化的问题是:层数一深,前向激活值的方差会沿层指数衰减或膨胀,进而触发梯度消失或爆炸. 一个直接的缓解思路是**方差缩放** (Variance Scaling)——根据当前层的输入/输出神经元数量自适应地调整初始化方差,使前向和反向方向上的方差近似稳定.

Xavier 初始化就是其中最具代表性的方案,它把权重方差表示为输入维 M_{l-1} 与输出维 M_l 的函数. 在近线性、小激活的近似下,取增益为 1,权重按 $\mathcal{N}(0, \frac{2}{M_{l-1}+M_l})$ 采样即可.



笔记

Xavier 初始化的推导细节可参考《神经网络与深度学习》第 7.3.2.1 节.



下面动手实现 Xavier 初始化,并观察它在本例中如何影响各层激活方差.

7.4.2.1 模型构建

由于 `torch.normal` 的参数是标准差而非方差, 先写一个 `xavier_normal_std` 函数, 根据 M_{l-1} 和 M_l 返回应当采样的标准差:

```
1 def xavier_normal_std(input_size, output_size):
2     return np.sqrt(2 / (input_size + output_size))
```



笔记

这里显式取增益为 1, 不承诺饱和激活下各层方差恒定. ReLU 可尝试 He 初始化的 $2/M_{l-1}$ 方差. 调用框架初始化函数时应同时检查 `gain` 或 `nonlinearity`; 函数名称相同并不意味着有效方差相同.



搭建一个可灵活配置初始化策略的多层感知器 MLP. 构造时通过 `layers_size` 给出每层神经元数, `init_fn_name` 选定初始化方式 (`normal`=Xavier 高斯、`uniform`=Xavier 均匀, 其他值为标准正态对照组), `init_fn` 回调用于在两种 Xavier 方案下计算具体标准差或区间半径, `act_fn` 选定激活函数:

```
1 class MLP(nn.Module):
2     def __init__(self, layers_size, init_fn_name, init_fn, act_fn):
3         """
4         多层网络初始化
5         输入:
6         - layers_size: 每层神经元的数量
7         - init_fn_name: 网络中参数初始化方法, 可以为 'normal' 或 'uniform'
8         - init_fn: 函数, 用来计算高斯分布标准差或均匀分布 r 值
9         - act_fn: 激活函数
```

```

10     """
11     super().__init__()
12     self.linear = nn.Sequential()
13     self.num_layers = len(layers_size) - 1
14     for i in range(self.num_layers):
15         input_size, output_size = layers_size[i], layers_size[i + 1]
16         if init_fn_name == 'normal':
17             # Xavier高斯分布初始化, 计算方差
18             layer = nn.Linear(input_size, output_size)
19             nn.init.normal_(layer.weight, mean=0, std=init_fn(input_size, output_size))
20         elif init_fn_name == 'uniform':
21             r = init_fn(input_size, output_size)
22             layer = nn.Linear(input_size, output_size)
23             nn.init.uniform_(layer.weight, a=-r, b=r)
24         else:
25             layer = nn.Linear(input_size, output_size)
26             nn.init.normal_(layer.weight)
27             nn.init.zeros_(layer.bias)
28         self.linear.add_module(str(i), layer)
29     self.act_fn = act_fn()
30     self.z = {}
31     # nn.Module 已经在 __call__ 中转发到 forward 并维护 hook,
32     # 因此不需要也不应自定义 __call__.
33     def forward(self, X):
34         """
35         前向计算
36         """
37         y = X
38         for num_layer in range(self.num_layers):
39             y = self.linear[num_layer](y)
40             if num_layer != self.num_layers - 1:
41                 y = self.act_fn(y)
42             self.z[num_layer] = y
43         return y

```

7.4.2.2 观察模型神经元的方差变化

高斯分布初始化 构造一个 5 层网络, 激活函数取 Tanh, 按 Xavier 高斯方案初始化权重:

```

1 torch.manual_seed(0)
2 # 定义网络每层神经元的数量
3 layers_size = [100, 200, 400, 300, 200, 100]
4 # 指定激活函数
5 activate_fn = torch.nn.Tanh

```

```

6 # 指定参数初始化方式
7 init_fn_name = 'normal'
8
9 model = MLP(layers_size, init_fn_name, init_fn=xavier_normal_std, act_fn=activate_fn)
10 inputs = torch.normal(0.0, 0.1, size=[1, 100])
11 y = model(inputs)

```

让一组随机输入逐层通过网络,并打印每层的输出方差:

```

1 for i in range(len(model.z) - 1):
2     print('layer %d: , %f'%(i, model.z[i].detach().numpy().var()))

```

输出结果为:

```

layer 0: , 0.006696
layer 1: , 0.004543
layer 2: , 0.005479
layer 3: , 0.006864

```

可以看到, Xavier 高斯初始化把每一层的方差稳定在了同一个数量级, 未出现典型的指数式衰减或膨胀.

均匀分布初始化 若改用区间 $[-r, r]$ 的均匀分布采样权重, Xavier 方案下 $r = \sqrt{6/(M_{l-1} + M_l)}$. 封装为 `xavier_uniform_r`:

```

1 def xavier_uniform_r(input_size, output_size):
2     return np.sqrt(6 / (input_size + output_size))

```

沿用同一组网络结构, 仅把初始化方式改成 Xavier 均匀:

```

1 torch.manual_seed(0)
2 # 指定激活函数
3 activate_fn = torch.nn.Tanh
4 # 指定参数初始化方式
5 init_fn_name = 'uniform'
6
7 model = MLP(layers_size, init_fn_name, init_fn=xavier_uniform_r, act_fn=activate_fn)
8 inputs = torch.normal(0.0, 0.1, size=[1, 100])
9 y = model(inputs)

```

同样打印各层输出方差:

```

1 for i in range(len(model.z) - 1):
2     print('layer %d: , %f'%(i, model.z[i].detach().numpy().var()))

```

输出结果为:

```

layer 0: , 0.005573
layer 1: , 0.003334
layer 2: , 0.003089
layer 3: , 0.003602

```

数值与高斯版本几乎在同一量级——两种 Xavier 方案在保持方差稳态上效果接近。

7.4.2.3 观察模型训练收敛性

下面在一个真实训练任务上对比 Xavier 初始化和 $N(0, 1)$ 初始化的收敛速度, 看初始化是否在训练损失曲线上也能体现差异。

构建数据集 复用第 3.1 节的 `make_moons` 生成一个小型二分类数据集:

```

1 from nnrl.data import make_moons
2
3 class MoonsDataset(Dataset):
4     def __init__(self, mode='train', num_samples=300, num_train=200):
5         super().__init__()
6         X, y = make_moons(n_samples=num_samples, shuffle=True, noise=0.5, seed=0)
7         y = y.unsqueeze(1)
8         if mode == 'train':
9             self.X, self.y = X[:num_train], y[:num_train]
10        else:
11            self.X, self.y = X[num_train:], y[num_train:]
12
13    def __getitem__(self, idx):
14        return self.X[idx], self.y[idx]
15
16    def __len__(self):
17        return len(self.y)

```

分别构造训练集、验证集并包装为 `DataLoader`:

```

1 torch.manual_seed(0)
2 train_dataset = MoonsDataset(mode='train')
3 dev_dataset = MoonsDataset(mode='dev')
4 train_loader = DataLoader(train_dataset, batch_size=10, shuffle=True,
5                             generator=torch.Generator().manual_seed(0))
6 dev_loader = DataLoader(dev_dataset, batch_size=10)

```

实例化两个结构完全相同的五层 MLP: 一个使用 Xavier 高斯初始化, 另一个使用未缩放的标准正态初始化作为对照. 两者均训练 100 个回合, 再比较损失曲线:

```

1 import nnrl
2

```

```

3 torch.manual_seed(0)
4 np.random.seed(0)
5 # 定义网络每层神经元的数量
6 layers_size = [2, 300, 500, 700, 400, 1]
7 # 指定激活函数
8 activate_fn = torch.nn.Tanh
9 # 指定参数初始化方式为Xavier高斯分布初始化
10 init_fn_name = 'normal'
11 model1 = MLP(layers_size, init_fn_name, init_fn=xavier_normal_std, act_fn=activate_fn)
12 opt1 = optimizer.SGD(model1.parameters(), lr=0.005)
13 loss_fn = F.binary_cross_entropy_with_logits
14 m = lambda logits, labels: ((logits > 0).float() == labels.reshape_as(logits)).float().
    mean().item()
15 runner1 = RunnerV3(model1, opt1, loss_fn, metric_fn=m)
16 runner1.fit(train_loader, dev_loader, num_epochs=100, log_every=None)
17 # 恢复随机状态与批次顺序, 仅改变初始化尺度
18 torch.manual_seed(0)
19 train_loader.generator.manual_seed(0)
20 # 指定参数初始化方式为 $N(0, 1)$ 高斯分布初始化
21 init_fn_name = 'basic'
22 model2 = MLP(layers_size, init_fn_name, None, act_fn=activate_fn)
23 opt2 = optimizer.SGD(model2.parameters(), lr=0.005)
24 runner2 = RunnerV3(model2, opt2, loss_fn, metric_fn=m)
25 runner2.fit(train_loader, dev_loader, num_epochs=100, log_every=None)

```

训练损失曲线见图7.14.

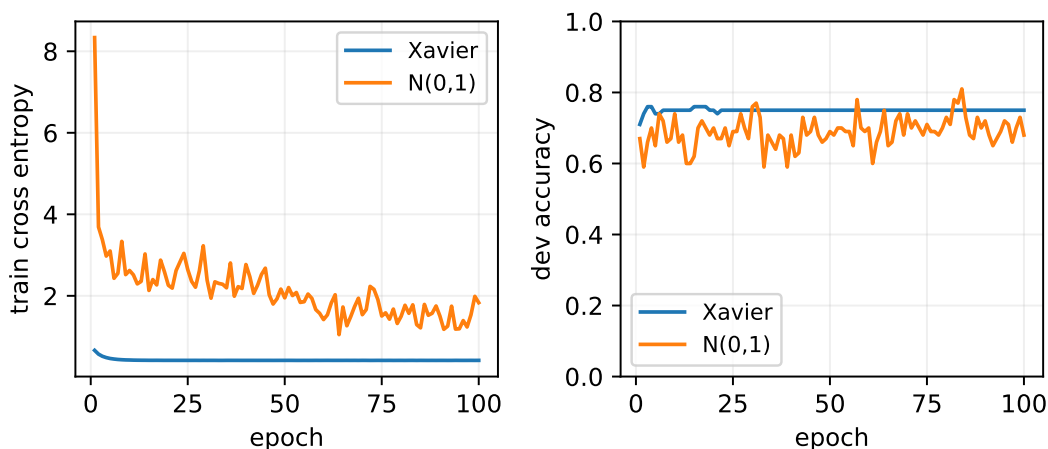


图 7.14 Xavier 与标准正态初始化的训练损失、验证准确率

可以看到,Xavier 初始化对应的损失曲线明显更低、收敛也更稳:它按每层输入输出维度调整

初始权重的方差,缓解了本例标准正态权重使 Tanh 较早饱和的问题;这不是对任意深度或学习率的收敛保证。



动手练习 7.9

按照 He 初始化的公式实现一个 `he_normal_std`, 把 MLP 的激活函数改为 ReLU 重新跑上面的实验,比较 He、Xavier、 $\mathcal{N}(0, 1)$ 三者的收敛差异。

7.5 逐层规范化

逐层规范化 (Layer-wise Normalization) 把控制数值尺度的思想用于网络中间表示: 按指定维度计算统计量并作变换,以改善训练条件。这里的“逐层规范化”是一类方法的统称,与其中具体的“层规范化 (LN)”需要区分。

随着前面若干层参数的更新,第 l 层接收到的输入分布也会变化,这种现象被称为**内部协变量偏移** (Internal Covariate Shift)。减轻这种变化是批量规范化提出时的动机之一。规范化通过控制中间表示的均值和方差等统计量改善训练条件,但并不保证整个分布保持不变,也不能仅用分布漂移解释其全部作用。

实践中最常用的两种逐层规范化是**批量规范化** (Batch Normalization, BN) (BN) 和**层规范化** (Layer Normalization, LN) (LN)。下面依次实现并对比。

7.5.1 批量规范化

批量规范化使用当前批次的统计量,对第 l 层净输入 $\mathbf{z}^{(l)}$ 逐特征或逐通道标准化,使标准化结果的均值接近 0、方差接近 1 (受 ϵ 影响)。这不会把任意分布变成正态分布。实践中规范化通常插入在线性层和激活函数之间:

$$\hat{\mathbf{z}}^{(l)} = \frac{\mathbf{z}^{(l)} - \boldsymbol{\mu}_B}{\sqrt{\sigma_B^2 + \epsilon}}, \quad (7.19)$$

其中 $\boldsymbol{\mu}_B, \sigma_B^2$ 是当前批次中该维度上的样本均值和方差。

后续计算未必需要固定的均值和方差。BN 在标准化之后引入可学习的**缩放**与**平移**参数 $(\boldsymbol{\gamma}, \boldsymbol{\beta})$, 让网络选择合适的尺度与偏移;固定的参数不能逐批恢复任意变化的原始分布:

$$\hat{\mathbf{z}}^{(l)} \triangleq \text{BN}_{\boldsymbol{\gamma}, \boldsymbol{\beta}}(\mathbf{z}^{(l)}) = \frac{\mathbf{z}^{(l)} - \boldsymbol{\mu}_B}{\sqrt{\sigma_B^2 + \epsilon}} \odot \boldsymbol{\gamma} + \boldsymbol{\beta}. \quad (7.20)$$

7.5.1.1 BatchNorm 算子

实现 BatchNorm 需要区分训练和推断两种模式: 训练时每个 mini-batch 直接按当前批次估计均值方差;推断时若仍依赖批次统计量,会让同一个样本的预测受同批其他样本影响。常见做法

是在训练过程中以指数移动平均的方式维护运行均值和方差的估计；它们并非完整训练集统计量的精确值,推断时改用这套运行统计量. 实现如下：

```

1  class BatchNorm(nn.Module):
2      def __init__(self, num_features, eps=1e-7, momentum=0.9, gamma=1.0, beta=0.0):
3          """
4          批量规范化初始化
5          输入:
6              - num_features: 输入特征数
7              - eps: 为保持数值稳定性而设置的常数
8              - momentum: 用于计算移动平均值
9              - gamma: 缩放的参数初值
10             - beta: 平移的参数初值
11          """
12          super().__init__()
13          shape = (1, num_features)
14          # gamma和beta是可学习参数, 必须用nn.Parameter包装才会被优化器更新
15          self.gamma = nn.Parameter(torch.full(shape, float(gamma)))
16          self.beta = nn.Parameter(torch.full(shape, float(beta)))
17          # moving_mean/moving_variance是不参与梯度的“缓冲区”, 用register_buffer注册后
18          # 会随model.to(device)一同迁移、并被state_dict保存/加载
19          self.register_buffer('moving_mean', torch.zeros(shape))
20          self.register_buffer('moving_variance', torch.ones(shape))
21          self.eps = eps
22          self.momentum = momentum
23
24      def forward(self, X):
25          assert X.dim() in (2, 4)
26          shape = (1, -1) if X.dim() == 2 else (1, -1, 1, 1)
27          if not self.training:
28              mean = self.moving_mean.view(shape)
29              var = self.moving_variance.view(shape)
30          else:
31              # 全连接层沿批量维统计, 卷积层沿批量、高度和宽度维统计
32              dims = (0,) if X.dim() == 2 else (0, 2, 3)
33              var, mean = torch.var_mean(X, dim=dims, keepdim=True, unbiased=False)
34              count = X.numel() // X.shape[1]
35              if count <= 1:
36                  raise ValueError("BatchNorm training needs more than one value per channel")
37              running_var = var * count / (count - 1)
38              # 缓冲区统一保存为[1, C], 参与计算时再按输入维度展开
39              self.moving_mean.mul_(self.momentum).add_(
40                  mean.detach().view_as(self.moving_mean), alpha=1 - self.momentum)
41              self.moving_variance.mul_(self.momentum).add_(
42                  running_var.detach().view_as(self.moving_variance), alpha=1 - self.momentum)

```

```

43         X = (X - mean) / torch.sqrt(var + self.eps)
44         return self.gamma.view(shape) * X + self.beta.view(shape)

```

本实现的momentum=0.9表示旧统计量的权重, 对应 PyTorch 中momentum=0.1. 训练前向采用除以观测数的方差; 写入运行方差时乘上 $K/(K-1)$ 作无偏修正, 这里的 K 是每通道参与统计的元素数. 全连接输入中 K 是批大小, 卷积输入中则包含空间位置. 推断时使用保存的统计量, 因此也要随模型保存这些缓冲区.

7.5.1.2 支持逐层规范化的 MLP 算子

为了在同一份 MLP 代码里灵活切换“无规范化 / BN / LN”, 把规范化层挂在原 MLP 上. 初始化时新增三个参数: norm_name 选择规范化类型 (None 表示不启用), gamma 和 beta 是规范化层的初始缩放与平移:

```

1  class MLP(nn.Module):
2      def __init__(self, layers_size, init_fn_name, init_fn, act_fn, norm_name=None, gamma=
        None, beta=None):
3          """
4          多层网络初始化
5          输入:
6              - layers_size: 每层神经元的数量
7              - init_fn_name: 网络中参数初始化方法
8              - init_fn: 计算高斯分布标准差或均匀分布r值
9              - act_fn: 激活函数
10             - norm_name: 使用哪一种逐层规范化
11             - gamma、beta: 缩放和平移变换的参数
12          """
13         super().__init__()
14         self.linear = nn.Sequential()
15         # 用 nn.ModuleList 保存规范化层; 无规范化的位置用 nn.Identity 占位,
16         # 这样 forward 里就不再需要分支判断
17         self.normalization = nn.ModuleList()
18         self.num_layers = len(layers_size) - 1
19         for i in range(self.num_layers):
20             input_size, output_size = layers_size[i], layers_size[i + 1]
21             if init_fn_name == 'normal':
22                 # Xavier高斯分布初始化, 计算方差
23                 layer = nn.Linear(input_size, output_size)
24                 nn.init.normal_(layer.weight, mean=0, std=init_fn(input_size, output_size))
25             elif init_fn_name == 'uniform':
26                 r = init_fn(input_size, output_size)
27                 layer = nn.Linear(input_size, output_size)
28                 nn.init.uniform_(layer.weight, a=-r, b=r)
29             else:
30                 layer = nn.Linear(input_size, output_size)

```

```

31         nn.init.normal_(layer.weight)
32     nn.init.zeros_(layer.bias)
33     self.linear.add_module(str(i), layer)
34     if norm_name == 'bn':
35         self.normalization.append(BatchNorm(output_size, gamma=gamma[i], beta=beta[i]
36                                           )))
37     elif norm_name == 'ln':
38         self.normalization.append(LayerNorm(gamma=gamma[i], beta=beta[i]))
39     else:
40         self.normalization.append(nn.Identity())
41     self.act_fn = act_fn()
42     # 中间激活值仅供可视化使用，故而单独保存
43     self.z = {}
44
45     def forward(self, X):
46         y = X
47         for num_layer in range(self.num_layers):
48             y = self.linear[num_layer](y)
49             if num_layer != self.num_layers - 1:
50                 y = self.normalization[num_layer](y) # Identity则等价于直通
51                 self.z[num_layer] = y
52                 y = self.act_fn(y)
53         return y

```

由于 BN 要在每个 mini-batch 里估计单个神经元的均值方差，因此对批大小有最低要求——批量太小，统计量会失真。

7.5.1.3 规范化后均值与方差的检查

下面构造两个结构相同但配置不同的模型：model1 不带 BN，model2 带 BN，观察 BN 是否能稳住中间层的分布：

```

1  torch.manual_seed(0)
2  # 定义网络每层神经元的数量
3  layers_size = [100, 200, 400, 300, 2, 2]
4
5  data = torch.randn(size=[200, 100])
6  print('data mean: ', data.numpy().mean())
7  print('data std: ', data.numpy().std())
8
9  activate_fn = torch.nn.Tanh
10 model1 = MLP(layers_size, 'basic', None, act_fn=activate_fn)
11 output = model1(data)
12 print('no batch normalization: ')
13 print('model output mean: ', model1.z[3].detach().mean(dim=0).numpy())
14 print('model output std: ', model1.z[3].detach().std(dim=0).numpy())

```

```

15
16 gamma = [1, 1, 1, 1, 1]
17 beta = [0, 0, 0, 0, 0]
18 model2 = MLP(layers_size, 'basic', None, act_fn=activate_fn, norm_name='bn', gamma=gamma,
               beta=beta)
19 output = model2(data)
20 print('with batch normalization: ')
21 print('model output mean: ', model2.z[3].detach().mean(dim=0).numpy())
22 print('model output std:', model2.z[3].detach().std(dim=0).numpy())

```

输出结果为:

```

data mean: -0.01193818
data std: 1.0046082
no batch normalization:
model output mean: [0.24915618 0.40605015]
model output std: [18.141748 17.855463]
with batch normalization:
model output mean: [9.5367430e-09 2.3841857e-09]
model output std: [1.0025095 1.0025095]

```

输入张量本来近似 $\mathcal{N}(0, 1)$. 穿过前 4 层后 (输出层之前、第 4 层净输入 $z[3]$), 无 BN 的版本均值和方差都明显偏离了初始值; 启用 BN ($\gamma = 1, \beta = 0$) 后, 相应通道的样本均值接近 0、方差接近 1. 这说明 BN 控制了这些统计量, 不能据此断定输出服从正态分布.

如果显式给每层设置不同的 γ 与 β , BN 就会把规范化后的分布缩放/平移到对应的均值、方差. 代码如下:

```

1 torch.manual_seed(0)
2
3 gamma = [1, 2, 3, 5, 4]
4 beta = [3, 2, 1, 2, 2]
5 model3 = MLP(layers_size, 'basic', None, act_fn=activate_fn, norm_name='bn', gamma=gamma,
               beta=beta)
6 output = model3(data)
7 print('batch normalization with different gamma and beta for different layer: ')
8 print('output means with bn 0: ', model3.z[0].detach().numpy().mean())
9 print('output stds with bn 0: ', model3.z[0].detach().numpy().std())
10 print('output means with bn 3: ', model3.z[3].detach().numpy().mean())
11 print('output stds with bn 3: ', model3.z[3].detach().numpy().std())

```

输出结果为:

```

batch normalization with different gamma and beta for different layer:
output means with bn 0: 3.0
output stds with bn 0: 1.0

```



```
output means with bn 3: 2.0
output stds with bn 3: 5.0
```

7.5.1.4 均值和方差的移动平均计算实验

接下来验证 BN 在训练阶段对 μ 、 σ^2 的移动平均累积. 这里让网络做 50 次前向 (不更新权重, 仅触发 BN 的统计累积逻辑), 最后打印第 3 层 BN 层累积的均值方差:

```
1 torch.manual_seed(0)
2
3 epochs = 50
4 for epoch in range(epochs):
5     inputs = torch.randn(size=[200, 100])
6     output = model3(inputs)
7     # 打印批量规范化中训练样本均值和方差的移动平均值
8     print('batch norm 3 moving mean: ', model3.normalization[3].moving_mean.numpy())
9     print('batch norm 3 moving variance: ', model3.normalization[3].moving_variance.numpy())
```

输出结果为:

```
batch norm 3 moving mean: [[-1.9143263  3.6829247]]
batch norm 3 moving variance: [[156.54797 189.72693]]
```

切换到推断模式 (`model3.eval()`), BN 会改用上面累积的均值方差代替当前批次的统计量:

```
1 torch.manual_seed(0)
2
3 inputs_test = torch.randn(size=[5, 100])
4 model3.eval() # 切换到推断模式: BN 改用累积的移动均值/方差
5 output = model3(inputs_test)
```

7.5.1.5 在 MNIST 数据集上使用带批量规范化的卷积网络

本节先观察 BN 对实际训练曲线的影响. 单次前向中的均值和方差检查, 不能说明训练期间分布如何变化, 也不足以确定 BN 改善优化的因果机制.

为了直观地体会这一点, 下面搭一个轻量级 CNN 在 MNIST 上做对照实验, 观察启用 BN 前后收敛速度的差异:

```
1 from torch.nn import Conv2d, MaxPool2d, Linear, BatchNorm2d
2 # 多层卷积神经网络实现
3 class MultiConvLayerNet(nn.Module):
4     def __init__(self, use_bn=False):
5         super().__init__()
6         # 定义卷积层, 输出特征通道out_channels设置为20, 卷积核的大小kernel_size为5, 卷积步长
           stride=1, padding=2
```

```

7         self.conv1 = Conv2d(in_channels=1, out_channels=20, kernel_size=5, stride=1,
                             padding=2)
8         # 定义汇聚层, 窗口的大小为2, 步长为2
9         self.max_pool1 = MaxPool2d(kernel_size=2, stride=2)
10        # 定义卷积层, 输出特征通道out_channels设置为20, 卷积核的大小kernel_size为5, 卷积步长
           stride=1, padding=2
11        self.conv2 = Conv2d(in_channels=20, out_channels=20, kernel_size=5, stride=1,
                             padding=2)
12        # 定义汇聚层, 窗口的大小为2, 步长为2
13        self.max_pool2 = MaxPool2d(kernel_size=2, stride=2)
14        # 定义一层全连接层, 输出维度是10
15        self.fc = Linear(980, 10)
16        if use_bn:
17            # 定义批量规范化层
18            self.batch_norm1 = BatchNorm2d(num_features=20)
19            self.batch_norm2 = BatchNorm2d(num_features=20)
20            self.use_bn = use_bn
21        # 定义网络前向计算过程
22        def forward(self, inputs):
23            x = self.conv1(inputs)
24            if self.use_bn:
25                x = self.batch_norm1(x)
26            x = F.relu(x)
27            x = self.max_pool1(x)
28            x = self.conv2(x)
29            if self.use_bn:
30                x = self.batch_norm2(x)
31            x = F.relu(x)
32            x = self.max_pool2(x)
33            x = torch.reshape(x, [x.shape[0], 980])
34            x = self.fc(x)
35            return x

```

分别实例化“无 BN”与“有 BN”两个网络在 MNIST 上训练:

```

1  bn_runners = {}
2  for use_bn in [False, True]:
3      torch.manual_seed(0)
4      model = MultiConvLayerNet(use_bn=use_bn)
5      train_loader = DataLoader(Subset(mnist_train, indices[:5000].tolist()),
6                                  batch_size=64, shuffle=True,
7                                  generator=torch.Generator().manual_seed(0))
8      dev_loader = DataLoader(mnist_dev, batch_size=128)
9      opt = torch.optim.Adam(model.parameters(), lr=0.01)
10     runner = RunnerV3(model, opt, F.cross_entropy, metric_fn=accuracy)

```

```

11 runner.fit(train_loader, dev_loader, num_epochs=5, log_every=None)
12 bn_runners[use_bn] = runner

```

训练损失和验证准确率曲线对比见图7.15: 两组使用同一份 5 000/1 000 条训练/验证划分、相同卷积及线性层初始权重和批次顺序. 比较损失与准确率时应一起看, 不能将一次 5 回合运行解释为稳定的普遍排序.

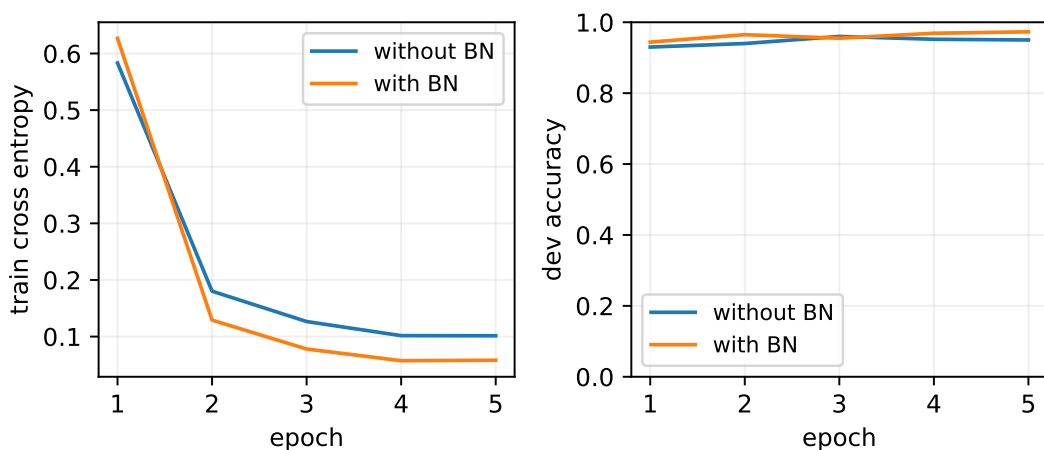


图 7.15 批量规范化的对比实验

7.5.2 层规范化

在本节的二维全连接表示中, BN 对同一特征在当前小批量内求均值和方差, LN 对同一样本的特征维求统计量, 随后都使用可学习的缩放与平移. 卷积 BN 通常还在空间维上汇总同一通道; Transformer 的 LN 则通常逐词元处理其隐藏向量.

层规范化的定义为

$$\hat{\mathbf{z}}^{(l)} = \frac{\mathbf{z}^{(l)} - \mu^{(l)}}{\sqrt{\sigma^{(l)^2} + \epsilon}} \odot \boldsymbol{\gamma} + \boldsymbol{\beta}, \quad (7.21)$$

$$\triangleq \text{LN}_{\boldsymbol{\gamma}, \boldsymbol{\beta}}(\mathbf{z}^{(l)}), \quad (7.22)$$

其中 $\mathbf{z}^{(l)}$ 是第 l 层的净输入, $\mu^{(l)}$ 和 $\sigma^{(l)^2}$ 是 $\mathbf{z}^{(l)}$ 沿特征维的均值和方差, $\boldsymbol{\gamma}, \boldsymbol{\beta}$ 是与 $\mathbf{z}^{(l)}$ 形状相同的可学习参数 (缩放与平移). 为简化实现, 下文代码将其取为标量、前向时按广播规则展开.

把小批量 $\mathbf{z}^{(l)} = [\mathbf{z}^{(1,l)}; \dots; \mathbf{z}^{(K,l)}]$ 写成 K 行矩阵, LN 对每行处理, BN 对每列处理. BN 常用于 CNN, LN 常用于序列模型; 选择依据包括结构、统计维度和有效批大小, 不能断言 BN 的收敛与泛化总优于 LN.

7.5.2.1 层规范化算子

实现层规范化的关键是把均值方差沿“样本内特征维”聚合,而非沿“跨样本批维”. LN 没有训练/推断双模式,也不需要维护移动平均,因此实现比 BN 更简洁:

```

1  class LayerNorm(nn.Module):
2      def __init__(self, eps=1e-7, gamma=1.0, beta=0.0):
3          """
4          层规范化初始化
5          输入:
6              - eps: 为保持数值稳定性而设置的常数
7              - gamma: 缩放的参数初值
8              - beta: 平移的参数初值
9          """
10         # nn.Module.__init__()不接受参数
11         super().__init__()
12         # gamma和beta是可学习参数;这里用标量形式,前向时会按广播规则展开
13         self.gamma = nn.Parameter(torch.tensor(float(gamma)))
14         self.beta = nn.Parameter(torch.tensor(float(beta)))
15         self.eps = eps
16
17     def forward(self, X):
18         # 二维/三维输入规范化最后一维;四维输入规范化C、H、W三维
19         assert X.dim() in (2, 3, 4)
20         dims = (1, 2, 3) if X.dim() == 4 else (-1,)
21         var, mean = torch.var_mean(X, dim=dims, keepdim=True, unbiased=False)
22         X = (X - mean) / torch.sqrt(var + self.eps)
23         return self.gamma * X + self.beta

```

7.5.2.2 层规范化的验证实验

随机生成一组形状 [10, 100] 的数据,输入带 LN 的 5 层网络. 给每层指定不同的 (γ, β) , 观察所选维度上的均值与标准差; ϵ 与浮点误差使结果只近似符合目标值:

```

1  torch.manual_seed(0)
2  # 定义网络每层神经元的数量
3  layers_size = [100, 200, 400, 300, 2, 2]
4  # 随机生成数据
5  data = torch.randn(size=[10, 100])
6  activate_fn = torch.nn.Tanh
7  gamma = [1, 2, 3, 5, 4]
8  beta = [3, 2, 1, 2, 2]
9  model = MLP(layers_size, 'basic', None, act_fn=activate_fn, norm_name='ln', gamma=gamma,
              beta=beta)
10 output = model(data)
11 print('layer normalization with different gamma and beta for different layer: ')

```

```

12 print('output means with ln 0: ', model.z[0].detach().mean(dim=-1).numpy())
13 print('output stds with ln 0: ', model.z[0].detach().std(dim=-1, unbiased=False).numpy())
14 print('output means with ln 1: ', model.z[3].detach().mean(dim=-1).numpy())
15 print('output stds with ln 1: ', model.z[3].detach().std(dim=-1, unbiased=False).numpy())

```

输出结果为：

```

layer normalization with different gamma and beta for different layer:
output means with ln 0: [3.    3.    3.    3.    3.0000002 2.9999998 3.
 3.    3.    3.    ]
output stds with ln 0: [1.    1.    1.    1.    1.    1.
 1.    1.    1.    0.99999994]
output means with ln 1: [2.  2.  2.  2.  2.  2.  2.  2.]
output stds with ln 1: [5.    5.    5.    5.    5.    5.
 4.9999995 5.    5.0000005]

```

由于 LN 只用样本自身的特征做归一化，不存在“训练时统计、推断时用”的双模式，因此训练/推断走的是同一段代码——这一点既是 LN 对小 batch 友好的根源，也是它和 BN 在工程行为上最显眼的区别。



动手练习 7.10

在 MNIST 上对比启用/不启用层规范化的卷积网络的收敛曲线，并与上一节 BN 的实验做横向对照。



7.6 网络正则化方法

当训练表现继续改善，而独立验证集上的表现停滞或变差时，应检查过拟合。**正则化**（Regularization）通过约束、扰动或提前停止等方式影响模型选择；训练与验证之间的差距还可能来自分布差异，不能一概归因于容量。

本节固定一个有噪声的小数据集与三层 MLP，在相同初始化和较长训练预算下比较不同处理。实验先报告训练与验证指标，再选定一个模型评价留出的测试集。

7.6.1 数据集构建

继续复用第 3.1.1 节的 make_moons：取 400 条样本，前 200 条作训练集、中间 100 条作验证集、最后 100 条留作测试集。样本量小、噪声大，正适合让深层网络“过拟合”：

```

1 torch.manual_seed(0)
2 # 固定生成一份数据，隔离训练、验证、测试
3 n_samples = 400

```

```

4 num_train = 200
5 # 根据make_moons生成二分类数据集
6 data_X, data_y = make_moons(n_samples=n_samples, shuffle=True, noise=0.5, seed=0)
7 X_train, y_train = data_X[:num_train], data_y[:num_train]
8 X_dev, y_dev = data_X[num_train:300], data_y[num_train:300]
9
10 y_train = y_train.reshape([-1, 1])
11 y_dev = y_dev.reshape([-1, 1])
12 X_test, y_test = data_X[300:], data_y[300:].reshape(-1, 1)
13 regularization_results = {}
14 print('train dataset X shape: ', X_train.shape)
15 print('train dataset y shape: ', y_train.shape)

```

输出结果为:

```

train dataset X shape: torch.Size([200, 2])
train dataset y shape: torch.Size([200, 1])

```

7.6.2 模型构建

为了让正则化的实现过程足够透明,本节采用书中自定义的Op框架而非`torch.nn`,搭一个三层MLPMLP_3L:隐藏层用ReLU,输出层是普通线性层,直接输出未归一化分数(logits)。

先实现ReLU算子,再复用第4.2.4.4节的Linear算子组装出MLP_3L。

7.6.2.1 ReLU 算子

记矩阵 $\mathbf{Z} \in \mathbb{R}^{N \times D}$ (N 个样本、 D 个特征)。ReLU的前向就是逐元素取正部:

$$\mathbf{A} = \max(\mathbf{Z}, 0) \in \mathbb{R}^{N \times D}, \quad (7.23)$$

其中 $\mathbf{A}$ 是激活后的活性值。

设上传来的梯度为 $\delta_{\mathbf{A}} = \partial \mathcal{R} / \partial \mathbf{A} \in \mathbb{R}^{N \times D}$,则ReLU的反向只需根据“前向是否 >0 ”做掩码:

$$\delta_{\mathbf{Z}} = \delta_{\mathbf{A}} \odot (\mathbf{A} > 0) \in \mathbb{R}^{N \times D}, \quad (7.24)$$

$\delta_{\mathbf{Z}}$ 即反向算子要返回的梯度。ReLU没有可学习参数,因此backward无需维护参数梯度,只把上游梯度按掩码透传即可:

```

1 class ReLU(Op):
2     def __init__(self):
3         self.inputs = None
4
5     def forward(self, inputs):
6         self.inputs = inputs

```

```

7         # clamp(min=0) 与 max(x, 0) 等价, 但均对负值输出0
8         return inputs.clamp(min=0)
9
10    def backward(self, outputs_grads):
11        # ReLU没有参数, 反向只把上游梯度按“前向是否>0”的掩码透传
12        return outputs_grads * (self.inputs > 0).to(outputs_grads.dtype)

```

7.6.2.2 自定义多层感知器

MLP_3L由三层`Linear`串联: 前两层后接ReLU、第三层是输出层(无激活, 直接给出 logits)。权重以标准正态采样初始化, 偏置置零。forward负责前向, backward负责反向并把参数梯度写进各`Linear`的`grads`属性, 再交给优化器更新:

```

1  from nndl import Op, LinearLayer, logistic
2
3  class MLP_3L(Op):
4      def __init__(self, layers_size):
5          self.fc1 = LinearLayer(layers_size[0], layers_size[1], name='fc1')
6          # ReLU激活函数
7          self.act_fn1 = ReLU()
8          self.fc2 = LinearLayer(layers_size[1], layers_size[2], name='fc2')
9          self.act_fn2 = ReLU()
10         self.fc3 = LinearLayer(layers_size[2], layers_size[3], name='fc3')
11         self.layers = [self.fc1, self.act_fn1, self.fc2, self.act_fn2, self.fc3]
12         # 父类 Op.__call__ 已经把单参调用转发到 forward, 无需再写 __call__
13         def forward(self, X):
14             z1 = self.fc1(X)
15             a1 = self.act_fn1(z1)
16             z2 = self.fc2(a1)
17             a2 = self.act_fn2(z2)
18             z3 = self.fc3(a2)
19             return z3
20
21         def backward(self, loss_grad_z3):
22             loss_grad_a2 = self.fc3.backward(loss_grad_z3)
23             loss_grad_z2 = self.act_fn2.backward(loss_grad_a2)
24             loss_grad_a1 = self.fc2.backward(loss_grad_z2)
25             loss_grad_z1 = self.act_fn1.backward(loss_grad_a1)
26             loss_grad_inputs = self.fc1.backward(loss_grad_z1)

```

7.6.2.3 损失函数算子

第4.2.4.2节介绍的`BinaryCrossEntropyLoss`默认输入概率。这里MLP_3L的最后一层直接输出 logits, 因此不能直接套用。

这里需要在交叉熵内部先把 logits 经 Logistic 函数映射为概率, 再做交叉熵; 数值实现则直接使用稳定的融合表达式, 避免先形成可能饱和的概率; PyTorch 的 `BCEWithLogitsLoss` 采用这种处理, 相比“先 `sigmoid` 再 `BCELoss`”更稳定。

把公式(3.5)稍作改写: 令 $\mathbf{y} \in \{0, 1\}^N$ 为真实标签, $\mathbf{o} \in \mathbb{R}^N$ 为模型输出的 logits 向量, 则二分类交叉熵为:

$$\mathcal{R}(\mathbf{y}, \mathbf{o}) = -\frac{1}{N} \left(\mathbf{y}^\top \log \sigma(\mathbf{o}) + (1 - \mathbf{y})^\top \log (1 - \sigma(\mathbf{o})) \right), \quad (7.25)$$

其中 σ 是 Logistic 函数。

损失关于未归一化分数 $\mathbf{o}$ 的梯度为:

$$\frac{\partial \mathcal{R}}{\partial \mathbf{o}} = -\frac{1}{N} (\mathbf{y} - \sigma(\mathbf{o})). \quad (7.26)$$

把上述前向与反向直接写到 `BinaryCrossEntropyWithLogits` 算子里:

```

1  class BinaryCrossEntropyWithLogits(Op):
2      def __init__(self, model):
3          self.predicts = None
4          self.labels = None
5          self.data_size = None
6          self.model = model
7          self.logistic = logistic
8
9      def forward(self, logits, labels):
10         self.predicts = self.logistic(logits)
11         self.labels = labels
12         self.data_size = self.predicts.shape[0]
13         # 直接从 logits 计算, 避免 sigmoid 饱和后出现 log(0)
14         return F.binary_cross_entropy_with_logits(logits, labels, reduction='mean')
15
16     def backward(self):
17         inputs_grads = 1./ self.data_size * (self.predicts - self.labels)
18         self.model.backward(inputs_grads)

```

`layer.params[key]`, `layer.grads[key]`, `self.init_lr`, `self.weight_decay` if `key == 'W'` else 0.0)

额外封装一个 `accuracy_logits`: 直接从 logits 计算准确率 (二分类按是否大于 0 判定、多分类按 `argmax`):

```

1  def accuracy_logits(logits, labels):
2      """
3      输入:
4          - logits: 预测值, 二分类时, size=[N, 1], N 为样本数量; 多分类时, size=[N, C], C 为类别数量
5          - labels: 真实标签, size=[N, 1]

```

```

6     输出:
7     - 准确率: size=[1]
8     """
9     # 判断是二分类任务还是多分类任务, preds.shape[1]=1时为二分类任务, preds.shape[1]>1时为多分类
      任务
10    if logits.shape[1] == 1:
11        # 二分类时, 判断每个 logits 是否大于 0, 当大于 0 时类别为 1, 否则类别为 0
12        # 使用 .to(torch.float32) 将 preds 的数据类型转换为 float32 类型
13        preds = (logits > 0).to(torch.float32)
14    else:
15        # 多分类时, 使用 'torch.argmax' 计算最大元素索引作为类别
16        preds = torch.argmax(logits, dim=1).to(torch.int32)
17
18    # 对齐形状: 多分类时 preds 为 [N]、labels 为 [N,1], 不对齐会广播成 [N,N]
19    labels = labels.reshape(preds.shape)
20    return torch.mean(torch.eq(preds, labels).to(torch.float32))

```

7.6.2.4 模型训练

下面用 `train_model` 串起整个训练流程: 接收数据、网络、优化器、损失函数、迭代步数, 循环执行前向、反向和更新, 并记录每 100 步的损失, 便于后续可视化:

```

1  def train_model(X_train, y_train, X_dev, y_dev, model, optimizer, loss_fn, num_iters, *
      args):
2      """
3      训练模型
4      输入:
5          - X_train, y_train: 训练集数据
6          - X_dev, y_dev: 验证集数据
7          - model: 定义网络
8          - optimizer: 优化器
9          - loss_fn: 损失函数
10         - num_iters: 训练迭代次数
11         - args: 在 dropout 中指定模型为训练模式或评价模式
12         """
13         losses = []
14         for i in range(num_iters):
15             # 前向计算
16             train_logits = model(X_train)
17             loss = loss_fn(train_logits, y_train)
18             # 反向计算
19             loss_fn.backward()
20             # 更新参数
21             optimizer.step()
22             if i % 100 == 0:

```

```

23         losses.append(loss.item())
24
25     train_logits = model(X_train, *args)
26     acc_train = accuracy_logits(train_logits, y_train)
27     test_logits = model(X_dev, *args)
28     acc_dev = accuracy_logits(test_logits, y_dev)
29     print('train accuracy:', acc_train.numpy())
30     print('dev accuracy:', acc_dev.numpy())
31     return losses

```

先用第4.2.4.6节的BatchGD做一次“无正则化基线”：训练 50 000 次，记录训练集与验证集准确率：

```

1  from nndl import BatchGD
2  torch.manual_seed(0)
3  layers_size = [X_train.shape[1], 20, 3, 1]
4  model = MLP_3L(layers_size)
5  opt = BatchGD(init_lr=0.2, model=model)
6  loss_fn = BinaryCrossEntropyWithLogits(model)
7  losses = train_model(X_train, y_train, X_dev, y_dev, model, opt, loss_fn, 50000)
8  regularization_results["baseline"] = model

```

输出结果为：

```

train accuracy: 0.885
dev accuracy: 0.72

```

本次基线的训练准确率为 88.5%，验证准确率为 72%。二者存在差距，但并没有记住所有训练点；还应结合损失曲线与决策边界，检查噪声、样本量和模型容量的影响。

辅助函数show_class_boundary的实现如下：

```

1  import matplotlib.pyplot as plt
2  from matplotlib.colors import ListedColormap
3
4  def show_class_boundary(model, X_train, y_train, *args):
5      palette = ListedColormap(['#4E79A7', '#E76F51'])
6      lower, upper = X_train.min(dim=0).values - 0.5, X_train.max(dim=0).values + 0.5
7      x1, x2 = torch.meshgrid(torch.linspace(lower[0], upper[0], 200),
8                              torch.linspace(lower[1], upper[1], 200), indexing='ij')
9      grid = torch.stack([x1.flatten(), x2.flatten()], dim=1)
10     labels = (model(grid, *args) > 0).reshape_as(x1)
11     plt.contourf(x1, x2, labels, levels=[-0.5, 0.5, 1.5], cmap=palette, alpha=0.25)
12     plt.scatter(X_train[:, 0], X_train[:, 1], c=y_train.flatten(), cmap=palette,
13               s=18, edgecolors='white', linewidths=0.4)
14     plt.xlabel('x1'); plt.ylabel('x2')

```

```
15 plt.tight_layout()
```

```
1 show_class_boundary(model, X_train, y_train)
```

图7.16中,散点颜色代表真实类别,背景代表预测类别.观察哪些训练点落在另一类别的区域,以及边界是否围绕少数点发生弯折,再与验证表现对照;边界复杂本身不是过拟合的充分证据.

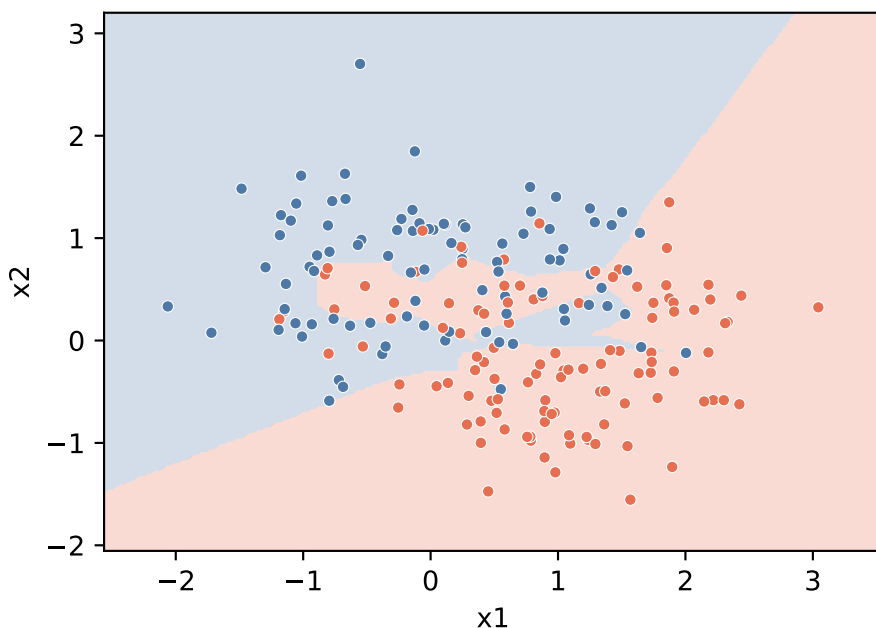


图 7.16 MLP_3L 的分类决策边界可视化

7.6.3 ℓ_1 和 ℓ_2 正则化

ℓ_1 和 ℓ_2 正则化是最经典的正则化手段:通过约束参数本身的 ℓ_1 或 ℓ_2 范数,限制模型容量,缓解过拟合.带正则项的优化问题为

$$\theta^* = \arg \min_{\theta} \frac{1}{B} \sum_{n=1}^B \mathcal{L}(y^{(n)}, f(\mathbf{x}^{(n)}; \theta)) + \lambda \Omega(\theta), \quad (7.27)$$

其中 $\mathcal{L}(\cdot)$ 是损失函数, B 是批大小, $f(\cdot)$ 是网络, θ 是参数, Ω 是惩罚函数, 如 $\|\theta\|_1$ 或 $\frac{1}{2}\|\theta\|_2^2$, λ 是正则化强度.

下面把 ℓ_2 正则化加到上面的过拟合实验里:本例沿用第 2 章的样本数缩放约定,前向多加 $\frac{\lambda}{2N} \sum_l \|\mathbf{W}_l\|_F^2$, 反向给各权重梯度加上 $\frac{\lambda}{N} \mathbf{W}_l$, 偏置不受惩罚.这里 $N = 200$ 是固定训练集大小;改用小批训练时不要把 N 替换为当前批大小,否则正则化强度也随之改变.

把这两部分写进BinaryCrossEntropyWithLogits:

```

1  class BinaryCrossEntropyWithLogits(Op):
2      def __init__(self, model, lamdb, num_train):
3          self.predicts = None
4          self.labels = None
5          self.data_size = None
6          self.model = model
7          self.logistic = logistic
8          self.lamdb = lamdb
9          self.num_train = num_train
10
11     def forward(self, logits, labels):
12         self.predicts = self.logistic(logits)
13         self.labels = labels
14         self.data_size = self.predicts.shape[0]
15         # 直接从 logits 计算, 避免 sigmoid 饱和后出现 log(0)
16         loss = F.binary_cross_entropy_with_logits(logits, labels, reduction='mean')
17         regularization_loss = 0
18         for layer in self.model.layers:
19             if isinstance(layer, LinearLayer):
20                 regularization_loss += torch.sum(torch.square(layer.params['W']))
21         loss += self.lamdb * regularization_loss / (2 * self.num_train)
22         return loss
23
24     def backward(self):
25         inputs_grads = 1./ self.data_size * (self.predicts - self.labels)
26         self.model.backward(inputs_grads)
27         #更新正则化项对应的梯度
28         for layer in self.model.layers:
29             if isinstance(layer, LinearLayer) and isinstance(layer.grads, dict):
30                 layer.grads['W'] += self.lamdb * layer.params['W'] / self.num_train

```

取 $\lambda = 0.7$, 其他配置完全不变重新训练 50 000 步:

```

1  torch.manual_seed(0)
2  model = MLP_3L(layers_size)
3  opt = BatchGD(init_lr=0.2, model=model)
4  loss_fn = BinaryCrossEntropyWithLogits(model, lamdb=0.7, num_train=len(X_train))
5  losses = train_model(X_train, y_train, X_dev, y_dev, model, opt, loss_fn, num_iters=50_000
6                        )
6  regularization_results["l2"] = model

```

输出结果为:

```
train accuracy: 0.83
```

```
dev accuracy: 0.76
```

加入 ℓ_2 惩罚后, 本次训练准确率为 83%, 验证准确率为 76%. 在相同初始化和训练预算下, 验证集多预测对 4 条样本; 这支持该系数在本次划分上有帮助, 不代表任意增大正则化都会改善泛化.

再画一次决策边界图作直观确认:

```
1 show_class_boundary(model, X_train, y_train)
```

图 7.17 展示加入惩罚后的边界. 与基线一起观察权重受约束后哪些区域改变, 避免只凭“更平滑”就判定模型更好.

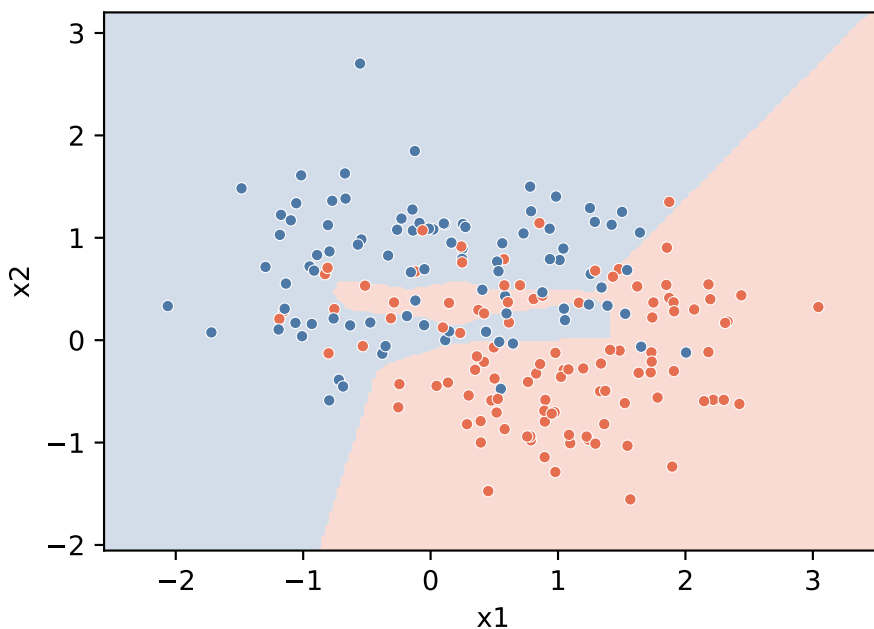


图 7.17 使用 ℓ_2 正则化的分类决策边界可视化

7.6.4 权重衰减

权重衰减 (Weight Decay) 是另一条常用思路: 不再修改损失函数, 而是在每次更新参数时让权重自身按比例缩小:

$$\theta_t \leftarrow (1 - \alpha\beta)\theta_{t-1} - \alpha\mathbf{g}_t, \quad (7.28)$$

其中 $\mathbf{g}_t$ 是未加入惩罚项的梯度, α 是学习率, β 为权重衰减系数, 每步直接缩小的比例是 $\alpha\beta$. 在无动量 SGD、相同参数范围与匹配系数下, 解耦衰减等价于在损失中加入 $\beta\|\theta\|_2^2/2$; 本例只衰减权重,

偏置不变。PyTorch 的 `weight_decay` 也采用需乘学习率的系数口径。对 Adam 这类自适应优化器，加入 ℓ_2 惩罚与解耦权重衰减不等价，这是 AdamW 的关键区别。

下面给 BatchGD 加上 `weight_decay` 参数，让每次更新都先把权重缩小一点：

```

1 class BatchGD(Optimizer):
2     def __init__(self, init_lr, model, weight_decay):
3         """
4         带权重衰减的批量梯度下降优化器初始化（此处覆盖前面 from nndl 导入的同名 BatchGD）
5         输入：
6             - init_lr: 初始学习率
7             - model: 模型，model.params 存储模型参数值
8             - weight_decay: 权重衰减系数 beta，每步先把权重缩小 (1-lr*beta) 倍，偏置不衰减
9         """
10        super().__init__(init_lr=init_lr, model=model)
11        self.weight_decay = weight_decay
12
13    def gradient_descent(self, x, gradient_x, init_lr, decay):
14        """
15        梯度下降更新一次参数
16        """
17        x = (1 - init_lr * decay) * x - init_lr * gradient_x
18        return x
19
20    def step(self):
21        """
22        参数更新
23        """
24        for layer in self.model.layers:
25            if hasattr(layer, 'params') and isinstance(layer.params, dict):
26                for key in layer.params.keys():
27                    layer.params[key] = self.gradient_descent(
28                        layer.params[key], layer.grads[key], self.init_lr,
29                        self.weight_decay if key == 'W' else 0.0)

```

取 $\beta = \lambda/N = 0.7/200$ ，使其与上一实验的 ℓ_2 系数匹配，关掉 ℓ_2 正则化 (`lambd=0`)，其他配置保持不变重新训练：

```

1 torch.manual_seed(0)
2 model = MLP_3L(layers_size)
3 opt = BatchGD(init_lr=0.2, model=model, weight_decay=0.7 / len(X_train))
4 loss_fn = BinaryCrossEntropyWithLogits(model, lambd=0, num_train=len(X_train))
5 losses = train_model(X_train, y_train, X_dev, y_dev, model, opt, loss_fn, num_iters=50000)
6 regularization_results["decay"] = model

```

输出结果为：

```
train accuracy: 0.835
dev accuracy: 0.77
```

匹配系数的权重衰减得到 83.5% 的训练准确率和 77% 的验证准确率, 与 ℓ_2 实验接近. 两种更新在实数运算下等价; 浮点运算顺序不同, 舍入误差经过 50 000 次非线性更新后可能改变少量预测. 本轮逐步核对了匹配系数的一次更新, 并未把长期训练的细小差异解释为算法收益.

再画一次决策边界图:

```
1 show_class_boundary(model, X_train, y_train)
```

图 7.18 应与匹配系数的 ℓ_2 实验一起阅读. 此处两者体现的是同一个无动量 SGD 更新关系, 不能当成互不相干的两种改进.

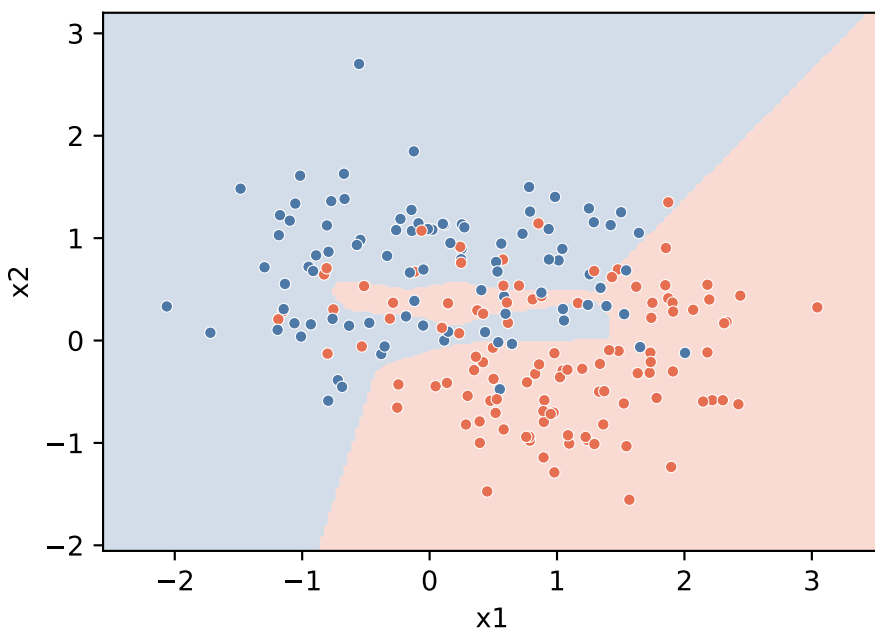


图 7.18 使用权重衰减的分类决策边界可视化

7.6.5 暂退法

暂退法 (Dropout) 的思路与前两种正则化截然不同: 训练时在每一步**随机**屏蔽一部分神经元 (输出强制设为 0), 给中间表示加入随机扰动, 减少对某些激活组合的依赖, 可能缓解过拟合. 最常见的做法是给每个神经元独立设定一个固定的丢弃概率 p .

记某中间层激活为 $\mathbf{X} \in \mathbb{R}^{B \times D}$ (B 批量、 D 维), 引入与之形状相同的二值掩码 $\mathbf{M}$, 其每个元素以概率 p 取 0、以概率 $1 - p$ 取 1.

仅做随机置零会使训练时的激活期望变为原来的 $1 - p$ 倍。常见做法是“inverted dropout”：训练阶段把保留下来的激活放大 $1/(1 - p)$ 倍，使给定输入时对随机掩码取平均的输出等于原输入，从而与推断阶段保持一致。这里保持的是期望，并非方差。

由此得到 Dropout 的前向定义：

$$\tilde{\mathbf{X}} = \text{dropout}(\mathbf{X}) \triangleq \begin{cases} (\mathbf{X} \odot \mathbf{M}) / (1 - p) & \text{当训练阶段时,} \\ \mathbf{X} & \text{当测试阶段时.} \end{cases} \quad (7.29)$$



提醒

公式(7.29)与《神经网络与深度学习》主书的公式(7.74)略有不同——未缩放的暂退法在训练阶段输出 $\mathbf{X} \odot \mathbf{M}$ ，推断阶段输出 $(1 - p)\mathbf{X}$ ，本书采用“训练阶段缩放、推断阶段保留”的 inverted dropout 版本，这也是 PyTorch、TensorFlow 等框架的默认实现。

反向同样需要把放大因子带回去：记 $\delta_{\tilde{\mathbf{X}}} = \partial \mathcal{L} / \partial \tilde{\mathbf{X}}$ ，则

$$\delta_{\mathbf{X}} = \delta_{\tilde{\mathbf{X}}} \odot \mathbf{M} / (1 - p). \quad (7.30)$$

注意：被屏蔽神经元的梯度也是 0，与前向行为完全对齐。

7.6.5.1 Dropout 算子

实现 Dropout 算子需要保存训练阶段使用的掩码，以便反向传播能复用相同的 mask；同时要区分训练/推断两种模式：

```
1 class Dropout(Op):
2     def __init__(self, drop_rate):
3         self.mask = None
4         if not 0 <= drop_rate < 1:
5             raise ValueError("drop_rate must be in [0, 1)")
6         self.drop_rate = drop_rate
7
8     def forward(self, inputs):
9         # 生成一个丢弃掩码
10        mask = torch.rand_like(inputs) > self.drop_rate
11        self.mask = mask
12        # 随机使一些神经元失效
13        inputs = torch.mul(inputs, mask)
14        # 使给定输入时的输出期望保持不变
15        inputs /= (1 - self.drop_rate)
16        return inputs
17
```

```

18     def backward(self, outputs_grad):
19         return torch.mul(outputs_grad, self.mask) / (1 - self.drop_rate)

```

把Dropout算子嵌进MLP_3L得到MLP_3L_dropout:在两层ReLU激活之后各插一个Dropout层,前向时按mode决定是否启用,反向时同步走对应的反向:

```

1
2 class MLP_3L_dropout(MLP_3L):
3     def __init__(self, layers_size, drop_rate):
4         super().__init__(layers_size)
5         self.dropout1 = Dropout(drop_rate)
6         self.dropout2 = Dropout(drop_rate)
7         self.layers = [self.fc1, self.act_fn1, self.fc2, self.act_fn2, self.fc3]
8
9     def forward(self, X, mode='train'):
10        self.mode = mode
11        z1 = self.fc1(X)
12        a1 = self.act_fn1(z1)
13        if self.mode == 'train':
14            a1 = self.dropout1(a1)
15        z2 = self.fc2(a1)
16        a2 = self.act_fn2(z2)
17        if self.mode == 'train':
18            a2 = self.dropout2(a2)
19        z3 = self.fc3(a2)
20        return z3
21
22    def backward(self, loss_grad_z3):
23        loss_grad_a2 = self.fc3.backward(loss_grad_z3)
24        if self.mode == 'train':
25            loss_grad_a2 = self.dropout2.backward(loss_grad_a2)
26        loss_grad_z2 = self.act_fn2.backward(loss_grad_a2)
27        loss_grad_a1 = self.fc2.backward(loss_grad_z2)
28        if self.mode == 'train':
29            loss_grad_a1 = self.dropout1.backward(loss_grad_a1)
30        loss_grad_z1 = self.act_fn1.backward(loss_grad_a1)
31        loss_grad_inputs = self.fc1.backward(loss_grad_z1)

```

取丢弃概率 $p = 0.3$,关掉 ℓ_2 正则化与权重衰减,单独看 Dropout 的效果:

```

1 torch.manual_seed(0)
2 model = MLP_3L_dropout(layers_size, drop_rate=0.3)
3 opt = BatchGD(init_lr=0.2, model=model, weight_decay=0)
4 loss_fn = BinaryCrossEntropyWithLogits(model, lambd=0, num_train=len(X_train))
5 losses = train_model(X_train, y_train, X_dev, y_dev, model, opt, loss_fn, 50000, 'dev')
6 regularization_results["dropout"] = model

```

输出结果为：

```
train accuracy: 0.82  
dev accuracy: 0.78
```

暂退率为 0.3 时, 本次训练准确率为 82%, 验证准确率为 78%。训练集评价也关闭随机暂退, 确保两项指标使用相同的前向规则。验证集只比权重衰减多预测对 1 条样本, 还不足以形成稳定的优劣结论。

再画一次决策边界确认：

```
1 show_class_boundary(model, X_train, y_train, 'dev')
```

图 7.19 画出了关闭暂退后的决策边界。暂退会改变训练中的梯度和有效表示, 但是否适合某个网络仍取决于数据量、结构、暂退位置与概率, 不能仅按网络深度选择。

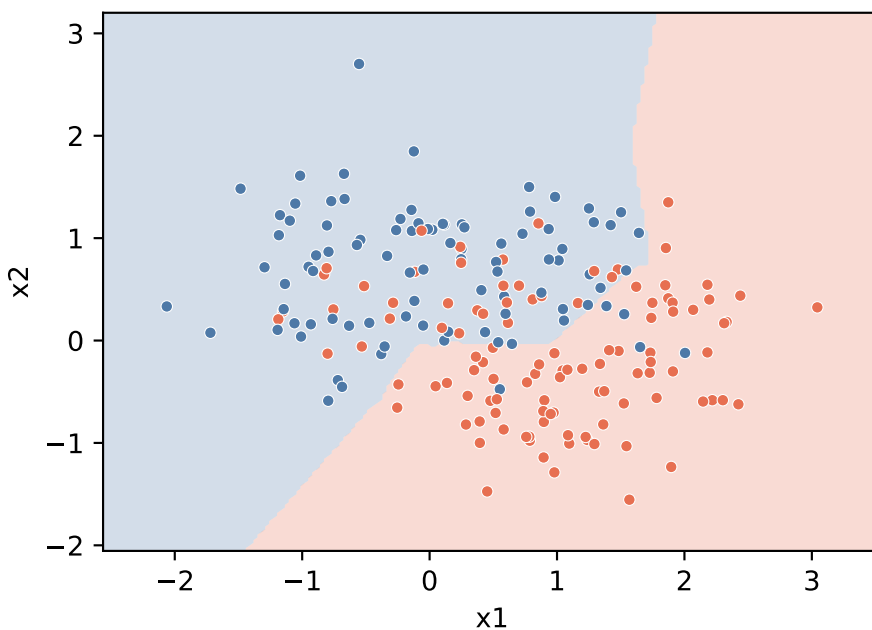


图 7.19 使用暂退法的分类决策边界可视化

动手练习 7.11

分别把正则化系数 λ 、权重衰减系数 β 和暂退率 p 调到很大 (如 $\lambda = 10$ 、 $\beta = 0.1$ 、 $p = 0.9$), 观察训练集准确率和验证集准确率会如何变化——同时记录训练损失, 区分正则化过强与优化尚未完成。

7.6.6 从验证集选择,再评价测试集

以上固定系数的对照只查看验证集. 如果还需选择正则化强度, 也应在验证集完成; 选定以后才评价留出的 100 条测试样本. 样本少时几个预测就能改变排序, 可以增加不同划分和随机种子的重复实验.

```
1 def evaluate_candidate(candidate, X, y):
2     args = ('dev',) if isinstance(candidate, MLP_3L_dropout) else ()
3     return accuracy_logits(candidate(X, *args), y).item()
4
5 chosen_name = max(regularization_results,
6                   key=lambda name: evaluate_candidate(regularization_results[name], X_dev,
7                                                         y_dev))
8 chosen_model = regularization_results[chosen_name]
9 print('chosen:', chosen_name)
10 print('test accuracy:', evaluate_candidate(chosen_model, X_test, y_test))
```

本次按验证准确率选中暂退法模型, 随后在 100 条测试样本上得到 83% 准确率. 这个数值只对应上述固定选择流程, 不再用于反向调整暂退率.

7.6.7 进一步实践: 训练预算与模型选择

第五章的 LeNet 需要增加训练回合才能学好训练样本, 第六章的影评模型则已经出现验证损失回升. 遇到类似现象, 应先区分优化不足和过拟合. 提前停止可以按预先选定的验证指标保存最佳快照, 并在若干次检查未改善后结束训练; 继续运行的最后一个状态不一定是最终要用的模型.

学习率调度也要明确计数单位. StepLR、指数衰减和余弦衰减可以按回合推进; OneCycleLR 通常按优化器更新推进, ReduceLROnPlateau 还需接收验证指标. 保存可恢复训练的检查点时, 应包含模型、优化器、调度器状态和当前进度. 配套 notebook 给出三种调度曲线, 读者可以接到同一训练实验中比较.

显存不足时可以跨多个小批次累积梯度. 若一个有效批次含 N 条样本, 各小批损失按样本数加权后再反向, 最后只更新一次参数, 才能对应整个有效批次的平均梯度. 各小批必须在相同参数处计算; 批量规范化的统计量仍来自各自的小批次, 因此含 BN 的网络通常不等同于一次真正的大批前向.

7.7 小结

同一条损失曲线背后, 可能是不同的更新次数、不同的梯度尺度, 也可能是统计方式不同. 本章先固定这些条件, 再观察优化器、初始化和规范化怎样改变学习过程.

优化部分从“批大小+学习率+梯度估计”三个方面切入：先用 2D Sphere 函数和 3D 鞍点函数把 SGD、AdaGrad、RMSprop、Momentum、Adam 等优化器画在同一张轨迹图上做直观对比，再补上 Xavier 初始化和 BN/LN 两种逐层规范化，区分参数更新、信号尺度和规范化轴的作用。

正则化部分围绕同一个易过拟合的 MLP 小实验，依次落地 ℓ_2 正则化、权重衰减和 Dropout，并通过决策边界图检查模型在固定验证集上的表现。权重衰减与 ℓ_2 惩罚的等价性还取决于更新规则和系数定义；调参完成后才读取测试集。

第8章 注意力机制

读一段影评时,“画面很美,但是情节拖沓”中的转折会改变我们对整句话的判断.第6章用双向LSTM编码上下文,再平均汇聚所有有效位置;现在可以追问:汇聚的权重能否也从分类任务中学出来?注意力机制(Attention Mechanism)提供了这种可微分的加权方式.先算相关性分数,再归一化并汇聚,就能把“该如何读这一组表示”纳入训练.

本章对应《神经网络与深度学习》第8章注意力机制.阅读前可先看一下图8.1中的关键知识点,方便概念与后续实现对照.

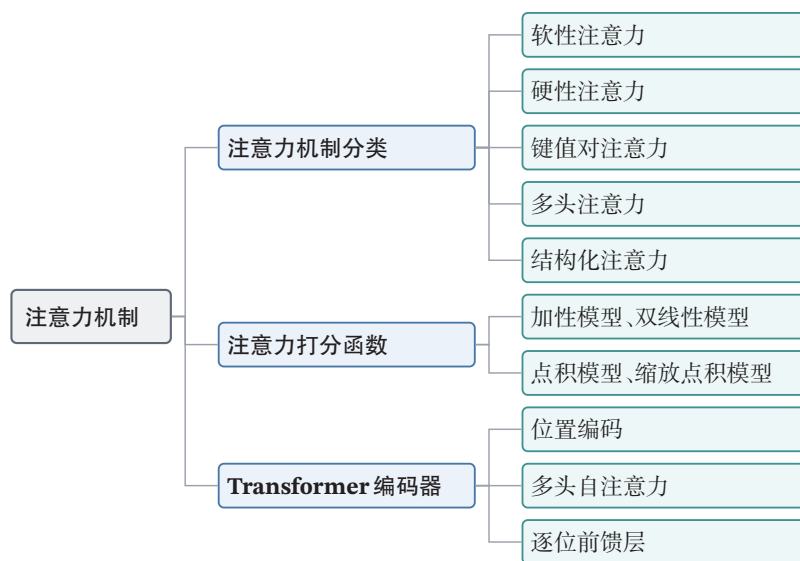


图 8.1 注意力机制关键知识点回顾

本章分为两部分:

- **模型解读**: 从最基础的注意力打分与汇聚开始,逐步实现键值对注意力、多头自注意力等变体,并用文本分类实验对比LSTM、LSTM+注意力、多头自注意力三种结构.
- **案例实践**: 实现基于Transformer编码器的文本语义匹配任务,完整体验Transformer的训练流程.

8.1 基于双向 LSTM 和注意力机制的文本分类

一次注意力计算可以拆成两步：先沿序列各位置算出一份注意力分布，再用它对各位置的表示做加权平均，得到**聚合**（Aggregation）表示。

给定 N 个输入向量 $[\mathbf{x}_1, \dots, \mathbf{x}_N]$ ，需要先说明此次汇聚服务于什么查询。**查询向量**（Query Vector）与各输入共同进入打分函数，产生一组待归一化的分数。查询可以是可学习的任务参数，也可以来自另一个网络或序列中的当前位置。

给定查询向量 $\mathbf{q}$ ，先用 Softmax 把打分归一化为**注意力分布**（Attention Distribution）：

$$\alpha_n = \frac{\exp(s(\mathbf{x}_n, \mathbf{q}))}{\sum_{j=1}^N \exp(s(\mathbf{x}_j, \mathbf{q}))}, \quad (8.1)$$

其中 $s(\mathbf{x}, \mathbf{q})$ 是**注意力打分函数**，常见有**加性模型**、**点积模型**、**缩放点积模型**和**双线性模型**等。注意力分布也称为**注意力权重**。

再用注意力分布对输入向量做加权平均，就得到序列的最终聚合表示：

$$\mathbf{z} = \sum_{n=1}^N \alpha_n \mathbf{x}_n. \quad (8.2)$$

第6.4节对双向 LSTM 的有效输出做平均汇聚。每个隐状态已编码周围语境，因此平均汇聚并不意味着各个原始词对预测的影响相等；注意力则进一步允许模型改变不同隐状态的汇聚系数。

本节保留原有编码器，用学到的权重替换固定均值，并通过同一份数据上的对照检验效果。权重多了一层灵活性，也带来了额外参数和优化问题，实验结果未必改善。

图8.2以 The movie is nice 为例。人可能先注意 nice；模型能否形成类似的分配，要由训练结果检验。图中的 sentiment 用于示意任务相关的查询，再由查询与各位置的表示计算权重。

笔记

图8.2里的 sentiment 只是一个用于说明“查询向量与任务相关”的示意例子；实际实现中，查询向量 $\mathbf{q}$ 通常直接作为一组随机初始化的可学习参数，在训练过程中自适应学习。

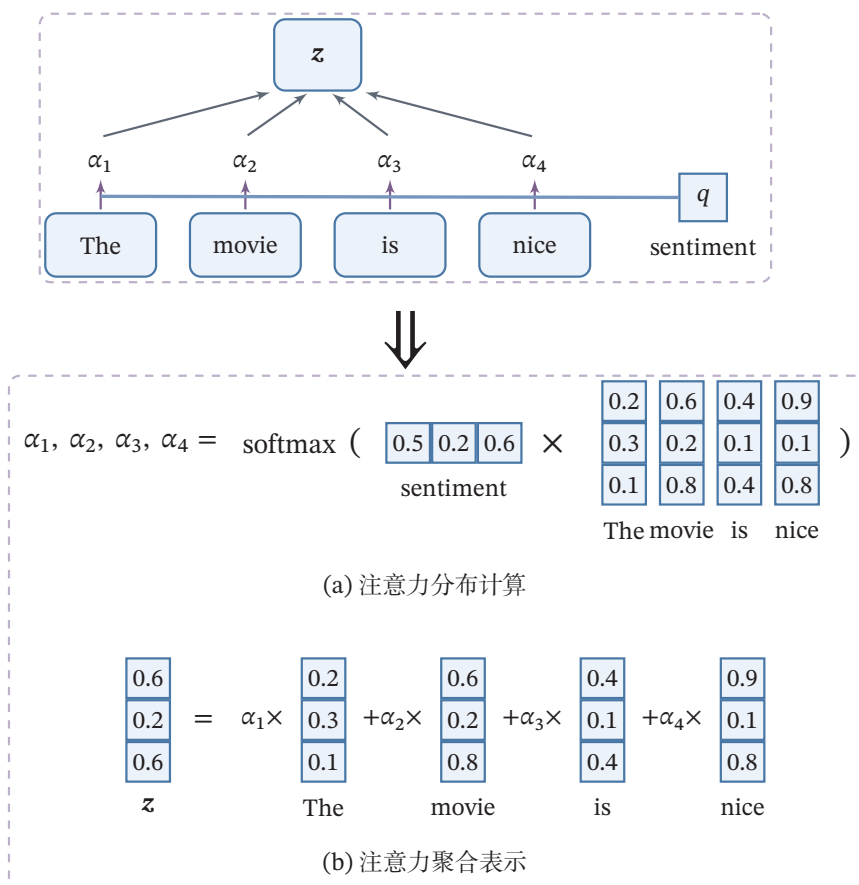


图 8.2 使用注意力机制来计算序列的聚合表示示例

8.1.1 数据介绍

继续使用第6.4.1节的IMDB电影评论数据集。默认采用5000/1000/1000条训练、验证、测试样本，嵌入与单向隐状态均为64维，最多读取128个词元。配套工具包复用第6章的数据处理和平均汇聚基线：

```

1 from pathlib import Path
2 from functools import partial
3 import torch
4 import torch.nn as nn
5 import torch.nn.functional as F
6 from torch.utils.data import DataLoader
7 from nndl import RunnerV3, accuracy
8 from nndl.text import (load_imdb_data, tokenize, balanced_subset, build_vocab,
9                        IMDBDataset, imdb_collate_fn, AveragePooling, Model_BiLSTM_FC)

```

```

10
11 # 在配套Notebook目录运行; 与第6章短程实验使用相同数据和词表.
12 data_dir = Path('../datasets/imdb')
13 train_data, dev_data, test_data = load_imdb_data(data_dir)
14 train_data = balanced_subset(train_data, 5000, seed=100)
15 dev_data = balanced_subset(dev_data, 1000, seed=101)
16 test_data = balanced_subset(test_data, 1000, seed=102)
17 word2id_dict = build_vocab(train_data, max_words=20000)
18 train_set, dev_set, test_set = [IMDBDataset(rows, word2id_dict)
19                                for rows in (train_data, dev_data, test_data)]
20 max_seq_len, batch_size, epochs = 128, 64, 5
21 imdb_collate = partial(imdb_collate_fn, max_seq_len=max_seq_len)
22 def make_train_loader():
23     return DataLoader(train_set, batch_size=64, shuffle=True,
24                       generator=torch.Generator().manual_seed(100), collate_fn=imdb_collate)
25 dev_loader = DataLoader(dev_set, batch_size=batch_size, collate_fn=imdb_collate)
26 test_loader = DataLoader(test_set, batch_size=batch_size, collate_fn=imdb_collate)
27 vocab_size, hidden_size, embedding_size, n_classes = len(word2id_dict), 64, 64, 2
28 torch.manual_seed(100)
29 initial_baseline = Model_BiLSTM_FC(vocab_size, embedding_size, hidden_size)

```

8.1.2 模型构建

整体模型结构见图8.3,按数据流向可拆为四块:

- 1) 嵌入层:把输入词序列转成向量序列.
- 2) LSTM层:双向 LSTM 编码上下文表示.
- 3) 注意力层:根据打分函数对 LSTM 的输出做加权汇聚.
- 4) 线性层:把聚合后的向量映射为类别得分.

嵌入层和双向 LSTM 直接复用第6.4节的实现,本节重点介绍注意力层.

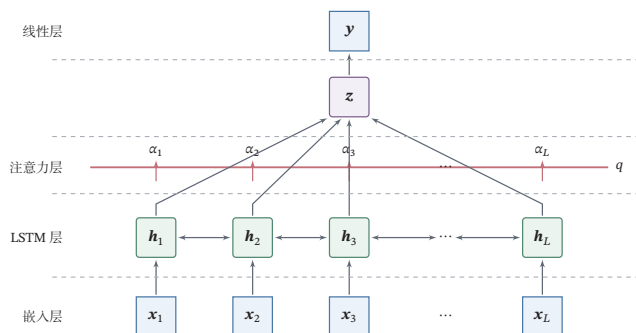


图 8.3 基于双向 LSTM 和注意力机制的文本分类模型

设输入序列长度为 L 。经过嵌入层和双向 LSTM 之后得到表示矩阵 $\mathbf{X} \in \mathbb{R}^{L \times D}$ (D 为每个位置的特征维度)。本节用注意力机制从 $\mathbf{X}$ 里挑出与任务相关的信息, 得到一个更有针对性的聚合表示。

下面按打分函数、分布计算、加权平均三步分别实现。

8.1.2.1 注意力打分函数

先实现公式(8.1)中的打分函数 $s(\mathbf{x}, \mathbf{q})$ 。本节实现加性模型与点积模型两种典型形式。

加性模型 记输入序列为 $\mathbf{X} \in \mathbb{R}^{B \times L \times D}$ (B 批量、 L 序列长度、 D 特征维度)。引入一个可学习的查询向量 $\mathbf{q} \in \mathbb{R}^D$ 。加性打分函数为

$$s(\mathbf{x}_n, \mathbf{q}) = \mathbf{v}^\top \tanh(\mathbf{W}\mathbf{x}_n + \mathbf{U}\mathbf{q}), \quad (8.3)$$

其中 $\mathbf{x}_n, \mathbf{q}, \mathbf{v} \in \mathbb{R}^D$ 按列向量记, $\mathbf{W}, \mathbf{U} \in \mathbb{R}^{D \times D}$ 为可学习矩阵。批处理代码把各位置放在张量的最后一维, 由 `nn.Linear` 完成同一个变换。

提醒

`nn.Linear` 的权重按“输出维度、输入维度”存储; 对行排列的输入, 下面实际计算 $\mathbf{Y} = \mathbf{X}\mathbf{W}_{\text{code}}^\top$ 。与数学公式对齐时应检查转置:

```
1 D = 3
2 X = torch.rand(1, 3, D)
3 W = torch.nn.Linear(D, D, bias=False)
4 Y = W(X) # 实际计算 X @ W.weight.T
```

把加性模型封装为加性注意力打分分子:

```
1 class AdditiveScore(nn.Module):
2     # hidden_size对应上面公式中的特征维度D
3     def __init__(self, hidden_size):
4         super().__init__()
5         self.W = nn.Linear(hidden_size, hidden_size, bias=False)
6         self.U = nn.Linear(hidden_size, hidden_size, bias=False)
7         self.v = nn.Linear(hidden_size, 1, bias=False)
8         # 查询向量使用均匀分布随机初始化
9         self.q = nn.Parameter(torch.empty(1, hidden_size))
10        nn.init.uniform_(self.q, -0.5, 0.5)
11
12    def forward(self, inputs):
13        """
14        输入:
15        - inputs: 输入张量, 形状为[批量大小, 序列长度, 隐状态维度]
16        输出:
17        - scores: 注意力分数矩阵, 形状为[批量大小, 序列长度]
```

```

18         """
19         # 输入张量形状: [批量大小, 序列长度, 隐状态维度]
20         batch_size, seq_len, hidden_size = inputs.shape
21         # 非线性变换后的张量形状: [批量大小, 序列长度, 隐状态维度]
22         scores = torch.tanh(self.W(inputs)+self.U(self.q))
23         # 注意力分数的形状: [批量大小, 序列长度]
24         scores = self.v(scores).squeeze(-1)
25         return scores
26     #测试加性注意力打分算子的打分输出, 输入是随机初始化的张量.
27     torch.manual_seed(2021)
28     inputs = torch.rand(size=[1, 3, 3])
29     additiveScore = AdditiveScore(hidden_size=3)
30     scores = additiveScore(inputs)
31     print(scores)

```

输出结果为:

```
tensor([[ 0.0263, -0.0281, -0.0866]], grad_fn=<SqueezeBackward1>)
```

思考



上面的加性打分函数直接照搬《神经网络与深度学习》主书公式(8.2), 将 $\mathbf{q}$ 和 $\mathbf{U}$ 分开维护. 但既然 $\mathbf{q}$ 本身就是可学习参数, $\mathbf{q}^T \mathbf{U}$ 完全可以合并为单一可学习向量. 请对比两种实现的参数量、训练稳定性和表达力差异.

点积模型 接下来实现点积版打分函数. 对于输入张量 $\mathbf{X} \in \mathbb{R}^{B \times L \times D}$ (B 批量、 L 序列长度、 D 特征维度) 和一个可学习查询向量 $\mathbf{q} \in \mathbb{R}^D$, 点积打分写作

$$s(\mathbf{X}, \mathbf{q}) = \mathbf{X}\mathbf{q}. \quad (8.4)$$

忽略批量维, 点积打分的主要运算为 $O(LD)$, 此处加性打分为 $O(LD^2)$. 这说明点积需要的算术运算较少, 实际耗时仍取决于张量规模、硬件和算子实现.

把它封装为**点积注意力打分算子**:

```

1 class DotProductScore(nn.Module):
2     def __init__(self, hidden_size):
3         super().__init__()
4         # 使用均匀分布随机初始化一个查询向量
5         self.q = nn.Parameter(torch.empty(hidden_size, 1))
6         nn.init.uniform_(self.q, -0.5, 0.5)
7
8     def forward(self, inputs):
9         """
10        输入:

```

```

11         - inputs: 输入张量, 形状为[批量大小, 序列长度, 隐状态维度]
12     输出:
13         - scores: 注意力分数矩阵, 形状为[批量大小, 序列长度]
14     """
15     # 输入张量形状: [批量大小, 序列长度, 隐状态维度]
16     batch_size, seq_length, hidden_size = inputs.shape
17     # 点积结果的形状: [批量大小, 序列长度, 1]
18     scores = torch.matmul(inputs, self.q)
19     # 移除末尾的单元素维度后, 分数形状为 [批量大小, 序列长度]
20     scores = scores.squeeze(-1)
21     return scores
22 # 测试点积注意力打分算子的打分输出, 输入是随机初始化的张量.
23 torch.manual_seed(2021)
24 inputs = torch.rand(size=[1, 3, 3])
25 dotScore = DotProductScore(hidden_size=3)
26 scores = dotScore(inputs)
27 print(scores)

```

输出结果为:

```
tensor([[ -0.2521, -0.1902,  0.1381]], grad_fn=<SqueezeBackward1>)
```

8.1.2.2 注意力分布计算

打分函数有了, 下一步是用 Softmax 把分数归一化为分布. 工程上还有一个绕不开的细节——文本批训练时不同样本长度不一, 需要用 [PAD] 补齐. 例如第 6.4.1.3 节的例子:

```

句子1: This movie was craptacular [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD]
句子2: I got stuck in traffic on the way to the theater

```

填充位置不应得到汇聚权重. 先把其分数设为 $-\infty$, 再做 Softmax; 只要每行至少有一个有效位置, 填充项的权重就是 0. 下例用有限负数演示同一效果, 实际代码直接使用负无穷, 避免 $-1e9$ 在半精度类型中溢出.

```

句子1: 0.1 0.2 0.4 0.6 -1e9 -1e9 -1e9 -1e9 -1e9 -1e9 -1e9
句子2: 0.3 0.2 0.4 0.1 0.2 0.5 0.4 0.6 0.7 0.9 1.0

```

再过一次 Softmax 得到注意力权重, 输出形如:

```

句子1: 0.19593432 0.21654092 0.26448367 0.32304109 0. 0. 0. 0. 0. 0. 0.
句子2: 0.07279236 0.06586525 0.080448 0.05959734 0.06586525 0.08890879 0.080448 0.09825941
        0.10859344 0.13263633 0.14658581

```

有效位置上的权重重新归一化为和为 1 的分布. 若整行都是填充, Softmax 没有可归一化的有效分数; 本章接口拒绝这种输入, 空文本在预处理时放入一个 [UNK].

整体写成矩阵形式: 记 $\mathbf{S} \in \mathbb{R}^{B \times L}$ 为一批样本的打分矩阵 (每行一个样本, L 是补齐后长度), 则带掩码的注意力分布为

$$\mathbf{A} = \text{softmax}(\mathbf{S} + \mathbf{M}) \in \mathbb{R}^{B \times L}, \quad (8.5)$$

$\text{softmax}(\cdot)$ 按行归一化; $\mathbf{M}$ 在填充位置取 $-\infty$ 、有效位置取 0. 代码以布尔掩码按位置填值.

先检查长度并生成布尔掩码, 再沿最后一维归一化. 这里的 True 表示保留该位置:

```

1 def sequence_mask(valid_lens, length, device):
2     if valid_lens.ndim != 1 or valid_lens.is_floating_point():
3         raise ValueError('valid_lens must be a one-dimensional integer tensor')
4     valid_lens = valid_lens.to(device)
5     if torch.any((valid_lens < 1) | (valid_lens > length)):
6         raise ValueError('each sequence must have 1..length valid tokens')
7     return torch.arange(length, device=device)[None, :] < valid_lens[:, None]
8
9 valid_lens = torch.tensor([2])
10 mask = sequence_mask(valid_lens, scores.shape[1], scores.device)
11 attn_weights = F.softmax(scores.masked_fill(~mask, float('-inf')), dim=-1)

```

8.1.2.3 加权平均

得到注意力分布后, 第三步是用分布作为权重对输入向量做加权平均, 得到聚合表示. 这一步本质上就是一次按序列维度的批矩阵乘:

```

1 X = inputs
2 context = torch.matmul(attn_weights.unsqueeze(1), X).squeeze(1)

```

8.1.2.4 注意力层汇总

最后把“打分、掩码、Softmax、加权”四步串成完整的注意力层. 通过 `use_additive` 开关在加性和点积模型之间切换, 便于后续做对照实验:

```

1 class Attention(nn.Module):
2     def __init__(self, hidden_size, use_additive=False):
3         super().__init__()
4         self.scores = AdditiveScore(hidden_size) if use_additive else DotProductScore(
5             hidden_size)
6         self._attention_weights = None
7
8     def forward(self, X, valid_lens):
9         scores = self.scores(X)
10        if len(valid_lens) != X.shape[0]:
11            raise ValueError('one valid length is required per sequence')
12        mask = sequence_mask(valid_lens, X.shape[1], X.device)

```

```

12         weights = F.softmax(scores.masked_fill(~mask, float('-inf')), dim=-1)
13         self._attention_weights = weights.detach()
14         return torch.bmm(weights.unsqueeze(1), X).squeeze(1)
15
16     @property
17     def attention_weights(self):
18         return self._attention_weights

```

8.1.2.5 模型汇总

注意力层就绪后, 把它接到整段网络里. 整体结构是“嵌入、双向 LSTM、注意力、线性分类”: 嵌入层把词索引转向量; 双向 LSTM 编码上下文得到每一步的隐状态; 注意力层从隐状态序列中筛选出聚合表示; 线性层映射到类别得分:

```

1  from torch.nn.utils.rnn import pack_padded_sequence, pad_packed_sequence
2
3  class Model_LSTMAttention(nn.Module):
4      def __init__(self, hidden_size, embedding_size, vocab_size, n_classes=2,
5                  n_layers=1, use_additive=False):
6          super().__init__()
7          # LSTM单元的隐藏神经元数量, 即隐状态向量和记忆单元向量的维度
8          self.hidden_size = hidden_size
9          # 表示词向量的维度
10         self.embedding_size = embedding_size
11         # 词表大小, 即包含词的数量
12         self.vocab_size = vocab_size
13         # 表示文本分类的类别数量
14         self.n_classes = n_classes
15         # 表示LSTM的层数
16         self.n_layers = n_layers
17         # 定义嵌入层
18         self.embedding = nn.Embedding(self.vocab_size, self.embedding_size, padding_idx=0)
19         # 定义LSTM, batch_first=True表示输入形状为[批量大小, 序列长度, 嵌入维度]
20         self.lstm = nn.LSTM(input_size=self.embedding_size, hidden_size=self.hidden_size,
21                             num_layers=self.n_layers, batch_first=True, bidirectional=True)
22         # 定义注意力层
23         self.attention = Attention(hidden_size * 2, use_additive=use_additive)
24         # 定义线性层, 用于将语义向量映射到相应的类别
25         self.cls_fc = nn.Linear(self.hidden_size * 2, self.n_classes)
26
27     def forward(self, input_ids, valid_lens):
28         # 获取词向量
29         embedded_input = self.embedding(input_ids)
30         # 用 pack_padded_sequence 处理变长序列, 让 LSTM 跳过 [PAD]
31         packed = pack_padded_sequence(embedded_input, valid_lens.cpu(),

```

```

32             batch_first=True, enforce_sorted=False)
33     packed_output, _ = self.lstm(packed)
34     last_layers_hiddens, _ = pad_packed_sequence(packed_output, batch_first=True)
35     # 使用注意力机制
36     last_layers_hiddens = self.attention(last_layers_hiddens, valid_lens)
37     # 通过线性层, 获得初步的类别数值
38     logits = self.cls_fc(last_layers_hiddens)
39     return logits

```

8.1.3 使用加性注意力模型进行实验

加性注意力的接法很简单: 把Model_LSTMAttention里的注意力层切换成加性版本, 其余完全不变, 让双向 LSTM 的每步隐状态送入加性打分函数后做加权平均, 再交给线性分类层。

8.1.3.1 模型训练

使用RunnerV3训练 5 个回合, Adam 学习率为 0.001, 并按验证准确率保存快照. 三种注意力模型均复制第 6 章基线的嵌入、LSTM 和分类头初值; 每次重建训练加载器, 固定批次顺序. 新增的注意力参数各自初始化, 所需运算量并不相同。

```

1  def align_initial_parameters(model):
2      model.embedding.load_state_dict(initial_baseline.embedding_layer.state_dict())
3      model.lstm.load_state_dict(initial_baseline.lstm_layer.state_dict())
4      model.cls_fc.load_state_dict(initial_baseline.output_layer.state_dict())
5
6  def train_attention_model(model, name):
7      align_initial_parameters(model)
8      runner = RunnerV3(model, torch.optim.Adam(model.parameters(), lr=0.001),
9                        nn.CrossEntropyLoss(), metric_fn=accuracy)
10     runner.fit(make_train_loader(), dev_loader, num_epochs=epochs, log_every=1,
11               best_path=f'./checkpoints/{name}.pt', grad_clip_norm=1.0)
12     return runner
13
14 torch.manual_seed(100)
15 model_atten = Model_LSTMAttention(hidden_size, embedding_size, vocab_size,
16                                   n_classes=n_classes, use_additive=True)
17 runner = train_attention_model(model_atten, 'imdb_additive')
18 attention_runners = {'additive': runner}

```

训练集与验证集的损失/准确率曲线见图8.4.

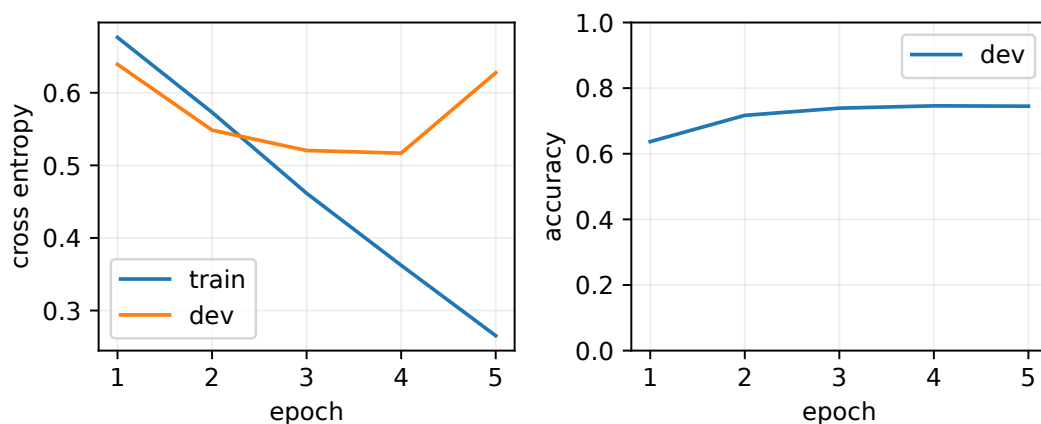


图 8.4 加性注意力模型的损失和准确率变化趋势

8.1.3.2 模型评价

加载最优快照,在测试集上做最终评估:

```
1 runner.load('./checkpoints/imdb_additive.pt')
2 _, test_accuracy = runner.evaluate(test_loader)
3 print(f"Evaluate on test set, Accuracy: {test_accuracy:.5f}")
```

输出结果为:

```
Evaluate on test set, Accuracy: 0.73500
```

8.1.4 使用点积注意力模型进行实验

切换为点积注意力的步骤几乎一样: 把Model_LSTMAttention的use_additive置为False, 其他配置完全保持不变.

8.1.4.1 模型训练

点积模型采用相同的样本、初始公共参数、批次顺序和训练回合数. 检查点使用独立文件名, 以免后面的实验覆盖前一个模型:

```
1 torch.manual_seed(100)
2 model_attn = Model_LSTMAttention(hidden_size, embedding_size, vocab_size,
3                                 n_classes=n_classes, use_additive=False)
4 runner = train_attention_model(model_attn, 'imdb_dot')
5 attention_runners['dot'] = runner
```

训练与验证曲线见图8.5. 需要同时看下降趋势和两者之间的差距, 不能只比较末回合训练损失.

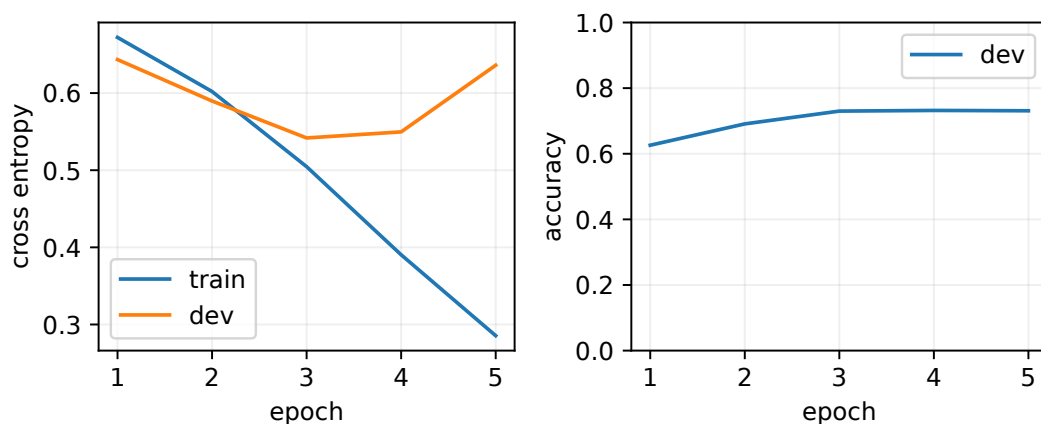


图 8.5 点积注意力模型的损失和准确率变化趋势

8.1.4.2 模型评价

载入最优快照,在测试集上做最终评估:

```
1 runner.load('./checkpoints/imdb_dot.pt')
2 _, test_accuracy = runner.evaluate(test_loader)
3 print(f"Evaluate on test set, Accuracy: {test_accuracy:.5f}")
```

输出结果为:

```
Evaluate on test set, Accuracy: 0.71200
```

第 6 章平均汇聚基线的测试准确率为 72.8%。本次加性注意力为 73.5%,点积注意力为 71.2%。增加注意力并未一致改善结果:它改变了优化路径和汇聚方式,不能凭一种写法就预先断定更好。以上仅为同一划分、单个随机种子的比较,差异的稳定性还需重复实验。

8.1.4.3 注意力可视化

除了查看预测是否正确,还可以检查汇聚时使用的权重。下面固定一条输入,加载点积模型的最佳快照,沿用训练时的小写与空白处理。所得权重描述这次计算中的分配,不能直接当作原始词对预测的因果贡献。

```
1 text = "this great science fiction film is really awesome"
2 # 分词
3 sentence = tokenize(text)
4 # 词映射成ID的形式
5 tokens = [
6     word2id_dict[word] if word in word2id_dict else word2id_dict["[UNK]"]
7     for word in sentence ]
8 # 取前max_seq_len的词
```



```

9 tokens = tokens[:max_seq_len] or [word2id_dict["[UNK]"]]
10 # 序列长度
11 seq_len = torch.tensor([len(tokens)])
12 # 转换成PyTorch的张量
13 input_ids = torch.tensor(tokens, dtype=torch.int64).unsqueeze(0)
14 # 模型开启评价模式
15 model_atten.eval()
16 # 设置不求梯度
17 with torch.no_grad():
18     # 预测输出
19     logits = model_atten(input_ids, seq_len)
20 # 提取注意力权重
21 atten_weights = model_atten.attention.attention_weights
22 print(f"输入的文本为: {text}")
23 print(f"转换成id的形式为: {input_ids.numpy()}")
24 print(f"注意力权重为: {atten_weights.numpy()}")
25 print("预测标签: ", logits.argmax(-1).item())

```

输出结果为:

```

输入的文本为: this great science fiction film is really awesome
转换成id的形式为: [[ 10  90 1297 2432 23   7  61 1489]]
注意力权重为: [[0.08253491 0.10558449 0.16977902 0.13937853 0.06430486 0.10768119
 0.14917436 0.18156256]]
预测标签:  1

```

每个权重对应一个上下文隐状态,而非孤立的词义。该隐状态还混合了周围词的信息;判断某个词的作用,需要结合替换、删除等干预,并注意这些改动也会改变其他位置的编码。

图8.6根据同一快照的实际权重绘制。本次 awesome 和 science 的权重较高,分别约为 0.182 和 0.170;不能据此把这些词的字面意义直接当作分类理由。

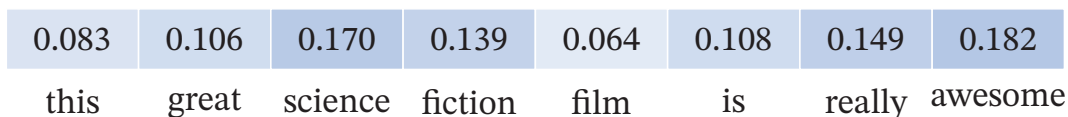


图 8.6 注意力权重的可视化示例



动手练习 8.1

仿照加性/点积模型的实现,再写出《神经网络与深度学习》第 8.2 节中的双线性、缩放点积等其他打分函数,并在同一份双向 LSTM 框架下做横向对比。

8.2 基于双向 LSTM 和多头自注意力的文本分类实验

上一节所有影评共享一个可学习的查询参数. 本节让每个位置从自身的上下文表示生成查询, 分别读取其他位置, 再将结果用于分类. 这就是**自注意力**: 变化发生在查询的来源和输出的组织方式.

8.2.1 自注意力模型

单层窄卷积只连接局部位置, 堆叠或空洞卷积可以扩大感受野; RNN 通过递推传递信息, 连接远处位置需要经过多个时间步. 全自注意力则在一层中直接计算任意两位置的关系, 缩短信息传播路径, 同时付出随序列长度平方增长的成对计算开销.

自注意力 (Self-Attention) 从设计上就直面长程依赖: 把序列中每个元素都视为全连接图上的一个节点, 节点间的连接权重不再固定, 而是由注意力机制根据当前内容动态计算得到.

下面按由浅入深的顺序依次介绍简单自注意力、QKV 自注意力与多头自注意力三种形式.

8.2.1.1 最简单的自注意力

设输入序列 $\mathbf{X} \in \mathbb{R}^{L \times D}$. 最朴素的自注意力把每个元素 $\mathbf{x}_m$ 都当作查询向量, 借助注意力机制从全序列中挑出与自己相关的信息, 得到当前元素的上下文表示 $\mathbf{h}_m \in \mathbb{R}^{1 \times D}$:

$$\mathbf{h}_m = \sum_{n=1}^L \alpha_{mn} \mathbf{x}_n \quad (8.6)$$

$$= \text{softmax}(\mathbf{x}_m \mathbf{X}^T) \mathbf{X}, \quad (8.7)$$

其中 α_{mn} 是第 m 个元素对第 n 个元素的注意力权重, 打分函数取点积.

仍以句子 `The movie is nice` 为例: 要计算 `movie` 的上下文表示, 就用 `movie` 本身作为查询向量, 对句中所有词算注意力分布, 再加权平均得到融合了全序列信息的 `movie` 表示. 与 CNN/RNN 相比, 自注意力可以一步“跨越”整个序列, 融合更远位置的上下文. 整个过程见图 8.7.

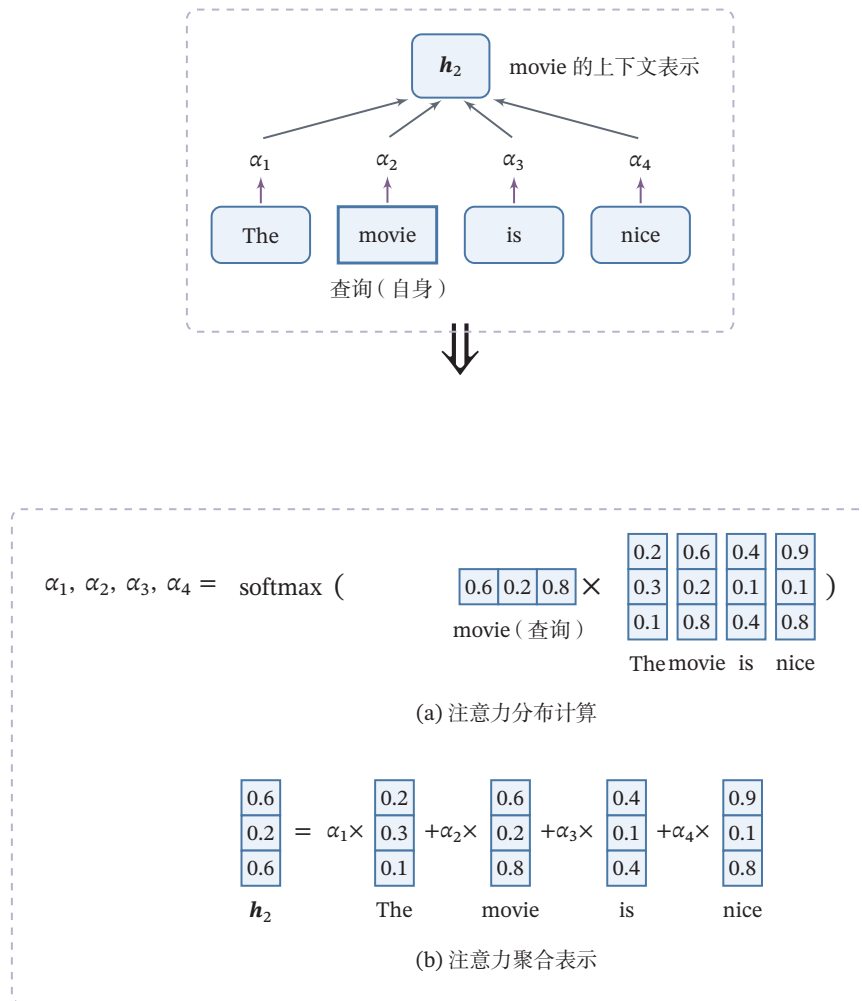


图 8.7 自注意力示例

对于输入序列 $\mathbf{X} \in \mathbb{R}^{L \times D}$, 自注意力可以表示为

$$\mathbf{Z} = \text{softmax}(\mathbf{X}\mathbf{X}^\top)\mathbf{X}, \quad (8.8)$$

其中 $\text{softmax}(\cdot)$ 按行归一化, $\mathbf{Z}$ 是各位置上下文表示的拼接. 整个流程见图 8.8.

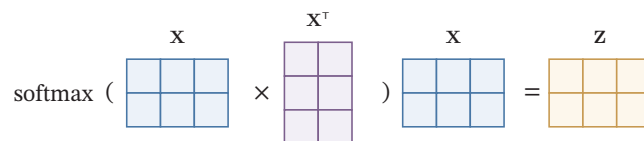


图 8.8 整个序列的自注意力示例



动手练习 8.2

把上面的简单自注意力实现出来, 插到第 8.1 节模型的双向 LSTM 和注意力层之间, 比较 IMDB 分类精度是否能进一步提升。

8.2.1.2 QKV 自注意力

前面的简单自注意力直接以输入元素自身作为查询, 全程没有引入任何可学习参数, 表达力受限。

为了赋予模型更强的容量, 工程上普遍采用**查询-键-值** (Query-Key-Value, QKV) 范式: 把输入序列分别通过三个独立的线性映射, 得到查询 Q 、键 K 、值 V 三组表示——查询负责发起匹配、键负责被匹配、值负责被加权。最终用 Q 与 K 算分布, 再以该分布加权 V 得到输出。

对 $X \in \mathbb{R}^{B \times L \times D}$, 三种映射可以写成:

$$Q = XW^Q \in \mathbb{R}^{B \times L \times D}, \quad (8.9)$$

$$K = XW^K \in \mathbb{R}^{B \times L \times D}, \quad (8.10)$$

$$V = XW^V \in \mathbb{R}^{B \times L \times D}, \quad (8.11)$$

其中 $W^Q, W^K, W^V \in \mathbb{R}^{D \times D}$ 均为可学习矩阵。为简化推导, 这里令 Q, K, V 的特征维度保持为 D 。

QKV 自注意力的统一表达式为

$$Z = \text{attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{D}}\right)V, \quad (8.12)$$

其中 $\text{softmax}(\cdot)$ 按行归一化, 整体计算流程见图 8.9。

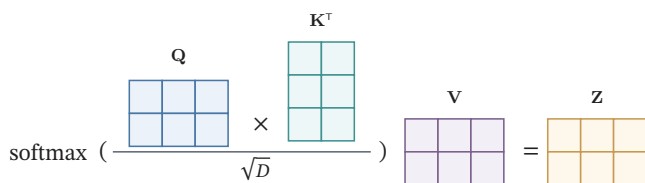


图 8.9 整个序列的 QKV 自注意力示例

若查询、键各分量相互独立, 均值为 0、方差为 1, 则内积方差为 D , 除以 $\sqrt{D}$ 可把该量级归一化为 1。这解释了缩放的动机; 训练后的分量未必满足假设, 因此缩放并不保证 Softmax 永不集中或梯度始终足够大。

掩蔽序列中的 [PAD] 变长序列在打包成等长张量时通常会在尾部填充 [PAD], 注意力计算时必须把这些位置屏蔽掉, 避免它们污染注意力分布。

图 8.10 按 False 位置将打分替换为 $-\infty$, 然后做 Softmax。这里是按掩码填值, 不是将打分与 0/1 逐元素相乘; 后者会把无效分数变成 0, 仍可能分到非零权重。

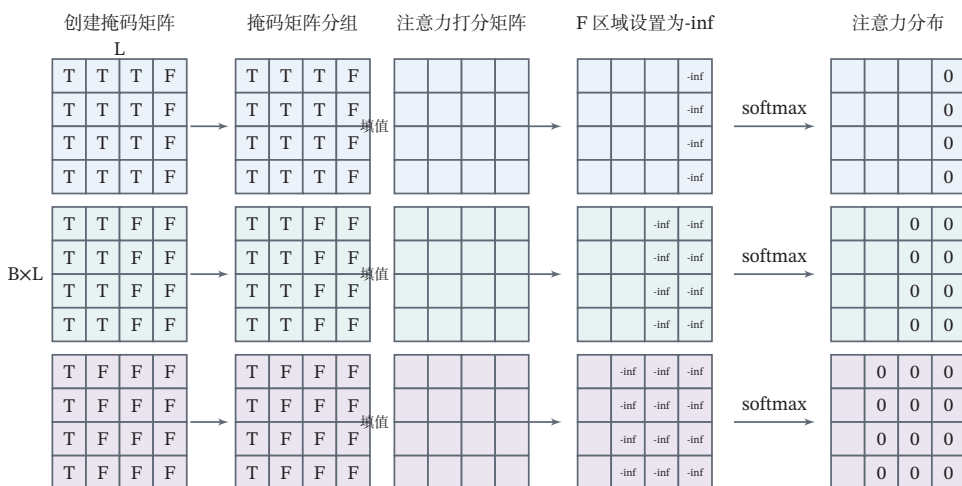


图 8.10 QKV 自注意力中掩蔽 [PAD] 位置的实现过程

具体落地分三步。

(1) 按真实长度构造 $B \times 1 \times L$ 的布尔掩码, 沿查询维广播到 $B \times L \times L$ 。无效的是键的位置:

```
1 torch.manual_seed(7)
2 Q = K = V = torch.randn(2, 3, 4)
3 score = Q @ K.transpose(-2, -1) / Q.shape[-1] ** 0.5
4 valid_lens = torch.tensor([2, 3])
5 mask = sequence_mask(valid_lens, K.shape[1], K.device)[: :, None, :]
```

(2) 将无效键的分数填为负无穷:

```
1 score = score.masked_fill(~mask, float('-inf'))
```

(3) 对打分做 Softmax 得到注意力分布, 被屏蔽位置的权重自然为 0:

```
1 # attention_weights: [batch_size, seq_len, seq_len]
2 attention_weights = F.softmax(score, -1)
```

把上述步骤整合为 QKVAttention 算子:

```
1 class QKVAttention(nn.Module):
2     def __init__(self, size, dropout=0.0):
3         super().__init__()
4         if size < 1:
5             raise ValueError('key dimension must be positive')
6         self.sqrt_size = size ** 0.5
7         self.dropout = nn.Dropout(dropout)
8         self.attention_weights = None
9
```

```
10     def forward(self, Q, K, V, valid_lens):
11         # Q: [B, Lq, Dk], K: [B, Lk, Dk], V: [B, Lk, Dv], 长度为 [B].
12         score = Q @ K.transpose(-2, -1) / self.sqrt_size
13         if len(valid_lens) != Q.shape[0] or K.shape[:2] != V.shape[:2]:
14             raise ValueError('incompatible batch, key/value or length shapes')
15         mask = sequence_mask(valid_lens, K.shape[1], K.device)[: , None, :]
16         weights = F.softmax(score.masked_fill(~mask, float('-inf')), dim=-1)
17         self._attention_weights = weights.detach()
18         return self.dropout(weights) @ V
19
20     @property
21     def attention_weights(self):
22         return self._attention_weights
```

8.2.1.3 多头自注意力

QKV 自注意力虽然引入了可学习的投影矩阵,但单组投影只能从一个角度衡量元素之间的关系。借鉴卷积“多通道”的思路,可以并行使用多组独立的 QKV 投影,每组叫一个**头** (head),再把它们的输出拼起来——这就是**多头自注意力** (Multi-Head Self-Attention, MHSA)。

每个头独立完成一次特征汇聚,相当于从不同子空间观察输入;拼接后再过一层线性投影做信息融合,得到最终输出。整体结构见图8.11,可以拆成三段:线性变换、单头 QKV 自注意力、多头融合。

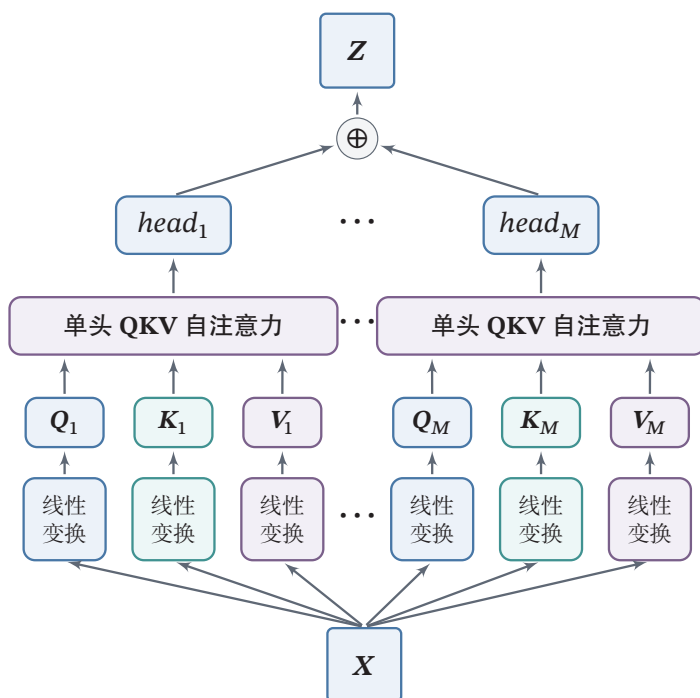


图 8.11 多头自注意力结构

设输入序列为 $\mathbf{X} \in \mathbb{R}^{B \times L \times D}$ (B 批量、 L 序列长度、 D 特征维度). MHSA 通过下面三步定义.

(1) **线性变换**: 每个头独立维护三组投影矩阵, 将 $\mathbf{X}$ 分别映射成自己的 Q、K、V:

$$\mathbf{Q}_m = \mathbf{X} \mathbf{W}_m^Q, \quad (8.13)$$

$$\mathbf{K}_m = \mathbf{X} \mathbf{W}_m^K, \quad (8.14)$$

$$\mathbf{V}_m = \mathbf{X} \mathbf{W}_m^V, \quad (8.15)$$

其中 $\mathbf{W}_m^Q, \mathbf{W}_m^K, \mathbf{W}_m^V \in \mathbb{R}^{D \times D_m}$ 是该头独立的可学习投影矩阵, D_m 为单头特征维度.

(2) **单头 QKV 自注意力**: 第 m 个头独立完成一次 QKV 注意力:

$$\text{head}_m = \text{attention}(\mathbf{Q}_m, \mathbf{K}_m, \mathbf{V}_m) \in \mathbb{R}^{B \times L \times D_m}, \quad (8.16)$$

其中 $\text{attention}(\cdot)$ 沿用公式(8.12)的定义.

(3) **多头融合**: 将所有头按特征维拼接, 再通过一次线性投影:

$$\mathbf{Z} = \text{MultiHeadSelfAttention}(\mathbf{X}) \triangleq \bigoplus (\text{head}_1, \text{head}_2, \dots, \text{head}_M) \mathbf{W}', \quad (8.17)$$

其中 $\bigoplus$ 表示沿最后一维拼接, $\mathbf{W}' \in \mathbb{R}^{(MD_m) \times D'}$ 是输出投影矩阵, D' 是输出维度.

工程上为简洁起见, 常令 $D_m = D/M$ (即把原特征维平均切 M 份)、 $D' = D$ (输出维与输入维相同), 这样多头机制不会改变张量形状.

动手实现 代码层面, 多头计算需要把 M 个头一起跑——直接循环 M 次效率太低. 常见做法是先做一次完整的 Q/K/V 投影得到 $[B, L, D]$, 再把特征维“切片”成 M 头并把头维与批维合并, 让 M 个头变成更大的 batch 一起做矩阵乘法. 实现原理见图 8.12.

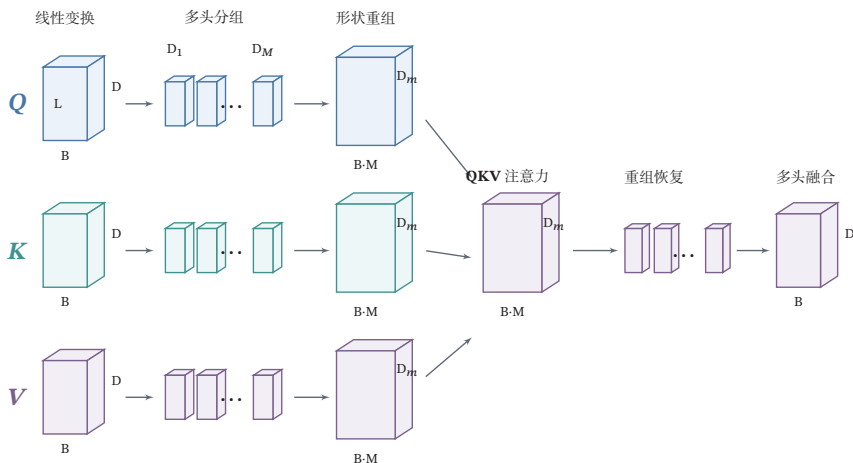


图 8.12 多头自注意力实现原理

完整流程分为六步.

(1) **线性变换**: 一次投影同时算出各个头的结果, 暂存在 $[B, L, D]$ 张量中. 不同头对应权重矩阵的不同列块; 合并计算不表示它们共享同一组投影参数:

```
1 inputs_size, heads_num, head_size = 4, 2, 2
2 X = torch.randn(2, 3, inputs_size)
3 valid_lens = torch.tensor([2, 3])
4 Q_proj = nn.Linear(inputs_size, inputs_size, bias=False)
5 K_proj = nn.Linear(inputs_size, inputs_size, bias=False)
6 V_proj = nn.Linear(inputs_size, inputs_size, bias=False)
7 batch_size, seq_len, hidden_size = X.shape
8 Q, K, V = Q_proj(X), K_proj(X), V_proj(X)
```

(2) **多头分组**: 把特征维度 D 切成 M 份, 每份对应一个头:

$$(B \times L \times D) \xrightarrow{\text{reshape}} (B \times L \times M \times D_m), \quad (8.18)$$

其中 M 是头数, D_m 是单头维度, $D = M \cdot D_m$.

(3) **形状重组**: 不同头之间相互独立, 因此把头维和批维合并, 相当于让 BM 个“虚拟样本”同时跑单头 QKV 注意力:

$$(B \times L \times M \times D_m) \xrightarrow{\text{transpose}} (B \times M \times L \times D_m) \xrightarrow{\text{reshape}} (BM \times L \times D_m), \quad (8.19)$$

对 Q, K, V 都做同样的 reshape+transpose, 得到 $Q, K, V \in \mathbb{R}^{BM \times L \times D_m}$.

经过这两步, 多头自注意力就被改写为 BM 个样本的单头注意力, 可以一次性走完. 把第(2)、(3)步合写到一个工具函数 `split_head_reshape` 里:

```

1 def split_head_reshape(X, heads_num, head_size):
2     """
3     输入:
4         - X: 输入张量, 形状为[批量大小, 序列长度, 隐状态维度]
5     输出:
6         - output: 输出多头的矩阵, size=[batch_size * heads_num, seq_len, head_size]
7     """
8     batch_size, seq_len, hidden_size = X.shape
9     # X: [batch_size, seq_len, heads_num, head_size]
10    # 多头分组
11    X = torch.reshape(X, [batch_size, seq_len, heads_num, head_size])
12    # X: [batch_size, heads_num, seq_len, head_size]
13    # 形状重组
14    X = X.permute(0, 2, 1, 3)
15    # X: [batch_size*heads_num, seq_len, head_size]
16    X = torch.reshape(X, [batch_size * heads_num, seq_len, head_size])
17    return X

```

(4) **QKV 自注意力**: 在重组后的 $Q, K, V \in \mathbb{R}^{BM \times L \times D_m}$ 上调用前面写好的 QKVAttention 即可:

$$H = \text{attention}(Q, K, V) \in \mathbb{R}^{(BM \times L \times D_m)}, \quad (8.20)$$

每个展开后的头需要一个有效长度, 故按与张量一致的顺序复制长度:

```

1 Q, K, V = [split_head_reshape(item, heads_num, head_size) for item in (Q, K, V)]
2 head_lens = valid_lens.repeat_interleave(heads_num)
3 attention = QKVAttention(head_size)
4 out = attention(Q, K, V, head_lens)

```

(5) **重组恢复**: 上面把头维合并进了批维, 现在反向走一次同样的 reshape+transpose, 把头维拆回来:

$$(BM \times L \times D_m) \xrightarrow{\text{reshape}} (B \times M \times L \times D_m) \xrightarrow{\text{transpose}} (B \times L \times M \times D_m), \quad (8.21)$$

实现如下:

```

1 # out: [batch_size, heads_num, seq_len, head_size]
2 out = torch.reshape(out, [batch_size, heads_num, seq_len, head_size])
3 # out: [batch_size, seq_len, heads_num, head_size]
4 out = out.permute(0, 2, 1, 3)

```

(6) **多头融合**: 先把头维与单头维合并回完整的 D 维, 再过一次输出投影完成跨头信息融合:

$$(B \times L \times M \times D_m) \xrightarrow{\text{reshape}} (B \times L \times D), \quad (8.22)$$

最终输出仍为 $[B, L, D]$, 可直接接入下一层. 代码同样用 `nn.Linear` 实现:

```

1  # 输出映射
2  out_proj = nn.Linear(inputs_size, inputs_size, bias=False)
3  # 多头自注意力输出拼接
4  # out: [batch_size, seq_len, heads_num * head_size]
5  out = torch.reshape(out, [out.shape[0], out.shape[1], out.shape[2] * out.shape[3]])
6  # 输出映射
7  out = out_proj(out)

```

多头自注意力算子 把上述六步打包到一个 `MultiHeadSelfAttention` 算子里, 对外接口与单头 QKV 注意力保持一致, 便于直接替换:

```

1  class MultiHeadSelfAttention(nn.Module):
2      def __init__(self, inputs_size, heads_num, dropout=0.0):
3          super().__init__()
4          if heads_num < 1 or inputs_size < 1 or inputs_size % heads_num:
5              raise ValueError('inputs_size must be positive and divisible by heads_num')
6          # 输入的词向量的维度
7          self.inputs_size = inputs_size
8          self.qsize, self.ksize, self.vsize = inputs_size, inputs_size, inputs_size
9          # 头的数目
10         self.heads_num = heads_num
11         # 每个头的输入向量的维度
12         self.head_size = inputs_size // heads_num
13         # 输入的维度 inputs_size 需要整除头的数目 heads_num
14         assert (self.head_size * heads_num == self.inputs_size), "embed_size must be
            divisible by heads_num"
15         # 查询、键、值三个线性映射
16         self.Q_proj = nn.Linear(self.qsize, inputs_size, bias=False)
17         self.K_proj = nn.Linear(self.ksize, inputs_size, bias=False)
18         self.V_proj = nn.Linear(self.vsize, inputs_size, bias=False)
19         # 输出映射
20         self.out_proj = nn.Linear(inputs_size, inputs_size, bias=False)
21         # QKV 自注意力
22         self.attention = QKVAttention(self.head_size, dropout)
23
24     def forward(self, X, valid_lens):
25         """
26         输入:
27         - inputs: 输入张量, 形状为[批量大小, 序列长度, 隐状态维度]

```

```

28         - valid_lens: 长度矩阵, size=[batch_size]
29     输出:
30         - output: 输出矩阵, 表示的是多头注意力的结果
31     """
32     self.batch_size, self.seq_len, self.hidden_size = X.shape
33     # Q,K,V: [batch_size, seq_len, hidden_size]
34     Q, K, V = self.Q_proj(X), self.K_proj(X), self.V_proj(X)
35     # Q,K,V: [batch_size*heads_num, seq_len, head_size]
36     Q, K, V = [
37         split_head_reshape(item, self.heads_num, self.head_size)
38         for item in [Q, K, V]
39     ]
40     # 按样本、头的展开顺序复制长度, 每个头保留一项
41     valid_lens = torch.repeat_interleave(
42         valid_lens, repeats=self.heads_num, dim=0)
43     # out: [batch_size*heads_num, seq_len, head_size]
44     out = self.attention(Q, K, V, valid_lens)
45     # out: [batch_size, heads_num, seq_len, head_size]
46     out = torch.reshape(
47         out, [self.batch_size, self.heads_num, self.seq_len, self.head_size] )
48     # out: [batch_size, seq_len, heads_num, head_size]
49     out = out.permute(0, 2, 1, 3)
50     # 多头注意力输出拼接
51     # out: [batch_size, seq_len, heads_num * head_size]
52     out = torch.reshape(out, [out.shape[0], out.shape[1], out.shape[2] * out.shape[3]])
53     # 输出映射
54     out = self.out_proj(out)
55     return out

```

对上面的实现进行验证, 输入的是形状为 $2 \times 2 \times 4$ 的张量, 表示两个序列, 每个序列有 2 个词, 每个词的长度是 4 维. 代码实现如下:

```

1 torch.manual_seed(2021)
2 X = torch.rand((2, 2, 4))
3 valid_lens = torch.tensor([1, 2])
4 print(f"输入向量 {X.numpy()}")
5 multi_head_attn = MultiHeadSelfAttention(heads_num=2, inputs_size=4)
6 context = multi_head_attn(X, valid_lens)
7 print(f"注意力的输出为 : {context.detach().numpy()}")
8 print(f"注意力权重为 : {multi_head_attn.attention.attention_weights.detach().numpy()}")

```

输出结果为:

```

输入向量 [[0.13043904 0.513387 0.7425571 0.715889 ]
          [0.5704716 0.16527355 0.04430902 0.9627712 ]]

[[0.29428095 0.09917057 0.8095681 0.01688558]

```

```

[0.82223636 0.12416631 0.7489104 0.3608024 ]]
注意力的输出为 : [[[ 0.15934913 -0.06445406 -0.26478574 0.3001002 ]
[ 0.15934913 -0.06445406 -0.26478574 0.3001002 ]]

[[ 0.08127859 -0.23955524 -0.19438154 0.11489068]
[ 0.08101125 -0.23980561 -0.19439408 0.11473196]]]
注意力权重为 : [[[1.      0.      ]
[1.      0.      ]]

[[1.      0.      ]
[1.      0.      ]]

[[0.51904374 0.48095626]
[0.52375513 0.47624484]]

[[0.50038457 0.49961546]
[0.49933293 0.50066704]]]

```

两组头的输出拼接后再投影。示例第一条序列只有一个有效键，因此两个查询位置都读取该键；填充查询的输出并不会自动变为 0。后续平均汇聚仍须按有效长度屏蔽查询位置，避免补齐长度影响最终分类。

8.2.2 基于 LSTM 和多头自注意力的文本分类的模型构建

把多头自注意力嵌入到上一节的 IMDB 文本分类管道，整体结构见图 8.13：

- 1) 嵌入层：把词索引映射为词向量；
- 2) LSTM 层：用双向 LSTM 编码上下文表示；
- 3) 自注意力层：在 LSTM 输出之上做一次多头自注意力，做跨位置的特征重组；
- 4) 汇聚层：对自注意力输出做平均汇聚，得到整段文本的固定长度表示；
- 5) 线性层：把表示映射为类别得分。

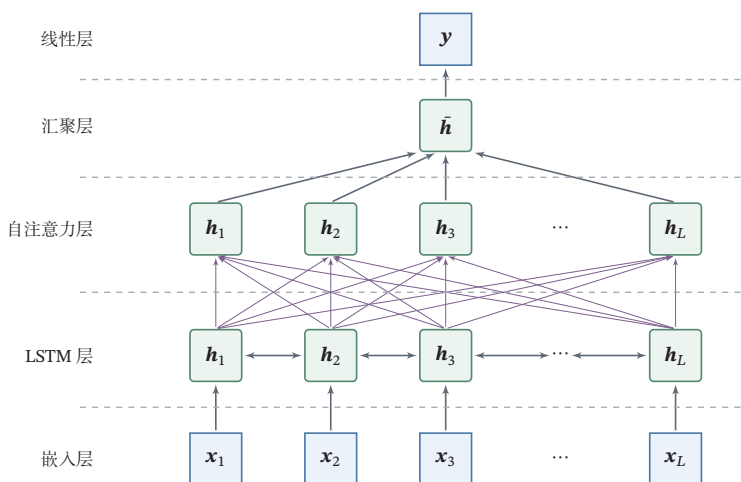


图 8.13 基于双向 LSTM 和多头自注意力的文本分类模型

嵌入与双向 LSTM 沿用第 6.4 节, 注意力层采用刚实现的多头算子, 最后接平均汇聚与线性分类头。

8.2.2.1 模型汇总

整套模型可以理解为: 在原有的双向 LSTM 基础上插入多头自注意力做特征重组, 再用平均汇聚和线性层得到最终类别得分:

```

1 class Model_LSTMSelfAttention(nn.Module):
2     def __init__(self, hidden_size, embedding_size, vocab_size, n_classes=2,
3         n_layers=1, attention=None):
4         super().__init__()
5         # LSTM单元的隐藏神经元数量, 即隐状态向量和记忆单元向量的维度
6         self.hidden_size = hidden_size
7         # 表示词向量的维度
8         self.embedding_size = embedding_size
9         # 词表大小, 即包含词的数量
10        self.vocab_size = vocab_size
11        # 表示文本分类的类别数量
12        self.n_classes = n_classes
13        # 表示LSTM的层数
14        self.n_layers = n_layers
15        # 定义嵌入层
16        self.embedding = nn.Embedding(
17            num_embeddings=self.vocab_size, embedding_dim=self.embedding_size, padding_idx
18            =0)
19        # 定义LSTM, batch_first=True表示输入形状为[批量大小, 序列长度, 嵌入维度]
20        self.lstm = nn.LSTM(input_size=self.embedding_size, hidden_size=self.hidden_size,
            num_layers=self.n_layers, batch_first=True, bidirectional=True)

```

```

21         self.attention = attention
22         # 实例化汇聚层, 汇聚层与循环网络章节使用一致
23         self.average_layer = AveragePooling()
24         # 定义线性层, 用于将语义向量映射到相应的类别
25         self.cls_fc = nn.Linear(self.hidden_size * 2, self.n_classes)
26
27     def forward(self, input_ids, valid_lens):
28         # 获取词向量
29         embedded_input = self.embedding(input_ids)
30         # 用 pack_padded_sequence 处理变长序列, 让 LSTM 跳过 [PAD]
31         packed = pack_padded_sequence(embedded_input, valid_lens.cpu(),
32                                       batch_first=True, enforce_sorted=False)
33         packed_output, _ = self.lstm(packed)
34         last_layers_hiddens, _ = pad_packed_sequence(packed_output, batch_first=True)
35         # 计算多头自注意力
36         last_layers_hiddens = self.attention(last_layers_hiddens, valid_lens)
37         # 使用汇聚层得到序列的聚合表示
38         last_layers_hiddens = self.average_layer(last_layers_hiddens, valid_lens)
39         # 将其通过线性层, 获得初步的类别数值
40         logits = self.cls_fc(last_layers_hiddens)
41         return logits

```

8.2.3 模型训练

按照RunnerV3的组装套路依次准备模型、优化器、损失函数和评价指标——这里模型组件由MultiHeadSelfAttention和上一节的双向 LSTM 拼装而成. 损失依旧是交叉熵、评价指标依旧是准确率, 与第8.1.3.1节保持一致以保证可比性:

```

1 hidden_size, embedding_size = 64, 64
2 torch.manual_seed(100)
3 multi_head_attn = MultiHeadSelfAttention(inputs_size=2 * hidden_size, heads_num=8)
4 model_attn = Model_LSTMSelfAttention(hidden_size, embedding_size, vocab_size,
5                                     n_classes=n_classes, attention=multi_head_attn)
6 runner = train_attention_model(model_attn, 'imdb_self')
7 attention_runners['self'] = runner

```

图8.14记录这次多头自注意力实验的训练损失和验证表现. 与前两种汇聚方法相比, 它还增加了位置之间的特征变换.

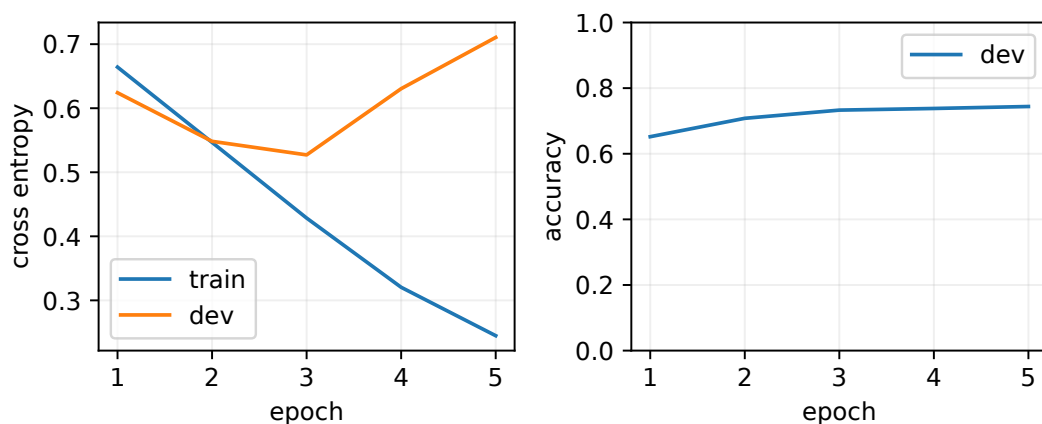


图 8.14 损失和准确率变化趋势

8.2.4 模型评价

载入最优模型,在测试集上做最终评估:

```
1 runner.load('./checkpoints/imdb_self.pt')
2 _, test_accuracy = runner.evaluate(test_loader)
3 print(f"Evaluate on test set, Accuracy: {test_accuracy:.5f}")
```

输出结果为:

```
Evaluate on test set, Accuracy: 0.75300
```

本次多头自注意力模型的测试准确率为 75.3%. 与平均汇聚 (72.8%)、加性注意力 (73.5%) 和点积注意力 (71.2%) 对照, 应把结果限定在当前 5 回合、64 维和固定子集的实验条件下. 多头结构增加了表示变换, 也增加了计算成本; 这里没有证据把它解释成尚未“释放”的必然优势.

思考

模型增加参数后, 结果变差可能来自优化不足, 也可能来自过拟合或结构不适合任务. 怎样用训练/验证损失、更多随机种子和受控的预算实验区分这些原因? 请先提出可验证的判断依据, 再调整配置.

动手练习 8.3

把公式(8.13)~(8.15)和公式(8.17)里的线性变换改成仿射变换 (引入偏置项), 重新跑实验, 观察准确率有无变化.

动手练习 8.4

把Model_LSTMSelfAttention里自写的MultiHeadSelfAttention替换成PyTorch内置的`nn.MultiheadAttention`, 在同一份双向LSTM管道上重跑IMDB情感分类, 对比两种实现的效果与开销:

1) 接好内置算子的接口差异: `nn.MultiheadAttention`默认`batch_first=False` (需显式设为`True`才与本书`[batch_size, seq_len, hidden_size]`的布局一致), 自注意力时`query`、`key`、`value`传同一个张量。

2) 重点对齐掩码语义: 本书QKVAttention用`valid_lens`构造的掩码里`True`表示有效位置, 而`nn.MultiheadAttention`的`key_padding_mask`里`True`表示该位置是[`PAD`]、需要被屏蔽——两者语义恰好相反。请由`valid_lens`推出正确的`key_padding_mask`, 并验证它确实把[`PAD`]位置的注意力权重压到了0。

3) 先复制Q/K/V及输出投影的参数, 并对齐偏置、暂退率, 核验同一输入的输出和梯度。随后才比较训练耗时; 调参使用验证集, 选定配置后评价测试集。

**8.2.5 模型预测**

挑一句英文测试一下情感预测:

```
1 text = "this movie is so great. I watched it three times already"
2 # 句子按照空格分开
3 sentence = tokenize(text)
4 # 词转ID
5 tokens = [
6     word2id_dict[word] if word in word2id_dict else word2id_dict["[UNK]"]
7     for word in sentence ]
8 # 取max_seq_len
9 tokens = tokens[:max_seq_len] or [word2id_dict["[UNK]"]]
10 # 长度
11 seq_len = torch.tensor([len(tokens)])
12 # 转换成张量
13 input_ids = torch.tensor([tokens], dtype=torch.int64)
14 # 预测
15 logits = runner.predict(input_ids, seq_len)
16 # 标签词表
17 id2label = {0: "消极", 1: "积极"}
18 # 取最大值的索引
19 label_id = torch.argmax(logits, dim=1).item()
20 # 根据索引取出输出
21 pred_label = id2label[label_id]
22 print("Label: ", pred_label)
```

输出结果为:

Label: 积极

输出为“积极”,与原句的实际情感一致.



动手练习 8.5

把模型中的双向 LSTM 去掉,仅保留多头自注意力+分类头重做实验,比较准确率变化并分析原因(提示:序列位置信息).



8.3 实践:基于自注意力模型的文本语义匹配

文本语义匹配 (Text Semantic Matching) 属于 NLP 的核心基础任务,信息检索、问答系统、文本蕴含等都会用到它.

举一个直观的例子.下面三句话中,A 和 C 在语义上更接近,但若仅按字面重合度判断,A 看起来反而最像 B:

-
- A. 什么花一年四季都开?
 - B. 什么花生一年四季都开?
 - C. 哪些花可以全年开放?
-

语义匹配的目标恰好就是让模型从“意思上”判断两句话是否一致——A 与 B 应输出 0、A 与 C 应输出 1.

本节去掉 LSTM,直接用 Transformer 编码器处理两句话.本案例的标签是二分类;语义匹配也可采用相关度打分或排序目标,具体由任务定义决定.

整体模型结构见图8.15.

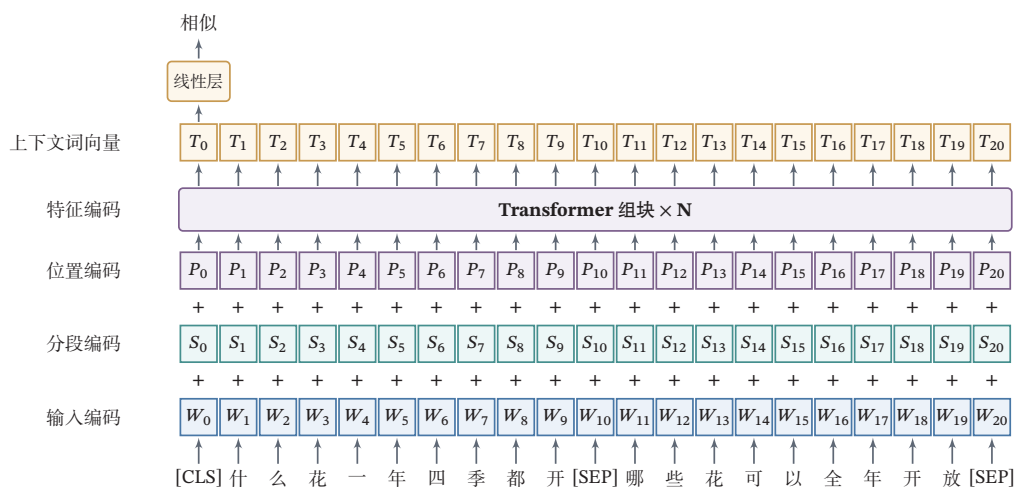


图 8.15 基于 Transformer 编码器的文本语义匹配模型结构

将两句按 [CLS]+ 句子 A+[SEP]+ 句子 B+[SEP] 拼接。[SEP] 标记边界, [CLS] 的位置用于读取分类表示。它要通过训练才可能汇集有用的信息,并不因名字特殊就天然代表整段语义。

接下来,每个字符同时携带三种编码:

- **输入编码 (词嵌入)**:刻画字符自身的语义;
- **位置编码**:补回顺序信息,弥补自注意力本身没有顺序感知的缺陷;
- **分段编码**:用 0/1 标记字符属于第一还是第二句话。

三种编码按位置相加得到最终的字符表示,输入 Transformer 编码器。编码器输出的 [CLS] 向量再经过一个线性分类头,得到“相似/不相似”的预测。

8.3.1 数据集构建

实验使用 LCQMC (Large-scale Chinese Question Matching Corpus) ——一个来自百度知道领域的中文问题匹配数据集,覆盖不同领域用户的真实提问。规模为:训练集 238 766 条、验证集 8 802 条、测试集 12 500 条。任务主要关注提问意图是否匹配,比逐字改写的一致性更宽泛。^①目录结构如下:

```
lcqmc/
|-- dev.txt.gz # 验证集
|-- test.txt.gz # 测试集
`-- train.txt.gz # 训练集
```

① LCQMC 原论文: <https://aclanthology.org/C18-1166/>。

每条样本由三列组成:前两列是两句中文文本,第三列是标签(1=相似、0=不相似)。例如:

什么花一年四季都开 什么花一年四季都是开的 1

8.3.1.1 数据加载

使用下载脚本归一化后的 gzip 文件,字段为两句文本与 0/1 标签. 默认从三个划分分别固定随机抽取 10 000/2 000/2 000 条样本,不额外重平衡;本次标签 1 的比例依次为 58.05%、49.05%、49.80%. 字符级切分按 Unicode 字符逐个查 BERT 中文字表,标点、数字和拉丁字母也占位置. 这里只借用词表,不加载 BERT 预训练权重:

```
1 import gzip
2 import random
3 from pathlib import Path
4 import numpy as np
5
6 def load_lcqmc_data(data_dir):
7     splits = []
8     for split in ('train', 'dev', 'test'):
9         with gzip.open(Path(data_dir) / f'{split}.txt.gz', 'rt', encoding='utf8') as f:
10             rows = [line.rstrip('\n').split('\t') for line in f if line.strip()]
11             splits.append([(a, b, int(label)) for a, b, label in rows])
12     return splits
13
14 train_data, dev_data, test_data = load_lcqmc_data('../datasets/lcqmc')
15 # 固定随机子集保留原始类比例,不用测试集决定样本数或训练配置.
16 full_lcqmc = False
17 if not full_lcqmc:
18     train_data = random.Random(100).sample(train_data, 10000)
19     dev_data = random.Random(101).sample(dev_data, 2000)
20     test_data = random.Random(102).sample(test_data, 2000)
21 with open('../datasets/bert-base-chinese/vocab.txt', encoding='utf8') as f:
22     word2id_dict = {word.rstrip('\n'): i for i, word in enumerate(f)}
23 assert word2id_dict['[PAD]'] == 0
24 print('train/dev/test:', len(train_data), len(dev_data), len(test_data))
```

8.3.1.2 先建立一个字符重合基线

“花”与“花生”的例子提醒我们:字面相似不等于意图相同. 不过,数据集中仍可能有不少词面线索. 先用两句字符集合的 Jaccard 相似度作基线,能帮助判断复杂模型究竟增加了多少收益. 阈值只在验证集上选择:

```
1 def char_jaccard(text_a, text_b):
2     a, b = set(text_a), set(text_b)
3     return len(a & b) / max(1, len(a | b))
```

```

4
5 def lexical_accuracy(rows, threshold):
6     return sum((char_jaccard(a, b) >= threshold) == label
7                 for a, b, label in rows) / len(rows)
8
9 thresholds = [i / 20 for i in range(21)]
10 threshold = max(thresholds, key=lambda t: lexical_accuracy(dev_data, t))
11 print(f'Jaccard threshold={threshold:.2f}, '
12       f'dev={lexical_accuracy(dev_data, threshold):.4f}, '
13       f'test={lexical_accuracy(test_data, threshold):.4f}')

```

```
Jaccard threshold=0.75, dev=0.6470, test=0.7015
```

该基线不理解语序、否定和多义词。它在本次子集上的结果为后面的训练提供参照;若调整模型方案,仍应回到验证集,不能围绕测试成绩反复试验。

8.3.1.3 构建 Dataset

下面的LCQMCDataset继承Dataset,负责单条句对的预处理。

它的核心职责包括:将文本按词表转换为ID序列(未登录字用[UNK]代替),按上节模板在两个句子之间插入[SEP]、在开头加入[CLS]。例如前述示例会被转换为:

```
[CLS] 什么花一年四季都开[SEP] 什么花一年四季都是开的[SEP]
```

每个词元除了对应的input_ids外,还要额外携带一份segment_ids(0/1)用来标记当前词元属于第一句还是第二句,与[SEP]一起明确两段语义的边界:

```

[101, 784, 720, 5709, 671, 2399, 1724, 2108, 6963, 2458, 102, 784, 720, 5709, 671, 2399,
 1724, 2108, 6963, 3221, 2458, 4638, 102]
[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1]

```

完整实现如下:

```

1 from torch.utils.data import Dataset
2
3 class LCQMCDataset(Dataset):
4     def __init__(self, data, word2id_dict, max_seq_len=128):
5         if max_seq_len < 5:
6             raise ValueError("reserve three special tokens and one token per sentence")
7         self.max_seq_len = max_seq_len
8         # 词表
9         self.word2id_dict = word2id_dict
10        # 数据
11        self.examples = data
12        # ['CLS']的id, 占位符

```

```

13         self.cls_id = self.word2id_dict['[CLS]']
14         # ['SEP']的id, 句子的分隔
15         self.sep_id = self.word2id_dict['[SEP]']
16
17     def __getitem__(self, idx):
18         # 返回单条样本
19         example = self.examples[idx]
20         text, segment, label = self.words_to_id(example)
21         return text, segment, label
22
23     def __len__(self):
24         # 返回样本的个数
25         return len(self.examples)
26
27     def words_to_id(self, example):
28         text_a, text_b, label = example
29         # text_a 转换成id的形式
30         input_ids_a = [self.word2id_dict[item] if item in self.word2id_dict else self.
31                        word2id_dict['[UNK]'] for item in text_a]
32         # text_b 转换成id的形式
33         input_ids_b = [self.word2id_dict[item] if item in self.word2id_dict else self.
34                        word2id_dict['[UNK]'] for item in text_b]
35         # 从较长的一句末尾截断, 保留两句各至少一个位置和全部特殊符号.
36         input_ids_a = input_ids_a or [self.word2id_dict['[UNK]']]
37         input_ids_b = input_ids_b or [self.word2id_dict['[UNK]']]
38         while len(input_ids_a) + len(input_ids_b) > self.max_seq_len - 3:
39             (input_ids_a if len(input_ids_a) > len(input_ids_b) else input_ids_b).pop()
40         # 加入[CLS], [SEP]
41         input_ids = [self.cls_id]+input_ids_a +[self.sep_id] +input_ids_b + [self.sep_id]
42         # 对句子text_a、text_b做id的区分, 进行分隔
43         segment_ids = [0]*(len(input_ids_a)+2)+[1]*(len(input_ids_b)+1)
44         return input_ids, segment_ids, int(label)
45
46     @property
47     def label_list(self):
48         # 0表示不相似, 1表示相似
49         return ['0', '1']
50
51 # 加载训练集
52 train_dataset = LCQMCDataset(train_data, word2id_dict)
53 # 加载验证集
54 dev_dataset = LCQMCDataset(dev_data, word2id_dict)
55 # 加载测试集
56 test_dataset = LCQMCDataset(test_data, word2id_dict)

```

8.3.1.4 构建 DataLoader

句对长度已在 Dataset 中控制:反复从较长的一句末尾截断,保留两句各至少一个位置和三个特殊符号;空句以 [UNK] 占位. `collate_fn`只按批内最长样本补齐,不能拼接后直接截取前缀,否则长句 A 可能挤掉句 B 和末尾分隔符.

```
1 from torch.nn.utils.rnn import pad_sequence
2 from torch.utils.data import DataLoader
3
4 def collate_fn(batch_data, pad_val=0):
5     # 长度控制已在LCQMCDataset完成,此处只补齐.
6     ids, segments, labels = zip(*batch_data)
7     return (pad_sequence([torch.tensor(x) for x in ids], batch_first=True, padding_value=
8         pad_val),
9         pad_sequence([torch.tensor(x) for x in segments], batch_first=True,
10             padding_value=0),
11         torch.tensor(labels))
12
13 batch_size = 64
14 train_loader = DataLoader(train_dataset, batch_size=batch_size, collate_fn=collate_fn,
15     shuffle=True, generator=torch.Generator().manual_seed(100))
16 dev_loader = DataLoader(dev_dataset, batch_size=batch_size, collate_fn=collate_fn)
17 test_loader = DataLoader(test_dataset, batch_size=batch_size, collate_fn=collate_fn)
18 print('首个验证批次的形状: ', [tuple(x.shape) for x in next(iter(dev_loader))])
```

输出结果为:

```
首个验证批次的形状: [(64, 57), (64, 57), (64,)]
```

输出的三个张量含义清晰:第一个张量 [32, 52] 是 `input_ids`, 32 为 batch、52 为本批次的最大长度 (不足者末尾补 0); 第二个张量 [32, 52] 是 `segment_ids`, 每个词元用 0/1 标记所属句子; 最后 [32] 是标签向量 (0/1).

8.3.2 模型构建

基于 Transformer 编码器的语义匹配模型由三大块组成:

- 1) 嵌入层: 把每个词元映射为向量. 除了词嵌入, 还要加入分段编码标记词元所属句子, 以及位置编码补回顺序信息.
- 2) Transformer 组块: 堆叠若干编码器块得到深层语义表示.
- 3) 线性层: 取 [CLS] 对应位置的输出向量, 经线性层得到二分类得分.

8.3.2.1 嵌入层

嵌入层负责把输入的 token 序列变成向量序列. 除了常规的词向量外, 由于自注意力本身无序、且输入由两句拼接而成, 还需要额外引入两类编码:

1) **分段编码 (Segment Encoding)**: 本任务输入是两段文本拼接, 需要给每个词元一个 0/1 标签, 便于模型区分它属于第一句还是第二句.

2) **位置编码 (Positional Encoding)**: 不含位置信息和方向掩码的全自注意力无法区分原始次序. 本节在输入端加位置向量提供顺序线索; 后文的 RoPE 则在查询和键上注入位置信息.

连同前面的词向量, 下面依次实现这三种编码.

(1) 输入编码: 把 token ID 映射为向量. 最直接的做法是用一张大查表——给定 ID 直接取出对应的嵌入向量. 本节用 `torch.nn.Embedding` 完成查表.

按原始 Transformer 的约定, 词嵌入用均值 0、标准差 $1/\sqrt{\text{emb_size}}$ 的正态分布初始化, 并在前向时乘上 $\sqrt{\text{emb_size}}$, 让词向量的范数与位置编码大致匹配, 便于后续相加:

```

1 class WordEmbedding(nn.Module):
2     def __init__(self, vocab_size, emb_size, padding_idx=0):
3         super().__init__()
4         # 词向量的维度
5         self.emb_size = emb_size
6         # 使用随机正态 (高斯) 分布初始化词向量
7         self.word_embedding = nn.Embedding(vocab_size, emb_size,
8             padding_idx=padding_idx)
9         # 用均值0、标准差 1/sqrt(emb_size) 的正态分布初始化, padding 行保持全 0
10        nn.init.normal_(self.word_embedding.weight, mean=0.0, std=emb_size ** -0.5)
11        with torch.no_grad():
12            self.word_embedding.weight[padding_idx] = 0.0
13
14        def forward(self, word):
15            word_emb = self.emb_size ** 0.5 * self.word_embedding(word)
16            return word_emb
17
18 torch.manual_seed(2021)
19 # 构造一个输入
20 X = torch.tensor([1, 0, 2])
21 # 表示构造的输入编码的词表的大小是10, 每个词的维度是4
22 word_embed = WordEmbedding(10, 4)
23 print(f"输入编码为: {X.numpy()}")
24 word_out = word_embed(X)
25 print(f"输出为: {word_out.detach().numpy()}")

```

输出结果为:

输入编码为: [1 0 2]

输出为: [[0.3243862 1.1296438 1.1001743 -0.1827752]

[0. 0. 0. 0.]

[-0.19344142 0.15751676 1.2502545 1.3996519]]

(2) **分段编码**: 当模型一次处理两个句子时, 需要让它区分每个词元的来源. 约定用 0 标记第一个句子、1 标记第二个句子, 然后用一张 2 行的嵌入表把 0/1 映射为向量. 实现上就是一张 2 行的嵌入表 (词表大小为 2 的 `nn.Embedding`), 不需要 `WordEmbedding` 里的缩放与特殊初始化.

以下面的句子为例,

什么花一年四季都开 什么花一年四季都是开的

对应的分段标记如下, 第一句填 0、第二句填 1:

0 0 0 0 0 0 0 0 0 1 1 1 1 1 1 1 1 1 1 1

实现上把分段 ID 映射为与词向量同维度的向量即可:

```
1 class SegmentEmbedding(nn.Module):
2     def __init__(self, vocab_size, emb_size):
3         super().__init__()
4         # 词向量的维度
5         self.emb_size = emb_size
6         # 分段编码
7         self.seg_embedding = nn.Embedding(vocab_size, emb_size)
8
9     def forward(self, word):
10        seg_embedding = self.seg_embedding(word)
11        return seg_embedding
```

(3) **位置编码**: 没有位置信息、没有方向掩码的全自注意力具有**排列等变性**: 打乱输入顺序, 输出按相同方式重排, 整串输出并非保持原样. 本节加入位置编码, 使模型能够区分词元的先后次序.

最经典的方案是正余弦位置编码: 维度 D 的位置编码每一维由正/余弦函数交替组成, 频率分布在多个尺度上:

$$p_{t,2i} = \sin\left(\frac{t}{10000^{2i/D}}\right), \quad (8.23)$$

$$p_{t,2i+1} = \cos\left(\frac{t}{10000^{2i/D}}\right), \quad (8.24)$$

其中 t 是当前位置, $0 \leq i < D/2$ 索引编码向量的维度——偶数维用 $\sin$ 、奇数维用 $\cos$, 频率随 i 的增大呈指数衰减. 代码实现如下:

```
1 # seq_len 为序列长度, hidden_size 为词向量的维度.
2 def get_sinuosoid_encoding(seq_len, hidden_size):
3     """位置编码"""
4     def cal_angle(pos, hidden_idx):
5         # hidden_idx 为词向量的维编号, 上面公式里的  $i = \text{hid\_idx} // 2$ 
6         return pos / np.power(10000, 2 * (hidden_idx // 2) / hidden_size)
7
```

```

8     def get_pos_angle_vec(pos):
9         return [cal_angle(pos, hidden_j) for hidden_j in range(hidden_size)]
10
11     sinusoid = np.array([get_pos_angle_vec(pos_t) for pos_t in range(seq_len)])
12     # dim 2i 偶数正弦
13     # 从0开始, 每隔2间隔求正弦值
14     sinusoid[:, 0::2] = np.sin(sinusoid[:, 0::2])
15     # dim 2i + 1 奇数余弦
16     # 从1开始, 每隔2间隔取余弦值
17     sinusoid[:, 1::2] = np.cos(sinusoid[:, 1::2])
18     # seq_len × hidden_size 得到每一个词的位置向量
19     return sinusoid.astype("float32")

```

封装为PositionalEmbedding算子并冻结梯度(位置编码是确定性变换,不参与训练):

```

1 class PositionalEmbedding(nn.Module):
2     def __init__(self, max_length, emb_size):
3         super().__init__()
4         self.emb_size = emb_size
5         # 用正余弦位置编码表初始化嵌入权重, 并冻结梯度(确定性变换, 不参与训练)
6         self.pos_encoder = nn.Embedding(max_length, emb_size)
7         sinusoid = get_sinusoid_encoding(max_length, emb_size)
8         with torch.no_grad():
9             self.pos_encoder.weight.copy_(torch.from_numpy(sinusoid))
10        self.pos_encoder.weight.requires_grad = False
11
12    def forward(self, pos):
13        return self.pos_encoder(pos)

```

直接画出来更直观——把前 100 个位置的若干维度提取出来逐位置画曲线(绘图代码见配套 notebook)。

结果见图8.16: 每一维都是一条频率不同的正弦曲线, 整体取值落在 $[-1, 1]$ 。频率梯度的设计让低维(高频)捕捉细粒度位置、高维(低频)捕捉粗粒度位置, 给注意力提供丰富的相对位置线索。

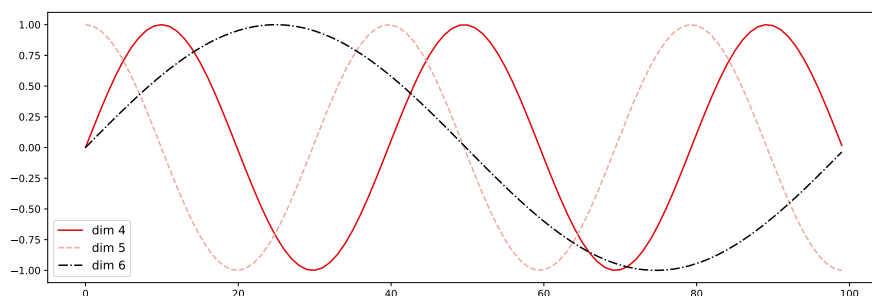


图 8.16 位置编码的可视化

**动手练习 8.6**

取两组不同位置的正余弦位置编码做内积, 绘制内积随相对距离的变化曲线, 验证“能否仅从内积推断出位置间的相对距离”。

**思考**

总结基于三角函数的位置编码的优缺点, 并思考更好的位置编码方式。

(4) **嵌入层汇总**: 把三种编码按位置相加, 再经一次层规范化和 Dropout 得到最终输入向量。层规范化稳定数值分布、Dropout 提供轻度正则化:

```
1 class TransformerEmbeddings(nn.Module):
2     """
3     包括输入编码、分段编码、位置编码
4     """
5     def __init__(self, vocab_size, hidden_size=768, hidden_dropout_prob=0.1,
6                   seq_len=512, segment_size=2):
7         super().__init__()
8         # 输入编码向量
9         self.word_embeddings = WordEmbedding(vocab_size, hidden_size)
10        # 位置编码向量
11        self.position_embeddings = PositionalEmbedding(seq_len, hidden_size)
12        # 分段编码向量
13        self.segment_embeddings = SegmentEmbedding(segment_size, hidden_size)
14        # 层规范化
15        self.layer_norm = nn.LayerNorm(hidden_size)
16        # 暂退操作
17        self.dropout = nn.Dropout(hidden_dropout_prob)
18
19    def forward(self, input_ids, segment_ids = None, position_ids = None):
```

```

20     if position_ids is None:
21         # 创建与输入形状相同的全1张量, 例如每行是 [1, 1, 1, 1]
22         ones = torch.ones_like(input_ids, dtype=torch.int64)
23         # 沿序列维累加, 得到从1开始的位置编号, 例如 [1, 2, 3, 4]
24         seq_length = torch.cumsum(ones, dim=-1)
25         # 减去1得到从0开始的位置索引, 例如 [0, 1, 2, 3]
26         position_ids = seq_length - ones
27         # 梯度不更新
28         position_ids.requires_grad=False
29     if segment_ids is None:
30         segment_ids = torch.zeros_like(input_ids)
31         # 输入编码
32         input_embeddings = self.word_embeddings(input_ids)
33         # 分段编码
34         segment_embeddings = self.segment_embeddings(segment_ids)
35         # 位置编码
36         position_embeddings = self.position_embeddings(position_ids)
37         # 输入张量、分段张量、位置张量进行叠加
38         embeddings = input_embeddings + segment_embeddings + position_embeddings
39         # 层规范化
40         embeddings = self.layer_norm(embeddings)
41         # 暂退操作
42         embeddings = self.dropout(embeddings)
43
44     return embeddings

```

8.3.2.2 Transformer 组块

Transformer 编码器是若干**组块** (block) 的堆叠, 每个组块的结构见图8.17: 多头自注意力、加与规范化 (Add&Norm)、逐位前馈、加与规范化。两条残差路径让信息可以越过任何一层直接前向/后向传递, 是深层 Transformer 能训得稳的关键。

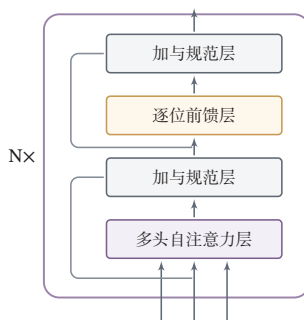


图 8.17 Transformer 组块结构

下面分别实现这四个模块.

多头自注意力层 这里直接调用 PyTorch 内置的 `nn.MultiheadAttention`: 它与第 8.2.1.3 节自实现的 `MultiHeadSelfAttention` 等价, 但掩码改用布尔 `key_padding_mask` 传入, 更适配后面变长文本任务的填充.

加与规范化层 **加与规范化** (Add&Norm) 层做两件事: 把子层输出与残差相加, 再做一次 **层规范化** (Layer Normalization, LN) 调节中间表示的统计量. 残差连接和规范化有助于训练深层网络, 但深度增加后仍可能出现梯度不稳定, 需要结合初始化、规范化位置和学习率设置处理.

记多头自注意力的输入为 $\mathbf{X} \in \mathbb{R}^{B \times L \times D}$ 、输出为 $\mathbf{H}$, 第一次 Add&Norm 写为

$$\mathbf{H} = \text{LN}(\mathbf{H} + \mathbf{X}), \quad (8.25)$$

其中 $\text{LN}(\cdot)$ 是层规范化操作.

实现上还在残差路径里加一次 Dropout, 对子层输出做轻度扰动:

```

1 class AddNorm(nn.Module):
2     """加与规范化"""
3     def __init__(self, size, dropout_rate):
4         super().__init__()
5         self.layer_norm = nn.LayerNorm(size)
6         self.dropout = nn.Dropout(dropout_rate)
7
8     def forward(self, X, H):
9         """
10         X: 表示被包裹的非线性层的输入
11         H: 表示被包裹的非线性层的输出
12         """
13         H = X + self.dropout(H)
14         return self.layer_norm(H)

```

思考

Transformer 中层规范化相对于残差路径的位置有两种主流写法——**Pre-LN** (Pre-LayerNorm) 和 **Post-LN** (Post-LayerNorm). 设子层为 $\mathbf{H} = f(\mathbf{X})$, 二者分别为:

$$\text{Pre-LN : } \mathbf{H} = \mathbf{X} + f(\text{LN}(\mathbf{X})),$$

$$\text{Post-LN : } \mathbf{H} = \text{LN}(f(\mathbf{X}) + \mathbf{X}),$$

在相同模型规模和训练预算下, 比较 Pre-LN 与 Post-LN 的梯度、收敛速度和验证表现. 两者应分别调节学习率与预热设置, 不能预设某一种配置总有更高的性能上限.

逐位前馈层 逐位前馈网络 (position-wise Feed Forward Networks, FFN) 是两层带 ReLU 激活的全连接, 独立作用在序列里的每个位置上——可以视为“核大小为 1”的卷积, 做的是非线性特征重映射, 而非跨位置混合。

输入张量 $\mathbf{H} \in \mathbb{R}^{B \times L \times D}$ (B 批量、 L 序列长度、 D 特征维度), FFN 计算公式为

$$\text{FFN}(\mathbf{H}) = \max(0, \mathbf{H}\mathbf{W}_1 + \mathbf{b}_1)\mathbf{W}_2 + \mathbf{b}_2, \quad (8.26)$$

其中 $\mathbf{W}_1 \in \mathbb{R}^{D \times D'}$ 、 $\mathbf{W}_2 \in \mathbb{R}^{D' \times D}$ 、 $\mathbf{b}_1 \in \mathbb{R}^{D'}$ 、 $\mathbf{b}_2 \in \mathbb{R}^D$ 都是可学习参数。 D' 通常取 $4D$, 先升维放大表达力, 再降维回原始尺寸。

实现如下:

```

1 class PositionwiseFFN(nn.Module):
2     """逐位前馈层"""
3     def __init__(self, input_size, mid_size, dropout=0.1):
4         super().__init__()
5         self.W_1 = nn.Linear(input_size, mid_size)
6         self.W_2 = nn.Linear(mid_size, input_size)
7         self.dropout = nn.Dropout(dropout)
8
9     def forward(self, X):
10        return self.W_2(self.dropout(F.relu(self.W_1(X))))

```

加与规范化层 逐位前馈之后再接一次 Add&Norm, 结构与第一次完全一致, 直接复用上面的 AddNorm 算子即可。

Transformer 组块汇总 把上述四个模块按图 8.17 的顺序串起来, 得到一个完整的 Transformer 组块:

```

1 class TransformerBlock(nn.Module):
2     def __init__(self, input_size, head_num, ffnn_size, dropout=0.1,
3         attn_dropout=None, act_dropout=None):
4         super().__init__()
5         # 输入数据的维度
6         self.input_size = input_size
7         # 多头自注意力多头的个数
8         self.head_num = head_num
9         # 逐位前馈层的大小
10        self.ffnn_size = ffnn_size
11        # 加与规范化的Dropout的参数
12        self.dropout = dropout
13        # 多头自注意力的Dropout参数
14        self.attn_dropout = dropout if attn_dropout is None else attn_dropout
15        # 逐位前馈层的Dropout参数
16        self.act_dropout = dropout if act_dropout is None else act_dropout

```

```

17     # 多头自注意力层, 使用 PyTorch 内置 nn.MultiheadAttention (注意 head 是小写 h)
18     # batch_first=True 表示输入形状是 [batch, length, embedding]
19     self.multi_head_attention = nn.MultiheadAttention(
20         self.input_size,
21         self.head_num,
22         dropout=self.attn_dropout,
23         batch_first=True)
24     # 逐位前馈层
25     self.ffn = PositionwiseFFN(self.input_size, self.ffn_size, self.act_dropout)
26     # 加与规范层
27     self.addnorm1 = AddNorm(self.input_size, self.dropout)
28     self.addnorm2 = AddNorm(self.input_size, self.dropout)
29
30     def forward(self, X, key_padding_mask=None):
31         # 多头自注意力层, 自注意力的 query/key/value 都是 X
32         # key_padding_mask 形状为 [batch, length], True 表示该位置是 [PAD], 会被掩蔽
33         X_attn, attn_weights = self.multi_head_attention(
34             X, X, X, key_padding_mask=key_padding_mask,
35             need_weights=True, average_attn_weights=False)
36         # 加与规范层
37         X = self.addnorm1(X, X_attn)
38         # 逐位前馈层
39         X_ffn = self.ffn(X)
40         # 加与规范层
41         X = self.addnorm2(X, X_ffn)
42         return X, attn_weights

```

8.3.2.3 模型汇总

最后把嵌入层、若干Transformer组块和分类头串起来,得到完整的语义匹配模型:

整体结构包含 5 个部分:词嵌入、分段编码、位置编码、Transformer 堆叠和最后的二分类头:

```

1 class Model_Transformer(nn.Module):
2     def __init__(self, vocab_size, n_block=2, hidden_size=768, heads_num=12,
3         intermediate_size=3072, hidden_dropout=0.1, attention_dropout=0.1,
4         act_dropout=0, seq_len=512, num_classes=2):
5         super().__init__()
6         # 词表大小, 即包含词的数量
7         self.vocab_size = vocab_size
8         # Transformer组块的数目
9         self.n_block = n_block
10        # 每个词映射成稠密向量的维度
11        self.hidden_size = hidden_size
12        # 多头自注意力的个数
13        self.heads_num = heads_num

```

```

14         # 逐位前馈层的维度
15         self.intermediate_size = intermediate_size
16         # Transformer组块的两个子模块中的暂退操作的参数
17         self.hidden_dropout = hidden_dropout
18         self.attention_dropout = attention_dropout
19         self.act_dropout = act_dropout
20         # 位置编码的大小seq_len
21         self.seq_len = seq_len
22         # 类别数
23         self.num_classes = num_classes
24         # 实例化输入编码、分段编码和位置编码
25         self.embeddings = TransformerEmbeddings(
26             self.vocab_size, self.hidden_size, self.hidden_dropout, self.seq_len)
27         # 实例化Transformer的编码器
28         self.layers = nn.ModuleList([])
29         for i in range(n_block):
30             encoder_layer = TransformerBlock(hidden_size, heads_num, intermediate_size,
31                 dropout=hidden_dropout, attn_dropout=attention_dropout, act_dropout=
32                     act_dropout,)
33             self.layers.append(encoder_layer)
34         # 全连接层
35         self.dense = nn.Linear(hidden_size, hidden_size)
36         # 双曲正切激活函数
37         self.activation = nn.Tanh()
38         # 最后一层分类器
39         self.classifier = nn.Linear(hidden_size, num_classes)
40
41     def forward(self, input_ids, segment_ids, position_ids=None, attention_mask=None):
42         # 构造 PyTorch MultiheadAttention 的 key_padding_mask: [batch, length] 布尔张量, True
43         # 表示 [PAD]
44         if attention_mask is None:
45             attention_mask = (input_ids == 0)
46         # 抽取特征向量
47         embedding_output = self.embeddings(
48             input_ids=input_ids, position_ids=position_ids, segment_ids=segment_ids )
49         sequence_output = embedding_output
50         self._attention_weights = []
51         # Transformer的输出和注意力权重的输出
52         for i, encoder_layer in enumerate(self.layers):
53             sequence_output, atten_weights = encoder_layer(
54                 sequence_output, key_padding_mask=attention_mask)
55             self._attention_weights.append(atten_weights.detach())
56         # 选择第0个位置的向量作为句向量
57         first_token_tensor = sequence_output[:, 0]
58         # 输出层

```

```

57         pooled_output = self.dense(first_token_tensor)
58         pooled_output = self.activation(pooled_output)
59         # 句子级别的输出经过分类器
60         logits = self.classifier(pooled_output)
61         return logits
62
63     @property
64     def attention_weights(self):
65         return self._attention_weights

```

8.3.3 模型训练

本次采用1个编码器组块,隐藏维度64,4个注意力头,FFN中间维度256,最大长度128,训练5个回合. AdamW学习率为0.001,权重衰减为0.01;一维偏置和LayerNorm参数不衰减. 较小规模便于检查完整流程,全量训练可打开full_lcqmc,其指标应另行记录:

```

1  torch.manual_seed(100)
2  vocab_size, num_classes, heads_num = len(word2id_dict), 2, 4
3  model = Model_Transformer(vocab_size=vocab_size, n_block=1, num_classes=num_classes,
4                             hidden_size=64, heads_num=heads_num, intermediate_size=256, seq_len
5                             =128)
6  decay_params, no_decay_params = [], []
7  for name, parameter in model.named_parameters():
8      (no_decay_params if parameter.ndim == 1 else decay_params).append(parameter)
9  optimizer = torch.optim.AdamW([
10      {'params': decay_params, 'weight_decay': 0.01},
11      {'params': no_decay_params, 'weight_decay': 0.0}], lr=0.001)
12  runner = RunnerV3(model, optimizer, nn.CrossEntropyLoss(), metric_fn=accuracy)
13  runner.fit(train_loader, dev_loader, num_epochs=5, log_every=1,
14             best_path='./checkpoints/lcqmc.pt', grad_clip_norm=1.0)

```

训练过程中的损失和准确率曲线见图8.18.

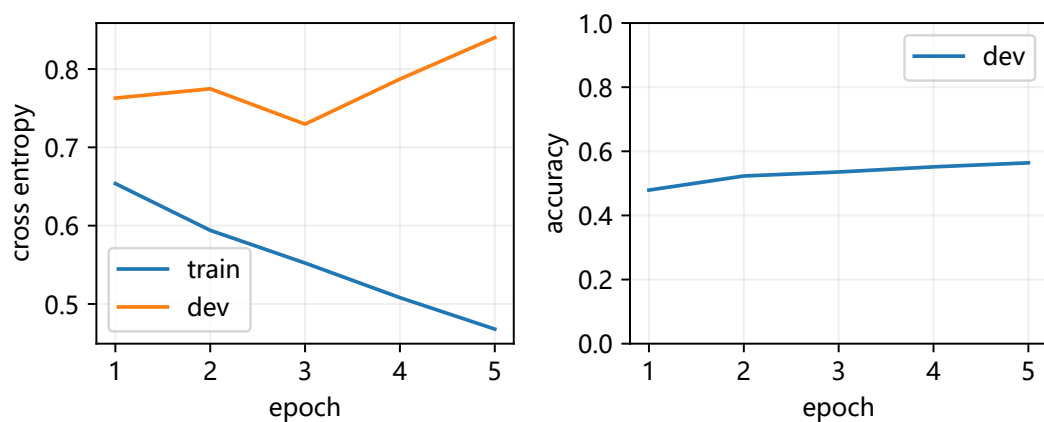


图 8.18 基于 Transformer 模型的语义匹配任务的损失和准确率变化趋势

本次训练损失从 0.654 降至 0.468, 但验证损失没有同步下降, 最后为 0.840; 验证准确率则由 47.9% 升至 56.4%。损失和准确率可以不同步, 因为前者还衡量预测置信度。这组结果不能支持“只是网络太小、增加深度即可解决”的判断, 还应检查样本规模、训练与验证的类别比例、输入表示及过拟合。

8.3.4 模型评价

载入最优快照, 在测试集上做最终评估:

```
1 runner.load('./checkpoints/lcqmc.pt')
2 _, test_accuracy = runner.evaluate(test_loader)
3 print(f"Evaluate on test set, Accuracy: {test_accuracy:.5f}")
```

输出结果为:

```
Evaluate on test set, Accuracy: 0.58750
```

这次小模型的测试准确率为 58.75%, 低于字符重合基线的 70.15%; 验证集也未超过该基线。因此本例完成了 Transformer 分类器的实现, 却没有证明它在当前预算下优于简单规则。扩大训练集、使用预训练编码器或改进训练设置都可以成为后续实验, 但应先固定新的比较规则。



动手练习 8.7

先与字符重合基线比较, 再把 `n_block` 从 1 增加到 2、4、6, 观察更深的编码器是否能进一步提升准确率, 同时记录显存和训练时长的变化。

8.3.5 模型预测

另外手写一对问句,沿用数据集中的句对截断与特殊符号规则,检查从原始文本到预测的完整路径:

```
1 text_a, text_b = '电脑怎么录像?', '如何在计算机上录视频'
2 example = LCQMCDataset([(text_a, text_b, 0)], word2id_dict)[0]
3 input_ids, segment_ids, _ = collate_fn([example])
4 logits = runner.predict(input_ids, segment_ids)
5 label_id = logits.argmax(dim=1).item()
6 print(f'预测的label标签 {label_id}')
```

输出结果为:

```
预测的label标签 1
```

该手写样例被预测为1,表示问题意图相似. 单个直观例子只是流程检查,不能替代独立测试集上的评价.

8.3.6 注意力可视化

分类结果只有一个标签,注意力矩阵则保留了每个头在各个查询位置上的分配. 以下从同一快照和输入提取矩阵,观察分布是否集中及各头有何不同:

```
1 # runner已加载验证集最佳快照,复用与预测相同的预处理.
2 model = runner.model
3 model.eval()
4 with torch.no_grad():
5     model(input_ids, segment_ids)
6 id2word = {index: word for word, index in word2id_dict.items()}
7 texts = [id2word[index] for index in input_ids[0].tolist()]
8 attention_matrix = model.attention_weights[0][0] # [head, query, key]
9 print('注意力矩阵形状:', tuple(attention_matrix.shape))
```

把每个头的权重画成热图(绘图代码见配套 notebook).

4个头的权重矩阵见图8.19. 位置编号与词元对应如下. 四个头使用同一颜色范围,以免每幅图独立拉伸色阶造成强弱错觉.

位置	0	1	2	3	4	5	6	7	8	9
词元	[CLS]	电	脑	怎	么	录	像	?	[SEP]	如
位置	10	11	12	13	14	15	16	17	18	19
词元	何	在	计	算	机	上	录	视	频	[SEP]

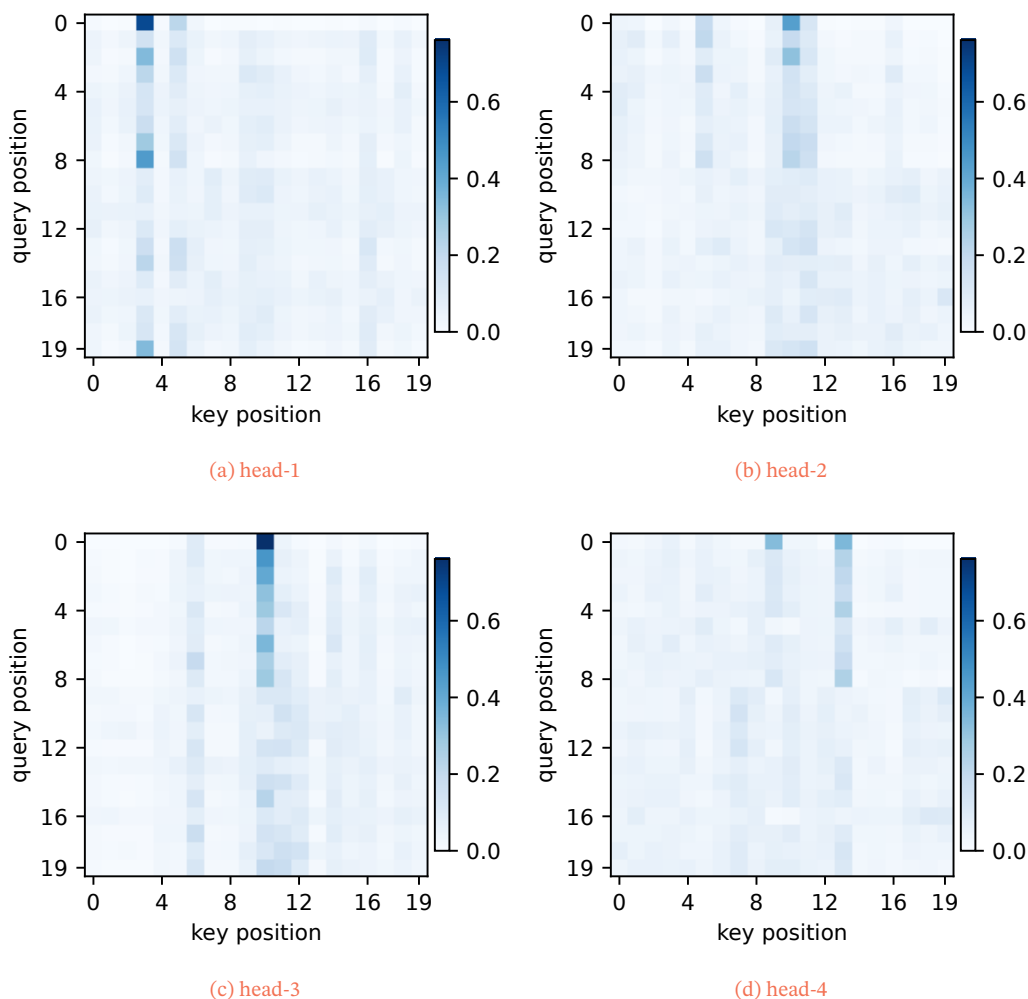


图 8.19 同一输入、同一快照的 4 个注意力头;颜色范围保持一致

热图中颜色越深表示权重越高. 不同注意力头对同一输入给出了不同的权重分布, 部分头的权重明显集中在少数键位置上. 要进一步判断这些集中位置是否对应词组关联或跨句语义对齐, 需要结合具体词元及更多样本分析, 不能仅凭一组热图得出结论.

动手练习 8.8



延续前面 “仅保留多头自注意力 + 分类头” 的练习思路, 将 `Model_Transformer` 直接用于上一节的 IMDB 文本分类任务. 提示: 分类任务中不需要分段编码, 只用词嵌入 + 位置编码即可.

8.4 现代 Transformer 的若干优化

前面的分类器组合了 2017 年 Transformer 编码器的正余弦位置编码、Post-LN 与逐位 FFN, 以及 BERT 式的分段嵌入和分类读取位置. 它是用于拆解计算流程的教学实现. 下面沿三条问题线继续: 深层网络如何传播梯度, 位置信息如何进入注意力, 以及长序列和大参数量如何降低执行开销.

8.4.1 Pre-LayerNorm vs Post-LayerNorm

原始 Transformer 采用 Post-LN:

$$\mathbf{x} \leftarrow \text{LayerNorm}(\mathbf{x} + \text{Sublayer}(\mathbf{x})). \quad (8.27)$$

另一种常用安排是 Pre-LN, 将规范化移到子层之前.^②

$$\mathbf{x} \leftarrow \mathbf{x} + \text{Sublayer}(\text{LayerNorm}(\mathbf{x})). \quad (8.28)$$

规范化的位置会影响深层网络中的梯度传播. Pre-LN 为残差路径提供了更直接的梯度通路, 通常能改善深层 Transformer 的训练稳定性; 但仍需要根据模型规模和优化设置选择学习率与预热策略.

8.4.2 旋转位置编码 RoPE

旋转位置编码 (Rotary Position Embedding, RoPE)^③ 是 Llama、GPT-NeoX 等模型选用的位置编码方案. 与正余弦位置编码“加到输入”不同, RoPE 直接对查询和键向量进行旋转来注入位置信息.

把 d 维向量按 $d/2$ 对 (x_{2i}, x_{2i+1}) 分组, 在 2D 平面上各自旋转角度 $m\theta_i$ (m 为位置索引, $\theta_i = 10000^{-2i/d}$, 不同维度对应不同频率):

$$\begin{pmatrix} x'_{2i} \\ x'_{2i+1} \end{pmatrix} = \begin{pmatrix} \cos(m\theta_i) & -\sin(m\theta_i) \\ \sin(m\theta_i) & \cos(m\theta_i) \end{pmatrix} \begin{pmatrix} x_{2i} \\ x_{2i+1} \end{pmatrix}. \quad (8.29)$$

RoPE 的好处

- **相对位置自然成立**: 对给定的查询和键, 旋转引入的位置依赖仅与相对位置有关; 注意力打分时仍取决于查询和键的内容;
- **可配合长上下文扩展**: 位置插值、YaRN 等方法调整位置或旋转频率, 并可结合长序列继续训练; RoPE 本身不保证在未见长度上可靠工作;

② 规范化位置与初始化梯度的分析见 <https://arxiv.org/abs/2002.04745>.

③ RoFormer 原论文: <https://arxiv.org/abs/2104.09864>.

- **零额外参数**: 纯几何变换, 不占用任何可学习参数.

```

1 def precompute_rope_freqs(dim, max_seq_len, base=10000.0, dtype=torch.float32):
2     if dim < 2 or dim % 2 or max_seq_len < 1 or base <= 0:
3         raise ValueError('RoPE needs a positive even dimension, length and base')
4     inv_freq = base ** (-torch.arange(0, dim, 2, dtype=dtype) / dim)
5     angles = torch.outer(torch.arange(max_seq_len, dtype=dtype), inv_freq)
6     return torch.polar(torch.ones_like(angles), angles)
7
8 def apply_rope(x, freqs_complex, start_pos=0):
9     # x:[..., seq, dim], start_pos用于分块或带KV缓存的解码.
10    if x.ndim < 2 or x.shape[-1] % 2 or not x.is_floating_point():
11        raise ValueError('x needs sequence and even feature dimensions')
12    length, dim = x.shape[-2:]
13    if start_pos < 0 or start_pos + length > len(freqs_complex):
14        raise ValueError('requested positions exceed precomputed frequencies')
15    if freqs_complex.shape[-1] != dim // 2:
16        raise ValueError('frequency dimension does not match x')
17    work = x.double() if x.dtype == torch.float64 else x.float()
18    pairs = work.reshape(*x.shape[:-1], -1, 2).contiguous()
19    z = torch.view_as_complex(pairs)
20    f = freqs_complex[start_pos:start_pos + length].to(device=x.device, dtype=z.dtype)
21    return torch.view_as_real(z * f).flatten(-2).to(x.dtype)

```

动手练习 8.9

借助本节给出的 `precompute_rope_freqs` 与 `apply_rope`, 把第 8.3 节文本语义匹配实验里 Transformer 用的正余弦位置编码替换成 RoPE, 体会两种位置编码的差异:

- 1) 注意 RoPE 的接入位置与正余弦编码不同: 正余弦编码是加到输入词向量上, 而 RoPE 要分别作用在每一层注意力的查询 **Q** 和键 **K** 上 (值 **V** 不旋转). 请把 `apply_rope` 插到自注意力算子算出 **Q**、**K** 之后、做缩放点积之前, 并去掉原来加在输入上的正余弦编码.
- 2) 在相同数据与超参下分别训练“正余弦编码”和“RoPE”两个版本, 对比收敛速度与最终验证准确率.
- 3) 测试外推能力: 把推断时的序列长度设为训练时最大长度的 2 倍, 比较两种位置编码的表现. 区分公式能否计算更远位置与模型能否有效利用更长上下文, 不预设 RoPE 一定更好.

8.4.3 三种架构范式

Transformer 家族常见的三种典型架构对比见表 8.1.

表 8.1 Transformer 的三种架构范式

架构	典型模型	注意力	适用任务
Encoder-only	BERT、RoBERTa	双向（全局）注意力	理解任务：分类、抽取、检索
Decoder-only	GPT、Llama、Qwen	因果（单向）注意力	生成任务：续写、对话、推理
Encoder-Decoder	原始 Transformer、T5、BART	encoder 双向 + decoder 因果 + cross-attention	序列到序列：翻译、摘要

架构与训练目标应一起选择 Decoder-only 可以把输入提示和待生成内容接成一个序列，用下一词元预测统一训练和生成接口。Encoder-decoder 则先编码输入，再由解码器通过交叉注意力读取；两者都能使用 KV 缓存。编码器也可以通过掩码重建等目标利用未配对文本。选择时要比较输入输出长度、训练目标、延迟和可用预训练模型，不能只凭架构名称判断容量或吞吐率。

8.4.4 混合专家 MoE

固定序列长度时，稠密 Transformer 中线性投影和前馈网络的计算量通常随隐藏维度平方增长；全自注意力还包含随序列长度平方增长的成对计算。**混合专家**（Mixture of Experts, MoE）常把一个稠密 FFN 替换为多个专家 FFN 与路由器，每个词元只选择其中一部分参与计算。选择数量与专家大小由模型设计决定，并非统一为 1 或 2 个。这样可以增加总参数容量，同时控制每词元的激活计算量；专家权重仍需存储。

$$\text{MoE}(\mathbf{x}) = \sum_{e=1}^E g_e(\mathbf{x}) \cdot E_e(\mathbf{x}), \quad (8.30)$$

其中 $g_e(\mathbf{x})$ 是路由器输出的稀疏门控权重（通常只在 top- k 个专家上为非零）， E_e 是第 e 个专家 FFN。代表模型有 Mixtral 8x7B（8 专家，每 token 选 2 个）、DeepSeek-V3 等。

下面给出一个 Top- k 路由 MoE 的最小可运行实现，仅供理解原理：

```

1 class MoFeedForward(nn.Module):
2     """简化版 Mixture-of-Experts FFN, Top-2 路由。"""
3
4     def __init__(self, n_embd, n_experts=4, top_k=2):
5         super().__init__()
6         if n_embd < 1 or not 1 <= top_k <= n_experts:
7             raise ValueError('require n_embd > 0 and 1 <= top_k <= n_experts')
8         self.n_experts = n_experts
9         self.top_k = top_k

```

```

10     # n_experts 个独立 FFN
11     self.experts = nn.ModuleList([
12         nn.Sequential(nn.Linear(n_embd, 4 * n_embd), nn.GELU(), nn.Linear(4 * n_embd,
13             n_embd))
14         for _ in range(n_experts)
15     ])
16     # 路由器: n_embd -> n_experts
17     self.gate = nn.Linear(n_embd, n_experts, bias=False)
18
19     def forward(self, x):
20         # x: [B, T, C]
21         B, T, C = x.shape
22         # 1. 每个 token 选 top-k 个专家
23         logits = self.gate(x) # [B, T, n_experts]
24         weights, idx = torch.topk(logits, self.top_k, dim=-1) # [B, T, top_k]
25         weights = F.softmax(weights, dim=-1) # 归一化
26         # 2. 把每个 token 路由到选中的专家
27         out = torch.zeros_like(x)
28         for e in range(self.n_experts):
29             # 哪些 token 选了第 e 个专家?
30             mask = (idx == e).any(dim=-1) # [B, T]
31             if mask.any():
32                 # 对应位置的路由权重
33                 w = (weights * (idx == e).float()).sum(dim=-1) # [B, T]
34                 out[mask] = out[mask] + (w[mask].unsqueeze(-1) * self.experts[e](x[mask]))
35         return out

```

提醒

本实现先选 Top-k, 再在选中项内部做 Softmax. 若 $k = 1$, 归一化后的唯一权重恒为 1, 任务损失不能通过该权重训练路由器; 选择索引本身也不可微. 因此默认使用 $k = 2$. 这只是路由与专家组合的演示, 真实 MoE 还需处理:

- 负载均衡: 需要防止词元过度集中到少数专家. 可采用辅助损失, 也可调整路由偏置等; DeepSeek-V3 的主要负载均衡策略不依赖辅助损失, 不能把加辅助损失视为必要条件;
- 分布式通信: 专家分布在不同 GPU 上, 路由会引入大量 all-to-all 通信;
- 专家容量: 容量受限的实现需要处理超额词元, 有些方案会丢弃超额分派; 也有不丢弃词元的实现, 但要处理相应的计算、存储和负载问题.

8.4.5 长上下文与 FlashAttention

固定隐藏维度时, 全自注意力的成对计算量随序列长度 N 按 $O(N^2)$ 增长; 朴素实现还会保存 $N \times N$ 矩阵, 带来二次存储开销. 减少计算次数与减少显存读写是不同的优化方向:

算法层面:稀疏注意力

- **滑动窗口**: 仅连接窗口内的位置. 双向编码可读取两侧邻居, 因果解码只能读取当前位置及之前的窗口; Longformer 还结合全局位置, Mistral 采用因果滑动窗口.
- **Dilated Attention**: 将注意力窗口按层次扩张以兼顾局部和全局信息, 代表如 LongNet;
- **压缩历史记忆**: 例如 Compressive Transformer 在循环记忆机制中压缩较旧的状态, 再供后续片段读取; 它不等同于对当前完整注意力矩阵直接做稀疏化.

系统层面: FlashAttention FlashAttention^④通过分块计算与在线 Softmax, 避免在 GPU 显存中保存完整的 $N \times N$ 打分矩阵, 减少显存与片上存储之间的数据搬运. 在通常浮点误差范围内, 它计算精确注意力; 全注意力的成对运算仍为二次量级:

- 将 Q, K, V 沿序列维拆成大小为 B_r, B_c 的块;
- 逐块算注意力, 并用 online softmax 技巧滚动更新中间结果;
- 块内计算利用片上存储, 并把较小的统计量和结果写回显存; 具体加速幅度依赖设备、形状与实现, 需要在目标配置上测量.

PyTorch 的 `scaled_dot_product_attention` 可以选择 FlashAttention 等后端; 实际选择取决于设备、数据类型、输入形状和后端限制, 不保证每次调用都使用 FlashAttention:

```
1 # [batch, head, length, head_dim], 该例没有填充.
2 q, k, v = [torch.randn(2, 4, 8, 16) for _ in range(3)]
3 out = F.scaled_dot_product_attention(q, k, v, is_causal=True, dropout_p=0.0)
4 print(out.shape)
```

`dropout_p` 不会自动随外部模块切换训练/评价状态, 推断时应传 0. 布尔 `attn_mask` 的 True 表示允许参与; 内置多头注意力的屏蔽掩码采用相反约定. 上例没有填充; 若同时需要因果和填充约束, 应先构造完整的允许位置, 再核对接口支持情况.

8.5 小结

从固定平均到可学习汇聚, 再到每个位置发出查询, 注意力的变化可以沿张量形状和可见位置逐步追踪. 实现时, Softmax 的归一化维度、键与查询的掩码, 以及多头展开顺序, 必须始终对应同一套数学定义.

实验也给出一个具体教训: 增加结构不保证在有限预算下改善分类. IMDB 中不同汇聚方式没有一致收益, LCQMC 的小型 Transformer 也未超过字符重合基线. 保留简单参照、记录训练和验证曲线, 比只报告一个看似不错的准确率更有助于找出下一步该查什么.

④ 算法与 IO 分析: <https://arxiv.org/abs/2205.14135>.

章末讨论的结构和执行策略解决不同问题,不能机械地全部堆到一个模型中. 第 10 章将从小型自回归语言模型出发,继续讨论训练、生成和工具调用;本章对查询、掩码、位置以及实验比较的检查方法仍会用到.

第9章 图神经网络

一间空手道俱乐部分裂后,成员会跟随哪一方?如果只知道两位核心成员的选择,以及成员之间的交往关系,能否推测其他人的去向?这次输入不再是一段影评或一幅图像,而是一张关系网:同一个人可以有1位朋友,也可以有十几位;调换成员编号,关系本身并没有改变.

图神经网络 (Graph Neural Network, GNN) 沿边交换信息,学习节点、边或整张图的表示. 把邻接矩阵展平成向量也能交给 MLP, 但这样的设计往往绑定节点数和编号. 我们希望构造的算子能适应邻居数量的变化,并在节点重新编号后给出相应重排的输出. 本章围绕4种消息传递方式展开:

- **GCN** (Graph Convolutional Network):图卷积;
- **GraphSAGE**:邻居采样 + 聚合;
- **GAT** (Graph Attention Network):图注意力;
- **GIN** (Graph Isomorphism Network):用求和与 MLP 研究图结构区分能力.

先在34个成员的小图上手写算子,看清每个节点从哪里收到信息,再用 PyTorch Geometric (PyG) 处理论文引用网络和蛋白质结构图. 途中将遇到三个不同的预测目标:成员属于哪一派、两人之间是否缺失一条边、整张图具有什么属性. 内容对应《神经网络与深度学习》第9章,知识点见图9.1.

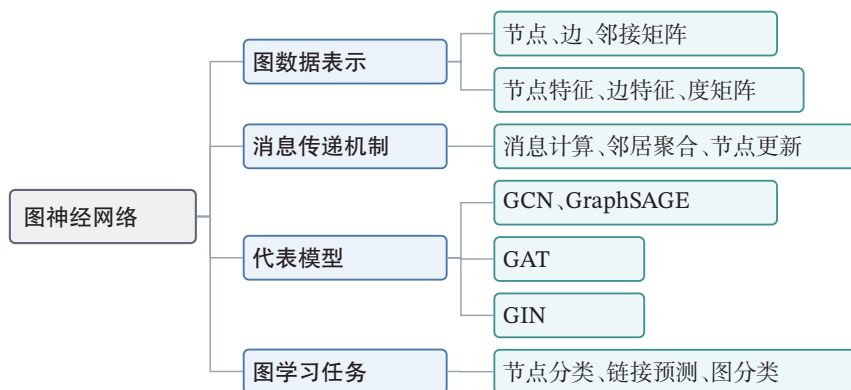


图 9.1 图神经网络关键知识点回顾

9.1 图数据的基本表示

一个图 $G = (V, E)$ 由节点集 V 与边集 E 组成. 与向量、网格、序列相比, 图的特点在于不规则: 节点数可变、邻居数不定、邻居之间没有先后顺序. 表示一张图本质上就是回答“谁连了谁”——也就是如何记录边的拓扑结构. 常见的存储方式有三种, 分别适合不同规模和算子:

- **邻接矩阵** (Adjacency Matrix) $A \in \{0, 1\}^{N \times N}$: 约定 $A_{ij} = 1$ 表示 $j \rightarrow i$, 即第 i 行收集发给节点 i 的消息. 无向图同时含两个方向, 矩阵对称. 存储开销为 $\mathcal{O}(N^2)$.
- **边列表** (Edge List): `edge_index` 的两行依次存源节点和目标节点, 因此 $j \rightarrow i$ 写成 $[j, i]$. 无向图通常每条边存两次; 后文写 $|E|$ 时指无向边数, 代码中的记录数为 $2|E|$. 存储开销与边数成正比.
- **邻接表** (Adjacency List): 每个节点维护一个邻居列表. 适合迭代式邻居聚合算法, 但难以做向量化运算.

除了拓扑结构, 每个图还可能配带:

- **节点特征矩阵** $X \in \mathbb{R}^{N \times D}$: 每个节点对应一个 D 维特征向量;
- **边特征** e_{ij} : 刻画一条边的属性, 比如分子里键的类型;
- **度矩阵** D : 对角矩阵, $D_{ii} = \sum_j A_{ij}$ 是节点 i 的度数, 许多归一化算子要用到.

9.1.1 Zachary's Karate Club: 一个“入门级”数据集

Zachary 记录了一家空手道俱乐部成员的交往关系, 以及分裂后成员加入哪一方俱乐部. NetworkX 提供的版本把这些去向存为节点的 `club` 属性. ① 本节只用节点 0 和 33 的派别训练, 其他 32 人的标签留作评价. 整张关系网在训练时可见: 这是在已知图中推断缺失标签的 **直推学习** 设定, 不是预测尚未出现的新成员.

```

1 import torch
2 import torch.nn as nn
3 import torch.nn.functional as F
4 import numpy as np
5 import matplotlib.pyplot as plt
6 import networkx as nx
7
8 torch.manual_seed(0)
9 np.random.seed(0)
10 # Karate Club 数据集 (NetworkX 内置)
```

① 数据来源、权重与编号约定见 https://networkx.org/documentation/stable/reference/generated/networkx.generators.social.karate_club_graph.html.

```

11 G = nx.karate_club_graph()
12 N = G.number_of_nodes()                # 34
13 E = G.number_of_edges()                # 78
14 print(f'节点数 N={N}, 边数 |E|={E}, 平均度数={2*E/N:.2f}')
15 # 三种存储方式 -----
16 # (1) 邻接矩阵 (weight=None 取 0/1 二值邻接: Karate Club 的边自带 1~7 的权重,
17 #   但本章节点分类只关心“是否相连”, 二值化后与下面 GCN/SAGE/GIN 的聚合口径一致)
18 A_dense = torch.tensor(nx.to_numpy_array(G, weight=None), dtype=torch.float32)
19 print('A_dense shape:', A_dense.shape, ', 占用',
20       A_dense.element_size() * A_dense.nelement(), '字节')
21 # (2) 边列表 (PyG 风格): [2, 2/E], 无向图把每条边写两次
22 edge_index = torch.tensor([u, v] for u, v in G.edges]
23               + [[v, u] for u, v in G.edges]).t()
24 print('edge_index shape:', edge_index.shape)
25 # (3) 稀疏邻接 (COO 格式)
26 A_sparse = A_dense.to_sparse_coo()
27 print('A_sparse nnz:', A_sparse._nnz())

```

输出:

```

节点数 N=34, 边数 |E|=78, 平均度数=4.59
A_dense shape: torch.Size([34, 34]), 占用 4624 字节
edge_index shape: torch.Size([2, 156])
A_sparse nnz: 156

```

二值邻接只记录是否相连, 忽略原始边权所表示的交往情境数量. 34 个节点时稠密存储很轻便; Cora 的 2708×2708 个 float32 元素约占 29.3MB, 而中间消息张量还要乘特征维数. 规模扩大时, 应同时估算图、特征和梯度占用.

9.1.2 图数据可视化

先把关系画出来, 有助于检查边与标签是否读对. 图9.2左侧按真实派别着色, 右侧为便于观察而按派别重排邻接矩阵; 训练仍使用原始节点编号. 布局距离只是绘图结果, 不等同于实际社交距离.

```

1 # 节点标签: 0=Mr. Hi 派, 1=Officer 派 (来自 NetworkX 内置 'club' 属性)
2 y = torch.tensor([0 if G.nodes[i]['club']=='Mr. Hi' else 1
3                   for i in range(N)])

```

从图中可以检查两件事:

- 同派成员之间的边较多, 这种标签与连接关系的相关性称为**同质性** (Homophily). 图中也有跨派边, 不能把“朋友同派”当作无例外规则.

- 按标签重排后更容易辨认两个较密的区域；矩阵的对称性则来自无向边约定，与节点排序无关。

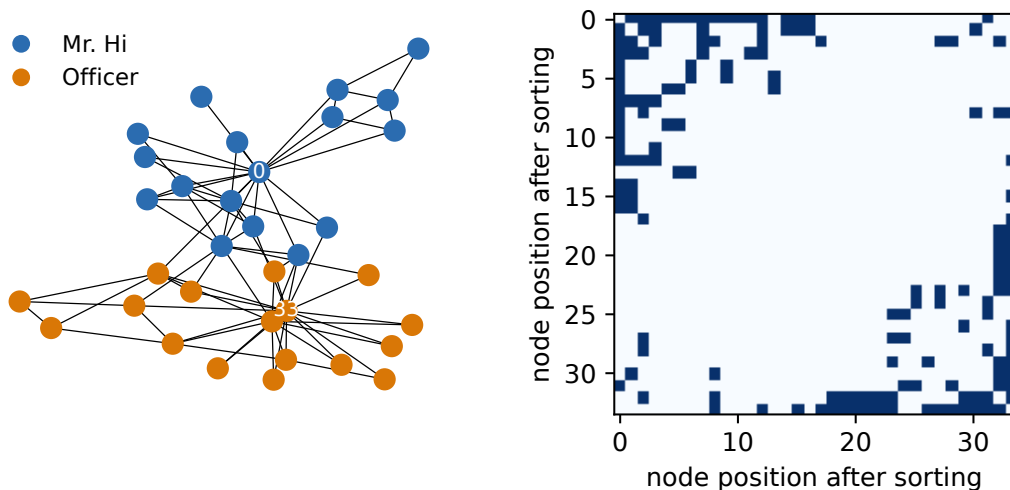


图 9.2 Karate Club 数据集可视化:图拓扑与重排后的邻接矩阵;仅标出两个训练节点

9.1.3 节点特征的选择

Karate Club 本身没有现成的节点属性，因此这里使用最简单的 **one-hot 特征**：每个节点的特征是一个 one-hot 向量，取值为 1 的位置正是它自己的编号。这相当于告诉模型“我是节点 i ”，剩下的全靠图结构本身去驱动节点表示的学习。

```

1 # 节点特征: one-hot (特征数=节点数)
2 X = torch.eye(N)
3 print('X shape:', X.shape)                # [34, 34]
4 # 节点度数 (GCN 归一化要用)
5 deg = A_dense.sum(dim=1)
6 print(f'度数分布: min={deg.min():.0f}, max={deg.max():.0f}, mean={deg.mean():.2f}')

```

笔记

真实数据集的节点特征往往是更有信息量的属性：

- **Cora**: 论文摘要的 1433 维词袋向量；
- **社交网络**: 用户画像（年龄、地区、兴趣标签等）；
- **QM9 分子数据集**: 原子的化学元素（one-hot）以及价电子数、电负性等理化属性；
- **推荐系统**: 用户/物品的 embedding 或 side information.

这里的 one-hot 输入是固定的；它乘以第一层权重，相当于取出各节点对应的可学习参数。普通 `torch.randn` 张量也不会自动成为可训练嵌入，需显式注册为 `Parameter` 或使用 `Embedding`。依赖节点编号的表示难以直接用于新节点；跨图应用通常需要可共享的属性或结构特征。



9.2 消息传递机制

看似五花八门的 GNN 其实共享一个统一框架——**消息传递**（Message Passing）：一层 GNN 分三步完成节点表示的更新：

- 1) **消息计算**（Message）：对每条边 (i, j) ，由源节点 j 算出要传给目标节点 i 的消息 $\mathbf{m}_{j \rightarrow i} = \text{Message}(\mathbf{h}_j, \mathbf{h}_i, \mathbf{e}_{ij})$ ；
- 2) **邻居聚合**（Aggregate）：节点 i 把所有邻居 $\mathcal{N}(i)$ 发来的消息合并为聚合消息 $\mathbf{m}_i$ 。常用的 `sum`、`mean`、`max` 都是**置换不变**的，与邻居顺序无关；
- 3) **节点更新**（Update）：把聚合得到的消息与节点自身的旧表示 $\mathbf{h}_i$ 组合，产出新表示 $\mathbf{h}'_i = \text{Update}(\mathbf{h}_i, \mathbf{m}_i)$ 。

整理为统一公式，即

$$\mathbf{h}'_i = \text{Update}\left(\mathbf{h}_i, \text{Aggregate}\{\text{Message}(\mathbf{h}_j, \mathbf{h}_i, \mathbf{e}_{ij}) : j \in \mathcal{N}(i)\}\right). \quad (9.1)$$

不同 GNN 之间的真正差异，落在 `Message`、`Aggregate`、`Update` 这三处函数的具体形式上。本章后续要实现的 GCN、GraphSAGE、GAT、GIN 都可以装进同一个框架——只是“消息怎么造、邻居怎么合、节点怎么更新”各有侧重。

9.2.1 抽象基类：把消息传递独立出来

先实现一个消息传递基类，用 `message`、`aggregate` 和 `update` 对应公式中的三步。后续 4 种模型会采用更紧凑的矩阵写法；这个基类用于观察信息流，与 PyG 的 `MessagePassing` 在设计上相通。

```
1 class MessagePassing(nn.Module):
2     """消息传递抽象基类.
```

```

3
4  Args:
5  aggr (str): 聚合方式, 可选 'sum' / 'mean' / 'max'.
6  """
7
8  def __init__(self, aggr='sum'):
9      super().__init__()
10     assert aggr in ('sum', 'mean', 'max')
11     self.aggr = aggr
12     # —— 三个钩子函数, 子类按需重写 ——
13     def message(self, h_src, h_dst, edge_attr=None):
14         """计算每条边从 src 流向 dst 的消息. 默认直接传 h_src. """
15         return h_src
16
17     def aggregate(self, messages, A):
18         """把 N x N x D 的边消息按 dst 聚合成 N x D 的节点消息. """
19         # messages: [N, N, D], A: [N, N]
20         if self.aggr == 'sum':
21             return torch.einsum('ij,ijd->id', A, messages)
22         if self.aggr == 'mean':
23             deg = A.sum(dim=1, keepdim=True).clamp(min=1)
24             return torch.einsum('ij,ijd->id', A, messages) / deg
25         # max
26         m = messages.clone()
27         m[A == 0] = float('-inf')
28         values = m.max(dim=1).values
29         has_neighbor = (A != 0).any(dim=1, keepdim=True)
30         return torch.where(has_neighbor, values, torch.zeros_like(values))
31
32     def update(self, h_old, h_agg):
33         """用聚合后的消息更新节点表示. 默认直接返回 h_agg. """
34         return h_agg
35     # —— 统一 forward: 子类一般不需要重写 ——
36     def forward(self, H, A, edge_attr=None):
37         # 广播成 [N, N, D] 的边消息张量
38         N, D = H.shape
39         h_src = H.unsqueeze(0).expand(N, N, D) # 沿 dst 维广播
40         h_dst = H.unsqueeze(1).expand(N, N, D) # 沿 src 维广播
41         msg = self.message(h_src, h_dst, edge_attr)
42         agg = self.aggregate(msg, A)
43         return self.update(H, agg)

```



笔记

这里显式构造 $N \times N \times D$ 的消息张量, 便于看清两个节点维度, 开销却随 N^2 增长. PyG 按实际边生成并聚合消息, 稀疏图上能减少存储. 后面的手写算子都面向二值、无向、无自环的小图; 对孤立节点, 空邻居集合的聚合结果约定为零.

9.2.2 一个示例: 用基类实现 Mean Aggregator

下面用上面的基类, 几行代码就能写一个“邻居均值”聚合器——和经典的 GraphSAGE 的 mean 变体几乎一致.

```
1 class MeanConv(MessagePassing):
2     """h_i' = W [ h_i || mean(N(i)) ]"""
3
4     def __init__(self, in_dim, out_dim, aggr='mean'):
5         super().__init__(aggr=aggr)
6         self.lin = nn.Linear(2 * in_dim, out_dim)
7
8     def update(self, h_old, h_agg):
9         return self.lin(torch.cat([h_old, h_agg], dim=1))
10
11 # 测试: 随便造一个 5 节点的小图
12 A_demo = torch.tensor([
13     [0, 1, 1, 0, 0],
14     [1, 0, 1, 0, 0],
15     [1, 1, 0, 1, 0],
16     [0, 0, 1, 0, 1],
17     [0, 0, 0, 1, 0],
18 ], dtype=torch.float32)
19 H_demo = torch.randn(5, 4)
20 out = MeanConv(4, 8)(H_demo, A_demo)
21 print(out.shape)    # torch.Size([5, 8])
```

后续不同 GNN 层的差异, 可以简洁地概括为:

- **GCN**: $\text{message} = \frac{1}{\sqrt{d_i d_j}} \mathbf{W} \mathbf{h}_j$, $\text{aggr} = \text{sum}$, $\text{update} = \text{identity}$;
- **GraphSAGE**: $\text{message} = \mathbf{h}_j$, $\text{aggr} = \text{mean}$, $\text{update} = \mathbf{W}[\mathbf{h}_i || \mathbf{m}_i]$;
- **GAT**: $\text{message} = \alpha_{ij} \mathbf{W} \mathbf{h}_j$ (注意力加权), $\text{aggr} = \text{sum}$, $\text{update} = \text{identity}$;
- **GIN**: $\text{message} = \mathbf{h}_j$, $\text{aggr} = \text{sum}$, $\text{update} = \text{MLP}((1 + \epsilon) \mathbf{h}_i + \mathbf{m}_i)$.



动手练习 9.1

- 1) 基于本节的 `MessagePassing` 基类, 把构造时的聚合方式 `aggr` 分别设为 `'sum'`、`'mean'`、`'max'`, 在同一张 5 节点小图 `A_demo` 和同一份输入 `H_demo` 上观察 `MeanConv` 输出的差异; 结合三种聚合的定义, 说明为什么 `mean` 对邻居数量不敏感、而 `sum` 和 `max` 各自“看重”邻居集合的哪一面。
- 2) 只重写 `message` 钩子 (例如让每条消息乘上一个固定标量, 或改成 `h_src - h_dst` 这种带目标节点信息的形式), 其余两个钩子保持不变, 重新前向一次, 验证消息计算、邻居聚合、节点更新这三步是否如预期相互解耦、互不影响。

9.3 GCN:图卷积网络

GCN 用一条紧凑的矩阵公式实现消息传递:

$$\mathbf{H}^{(l+1)} = \sigma\left(\tilde{\mathbf{D}}^{-\frac{1}{2}} \tilde{\mathbf{A}} \tilde{\mathbf{D}}^{-\frac{1}{2}} \mathbf{H}^{(l)} \mathbf{W}^{(l)}\right), \quad (9.2)$$

其中:

- $\tilde{\mathbf{A}} = \mathbf{A} + \mathbf{I}$: 加自环, 让每个节点也“传消息给自己”;
- $\tilde{\mathbf{D}}$: $\tilde{\mathbf{A}}$ 对应的度矩阵;
- $\tilde{\mathbf{D}}^{-1/2} \tilde{\mathbf{A}} \tilde{\mathbf{D}}^{-1/2}$: 对称归一化的邻接矩阵, 防止度数大的节点主导聚合;
- $\mathbf{W}^{(l)}$: 可学习的权重矩阵;
- σ : 非线性激活, 通常为 `ReLU`.

先将邻居与自身的特征按度数归一化求和, 再作线性变换. 下面的 `GCNLayer` 还加入偏置, 实际输出是 $\hat{\mathbf{A}}\mathbf{H}\mathbf{W} + \mathbf{1}\mathbf{b}^\top$, 激活放在模型外层. 偏置在聚合后相加; 若先加偏置再乘 $\hat{\mathbf{A}}$, 不同节点的偏置会被不同的行和缩放.

```
1 def normalize_adj(A):
2     """GCN 的对称归一化:  $\mathbf{A}_{\text{hat}} = \mathbf{D}^{-1/2} (\mathbf{A} + \mathbf{I}) \mathbf{D}^{-1/2}$ ."""
3     N = A.shape[0]
4     # 用 A.device / A.dtype 保证单位阵与 A 同设备、同精度, GPU 训练时不会爆
5     A_tilde = A + torch.eye(N, device=A.device, dtype=A.dtype)
6     D_tilde = A_tilde.sum(dim=1)
7     D_inv_sqrt = D_tilde.pow(-0.5)
8     D_inv_sqrt[torch.isinf(D_inv_sqrt)] = 0.0
9     # 将 A_tilde 的每一行、每一列分别乘以对应节点度数的负二分之一次方
10    # 等价于两侧乘以对角矩阵, 广播逐元素相乘的时间复杂度为  $O(N^2)$ 
11    return D_inv_sqrt[:, None] * A_tilde * D_inv_sqrt[None, :]
12
13 class GCNLayer(nn.Module):
```

```

14     """线性 GCN 层:  $H' = A_{\text{hat}} H W + b$ ; 激活放在外部. """
15
16     def __init__(self, in_dim, out_dim):
17         super().__init__()
18         self.lin = nn.Linear(in_dim, out_dim)
19
20     def forward(self, H, A_hat):
21         # H为节点特征矩阵, 形状为[节点数, 输入特征维度]
22         # A_hat为对称归一化后的邻接矩阵, 形状为[节点数, 节点数]
23         return self.lin(A_hat @ H) # 聚合后加偏置, 与 GCNConv 对齐
24
25 class GCN(nn.Module):
26     """两层 GCN: [N, in] -> [N, hidden] -> [N, out]."""
27
28     def __init__(self, in_dim, hidden, out_dim, dropout=0.5):
29         super().__init__()
30         self.conv1 = GCNLayer(in_dim, hidden)
31         self.conv2 = GCNLayer(hidden, out_dim)
32         self.dropout = dropout
33
34     def forward(self, H, A_hat):
35         H = F.relu(self.conv1(H, A_hat))
36         H = F.dropout(H, p=self.dropout, training=self.training)
37         return self.conv2(H, A_hat)

```

半监督节点分类 只让节点 0 和 33 的标签进入交叉熵损失, 固定训练 200 回合; 准确率仅在其余 32 个节点上计算. 为了观察学习过程, 每回合记录这 32 人的预测, 但不据此选择回合或调参. 若要系统选择配置, 还需要额外的验证标签或独立任务.

```

1  A_dense_t = A_dense # 沿用前面定义
2  A_hat = normalize_adj(A_dense_t)
3  torch.manual_seed(0)
4  model = GCN(in_dim=N, hidden=16, out_dim=2, dropout=0.3)
5  optimizer = torch.optim.Adam(model.parameters(), lr=0.05, weight_decay=5e-4)
6  # 半监督设定: 只用 2 个节点的标签
7  train_mask = torch.zeros(N, dtype=torch.bool)
8  train_mask[0] = True # Mr. Hi 派
9  train_mask[33] = True # Officer 派
10
11 eval_mask = ~train_mask # 只评价未参与监督的 32 个节点
12
13 history = {'loss': [], 'acc': []}
14 for epoch in range(200):
15     model.train(); optimizer.zero_grad()
16     logits = model(X, A_hat)

```

```

17     loss = F.cross_entropy(logits[train_mask], y[train_mask])
18     loss.backward(); optimizer.step()
19     with torch.no_grad():
20         model.eval()
21         pred = model(X, A_hat).argmax(1)
22         acc = (pred[eval_mask] == y[eval_mask]).float().mean().item()
23         history['loss'].append(loss.item()); history['acc'].append(acc)
24
25     print(f'GCN unlabeled acc: {history["acc"][-1]:.4f}')

```

本次固定 200 回合后, GCN 在 32 个未标注节点中预测正确 31 个, 准确率为 96.875%。训练曲线见图 9.3: 训练损失来自 2 个标签, 评价准确率来自另外 32 个节点, 两条曲线回答的问题不同。

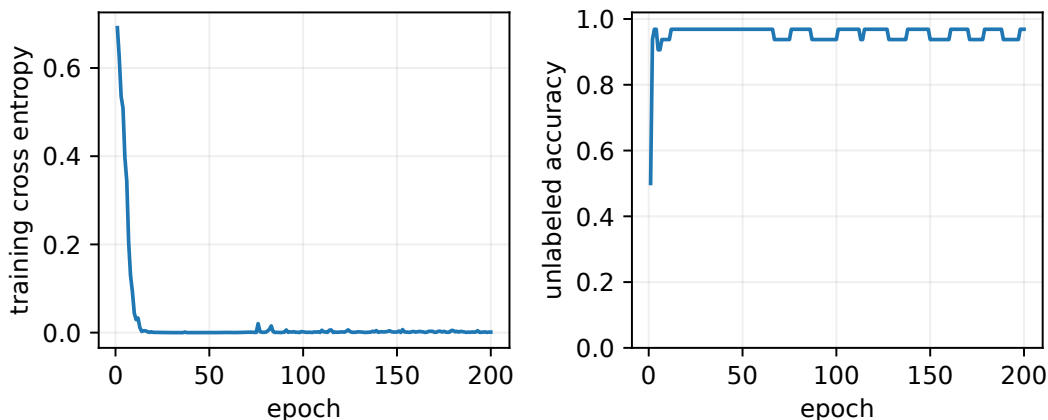


图 9.3 两层 GCN 在 Karate Club 上的训练曲线

节点嵌入的可视化 GCN 每一层都把节点投到一个新的表示空间。取倒数第二层 (hidden=16 维) 的输出, 再降到 2 维, 应该能看到同派节点聚成簇的现象:

```

1  from sklearn.manifold import TSNE
2
3  model.eval()
4  with torch.no_grad():
5      H1 = F.relu(model.conv1(X, A_hat)).numpy() # [N, 16]
6  emb = TSNE(n_components=2, perplexity=8, random_state=0).fit_transform(H1)

```

把 emb 按真实派别着色, 得到图 9.4。t-SNE 用于检查局部邻域, 不保留所有距离和全局几何关系; 分类效果仍应由预先规定的评价指标判断。

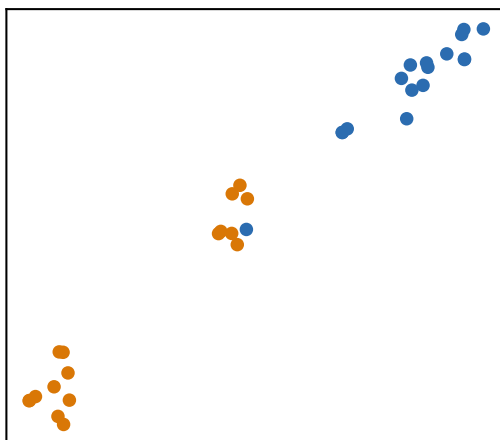


图 9.4 两层 GCN 第一层输出经 t-SNE 可视化

同标签节点出现聚集说明表示中含有与派别相关的信息,但图形会随降维参数和随机种子变化.不能仅凭“两团分开”断言模型已学会普遍的社区规则.

9.4 GraphSAGE: 邻居采样 + 聚合

前面的稠密实现需要整张邻接矩阵;这是实现方式的限制,GCN 本身也可以使用稀疏运算和采样.接下来把更新写成中心节点与邻居特征的组合,便于说明局部计算.

GraphSAGE (Sample and AggreGatE) 学习共享的邻居聚合函数,并用邻居采样控制每批计算量.^②有可共享的节点属性时,这个函数也能作用于训练中没出现的节点或图.下面先使用全部邻居;单层更新为

$$\mathbf{h}_i^{(l+1)} = \sigma(\mathbf{W}^{(l)} \cdot \text{CONCAT}(\mathbf{h}_i^{(l)}, \text{AGG}\{\mathbf{h}_j^{(l)} : j \in \mathcal{N}(i)\})), \quad (9.3)$$

本节实现 mean 聚合,不包含邻居采样和逐层 L2 规范化.求和、均值和最大值不依赖邻居顺序;若改用 LSTM 聚合,还要处理其顺序敏感性,不能直接沿用置换不变的结论.

```
1 class SAGELayer(nn.Module):
2     """GraphSAGE 一层 (mean aggregator): concat[h_i, mean(N(i))] -> linear."""
3
4     def __init__(self, in_dim, out_dim):
5         super().__init__()
6         self.lin = nn.Linear(2 * in_dim, out_dim)
7
```

^② GraphSAGE 原论文: <https://arxiv.org/abs/1706.02216>.

```

8     def forward(self, H, A):
9         # H: [N, in_dim]; A: [N, N] (0/1 邻接矩阵, 不需要自环)
10        deg = A.sum(dim=1, keepdim=True).clamp(min=1)
11        nbr_mean = (A @ H) / deg # 邻居均值
12        return self.lin(torch.cat([H, nbr_mean], dim=1))

```



笔记

大图可用NeighborLoader为一批目标节点抽取计算子图,每层限制采样数.深度为 L 、每层采 K 个邻居时,依赖节点数仍可能随 K^L 增长;采样改善预算控制,并不意味着可以忽略图存储和多跳展开成本。

9.5 GAT:图注意力网络

GCN中的邻居系数由度数决定.若不同邻居的特征对当前预测有不同作用,可以让系数随输入变化;上一章的注意力机制正好提供了这种写法.

GAT在邻居与自身之间计算归一化权重,再加权聚合.权重表示本次算子的分配,不是某段社交关系重要性的直接证明:

$$\alpha_{ij} = \text{softmax}_{j \in \mathcal{N}(i) \cup \{i\}} \left(\text{LeakyReLU}(\mathbf{a}^\top [\mathbf{W}\mathbf{h}_i \parallel \mathbf{W}\mathbf{h}_j]) \right), \quad (9.4)$$

$$\mathbf{h}'_i = \sigma \left(\sum_{j \in \mathcal{N}(i) \cup \{i\}} \alpha_{ij} \mathbf{W}\mathbf{h}_j \right). \quad (9.5)$$

实际使用中通常采用**多头注意力**:跑 K 套独立的 $(\mathbf{W}, \mathbf{a})$ 再把输出concat起来.下面给出单头版本:

```

1 class GATLayer(nn.Module):
2     """单头 GAT 层. 多头通过堆多个实例 + concat 实现. """
3
4     def __init__(self, in_dim, out_dim):
5         super().__init__()
6         self.W = nn.Linear(in_dim, out_dim, bias=False)
7         self.a = nn.Parameter(torch.randn(2 * out_dim) * 0.1)
8         self.leaky = nn.LeakyReLU(0.2)
9
10    def forward(self, H, A):
11        N = H.shape[0]
12        H_w = self.W(H) # 变换后的节点特征, 形状为 [节点数, 输出特征维度]
13        # 计算所有节点对的未归一化注意力分数
14        # 将拼接后的向量与参数向量的点积分解为两项, 再施加非线性激活
15        a_target, a_source = self.a[:H_w.shape[1]], self.a[H_w.shape[1]:]

```

```

16     E = (H_w @ a_target).unsqueeze(1) + (H_w @ a_source).unsqueeze(0)
17     E = self.leaky(E)
18     # 屏蔽非邻居位置, 仅在邻居节点和自身之间归一化注意力分数
19     A_self = A + torch.eye(N, device=A.device, dtype=A.dtype)
20     E = E.masked_fill(A_self == 0, float('-inf'))
21     alpha = F.softmax(E, dim=1)
22     return alpha @ H_w                                     # 加权聚合

```

9.6 GIN:图同构网络

关于 GNN 的表达能力, 理论分析关注它能否区分不同图结构. 在节点特征域等适当条件下, 邻居聚合、节点更新与图级读出若均保留单射性质, 模型可以达到 1-WL 图同构测试的区分能力.^③这是表示能力的结论, 不保证某次训练收敛, 也不保证测试准确率高于其他网络.

GIN 用一组极为简洁的设计实现这一点:

$$\mathbf{h}_i^{(l+1)} = \text{MLP}^{(l)}\left((1 + \epsilon) \cdot \mathbf{h}_i^{(l)} + \sum_{j \in \mathcal{N}(i)} \mathbf{h}_j^{(l)}\right). \quad (9.6)$$

设计要点:

- 使用`sum`聚合而非 `mean/max`——在适当的特征映射下, 求和能构造对邻居多重集的单射表示, 并保留邻居数量信息; 对任意原始向量直接求和并不自动具有单射性;
- 使用`MLP`而非单层 `linear`——更灵活的非线性映射有助于保留聚合后的区分信息, 但训练得到的映射并不自动满足单射条件;
- ϵ 既可固定也可设为可学习参数, 用来突出中心节点自身的贡献.

```

1  class GINLayer(nn.Module):
2      def __init__(self, in_dim, out_dim, eps=0.0, train_eps=False):
3          super().__init__()
4          self.eps = nn.Parameter(torch.tensor(eps)) if train_eps else eps
5          self.mlp = nn.Sequential(
6              nn.Linear(in_dim, out_dim),
7              nn.ReLU(),
8              nn.Linear(out_dim, out_dim),
9          )
10
11     def forward(self, H, A):
12         out = (1 + self.eps) * H + A @ H # 邻居 sum + (1+eps) * self
13         return self.mlp(out)

```

^③ 条件与反例见 GIN 原论文: <https://arxiv.org/abs/1810.00826>.

9.7 四种 GNN 在 Karate Club 上的对比

统一 200 回合、Adam、学习率 0.05、权重衰减 5×10^{-4} 、隐藏维数 16 与暂退率 0.3, 比较 4 种聚合方式. GAT 使用单头, 正文与 Notebook 一致. 不同结构的参数个数和优化难度仍有差异, 因此这个小实验只比较指定预算下的结果.

```

1 def train_gnn(model, A_input, n_epoch=200, lr=0.05, wd=5e-4, seed=0):
2     """通用训练循环: 返回每个 epoch 的 (loss, acc). """
3     torch.manual_seed(seed)
4     opt = torch.optim.Adam(model.parameters(), lr=lr, weight_decay=wd)
5     hist = {'loss': [], 'acc': []}
6     for _ in range(n_epoch):
7         model.train(); opt.zero_grad()
8         logits = model(X, A_input)
9         loss = F.cross_entropy(logits[train_mask], y[train_mask])
10        loss.backward(); opt.step()
11        with torch.no_grad():
12            model.eval()
13            pred = model(X, A_input).argmax(1)
14            acc = (pred[eval_mask] == y[eval_mask]).float().mean().item()
15            hist['loss'].append(loss.item()); hist['acc'].append(acc)
16    return hist
17 # ——— 4 个模型 ———
18 class SAGE(nn.Module):
19     def __init__(self, in_dim, hidden, out_dim, dropout=0.3):
20         super().__init__()
21         self.conv1 = SAGELayer(in_dim, hidden)
22         self.conv2 = SAGELayer(hidden, out_dim)
23         self.dropout = dropout
24     def forward(self, H, A):
25         H = F.relu(self.conv1(H, A))
26         H = F.dropout(H, self.dropout, training=self.training)
27         return self.conv2(H, A)
28
29 class GAT(nn.Module):
30     def __init__(self, in_dim, hidden, out_dim, dropout=0.3):
31         super().__init__()
32         self.conv1 = GATLayer(in_dim, hidden)
33         self.conv2 = GATLayer(hidden, out_dim)
34         self.dropout = dropout
35     def forward(self, H, A):
36         H = F.elu(self.conv1(H, A))
37         H = F.dropout(H, self.dropout, training=self.training)
38         return self.conv2(H, A)

```

```

39
40 class GIN(nn.Module):
41     def __init__(self, in_dim, hidden, out_dim, dropout=0.3):
42         super().__init__()
43         self.conv1 = GINLayer(in_dim, hidden)
44         self.conv2 = GINLayer(hidden, out_dim)
45         self.dropout = dropout
46     def forward(self, H, A):
47         H = F.relu(self.conv1(H, A))
48         H = F.dropout(H, self.dropout, training=self.training)
49         return self.conv2(H, A)
50 # 种子在构造前设置; 每次固定训练 200 回合, 不按评价曲线选择回合.
51 results, seed_scores = {}, {name: [] for name in ['GCN', 'SAGE', 'GAT', 'GIN']}
52 model_classes = {'GCN': GCN, 'SAGE': SAGE, 'GAT': GAT, 'GIN': GIN}
53 for seed in range(5):
54     for name, cls in model_classes.items():
55         torch.manual_seed(seed)
56         m = cls(N, 16, 2, dropout=0.3)
57         h = train_gnn(m, A_hat if name == 'GCN' else A_dense_t, seed=seed)
58         seed_scores[name].append(h['acc'][-1])
59         if seed == 0:
60             results[name] = h
61 for name, scores in seed_scores.items():
62     print(f'{name:5s} seed0={scores[0]:.4f} '
63           f'mean={np.mean(scores):.4f} std={np.std(scores, ddof=1):.4f}')

```

输出 (种子 0 至 4; 标准差按样本标准差计算):

```

GCN  seed0=0.9688 mean=0.9688 std=0.0000
SAGE seed0=0.8750 mean=0.8500 std=0.0677
GAT  seed0=0.9375 mean=0.9000 std=0.0342
GIN  seed0=0.9062 mean=0.8000 std=0.1857

```

图9.5画出种子0的训练损失与32个未标注节点上的准确率. 训练误差较小并不意味着对其他节点预测得更准; 评价曲线中的波动正是需要重复种子的原因.

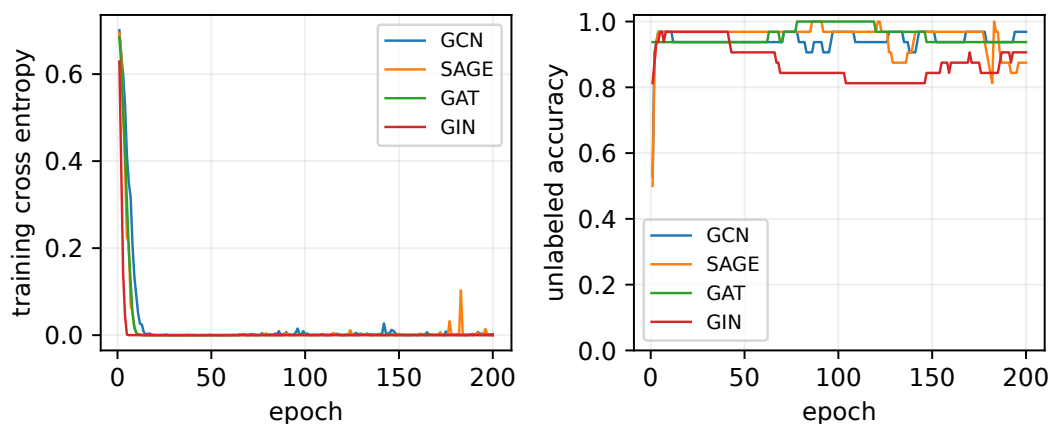


图 9.5 4 种 GNN 在 Karate Club 上的训练曲线对比

表 9.1 汇总种子 0 至 4 的最终结果. 只改变初始化与暂退随机性, 仍使用同一张图和同两个训练标签, 因而没有覆盖换图、换标签划分的不确定性.

表 9.1 4 种 GNN 在 Karate Club 数据集上的对比 (5 个种子的均值和样本标准差)

模型	平均准确率	标准差	特点
GCN	0.9688	0.0000	度数归一化加权
GraphSAGE	0.8500	0.0677	邻居均值与自身拼接
GAT	0.9000	0.0342	单头注意力加权
GIN	0.8000	0.1857	邻居求和与 MLP

笔记



每个未标注节点占准确率的 $1/32$, 即 3.125 个百分点. 在这样的小样本上, 几次预测变化就能改变排序. 后面的 Cora 实验增加节点数和标签数, 但仍要保留划分与预算, 不能把一个数据集上的排名推广到所有图.

动手练习 9.2



1) 在 Karate Club 上将训练标签数从 2 个增加到 10 个, 观察 GCN 准确率的变化曲线.

9.8 过度平滑与深层训练

堆叠消息传递可以访问更远的邻居，但层数增加也会让信息反复混合。过度平滑（Over-smoothing）指节点表示逐渐难以区分的现象。它是深层图网络需要检查的原因之一，还应同时排查优化失败、过拟合和信息传播瓶颈。

可以先考察省略权重变换和非线性后的传播 $\mathbf{H}^{(l+1)} = \hat{\mathbf{A}}\mathbf{H}^{(l)}$ 。在连通且加入自环的无向图上，反复传播会逐渐保留 $\hat{\mathbf{A}}$ 主特征向量对应的分量，节点间的差异因而减少。对这里的对称归一化，极限表示还受节点度数缩放，并不意味着所有节点向量数值完全相同。完整 GCN 还包含可学习变换和非线性，这个简化分析用于解释过度平滑的趋势。

9.8.1 实验：把 GCN 堆到 2 层、4 层、6 层、8 层

我们重新写一个支持任意层数的 GCN，跑一组对比：

```

1 class DeepGCN(nn.Module):
2     """L 层 GCN. """
3
4     def __init__(self, in_dim, hidden, out_dim, n_layers=2, dropout=0.5):
5         super().__init__()
6         dims = [in_dim] + [hidden]*(n_layers-1) + [out_dim]
7         self.layers = nn.ModuleList([GCNLayer(dims[i], dims[i+1])
8                                     for i in range(n_layers)])
9         self.dropout = dropout
10
11     def forward(self, H, A_hat):
12         for i, layer in enumerate(self.layers):
13             H = layer(H, A_hat)
14             if i < len(self.layers) - 1:
15                 H = F.relu(H)
16                 H = F.dropout(H, p=self.dropout, training=self.training)
17         return H
18
19     def node_similarity(H):
20         """节点表示两两余弦相似度的均值——衡量过度平滑的指标. """
21         H = F.normalize(H, dim=1)
22         n = H.shape[0]
23         pair_mask = ~torch.eye(n, dtype=torch.bool, device=H.device)
24         return (H @ H.t())[pair_mask].mean().item()
25
26 depths = [2, 4, 6, 8]
27 res = {}
28 for L in depths:
29     torch.manual_seed(0)

```

```

30 m = DeepGCN(N, 16, 2, n_layers=L, dropout=0.3)
31 h = train_gnn(m, A_hat, n_epoch=200)
32 m.eval()
33 with torch.no_grad():
34     # 取倒数第二层 (hidden 维) 做相似度统计
35     H_hidden = X
36     for layer in m.layers[:-1]:
37         H_hidden = F.relu(layer(H_hidden, A_hat))
38     res[L] = {'acc': h['acc'][-1], 'sim': node_similarity(H_hidden)}
39     print(f'L={L} acc={res[L]["acc"]:.4f} cosine={res[L]["sim"]:.4f}')

```

输出:

```

L=2 acc=0.9688 cosine=0.7335
L=4 acc=1.0000 cosine=0.6288
L=6 acc=0.9688 cosine=0.4934
L=8 acc=0.5000 cosine=0.7610

```

对照图9.6时,应分开看准确率与非对角节点对的余弦相似度. 高余弦说明方向接近,不代表向量完全相同;若含零向量, `F.normalize` 的零向量约定也会影响均值. 本次 8 层模型的准确率下降,但其相似度并非随深度单调上升. 仅靠这组训练后结果,不能把性能下降全部归因于过度平滑. 可以另做一个不训练的控制实验:令 $\mathbf{H} = \mathbf{I}$, 反复计算 $\mathbf{H} \leftarrow \hat{\mathbf{A}}\mathbf{H}$, 观察去掉度数缩放后的行差异是否衰减;配套 Notebook 给出了这一检查.

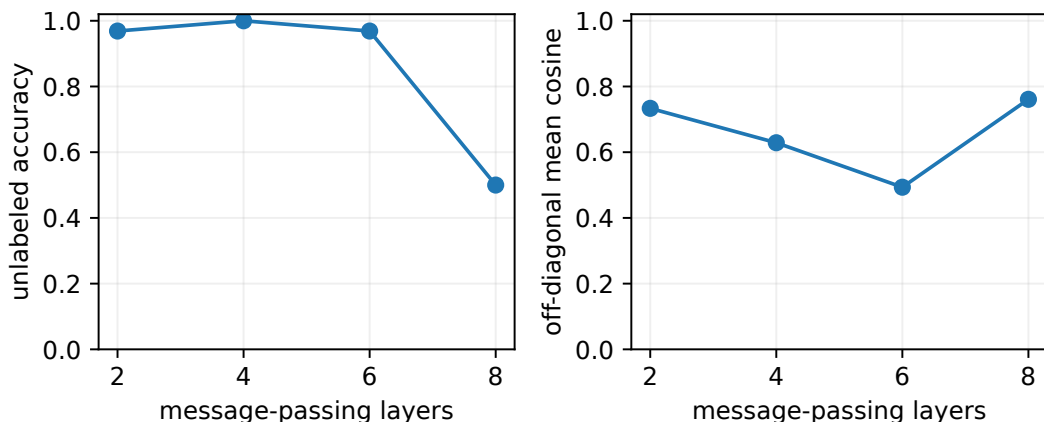


图 9.6 GCN 层数对准确率与节点表示余弦相似度的影响

9.8.2 缓解过度平滑的三种思路

过度平滑是图卷积“扩散”属性带来的代价,常见的缓解方法有 3 类:

- **残差连接 (ResNet 思路)**: 在维度相容时加入 $H = H + \text{GCNLayer}(H, A_{\text{hat}})$, 为浅层信息和梯度提供直接通路, 例如 ResGCN; JK-Net 则通过汇合不同层的表示利用多尺度信息. 这些方法可以缓解深层训练困难, 但不保证深层模型一定优于浅层模型;
- **表示归一化**: PairNorm、NodeNorm 在每层后把节点表示重新拉开距离, 防止收敛到同一点;
- **图稀疏化**: DropEdge 随机丢弃一部分边, 相当于对邻居集合做随机扰动, 降低“扩散”的速度.

```

1 class ResGCN(nn.Module):
2     """带残差连接的 L 层 GCN. """
3
4     def __init__(self, in_dim, hidden, out_dim, n_layers=6, dropout=0.3):
5         super().__init__()
6         self.in_lin = nn.Linear(in_dim, hidden)
7         self.layers = nn.ModuleList([GCNLayer(hidden, hidden)
8                                     for _ in range(n_layers)])
9         self.out_lin = nn.Linear(hidden, out_dim)
10        self.dropout = dropout
11
12    def forward(self, H, A_hat):
13        H = F.relu(self.in_lin(H))
14        for layer in self.layers:
15            H = F.relu(layer(H, A_hat)) + H          # 残差
16            H = F.dropout(H, p=self.dropout, training=self.training)
17        return self.out_lin(H)
18
19 torch.manual_seed(0)
20 h_res = train_gnn(ResGCN(N, 16, 2, n_layers=6), A_hat)
21 print(f'ResGCN-6L acc={h_res["acc"][-1]:.4f}') # 6 次消息传递, 另有输入和输出线性层

```

提醒



GNN 适合的深度取决于图结构、任务和模型设计. 每增加一层局部消息传递, 就多覆盖一跳邻居; Karate Club 图的直径为 5, 少量层数已能覆盖许多节点. 继续增加层数是否有益, 应结合远距离依赖、过度平滑和实际验证结果判断, 不能把某个小图上的最优层数当作通用上限.

动手练习 9.3



1) 用 GAT 替代 GCN, 固定训练预算并重复多个种子, 比较准确率和表示相似度. 注意力是否在当前任务上缓解了深层退化? 请根据结果回答, 不预设某个模型更鲁棒.

9.9 链接预测:让模型“补全”图

到此为止,本章所有任务都属于**节点分类**:给定整张图,预测每个节点的标签. GNN 还有另一类同样经典的任务——**链接预测** (link prediction):基于图的一部分已知边,预测两个节点之间**是否本就连着**(或将来是否会形成)一条边.这类任务在现实中俯拾皆是:

- 社交网络的“可能认识的人”;
- 知识图谱补全:“爱因斯坦-出生地-?”;
- 推荐系统的“user-item”二部图;
- 蛋白质相互作用网络补全.

9.9.1 思路:编码器 + 解码器

链接预测的常见范式是“编码器-解码器”(encoder-decoder):

$$\mathbf{z}_i = \text{GNN}(\mathbf{X}, \mathbf{A})_i, \quad (\text{encoder}) \quad (9.7)$$

$$\hat{p}_{ij} = \text{sigmoid}(\mathbf{z}_i^\top \mathbf{z}_j). \quad (\text{decoder}) \quad (9.8)$$

编码器生成节点向量,内积解码器为节点对评分,再通过 Sigmoid 映射到 0 至 1. 内积对称,适合这里的无向关系;有向边或不同关系类型需要相应的解码器. 负采样改变了类别比例,因此这里的 Sigmoid 分数不能直接当作真实世界中未来连边的概率.

9.9.2 负采样:从“缺边”里造训练样本

在本章的封闭数据设定中,完整图里没有记录的节点对视为负例;训练时每回合抽取与正例等量的非边. 这一比例控制训练预算. 实际网络中的“未观测”未必等于“不存在”,时间预测还要按时间构造可见图与候选集合.

```

1 def sample_neg_edges(A, n_neg, exclude=None, rng=None):
2     """从完整图的非边中无放回采样, 每条无向边只保留 i < j. """
3     N = A.shape[0]
4     blocked = A.bool() | A.t().bool()
5     if exclude is not None:
6         blocked = blocked | exclude.bool() | exclude.t().bool()
7     candidates = torch.triu(~blocked, diagonal=1).nonzero()
8     if not 0 <= n_neg <= candidates.shape[0]:
9         raise ValueError('可用非边数量不足, 或采样数量为负')
10    rng = np.random.default_rng() if rng is None else rng
11    chosen = rng.choice(candidates.shape[0], n_neg, replace=False)
12    chosen = torch.as_tensor(chosen, device=A.device)
13    return candidates[chosen].t().contiguous() # [2, n_neg]

```

9.9.3 切分训练、验证与测试边

从 78 条无向边中留出 15 条验证、15 条测试正边, 剩余 48 条构成编码器可见的训练图; 删除时两个方向都要移除. 验证与测试负边一次抽定、互不重叠, 并从训练负样本候选中排除.

```

1 def split_edges(A, val_ratio=0.2, test_ratio=0.2, seed=0):
2     """无向二值无自环图: 训练图不含验证、测试正边的任何方向. """
3     if val_ratio <= 0 or test_ratio <= 0 or val_ratio + test_ratio >= 1:
4         raise ValueError('验证与测试比例须为正, 且总和小于 1')
5     edges = torch.triu(A != 0, diagonal=1).nonzero()
6     rng = np.random.default_rng(seed)
7     edges = edges[torch.as_tensor(rng.permutation(len(edges)), device=A.device)]
8     n_val, n_test = int(len(edges) * val_ratio), int(len(edges) * test_ratio)
9     if min(n_val, n_test) == 0:
10        raise ValueError('图太小, 验证或测试边集为空')
11    val_pos, test_pos = edges[:n_val].t(), edges[n_val:n_val+n_test].t()
12    train_pos = edges[n_val+n_test:].t()
13    A_train = torch.zeros_like(A)
14    A_train[train_pos[0], train_pos[1]] = 1
15    A_train[train_pos[1], train_pos[0]] = 1
16    return A_train, train_pos, val_pos, test_pos
17
18 A_train, train_pos, val_pos, test_pos = split_edges(A_dense_t, seed=0)
19 print('训练/验证/测试正边数: ', train_pos.shape[1], val_pos.shape[1], test_pos.shape[1])

```

9.9.4 完整的 GCN 链接预测模型

```

1 class GCNEncoder(nn.Module):
2     """GCN encoder: 输出每个节点的 d 维 embedding. """
3
4     def __init__(self, in_dim, hidden, out_dim):
5         super().__init__()
6         self.conv1 = GCNLayer(in_dim, hidden)
7         self.conv2 = GCNLayer(hidden, out_dim)
8
9     def forward(self, H, A_hat):
10        H = F.relu(self.conv1(H, A_hat))
11        return self.conv2(H, A_hat)
12
13    def decode(Z, edge_index):
14        """对一组边 (i, j) 做内积评分. """
15        return (Z[edge_index[0]] * Z[edge_index[1]]).sum(dim=1)
16    from sklearn.metrics import roc_auc_score, average_precision_score, roc_curve
17

```

```

18 torch.manual_seed(0)
19 edge_rng = np.random.default_rng(1)
20 A_hat_train = normalize_adj(A_train)
21 # 一次抽出互不重叠的验证、测试非边，再从训练候选中排除。
22 held_neg = sample_neg_edges(A_dense_t, val_pos.shape[1] + test_pos.shape[1], rng=edge_rng)
23 val_neg, test_neg = held_neg[:, :val_pos.shape[1]], held_neg[:, val_pos.shape[1]:]
24 exclude_train = A_dense_t.bool().clone()
25 exclude_train[held_neg[0], held_neg[1]] = True
26 exclude_train[held_neg[1], held_neg[0]] = True
27 encoder = GCNEncoder(N, 32, 16)
28 opt = torch.optim.Adam(encoder.parameters(), lr=0.01, weight_decay=5e-4)
29
30 def edge_scores(positive, negative):
31     encoder.eval()
32     with torch.no_grad():
33         Z = encoder(X, A_hat_train)
34         pos = decode(Z, positive).cpu().numpy()
35         neg = decode(Z, negative).cpu().numpy()
36     return np.r_[np.ones(len(pos)), np.zeros(len(neg))], np.r_[pos, neg]
37
38 train_loss_hist, val_auc_hist = [], []
39 best_val_auc, best_state, best_epoch = -float('inf'), None, 0
40 for epoch in range(1, 201):
41     encoder.train(); opt.zero_grad()
42     Z = encoder(X, A_hat_train)
43     pos_score = decode(Z, train_pos)
44     neg_idx = sample_neg_edges(A_dense_t, train_pos.shape[1],
45                               exclude=exclude_train, rng=edge_rng)
46     neg_score = decode(Z, neg_idx)
47     logits = torch.cat([pos_score, neg_score])
48     target = torch.cat([torch.ones_like(pos_score), torch.zeros_like(neg_score)])
49     loss = F.binary_cross_entropy_with_logits(logits, target)
50     loss.backward(); opt.step()
51     train_loss_hist.append(loss.item())
52     val_y, val_scores = edge_scores(val_pos, val_neg)
53     val_auc = roc_auc_score(val_y, val_scores)
54     val_auc_hist.append(val_auc)
55     if val_auc > best_val_auc:
56         best_val_auc, best_epoch = val_auc, epoch
57         best_state = {k: v.detach().clone() for k, v in encoder.state_dict().items()}
58     if epoch % 50 == 0:
59         print(f'epoch {epoch:3d} loss={loss.item():.4f} val_AUC={val_auc:.4f}')
60     encoder.load_state_dict(best_state)
61     labels, scores = edge_scores(test_pos, test_neg)
62     auc = roc_auc_score(labels, scores)

```

```
63 ap = average_precision_score(labels, scores)
64 print(f'best_epoch={best_epoch}, test_AUC={auc:.4f}, test_AP={ap:.4f}')
```

输出:

```
epoch 50 loss=0.4513 val_AUC=0.7111
epoch 100 loss=0.3967 val_AUC=0.6622
epoch 150 loss=0.4195 val_AUC=0.7289
epoch 200 loss=0.4219 val_AUC=0.6933
best_epoch=118, test_AUC=0.7733, test_AP=0.7858
```

9.9.5 评估:AUC与平均精确率

稀疏图的全节点对评价容易被大量非边支配. 本次在固定的 15 条正边与 15 条负边上报告:

- **AUC** (ROC 曲线下面积): 衡量正负样本之间的“可分性”;
- **平均精确率 (AP)**: 按分数排序, 综合不同召回位置的精确率. 它受候选集合中的正例比例影响, 比较时必须使用相同构造.

把训练损失和 ROC 曲线一起画出来, 结果见图 9.7.

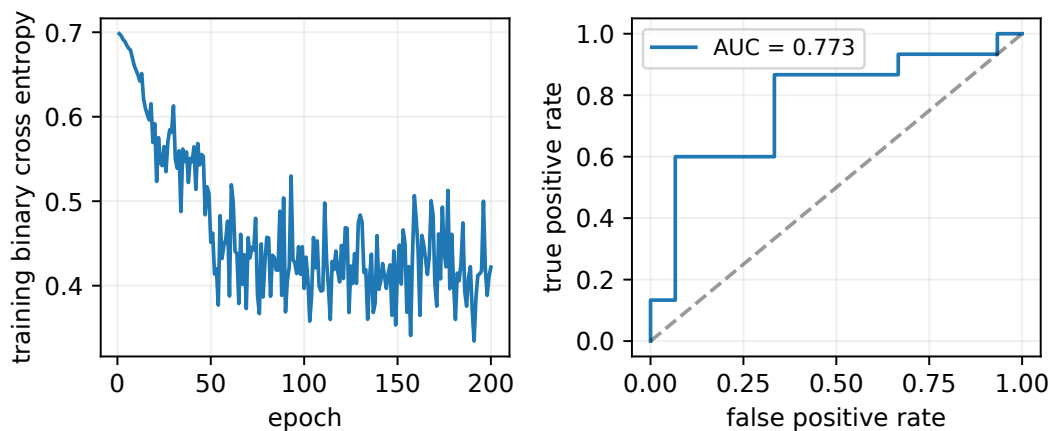


图 9.7 链接预测的训练损失与 ROC 曲线

验证 AUC 首次达到最佳值的回合是 118; 加载该快照后, 15 条测试正边与 15 条固定负边上的 AUC 为 0.7733, AP 为 0.7858. 每个样本的影响都较大, 需更换划分和种子才能评估稳定性. 若改用 Hits@K, 还须固定候选负边数量和并列排名规则.



动手练习 9.4

在配套 notebook 中画出链接预测的训练损失曲线和 ROC 曲线, 结果可与图 9.7 对照。



笔记

链接预测的 encoder 和 decoder 是解耦的, 可以自由组合:

- **encoder**: GCN、GraphSAGE、GAT 等任何节点表示模型;
- **decoder**: 内积是最简单的; 更复杂的有 DistMult、ComplEx (知识图谱) 和双线性层 $\mathbf{z}_i^T \mathbf{W} \mathbf{z}_j$.

VGAE 使用图卷积编码器与内积解码器, 可作为进一步阅读的起点。^a 关系型知识图谱还需把关系类型加入评分, 不能直接复用本节的无向内积假设。

^a <https://arxiv.org/abs/1611.07308>.

9.10 图级任务:随机图回归

GNN 的一个重要应用是图级任务——给一个完整的图, 预测一个标量或类别。典型场景是分子性质预测: 给分子结构图, 预测能量、毒性、溶解度等。

图级任务相比节点级任务多了一步读出 (Readout): 把所有节点表示聚合成一个图级表示。

$$\mathbf{h}_G = \text{READOUT}(\{\mathbf{h}_i^{(L)} : i \in V\}). \quad (9.9)$$

READOUT 可以是 sum、mean、max 或者更复杂的 Set2Set。

先用可控的随机图检查读出和训练流程。每图 5 至 10 个节点, 每个无向节点对以 0.4 的概率连边, 节点特征为 3 维标准正态向量。目标定义为 $|E| + 0.3 \sum_{i,d} X_{id}$, 随图规模和特征总量变化; 它是人为构造的数值, 不是分子的物理性质。训练、验证和测试分别独立生成 200、50 和 50 张图。

```
1 class GraphGIN(nn.Module):
2     """图级 GIN: 多层消息传递 + 全图 readout (拼接每层 sum readout). """
3
4     def __init__(self, in_dim, hidden, n_layers=3, dropout=0.0):
5         super().__init__()
6         self.convs = nn.ModuleList()
7         for i in range(n_layers):
8             d_in = in_dim if i == 0 else hidden
9             self.convs.append(GINLayer(d_in, hidden))
10        # 拼接每层 readout 后回归
11        self.lin = nn.Linear(hidden * n_layers, 1)
```

```

12         self.dropout = dropout
13
14     def forward(self, H, A):
15         outs = []
16         for conv in self.convs:
17             H = F.relu(conv(H, A))
18             outs.append(H.sum(dim=0))          # sum readout
19         h_graph = torch.cat(outs, dim=0)       # 多层 readout 拼接
20         return self.lin(F.dropout(h_graph, p=self.dropout, training=self.training)).squeeze()
21
22     def make_graphs(n_graphs, seed):
23         gen = torch.Generator().manual_seed(seed)
24         graphs = []
25         for _ in range(n_graphs):
26             n = int(torch.randint(5, 11, (1,), generator=gen))
27             upper = torch.triu((torch.rand(n, n, generator=gen) < 0.4).float(), diagonal=1)
28             adjacency = upper + upper.t() # 每个无向节点对只采一次
29             features = torch.randn(n, 3, generator=gen)
30             target = adjacency.sum() / 2 + 0.3 * features.sum()
31             graphs.append((features, adjacency, target))
32         return graphs
33
34     toy_train, toy_val, toy_test = [make_graphs(n, seed) for n, seed in [(200, 10), (50, 11),
35                                     (50, 12)]]
36
37     torch.manual_seed(0)
38     graph_model = GraphGIN(3, 16)
39     graph_opt = torch.optim.Adam(graph_model.parameters(), lr=0.001)
40     order_gen = torch.Generator().manual_seed(0)
41
42     @torch.no_grad()
43     def graph_mse(graphs):
44         graph_model.eval()
45         return np.mean([(graph_model(x, a) - y).square().item() for x, a, y in graphs])
46
47     toy_history = {'train_loss': [], 'val_loss': []}
48     best_loss, toy_best = float('inf'), None
49     for epoch in range(50):
50         graph_model.train(); losses = []
51         for i in torch.randperm(len(toy_train), generator=order_gen):
52             xg, ag, yg = toy_train[int(i)]
53             graph_opt.zero_grad()
54             loss = (graph_model(xg, ag) - yg).square()
55             loss.backward(); graph_opt.step()
56             losses.append(loss.item())

```

```

55     val_loss = graph_mse(toy_val)
56     toy_history['train_loss'].append(np.mean(losses))
57     toy_history['val_loss'].append(val_loss)
58     if val_loss < best_loss:
59         best_loss = val_loss
60         toy_best = {k: v.detach().clone() for k, v in graph_model.state_dict().items()}
61     graph_model.load_state_dict(toy_best)
62     toy_test_mse = graph_mse(toy_test)
63     train_target_mean = np.mean([y.item() for _, _, y in toy_train])
64     toy_baseline_mse = np.mean([(y.item() - train_target_mean)**2 for _, _, y in toy_test])
65     print(f'GraphGIN test MSE={toy_test_mse:.4f}, constant baseline={toy_baseline_mse:.4f}')

```

本次 50 回合、按验证 MSE 选定模型后, 测试 MSE 为 0.3014; 恒预测训练目标均值的基线为 37.0646. 这说明模型学到了本数据生成规则中的关系, 不能据此推断真实分子性质预测能力.

提醒

图级任务有几个关键点:

- 读出与目标匹配: **sum** 保留总量信息, **mean** 突出平均性质; 本节目标有随图规模增长的分量, 采用 **sum**, 并与恒预测训练目标均值的基线比较.
- 多层 **readout** 拼接: 不同深度的层捕捉不同尺度的子结构信息, GIN 原文推荐这种做法;
- 按图组织训练: 教学小图可以逐图处理; 图批处理则能减少调度开销. 无论哪种方式, 统计图级平均损失时都应按图数加权.



动手练习 9.5

- 1) 在本节的随机图回归任务上, 把 GraphGIN 的读出 (**readout**) 从 **sum** 分别换成 **mean** 和 **max** (即把 `H.sum(dim=0)` 改成 `H.mean(dim=0)` 或 `H.max(dim=0).values`), 其余设置不变, 比较三种读出在验证集上的 MSE, 再对选定方案作测试.
- 2) 再把“多层读出拼接”去掉, 只用最后一层的图级表示做回归 (相应调整 `self.lin` 的输入维度), 观察验证 MSE 的变化.
- 3) 将目标改成 $|E|/|V| + 0.3 \sum_{i,d} X_{id}/|V|$, 重新比较读出方式. 目标从总量变为平均量后, 原来的结论是否仍成立?



9.11 PyTorch Geometric:从“学具”到“工程”

前面的稠密张量让每个索引都能看清. 处理更大的稀疏图时, **PyTorch Geometric** (PyG)^④ 提供按边运算、数据集接口与图批处理, 免去重复实现这些组件:

- 各种图数据集 (Cora、Citeseer、PubMed、OGB、QM9、TUDataset 等) 的一键加载;
- 高效的稀疏邻接表示 (edge_index: [2, E]) 的长整型张量): 对稀疏大图 (百万节点、亿条边) 才装得下;
- 邻居采样、mini-batch (Batch/DataLoader)、异构图、稀疏scatter聚合等高级特性;
- 多种现成 GNN 层 (GCNConv、SAGEConv、GATConv、GINConv 等).

先安装与设备相符的 PyTorch, 再安装 PyG:

```
python -m pip install torch_geometric
```

邻居采样等功能可能需要额外扩展, 应按当前安装页选择与 PyTorch、CUDA 匹配的构建.^⑤ 本次复核使用 PyTorch2.12.0 CPU 与 PyG2.7.0.

9.11.1 把前面的 GCN 换成 PyG 版

```

1 from torch_geometric.nn import GCNConv
2
3 class GCN_PyG(nn.Module):
4     def __init__(self, in_dim, hidden, out_dim, dropout=0.5):
5         super().__init__()
6         self.conv1 = GCNConv(in_dim, hidden)
7         self.conv2 = GCNConv(hidden, out_dim)
8         self.dropout = dropout
9
10    def forward(self, x, edge_index):
11        x = F.relu(self.conv1(x, edge_index))
12        x = F.dropout(x, p=self.dropout, training=self.training)
13        return self.conv2(x, edge_index)

```

注意接口变化: forward 的第二个参数是 edge_index (稀疏边表) 而非 A_hat (稠密归一化矩阵). GCNConv 内部按需做了对称归一化, 不用我们手动算 $D^{-1/2}$.

^④ <https://pytorch-geometric.readthedocs.io/>.

^⑤ <https://pytorch-geometric.readthedocs.io/en/latest/install/installation.html>. PyG 官方对较新 PyTorch 版本推荐 pip, 不再提供相应 conda 包.

9.12 真实数据集实战:Cora 论文引用网络

Cora把论文作为节点、引用联系作为边。这里使用 Planetoid 公开划分的无向处理版本,任务是利用论文词袋特征与引用关系预测类别:

- 2708 个节点(论文),每篇论文由 1433 维词袋向量表示(特征是否含某关键词);
- 5278 条无向边,在 PyG 的边表中按两个方向存为 10556 条记录;
- 7 个类别(神经网络、强化学习、概率方法、规则学习、遗传算法、理论、案例研究);
- 标准 train/val/test 切分:140 个训练标签(每类 20 个)、500 个验证、1000 个测试。

这 140 个训练标签只占节点数的约 5.2%。另外 1068 个节点不属于这三个标签集合,也不纳入验证或测试准确率;它们的特征与边仍可参与直推式消息传递。

9.12.1 数据处理

```
1 from torch_geometric.datasets import Planetoid
2 from torch_geometric.transforms import NormalizeFeatures
3
4 dataset = Planetoid(root='data/Planetoid', name='Cora',
5                     transform=NormalizeFeatures())
6 data = dataset[0]
7 print(data)
8 print(f'类别数 = {dataset.num_classes}, 特征维数 = {dataset.num_features}')
9 print(f'训练/验证/测试 = {data.train_mask.sum()} / '
10       f'{data.val_mask.sum()} / {data.test_mask.sum()}')
```

输出:

```
Data(x=[2708, 1433], edge_index=[2, 10556], y=[2708],
     train_mask=[2708], val_mask=[2708], test_mask=[2708])
类别数 = 7, 特征维数 = 1433
训练/验证/测试 = 140 / 500 / 1000
```

PyG 的 Data 对象统一管理图的所有信息: x 是节点特征、edge_index 是边表、y 是节点标签、*_mask 是布尔型的训练/验证/测试切分。

9.12.2 模型构建

Cora 上直接复用上一节用 PyG 重写的 GCN_PyG: 两层 GCNConv, 隐藏层 16 维, 中间加 dropout=0.5。把模型和数据都搬到同一设备上, 再配一个带权重衰减的 Adam 优化器。

```

1 device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
2 data = data.to(device)
3 torch.manual_seed(0)
4 model = GCN_PyG(in_dim=dataset.num_features,
5                 hidden=16, out_dim=dataset.num_classes,
6                 dropout=0.5).to(device)
7 opt = torch.optim.Adam(model.parameters(), lr=0.01, weight_decay=5e-4)

```

9.12.3 模型训练

每回合只在训练标签上计算损失, 用验证准确率选择最早达到最佳分数的参数. 测试标签不进入回合选择; 加载选定快照后才评价一次. 把训练封装成函数, 稍后让 MLP 和 GAT 沿用相同的评价流程.

```

1 def fit_node_model(model, data, opt, n_epoch=200):
2     best_val_acc, best_state, best_epoch = -float('inf'), None, 0
3     history = {'train_loss': [], 'val_acc': []}
4     for epoch in range(1, n_epoch + 1):
5         model.train(); opt.zero_grad()
6         out = model(data.x, data.edge_index)
7         loss = F.cross_entropy(out[data.train_mask], data.y[data.train_mask])
8         loss.backward(); opt.step()
9         model.eval()
10        with torch.no_grad():
11            pred = model(data.x, data.edge_index).argmax(dim=1)
12            val_acc = (pred[data.val_mask] == data.y[data.val_mask]).float().mean().item()
13        history['train_loss'].append(loss.item())
14        history['val_acc'].append(val_acc)
15        if val_acc > best_val_acc:
16            best_val_acc, best_epoch = val_acc, epoch
17            best_state = {k: v.detach().clone() for k, v in model.state_dict().items()}
18        if epoch % 50 == 0:
19            print(f'epoch {epoch:3d} loss={loss.item():.4f} val={val_acc:.4f}')
20        model.load_state_dict(best_state); model.eval()
21        with torch.no_grad():
22            pred = model(data.x, data.edge_index).argmax(dim=1)
23            final_test = (pred[data.test_mask] == data.y[data.test_mask]).float().mean().item()
24        print(f'best_epoch={best_epoch}, test_acc={final_test:.4f}')
25        return history, final_test, best_epoch
26
27 history, final_test, best_epoch = fit_node_model(model, data, opt)
28 cora_model, cora_history = model, history
29 cora_results = {'GCN': final_test}

```

输出:

```
epoch 50 loss=1.1713 val=0.7580
epoch 100 loss=0.5719 val=0.7860
epoch 150 loss=0.4109 val=0.7880
epoch 200 loss=0.3846 val=0.7840
best_epoch=93, test_acc=0.8080
```

9.12.4 模型评价

把训练完的隐层表示用 t-SNE 降到 2 维, 看看 7 个类别是否被 GNN 分开 (完整绘图代码见配套 notebook)。

图 9.8 按数据中的类别编号着色。类别编号到英文名称的映射应从当前数据版本读取, 不能凭顺序猜测; 降维图中簇的远近也不能直接解释成研究方向的语义距离。

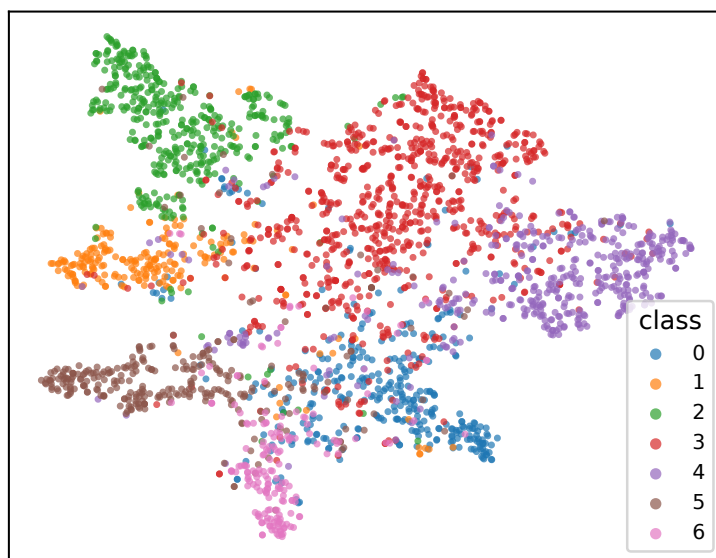


图 9.8 Cora 节点经 GCN 编码后的 t-SNE 可视化

9.12.4.1 对比基线

将特征信息与连接信息拆开, 有助于判断它们各自贡献多少 (表 9.2):

- **MLP**: 只用节点特征 $\mathbf{X}$, 完全无视边;
- **Label Propagation**: 完全不用特征, 只靠图结构传标签。

```

1  class MLP(nn.Module):
2      def __init__(self, in_dim, hidden, out_dim, dropout=0.5):
3          super().__init__()
4          self.lin1 = nn.Linear(in_dim, hidden)
5          self.lin2 = nn.Linear(hidden, out_dim)
6          self.dropout = dropout
7      def forward(self, x, edge_index=None):          # 接口对齐
8          return self.lin2(F.dropout(F.relu(self.lin1(x)),
9                                     p=self.dropout, training=self.training))
10
11 torch.manual_seed(0)
12 mlp = MLP(dataset.num_features, 16, dataset.num_classes).to(device)
13 mlp_opt = torch.optim.Adam(mlp.parameters(), lr=0.01, weight_decay=5e-4)
14 _, cora_results['MLP'], _ = fit_node_model(mlp, data, mlp_opt)
15
16 from torch_geometric.nn import LabelPropagation, GATConv
17 lp = LabelPropagation(num_layers=50, alpha=0.9)
18 lp_pred = lp(data.y, data.edge_index, mask=data.train_mask).argmax(dim=1)
19 cora_results['LP'] = (lp_pred[data.test_mask] == data.y[data.test_mask]).float().mean().
    item()
20
21 class GAT_PyG(nn.Module):
22     def __init__(self, in_dim, out_dim):
23         super().__init__()
24         self.conv1 = GATConv(in_dim, 8, heads=8, dropout=0.6)
25         self.conv2 = GATConv(64, out_dim, heads=1, concat=False, dropout=0.6)
26     def forward(self, x, edge_index):
27         x = F.dropout(x, 0.6, training=self.training)
28         x = F.elu(self.conv1(x, edge_index))
29         x = F.dropout(x, 0.6, training=self.training)
30         return self.conv2(x, edge_index)
31
32 torch.manual_seed(0)
33 gat = GAT_PyG(dataset.num_features, dataset.num_classes).to(device)
34 gat_opt = torch.optim.Adam(gat.parameters(), lr=0.005, weight_decay=5e-4)
35 _, cora_results['GAT'], _ = fit_node_model(gat, data, gat_opt)
36 for name, value in cora_results.items():
37     print(f'{name}: {value:.4f}')

```

表 9.2 Cora 数据集上不同模型的测试准确率 (本次固定种子与公开划分)

模型	Test Acc	用了什么信息
MLP (只用特征)	0.5690	只用 $\mathbf{X}$, 不看图
Label Propagation	0.7130	只用 $\mathbf{A}$, 不看 $\mathbf{X}$
GCN (2 层)	0.8080	同时用 $\mathbf{A}$ 和 $\mathbf{X}$
GAT (2 层 8 头)	0.8330	同上 + 注意力

MLP 与 GCN 使用相同隐藏维数、暂退率和优化器设置. GAT 采用首层 8 头、每头 8 维、暂退率 0.6、学习率 0.005; 三种可训练模型都训练 200 回合、按验证集选择快照. 标签传播固定 50 步、 $\alpha = 0.9$. 这些结果用来比较信息来源与具体配置, 不能把差异全部归因于注意力.

提醒



同一公开划分上的一次结果有利于复查代码, 但不代表跨数据集性能. 若继续调参, 应保留测试集, 使用验证集作选择, 并报告不同种子. MNIST 与 Cora 的样本结构和监督方式不同, 不宜据标签数量直接比较难度.

9.12.5 模型预测

最后用训练好的模型对单个节点做一次预测, 直观感受“给定一篇论文, 模型判断它属于哪个研究方向”:

```

1 # 直接显示数据集类别编号; 类别名须使用与当前数据版本匹配的映射
2 model.eval()
3 with torch.no_grad():
4     probs = F.softmax(model(data.x, data.edge_index), dim=1)
5 # 任取一个测试集中的节点
6 node = data.test_mask.nonzero(as_tuple=True)[0][0].item()
7 pred = probs[node].argmax().item()
8 print(f'节点 {node}: 预测类别 = {pred}')
9     f' (Softmax分数 {probs[node, pred]:.2f}), 真实类别 = {data.y[node].item()}'

```

输出:

节点 1708: 预测类别 = 1 (Softmax分数 0.19), 真实类别 = 3

对照 `data.y` 可检查该节点是否预测正确. 打印的最大 Softmax 值只是模型分数, 未经校准不能直接解释为正确概率. 要分析错误原因, 可在验证节点上检查词袋特征、邻居标签分布和节点度数, 再设计对照实验.



动手练习 9.6

- 1) 在 Cora 上用 GATConv 替换 GCNConv, 尝试 4 头和 8 头多头注意力, 在验证集上选择配置; 在 GATConv 前向调用时传入 `return_attention_weights=True`, 从返回的边索引与注意力权重中可视化某个节点的注意力分布。
- 2) 在 Cora 上做 [链接预测](#) (参考 §9.9): 随机移除 10% 的边作为正测试样本, 重新训练 GCN encoder, 报告 AUC 和 Hits@20。

9.13 图分类实战:蛋白质结构图分类

随机图回归已经让每个样本对应一张图。现在换成 PROTEINS: 输入一张蛋白质结构图, 预测图级类别。与 Cora 只遮住同一图中的标签不同, 这里训练、验证、测试包含不同的图。

9.13.1 数据处理

PROTEINS 包含 1113 张蛋白质结构图。节点表示二级结构单元; 边表示这些单元在序列或空间上的邻接关系。默认加载的 3 维节点特征为节点类别的独热编码, 图级目标区分酶与非酶。代码按类别分层固定约 80%/10%/10% 的划分, 显示原始加载后的类别编号; 没有核对映射前, 不自行给 0、1 贴上语义名称。

```
1 from torch_geometric.datasets import TUDataset
2 from torch_geometric.loader import DataLoader
3 from sklearn.model_selection import train_test_split
4
5 dataset = TUDataset(root='data/TUDataset', name='PROTEINS')
6 print(dataset)
7 print(f'类别数 {dataset.num_classes}, 节点特征维数 {dataset.num_node_features}')
8 indices = np.arange(len(dataset))
9 graph_labels = np.array([int(g.y) for g in dataset])
10 train_ids, held_ids = train_test_split(indices, test_size=0.2,
11                                       stratify=graph_labels, random_state=0)
12 val_ids, test_ids = train_test_split(held_ids, test_size=0.5,
13                                     stratify=graph_labels[held_ids], random_state=1)
14 train_set, val_set, test_set = [dataset[ids.tolist()] for ids in [train_ids, val_ids,
15                                                                    test_ids]]
16 print('训练/验证/测试图数: ', len(train_set), len(val_set), len(test_set))
17 train_loader = DataLoader(train_set, batch_size=64, shuffle=True,
18                           generator=torch.Generator().manual_seed(0))
19 val_loader = DataLoader(val_set, batch_size=64)
20 test_loader = DataLoader(test_set, batch_size=64)
```

数据统计与原始论文见<https://chrsmrrs.github.io/datasets/docs/datasets/>。

9.13.1.1 Mini-batch:多张图“粘”在一起

PyG 把一个大批量里的图组合成一张大图,不同图之间没有边,其邻接矩阵呈块对角结构. 各图可以一起做消息传递,再用batch向量记录每个节点所属的图;读出时按这个向量分组汇聚,就能得到每张图的表示.

```

1 batch = next(iter(DataLoader(train_set[:2], batch_size=2)))
2 print(batch)
3
4 print('batch 向量前 20 个:', batch.batch[:20].tolist())

```

9.13.2 模型构建

下面用 3 层 GIN 构建分类器. 每层先更新节点表示,再按图求和,最后拼接各层读出. 求和保留节点数量信息;训练效果仍需由验证集判断.

```

1 from torch_geometric.nn import GINConv, global_add_pool
2
3 class GINGraphClassifier(nn.Module):
4     def __init__(self, in_dim, hidden, out_dim, n_layers=3, dropout=0.3):
5         super().__init__()
6         self.convs = nn.ModuleList()
7         for i in range(n_layers):
8             d_in = in_dim if i == 0 else hidden
9             mlp = nn.Sequential(
10                 nn.Linear(d_in, hidden), nn.ReLU(),
11                 nn.Linear(hidden, hidden))
12             self.convs.append(GINConv(mlp, train_eps=True))
13         # 拼接每层 readout
14         self.classifier = nn.Sequential(
15             nn.Linear(hidden * n_layers, hidden),
16             nn.ReLU(), nn.Dropout(dropout),
17             nn.Linear(hidden, out_dim))
18
19     def forward(self, x, edge_index, batch):
20         readouts = []
21         for conv in self.convs:
22             x = F.relu(conv(x, edge_index))
23             readouts.append(global_add_pool(x, batch)) # 按图分组求和
24         h_graph = torch.cat(readouts, dim=1)
25         return self.classifier(h_graph)

```

9.13.3 模型训练

```

1 device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
2 torch.manual_seed(0)
3 model = GINGraphClassifier(
4     in_dim=dataset.num_node_features,
5     hidden=64, out_dim=dataset.num_classes, n_layers=3).to(device)
6 opt = torch.optim.Adam(model.parameters(), lr=1e-3, weight_decay=5e-4)
7
8 def evaluate(loader):
9     model.eval(); n_correct = 0; n_total = 0
10    with torch.no_grad():
11        for batch in loader:
12            batch = batch.to(device)
13            pred = model(batch.x, batch.edge_index, batch.batch).argmax(dim=1)
14            n_correct += (pred == batch.y).sum().item()
15            n_total += batch.num_graphs
16    return n_correct / n_total
17
18 history = {'train_loss': [], 'val_acc': []}
19 best_val = -float('inf'); best_state = None
20 for epoch in range(1, 101):
21     model.train(); ep_loss = 0.0; ep_n = 0
22     for batch in train_loader:
23         batch = batch.to(device); opt.zero_grad()
24         logits = model(batch.x, batch.edge_index, batch.batch)
25         loss = F.cross_entropy(logits, batch.y)
26         loss.backward(); opt.step()
27         ep_loss += loss.item() * batch.num_graphs
28         ep_n += batch.num_graphs
29     train_loss = ep_loss / ep_n
30     val_acc = evaluate(val_loader)
31     history['train_loss'].append(train_loss)
32     history['val_acc'].append(val_acc)
33     if val_acc > best_val:
34         best_val = val_acc
35         best_state = {k: v.detach().clone() for k, v in model.state_dict().items()}
36     if epoch % 10 == 0:
37         print(f'epoch {epoch:3d} loss={train_loss:.4f} '
38               f'val={val_acc:.4f}')
39
40 model.load_state_dict(best_state)
41 final_test = evaluate(test_loader)
42 print(f'\nBest-val 模型 test_acc = {final_test:.4f}')

```

9.13.4 模型评价

二分类任务最直观的诊断是**混淆矩阵**和**ROC 曲线**:

```
1 from sklearn.metrics import confusion_matrix, roc_auc_score, roc_curve
2
3 model.eval(); all_pred, all_score, all_y = [], [], []
4 with torch.no_grad():
5     for batch in test_loader:
6         batch = batch.to(device)
7         logits = model(batch.x, batch.edge_index, batch.batch)
8         all_pred.append(logits.argmax(dim=1).cpu())
9         all_score.append(F.softmax(logits, dim=1)[ :, 1].cpu())
10        all_y.append(batch.y.cpu())
11 y_true = torch.cat(all_y).numpy()
12 y_pred = torch.cat(all_pred).numpy()
13 y_score = torch.cat(all_score).numpy()
14
15 cm = confusion_matrix(y_true, y_pred)
16 auc = roc_auc_score(y_true, y_score)
17 fpr, tpr, _ = roc_curve(y_true, y_score)
```

本次测试准确率为 80.36%, AUC 为 0.8418. 图9.9按类别编号给出混淆矩阵, 以类别 1 为 ROC 正例. 这 112 张图的结果用于复核当前划分, 还不能代表不同蛋白质分布下的表现.

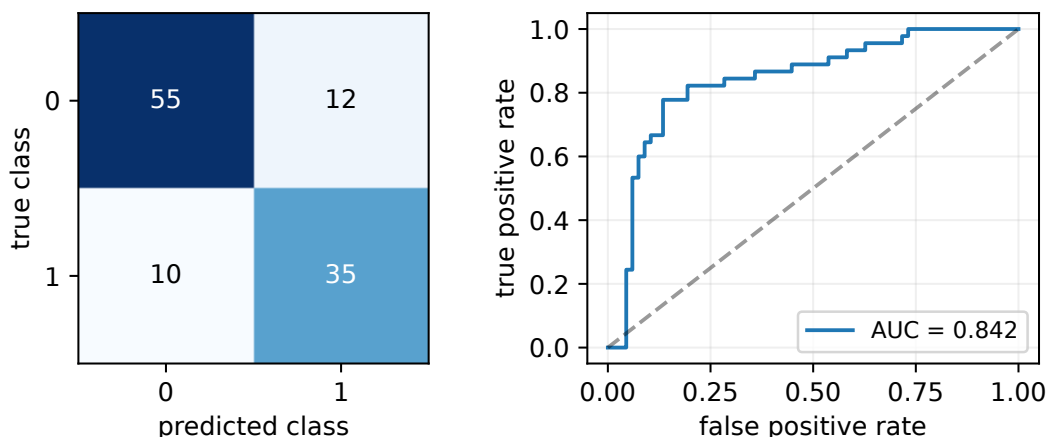


图 9.9 PROTEINS 测试集上的混淆矩阵与 ROC 曲线

**动手练习 9.7**

在配套 notebook 中用 cm 画出混淆矩阵热力图,并用 fpr、tpr 画出 ROC 曲线,与图 9.9 对照。

笔记

图级表示还可用于以下任务,但输入定义与数据划分要随任务改变:

- **药物发现**: DeepChem、Stanford OGB 提供的 ogbg-mol* 系列,专门用于分子属性预测;
- **量子化学**: QM9、PCQM4Mv2 上做能量、HOMO-LUMO 间隙回归;
- **程序分析**: 把代码 AST/CFG 当成图,预测漏洞或 bug;
- **生物分子**: 蛋白质-蛋白质对接、酶催化预测。

若任务要求整图输出,需要将节点信息读出为图级表示。分子模型还可能需键类型、三维坐标等输入,本章忽略边属性的 GIN 不能直接替代这些专门模型。对于蛋白质,随机分图也不等于向全新家族泛化,后者需要按序列或结构相似性设计划分。

9.13.5 模型预测

取一张保留的测试图演示完整前向过程。单图的 batch 向量全为 0,与批处理中“属于哪张图”的约定一致:

```
1 model.eval()
2 g = test_set[0].to(device)
3 with torch.no_grad():
4     # 单张图, batch 向量全为 0
5     batch_vec = torch.zeros(g.num_nodes, dtype=torch.long, device=device)
6     pred = model(g.x, g.edge_index, batch_vec).argmax(dim=1).item()
7     print(f'预测类别: {pred}, 真实类别: {int(g.y)}')
```

输出:

预测类别: 0, 真实类别: 0

类别编号便于核对代码;解释为酶或非酶前,需核验当前数据加载器与原始标签的对应关系。错误分析和后续模型选择应回到验证集。



动手练习 9.8

1) 把 GIN 换成 2 层 GCN 与全局平均汇聚, 固定划分、训练预算和模型选择规则, 比较验证表现. 哪部分差异来自层数、参数量或读出方式? 一次只改变一个因素再作判断.

9.14 小结

从俱乐部成员的派别到缺失的关系, 再到蛋白质整图标签, 预测对象改变时, 监督位置、数据划分和读出方式也随之改变. 4 种算子都沿边传递消息, 但它们选择邻居系数、保留数量信息的方式不同, 见表 9.3. 实际效果还受到标签数量、优化过程和任务结构的影响.

表 9.3 4 种 GNN 的关键差异

模型	关键思路	公式核心
GCN	对称归一化的图卷积	$\tilde{D}^{-1/2} \tilde{A} \tilde{D}^{-1/2} H W$
GraphSAGE	邻居采样 + 聚合	$\sigma(W[h_i \parallel \text{AGG}(\mathcal{N}(i))])$
GAT	学习邻居权重	$\sum_j \alpha_{ij} W h_j$
GIN	单射聚合 + MLP	$\text{MLP}((1 + \epsilon) h_i + \sum_j h_j)$

把实验带到新图上 先写清节点和边代表什么、推断时哪些关系可见, 以及评价对象是节点、边还是整张图. 然后建立简单基线: 节点任务比较只用特征的 MLP 与只用关系的标签传播, 链接任务固定负样本候选, 图级任务检查读出是否匹配总量或平均量.

同质性描述相邻节点标签是否相似; 异构图描述是否有多种节点或关系类型, 二者不是一回事. 换成 GAT 或 GraphSAGE 不会自动解决弱同质性. 可以检查自身特征与邻居特征是否需要分开处理, 并用验证结果判断.

深层图网络还有**过度压缩** (Over-squashing) 问题: 远处大量信息要经过少数边和固定宽度表示传递, 关键差异可能在瓶颈处丢失; 它不同于反复混合造成的过度平滑. 层数、宽度、残差、采样和图结构修改各影响不同环节, 不能用“加深网络”或“加入注意力”概括所有补救方式.

下一章我们进入**大语言模型与智能体**: 把注意力机制扩展到序列上的 Transformer 模型, 并构建一个简单的智能体.

第10章 大语言模型与智能体

在输入框里写下“ROMEO:”，让程序接着往下写，会发生什么？随机初始化的网络先吐出杂乱字符；训练后，空格、称呼和对白格式可能逐渐出现。可如果把提示改成“将这个单词转为大写”，同一个模型未必照做。再问一句需要查资料的问题，仅靠续写也无法补上缺失的信息。本章从这些不同的要求出发，逐步搭建语言模型与它周围的执行程序。

大语言模型（Large Language Model, LLM）的完整训练超出普通个人设备的预算。这里先用 813568 个参数的字符模型检查自回归训练，再用 LoRA 微调和偏好比较观察目标如何改变模型输出。工具调用与检索部分则采用明确标记的模拟接口，让读者能单独核验执行流程。小模型实验与模拟接口各自回答不同问题，不能把它们连起来就宣称得到了可靠助手。

本章对应《神经网络与深度学习》第13章。图10.1概括了从下一词元预测、模型适配到工具与资料使用的路径；第8章的注意力和第7章的优化知识会在代码中再次出现。

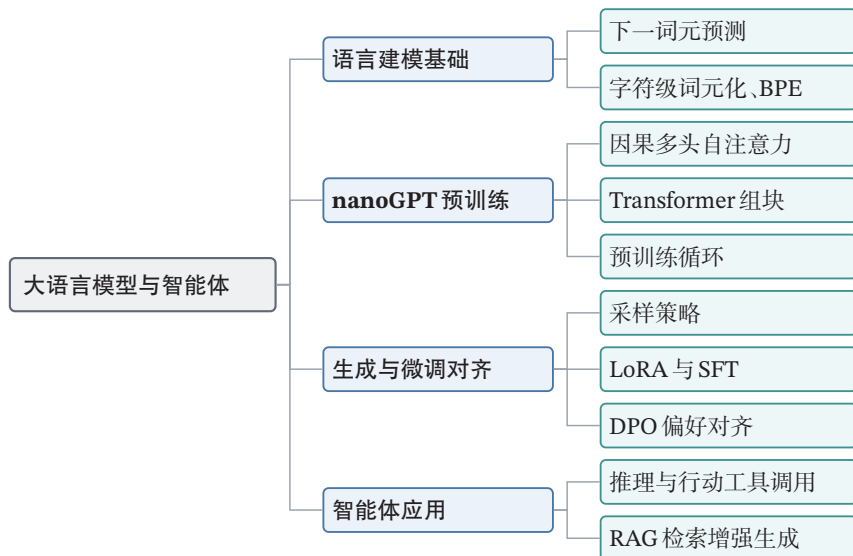


图 10.1 大语言模型与智能体关键知识点回顾

10.1 语言模型回顾

语言模型对词元序列的概率分布进行建模. 本章采用**自回归语言模型**: 给定前序词元 $x_{<t}$, 估计下一个词元 x_t 的概率. 序列的联合分布可以分解为:

$$p(x_1, x_2, \dots, x_T) = \prod_{t=1}^T p(x_t | x_{<t}). \quad (10.1)$$

这种建模方式就是**自回归** (autoregressive): 模型根据当前上下文中的前序词元预测下一个词元, 生成时的前序词元也可以是模型此前生成的结果. 训练目标是**极大化训练语料的对数似然**, 等价于最小化下一词元预测的交叉熵:

$$\mathcal{L} = - \sum_{t=1}^T \log p_{\theta}(x_t | x_{<t}). \quad (10.2)$$

工程实现上, 这等价于把序列 $[x_0, x_1, \dots, x_{T-1}]$ 作为输入、 $[x_1, x_2, \dots, x_T]$ 作为目标 (错开一位), 让模型并行预测每个位置的下一个词元.

10.2 实践: 从零搭建字符级 nanoGPT

先固定一件事: 给定前缀, 预测下一个字符. 数据、损失和采样都围绕这一目标组织; 后面再改变监督样本和执行流程, 观察哪些行为来自权重更新, 哪些来自外部程序.

10.2.1 数据处理

本节使用 TinyShakespeare 文本, 共 1115394 个字符. ①按原始文本顺序分为 90% 训练、5% 验证、5% 测试, 分别为 1003854、55770 和 55770 个字符; 切取窗口时不跨集合边界. 连续切分便于复核, 但其中作品、人物和文本风格的分布不一定相同.

采用**字符级词元化**, 在训练文本中建立 65 字符的词表; 本语料验证与测试中没有新增字符. 实际使用时若出现词表外字符, 当前编码器会报错; 字节级方案或显式未知词元需要另外设计.

```

1 import math
2 import urllib.request
3 from pathlib import Path
4 import torch
5 import torch.nn as nn
6 import torch.nn.functional as F
7
```

① 公开教学语料: <https://github.com/karpathy/char-rnn/tree/master/data/tinyshakespeare>.

```

8 # 下载 TinyShakespeare (约 1.1 MB)
9 data_path = Path('tinyshakespeare.txt')
10 if not data_path.exists():
11     url = 'https://raw.githubusercontent.com/karpathy/char-rnn/master/data/tinyshakespeare/
        input.txt'
12     urllib.request.urlretrieve(url, data_path)
13 text = data_path.read_text(encoding='utf-8')
14 # 字符级词表
15 n_train, n_val = int(0.9 * len(text)), int(0.95 * len(text))
16 chars = sorted(set(text[:n_train]))
17 vocab_size = len(chars)
18 stoi = {ch: i for i, ch in enumerate(chars)}
19 itos = {i: ch for i, ch in enumerate(chars)}
20
21 def encode(s):
22     return torch.tensor([stoi[c] for c in s], dtype=torch.long)
23
24 def decode(ids):
25     return ''.join(itos[int(i)] for i in ids)
26
27 data = encode(text)
28 train_data, val_data, test_data = data[:n_train], data[n_train:n_val], data[n_val:]
29 print('字符数/词表: ', len(text), vocab_size)
30 print('训练/验证/测试长度: ', len(train_data), len(val_data), len(test_data))

```

笔记



真实大模型常采用子词级词元化,在词表大小和序列长度之间折中。字节对编码 (Byte Pair Encoding, BPE) 和 Unigram 是词元化算法; SentencePiece 是支持这些算法的工具。本章先采用字符级方案,下一节再用一个小实验展开 BPE 的工作原理。

10.2.1.1 从字符到子词: BPE 是怎么工作的

字符级词元化会使一些常见片段占用多个位置:“the”要花 3 个词元,“Shakespeare”要花 11 个词元。整词词元化则需要容纳更多词条,还会遇到未登录词 (Out-of-Vocabulary, OOV) 问题,即输入中出现词表没有收录的词。子词方案在这两端之间取得折中。

BPE 是一种常见的子词算法。下面以从字符开始的教学版本说明它的过程:

- 1) 一开始把每个字符当作一个词元;
- 2) 在整个语料里统计相邻词元对 (a, b) 的出现频次;
- 3) 把出现最频繁的那对合并成一个新词元 ab ;
- 4) 重复 2-3,直到词表达到目标大小。

合并规则由当前语料中的频次决定. 这里先按空白切成词, 再在词内合并, 不跨空格, 也没有加入词尾标记; 它是说明相邻片段合并的教学版本. 不要从 30 次合并直接推断一定会形成哪些完整单词.

```
1 from collections import Counter
2
3 def get_pair_freqs(corpus):
4     """统计相邻词元对的出现频次. """
5     pairs = Counter()
6     for tokens, freq in corpus.items():
7         for a, b in zip(tokens, tokens[1:]):
8             pairs[(a, b)] += freq
9     return pairs
10
11 def merge_pair(pair, corpus):
12     """把语料中所有 pair=(a,b) 合并成 'ab'. """
13     new_corpus = Counter()
14     a, b = pair
15     for tokens, freq in corpus.items():
16         new_tokens = []
17         i = 0
18         while i < len(tokens):
19             if i < len(tokens) - 1 and tokens[i] == a and tokens[i+1] == b:
20                 new_tokens.append(a + b); i += 2
21             else:
22                 new_tokens.append(tokens[i]); i += 1
23         new_corpus[tuple(new_tokens)] += freq
24     return new_corpus
25
26 def train_bpe(words, n_merges):
27     """words: dict[str, int], 每个词及其频次. """
28     corpus = {(tuple(w): freq for w, freq in words.items())}
29     merges = []
30     for _ in range(n_merges):
31         pairs = get_pair_freqs(corpus)
32         if not pairs: break
33         best = pairs.most_common(1)[0][0]
34         corpus = merge_pair(best, corpus)
35         merges.append(best)
36     return merges, corpus
37 # 演示: 用前 1 万个字符的莎士比亚做训练
38 sample = text[:10000]
39 word_freqs = Counter(sample.split())
40 merges, vocab = train_bpe(word_freqs, n_merges=30)
41 print('前 10 次 merge:', merges[:10])
```

输出:

```
前 10 次 merge: [('t', 'h'), ('th', 'e'), ('o', 'u'), ('a', 't'),  
                ('r', 'e'), ('s', 't'), ('i', 'n'), ('e', 'n'),  
                ('a', 'n'), ('i', 't')]
```

本次先合并“t”和“h”,再合并“th”和“e”. 频次相同时,代码采用 Counter 保留的出现顺序决定先后;换语料或规则,结果也会改变. 下面对同一个名字应用不同数量的合并规则:

```
1 def tokenize(word, merges):  
2     """按已学到的 merge 顺序应用到一个新词上. """  
3     tokens = list(word)  
4     for a, b in merges:  
5         i = 0  
6         while i < len(tokens) - 1:  
7             if tokens[i] == a and tokens[i+1] == b:  
8                 tokens[i:i+2] = [a + b]  
9             else:  
10                i += 1  
11     return tokens  
12  
13 for n in [0, 10, 30, 100]:  
14     ms, _ = train_bpe(word_freqs, n_merges=n)  
15     print(f'{n:>3d} merges: Shakespeare -> {tokenize("Shakespeare", ms)}')
```

输出:

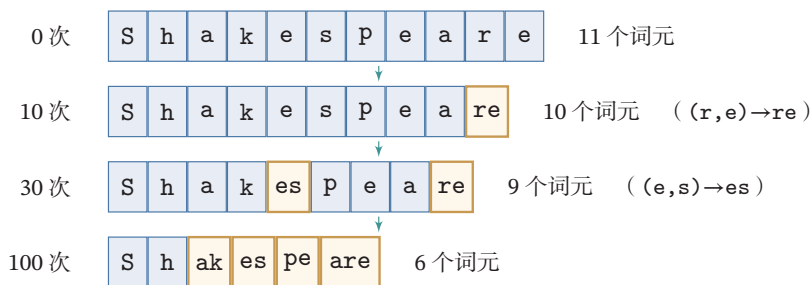
```
0 merges: Shakespeare -> ['S','h','a','k','e','s','p','e','a','r','e']  
10 merges: Shakespeare -> ['S','h','a','k','e','s','p','e','a','re']  
30 merges: Shakespeare -> ['S','h','a','k','es','p','e','a','re']  
100 merges: Shakespeare -> ['S','h','ak','es','pe','are']
```

随着合并次数增加,“es”、“pe”、“are”等片段逐步登场,本例中“Shakespeare”的词元数量从 11 降到 6. 切分结果取决于训练语料和合并规则;高频片段可能恰好是一个词或词缀,也可能横跨构词边界. BPE 统计的是相邻片段频次,并不显式分析词根和词缀. 图 10.2 展示了本实验从字符出发逐步合并的过程.

语料: *TinyShakespeare* 前 1 万字符; 在全部词上统计相邻 *pair* 频次, 每轮贪心合并最高频对. 下面的合并顺序与切分均由正文代码实际跑出.

学到的合并 (前 7, 按顺序): th the ou at re st in ...
(按学习到的规则顺序应用; “the” = th + e, 新词元继续参与后续合并)

把合并规则依次应用到示例词 *Shakespeare*:



规则由词内相邻片段的频次决定, 既不跨空格, 也不保证片段对应词根或词缀. 示例词的词元数从 11 降到 6; 不同语料、预处理或并列频次处理方式会得到不同切分.

图 10.2 BPE 训练过程

10.2.1.2 选择词元化粒度

字符、字节、子词各保留了不同的信息, 也会改变序列长度与计算预算:

- **序列变长**: 高频片段不能合并时, 同一文本通常需要更多词元, 增幅取决于语言与词表. 固定隐藏维度时, 全自注意力的成对计算为 $O(T^2)$; 序列翻倍使这一部分的计算量约增至 4 倍, 不代表整个模型的计算都增加 4 倍;
- **组合跨度增加**: 英文字符通常需要组合才能构成词, 模型要跨多个位置学习这种关系; 汉字则常能承载语义, 不能一概说单个字符没有含义;
- **表示粒度不同**: 字符级保留了拼写信息, 但需要模型自己组合常见片段; 子词方案能缩短一些组合路径, 却也不保证按“run+ing”这样的构词结构切分.

教学中采用字符级方案, 便于直接检查词表和生成结果; 实际模型则按语言覆盖、序列长度和任务选择粒度. BPE、WordPiece、Unigram 属于算法, SentencePiece 和 tiktoken 属于工具库, 使用时应核对具体模型的分词器与词表.

提醒



从 GPT-2 开始, 字节级 BPE 成为大模型词元化的一条重要路线: 先把文本编码为 UTF-8 字节流, 再在字节序列上做 BPE. 只要词表保留完整的基础字节集合, 就可以表示任意输入文本而不产生未知词元. 一个汉字或词语最终对应多少词元取决于学到的合并规则, 不能用固定的每字词元数概括.



动手练习 10.1

1) 把字符级 nanoGPT 改造成 BPE 级 (用 tokenizers 库训一个小词表), 观察验证损失和生成质量的变化。

10.2.2 模型构建

第8章已经介绍多头自注意力、Pre-LN 和位置编码,本章复用这些组件,再加上因果掩码,组装一个**仅解码器**(Decoder-only) Transformer. 这里采用接近 GPT-2 的教学结构; Llama 等模型在位置编码、规范化和前馈网络上还有不同设计:

- 1) 词元嵌入 + 位置编码;
 - 2) N 个 Transformer Block (每个 block 依次包含层规范化、因果多头自注意力、残差, 以及层规范化、前馈网络 (4 倍隐藏维度)、残差);
 - 3) 末端层规范化;
 - 4) 线性层映射到词表大小, 得到未归一化分数。
- 几个关键的工程决定:

- **因果掩码**: 训练时若不加约束, 自注意力会“看到未来”, 等于直接抄答案. 所以这里强制套上三角 mask, 屏蔽掉每个位置右侧的所有词元;
- **前规范化** (Pre-LN): 层规范化放在注意力与前馈子层之前, 相比原始 Transformer 的后规范化 (Post-LN), 常能改善深层网络的训练稳定性; 实际效果仍取决于初始化和优化设置;
- **位置编码**: 采用可学习的绝对位置嵌入, 适用于预先规定的上下文长度; 它不能自动外推到从未训练的位置。

10.2.2.1 因果多头自注意力

```
1 class CausalSelfAttention(nn.Module):
2     def __init__(self, n_embd, n_head, block_size, dropout=0.0):
3         super().__init__()
4         if n_head <= 0 or n_embd % n_head != 0:
5             raise ValueError('头数须为正并整除隐藏维数')
6         self.n_head = n_head
7         self.head_dim = n_embd // n_head
8         # q, k, v 投影合在一个矩阵里
9         self.qkv = nn.Linear(n_embd, 3 * n_embd, bias=False)
10        self.proj = nn.Linear(n_embd, n_embd, bias=False)
11        self.dropout = nn.Dropout(dropout)
12        # 因果掩码
13        mask = torch.triu(torch.ones(block_size, block_size), diagonal=1).bool()
14        self.register_buffer('mask', mask)
```

```

15
16     def forward(self, x):
17         B, T, C = x.shape
18         if not 1 <= T <= self.mask.shape[0]:
19             raise ValueError('序列长度超出注意力窗口')
20         qkv = self.qkv(x)
21         q, k, v = qkv.split(C, dim=2)
22         q = q.view(B, T, self.n_head, self.head_dim).transpose(1, 2)
23         k = k.view(B, T, self.n_head, self.head_dim).transpose(1, 2)
24         v = v.view(B, T, self.n_head, self.head_dim).transpose(1, 2)
25         att = (q @ k.transpose(-2, -1)) / math.sqrt(self.head_dim)
26         att = att.masked_fill(self.mask[:T, :T], float('-inf'))
27         att = F.softmax(att, dim=-1)
28         att = self.dropout(att)
29         y = att @ v
30         y = y.transpose(1, 2).contiguous().view(B, T, C)
31         return self.proj(y)

```

动手练习 10.2



1) CausalSelfAttention里手写的注意力是“ $q @ k.transpose(-2, -1)$ 除以 $\sqrt{d}$ 、再masked_fill套因果掩码、最后F.softmax”这一串。请把这段替换为PyTorch自带的F.scaled_dot_product_attention(设is_causal=True,从而省去显式的三角掩码缓冲),在相同随机种子下对比两种实现的训练损失曲线与单步耗时,并说说这点差异从何而来。

10.2.2.2 Block与整体结构

```

1  class FeedForward(nn.Module):
2      def __init__(self, n_embd, dropout=0.0):
3          super().__init__()
4          self.net = nn.Sequential(
5              nn.Linear(n_embd, 4 * n_embd, bias=False),
6              nn.GELU(), # GPT-2 风格
7              nn.Linear(4 * n_embd, n_embd, bias=False),
8              nn.Dropout(dropout),
9          )
10
11     def forward(self, x):
12         return self.net(x)
13
14     class Block(nn.Module):
15         def __init__(self, n_embd, n_head, block_size, dropout=0.0):
16             super().__init__()

```

```

17         self.ln1 = nn.LayerNorm(n_embd)
18         self.attn = CausalSelfAttention(n_embd, n_head, block_size, dropout)
19         self.ln2 = nn.LayerNorm(n_embd)
20         self.ffn = FeedForward(n_embd, dropout)
21
22     def forward(self, x):
23         x = x + self.attn(self.ln1(x))
24         x = x + self.ffn(self.ln2(x))
25         return x
26
27 class NanoGPT(nn.Module):
28     def __init__(self, vocab_size, block_size, n_layer=4, n_head=4, n_embd=128, dropout
29         =0.0):
30         super().__init__()
31         self.block_size = block_size
32         self.tok_emb = nn.Embedding(vocab_size, n_embd)
33         self.pos_emb = nn.Embedding(block_size, n_embd)
34         self.drop = nn.Dropout(dropout)
35         self.blocks = nn.Sequential(*[
36             Block(n_embd, n_head, block_size, dropout) for _ in range(n_layer)
37         ])
38         self.ln_f = nn.LayerNorm(n_embd)
39         self.head = nn.Linear(n_embd, vocab_size, bias=False)
40
41     def forward(self, idx, targets=None):
42         B, T = idx.shape
43         if not 1 <= T <= self.block_size:
44             raise ValueError('输入长度须在 1 与 block_size 之间')
45         tok = self.tok_emb(idx)
46         pos = self.pos_emb(torch.arange(T, device=idx.device))
47         x = self.drop(tok + pos)
48         x = self.blocks(x)
49         x = self.ln_f(x)
50         logits = self.head(x)
51         loss = None
52         if targets is not None:
53             loss = F.cross_entropy(logits.reshape(-1, logits.size(-1)), targets.reshape(-1))
54         return logits, loss

```

10.2.3 模型训练

训练流程:

- 1) 从语料里随机切batch_size个长度为block_size的连续片段;

- 2) 模型输入是片段的前 T 个词元,目标是后 T 个词元(错开一位);
- 3) AdamW 优化器,权重衰减 0.1;
- 4) 学习率预热(线性升)+余弦衰减;
- 5) 每若干步在 train / val 上各采样若干个 batch 做评估.

```

1 # 默认使用 CPU, 以便复核本章结果.
2 device = torch.device('cpu')
3 block_size = 64; batch_size = 32; n_layer = 4; n_head = 4; n_embd = 128
4 max_iters = 1500; warmup = 100; lr_max = 3e-3; lr_min = 1e-4
5 train_generator = torch.Generator().manual_seed(100)
6
7 def get_batch(split, generator=None):
8     datasets = {'train': train_data, 'val': val_data, 'test': test_data}
9     sequence = datasets[split]
10    if len(sequence) <= block_size:
11        raise ValueError('语料须至少有 block_size + 1 个词元')
12    if generator is None:
13        generator = train_generator
14    ix = torch.randint(len(sequence) - block_size, (batch_size,), generator=generator)
15    x = torch.stack([sequence[i:i+block_size] for i in ix])
16    y = torch.stack([sequence[i+1:i+block_size+1] for i in ix])
17    return x.to(device), y.to(device)
18
19 def get_lr(step):
20     # step 表示第几次参数更新, 从 1 开始; 第 warmup 步达到 lr_max.
21     if step <= warmup:
22         return lr_max * step / warmup
23     progress = min(1.0, (step - warmup) / (max_iters - warmup))
24     return lr_min + 0.5 * (lr_max - lr_min) * (1 + math.cos(math.pi * progress))
25
26 torch.manual_seed(0)
27 config = dict(vocab_size=vocab_size, block_size=block_size, n_layer=n_layer,
28              n_head=n_head, n_embd=n_embd, dropout=0.1)
29 model = NanoGPT(**config).to(device)
30 print('模型参数数: ', sum(p.numel() for p in model.parameters()))
31 # 矩阵使用权重衰减; LayerNorm 的一维缩放、偏置不衰减.
32 groups = [dict(params=[p for p in model.parameters() if p.ndim >= 2], weight_decay=0.1),
33           dict(params=[p for p in model.parameters() if p.ndim < 2], weight_decay=0.0)]
34 optimizer = torch.optim.AdamW(groups, lr=lr_max, betas=(0.9, 0.95))
35
36 @torch.no_grad()
37 def generate_snapshot(prompt_ids, max_new_tokens=120):
38     was_training = model.training
39     model.eval()

```

```

40     # 独立抽样源, 不改变训练批次或暂退的随机状态.
41     generator = torch.Generator(device=device).manual_seed(42)
42     idx = prompt_ids.clone().to(device)
43     try:
44         for _ in range(max_new_tokens):
45             logits, _ = model(idx[:, -block_size:])
46             probs = F.softmax(logits[:, -1, :], dim=-1)
47             idx = torch.cat([idx, torch.multinomial(probs, 1, generator=generator)], dim=1)
48         return idx
49     finally:
50         model.train(was_training)
51
52 def train_step(step):
53     model.train()
54     for group in optimizer.param_groups:
55         group['lr'] = get_lr(step)
56     x, y = get_batch('train')
57     _, loss = model(x, y)
58     optimizer.zero_grad(); loss.backward()
59     torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
60     optimizer.step()
61     return loss

```

训练固定 1500 次参数更新. 学习率按更新次数从 1 开始计数, 第 100 次达到峰值, 最后降到最低值; 不把学习率为零的一步算作有效训练. 记录运行时间时还要计入评价与采样, 耗时随设备和线程设置变化.

10.2.3.1 训练过程可视化

每 50 次更新检查固定抽样的训练与验证窗口, 并在指定阶段用同一提示和抽样种子生成文本. 训练批次、评价窗口与生成采样采用独立随机源, 增加一次观察不会消耗训练用的随机数. 最终按验证损失选择快照, 再评价测试窗口.

```

1 hist = {'step': [], 'train_loss': [], 'val_loss': [], 'lr': []}
2 snapshots = {}
3
4 @torch.no_grad()
5 def estimate_loss(splits=('train', 'val'), eval_iters=20):
6     # 每次使用相同的评价窗口, 减少抽样噪声, 也不消耗训练随机数.
7     was_training = model.training; model.eval(); out = {}
8     try:
9         for split in splits:
10             generator = torch.Generator().manual_seed({'train': 200, 'val': 201, 'test':
11                 202}[split])
12             losses = []

```

```

12         for _ in range(eval_iters):
13             x, y = get_batch(split, generator)
14             _, loss = model(x, y)
15             losses.append(loss.item())
16             out[split] = sum(losses) / len(losses)
17         finally:
18             model.train(was_training)
19         return out
20
21 sample_steps = {50, 200, 500, 1000, max_iters}
22 best_val, best_step, best_state = float('inf'), 0, None
23 for step in range(max_iters + 1):
24     if step > 0:
25         train_step(step)
26     if step % 50 == 0 or step == max_iters:
27         losses = estimate_loss()
28         hist['step'].append(step); hist['lr'].append(get_lr(step))
29         hist['train_loss'].append(losses['train']); hist['val_loss'].append(losses['val'])
30         print(f"step {step:4d} train={losses['train']:.4f} "
31               f"val={losses['val']:.4f} lr={get_lr(step):.6f}")
32         if losses['val'] < best_val:
33             best_val, best_step = losses['val'], step
34             best_state = {k: v.detach().cpu().clone() for k, v in model.state_dict().items()}
35
36     if step in sample_steps:
37         out = generate_snapshot(encode('ROMEO:').unsqueeze(0))
38         snapshots[step] = decode(out[0].tolist())
39 model.load_state_dict(best_state)
40 test_loss = estimate_loss(('test',))['test']
41 print(f'best_step={best_step}, test_loss={test_loss:.4f}, perplexity={math.exp(test_loss)
42       :.3f}')
43 # 模型结构与词表随权重保存, 下篇严格加载同一结构.
44 checkpoint = {'config': config, 'chars': chars, 'model': best_state,
45               'best_step': best_step, 'test_loss': test_loss}
46 torch.save(checkpoint, 'nanogpt_pretrained.pt')

```

10.2.3.2 损失和学习率曲线

图10.3来自本次实际运行. 训练和验证使用各 20 批固定窗口估计损失, 每批 32 条、每条 64 个预测位置; 窗口可能重叠, 因此它不是对整份语料逐位置无重叠求平均. 固定窗口能减少评价抽样波动, 但观察小幅变化时仍应结合数据量. 本次 1500 步结束时, 训练损失为 1.5141, 验证损失为 1.6482. 验证最优快照来自第 1500 步, 保留测试窗口的损失为 1.7388, 字符级困惑度为 5.691.

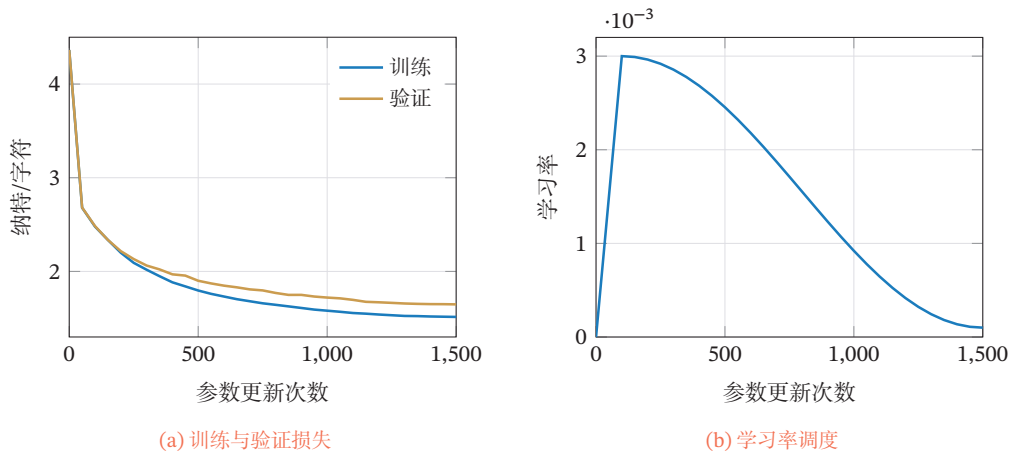


图 10.3 nanoGPT 在 TinyShakespeare 上的训练曲线与学习率调度

10.2.4 模型评价

损失下降意味着预测真实后继字符的平均概率提高,生成样本则能暴露拼写、重复和连贯性问题. 表10.1保留同一提示在 5 个阶段的实际输出,作为两种观察的对照.

表 10.1 nanoGPT 在不同训练步数的实际生成片段 (提示为“ROMEO:”,生成 120 个新词元;表中截取前 75 个字符,竖线表示换行)

step	val_loss	生成片段
50	2.681	ROMEO: : DYOLMour d may: G: my h: T nw, ozounnqurnd fe wVokerear heso land ...
200	2.216	ROMEO: Kitloulf a daysund for grows no, onot nournd fe thowe? MENTIO: Phi...
500	1.900	ROMEO: O clould a dids now dight: sing, Stall, be mane they fear heed And t...
1000	1.721	ROMEO: As look shadvess not die chard's to fatness'd for Vold? POLIXENES: L...
1500	1.648	ROMEO: As look shade, you more chard's, on annobled fear of that he delike,...

可以逐项检查角色名后的冒号、空格与换行、短词拼写及重复片段,而不急着把每个阶段命名为“理解了语法”或“掌握了语义”.一段恰好流畅的抽样也不足以代表所有提示的表现.

这个约 0.81M 参数的模型用于观察字符预测和风格学习. 角色行为与情节可能前后不连贯; 是否掌握某种知识或推理, 需要在未见样例与明确任务上另行检验. 扩大模型和数据会改变实验条件, 也需要重新评价.



提醒

交叉熵取决于词元化、语料与上下文. 本例均匀预测 65 个字符时, 损失为 $\ln 65 \approx 4.17$ 纳特/字符; 困惑度为交叉熵的指数. 不同词表下的词元级损失不能直接比较, 必要时可换算为同一原始文本单位的信息量, 并保持评价数据一致.

10.2.5 模型预测

模型训练完成后, 怎么用它生成文本? 做法很自然: 给一段前缀, 模型输出下一个词元的概率分布, 从中**采样**一个词元追加到序列末尾, 再以新前缀重复整个过程, 直到触发结束条件.

“怎么采样”这一步对最终文本风格影响巨大, 常用的几种策略各有取舍:

- **贪心解码**: 每次选概率最大的词元, 不作随机抽样; 它计算简单, 但在某些提示下容易陷入重复;
- **温度采样**: 将未归一化分数除以温度 $\tau > 0$, 再做 Softmax 并采样. $0 < \tau < 1$ 使分布更尖锐, $\tau > 1$ 使分布更平坦; 最大得分唯一时, $\tau \rightarrow 0$ 的极限趋向贪心解码;
- **Top-k 采样**: 只保留概率最高的 k 个词元, 重新归一化后采样;
- **Top-p 采样** (又称**核采样**): 按概率从高到低排列候选, 保留累积概率首次达到 p 的最小前缀, 重新归一化后采样.

```

1 def sample_next(logits, temperature=1.0, top_k=None, top_p=None, generator=None):
2     if not math.isfinite(temperature) or temperature < 0:
3         raise ValueError('temperature 须为有限非负数')
4     if top_k is not None and (not isinstance(top_k, int) or not 1 <= top_k <= logits.shape
5         [-1]):
6         raise ValueError('top_k 须在 1 与词表大小之间')
7     if top_p is not None and not 0 < top_p <= 1:
8         raise ValueError('top_p 须在 (0, 1] 内')
9     if temperature == 0:
10         return logits.argmax(dim=-1, keepdim=True)
11     logits = logits / temperature
12     if top_k is not None:
13         values, indices = logits.topk(top_k, dim=-1)
14         logits = torch.full_like(logits, -float('inf')).scatter(-1, indices, values)
15     if top_p is not None:
16         values, indices = logits.sort(descending=True, dim=-1)

```

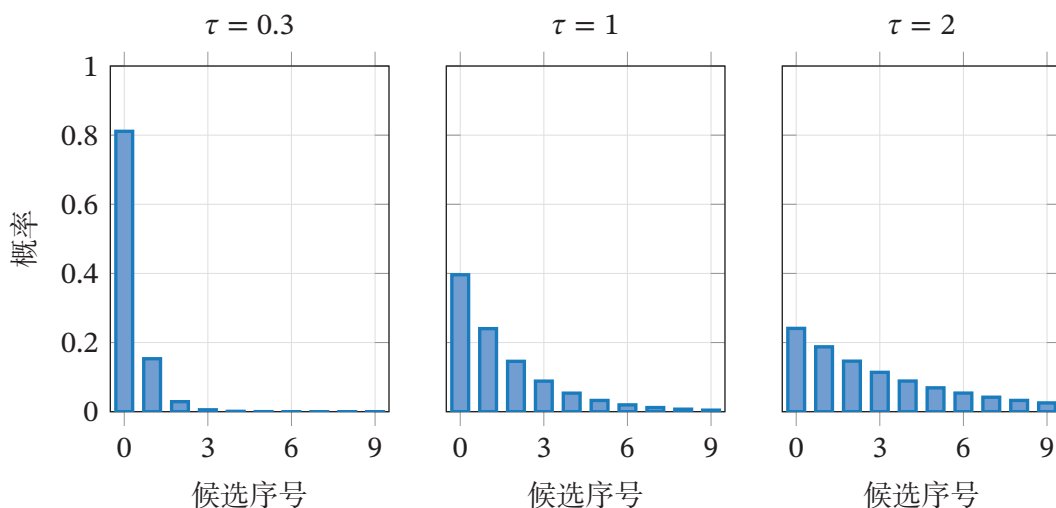
```

16         cumulative = F.softmax(values, dim=-1).cumsum(dim=-1)
17         remove = cumulative >= top_p
18         remove[..., 1:] = remove[..., :-1].clone()
19         remove[..., 0] = False
20         values = values.masked_fill(remove, -float('inf'))
21         logits = torch.full_like(logits, -float('inf')).scatter(-1, indices, values)
22     return torch.multinomial(F.softmax(logits, dim=-1), 1, generator=generator)
23
24 @torch.no_grad()
25 def generate(model, prompt_ids, max_new_tokens, temperature=1.0,
26             top_k=None, top_p=None, stop_id=None, generator=None):
27     if prompt_ids.ndim != 2 or prompt_ids.shape[1] == 0 or max_new_tokens < 0:
28         raise ValueError('提示须为非空二维词元张量, 生成长度不得为负')
29     was_training = model.training; model.eval()
30     idx = prompt_ids.clone().to(next(model.parameters()).device)
31     finished = torch.zeros(len(idx), dtype=torch.bool, device=idx.device)
32     try:
33         for _ in range(max_new_tokens):
34             logits, _ = model(idx[:, -model.block_size:])
35             next_id = sample_next(logits[:, -1, :], temperature, top_k, top_p, generator)
36             if stop_id is not None:
37                 next_id = torch.where(finished[:, None], stop_id, next_id)
38                 finished |= next_id[:, 0] == stop_id
39             idx = torch.cat([idx, next_id], dim=1)
40             if stop_id is not None and finished.all():
41                 break
42         return idx
43     finally:
44         model.train(was_training)

```

10.2.5.1 温度对概率分布的影响

图10.4固定同一个十候选打分向量,只改变温度.低温使概率更集中,高温使分布更平坦;具体变化由候选分数的差距决定,不能保证某个温度下“几乎总选最大项”。

图 10.4 温度 τ 对采样概率分布的影响

低温使输出更集中, 高温增加随机性. 它们改变的是选择方式, 并不保证答案更正确. 不同任务可以这样比较:

- **代码生成、数学题、事实问答**: 比较贪心解码与低温采样, 并用测试、计算器或来源核对答案; 若生成多个候选再筛选, 也可以尝试更高温度;
- **对话、写作**: 以 $\tau = 1$ 为参照, 比较降低温度后的一致性与表达多样性;
- **创意写作、头脑风暴**: 可以提高温度探索不同组合, 同时检查连贯性. 接口中的温度 0 通常表示贪心解码, 不能直接代入除以 τ 的公式.

10.2.5.2 不同采样策略的生成对比

把同一个提示词送给 5 种采样策略, 观察生成结果的差异, 结果见表 10.2:

```

1 ctx = encode('ROMEO:\n').unsqueeze(0)
2 configs = {
3     'greedy': dict(temperature=0),
4     'tau=1.0': dict(temperature=1.0),
5     'tau=1.5': dict(temperature=1.5),
6     'top-k=10': dict(temperature=1.0, top_k=10),
7     'top-p=0.9': dict(temperature=1.0, top_p=0.9),
8 }
9 sample_outputs = {}
10 for name, cfg in configs.items():
11     generator = torch.Generator(device=device).manual_seed(42)
12     out = generate(model, ctx, max_new_tokens=80, generator=generator, **cfg)
13     sample_outputs[name] = decode(out[0].tolist())

```

```
14 print(f'{{name:>10s}}: {{sample_outputs[name]!r}}')
```

表 10.2 nanoGPT 不同采样策略的实际生成对比 (提示为“ROMEO:”后接换行,生成 80 个字符;竖线表示换行)

策略	生成片段
贪心解码	ROMEO: The shall stand the soul of the sent of the country of the sea The see of the se
$\tau = 1.0$	ROMEO: Yield, lose did you more come and on thney, And king eye and go like on, dreams
$\tau = 1.5$	ROMEO: Yiell, lose didst: we nught run. SAMWLnqu0: And thou fear Engo lampt-doquire of
Top-k ($k = 10$)	ROMEO: Yield, I save you and not the rest to the body. CAPULET: My go lame on, the cra
核采样 ($p = 0.9$)	ROMEO: Yes, thou art is not disersivent, on any be man. VOLUMNIA: New like the dream.

比较时保持提示、生成长度和抽样种子一致,重点检查重复、拼写和前后呼应. 贪心解码不抽样;温度、Top-k 和 Top-p 改变候选分布,不会识别事实错误. 本实现按明确的候选索引保留 k 项,累积概率恰好达到 p 时即停止扩展前缀.

KV 缓存 朴素生成每步重算当前前缀;对长度 T 的全注意力,一步的成对计算为 $O(T^2)$. **KV 缓存** (KV Cache) 保存各层已有位置的键和值,新步只计算新位置的查询、键和值,再与已有缓存组合,注意力部分变为 $O(T)$. 配套 Notebook 提供整模型缓存前向,并先核验逐块输入与整段输入的 logits 一致,再比较贪心结果和耗时. **本实现只在原始绝对位置窗口内缓存**;超出窗口会报错,不能把旧缓存裁剪后直接当作重新编号的前缀.

动手练习 10.3



1) 在 TinyShakespeare 上,用贪心解码、Top-k 采样 ($k = 10$)、核采样 ($p = 0.9$) 三种策略分别对同一提示词进行采样,比较生成文本的多样性与重复度.

动手练习 10.4



1) 用配套 forward_cached 和 generate_cached,在相同权重、评价模式和位置编号下比较结果. 先核验完整前缀、多词元增量与单词元增量的 logits,再分别计时首轮前缀处理和后续解码. 一次短序列测时不足以推出通用加速倍数.

10.3 监督微调与 LoRA

预训练目标奖励正确的续写, 并没有单独告诉模型“这里是一条需要执行的指令”。**监督微调** (Supervised Fine-Tuning, SFT) 用带目标输出的样本继续训练; 样本采用指令与回答形式时, 可用于**指令微调** (Instruction Tuning)。本节用大小写转换观察这种适配。实际业务中, 确定性的字符串转换应直接调用程序; 这里选它是因为答案能精确核验。

不过对一个 7B 级别的大模型做全量微调代价昂贵: 除了模型权重, 还要存储梯度、优化器状态和中间激活。**低秩适配** (Low-Rank Adaptation, LoRA) 则只更新少量参数: 冻结预训练权重 W_0 , 只学习一个低秩**增量**。原始方法见 LoRA 论文^②。省略缩放系数时, $\Delta W = BA$, 其中 $A \in \mathbb{R}^{r \times d}$ 、 $B \in \mathbb{R}^{d \times r}$, 秩 $r \ll d$ 。前向变为

$$h = W_0 x + BAx. \quad (10.3)$$

对这个 $d \times d$ 的线性层, 可训练参数量从 d^2 变为 $2dr$, 缩减倍数为 $d/(2r)$; 具体数值取决于层维度、秩和注入位置。下面代码另以 α/r 缩放增量, 这不会改变可训练参数量; 合并权重时也应包含该系数。

```

1  class LoRALinear(nn.Module):
2      """包装一个 nn.Linear, 加上 LoRA 增量 (rank-r) 适配器。"""
3
4      def __init__(self, base: nn.Linear, r=4, alpha=16):
5          super().__init__()
6          if not isinstance(r, int) or r <= 0:
7              raise ValueError('LoRA秩须为正整数')
8          self.base = base
9          self.base.requires_grad_(False)
10         self.r = r
11         self.scaling = alpha / r
12         in_f, out_f = base.in_features, base.out_features
13         self.A = nn.Parameter(base.weight.new_zeros(r, in_f))
14         self.B = nn.Parameter(base.weight.new_zeros(out_f, r))
15         nn.init.kaiming_uniform_(self.A, a=math.sqrt(5))
16         # B 保持全 0: 训练开始时 BA = 0, 等价于原模型
17
18     def forward(self, x):
19         return self.base(x) + (x @ self.A.t() @ self.B.t()) * self.scaling

```

② <https://arxiv.org/abs/2106.09685>.

10.3.1 LoRA 的结构与参数量

LoRA 用两个细长矩阵 $\mathbf{BA}$ 表示低秩增量, 中间维度为 r (图 10.5)。例如, $d = 4096, r = 8$ 时, 该层可训练参数量从 $4096^2 = 16\,777\,216$ 降到 $2 \cdot 4096 \cdot 8 = 65\,536$, 为原来的 0.39%, 即约缩小 256 倍。原始权重仍被保留; 低秩分解作用于增量, 不是把预训练矩阵本身压缩掉。

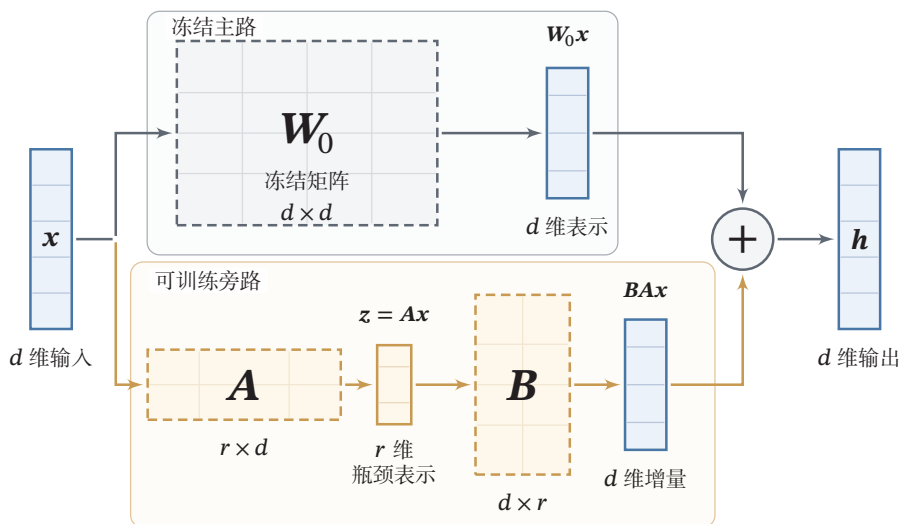


图 10.5 LoRA 的结构

10.3.2 全量微调与 LoRA 的存储开销

以原始 Llama-7B 的 32 层、隐藏维度 4096 为例, 只对各层注意力的 Q/K/V/O 投影注入秩 8 适配器, 计算参数量。下面不加载模型, 也不测量峰值显存:

```
1 # Llama-7B 的近似规格
2 d_model = 4096; n_layers = 32; vocab = 32000
3 # 全量微调: 所有参数都要算梯度
4 full_finetune_params = 7e9
5 # LoRA: 通常只对 attention 的 Q/K/V/O 4 个矩阵加 LoRA
6 # 每层 4 个矩阵, 每个矩阵参数量 = 2 * d * r
7 lora_r = 8
8 lora_params = n_layers * 4 * 2 * d_model * lora_r # 8.4M
9
10 print(f'Llama-7B 全量微调参数: {full_finetune_params/1e9:.1f}B')
11 print(f'LoRA-8 训练参数 : {lora_params/1e6:.2f}M')
12 print(f'LoRA / Full = {lora_params/full_finetune_params*100:.3f}%')
```

输出:

Llama-7B 全量微调参数: 7.0B
 LoRA-8 训练参数 : 8.39M
 LoRA / Full = 0.120%

这里只需训练约 0.12% 的参数. 显存节省主要来自梯度和优化器状态的减少, 中间激活并不按可训练参数比例同步减少:

- 全量 SFT 需要为全部参数保存梯度及优化器状态, 具体字节数取决于精度与优化器实现;
- LoRA 只为适配器参数保存这些状态, 但仍需加载冻结的基模型, 并保留反向传播需要的中间激活.

表 10.3 列出可训练参数量与基模型权重的存储量级. 表中 GB 按 10^9 字节计算, 不包含激活、梯度、优化器状态、量化元数据和工作区, 不能直接作为峰值显存需求; 后者还受批大小、序列长度和激活重计算等设置影响.

表 10.3 7B 模型微调的参数与基模型权重存储量级

策略	可训练参数	基模型权重	说明
Full SFT	7.0B	~14 GB	按 16 位权重估计
LoRA-8	8.4M	~14 GB	冻结 16 位基模型
LoRA-16	16.8M	~14 GB	适配器参数量增加
QLoRA-8	8.4M	~3.5 GB	4 位基模型的理想打包大小

QLoRA 将冻结基模型的量化存储与低秩适配结合.^③表中的 4 位理想打包大小没有计入量化元数据、适配器、优化器状态和激活, 不能当作实际训练显存.

10.3.3 把 LoRA 注入预训练 nanoGPT

加载保存的结构配置、词表和选定权重, 冻结底模, 再为名称恰好是 qkv 或 proj 的线性层加入 LoRA. 适配器张量继承底层权重的设备和精度; 重复注入或没有匹配层会报错, 防止无意改变训练对象.

```

1 def apply_lora_to_model(model, r=8, alpha=16, target_names=('qkv', 'proj')):
2     """精确匹配叶块名; 避免再次注入导致嵌套适配器. """
3     if any(isinstance(m, LoRALinear) for m in model.modules()):
4         raise ValueError('模型已经含有LoRA; 请从未注入的副本开始')
5     matches = [(name, m) for name, m in model.named_modules()
6               if isinstance(m, nn.Linear) and name.split('.')[1] in target_names]
```

③ <https://arxiv.org/abs/2305.14314>.

```

7     if not matches:
8         raise ValueError('没有匹配的线性层, 请核对模块名称')
9     for name, module in matches:
10        parent = model
11        *path, leaf = name.split('.')
12        for part in path:
13            parent = getattr(parent, part)
14            setattr(parent, leaf, LoRALinear(module, r=r, alpha=alpha))
15        n_train = sum(p.numel() for p in model.parameters() if p.requires_grad)
16        n_total = sum(p.numel() for p in model.parameters())
17        print(f'可训练 {n_train:,} / 总 {n_total:,} ({n_train/n_total*100:.2f}%)')
18    return model
19
20 # 重新加载选定预训练快照; 微调时把所有暂退率设为0, 便于小实验复核.
21 checkpoint = torch.load('nanogpt_pretrained.pt', map_location=device, weights_only=True)
22 assert checkpoint['chars'] == chars
23 sft_config = dict(checkpoint['config'], dropout=0.0)
24 model = NanoGPT(**sft_config).to(device)
25 model.load_state_dict(checkpoint['model'])
26 model.requires_grad_(False)
27 torch.manual_seed(300)
28 apply_lora_to_model(model, r=8, alpha=16)

```

动手练习 10.5



1) 对照注入前后的参数列表. 若不冻结整个底模, LoRALinear 仍会冻结它包装的线性层, 但嵌入、前馈层和规范化参数可能继续训练. 计算这两种设置的实际参数比例, 并解释为什么“使用 LoRA 层”不等于“只训练 LoRA 参数”。

10.3.4 SFT 数据格式与损失遮蔽

本实验只在回答词元上计算损失. 输入位置 t 预测原序列的 $t+1$, 所以最后一个提示词元对应的预测仍要保留, 用来监督回答的首词元. 右侧补齐的目标置为 -100 ; 因果注意力使有效位置看不到右侧补齐, 补齐本身也不贡献损失. 超过窗口时保留完整提示、截短回答; 若提示已没有回答空间则报错.

```

1 def build_sft_batch(pairs, encode_fn=encode, max_len=64):
2     """max_len是输入长度上限; 保留整个提示, 回答最多保留剩余位置. """
3     if not pairs or max_len < 1:
4         raise ValueError('批次非空, max_len须为正')
5     xs, ys = [], []
6     for prompt, response in pairs:
7         p_ids = encode_fn(prompt).tolist()

```

```

8         r_ids = encode_fn(response).tolist()
9         if not p_ids or not r_ids:
10             raise ValueError('本例要求非空提示和回答')
11         if len(p_ids) > max_len:
12             raise ValueError('提示占满上下文，无法监督回答；请缩短提示')
13         r_ids = r_ids[:max_len + 1 - len(p_ids)]
14         ids = torch.tensor(p_ids + r_ids, dtype=torch.long)
15         x, y = ids[:-1], ids[1:].clone()
16         y[:len(p_ids)-1] = -100 # 最后一个提示位置预测第一个回答词元
17         xs.append(x); ys.append(y)
18         from torch.nn.utils.rnn import pad_sequence
19         x = pad_sequence(xs, batch_first=True, padding_value=0)
20         y = pad_sequence(ys, batch_first=True, padding_value=-100)
21         return x, y

```

10.3.5 SFT 训练曲线

训练集包含 14 个固定单词,验证与测试各保留 6 个不同单词. 训练 200 步,每 20 步评价回答词元损失,按验证损失选择快照;最后报告训练词与保留测试词的完整字符串匹配数. 对训练词的准确输出是记忆检查,保留单词才用于观察词面泛化. 这个小实验仍不足以评价开放域指令遵循.

```

1 # 固定单词划分: 同一单词不跨训练、验证、测试集合.
2 WORDS = ['hello', 'world', 'pytorch', 'transformer', 'language', 'model',
3          'small', 'cat', 'dog', 'machine', 'learn', 'deep', 'neural', 'net']
4 VAL_WORDS = ['book', 'river', 'music', 'dream', 'school', 'garden']
5 TEST_WORDS = ['attention', 'tensor', 'network', 'science', 'paper', 'graph']
6
7 def make_pair(word):
8     return f'Convert: {word}: ', word.upper() + '\n'
9
10 @torch.no_grad()
11 def sft_loss_on(words):
12     was_training = model.training; model.eval()
13     try:
14         x, y = build_sft_batch([make_pair(w) for w in words], max_len=model.block_size)
15         _, loss = model(x.to(device), y.to(device))
16         return loss.item()
17     finally:
18         model.train(was_training)
19
20 sft_generator = torch.Generator().manual_seed(300)
21 opt = torch.optim.AdamW([p for p in model.parameters() if p.requires_grad],
22                          lr=1e-3, weight_decay=0.0)
23 sft_hist = {'step': [], 'train_loss': [], 'val_loss': []}

```

```

24 sft_best_loss, sft_best_state, sft_best_step = float('inf'), None, 0
25 for step in range(201):
26     if step > 0:
27         model.train()
28         indices = torch.randint(len(WORDS), (8,), generator=sft_generator)
29         x, y = build_sft_batch([make_pair(WORDS[int(i)]) for i in indices], max_len=model.
                               block_size)
30         _, loss = model(x.to(device), y.to(device))
31         opt.zero_grad(); loss.backward(); opt.step()
32     if step % 20 == 0:
33         train_loss, val_loss = sft_loss_on(WORDS), sft_loss_on(VAL_WORDS)
34         sft_hist['step'].append(step); sft_hist['train_loss'].append(train_loss)
35         sft_hist['val_loss'].append(val_loss)
36         print(f'SFT step={step:3d} train={train_loss:.4f} val={val_loss:.4f}')
37         if val_loss < sft_best_loss:
38             sft_best_loss, sft_best_step = val_loss, step
39             sft_best_state = {k: v.detach().cpu().clone() for k, v in model.state_dict().
                               items()}
40 model.load_state_dict(sft_best_state)
41
42 @torch.no_grad()
43 def conversion_results(words):
44     rows = []
45     for word in words:
46         prompt, target = make_pair(word)
47         ids = encode(prompt).unsqueeze(0)
48         out = generate(model, ids, max_new_tokens=32, temperature=0, stop_id=stoi['\n'])
49         response = decode(out[0, ids.shape[1]:].tolist())
50         rows.append({'word': word, 'response': response, 'correct': response == target})
51     return rows
52
53 sft_train_predictions = conversion_results(WORDS)
54 sft_test_predictions = conversion_results(TEST_WORDS)
55 sft_test_loss = sft_loss_on(TEST_WORDS)
56 print('SFT best_step:', sft_best_step)
57 print('训练词完全匹配: ', sum(x['correct'] for x in sft_train_predictions), '/', len(WORDS))
58 print('保留测试词完全匹配: ', sum(x['correct'] for x in sft_test_predictions), '/', len(
    TEST_WORDS))
59 for row in sft_test_predictions:
60     print(row['word'], '->', repr(row['response']))
61 torch.save({'config': sft_config, 'chars': chars, 'model': sft_best_state,
62           'lora': {'r': 8, 'alpha': 16}}, 'nanogpt_sft.pt')

```

图10.6同时保留训练与验证损失。逐词元损失下降不等于整条回答完全正确：少一个字母、多一个空格或没输出换行，都会使本例的完整匹配失败。最终以实际生成结果核对。

本次运行的验证最优点在第 40 步,验证损失为 2.7898。到第 200 步,训练损失继续降至 0.0417,验证损失却升至 4.1482,出现明显的训练集适应与保留词泛化之间的分歧。恢复第 40 步快照后,训练词完整匹配为 0/14,测试词为 0/6,测试回答损失为 2.6819;例如,attention 得到的是 NEE 后接换行。它还没有学会大小写转换规则。可以增加覆盖不同字母和长度的训练样本,再用新的验证结果判断,而不能靠展示几个训练词证明规则已被掌握。

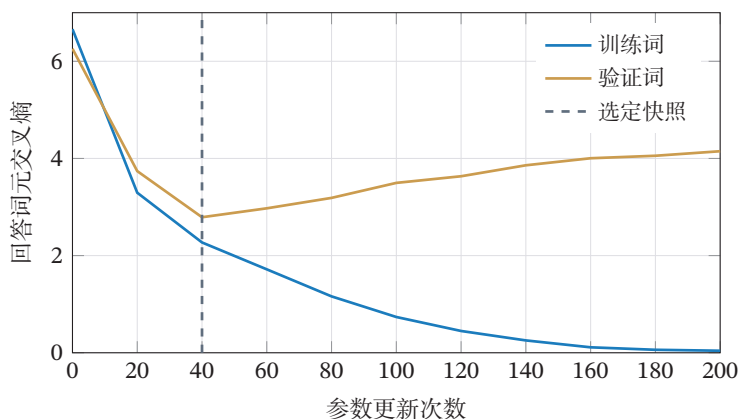


图 10.6 LoRA 与 SFT 的回答词元损失:验证最优点为第 40 步,后续训练损失下降而验证损失上升

笔记

选择 LoRA 时,还可以考虑以下使用方式:

- **可合并部署**:对可合并的线性层,可在部署前把低秩增量并入 W_0 ,避免额外适配器分支的计算;保留独立适配器或采用量化时,开销需另行评估;
- **任务存储较小**:不同任务可以共享基模型,分别保存适配器;适配器仍需加载、切换或合并,成本取决于大小与部署方式;
- **便于尝试组合**:在同一底座和兼容结构下,可以尝试组合多个低秩增量,但可能发生任务干扰,组合效果仍需验证。

10.4 偏好对齐:DPO

SFT 通过示范答案也会传递风格和行为偏好,但通常不显式比较同一问题的多个候选。偏好训练把这种比较直接作为信号:例如,在没有实时天气资料时,标注者可以选择说明信息不足的回答,而拒绝凭空断言“晴朗”的回答。偏好可以涉及准确性、帮助程度和行为边界,并不限于文风。

基于人类反馈的强化学习 (Reinforcement Learning from Human Feedback, RLHF) 的经典做法分为三步:

- 1) 收集人类偏好数据 (x, y_w, y_l) ——同一个提示词 x , 标注员认为回答 y_w 比 y_l 更好;

2) 用偏好数据训练奖励模型 $r_{\phi}(x, y)$;

3) 用 PPO 等强化学习算法提高奖励, 同时用 KL 项惩罚相对参考模型的过大偏离; 经典设置通常以 SFT 模型作为参考。

直接偏好优化 (Direct Preference Optimization, DPO) ^④ 利用带 KL 正则项的奖励最大化问题中, 最优策略与奖励函数的关系, 将偏好拟合改写为下面的损失。原始离线 DPO 直接使用偏好样本, 无须单独训练奖励模型或执行 PPO 式的在线采样更新:

$$\mathcal{L}_{\text{DPO}} = -\log \sigma \left(\beta \log \frac{\pi_{\theta}(y_w | x)}{\pi_{\text{ref}}(y_w | x)} - \beta \log \frac{\pi_{\theta}(y_l | x)}{\pi_{\text{ref}}(y_l | x)} \right), \quad (10.4)$$

其中 π_{θ} 是待训练的策略 (语言模型), π_{ref} 是冻结的参考模型 (通常取 SFT 之后的模型副本), β 控制偏离参考模型的强度。

这个损失提高胜出回答相对落选回答的**对数概率比优势**, 并以冻结的参考模型为基准。它不保证每个胜出回答的绝对概率都上升, 也不构成“偏离绝不超过某个阈值”的硬约束。

```

1 def logprob_of_sequence(model, prompt, response):
2     """字符级条件对数概率之和; 不允许截断改变偏好样本。"""
3     if len(encode(prompt)) + len(encode(response)) - 1 > model.block_size:
4         raise ValueError('偏好样本超出上下文, 应在数据准备时统一处理')
5     x, y = build_sft_batch([(prompt, response)], max_len=model.block_size)
6     model_device = next(model.parameters()).device
7     x, y = x.to(model_device), y.to(model_device)
8     logits, _ = model(x)
9     token_logits = F.log_softmax(logits, dim=-1).gather(-1, y.clamp_min(0).unsqueeze(-1)).
        squeeze(-1)
10    return token_logits.masked_select(y != -100).sum()
11
12 def dpo_loss(policy, ref, prompt, y_win, y_lose, beta=0.1):
13     if beta <= 0:
14         raise ValueError('beta须为正')
15     lp_w = logprob_of_sequence(policy, prompt, y_win)
16     lp_l = logprob_of_sequence(policy, prompt, y_lose)
17     with torch.no_grad():
18         lr_w = logprob_of_sequence(ref, prompt, y_win)
19         lr_l = logprob_of_sequence(ref, prompt, y_lose)
20     return -F.logsigmoid(beta * ((lp_w - lr_w) - (lp_l - lr_l)))
21
22 # 教学偏好: 同一词的大写回答胜过小写回答, 不声称这是人类标注数据。
23 import copy
24 policy = copy.deepcopy(model).eval()
25 ref = copy.deepcopy(model).requires_grad_(False).eval()
26 dpo_optimizer = torch.optim.AdamW([p for p in policy.parameters() if p.requires_grad],

```

^④ 原始离线 DPO 的推导: <https://arxiv.org/abs/2305.18290>.


```

27             lr=1e-4, weight_decay=0)
28 dpo_generator = torch.Generator().manual_seed(400)
29
30 def preference(word):
31     prompt, win = make_pair(word)
32     return prompt, win, word + '\n'
33
34 dpo_hist = []
35 for step in range(101):
36     if step > 0:
37         chosen = torch.randint(len(WORDS), (4,), generator=dpo_generator)
38         loss = torch.stack([dpo_loss(policy, ref, *preference(WORDS[int(i)])) for i in
39                             chosen]).mean()
40         dpo_optimizer.zero_grad(); loss.backward(); dpo_optimizer.step()
41     if step % 20 == 0:
42         with torch.no_grad():
43             train_loss = torch.stack([dpo_loss(policy, ref, *preference(w)) for w in WORDS])
44             .mean().item()
45             val_loss = torch.stack([dpo_loss(policy, ref, *preference(w)) for w in VAL_WORDS
46                                     ]).mean().item()
47             dpo_hist.append({'step': step, 'train_loss': train_loss, 'val_loss': val_loss})
48             print(f'DPO step={step:3d} train={train_loss:.4f} val={val_loss:.4f}')
49     with torch.no_grad():
50         dpo_test_loss = torch.stack([dpo_loss(policy, ref, *preference(w)) for w in TEST_WORDS
51                                     ]).mean().item()
52     print(f'DPO held-out preference loss={dpo_test_loss:.4f}')

```

笔记



本节在同一批训练词上构造“大写优于小写”的偏好对，并固定 SFT 副本作为参考；不把这些规则样本称为人类偏好数据。初始策略与参考完全一致且关闭暂退时，DPO 损失应为 $\ln 2$ 。100 步实验用于观察偏好损失变化；它不替代多样化偏好数据和生成质量评价。

本次 DPO 实验在相同策略与参考下从 $\ln 2$ 开始，第 100 步训练偏好损失为 0.1148，验证偏好损失为 0.0835，保留测试词的偏好损失为 0.0426（图 10.7）。这说明模型在所构造的两类回答之间增加了相对偏好；它与 SFT 的逐词元损失和完整匹配率是不同指标，不能据此声称任意单词都能正确转换。

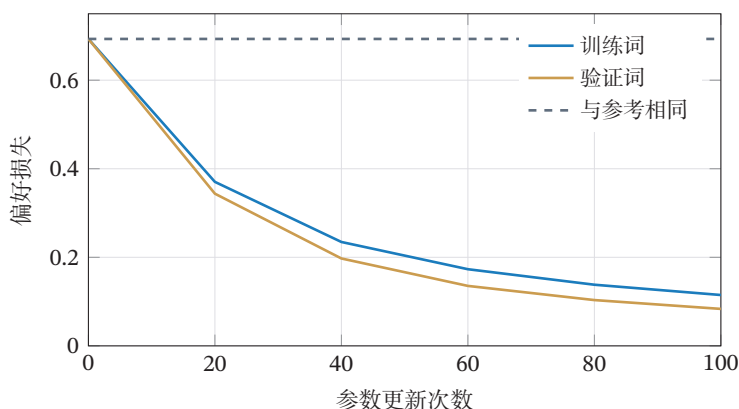


图 10.7 规则构造的偏好对上, DPO 训练与验证损失的变化

10.5 推理与行动智能体

智能体 (Agent) 是围绕目标感知环境、作出决策并采取行动的系统。在以语言模型为核心的实现中，模型提出下一步，外部程序负责检查和执行工具调用，再把结果交回模型；工具可以是搜索引擎、代码执行器或数据库。

ReAct (Reasoning + Acting) 把推理与行动交替组织。^⑤教学上可把一轮拆成：

- **Thought**: 模型形成下一步的推理或计划；教学中用文本展示，不要求实际产品公开内部思维链；
- **Action**: 调用一个工具（比如 `Search(...)`、`Calculator(...)`）；
- **Observation**: 工具返回的结果，输入回模型。

模型给出最终回答后结束；没有在步数上限内完成，则返回上限状态。下面用固定规则模拟决策模块，工具只有离线资料查询和有界四则运算。动作采用结构化字段，程序核验类型、工具名称和参数，再把成功或错误结果写入历史。它演示执行循环，不表示前面的字符模型已学会工具调用。

```

1 import ast
2 import operator
3 import re
4
5 def calculator(expression):
6     """仅解释有界四则运算，不执行Python表达式中的任意操作。"""
7     if not isinstance(expression, str) or not 1 <= len(expression) <= 128:
8         raise ValueError('表达式长度须在1至128字符之间')
```

^⑤ <https://arxiv.org/abs/2210.03629>.

```

9     tree = ast.parse(expression, mode='eval')
10    if len(list(ast.walk(tree))) > 50:
11        raise ValueError('表达式太复杂')
12    binary = {ast.Add: operator.add, ast.Sub: operator.sub,
13              ast.Mult: operator.mul, ast.Div: operator.truediv}
14    unary = {ast.UAdd: operator.pos, ast.USub: operator.neg}
15    def visit(node, depth=0):
16        if depth > 12:
17            raise ValueError('表达式嵌套太深')
18        if isinstance(node, ast.Constant) and type(node.value) in (int, float):
19            value = node.value
20        elif isinstance(node, ast.BinOp) and type(node.op) in binary:
21            value = binary[type(node.op)](visit(node.left, depth+1), visit(node.right, depth
22                                          +1))
23        elif isinstance(node, ast.UnaryOp) and type(node.op) in unary:
24            value = unary[type(node.op)](visit(node.operand, depth+1))
25        else:
26            raise ValueError('只支持数字、括号和四则运算')
27        if not math.isfinite(value) or abs(value) > 1e12:
28            raise ValueError('数值超出教学工具范围')
29        return value
30    return str(visit(tree.body))
31
32 def search(query):
33     # 固定的离线资料，并非联网搜索。
34     entries = {'法国首都': '巴黎', 'capital of france': 'Paris',
35               '真空光速': '299792458 m/s'}
36     for key, value in entries.items():
37         if key in query.lower():
38             return value
39     raise LookupError('离线资料中没有相关条目')
40
41 TOOLS = {'Calculator': calculator, 'Search': search}
42
43 def mock_llm(question, history):
44     """按固定规则返回结构化动作，用于检查执行循环。"""
45     if history and history[-1]['kind'] == 'observation':
46         observation = history[-1]
47         answer = observation['result'] if observation['ok'] else '工具未能完成: ' +
48                 observation['result']
49         return {'kind': 'answer', 'text': answer}
50     match = re.search(r'[0-9][0-9+*/(). x÷-]*', question)
51     if match and any(op in match.group() for op in '+-*/x÷'):
52         return {'kind': 'tool', 'name': 'Calculator',
53               'input': match.group().strip().replace('x', '*').replace('÷', '/')}

```

```

52     return {'kind': 'tool', 'name': 'Search', 'input': question}
53
54 def react_loop(question, max_steps=5, model_fn=mock_llm):
55     if max_steps < 1:
56         raise ValueError('max_steps须为正')
57     history = []
58     for _ in range(max_steps):
59         action = model_fn(question, history)
60         if not isinstance(action, dict):
61             return {'status': 'invalid', 'history': history}
62         kind = action.get('kind')
63         if kind == 'answer' and isinstance(action.get('text'), str):
64             history.append(action)
65             return {'status': 'answered', 'answer': action['text'], 'history': history}
66         if kind != 'tool' or not isinstance(action.get('name'), str) or not isinstance(
67             action.get('input'), str):
68             return {'status': 'invalid', 'history': history}
69         history.append(action)
70         try:
71             if action['name'] not in TOOLS:
72                 raise ValueError('未知工具')
73             if len(action['input']) > 256:
74                 raise ValueError('工具输入过长')
75             result = TOOLS[action['name']](action['input'])
76             observation = {'kind': 'observation', 'ok': True, 'result': result}
77         except (ValueError, SyntaxError, ArithmeticError, LookupError) as error:
78             observation = {'kind': 'observation', 'ok': False, 'result': str(error)}
79         history.append(observation)
80     return {'status': 'limit', 'history': history}
81
82 for question in ['计算 (15 + 27) * 3', '法国首都是什么?', '计算 1 / 0']:
83     result = react_loop(question)
84     print(question, '->', result.get('answer', result['status']))

```

10.6 检索增强生成 (RAG)

检索增强生成 (Retrieval-Augmented Generation, RAG) 先取回相关资料, 再让生成模型据此回答. 它可以是一条固定流程, 也可以是智能体调用的工具. 下面先明确各环节的职责:

- 1) 把知识库切成小段 (chunks), 用 embedding 模型逐段编码成向量;
- 2) 建立检索索引, 例如用 FAISS 向量检索库, 或用 Milvus 等向量数据库存储和检索向量;

<https://arxiv.org/abs/2005.11401>.

3) 用户提问时,把问题也编码成向量,从向量库中检索 Top-k 相似 chunks;

4) 把检索结果作为上下文拼到提示词中,生成回答并核对资料是否支持结论. 检索可能漏掉证据或找到错误资料,模型也可能误用资料;RAG 不能保证消除幻觉.

本例使用虚构的课程手册,以字符 n-gram 的 TF-IDF 向量作词面检索基线,并保留文档 ID 与来源. 回答环节暂用“返回首条资料摘录”的函数替代真实生成;没有检索到足够相关资料时明确返回缺少信息. 运行后可分别检查相关资料是否入选、引用是否对得上,以及资料之外的问题是否被错误回答.

```

1 from sklearn.feature_extraction.text import TfidfVectorizer
2
3 # 教学虚构的课程手册; 保留文档ID与来源, 便于核对检索结果.
4 kb_chunks = [
5     {'id': 'gpu', 'source': '课程手册第1节', 'text': 'GPU单次预约上限是2小时. 取消预约后需要重新提交. '},
6     {'id': 'report', 'source': '课程手册第2节', 'text': '实验报告应包括训练曲线、验证指标和数据划分方法. '},
7     {'id': 'backup', 'source': '课程手册第3节', 'text': '模型文件保存在checkpoints目录, 提交报告前请另存一份备份. '},
8 ]
9 # 字符n-gram的稀疏向量是词面检索基线, 不是预训练语义嵌入.
10 vectorizer = TfidfVectorizer(analyzer='char', ngram_range=(2, 4), norm='l2')
11 kb_vectors = vectorizer.fit_transform([chunk['text'] for chunk in kb_chunks])
12
13 def retrieve(question, k=2, min_score=0.1):
14     if not isinstance(k, int) or not 1 <= k <= len(kb_chunks):
15         raise ValueError('k超出知识库大小')
16     q = vectorizer.transform([question])
17     scores = (kb_vectors @ q.T).toarray().ravel()
18     indices = scores.argsort(kind='stable')[::-1][:k]
19     return [dict(kb_chunks[i], score=float(scores[i])) for i in indices if scores[i] >=
20             min_score]
21
22 def mock_grounded_answer(question, evidence):
23     # 用摘录代替真实生成, 明确标记来源; 不把证据内容当作可执行指令.
24     if not evidence:
25         return '现有课程手册没有足够资料回答. '
26     first = evidence[0]
27     return f"资料摘录: {first['text']} [{first['id']}; {first['source']}]"
28
29 def rag(question, k=2, answer_fn=mock_grounded_answer):
30     evidence = retrieve(question, k)
31     return {'answer': answer_fn(question, evidence), 'evidence': evidence}
32
33 for question in ['GPU预约上限是多少?', '实验报告应包含什么?', '明天天气如何?']:

```

```

33     result = rag(question)
34     print(question, '->', result['answer'])

```



动手练习 10.6

1) 实现一个带记忆的推理与行动智能体——把过往对话作为知识入库, 每轮对话开始前先检索相关历史。

10.7 小结

从“ROMEO:”的续写开始, 我们训练了字符模型, 观察采样与缓存, 再在同一权重上检查微调 and 偏好损失. 工具与检索的模拟实验则展示了模型输出进入外部程序后还需要哪些检查. 各项技术的核心思想速查见表10.4.

表 10.4 大语言模型与智能体关键技术速查

技术	解决什么	关键代码 / 工具
nanoGPT	自回归语言建模	Transformer decoder + 因果注意力
KV 缓存	推断加速	缓存历史 K / V, 每词元注意力 $O(T^2) \rightarrow O(T)$
采样策略	控制生成多样性	温度/Top-k/Top-p
LoRA	高效微调, 省显存	$\Delta W = BA$, 秩 $r \ll d$
SFT	用示范输出适配任务	本例仅在回答上算损失
DPO	从候选比较中学习偏好	$-\log \sigma(\beta[\Delta_w - \Delta_l])$
推理与行动	让模型主动调工具	Thought / Action / Observation 循环
RAG	让模型查知识库	向量检索 + 检索结果拼提示词

本章的迷你 GPT 与许多大模型共享自回归训练、因果注意力和残差组块等基本思路. 扩大规模还需要数据治理、稳定训练、结构改进、分布式计算和任务评测; 把参数和语料放大, 并不会自动得到可靠的对话或智能体能力.

从教学实验走向完整系统 预训练、SFT、偏好训练和推断时计算解决不同问题. 原始离线 DPO 使用给定的候选比较; 强化学习还可以让当前模型生成候选, 再用可核验的答案、测试或奖励模型提

供反馈。GRPO 与 DeepSeek-R1 展示了这类推理训练路径。增加推理词元或采样次数意味着更多计算,收益取决于任务和筛选机制;回答更长并不自动更正确。

接入真实模型前,先规定工具的参数结构、错误状态、资源上限和允许的副作用。模型返回工具名称只是一个提议,执行权限由外部程序决定;工具和检索返回的文字是资料,不应被提升为新的操作指令。评价既要检查最终任务是否完成,也要检查工具参数、失败处理、引用支持程度与调用成本。

RAG 可以分别评估检索召回与回答依据:有标准证据的问题用于检查 Top- k 是否包含所需段落,答案的每项事实再与引用段落核对。换用语义嵌入、加入重排或扩大上下文都需要比较;换一个框架不会自动修复缺失证据或错误数据。模型、词元化器、适配器与知识库版本应一并记录,才能复查一次回答的来源。

思考



把模拟接口替换成一个支持结构化工具调用的真实模型时,需要怎样的适配器?请列出正常动作、未知工具、参数格式错误、工具失败和步数耗尽的预期行为,并先用模拟返回逐项验证,再进行真实调用。



索引

- 边列表 (Edge List) , 319
- 标准化 (Standardization) , 57
- 步长 (Stride) , 131
- 残差网络 (Residual Network, ResNet) , 149
- 层规范化 (Layer Normalization, LN) , 240, 304
- 查询-键-值 (Query-Key-Value, QKV) , 280
- 查询向量 (Query Vector) , 266
- 词表 (Vocabulary) , 195
- 词嵌入 (Word Embedding) , 195
- 大语言模型 (Large Language Model, LLM) , 356
- 带泄露的 ReLU (Leaky ReLU) , 91
- 低秩适配 (Low-Rank Adaptation, LoRA) , 373
- 动量法 (Momentum Method) , 223
- 读出 (Readout) , 341
- 对数几率 (Logit) , 66
- 多分类 (Multi-class Classification) , 75
- 多头自注意力 (Multi-Head Self-Attention, MHSA) , 282
- 多维数组 (ndarray) , 4
- 多项式基函数 (Polynomial Basis Function) , 47
- 方差缩放 (Variance Scaling) , 235
- 非线性基函数 (Non-linear Basis Function) , 48
- 飞桨 (PaddlePaddle) , 1
- 分段编码 (Segment Encoding) , 299
- GraphSAGE (Sample and AggreGatE) , 328
- 广播 (Broadcasting) , 16
- 过度平滑 (Over-smoothing) , 334
- 过度压缩 (Over-squashing) , 355
- 过拟合 (Overfitting) , 37
- 互相关 (Cross-Correlation) , 129
- 回归 (Regression) , 38
- 汇聚层 (Pooling) , 139
- 混合专家 (Mixture of Experts, MoE) , 314
- 基于人类反馈的强化学习 (Reinforcement Learning from Human Feedback, RLHF) , 379
- 机器学习 (Machine Learning, ML) , 34
- 计算图 (Computational Graph) , 22
- 加与规范化 (Add&Norm) , 304
- 加州房价预测数据集 (California Housing Prices) , 55
- 监督微调 (Supervised Fine-Tuning, SFT) , 373
- 检索增强生成 (Retrieval-Augmented Generation, RAG) , 384
- 简单循环网络 (Simple Recurrent Network, SRN) , 168
- 结构风险 (Structure Risk) , 37
- 节点更新 (Update) , 322
- 仅解码器 (Decoder-only) , 362
- 经验风险 (Empirical Risk) , 37
- 聚合 (Aggregation) , 266
- 卷积神经网络 (Convolutional Neural Network, CNN) , 128
- 均方误差 (Mean Squared Error, MSE) , 43
- KV 缓存 (KV Cache) , 372
- 链接预测 (link prediction) , 337
- 邻接表 (Adjacency List) , 319
- 邻接矩阵 (Adjacency Matrix) , 319
- 邻居聚合 (Aggregate) , 322
- 零填充 (Zero Padding) , 132
- 轮 (Epoch) , 83
- 门控机制 (Gating Mechanism) , 187
- 模型部署 (Model Deployment) , 35
- 模型测试 (Model Test) , 35
- 模型推断 (Model Inference) , 35
- 内部协变量偏移 (Internal Covariate Shift) , 240
- PyTorch (PyTorch) , 1
- PyTorch Geometric (PyG) , 344
- 批大小 (Batch Size) , 118, 206
- 批量规范化 (Batch Normalization, BN) , 240
- 批量规范化层 (BatchNorm) , 151
- 批量梯度下降法 (Batch Gradient Descent, BGD) , 118
- 评价指标 (Evaluation Metric) , 38
- 前规范化 (Pre-LN) , 362

- 前馈神经网络 (Feedforward Neural Network, FNN), 89
- 嵌入 (Embedding), 173
- 嵌入矩阵 (Embedding Matrix), 173
- 情感分析 (Sentiment Analysis), 194
- 权重衰减 (Weight Decay), 257
- 神经网络 (Neural Network, NN), 89
- 数据类型 (dtype), 4
- 算子 (Operator, Op), 1
- 算子库 (nndl), 26
- 随机梯度下降法 (Stochastic Gradient Descent, SGD), 118
- 随时间反向传播 (Backpropagation Through Time, BPTT), 168
- 损失函数 (Loss Function), 37
- 特征缩放 (Feature Scaling), 57
- 特征图 (Feature Map), 136
- 梯度下降法 (Gradient Descent Method), 38
- 提前停止 (Early Stopping), 38, 72
- 同质性 (Homophily), 320
- 头 (head), 282
- 图神经网络 (Graph Neural Network, GNN), 318
- 图像分类 (Image Classification), 159
- 微调 (Fine-Tuning), 234
- 位置编码 (Positional Encoding), 299
- 未登录词 (Out-of-Vocabulary, OOV), 358
- 未归一化分数 (Logits), 121, 176
- 文本语义匹配 (Text Semantic Matching), 293
- 下采样 (Downsampling), 131
- 线性回归 (Linear Regression), 38
- 消息传递 (Message Passing), 322
- 消息计算 (Message), 322
- 小批量梯度下降 (Mini-Batch Gradient Descent), 118, 206
- 形状 (Shape), 4
- 旋转位置编码 (Rotary Position Embedding, RoPE), 312
- 循环神经网络 (Recurrent Neural Network, RNN), 167
- 循环神经网络单元 (RNN Cell), 168
- 掩码 (Mask), 200
- 原子操作 (Elementary Operation), 22
- 暂退法 (Dropout), 259
- 张量 (Tensor), 1, 4
- 长程依赖问题 (Long-Term Dependencies Problem), 168
- 长短期记忆网络 (Long Short-Term Memory, LSTM), 187
- 长时间间隔 (Long Range), 168
- 正则化 (Regularization), 249
- 直接偏好优化 (Direct Preference Optimization, DPO), 380
- 直连边 (Skip Connection), 149
- 指令微调 (Instruction Tuning), 373
- 智能体 (Agent), 382
- 轴 (Axis), 4
- 逐层规范化 (Layer-wise Normalization), 240
- 逐元素积 (Element-wise Product), 188
- 注意力分布 (Attention Distribution), 266
- 注意力机制 (Attention Mechanism), 265
- 状态 (State), 167
- 准确率 (Accuracy), 71
- 字节对编码 (Byte Pair Encoding, BPE), 358
- 自动梯度计算 (autograd), 106
- 自动微分 (Automatic Differentiation, AD), 22
- 自回归 (autoregressive), 357
- 自注意力 (Self-Attention), 278
- 组块 (block), 303
- 最小—最大归一化 (Min-Max Normalization), 57
- 最小二乘法 (Least Squares Method, LSM), 44